

Digesting Gibbs Sampling Using R

MAHDI TEIMOURI YANESARI¹

August 15, 2026

¹profs.gonbad.ac.ir/fa/teimouri

Dedicated to my parents

Contents

1	An introduction to R	3
1.1	Data and objects in R	4
1.2	Condition and repeat	13
1.2.1	Conditional decision using if(...)	14
1.2.2	Repeat using while(...)	14
1.2.3	Repeat using for(...)	15
1.3	Data importing and exporting	16
1.3.1	Reading built-in data	16
1.3.2	Importing data from R package	17
1.3.3	Importing data from other application	17
1.3.4	Importing xls and xlsx data	18
1.3.5	Data exporting	19
1.3.6	Apply family	19
1.4	Graphical summary	21
1.4.1	Plot(...)	21
1.4.2	Pairwise visualization	25
1.4.3	Contingency table	25
1.5	Function in R	27
1.5.1	Built-in function for character	28
1.6	External R package	29
1.6.1	External R package initialization	29
1.6.2	External R package installation	30
1.6.3	Normality test	31
1.7	Regression analysis in R	32
1.7.1	Linear regression	32
1.7.2	Non-linear regression	34
2	Common simulation techniques	37
2.1	Metropolis-Hastings algorithm	37
2.2	Inverse transform method	41
2.3	Distributional identity	43
2.3.1	Multinomial distribution	45
2.3.2	Finite mixture model	45
2.3.3	Dirichlet distribution	48
2.3.4	Chi-squared distribution	48
2.3.5	Inverse Gaussian distribution	49
2.3.6	α -Stable distribution	50
2.3.7	Birnbaum-Saunders distribution	53
2.4	Rejection sampling	54
2.5	Adaptive rejection sampling	57
2.5.1	Two-dimensional single rejection sampling	58

2.6	Ratio of uniforms method	61
2.7	Generalized ratio of uniforms method	64
2.8	Simulating multivariate distributions	65
2.8.1	Gaussian distribution	65
2.8.2	Properties of Gaussian distribution	66
2.8.3	Generation from Gaussian distribution	67
3	Importance Sampling	69
3.1	Importance sampling	69
3.2	Variance reduction	72
3.3	Estimators with negative covariance	74
3.3.1	Computing the CDF of univariate Gaussian distribution	78
4	Gaussian Scale mixture models	79
4.0.1	Skewed Gaussian models	79
4.0.2	Gaussian scale/location-scale mixture models	80
4.0.3	Gaussian mixture location-scale model	82
4.1	Skewed Gaussian scale mixture models	83
4.1.1	Special models	85
4.1.2	Computing the CDF of multivariate Gaussian distribution	89
4.1.3	Computing CDF of the multivariate Student's t distribution	92
4.2	Univariate Gaussian quadrature	96
4.2.1	Gaussian quadrature rules	98
4.2.2	Golub-Welsch algorithmmmmmmmmmmm	103
4.3	Multivariate Gaussian quadrature	116
4.4	Computing the CDF of multivariate Gaussian distribution using Gaussian Quadrature	127
4.4.1	Computing the CDF of bivariate Gaussian distribution	127
4.4.2	Computing the CDF of bivariate Student's t distribution	132
4.4.3	Computing the CDF of bivariate Gaussian scale mixture distribution	134
4.4.4	Computing the CDF of trivariate Gaussian distribution	136
4.4.5	Computing the CDF of trivariate Student's t distribution	141
4.4.6	Rectangular Gaussian probability with positive correlation matrix	145
4.4.7	Computing the CDF of multivariate Gaussian distribution: General case	146
4.5	The EM algorithm	149
4.5.1	How does the EM algorithm work?	150
4.5.2	Convergence of the EM Algorithm	150
4.5.3	Extensions of the EM algorithm	151
4.6	Robust linear regression	175
4.7	Model and notation	176
4.7.1	Coefficient estimator	177
4.8	Moments of truncated distributions	180
4.9	Moments of truncated Gaussian distribution in univariate case	181
4.9.1	First two moments of truncated Gaussian distribution	182
4.10	First moment of truncated multivariate Gaussian distribution	183
4.11	Second moment of truncated Gaussian distribution	187
4.12	Truncated skew Gaussian distribution	193
4.13	Moment of truncated Student's t distribution	194
4.13.1	First two moments of truncated Student's t distribution distribution in univariate case	194
4.14	First moment of truncated Student's t distribution in multivariate case	197

5	Bayesian inference	201
5.1	Prior selection	201
5.1.1	Uniform prior	202
5.1.2	Reference prior	202
5.1.3	Jeffreys prior	203
5.1.4	Conjugate prior	203
5.2	Empirical Bayes	204
5.3	Credible interval	207
6	Gibbs sampling	211
6.1	Gibbs sampling in broad sense	211
6.2	Moments of truncated Gaussian distribution: Gibbs sampling scheme	213
6.3	Gibbs sampling in Bayesian paradigm	215
6.4	Gibbs sampling in Bayesian paradigm with one latent variable	226
6.5	Gibbs sampling in Bayesian paradigm with two latent variables	232
6.6	Gibbs sampling in Bayesian paradigm with more than two latent variables: Model-based Clustering	239
6.6.1	Generating from Gaussian finite mixture model	240
6.6.2	Model selection	242
6.6.3	Finite mixture of Gaussian distributions	243
6.7	Simulating from truncated Gaussian distribution	248

List of Figures

1.1	Scatterplot of (a): height vs. girth and (b): height and volume vs. girth.	21
1.2	Commonly used symbols for <code>pch</code> in R and the pertinent code.	22
1.3	Two sample plots produced based on some supplementary calls of Table 1.3. . . .	23
1.4	Pairwise scatterplots of <code>iris</code> data.	25
1.5	Pairwise scatterplots of <code>iris</code> data with fitted Gaussian curve to the marginals. .	26
1.6	Mosaicplot for <code>Wage</code> data.	27
1.7	(left subfigure): q-q plot and (right subfigure): Tukey mean-difference q-q plot. .	32
1.8	(a): Scatterplot of <code>women</code> data: (a) fitted regression line and (b): prediction interval.	33
1.9	Scatterplot of height and diameter data with fitted Weibull (left subfigure) and logistic (right subfigure) growth curves.	36
2.1	(a): Histogram (constructed based on chain's output between 201 th up to 1000 th iterations); superimposed is $f(x \theta = 5)$ and (b): Chain's motion across iterations for sampling from $f(x \theta = 5)$ with starting value $x_0 = 4$	41
2.2	(a): a schematic of three-piece CDF and (b): the corresponding pieces.	43
2.3	(a): Histogram of $n = 2000$ simulated data from family with CDF (2.13). The red-colored superimposed curve is the corresponding PDF. (b): The population CDF and empirical CDF.	43
2.4	(a): histogram of $n = 5000$ simulated data from: (a) two-component Gaussian mixture model (2.16) and (b): two-component gamma mixture model (2.18). The red-colored superimposed curve is the corresponding PDFs given in (2.16) and (2.19), respectively.	47
2.5	(a): Histogram constructed based on 1000 samples generated from $\mathcal{W}(\theta = 2, 1)$. Superimposed are $Mg(x \theta)$ (red line) and $\mathcal{W}(x \theta = 2, 1)$ (blue line). (b): Generated samples across iterations for sampling from $f(x \theta = 2) = \mathcal{W}(x \theta = 2, 1)$. . .	54
2.6	(a): Histogram constructed based on 500 samples generated from $\mathcal{SG}(0, 1, \lambda = 4)$. Superimposed are $Mg(x \theta) = 2\phi(x)$ (red line) and $f(x 0, 1, \lambda = 4)$ (blue line). (b): Generations across iterations for sampling from $f(x 0, 1, \lambda = 4) \sim \mathcal{SG}(0, 1, \lambda = 4)$	56
3.1	The top row shows the bias and $\log(1+\text{rmse})$ for $\hat{\theta}_{MLE}$, $\hat{\theta}^*$, $\hat{\theta}_1$, and $\hat{\theta}_2$ computed based on $N = 1000$ independent sample each of size $n = 100$ generated from $\mathcal{BS}(\alpha, \beta = 1)$. The bottom row displays the same graph with the same parameters except $n = 500$	75
3.2	Components of random vector $(U, V)^\top$ come independently form $\mathcal{U}(-1, 1)$	76
3.3	The bias (a) and rmse (b) of $\hat{\pi}$, $\hat{\pi}_1$, and $\hat{\pi}_2$ computed based on $N = 1000$ independent samples of size n	77
4.1	Plot of integrand $f(x) = x \exp\{-x\}$ and nodes x_1 and x_2 . The area under the integrand is shown by shaded area.	98

4.2	Graph of (a): Hermite and (b): Legendre polynomials of orders $n = 2$ (solid orange line), $n = 3$ (dashed black line), $n = 4$ (dotted red line), and $n = 5$ (dotted-dashed blue line).	100
4.3	Graph of integrand (4.108). (a): $p = 1$ and $k = 5$, (b): $p = 2$ and $k = 10$	114
4.4	Graph of integrand (4.110) for (a): $z = 2$ and $k = 5$, (b): $z = 4$ and $k = 15$	115
4.5	(a): Graph of $g(x_1, x_2 \boldsymbol{\theta} = (0.5, 1, 0.5)^\top)$ on region $[0, 0.5] \times [0, 0.5]$, (b): Graph of $g(x_1, x_2 \boldsymbol{\theta} = (0.5, 1, 0.5)^\top)$ transformed to region $[-1, 1] \times [-1, 1]$, (c): Graph of $g(x_1, x_2 \boldsymbol{\theta} = (5, 1, 5)^\top)$ on region $[0, 10] \times [0, 10]$, (b): Graph of $g(x_1, x_2 \boldsymbol{\theta} = (5, 1, 5)^\top)$ transformed to region $[-1, 1] \times [-1, 1]$, (e): Coordinates of nodes computed using Gauss-Legendre rule with $k_1 = k_2 = 5$ points, and (f): Product of weights computed using Gauss-Legendre rule with $k_1 = k_2 = 5$ points.	120
4.6	First row: contour plot of $g(z_{1i}, z_{2j})$ given in the RHS of (4.151). The filled black points show the address of nodes based on the Cholesky decomposition for (a): $\mathbf{a} = (-2, -2)^\top$, $\mathbf{b} = (2, 2)^\top$, and $\rho = -0.50$ and (b): $\mathbf{a} = (0, 0)^\top$, $\mathbf{b} = (\infty, \infty)^\top$, and $\rho = 0.50$. Second row: contour plot of $g(z_{1i}, z_{2j})$ given in the RHS of (4.246). The filled black points show the address of nodes based on the spectral decomposition for (c): $\mathbf{a} = (-2, -2)^\top$, $\mathbf{b} = (2, 2)^\top$, and $\rho = -0.50$ and (d): $\mathbf{a} = (0, 0)^\top$, $\mathbf{b} = (\infty, \infty)^\top$, and $\rho = 0.50$	148
4.7	The red-colored solid curve is $\ell(\mu \mathbf{y}) = \log L(\mu \mathbf{y})$. Superimposed is $Q(\mu \mu^{(k)})$ at iterations $k = \{1, 2, 3, 20\}$ for four combinations of sample size and starting value as (a): $n = 10, \mu^{(0)} = -4$, (b): $n = 10, \mu^{(0)} = 1$, (c): $n = 40, \mu^{(0)} = -4$, and (d): $n = 40, \mu^{(0)} = 1$	151
4.8	The output of EM and MCEM algorithms for estimating (a): α and (b): λ , when $\mathcal{GE}$ distribution is fitted to 112 patients with plasma cell myeloma.	157
4.9	The output of EM and MCEM algorithms for estimating (a): δ and (b): γ , when SaS distribution is fitted to $n = 100$ simulated realizations.	159
4.10	The output of EM algorithm for estimating (a): $\mu = 0$ and (b): $\sigma = 1$	165
4.11	(a): The PDF of bivariate Gaussian distribution with parameters $\mu_1 = \mu_2 = 0$, $\sigma_1^2 = 1$, $\sigma_2^2 = 2$, and $\rho = 0.3535$. (b): The PDF of the corresponding truncated bivariate Gaussian distribution on area with lower limits $\mathbf{a} = (-1, -1)^\top$ and upper limits $\mathbf{b} = (2, 2)^\top$. (c) The PDF of bivariate Gaussian distribution with parameters $\mu_1 = \mu_2 = 0$, $\sigma_1^2 = 2$, $\sigma_2^2 = 2$, and $\rho = 0.25$. (b): The PDF of the corresponding truncated bivariate Gaussian distribution on area with lower limits $\mathbf{a} = (0, 0)^\top$ and upper limits $\mathbf{b} = (+\infty, +\infty)^\top$	186
5.1	Schematic diagram for PDF of underlying, prior, and posterior when (a): $n = 50$ and (b): $n = 100$	208
5.2	Schematic diagram for credible interval.	208
5.3	(a): the 95% empirical credible interval for parameter α based on a sample of $n = 200$ realizations generated from $\mathcal{G}(\alpha = 2, \beta = 2)$ distribution and (b): histogram of the generated samples with fitted posterior $\pi(\alpha \mathbf{x}, \boldsymbol{\theta}_0)$	210
6.1	(a): Scatterplot with fitted contours based on 3000 samples generated from $\mathcal{N}_2(\mathbf{0}, \Sigma)$ in which $\Sigma = [(1, 0.5)^\top, (0.5, 1)^\top]$, (b): histogram of generated samples from marginal X_1 , and (c): histogram of generated samples from marginal X_2	213
6.2	Scatterplot of 5000 samples generated from full conditionals: (a) $\theta_1 (\theta_2, \theta_3, \mathbf{x})$, (b): $\theta_2 (\theta_1, \theta_3, \mathbf{x})$, and (c): $\theta_3 (\theta_1, \theta_2, \mathbf{x})$	220
6.3	Scatterplot of 5000 samples generated from full conditionals: (a) $\beta_0 (\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\beta_0)})$, (b): $\beta_1 (\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\beta_1)})$, and (c): $\sigma (\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\sigma)})$. The blue line, in each subfigure, shows the Bayesian estimator.	223

6.4	Scatterplot of 5000 samples generated from full conditionals: (a) $\beta_0 (\mathbf{y}, \mathbf{x}, \Psi_{(-\beta_0)})$, (b): $\beta_1 (\mathbf{y}, \mathbf{x}, \Psi_{(-\beta_1)})$, (c): $\beta_2 (\mathbf{y}, \mathbf{x}, \Psi_{(-\beta_2)})$, and (d): $\sigma^2 = \delta (\mathbf{y}, \mathbf{x}, \Psi_{(-\delta)})$. The blue line, in each subfigure, shows the Bayesian estimator.	226
6.5	Scatterplot of trees data. Superimposed is the fitted regression plane whose coefficients are estimated through the Bayesian paradigm.	227
6.6	Scatterplot of 5000 samples generated from full conditionals: (a) $\mu_1 (\mathbf{y}, \mathbf{g}, \Psi_{(-\mu_1)})$, (b): $\mu_2 (\mathbf{y}, \mathbf{g}, \Psi_{(-\mu_2)})$, (c): $\Sigma_{11} (\mathbf{y}, \mathbf{g}, \Psi_{(-\Sigma_{11})})$, (d) $\Sigma_{12} (\mathbf{y}, \mathbf{g}, \Psi_{(-\Sigma_{12})})$, (e) $\Sigma_{22} (\mathbf{y}, \mathbf{g}, \Psi_{(-\Sigma_{22})})$, and (f) $\nu (\mathbf{y}, \mathbf{g}, \Psi_{(-\nu)})$. The blue line, in each subfigure, shows the Bayesian estimator.	254
6.7	The PDF of skewed Student's t distribution. (a): true model and (b): true and fitted contour plots.	255
6.8	Scatterplot of 5000 samples generated from full conditionals involved in Bayesian paradigm for skewed Student's t distribution in Example 6-7.	255
6.9	(a): Ten data points that can constitute a cluster, (b): Ten data points that should be clustered into two groups, (c): Ten data points in part (a) that can be modelled through a single-component (homogeneous) model with PDF $f(\mathbf{y} \Theta)$, and (d): Ten data points in part (b) that can be modelled through a two-component (non-homogeneous) mixture model with PDF $0.4f(\mathbf{y} \Theta_1)+0.6f(\mathbf{y} \Theta_2)$	256
6.10	(a): Scatterplot of 300 data points drawn from $\mathbf{Y} = (Y_1, Y_2)^\top$ following a three-component Gaussian mixture model and (b): Computed BIC versus number of clusters.	256
6.11	(a): Scatterplot of faithful data, (b): contour plot fitted to standardized faithful data, and (c): scatterplot of clustered standardized faithful data.	257

List of Tables

1.1	Some commands for numeric vector algebra in R.	9
1.2	Arguments of <code>matrix</code> in R.	11
1.3	Some commands for matrix algebra in R.	13
1.4	Some commonly used operators in R.	15
1.5	Details about main arguments of <code>read.table(...)</code> in R.	17
1.6	Functions for importing data in R.	18
1.7	Details about main arguments of <code>write.table(...)</code> in R.	19
1.8	Details about main arguments of <code>plot</code> in R.	22
1.9	Details about arguments of low-level graphical facilities.	24
1.10	Details about arguments of <code>cut(...)</code>	25
1.11	Some primitive built-in functions in R.	28
1.12	Details about main arguments of <code>grep</code> in R.	29
1.13	Arguments of <code>substr(...)</code> in R.	29
1.14	Details about output of <code>lm(...)</code> in R.	33
1.15	Details about main arguments of <code>nls(...)</code> in R.	34
1.16	Details about main arguments of <code>tryCatch(...)</code> in R.	34
2.1	Commands for simulating from some statistical families in R.	65
3.1	Summary statistics for computing $\Omega = E(\log(1+X))$ where $X \sim \mathcal{G}(\alpha, \beta)$ through three instrumentals.	71
3.2	Summary statistics for computing $\Omega = E(X)$ where $X \sim \mathcal{SG}(0, 1, \lambda)$ through three instrumentals.	73
4.1	Examples of the GSM models.	81
4.2	PDF of the GSM model in Table 4.1.	81
4.3	Three members of CFUSG scale mixture models.	87
4.4	Most commonly used orthogonal polynomials.	102
4.5	The nodes and weights associated with the Gauss-Hermite rule of third kind. . .	108
4.6	ARE for approximating integral in (4.108).	114
4.7	ARE for approximating integral in (4.110) for $\alpha = 2$ using Gauss-Hermite rule. .	115
4.8	Absolute error of the Gaussian quadrature rules in approximating $T(\sqrt{2}, a)$. . .	117
4.9	ARE for approximating CDF (4.119) for $\beta = 1$ using Gauss-Legendre rule. . . .	119
4.10	Results for approximating $E(T_1^2 T_2^2)$ using k -point Gauss-Legendre rule.	121
4.11	Fatigue life of 10 bearings of a certain type (in hours).	125
4.12	Estimators and associated GOF statistics for gamma distribution fitted to to fatigue life data given in Table 4.11.	125
4.13	Computed ARF for approximating the CDF of Student's t using k -point Gaussian- Legendre and Gauss-Hermite rules.	132
4.14	ARE for approximating $\mathcal{P} = P(\mathbf{a} \leq \mathbf{X} \leq \mathbf{b})$ using k -point Gauss-Legendre rule based on Cholesky and spectral decompositions.	147
4.15	Output of the EM algorithm for multinomial distribution.	163

4.16	Summary statistics for computing $\mathbf{\Omega}$ through the exact and importance sampling with $\mathcal{N}_2(\mathbf{0}, \Sigma)$ as the instrumental.	185
4.17	Summary statistics for computing $\mathbf{\Omega} = E(\mathbf{X}_{\mathcal{R}^p})$ through the exact and importance sampling methods.	199

Preface

In general, the statistical simulation approaches are referred to as the *Monte Carlo methods* as a whole. The broad class of the Monte Carlo methods involves the Markov chain Monte Carlo (MCMC) techniques that attract the attention of researchers from a wide variety of study fields. The main focus of this report is to provide a framework for all users who are interested in implementing the MCMC approaches in their investigations, especially the *Gibbs* sampling. I have tried, if possible, to eliminate the proofs, but reader is expected to know some topics in elementary calculus (including mathematical function, limit, derivative, partial derivative, simple integral) and statistics (including discrete and continuous random variables, probability distribution, expected value and variance, moment generating function, multivariate distribution, distribution of a functions of random variable, and the central limit theorem).

Almost all of researchers use statistical tools such as *confidence interval* or *hypothesis testing* for supporting their study results. Indeed, through both aforementioned tools, investigator can test a statement about the unknown population parameter such as population mean. Herein, the accuracy of decision made about the unknown parameter depends on a concept called *statistic* that is the sample counterpart of the unknown parameter under testing that is herein, for example, the sample mean or the sample median. The critical question is that which one is better the sample mean or the sample median ? The simulation process enables investigator to answer this question quite well. In fact, investigator will answer this question within a statistical laboratory by simulating sample evidence through the Monte Carlo methods, and then computing the sample mean or median based on simulated evidence. The accuracy of both statistics is accessible easily by repeating the simulation procedure. The Monte Carlo methods are widely used in several fields of study such as biology, engineering, econometrics, medicine, etc. The main concern of this report is placed on showing how user can apply the Monte Carlo methods in practice. I have tried to give the R codes accompanying with examples in order to help user for understanding the theoretical topics in the fields of statistics well. As the last, but not the least, I have used throughout the equivalent phrases “Gaussian” and “normal” interchangeably. I did my the best to avoid mistakes, specially for the formulas, however, I am sure that still there are several mistakes. Let me know please if you find anyone or have comments through the following email address.

Acknowledgements

- I would like to express my sincere thanks to my family and my colleagues that indirectly contributed to this work.
- I would like to express my sincere thanks to Amber Jain (<http://amberj.devio.us/>) for a free-to-edit LaTeX template.

Mahdi Teimouri Yanesari,

Assistant Professor in Statistics, Gonbad Kavous University, Gonbad Kavous, Iran.

Email:mahdiba_2001@yahoo.com

1

An introduction to R

The **R** (or **R** language) is an environment developed mainly for *statistical analysis*, *statistical computing*, and *data visualization*. In contrast to **SPSS** or **Minitab**, **R** lacks the graphical user interface (GUI) that allows user to accomplish statistical analysis using some graphical components such as button, dialog box, and icon. This means that **R** is a programming language or *command-line* interface in which user must type codes within some sessions called **console** or **script** for implementing the statistical methods.

The first nonformal version on **R** was created by Ross Ihaka and Robert Gentleman, two statisticians from University of Auckland in August 1993 and then introduced formally in 1996 [Ihaka and Gentleman, 1996]. The first formal version of **R**, that is version 1.0, was created in February 29, 2000. This language has won a great deal of advocate among data scientists since it maintains some advantages among them we mention the following:



- i. The **R** is loaded quickly.
- ii. The **R** is user-friendly interface.
- iii. The **R** is an open-source software that can be distributed freely.
- iv. The **R** user can call (or interfaces) the **C** or **Fortran** for implementing a project faster.
- v. The codes or programs in a new structure called **R** package that can be exported/imported simply while they are often accompanied by a long-form vignette (documentation) in order to help user for using it.

Another computational software written by data analysis researchers at Bell Labs is the **S** that has its origin from the joint project of Rick Becker and John M. Chambers on May 5 1976 Chambers [2020]. The **S** language was written to provide a framework for research in data analysis and collaborations to apply that research, rather than as a separate project to create a programming language Chambers [2020]. The **S** calls **Fortran** subroutines and has some extensions such as **S3** and **S4**. The **R** and **S3** are very close together. Herein, we do not deal with the differences between these extensions and their relations with **R**. Some redecorated and absolutely more attractive versions of **R**, called **R Studio** or **Visual R Studio**, are available for linting, compiling code, highlighting syntax, having a look at data, variables, packages, etc. Herein, we aim to provide a general, but not comprehensive, knowledge for users who are interested in working or being familiar with **R**. Since **R** is free for distributing or redistributing, the user is able to download it from www.r-project.org. To do this, the user just needs to enter <https://cran.r-project.org/mirrors.html>, and then download **R**, after selecting a mirror from a listing of mirrors that basically are servers located in different locations (countries) which host the same **R** and corresponding packages. It is suggested to select a close mirror in order to speed downloading. We point out that all notes given here can be found in more

details at <https://cran.r-project.org/manuals.html>. There is more detailed information on this in [Becker et al., 1988, Chambers and Hastie, 1992, Chambers, 1998, Nolan and Speed, 2000, Pinheiro and Bates, 2000, Therneau and Grambsch, 2000, Venables and Ripley, 2000, Harrell, 2001, Fox, 2002, Gareth et al., 2013, Hothorn and Everitt, 2014, Bloomfield, 2014, Daróczi, 2015, Blangiardo and Cameletti, 2015, Kohl, 2015, Sun, 2015, Dayal, 2015, Leemis, 2016, Murray, 2017, Hastie et al., 2017, Rahlf, 2017, Kelley, 2018, Fox and Weisberg, 2018].

1.1 Data and objects in R

There are, in general, five types of data in R including:

- i. *character* (or *string*): the name of people, cities, mountains, or marital status, your gender, etc., that evidently cannot be recorded numerically.
- ii. *complex*: a complex number such as $1 + 2i$ that is represented in R as `1+2i`.
- iii. *integer*: a natural or whole number such as 2 that is represented in R when is followed by letter L as `2L`.
- iv. *logical*: takes on only two states, that is, `TRUE` (or T) and `FALSE` (or F)
- v. *numeric* (or *double*): a measurable quantity such as age, height, temperature, etc., that evidently can be recorded numerically, or the chemical, mathematical, or physical constants such as Avogadro's number (6.023×10^{23}), the ratio of a circle's circumference to its diameter ($\pi = 3.14159265\dots$), and the gravity on the Earth ($g = 9.8m/s^2$).

The R user has not permission of access to the stored information directly, but R provides some data structure that is referred to as *object*. In point of view of some other programming languages *object* is a *variable*. Hence, everything in R is an *object*. All variables, regardless of their type, and all functions are objects. For example, the value such as 2 is a datum of type "double", but if we have `a=2`, then `a` is an object. Each object in R belongs to a particular *class* that assigns some *attributes* or *properties*. These properties determine how the generic functions operate with the corresponding object. In summary, any object in R has a specific *type*, is stored in a specific *mode*, and has a particular class in order to determine the particular *object* can be used. These features are important when we are working on objects with more structures such as *vector*, *data frame*, *list*, *matrix*, etc.

R session

The R has two main sessions including *console* and *script* (or code editor). Once we run R, then the console session comes up in which a ">" prompt appears after which we must write the input command (call). One of the main objects in R is vector that has two types including: *atomic* and *nonatomic*. The elements of each atomic vector have the same type while elements of the latter are not of the same type. For example, suppose we would like to define an atomic vector consisting of numbers one, two, and three. To do this, we may proceed as follows.

```
R> v0 <- 1:3 # we may write v0 = 1:3
R> v0
```

and then press the Enter key "↵" to see the output as follows.

```
[1] 1 2 3
```

Throughout, as seen in above, we show the R command and the R output in red- and blue-colored texts, respectively. As it is seen, we can insert an explanatory expression in front of vector `v0` in above after sharp symbol "#". We note that another way to define above vector is to run command

```
R> v1 <- c(1,2,3); v1
```

Of course, we can write several commands (calls) in one line provided that each command is separated with the next one by *just one* semicolon “;” symbol. As we see, objects `v0` and `v1`

R code	output
<code>R> mode(v0); typeof(v0); class(v0)</code>	<code>[1] "numeric"</code> <code>[1] "integer"</code> <code>[1] "integer"</code>
<code>R> mode(v1); typeof(v1); class(v1)</code>	<code>[1] "numeric"</code> <code>[1] "double"</code> <code>[1] "numeric"</code>

have the same `mode`, but different `type` and `class`. We note that, different objects may have the same class. In practice there is no difference between `v0` and `v1` defined in above and can be used interchangeably in many applications. Alternatively, we can produce the output given in above by typing `v1 <-c(1,2,3)` in the script session and then pressing `(Ctrl) + (R)`. The script is a new window that is appeared by following the track `File >> New script`. Once we have created a script session, we follow the track `File >> Save as` to save the script under an arbitrary name, for example “myfile.R” in which, as we see, suffix is `.R`. Care should be taken about the location in which the script will be saved. The default path can be changed straightforwardly. The default directory for saving R files is obtained by typing the following command in console or execute it from script session.

```
R> getwd()
```

The output is

```
[1] "C:/Users/NikPardaz/Documents"
```

If we no longer interested in directory given above, for changing it, we may put our desired address inside the command `setwd()`. For example, if we would like to change the path `C > Users > NikPardaz > Documents` to `E > my R work`, then we may use the command given below.

```
R> setwd("E:/my R work")
```

We note that R is case sensitive that implies, for example, letters “A” and “a” are different objects. There are different ways in R for getting help with functions and application of phrases. Herein, we suggest the most commonly used ways to get more information about phrase `apply`. The latter is a *built-in* function that is applied to the margins of an array or matrix. To do this, we may use either command

```
R> help(apply)
```

or

```
R> ?(apply)
```

The version of R that we are using is very important when installing the external packages. More detailed information on this and related features can be obtained using the command

```
R> R.version
```

We may use command

```
R> print(v1)
```

to observe the elements of vector `v1`. A nonatomic vector such as `v2` may be defined in R as

```
R> v2 <- c(1, 2, 'male', 'female')
R> str(v2)

chr [1:4] "1" "2" "male" "female"
```

As it is seen, two last elements of object `v2` are of type character. If at least one of elements in a given object, such as `v2`, are character, then the given object is considered as a character shown by “chr”. It is worth to note that objects in R whose type is character (nonnumeric) must be surrounded by a single or double quotation marks. The elementary mathematical operations including addition, subtraction, and multiplication are meaningless for objects such as `v2` defined in above. Sometimes, objects `v1` and `v2` are called the *numeric* and *non-numeric* vectors. Regardless of the its type, the element(s) of each vector can be found easily in R. For example, the second element of vector `v1` is determined by command `v1[2]`. If one needs to determine more elements of a given vector, then the address of those elements must be put inside the square bracket. Let vector `v3` is defined as

```
R> v3 <- 1:10 # equivalently we may write v3<-c(1,2,3,4,5,6,7,8,9,10)
```

For finding the first five elements of `v3`, we may use command

```
R> v3[1:5]
```

Equivalently, we may do this task through the command

```
R> v3[v3<=5]
```

Finally, if we want to save all elements of vector `v3` that are between 3 and 8 (excluding ending points) in new vector called `v4`, we proceed as follows.

```
R> v4 <- v3[ v3<=7 & v3>=4 ]
```

If the desired elements are not well-addressed, then we have to determine the addresses through a new vector. For example, for picking up the elements of `v3` whose addresses are odd (or even), we have

```
R> v.odd <-v3[ c(1,3,5,7, 9) ]# elements of v3 whose indices are odd number
R> v.even1<-v3[ c(2,4,6,8,10) ]# elements of v3 whose indices are even number
```

Sometimes, we are interested eliminating one or more items from a vector. For example, the last element of vector `v3` is removed using command `v3[-10]`. Hence, another method for creating vector `v.even` in above is to use command

```
R> v.even2<-v3[-c(1,3,5,7,9)]# elements of v3 whose indices are even numbers
```

This means that vectors `v.even1` and `v.even2` are the same. The basic mathematical operations such as subtraction (addition) and multiplication (division) are applied to two vectors pointwise provided that the vectors are of the same lengths. For instance, we can compute `v.odd/v.even1` as follows.

```
R> v.div <- v.odd/v.even1 # v.odd divided by v.even1
R> print(v.div)
```

```
0.5000000 0.7500000 0.8333333 0.8750000 0.9000000
```

The name assigned to a vector is quite optional, but we suggest the user to avoid using letters c, D, F, I, etc. Typically, an English letter, English letter followed by an integer number, English letter followed by English letter, an English letter followed by dot “.”, or English letter followed by an underscore “_” is suggested. In above, we assigned the name `v.div` to vector `v.odd/v.even1`. The format and length of each element in output above can be determined by user. For example, we may use command `options(digits=4)` to see that

```
0.5000 0.7500 0.8333 0.8750 0.9000
```

As we see, the total digits after the decimal point is 4. If the output is greater than one, then the sum of total digits before and after the decimal point would be 4. If a vector multiplied by a numeric value, then all elements of the given vector is multiplied by that numeric value. This holds true also for subtraction, division, and addition operations. We have

```
R> v4 <- 1:3
R> v5 <- 3:5
R> v6 <- v4/v5
```

```
0.3333 0.5000 0.6000
```

Above three commands can be written in shorter form in order to save the space. For this purpose, all commands must be written sequentially, but each is separated from the others by symbol “;”. We have

```
R> v4 <- 1:3; v5 <- 3:5; v6 <- v4/v5
```

Either both commands `ls()` or `objects()` can be used to show a listing of variables and objects created when working with R. Moreover, the objects created during an R session are stored in machine’s memory and are ready when R is started at later time. We suggest to remove all variables at the end of each R session. For example, if we would like to remove either object `x` or both of objects `x` and `y` from the set of objects, we can use command

```
R> rm('x')
```

and

```
R> rm(list=c('x', 'y'))
```

respectively. In general command

```
R> rm(list=ls())
```

can be used to remove all objects from the current session. We note that, in general, R does not discriminate between commands `rm('x')` and `rm("x")`. It is worth to note that the R user is free to put blank or empty space between object(s) and round brackets (), square brackets [], and curly bracket { }. For example, all commands `rm("x" )`, `rm( "x")`, `rm ("x")`, and `rm(list=c('x', 'y'))` are correct and do the same task. Furthermore, the R is case sensitive language. That is, the *capital* and *small* letters have different meaning and do different tasks. For instance, objects `x` and `X` are two different objects and command `Rm("x")` is meaningless. When using the latter, the output would be an error message as follows.

```
R> Rm("x")
Error in Rm("x") : could not find function "Rm"
```

The remainder of this subsection is devoted to introduce more complicated objects (or data structures) in R including sequence, data frame, matrix, array, and list.

Sequence

Herein, we introduce two important methods for creating vectors that have specified structures that typically are helpful in statistical analysis. For repeating some specified numeric values under some given pattern, we may use `rep(.)`. For example, in order to create sequence 1,1,2,2,3,3,4,4, we may proceed as follows.

```
R> r1 <- rep(1:4, each = 2)
```

Furthermore, for creating sequence 1,2,3,4,1,2,3,4, we have

```
R> r2 <- rep(1:4, 2)
```

and finally, we write

```
R> r3 <- rep(1:4, 1:4)
```

to produce sequence 1,2,2,3,3,3,4,4,4,4.

For producing numeric values within an interval, $[a, b]$ say, with a given increment `delta` (or length `n`), we use command `seq(a,b,delta)` (or `seq(a,b,length=n)`). For example, in order to create vector 0,0.1,0.2,...,0.9,1, we can use the following command.

```
R> s1 <- seq(0, 1, 0.1)
```

The output is

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

Alternatively, above output can be created using command

```
R> s2 <- seq(0, 1, length=11)
```

When working with vectors, we must take care about the observations that are recorded as missing or Not Available (NA), Not Available as Number (NaN), and infinity (Inf). In R, all observations that have been not recorded during the sampling stage are denoted by NA. The symbol NaN accounts for the indeterminate forms (including $0/0$, $0 \times \log(0)$, $\sqrt{-1}$, or $0 \times \infty$) and hence is called *Not a Number*. Evidently, when a nonzero value is divided by zero, the result would be Inf. The mathematical operations cannot be applied on NA, NaN, and Inf cases. Hence, to avoid problems arising from these cases, we should have our diagnostics. One simple way, is the use of command `summary`. We have

```
R> z3 <- c(NA, 1:5)
```

```
R> summary(z3)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
1.00	3.25	5.50	5.50	7.75	10.00	1

Based on above output, there is one missing value in vector `z3`. Typically, we need to apply the mathematical/statistical functions on elements of a vector. Table 1.1 shows a listing of statistical functions in R with a widespread use in applications.

Table 1.1: Some commands for numeric vector algebra in R.

Function	output/task
<code>all(x, na.rm=FALSE)</code>	<code>logical</code> is TRUE if all elements of vector <code>x</code> holds true for the given property, otherwise is FALSE.
<code>any(x, na.rm=FALSE)</code>	<code>logical</code> is TRUE if at least one elements of vector <code>x</code> holds true for some given property, otherwise is FALSE.
<code>as.vector(x)</code>	considering sequence <code>x</code> as a vector.
<code>cbind(x1,x2,...)</code>	creating a matrix whose columns are vectors <code>x1,x2,...</code> .
<code>ceiling(x)</code>	rounding elements of <code>x</code> to the smallest integer greater than <code>x</code> .
<code>cumsum(x)</code>	cumulative sum of elements of <code>x</code> .
<code>diff(x,lag=k)</code>	difference of <code>x</code> values with lag <code>k</code> .
<code>floor(x)</code>	rounding elements of <code>x</code> to the greatest integer less than <code>x</code> .
<code>intersect(x,y)</code>	intersect of <code>x</code> and <code>y</code> .
<code>is.numeric(x)</code>	<code>logical</code> : if FALSE, then <code>x</code> is not a numeric vector. and TRUE otherwise.
<code>length(x)</code>	length of <code>x</code> .
<code>max(x,na.rm=FALSE)</code>	maximum of <code>x</code> , by default NA is not removed.
<code>min(x,na.rm=FALSE)</code>	minimum of <code>x</code> , by default NA is not removed.
<code>mean(x)</code>	average of <code>x</code> .
<code>median(x,na.rm=FALSE)</code>	median of <code>x</code> .
<code>numeric(n)</code>	a numeric vector of length <code>n</code> .
<code>NULL</code>	a numeric vector regardless of its length.
<code>cumprod(x)</code>	cumulative product of elements of <code>x</code> .
<code>quantile(x,q)</code>	<code>q</code> th quantile of <code>x</code> .
<code>range(x,na.rm=FALSE)</code>	range of <code>x</code> , by default NA is not removed.
<code>rbind(x1,x2,...)</code>	creating a matrix whose rows are vectors <code>x1,x2,...</code> .
<code>rev(x)</code>	representing elements of <code>x</code> in reversed representation of <code>x</code> .
<code>round(x)</code>	rounding elements of <code>x</code> to the nearest integer.
<code>sd(x,na.rm=FALSE)</code>	standard deviation of <code>x</code> , by default NA is not removed.
<code>sum(x,na.rm=FALSE)</code>	sum of <code>x</code> , by default NA is not removed.
<code>summary(x)</code>	6-point summary of <code>x</code> .
<code>union(x,y)</code>	union of <code>x</code> and <code>y</code> .
<code>var(x,na.rm=FALSE)</code>	variance of <code>x</code> , by default NA is not removed.
<code>which.min(x)</code>	index in which <code>min(x)</code> occurs.
<code>which.max(x,na.rm=FALSE)</code>	index in which <code>max(x)</code> occurs.

Data frame

A rectangular structure of data with different types in R can be constructed using a concept called *data frame*. The data frame can be used to represent all types of data including numeric and non-numeric, but in a rectangular (balanced) form. The general command for data frame is

```
data.frame(name1=object1,name2=object2,...,row.names=NULL)
```

For example of data frame may be

```
R> name <- c('row1', 'row2', 'row3')
R> df1 <- data.frame(col1=1, col2=v4, col3=v5, row.names=name)
R> df1
```

The output becomes

```
      col1 col2 col3
row1     1     1     3
row2     1     2     4
row3     1     3     5
```

As another example, let we want to construct a *design of experiment* in which there are two factors (each of three levels) with one repetition. To do this, we may construct a data frame as follows.

R code	output			
R> Level<-c('low','medium','high')	Factor1	Factor2	response	
R> n.L<-length(Level); Rep<-1	1	low	low	1
R> factor1<-rep(Level,times=Rep,	2	low	low	2
+ each=n.L*n.L)	3	low	low	3
R> factor2<-rep(Level,times=n.L,	4	low	medium	4
+ each=n.L)	5	low	medium	5
R> df1<-data.frame(Factor1=	6	low	medium	6
+ factor1, Factor2=factor2,	7	low	high	7
+ response=1:27)	8	low	high	8
	...	...	...	...
	26	high	high	26
	27	high	high	27

Sometimes we need to manipulate or organize some part of a data frame (or matrix) in terms of its other parts. A useful function for this purpose is `subset(df,subset,select)` in which, `df` denotes the data frame involving whole data, `subset` is the constraint we are willing to apply, and `select` is the name of column(s) to be select from a data frame. For example, in `airquality` data, suppose we are willing to selecting all levels of variables `Temp` and `Wind` for which variables `Ozone` and `Solar.R` are not missing. To do this is, we may use command `subset` as follows.

```
dd<-subset(airquality, !(Ozone=="NA") & !(Solar.R=="NA"), select=c(Wind,Temp))
```

When data frame is large, then we would prefer to manipulate the data frame in terms of its column's name rather than the column's order or number. In such a situation, one may use the R package `dplyr` available at R Comprehensive archive <https://cran.r-project.org/web/packages/dplyr/index.html>.

List

A useful and informative structure of data can be constructed in R using a concept called *list*. The list can be used to represent all types of data including matrix, numeric data, non-numeric data, and logical. To create a list (called here L1) consisting of vector `v2` and data frame `df1` defined earlier, we can write

```
R> L1 <- list(x1 = v2, x2 = df1); L1

$x1
[1] "1"      "2"      "male"    "female"

$x2
  x y z
1 1 1 4
2 1 2 5
3 1 3 6
```

To access to each part of data frame, for example the first part, we may write `L$x1` and also for the second row of the second part we may write `L$x2[2,]`, and finally elements consisting of digit 3 in last part of list L is accessible through command `L$x3[4:6]`. Of course, alternatively, the latter is accessible using `L[[2]][2,]`. We note that if we use command `list(v2,df1)`, then names of two parts in the list L will be disappeared.

Matrix

One of the most commonly used structures in R is matrix. Recall that elements in each column of data frame are of the same type while different columns may have different types. The matrix is a rectangular form of data with the same type for which the numeric one is commonly used. In general, for constructing a matrix called A, we use the command given by the following.

```
R> A <- matrix(data=NA, nrow=1, ncol=1, byrow=FALSE, dimnames=NULL)
```

Herein `data`, `nrow`, `ncol`, `byrow`, and `dimnames` are arguments of function `matrix`. Table 1.2 below gives more explanations about the duty of each argument. It is worthwhile to note that

Table 1.2: Arguments of `matrix` in R.

Argument	output/task
<code>data</code>	a numeric vector that can include even NA, NAN, and Inf.
<code>nrow</code>	number of rows.
<code>ncol</code>	number of columns.
<code>byrow</code>	logical: if FALSE (by default), then the matrix is filled by columns, otherwise the matrix is filled by rows.
<code>dimnames</code>	a list of length 2 that gives the row and column names, respectively.

R automatically fills out the elements of matrix *column-by-column*. If user forgets the order of arguments in `matrix` function, it suffices to run command `args(matrix)` as

```
R> args(matrix)
```

to see the arguments of function `matrix` as follows.

```
function (data=NA, nrow=1, ncol=1, byrow=FALSE, dimnames=NULL)
NULL
```

Hence, if constructing a 2×2 matrix A represented as

$$A = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}.$$

is of interest, then we may use two commands to define matrix A as follows. If we want to

R code	output
<code>R>A<-matrix(c(1,2,3,4),nrow=2,ncol=2)# method 1</code>	<code>[,1] [,2]</code>
<code>R>A<-matrix(1:4,nrow=2,ncol=2)# method 2</code>	<code>[1,] 1 3</code>
<code>R>A</code>	<code>[2,] 2 4</code>

assign names to the rows and columns, and further, fill out the matrix A by rows, we proceed as follows. To access (i, j) th element of matrix A , we must write $A[i, j]$. Moreover, the elements

R code	output
<code>R>L2<-list(c("row1","row2"),c("col1","col2"))</code>	<code>col1 col2</code>
<code>R>A<-matrix(1:4,nrow=2,ncol=2,byrow=TRUE,</code>	<code>row1 1 2</code>
<code>+ dimnames=L2)</code>	<code>row2 3 4</code>

of i th row of matrix A are obtained by command $A[i,]$, and accordingly, the elements of j th column of matrix A are obtained by command $A[, j]$. For instance, the output of commands $A[1,1]$, $A[1,]$, and $A[, 2]$ are given as follows.

```
R> A[1,1]    R> A[1,]    R> A[,2]
[1] 1        [1] 1 2    [1] 2 4
```

The matrix algebra typically is used in statistical methods. Table 1.3 lists the most commonly used functions (commands) associated with the matrix algebra.

Array

The array can be a one-, two-, or even three-dimensional object. The general command for array is `array(data=NA, dim=c(nr,nc,nz), dimnames=NULL)` in which `data` is the vector or sequence of array's elements, `nr` is the number of rows, `nc` is the number of columns, `nz` is the number of third coordinate, and `dimnames` is a list of names for three coordinates. There is also one- or two-dimensional array. The array can be reduced to construct a matrix. A simple example of three-, two-, and one-dimensional array can be constructed using vector `1:8`, `1:4`, and `1:2` as follows.

```
R> array(1:8,dim=c(2,2,2))  R> array(1:4,dim=c(2,2))  R> array(1:2,dim=2)

, , 1                [,1] [,2]                [1] 1 2
      [,1] [,2]      [1,] 1 3                [1,] 1 2
[1,] 1 3            [2,] 2 4                [2,] 1 2
[2,] 2 4
, , 2
      [,1] [,2]
[1,] 5 7
[2,] 6 8
```

A useful summary about a data frame may be obtained though call `str(...)`. This some extension of command `type` defined formerly for vector object. The `str(...)` displays the

Table 1.3: Some commands for matrix algebra in R.

Function	output/task
<code>A*B</code>	pointwise product of the given matrices <code>A</code> and <code>B</code> .
<code>A%*%B</code>	product of the given matrices <code>A</code> and <code>A</code> .
<code>as.matrix(df)</code>	considers a data frame <code>df</code> as a matrix.
<code>asplit(df,MARGIN)</code>	splitting an array, data frame, list, or matrix by its margins (1 for rows, 2 for columns, and <code>c(1,2)</code> for rows and columns).
<code>crossprod(A,B)</code>	computing $t(A) \%*\% B$.
<code>colMeans(A)</code>	computing average of columns of <code>A</code> .
<code>colnames(A)</code>	assigning names to columns of <code>A</code> .
<code>colSums(A)</code>	computing sum of columns of <code>A</code> .
<code>det(A)</code>	computing determinant of matrix <code>A</code> .
<code>dim(A)</code>	dimensions of matrix <code>A</code> .
<code>diag(A)</code>	diagonal of matrix <code>A</code> .
<code>diag(a)</code>	constructing identity matrix whose diagonal elements are elements of vector <code>a</code> .
<code>eigen(A)</code>	eigen vectors and values of matrix <code>A</code> .
<code>identical(A,B)</code>	logical: <code>TRUE</code> if matrices <code>A</code> and <code>B</code> are identical; and <code>FALSE</code> otherwise.
<code>is.matrix(df)</code>	logical: <code>TRUE</code> if the given data frame <code>df</code> is a matrix, and <code>FALSE</code> otherwise.
<code>isSymmetric(A)</code>	logical: <code>TRUE</code> if matrix <code>A</code> is symmetric, and <code>FALSE</code> otherwise.
<code>mahalanobis(x,mu,S)</code>	Mahalanobis distance, that is $(x-\mu)^T S^{-1} (x-\mu)$.
<code>ncol(A)</code>	the number of columns of matrix (data frame) <code>A</code> .
<code>qr(A)</code>	QR decomposition of matrix <code>A</code> .
<code>rowMeans(A)</code>	computing average of rows of <code>A</code> .
<code>rownames(A)</code>	assigning names to rows of <code>A</code> .
<code>rowSums(A)</code>	computing sum of rows of <code>A</code> .
<code>scale(A,center=TRUE,scale=TRUE)</code>	matrix whose columns are standardized columns of matrix <code>A</code> .
<code>solve(A)</code>	inverse of matrix <code>A</code> .
<code>t(A)</code>	transpose of matrix <code>A</code> .

internal structure of an complicated object such as array, data frame, list, or matrix. For example, recall data frame `df1`, we have

```
R> str(df1)
'data.frame': 27 obs. of 3 variables:
 $Factor1:Factor w/ 3 levels "high","low","medium":2 2 2 2 2 2 2 2 2 3 ...
 $Factor2:Factor w/ 3 levels "high","low","medium":2 2 2 3 3 3 1 1 1 2 ...
 $response:int 1 2 3 4 5 6 7 8 9 10 ...
```

The phrase 'data.frame' in output indicates that the type of object is data frame that consists of 27 observations on three variables. Then, a summary is given on each of three variables `Factor1`, `Factor2`, and `response`.

1.2 Condition and repeat

In what follows, we deal with briefly some tools that are useful for creating desired data structure, data refinement, comparisons, etc. Some tools are needed to create a user-defined function well. These tools have been defined for R and include `if(...)`, `ifelse(...)`, `while(...)`, and `for(...)`. In what follows, we briefly discuss aforementioned tools.

1.2.1 Conditional decision using if(...)

For accomplishing some decision under some predetermined *condition* (or nature state), we use this command. In general, let nature space Ω consist of mutually disjoint states **State 1**, **State 2**, and **State 3** so that

$$\text{State 1} \cup \text{State 2} \cup \text{State 3} = \Omega.$$

Furthermore, suppose we are willing to make **decision 1** (if **State 1** holds true), **decision 2** (if **State 2** holds true), and **decision 3** (if **State 3** holds true). For example, we can use the command `if(...)` for evaluating the sign of a real given number x . To this end, the pertinent pseudo and R codes are given by the following.

Pseudo code for command	R code for <code>if(...)</code> command
<code>if(State 1 holds true)</code>	<code>if(x==0)</code>
<code>{decision 1</code>	<code>{print(0) # decision 1</code>
<code>}else if(State 2 holds true){</code>	<code>}else if(x>0){</code>
<code>decision 2</code>	<code>print(1) # decision 2</code>
<code>}else{</code>	<code>}else{</code>
<code>decision 3}</code>	<code>print(-1) # decision 3</code>
<code>}</code>	<code>}</code>

If nature states consist of only two states, then we may use a simpler form of `if(...)` command called `ifelse(test,yes,no)`. Of course, the `if(...)` command still works well. Suppose, we are interested in turning nominal variable x that takes on either 'male' or 'female' into a numeric variable that takes on either 1 or 0, respectively. The pertinent `if(...)` and `ifelse(test,yes,no)` commands are as follows.

```
R> if(x=='male')      R> x<-ifelse(x=='male',1,0)
+ {
+   x<-1
+ } else {
+   x<-0
+ }
```

While using `if(...)`, we may use some operators. Table 1.4 lists some commonly used operators. It is worth to note that operator `a||b` evaluates b only if a is **FALSE** [Fox and Weis-

berg, 2018]. Let `c1<- c(1,1,2,2,1,2,1)` and `c2<- c(1,2,2,2,1,1,1)` denote the clustering results of two populations each consisting of two clusters 1 and 2. This fact that two clusters `c1` and `c2` are not fully matched can be shown by the following commands.

1.2.2 Repeat using while(...)

For repeating some task until some statement holds true, we may use command `while(...)`. This command essentially needs some initialization in relation to the given statement. In general, we should consider some counter

```
initialization
while(statement holds true)
{
body of while
}
```

Table 1.4: Some commonly used operators in R.

Operator	output/task
<=	less than or equal.
>=	greater than or equal.
!=	not equals.
a%%b	remainder of division a/b.
a%/%b	quotient of division a/b.
a==round(a)	logical: returns TRUE if a is an integer value, otherwise returns FALSE.
a b	logical: it applies element-wise to vectors a and b, and returns a vector of logical values.
a b	logical: it applies to numeric values a and b, and returns a single logical value. The output is TRUE if either a or b holds true.
a&b	logical: it applies element-wise to vectors a and b, and returns a vector of logical values. Each element of output is TRUE if either a or b holds true.
a&&b	logical: it applies to numeric values a and b, and returns a single logical value. The output is TRUE if both of a and b hold true.

```

R> a1<-ifelse(      R> a2<-ifelse(      R> if(all(c1-c2==0)
+all(c1-c2 == 0),    +any(c1/c2 != 1),    +|| all(c1/c2==1)
+"full", "not full") +"not full", "full") +a3<-"full" else a3<-"not full")
R> a1              R> a2              R> a3
[1] "not full"      [1] "not full"      [1] "not full"

```

Suppose we would like to print sum of nonnegative integer numbers provided that sum of these numbers does not exceed 10. Below, we give the command `while(...)` for this purpose.

R code	output
R> Sum <- 0; i <- 0 #initialization	[1] 0
R> while(Sum < 10){ #while(statement)	[1] 1
+ Sum <- Sum + i	[1] 3
+ print(Sum)	[1] 6
+ i <- i + 1	[1] 10
+ }	

1.2.3 Repeat using for(...)

For doing some task repeatedly for a given number, we may use command `for(...)`. Note that in contrast to `while(...)`, the number of iterations in `for(...)` is fixed and known. Let the non-numeric vector `sex` is defined as `sex=c('male','female','male','female','female','male')` that represents the sex of six persons. If we are willing to replace 'male' and 'female', respectively, with 1 and 0, then we can command `for(...)` as follows.

```

1 R> sex <- c('male','female','male','female','female','male')
2 R> n <- length(sex)
3 R> x <- rep(0, n) # creating a numeric vector called x of size n
4 R> for(i in 1:n)
5 + {
6 +   if( sex[i] == 'female')
7 +   {
8 +     x[i] <- 0
9 +   }else{
10 +     x[i] <- 1
11 +   }

```

We note that in order to constructing vector `x` in above, we may obtain `x` simply using command `x<-ifelse(sex=='female',0,1)`. The next example uses command `for(...)` to create a vector involving the names of rows (or columns) for a 10×10 matrix. The pertinent R code and output are as follows.

```
R> N1<-rep(NA,10)
R> for(i in 1:length(N1)){
+ N1[i]<-paste('a',
+ sep="-", i)}

[1] "a-1" "a-2" "a-3" "a-4"
[5] "a-5" "a-6" "a-7" "a-8"
[9] "a-9" "a-10"

R> N2<-rep(NA,10)
R> for(i in 1:length(N2)){
+ N2[i]<-paste(letters[i],
+ sep="", i) }

[1] "a1" "b2" "c3" "d4"
[5] "e5" "f6" "g7" "h8"
[9] "i9" "j10"
```

1.3 Data importing and exporting

Herein, we deal with reading *built-in* data sets existing in R and further discussing its nice property of exporting (writing) and importing (reading) data from other applications.

1.3.1 Reading built-in data

There are some applied data sets available in R that are known as *built-in* data. The list of data is ready for using by running command `R>data()`. There appears a list of several data sets. For calling some special data set, we need just to put the name of data inside the former command. Herein, we are willing to call two famous data sets known as `airquality` Chambers [2018], `iris` Fisher [1936] data. The first set is a data frame with 153 observations on 6 variables representing the daily air quality measurements in New York from May to September 1973. The second set consists of 50 measurements of features sepal length, sepal width, petal length, and petal width sampled from species *Setosa*, *Versicolor*, and *Virginica* of iris flower. For example, we may run the command

```
R> data(airquality)
R> head(airquality,6) #we may use tail(airquality,10) for last 10 lines
```

to see the first six rows of `airquality` data as follows.

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41	190	7.4	67	5	1
2	36	118	8.0	72	5	2
3	12	149	12.6	74	5	3
4	18	313	11.5	62	5	4
5	NA	NA	14.3	56	5	5
6	28	NA	14.9	66	5	6

Alternatively, one may use command `View(airquality)` to have a look at data in a new frame. It is worth to note that if you have access to some built-in data called `x` that other do not, one way to move it is as follows. First run command `dput(x)` and then copy the blue-colored output that starts with `structure(list(...` and then insert it at a new variable called `y` or email the blue-colored output to any destination.

1.3.2 Importing data from R package

We can read data in R from other applications. If data are appended to an R package, then we need to load the package that involves the data set. This is done through command `library(package name)` or `require(package name)`. For example, to access the DBH data existing in R package `ForestFit`, we proceed as follows.

```
R> library(ForestFit) # loading package
R> data(DBH)          # loading the pertinent data called DBH
R> head(DBH,2)        # representing first 2 lines of data
```

1.3.3 Importing data from other application

The prepared data for reading should be in tabular structure such as data frame or matrix and the acceptable formats are variants of `txt` including comma-, pipe-, or tab-delimited. Herein, the phrase delimiter refers to an object that separates the entries of two columns (fields). The comma-delimited format is also known as the comma separated values (CSV). Typically, the raw data are available from an external resource and reading data using command `read.table(...)` saves the user time and avoids some potential error arising from manual data entry. The main difference between all above mentioned types of `txt` data is the type of delimiter between columns. In general, command `read.table(...)` is a function that reads a tabular structure of data and converts it into a data frame. The arguments and the pertinent details are given in Table 1.5. Although `read.table(...)` can be used for all formats, but we may use commands

Table 1.5: Details about main arguments of `read.table(...)` in R.

Argument	output/task
<code>file</code>	the full address of file that is ready for reading. Each row of the tabular data would be a line of the output file.
<code>header</code>	logical: if <code>TRUE</code> , then the file contains the names of the variables as its first line; otherwise, a vector of letter "V" followed by column number will be assigned to columns as the name.
<code>row.names</code>	a vector of row names.
<code>col.names</code>	a vector of names for the variables. The default is a set consist of letters "V" followed by the column number.
<code>sep</code>	this parameter allows to set different delimiters including comma, pipe, semicolon tab, and one space for separating columns. It is assumed that the entries at each line of the imported file are separated by the character in front of <code>sep</code> parameter. If columns are comma-, pipe-, or tab-delimited, then <code>sep=","</code> , <code>sep=" "</code> , or <code>sep="\t"</code> , is recommended, respectively.
<code>na.strings</code>	a character or string that is recognized as missing value of input data. By default <code>na.strings="NA"</code> . If not, put the missing symbol instead of <code>NA</code> inside the quotation marks.
<code>dec</code>	a character assumed for decimal points.

`read.csv(...)`, `read.csv2(...)`, and `read.delim(...)` for reading tabular data with different delimiters as shown by Table 1.6.

For example, suppose we have a tabular data whose name is `A.csv` as

```
"", "a", "b", "c", "d", "e"
"a", 1, 2, 3, 4, .
```

Table 1.6: Functions for importing data in R.

Function	input data specifications
<code>read.csv(...)</code>	comma delimited (separated), that is <code>sep=','</code> , and <code>dec='.'</code>
<code>read.csv2(...)</code>	semi colon separates, that is <code>sep=';'</code> , and <code>dec=','</code>
<code>read.delim(...)</code>	tab separated, that is <code>'\t'</code> , and <code>dec='.'</code>

```
"b", 6, 7, 8, 9, 10
"c", 11, 12, 13, 14, 15
"d", 16, 17, 18, 19, *
```

existing at address `E:\my R work\A.csv`. As it may be seen, two characters “.” and “*” are denoting missing data and further letters `a,b,c,d,e` are assigned as names to the rows and columns. Using command `read.csv(...)`, then the following output is obtained.

Command	output
<code>R> out <- read.csv("E:/my R code/A.csv",</code>	<code>X a b c d e</code>
<code>+ header = T, na.strings=c('*', '.'))</code>	<code>1 a 1 2 3 4 NA</code>
<code>R> out</code>	<code>2 b 6 7 8 9 10</code>
	<code>3 c 11 12 13 14 15</code>
	<code>4 d 16 17 18 19 NA</code>

1.3.4 Importing xls and xlsx data

Sometimes the original data are needed to read from Microsoft Excel application. One way to read such data in R is to install the package `readxl`¹ developed for R (≥ 3.6). For instance, suppose the well known built-in data `mtcars` is available in `xls` format at path `E:\my R work\Mtcars.xls` and we are willing to import it using package `readxl`. To do this, we proceed as follows.

```
R> library(readxl)
R> datxl1 <- read_excel("E:/my R work/Mtcars.xls", sheet=1, range="A1:L33")
R> datxl2 <- as.data.frame(datxl1)
```

All `Mtcars.xls` data are collected in just one sheet. If there are other sheets, then the argument `sheet` allows user to import another existing data sheet. If user interested in importing a predetermined range of data sheet, then the argument `range` can be changed appropriately. The range of `Mtcars.xls` data is `"A1:L33"`. For example, some other types of importing a predetermined part of data sheet is given as follows.

```
read_excel("Mtcars.xls", range="mtcars!B1:D5")
read_excel("Mtcars.xls", range=cell_rows(1:4))
read_excel("Mtcars.xls", range=cell_cols("B:D"))
```

Alternatively, for `xls` format as above, the commands `read_xls` also works. Unless stated otherwise, any blank cell in the original Excel data sheet are considered as a missing value. Any change is permitted using argument `na=" "`. More detailed information on other arguments is accessible running `args(read_excel)`. Moreover, for data of format `xlsx`, the command `read_xlsx(...)`

¹<https://cran.r-project.org/package=readxl>.

1.3.5 Data exporting

Likewise, data can be exported from R for other applications as an input. Table 1.7 shows the arguments of command `write.table(...)`. If the format of output data is `csv`, then the command `write.csv(...)` is preferable. For example, we may apply command `write.table(...)`

Table 1.7: Details about main arguments of `write.table(...)` in R.

Argument	output/task
<code>x</code>	the name of file to be exported.
<code>file=""</code>	the full address of the output file.
<code>append</code>	logical: By default is <code>FALSE</code> that means the old output is replaced with the new one, otherwise the new output will be added at the end of the old one.
<code>na="NA"</code>	the missing values in the output file is denoted by "NA" by default, otherwise the character must be determined by user.

to the first 6 lines of the `airquality` data to produce a `txt` file, namely `air.txt`, for which columns are tab-separated, decimal points are represented as slash "/", missing data are denoted by "*" rather than "NA", and finally row names are removed.

Command	output					
<code>R>write.table(head(airquality,6), +"E:/my R code/air.txt", sep='\t', + dec='/', na='*')</code>	"Ozone"	"Solar.R"	"Wind"	"Temp"	"Month"	"Day"
	41	190	7/4	67	5	1
	36	118	8	72	5	2
	12	149	12/6	74	5	3
	18	313	11/5	62	5	4
	*	*	14/3	56	5	5
	28	*	14/99	66	5	6

1.3.6 Apply family

A wide range of functions can be applied to the members of array, data frame, matrix, and vector through `apply` family of functions. Herein, we will briefly discuss these functions.

1. `apply(A, MARGIN, FUN, ...)`: The data structure `A` can be matrix, array, or data frame (with numeric values). If `MARGIN=1` (or `2`), then function `FUN` is applied to the rows (or columns) of `A`. The output of `apply` would be a vector. It is worth to note that "..." accounts for some additional arguments that are passed to the function `apply`. For example, we may add argument `na.rm=T` that means the missing values are removed when applying function `FUN` to the coordinates of data structure `A`. For instance, the sum of rows, mean of columns, and variance of rows of matrix `A` denoted as above is obtained as follows.

```
R> apply(A, 1, sum)      R> apply(A, 2, mean)      R> apply(A, 1, var)
[1] 4      6              [1] 1.5      3.5              [1] 2      2
```

2. `lapply(A, FUN, ...)`: The data structure `A` can be data frame, list, matrix, or vector. The command `lapply` applies function `FUN` to each element of object `A`. The output of `lapply` would be a list. For example, let function `f1` is defined as

```
R> f1<- function(x) 10*x
```

then the command `lapply(df2, f1)` applies function `f1` to the columns (members) of data frame `df2` defined as `df2 <- data.frame(x=1:2, y=c(-2,2))`. The output is

```
$x
[1] 10 20
$y
[1] -20 20
```

3. `sapply(A, INDEX, FUN,...)`: Both functions `lapply` and `sapply` do the same task, but the input of `sapply` is vector, data frame, or list and moreover, the output of the `lapply` is a list while the results of `sapply` is vector. Let we are interested in standardizing the columns of data frame `df1` defined earlier. To do this, we have

Command	output		
<code>R> sapply(df1, function(x)</code>	<code>col1</code>	<code>col2</code>	<code>col3</code>
<code>+ c(x-mean(x))/sd(x))</code>	<code>[1,]</code>	<code>NaN</code>	<code>-1</code>
	<code>[2,]</code>	<code>NaN</code>	<code>0</code>
	<code>[3,]</code>	<code>NaN</code>	<code>1</code>

Not surprisingly, the first column of output in above is NAN since we have undetermined case 0/0.

4. `tapply(A, INDEX, FUN,...)`: The function `tapply` in R can be used when we want to apply some function to a vector that is grouped by some other vector. The input of `tapply` can be array, data frame, matrix, or list. In what follows, suppose we would like to compute the variance of variable `Sepal.Length` within `iris` data in terms of variable `Species`. We have

<code>R>data(iris)</code>	<code>R>data(iris)</code>
<code>R>tapply(iris\$Petal.Length,</code>	<code>R>tapply(iris\$Petal.Length,</code>
<code>+ iris\$Petal.Width<1,var)</code>	<code>+ iris\$Species=="setosa",var)</code>
<code>FALSE</code>	<code>FALSE</code>
<code>TRUE</code>	<code>TRUE</code>
<code>0.68157980</code>	<code>0.68157980</code>
<code>0.03015918</code>	<code>0.03015918</code>

As it may be seen, not surprisingly, both parts in output above are the same due to the fact that `iris$Petal.Width<1` and `iris$Species=="setosa"` are equal sets. We conclude that the variance of `Petal.Length` when variable `Petal.Width` is less (more) than one is 0.03015918 (0.68157980). The function `tapply` can be employed for applying `FUN` on a vector that is grouped by two or more variables. For example, suppose we are willing to compute the average of `response` variable involved in data frame `df1` introduced earlier while two constraints including `Factor1='low'` and `Factor2='high'` are applied. The pertinent command and output are given by the following.

<code>R>tapply(df1\$response, list(</code>	<code>R>tapply(df1\$response, list(</code>
<code>+ df1\$Factor1=='low', df1\$</code>	<code>+ Factor1=df1\$Factor1, Factor2=</code>
<code>+ Factor2=='high'), mean)</code>	<code>+ df1\$Factor2), mean)</code>
<code>FALSE</code>	<code>Factor2</code>
<code>TRUE</code>	<code>high low medium</code>
<code>FALSE</code>	<code>Factor1</code>
<code>TRUE</code>	<code>high</code>
	<code>low</code>
	<code>medium</code>
<code>17.0</code>	<code>26</code>
<code>21.5</code>	<code>20</code>
<code>3.5</code>	<code>8</code>
<code>8.0</code>	<code>2</code>
	<code>5</code>
	<code>17</code>
	<code>11</code>
	<code>14</code>

1.4 Graphical summary

Data visualization is one of the most crucial elements in data analysis. A main part of statistical evaluations and comparisons is based on graphical summary. We may use different visualization facilities in R, depending on the type of data set. In general, graphical summaries are either high-level or low-level. High-level graphical facilities always create a new plot window and erase the current plot if necessary; rather the low-level ones, in general, do not create a new plot window. Among the high-level summaries, for example, we may mention the scatterplot for representing two sets of numeric data running `plot(...)`. If two data sets are of non-numeric or different types, then a contingency table may be suggested for graphical summary representation using `table(...)`. In what follows, we list some important high- and low-level graphical facilities.

- i. **High-level:** `plot(...)`, `pers(...)`, `pairs(...)`, `image(...)`, `hist(...)`, `dotchart(...)`, `contour(...)`, `coplot(...)`, `boxplot(x~y,...)`, and `barplot(...)`.
- ii. **Low-level:** `abline(...)`, `axis(...)`, `curve(...)`, `expression(...)`, `legend(...)`, `lines(...)`, `par(...)`, `segments(...)`, and `text(...)`.

In what follows we deal with the `plot(...)` among the other high-level graphical facilities.

1.4.1 Plot(...)

We may call `plot(x,y,...)` to display a two-dimensional visual summary of horizontal (x) and vertical (y) coordinates of numerical data points. The three dots `...` represent further arguments to be passed to object `plot(x,y,...)` that are optional. Table 1.8 lists some descriptions for some important arguments of this object. Herein, we call `trees` built-in data using command `data(trees)` for our graphical illustration. This data set consists of 31 measurements of felled black cherry trees on variables: girth (diameter), height, and volume². Figure 1.1(a) shows the scatterplot of height versus girth and Figure 1.1(b) displays the scatterplot of height and volume versus girth simultaneously. The command for producing these figures are given as follows.

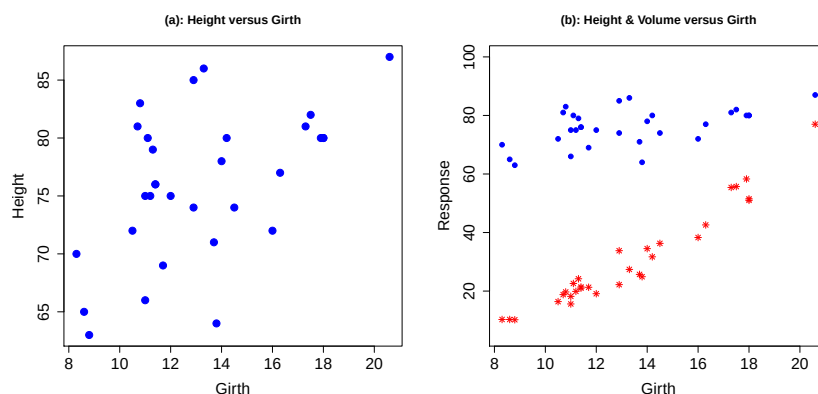


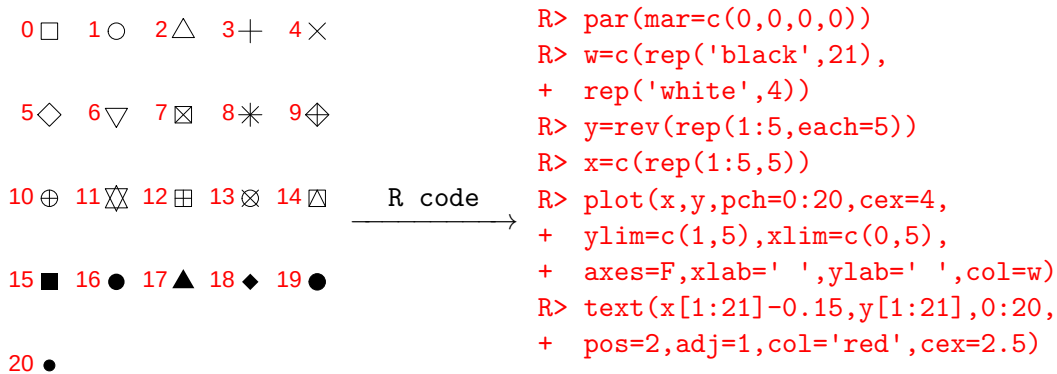
Figure 1.1: Scatterplot of (a): height vs. girth and (b): height and volume vs. girth.

```
R> # commands for producing Subfigure(a)
R> data(trees); x<-trees$Girth; y<-trees$Height; z<-trees$Volume; m<-1.5
R> plot(x, y, main="Height versus Girth", xlab="Girth", ylab="Height",
+ type="p", col="blue", pch=19, cex=m, lwd=m, cex.lab=m, cex.axis=m)
R> # commands for producing Subfigure(b)
R> plot(x, y, main="Height & Volume versus Girth", xlab="Girth", ylab=
+ "Response", type="p", col="blue", pch=19, ylim=c(5,100), lwd=m,
```

²Diameter is recorded at around 1.3 meters (or 4 to 6 feet) above the ground

Table 1.8: Details about main arguments of `plot` in R.

Argument	output/task
<code>cex</code>	the size of symbol suggested by <code>pch</code> .
<code>cex.axis</code>	the size of axes labels.
<code>cex.lab</code>	the size of axes title.
<code>cex.main</code>	the size of main title.
<code>col</code>	name of color defined for plot points inside the quotation mark.
<code>font</code>	the font type and font style of axes labels that is determined using object <code>Hershey+</code> for font type and its style.
<code>lty</code>	an integer number representing the line type: 0=blank, 1=solid (default), 2=dashed, 3=dotted, 4=dotdash, 5=longdash, 6=twodash.
<code>lwd</code>	the thickness (width) of lines and symbols.
<code>mgp=c(a,b,c)</code>	it adjusts distance between axes lines and axes title (through <code>a</code>), axes labels (through <code>b</code>), and axes ticks (through <code>c</code>)
<code>pch</code>	an integer number from 0 to 20 denoting the symbol of data points shown as Figure 1.2.
<code>type</code>	it is "b" (for both points and lines), "c" (for empty points joined by lines), "h" (for histogram-like vertical lines), "l" (for lines), "n" (for any points or lines), "o" (for overplotted points and lines), "p" (for points), "s" (for a stair steps), and etc.
<code>xlab</code>	label for horizontal axis inside the quotation mark.
<code>ylab</code>	label for vertical axis inside the quotation mark.
<code>xlim</code>	<code>c(a,b)</code> that restricts the range of horizontal axis from <code>a</code> to <code>b</code> .
<code>ylim</code>	<code>c(a,b)</code> that restricts the range of vertical axis from <code>a</code> to <code>b</code> .
<code>yaxt="n"</code>	removes the labels of vertical axis.
<code>xaxt="n"</code>	removes the labels of horizontal axis.

Figure 1.2: Commonly used symbols for `pch` in R and the pertinent code.

```
+ cex.lab=m, cex.axis=m)
R> lines(x,z,type="p",col="red",pch=8)
```

After producing figure, user can save the produced figure freely in such formats as `bmp`, `jpeg`, `pdf`, `png`, and `tiff`. Herein, we use command `pdf(...)` for this end. Details about arguments of these calls are given using, for example, `?tiff`. Suppose the name of created figure is `FigA` and we would like to save it with portable document format (`pdf`) in track `E\my R work`. To do this, we have

```
R> setwd("E:/my R work")
```

```
R> pdf(file="FigA.pdf",width=4,height=4) # width and height are in inches
R> # put here command plot(x,y,...)
R> # put here command line(x,y,...)
R> dev.off()
```

More manipulations can be made on produced plot using low-level graphical facilities. Table 1.9. Figure 1.3 shows two sample plots produced based on some supplementary arguments given in Table 1.9. The corresponding R codes are given as follows.

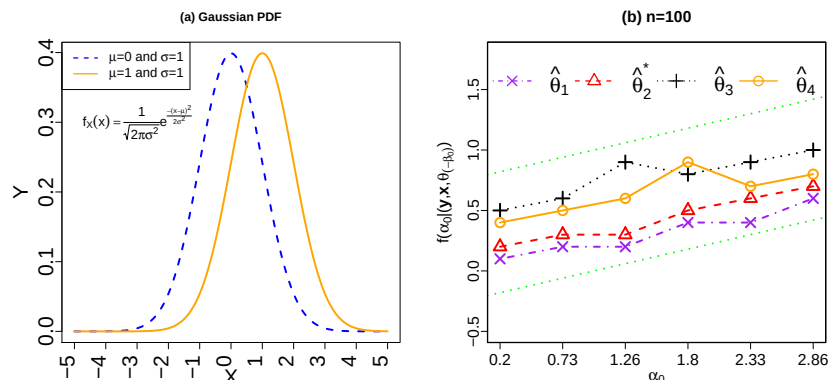


Figure 1.3: Two sample plots produced based on some supplementary calls of Table 1.3.

```
R># R code for producing Figure 1.2(a)
R>x<-seq(-5,5,.01); posi<-seq(-5,5,1)
R>plot(x,dnorm(x),col="blue",type='l',xlab='X',main="
+(a) Gaussian PDF",mgp=c(2.5,.5,0), ylab='Y',cex=1,
+lty=2,lwd=3,xaxt='n',cex.lab=2,cex.axis=2,pch=8)
R>lines(x,dnorm(x,1,1),col='orange',lwd=3, pch=19)
R>Leg1<-expression( paste(mu,"=0 and ",sigma,"=1"))
R>Leg2<-expression( paste(mu,"=1 and ",sigma,"=1"))
R>mytext<-expression(paste(f[X](x)==frac(1,sqrt(2*
+pi*sigma^2))*e^{frac(-(x-mu)^2,2*sigma^2)} ))
R>text(-3,0.3,mytext,cex=1.25)
R>axis(1,at=posi,lab=posi,hadj=1,padj=0,las=2,cex.axis=2)
R>legend('topleft',legend=c(Leg1,Leg2),lty=c(2,1),col=
+c('blue','orange'),cex=1.25,pch=c(8,19),lwd=2)
R># R code for producing Figure 1.2(b)
R>x1<-c(0.1,0.2,0.2,0.4,0.4,0.6);x2<-c(0.2,0.3,0.3,0.5,0.6,0.7)
R>x3<-c(0.5,0.6,0.9,0.8,0.9, 1);x4<-c(0.4,0.5,0.6,0.9,0.7,0.8)
R>vmle <-expression(paste( hat(theta)[1] ))
R>vstar <-expression(paste(hat(theta)[2]^*))
R>vhat1 <-expression(paste( hat(theta)[3] ))
R>vhat2 <-expression(paste( hat(theta)[4] ))
R>valpha<-expression(paste( alpha[0] ))
R>vylab <- expression(paste("f(",alpha[0],"|(", bold(y),"",
+bold(x),"", bold(theta)[(-beta[0])]),")"))
R>x0 <- seq(0.2, 5, .5); v1<-c(vmle, vstar, vhat1, vhat2); m<-1.5;
R>plot(x1,cex.lab=m,cex.main=m,cex=2,main="(b) n=100",cex.axis=m,
+col="purple",lty=4,lwd=3,mgp=c(2.45,0.5,0),pch=4,type="o",
+xaxt='n',xlab=valpha,ylab=vylab,ylim=c(-0.5,1.8))
R>lines(x2, cex=2, col="red" ,lty=2, lwd=3, pch=2 ,type="o")
R>lines(x3, cex=2, col="black" ,lty=3, lwd=3, pch=3 ,type="o")
R>lines(x4, cex=2, col="orange",lty=1, lwd=3, pch=1 ,type="o")
R>axis(1, at=seq(1,length(x0)), cex.axis=m,+labels=c("0.2","0.73",
+"1.26","1.8","2.33","2.86","3.4","3.93","4.46","5"))
```

Table 1.9: Details about arguments of low-level graphical facilities.

Function/argument	output/task
<code>abline(a=NULL,b=NULL,h=NULL,v=NULL,...)</code>	it adds one or more straight lines to the plot. Parameters <code>a</code> and <code>b</code> are, accordingly, the intercept and slope of the fitted regression line. Parameters <code>h</code> and <code>v</code> are, respectively, the position of drawn horizontal and vertical lines to the plot.
<code>axis(side,at=NULL,labels=TRUE,...)</code>	
<code>side</code>	an integer number indicating the side of the plot we are willing to work on. That includes 1 (for <code>below</code>), 2 (for <code>left</code>), 3 (for <code>above</code>), and 4 (for <code>right</code>).
<code>at</code>	a vector of numerical values indicating the location at which the tick marks are shown.
<code>labels</code>	a vector of numeric or non-numeric data to be inserted at points determined in <code>at</code> argument.
<code>las</code>	an integer (0=always parallel to the axis [default] and 1=always perpendicular to the axis) value.
<code>hadj</code>	a numeric number for moving the position of all axis labels simultaneously parallel to the reading direction.
<code>padj</code>	a numeric number for moving the position of all axis labels simultaneously perpendicular to the reading direction.
<code>curve(expr,from=NULL,to=NULL,n=101,add=FALSE)</code>	it works similar to call <code>plot(...)</code> for depicting a curve with formula <code>expr</code> over the given range starting at <code>from</code> and ending at <code>to</code> . If we want to add the output of call <code>curve(...)</code> to the output of plot, we must set <code>add=TRUE</code> .
<code>expression(paste(...))</code>	this command is useful when we want to add some annotations that typically involves mathematical formula. A full list of to a full list of mathematical symbols and formula run <code>demo(plotmath)</code> .
<code>legend('position',legend,lwd,col,horiz=FALSE,...)</code>	
<code>position</code>	one of <code>top</code> , <code>bottom</code> , <code>topleft</code> , <code>topright</code> , <code>bottomleft</code> , or <code>bottomright</code> . If some position else is desired, the <code>x</code> and <code>y</code> coordinates must be given instead.
<code>legend</code>	a vector of explanations for each line existing in the plot.
<code>lwd</code>	width of lines (or symbols) existing in argument <code>legend</code> .
<code>col</code>	color of lines existing in <code>legend</code> .
<code>horiz=FALSE</code>	logical: if <code>TRUE</code> , then the expressions are shown horizontally.
<code>lines(x,y,...)</code>	it adds a new curve to object <code>plot(...)</code> and its arguments are the same as <code>plot(...)</code> .
<code>par(...)</code>	it includes 72 graphical parameters or queries. The full list of these parameters is accessible by running <code>par(...)</code> . It is worth to note that if user changes the setting of object <code>par</code> , then these change will be applied to all graphs whenever the session is working. To restore the default settings, for other plots, calling <code>on.exit(...)</code> is suggested.
<code>segments(x0,y0,x1=x0,y1=y0,...)</code>	it draws a line from starting point whose <code>x</code> and <code>y</code> coordinates are, accordingly, <code>x0</code> and <code>y0</code> to end point whose <code>x</code> and <code>y</code> coordinates are, accordingly, <code>x1</code> and <code>y1</code> .
<code>text(a,b,'note',font,...)</code>	it puts a <code>note</code> at a position whose <code>x</code> and <code>y</code> coordinates are <code>a</code> and <code>b</code> , respectively. Parameter <code>font</code> determines the <code>note</code> , for which a list of fonts and styles can be found in object <code>Hershey</code> . Finally, three dots <code>...</code> represents further arguments that must be passed, for example <code>col</code> (for color), <code>cex</code> (for size), etc.

```
R>cols<-c("purple","red","black","orange")
R>legend("topleft",horiz=TRUE, legend=v1,bty="n",lwd=2,cex=2,
+col=cols,pch=c(4,2,3,1),lty=c(4,2,3,1))
R>abline(a=0.7,b=0.12,lty=3,lwd=3,col='green')
R>abline(a=-0.3,b=0.12,lty=3,lwd=3,col='green')
R>box(lwd=2)
```

1.4.2 Pairwise visualization

When there are more than two sets of numeric data, then a more useful graphical summary about the variables involved in study and associated relationships can be found using object `pairs(...)`. There is a long list of arguments for this object, among them in order to save space, we mention the `lower.panel`, `upper.panel`, and `diag.panel` that are more important than others. Suppose we are interested in relationship between last three variables, excluding `Species` iris data. A graphical summary may be obtained such as Figure 1.4 that displays the pairwise scatterplots of these three variables. More changes on produced pairwise scatterplot is possible by changing the arguments `lower.panel`, `upper.panel`, and `diag.panel` appropriately. Herein, we use our user-defined function by setting `diag.panel=panel.hist` for presenting the histogram of each marginal within the diagonal cells of pairwise scatterplots of Figure 1.5 while the superimposed appropriate Gaussian curve helps in discovering departure from normality.

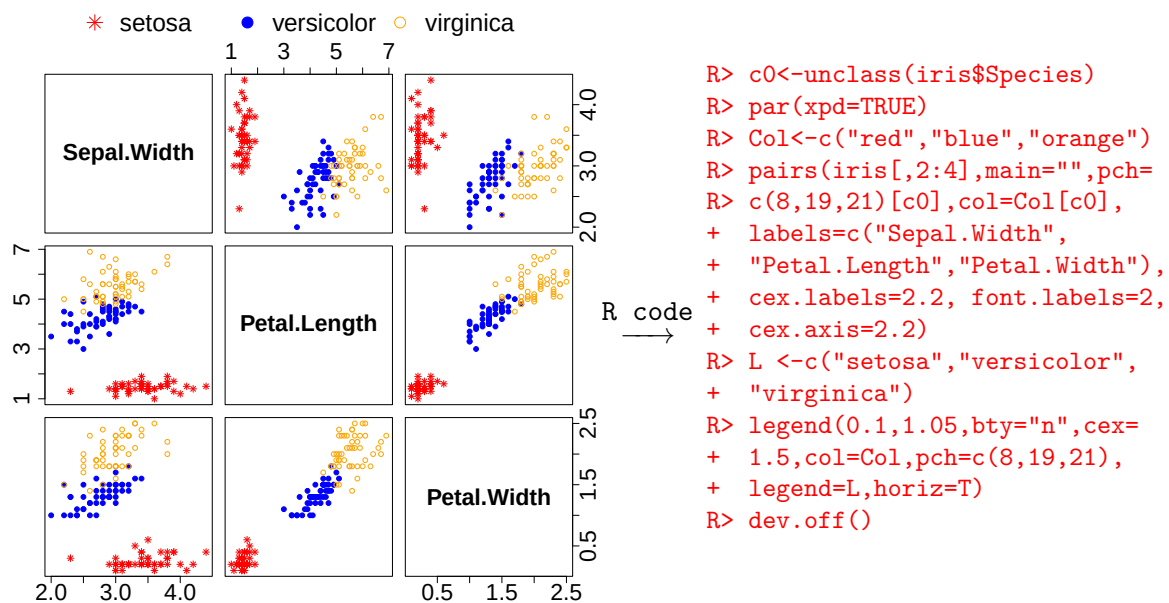


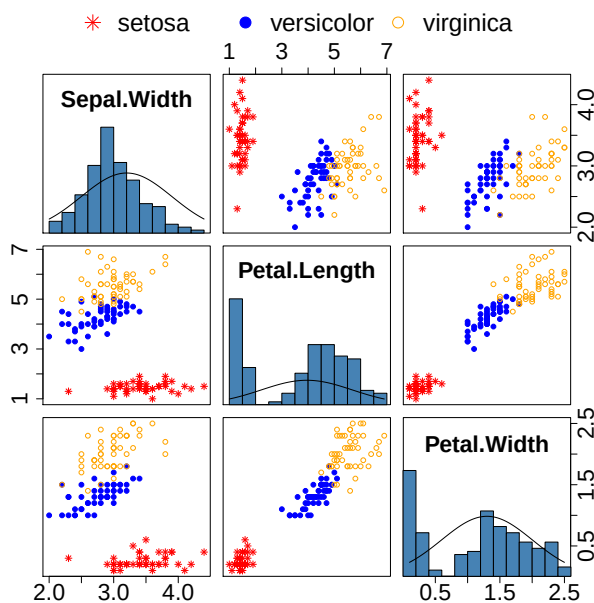
Figure 1.4: Pairwise scatterplots of iris data.

1.4.3 Contingency table

One useful graphical summary associated with both numeric and non-numeric data is contingency table. For constructing a contingency table we may use object `cut(...)` that splits numeric data into a number of mutually disjoint intervals (groups). Details about arguments of this object is given by Table 1.10. Recalling `women` data, we call `cut(...)` for grouping this

Table 1.10: Details about arguments of `cut(...)`.

Argument	output/task
<code>x</code>	a given numeric vector for grouping.
<code>breaks (br)</code>	either a numeric vector representing cut points or a single value > 2 representing the number of groups.
<code>labels=NULL</code>	optional labels for the created groups.
<code>include.lowest</code>	logical: if TRUE and $x[i]$ coincides with the lower bound of group, then group includes $x[i]$, otherwise $x[i]$ belongs to another group.
<code>right</code>	logical: if TRUE, then each group takes the form $(a,b]$, otherwise it would be of the form $[a,b)$.
<code>...</code>	a number of arguments to be passed.



R code

```
R> panel.hist<-function(x,col,...){
+   usr<-par("usr");on.exit(par(usr))
+   par(usr=c(usr[1:2],0,1.5))
+   h<-hist(x, plot=FALSE)
+   breaks<-h$breaks
+   nB<-length(breaks)
+   y<-h$counts; y<-y/max(y)
+   rect(breaks[-nB],0,breaks[-1],
+   y,col="steelblue",...)
+   curve(dnorm(x,mean=mean(x),
+   sd=sd(x)),add=T)
+}
R> c0<-unclass(iris$Species)
R> par(xpd=TRUE)
R> Col<-c("red","blue","orange")
R> pairs(iris[,2:4],main="",pch=
R> c(8,19,21)[c0],col=Col[c0],
+   diag.panel =panel.hist,
+   labels=c("Sepal.Width",
+   "Petal.Length","Petal.Width"),
+   cex.labels=2.2, font.labels=2,
+   cex.axis=2.2)
R> L <-c("setosa","versicolor",
+   "virginica")
R> legend(0.1,1.05,bty="n",cex=
+   1.5,col=Col,pch=c(8,19,21),
+   legend=L,horiz=T)
R> dev.off()
```

Figure 1.5: Pairwise scatterplots of `iris` data with fitted Gaussian curve to the marginals.

set of data as follows.

Command	output			
R>br.h<-c(50,60,70,80)	W			
R>br.w<-c(110,130,150,170)	H	(110,130]	(130,150]	(150,170]
R>H<-cut(women\$height,br=br.h)	(50,60]	3	0	0
R>W<-cut(women\$weight,br=br.w)	(60,70]	3	6	1
R>table(H,W)	(70,80]	0	0	2

The focus of the second example is in `wage` (wage and other information of 3000 male workers in the Mid-Atlantic region) data available at package `ISLR`. Suppose we are interested in relationship between marital status and wage variables. The former is non-numeric while the latter is numeric. One way to deal with relation between these two variables, first we may divide wage variable into two levels `High` (greater than median) and `Low` (less than median) and then investigating the chi-squared independence test. The output of chi-square independence test, after running `chisq.test(Table)`, is given by the following.

```
Pearson's Chi-squared test
data: Table
X-squared = 541.37, df = 4, p-value < 2.2e-16
```

Obviously, as it may be seen from Figure 1.6, we decide to reject the null hypothesis of H_0 : “Wage and education variables are independent” against H_1 : “Wage and education variables are not independent”.

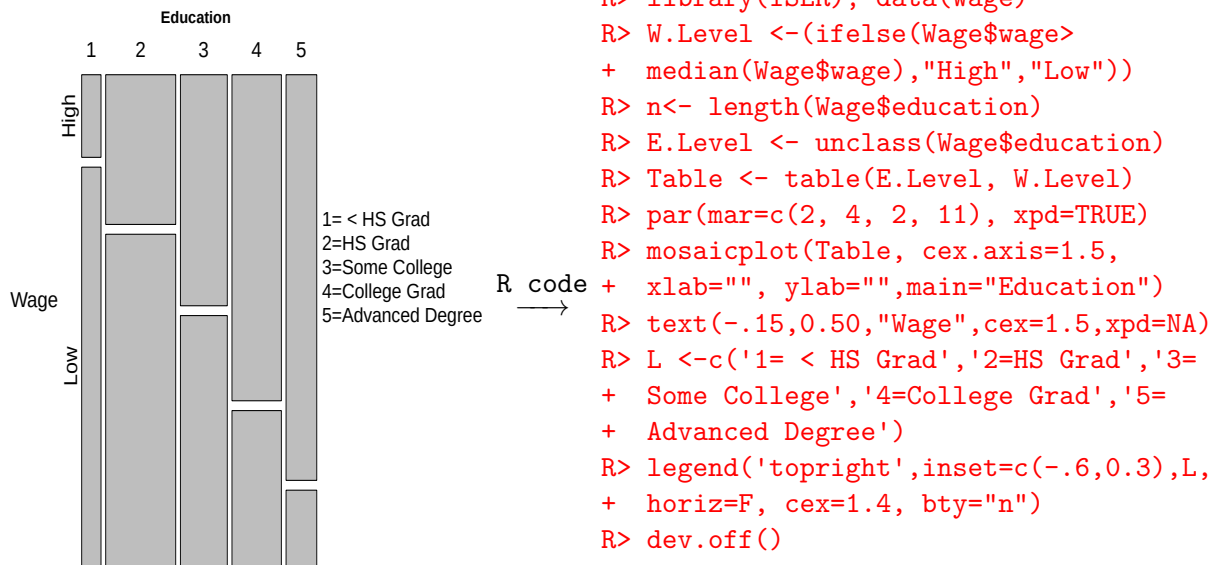


Figure 1.6: Mosaicplot for *Wage* data.

1.5 Function in R

When we would like to accomplish some task(s) repeatedly, it is recommended to use some structure called *function* in order to saving space, reducing the computational costs, debugging errors and warnings easily, etc., within an R project. In general, an R function is either *built-in* or *user-defined*. Tables 1.1, 1.3, and 1.11 involve some built-in functions. The user-defined function are those created by user for some special purpose. Each function has four parts including *name*, *input*, *body*, and *output*. Two key points when creating a user-defined function are: i) to avoid constructing a function whose built-in counterpart does exist and ii) the name of the user-defined and existing built-in functions must be different; otherwise the user-defined function will *mask* the built-in function. The general structure of user-defined function is given as follows.

```
name <- function(input)
{
  body of function
  return(output) # we may write output instead
}
```

The input may be array, data frame, list, matrix, single value, vector, or a combination of these. The body of function is a set of R commands in order to achieve the goal (output). If the output has just one part, then user is free to write *output* rather than *return(output)* for returning the output of function. It sounds worthwhile to note that if the body of function consists of only one sentence then the curly brackets (braces) and *return(output)* can be removed in order to relax the created function. Table 1.11 lists with some popular primitive built-in functions.

For example, if we want repeatedly standardize a numeric vector, then we may create a function called *my.scale* for this purpose as follows.

```
R> my.scale <- function(v) (v - mean(v)) / sd(v)
```

Of course, we do not suggest to use the user-defined function *my.scale* for practical purposes since its built-in counterpart exists. Hence, the user-defined are mainly developed to perform

Table 1.11: Some primitive built-in functions in R.

Function	output/task
<code>abs(x)</code>	absolute value of <code>x</code> .
<code>atan(x)</code>	inverse of trigonometric function <code>tan(x)</code> .
<code>exp(x)</code>	natural exponential of <code>x</code> .
<code>Im(x)</code>	imaginary part of <code>x</code> .
<code>log(x)</code>	logarithm of <code>x</code> based on number <code>y=exp(1)</code> by default.
<code>lgamma(x)</code>	logarithm of $\Gamma(x)$.
<code>Re(x)</code>	real part of complex number <code>x</code> .
<code>sign(x)</code>	takes on values -1, 0, or 1, if <code>x<0</code> , <code>x=0</code> , or, <code>x>0</code> , respectively.
<code>sqrt(x)</code>	square root of <code>x</code> .
<code>x^y</code>	<code>x</code> to power <code>y</code> (often symbol <code>^</code> is created by <code>[Shift] + [6]</code>).

those tasks for which the built in function is not applicable. As another example, suppose we would like to create a function that computes the sign of a given real value. Recall that we have just computed the sign of vector `sex`. A little more work required to create function called `my.sign` for computing sign of given vector `x`. The function `my.sign` is given as follows.

```

1  R> my.sign <- function(x)
2  +{
3  + n <- length(x)
4  + y <- rep(0, n) # a numeric vector called y of size n as output
5  + for(i in 1:n)
6  + {
7  +   if( x[i] == 0)
8  +   {
9  +     y[i] <- 0
10 +   }else if (x[i] > 0){
11 +     y[i] <- 1
12 +   }else{
13 +     y[i] <- -1
14 +   }
15 + }
16 + return(y)
17 +}
```

It is worthwhile to note that each user-defined function must be checked to see does it work well or not, and next priorities are speed, memory usage, and etc.

1.5.1 Built-in function for character

Working with character or string is also important. The built-in functions have been developed effectively to deal with such type of variables in R. Herein, among them, we briefly introduce `grep`, `grepl`, `gregexpr`, `letters`, `regexec`, `regexpr`, `sub`, `gsub`, `substr`, `strsplit`, `toupper`, `tolower`. These functions, in general, have been developed for finding some given pattern within a non-numeric vector. Among aforementioned commands, we are willing to discuss briefly the commands `grep` and `substr`. Details about arguments of `grep` are given in Table 1.12. Suppose we would like to find pattern 'Bd' inside the non-numeric atomic vector

`x <- c('abcd', 'Bdcd', 'abcdabcd', 'bd2')`. Hence, we define

```
R> x <- c('abcd', 'Bdcd', 'abcdabcd', 'bd2'); pattern <- 'Bd'
```

We run command `grep(...)` for four different combinations of its arguments as follows.

```
R>grep(pattern,x,ignore.case=T,value=T) R>grep(pattern,x,ignore.case=T,value=F)
```

Table 1.12: Details about main arguments of **grep** in R.

Argument	output/task
pattern	a sequence of character(s) that we want to find it within some given non-numeric vector.
x	given non-numeric vector.
ignore.case	logical: if TRUE, the matched sequence will be found regardless of being upper or lowercase.
value	logical: if TRUE, then it returns the vector of matched sequence; otherwise, the indices vector is returned.

```
[1] "Bdcd" "bd2" [1] 2 4
```

```
R>grep(pattern,x,ignore.case=F,value=T) R>grep(pattern,x,ignore.case=F,value=F)
```

```
[1] "Bdcd" [1] 2
```

Details about arguments of function **substr(...)** is given in Table 1.13. This command is mainly used to extract or replace substrings in a given non-numeric vector. For example, if

Table 1.13: Arguments of **substr(...)** in R.

Argument	output/task
x	a non-numeric vector.
start	first element to be extracted (or replaced).
stop	last element to be extracted (or replaced).
value	a string to be replaced.

we want to substitute the string 'sm' into the second and third places of elements of vector **x** defined earlier, or further, extract the second and third elements of vector **x**. The corresponding commands and outputs are given as follows. In a further example, e are willing to separate digits

R code	output
<pre>R>x<-c('abcd','Bdcd','abcdabcd','bd2')</pre>	
<pre>R>value<-'sm'; substr(x,2,3)<-value</pre>	
<pre>R>x</pre>	<pre>[1] "asmd" "Bsmd" "asmdabcd" "bsm"</pre>
<pre>R>x<-c('abcd','Bdcd','abcdabcd','bd2')</pre>	
<pre>R>substr(x,2,3)</pre>	<pre>[1] "bc" "dc" "bc" "d2"</pre>

of a given number. To this end, we may use **gsub** command to separate digits of 2024 as follows.

R code	output
<pre>R> val1 <- as.character(2024)</pre>	<pre>[1] 2 0 2 4</pre>
<pre>R> val2 <- gsub("(.)", "\\1 ", val1)</pre>	
<pre>R> val3 <- strsplit(val2, " ")[[1]]</pre>	
<pre>R> val4 <- as.numeric(val3); val4</pre>	

1.6 External R package

1.6.1 External R package initialization

As mentioned before, attaching R packages is privileged to users for implementing the advanced or more technical statistical methods. Herein, we would like to deal with utilizing an

external R package. To this end, the first step is prepare the package's documentation (or Reference manual) that is given in pdf available at <https://cran.r-project.org/web/packages/pakagename>. The documentation gives necessary information on R package. There are several features about each R package that, for example, may be seen at <https://cran.r-project.org/doc/manuals/r-devel/R-exts.html#The-DESCRIPTION-file>. These features must be investigated whenever user wants to install and then use a package, among them we mention the following.

- i. **Depends:** This field gives a numbers of comma-separated package names that the desired package depends on. The prerequisite packages will be installed while the current package is installed. This field also may specify a dependence on a certain version of R. For instance, if the desired package works only with version 4 or later, we may see `R (>= 4.0)` in the "Depends" field.
- ii. **Suggests:** This field has the same characteristics as field "Depends" does. But, the list of packages in front of this field are not necessary to install. The prerequisite package(s) are just used for implementing examples, tests existing in the desired package.
- iii. **Package source:** This fields contains the compressed source code of the desired package. For example, the compressed source code of package `nortest` is given by `nortest_1.0-4.tar.gz` in this field.

1.6.2 External R package installation

In this part, we are willing to deal with installing an external Rpackage. To this end, we select the `nortest` available at <https://cran.r-project.org/package=nortest>. The first step for installing an external Rpackage is study the documentation file of `nortest` as mentioned before. If all features hold true, then we use the steps given by the following for using this package for all practical purposes.

```
R > address = "E:/my R code"
R > install.packages("nortest", lib=address,
+   rpos="https://cran.r-project.org/package=nortest")
```

Running above two lines there appears a box entitles "Secure CRAN mirrors" that lists all available mirrors for installing the desired package. We suggest to select the nearest country, if you country is not listed there, and then approving to continue the installing process. Depending on the desired package's dependency on other packages, the installation process may take several seconds or minutes. Finally, a blue-colored message appears in console session indicating that the installation process is accomplished successfully. After installing, we must call the desired package through

```
R > library(nortest) # or require(nortest)
```

It is worth to mention that the argument `address` is quite optional and determined by user. The argument `rpos` is obtained from the last line in the package's page at CRAN. The above process must be done for using all external packages. Sometimes, due to some technical problems, user is not able to install the desired package. One way to avoid such a issue is to install the desired package manually. To this end, user just needs to download the package's compressed source code available at **Package source** part. Then, we determine the address of the package source through

```
install.packages(file.choose( ), repos = NULL, type="source")
```

to complete the installation process.

1.6.3 Normality test

Testing the normality of a data set is a vital statistical hypothesis in many parametric and non-parametric statistical methods [Thode, 2002, Stephens, 2017]. Though implementing the goodness-of-fit tests does not need an in-depth R programming skill, but we would prefer to use some external package for this purpose rather to create our user-defined function. After installing package `nortest`, having a look at topics, we can see that function `ad.test` has been developed for accomplishing the Anderson-Darling test of normality. There is more detailed information on this function in the pertinent page. Also, the package `nortest` allows user for running more normality test such as Cramer-von Mises, Lilliefors (or Kolmogorov-Smirnov), Pearson chi-square, and Shapiro-Wilk. In what follows, we give the pertinent commands and outputs when these goodness-of-fit tests are applied to a data set of size $n = 100$ randomly generated from Gaussian distribution with mean 5 and standard deviation 3. Each test consists of the *test statistic* and corresponding *p-value*. Since all p-values are greater than significance level, $\alpha = 0.05$ say, then *based on the sample evidence*, we decide to accept the null hypothesis of H_0 : “data follow Gaussian distribution” against H_1 : “data do not follow Gaussian distribution” at this level.

```
R> set.seed(20241204)          R> ad.test(x)
R> x<-rnorm(100,mean=5,sd=3)    Anderson-Darling normality test
R> #we use rnorm() to generates data:  x
R> #from Gaussian distribution  A = 0.3784, p-value = 0.4007

R> cvm.test(x)                 R> lillie.test(x)
Cramer-von Mises normality test  Lilliefors (Kolmogorov-Smirnov) normality test
data:  x                        data:  x
W = 0.055022, p-value = 0.4364  D = 0.063051, p-value = 0.4239

R> pearson.test(x)             R> sf.test(x)
Pearson chi-square normality test Shapiro-Francia normality test
data:  x                       data:  x
P = 9.72, p-value = 0.4654      W = 0.98913, p-value = 0.5087
```

More diagnostics for assessing the normality are discussed through R package `EnvStats` available at <https://cran.r-project.org/package=EnvStats>. It is worthwhile to note that this package needs an R(>= 3.5.0) and imports packages `MASS`, `ggplot2`, `nortest`. Hence, installing `EnvStats`, we should take care about the imported packages and their dependencies. The package `EnvStats` is enable to draw quantile-quantile plot (q-q plot). Suppose we would like to display the q-q plot of the data already generated from Gaussian distribution. To this end, we run the commands as

```
R> library(EnvStats); set.seed(20241204); x<-rnorm(100,mean=5,sd=3)
R> qqPlot(x, dist = "norm", estimate.params = T, digits = 2,
+ points.col = "blue", add.line = TRUE)
R> qqPlot(x, dist = "norm", param.list=list(mean=5, sd=3),
+ estimate.params = F, digits = 2, points.col = "blue", add.line =
+ TRUE, plot.type="Tukey Mean-Difference Q-Q")
```

to display graphical summaries depicted in Figure 1.7. It is worthwhile to say that for producing the normal q-q plot (left subfigure), we set argument `estimate.params=T` and hence the package automatically sets `param.list=list(mean=mu, sd=sigma)` in which `mu=5.4` and `sigma=2.7` are the average and standard deviation of sample evidence `x`. For producing the

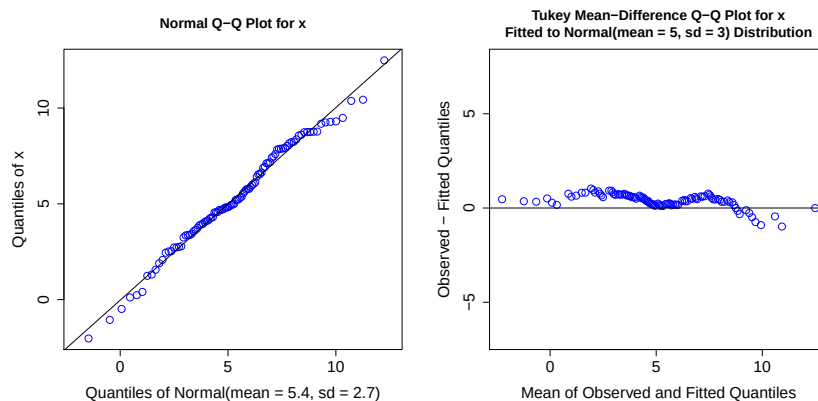


Figure 1.7: (left subfigure): q-q plot and (right subfigure): Tukey mean-difference q-q plot.

Tukey mean-difference q-q plot (right subfigure), we set `estimate.params=F` and hence we need to put the true mean and standard deviation using argument `param.list=list(mean=5, sd=3)`.

1.7 Regression analysis in R

The simple linear, multiple linear, and non-linear regression analysis are among applied topics in statistics with widespread use in other fields of study. Herein, we are willing to discuss how user can implement the regression analysis in R.

1.7.1 Linear regression

Suppose the amount of some variable that is known in the literature as *response* or *dependent* variable depends *linearly* on one (simple case) or more (multiple case) variable(s) namely *independent* variable(s) or *covariate*. The linear regression enables us to estimate the value of dependent variable y when a level of independent x is given. The set of dependent and independent variables should be provided in a data frame structure. For practical purposes, we will focus on *built-in* **women** data that consists of two variables average heights and weights for American women. Hence, for detecting how the dependent (weight) and independent (height) variables are linearly correlated, we proceed as follows.

```
R> data(women)
R> dat <- data.frame(y=women$weight, x=women$height)
R> model <- lm(y~x, data=dat) # lm accounts for linear model
R> out <- summary(model)
```

The summary of regression analysis such as estimated coefficients, estimated residuals, etc., are used for further analysis. Table 1.14 lists some commands to access more detailed summary related to the simple linear regression analysis. Figure 1.7 displays our user-defined function

created for depicting the dependent and independent scatterplot. The pertinent R function called `reg_plot(x,y,...)` is given as follows.

```
1 R> reg_plot <- function(x,y,PI=FALSE, interval=c("c","p"),...)
2 + {# 'c' for confidence and 'p' for prediction interval
3 + model <- lm(y~x)
4 + out <- summary(model)
5 + b0 <- round(out$coefficien[1,1],4)
6 + b1 <- round(out$coefficien[2,1],4)
```

Table 1.14: Details about output of `lm(...)` in R.

Command	output
<code>out\$coefficients[1,1]</code>	estimated intercept.
<code>out\$coefficients[2,1]</code>	estimated slope.
<code>out\$coefficients[1,2]</code>	standard error of estimated intercept.
<code>out\$coefficients[2,2]</code>	standard error of estimated slope.
<code>out\$coefficients[1,4]</code>	p-value corresponds to null hypothesis of intercept is zero.
<code>out\$coefficients[2,4]</code>	p-value corresponds to null hypothesis of slope is zero.
<code>out\$sigma</code>	estimated response variable standard error.
<code>out\$r.squared</code>	coefficient of determination R^2 .
<code>confint(model, level=0.95)</code>	0.95% confidence interval for regression coefficients.
<code>out\$residuals</code>	residuals of fitted model.
<code>anova(model)</code>	analysis of variance table.
<code>predict(object, new, ...)</code>	object is <code>lm(y ~ x)</code> and <code>new</code> is a single or vector of values for which a confidence ("c") or prediction ("p") interval at <code>level=0.95</code> is constructed.

```

7 + note1 <- paste("Y=", b0, "+", b1, "X")
8 + note2 <- paste("R.squared=", round(out$r.squared, 4))
9 + n <- length(x)
10 + new <- data.frame(x=seq(min(x), max(x), length = n))
11 + if(PI == FALSE)
12 + {
13 +   plot(x, y, ...)
14 +   curve(b0 + b1*x, add=T)
15 +   lines(new$x, b0 + b1*new$x)
16 +   text(mean(x), max(y), note1)
17 +   text(mean(x), quantile(y, 0.90), note2)
18 + } else if(PI == TRUE){
19 +   pred <- predict(model, new, interval = interval)
20 +   plot(x, y, ...)
21 +   x0 <- c(new$x[1:n], new$x[n:1])
22 +   y0 <- c(pred[, 2], pred[n:1, 3])
23 +   polygon(x0, y0, col="steelblue", border="red")
24 +   text(mean(x), max(y), note1)
25 +   text(mean(x), quantile(y, 0.90), note2)
26 +   points(x, y, ...)
27 +   curve(b0 + b1*x, add=T)
28 + }
29 + }

```

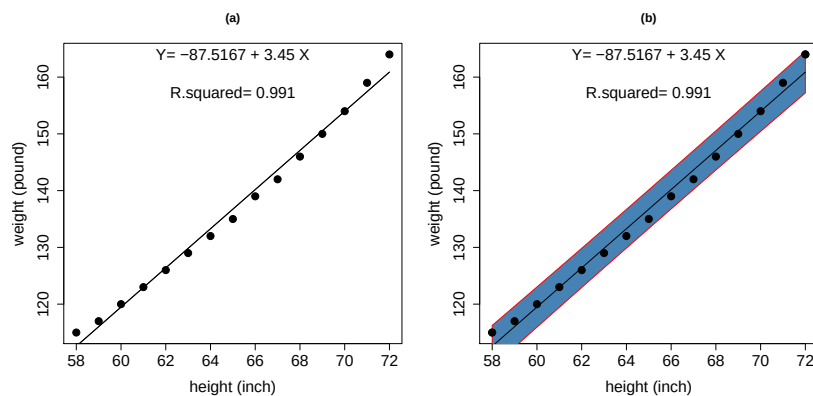


Figure 1.8: (a): Scatterplot of `women` data: (a) fitted regression line and (b): prediction interval. We run the R commands as follows for producing Figure 1.8.

```

R> data(women)
R> x <- women$height; y <- women$weight
R> reg_plot(x,y,PI=F, interval="p",pch=19,cex=1.5,
+ cex.lab=1.5,cex.axis=1.5,main="(a)",
+ xlab="height (inch)",ylab="weight (pound)")# for subfigure (a)
R> reg_plot(x,y,PI=T, interval="p",pch=19,cex=1.5,
+ cex.lab=1.5,cex.axis=1.5,main="(b)",
+ xlab="height (inch)",ylab="weight (pound)")# for subfigure (b)

```

1.7.2 Non-linear regression

In some applications relation between dependent and independent variables is not linear. There are infinite number of non-linear mathematical models for fitting to data. Herein, we would like to deal with the growth model. An application of such models can be found in forestry for describing the relation between tree's height (h in m/foot) and its diameter at breast height (dbh in cm/inch) Teimouri et al. [2020]. Among the most commonly used growth models, we focus on Weibull model whose relation is given by Yang et al. [1978]:

$$h = 1.3 + b1[1 - e^{-b2d^{b3}}], \quad (1.1)$$

where $b1 > 0$, $b2 > 0$ and $b3 > 0$ are the model's parameters³. There are different ways for estimating the model's parameters based on the nonlinear least squares method, among them herein, we suggest the use of function `nls(formula,data,start,...)`. Detail on main arguments of the former function is shown by Table 1.15.

Table 1.15: Details about main arguments of `nls(...)` in R.

Argument	output/task
formula	mathematical expression such as formula (1.1).
data	data frame consisting of dependent and independent data.
start	vector of starting (initial) values of the model's parameters.

The `nls(...)` uses some iterative algorithms to estimate the parameters. But, the performance (being convergent or not) of these algorithms in general is sensitive on the initial values. Not surprisingly, if starting values are close to the global maximum, then algorithm converges soon. Otherwise, the algorithm will not converge and `nls(...)` fails to estimate model's parameters. One way to deal this practical issue is the use of function `tryCatch(...)` that has mainly four parts described as Table 1.16. In what follows, we run function `nls(...)` for

Table 1.16: Details about main arguments of `tryCatch(...)` in R.

Argument	output/task
expr	the task to be accomplished.
error	the task to be done if the given task in part expr if an error is caught.
warning	the task to be done if accomplishing the given task in part expr does not fail, but a warning is caught.
finally	the task to be done regardless of accomplishing the given task in part expr is successful or not successful.

DBH data involved in package `ForestFit` under two scenarios including: i) true parameters are assumed to be known and function `tryCatch(...)` is not employed and ii) true parameters

³The constant 1.3 is model's intercept since the experimenter's height at breast level is on the average 1.3 m.

are assumed to be unknown and function `tryCatch(...)` is employed. We deal with these two scenarios as follows.

- i. **First scenario:** We assume that true parameters are $b_1 = 25.011$, $b_2 = 0.0167$, and $b_3 = 1.1561$. Using this setting, we use the command

```
R> library(ForestFit); data(DBH)
R> d <- DBH[DBH[,1]==55,10]; h <- DBH[DBH[,1]==55,11]
R> start <- c(25.011,0.016, 1.156)
R> relation <- as.formula(h~1.3+b1*(1-exp(-b2*d^b3)))
R> out <- summary( nls( relation, data=data.frame(h=h, d=d),
+ start = list( b1 = start[1], b2 = start[2], b3 = start[3] ) ) )
R> out
```

to see the output as follows.

```
Formula: h ~ 1.3 + b1 * (1 - exp(-b2 * d^b3))
Parameters:
      Estimate Std. Error t value Pr(>|t|)
b1 25.011868   2.400674  10.419 1.26e-14 ***
b2  0.016704   0.004924   3.392  0.00129 **
b3  1.156198   0.124108   9.316 6.63e-13 ***
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Residual standard error: 1.558 on 55 degrees of freedom
Number of iterations to convergence: 4
Achieved convergence tolerance: 8.725e-06
```

- ii. **Second scenario:** We let the model's parameter are not precisely known. Hence, we use the function `tryCatch(...)` to implement the `nls(...)` function. The pertinent R code and output are given by the following.

```
R> library(ForestFit); data(DBH)
R> d <- DBH[DBH[,1]==55,10]
R> h <- DBH[DBH[,1]==55,11]
R> start <- c(1, 1, 1)
R> relation<-as.formula(h~1.3+b1*(1-exp(-b2*d^b3)))
R> f<-function(x, par)
+ {
+   b1<-par[1]; b2<-par[2]; b3<-par[3]; 1.3+b1*(1-exp(-b2*x^b3))
+ }
R> it <- 1; i <- 0
R> while(it <= 1)
+ {
+   r1<-runif(1,max(0,start[1]-10),start[1]+10) # b1 is selected randomly
+   r2<-runif(1,max(0,start[2]-10),start[2]+10) # b2 is selected randomly
+   r3<-runif(1,max(0,start[3]-10),start[3]+10) # b3 is selected randomly
+   out <- tryCatch(summary( nls( relation, data=data.frame(h=h, d=d),
+ start = list( b1 = r1, b2 = r2, b3 = r3 ) ) ),
+ error=function(e)( "fail" ) )
+   if( out[1] == "fail" )
+   {
```

```

+       it <- 1
+     }else{
+       it <- 2
+     }
+ i <- i + 1
+}
R> out

```

As it may be seen, we set the starting value as `start=c(1,1,1)` in second scenario and function `tryCatch(...)` allows to test different starting values during each test within `while(...)` function. As expected, the estimated regression coefficients under both both scenario are the same. We further applied the function `tryCatch(...)` to estimate three parameters of the logistic growth curve. It suffices to consider as follows.

```

R> relation<-as.formula(h~1.3+b1/(1+b2*exp(-b3*d)))
R> f<-function(x, par)
+ {
+   b1<-par[1]; b2<-par[2]; b3<-par[3]; 1.3+b1/(1+b2*exp(-b3*x))
+ }

```

Figure 1.9 displays the scatterplot of height and diameter variables while the estimated Weibull (left subfigure) and logistic (right subfigure) growth curves are added to the plot. It is worth to note that the output corresponds to fitting logistic growth curve is as follows.

```

Formula: h ~ 1.3 + b1/(1 + b2 * exp(-b3 * d))
Parameters:
      Estimate Std. Error t value Pr(>|t|)
b1 21.688329   0.814105  26.641  < 2e-16 ***
b2  6.878022   1.212616   5.672 5.42e-07 ***
b3  0.086529   0.009079   9.530 3.04e-13 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
Residual standard error: 1.802 on 55 degrees of freedom
Number of iterations to convergence: 9
Achieved convergence tolerance: 6.395e-06

```

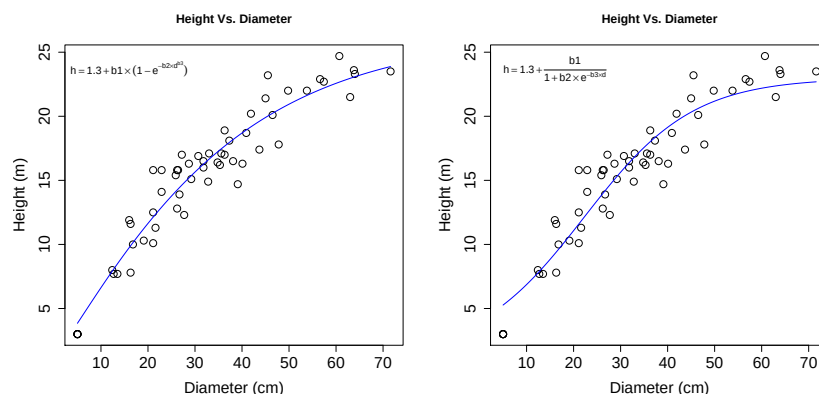


Figure 1.9: Scatterplot of height and diameter data with fitted Weibull (left subfigure) and logistic (right subfigure) growth curves.

2

Common simulation techniques

This section focuses on some important simulation approaches including: *Metropolis-Hastings algorithm*, *Inverse transform method*, *Distributional identity*, *Rejection sampling*, *Adaptive rejection sampling*, and *Ratio of uniforms method*, and *Generalized ratio of uniforms method*. We further deal with the methods for simulating from multivariate Gaussian distributions. Throughout, we assume that generation from uniform distribution on $(0,1)$, that is $\mathcal{U}(0,1)$, is accomplished by the computer machine.

2.1 Metropolis-Hastings algorithm

In this part, we describe the Metropolis-Hastings algorithm. To this end, first, we review some preliminaries.

Definition 2.1.1. A collection of sample evidence (random variables) indexed by time set $t \in \{0, 1, 2, \dots\}$, represented as $\{X_t\}_t$, is called a stochastic or random process.

Definition 2.1.2. A stochastic process $\{X_t\}_{t \geq 0}$ is said to be Markov process if

$$p(X_n = x_n | X_0 = x_0, \dots, X_{n-1} = x_{n-1}) = p(X_n = x_n | X_{n-1} = x_{n-1}), \quad (2.1)$$

that means the position (or state) of a Markov process (chain) at time n just depends on its position at previous state $n - 1$ for $n \geq 1$.

Definition 2.1.3. For a given pair (i, j) , if there is a positive probability that chain will ever visit state j while it starts with state i , then this chain is called irreducible.

Let π_j denote the long-run proportion of times that an irreducible Markov chain visits state j . It can be shown that π_j exists, independent of the chain's initial state i , and is given as the solution of the equations

$$\pi_j = \sum_{i=1}^N \pi_i P_{ij}, \text{ for } j = 1, \dots, N. \quad (2.2)$$

The system of equations in (2.2) can be represented in matrix form as

$$\boldsymbol{\pi}^\top = \begin{bmatrix} \pi_1 \\ \pi_2 \\ \vdots \\ \pi_j \\ \vdots \\ \pi_N \end{bmatrix}^\top = \begin{bmatrix} \pi_1 \\ \pi_2 \\ \vdots \\ \pi_j \\ \vdots \\ \pi_N \end{bmatrix}^\top \times \begin{bmatrix} P_{11} & P_{12} & \cdots & P_{1N} \\ P_{21} & P_{22} & \cdots & P_{2N} \\ \vdots & \vdots & \vdots & \vdots \\ P_{j1} & P_{j2} & \cdots & P_{jN} \\ \vdots & \vdots & \vdots & \vdots \\ P_{N1} & P_{N2} & \cdots & P_{NN} \end{bmatrix}. \quad (2.3)$$

The vector $\boldsymbol{\pi}$ is known as the *limiting distribution* or the *stationary distribution* and π_j is called the *limiting probability* that the Markov chain visits state j .

Definition 2.1.4. An irreducible Markov chain is said to be aperiodic if we have

$$P(X_n = j | X_0 = j) > 0, \text{ for } n \geq 0. \quad (2.4)$$

Definition 2.1.5. A Markov chain with limiting probability π_j and transition probabilities P_{ij} is said to be time-reversible if we have $\pi_i P_{ij} = \pi_j P_{ji}$ for all $i \neq j$.

Suppose we are willing to simulate from random variable for which we have $P(X = j) = \pi_j$ for $j = 1, \dots, m$ (m is large). One way is the use of MH algorithm that constructs a time-reversible Markov chain $\{X_n\}_{n \geq 0}$ whose limiting probability is π_j . To this end, we consider an irreducible Markov chain with integer states $\{1, \dots, m\}$ whose transition probability matrix is $\mathbf{Q}$, see Ross [2022]. Let the (i, j) -element of $\mathbf{Q}$ is denoted as $q(i, j)$. If $X_n = i$, then generation X is drawn from distribution with probability mass function $P(X = j) = q(i, j)$ for $j = 1, \dots, m$. If $X = j$, then X_{n+1} would be i and j , accordingly, with probabilities $1 - \alpha(i, j)$ and $\alpha(i, j)$. The transition probabilities of the constructed Markov chain becomes

$$\begin{aligned} P_{i,j} &= q(i, j)\alpha(i, j), \text{ for } j \neq i, \\ P_{i,i} &= q(i, i) + \sum_{k \neq i} q(i, k)[1 - \alpha(i, k)]. \end{aligned}$$

The time-reversibility of this Markov chain with limiting distribution π_j holds true if

$$\pi_i P_{i,j} = \pi_j P_{j,i} \text{ for } j \neq i,$$

that implies

$$\pi_i q(i, j)\alpha(i, j) = \pi_j q(j, i)\alpha(j, i). \quad (2.5)$$

It is not hard to see that the identity in (2.5) is satisfied if we have

$$\alpha(i, j) = \min \left\{ 1, \frac{\pi_j q(j, i)}{\pi_i q(i, j)} \right\}. \quad (2.6)$$

The limiting probability π_j may be represented as $\pi_j = b_j/B$ in which $B = \sum_{j=1}^m b_j$ for $b_j > 0$ and sufficiently large m . Evidently, factor B plays the role of normalizing constant, and hence the right-hand side (RHS) of (2.6) can be represented as

$$\alpha(i, j) = \min \left\{ 1, \frac{b_j q(j, i)}{b_i q(i, j)} \right\}. \quad (2.7)$$

The steps of the MH algorithm is given by Algorithm 1. In practice, the MH algorithm is use frequently to simulate from continuous distribution with probability density function (PDF) $f(\cdot|\theta)$ that coincides with the limiting distribution π_j , of irreducible Markov chain. But, as it is seen from Algorithm 1, the MH algorithm produces dependent generations sine the next generation is created based on the current one. Hence, all estimators constructed based on the generated samples are biased and inefficient. To avoid this problem, the initial generated samples generated sample are removed from the whole [Robert et al., 2004]. The length L , of the removed initial samples is known as *burn-in* or *warm-up* for which some criteria has been determined [Hobert and Robert, 2004]. It can be shown that after sufficiently large number M of generations, the generated samples are statistically uncorrelated and converge to the chain's limiting distribution that coincides with distribution of PDF $f(\cdot|\theta)$. We follow the steps of Algorithm 2 for implementing the MH algorithm to simulate one generation from PDF $f(\cdot|\theta)$.

Algorithm 1 MH-algorithm for generation one sample from target PDF $\pi_j = b_j/B$

- 1: Construct an irreducible Markov chain with transition probabilities $q(i, j)$ (for $i, j = 1, \dots, m$) and suggest an integer number c such that $1 \leq c \leq m$;
 - 2: Set $n = 0$ and $x_0 = c$;
 - 3: Generate x_{new} from probability mass function $P(X_{new} = j) = q(x_n, j)$;
 - 4: Generate a random variable $u \sim \mathcal{U}(0, 1)$;
 - 5: **if** $u < [b_x q(x_{new}, x_n)] / [b_{x_n} q(x_n, x_{new})]$ **then**
 - 6: $x_{new} \leftarrow x$
 - 7: **else**
 - 8: $x_{new} \leftarrow x_n$;
 - 9: **end**
 - 10: $n \leftarrow n + 1$ and $x_n \leftarrow x_{new}$;
 - 11: go to step 3.
-

It is worthwhile to mention that $q(\cdot|\cdot)$ is known as the *proposal* or *candidate* distribution and $f(\cdot|\theta)$ is also referred to as *target* distribution in the literature. The proposal must be easy to sampling from and moreover we have $\mathcal{S}_q = \mathcal{S}_f$ ¹. If the next state is obtained by adding some random variable to the current state then we have a *random walk* proposal and the algorithm is called sometimes random walk Metropolis algorithm. Furthermore, if the random variable follows a symmetric distribution, then the acceptance rate or transition probability takes a simpler form. In such a case, we have $q(x_{n+1}|x_n) = q(x_n|x_{n+1})$. The original Metropolis [Metropolis et al., 1953] algorithm was constructed based on a symmetric random walk proposal. For example, suppose that chain's current state is X_n and we use a standard Gaussian distribution, as the proposal, for determining the next state X_{n+1} . It follows that

$$X_{n+1} = X_n + Y,$$

where $Y \sim \mathcal{N}(\mu = 0, \sigma = 1)$. It is easy to check that PDF of the next state becomes $\mathcal{N}(x_{n+1}|\mu = x_n, \sigma = 1)$ or equivalently

$$q(x_{n+1}|x_n) = \mathcal{N}(x_{n+1}|\mu = x_n, \sigma = 1).$$

Obviously, since $\mathcal{N}(x_{n+1}|\mu = x_n, \sigma = 1) = \mathcal{N}(x_n|\mu = x_{n+1}, \sigma = 1)$, we have

$$q(x_{n+1}|x_n) = q(x_n|x_{n+1}). \quad (2.8)$$

The above argument holds true for any $\sigma > 0$. Therefore the acceptance rate given in Algorithm has the simpler form

$$A = \min \left\{ 1, \frac{f(x_{n+1}|\theta)}{f(x_n|\theta)} \right\}.$$

Another example for a random walk Metropolis algorithm is obtained by considering the uniform distribution on $(a - 3b, a + 3b)$ as the proposal in which $a \in \mathbb{R}$ and $b > 0$. In this case, we have $q(x_{n+1}|x_n) = q(x_n|x_{n+1}) = 1/(6\sigma)$. The algorithm proposed by Metropolis et al. [1953] was then developed by Hastings [1970] for general case including both symmetric and asymmetric [$q(x_{n+1}|x_n) \neq q(x_n|x_{n+1})$] cases. Today the latter case is known as the Metropolis-Hasting algorithm. A privilege of the MH algorithm is its capability to apply for the cases that the target distribution needs only to be specified up to proportionality constant. The care must taking into account that target PDF must never be zero at the initial state x_0 and the candidate

¹ $\mathcal{S}_f = \{x \in \mathbb{R} | f(x|\theta) \geq 0, \int_{\mathcal{S}_f} f(x|\theta) dx = 1\}$.

$q(\cdot|\cdot)$ must have a broad enough support to visit any state of $\mathcal{S}_f$ with a positive probability while gets as simple as possible form. We give more details on finding a suitable proposal through the following example.

Algorithm 2 MH-algorithm for generation one sample from target PDF $f(\cdot|\theta)$

- 1: Set $n = 0$, read L , and choose the current state (initial generation) x_0 ;
- 2: **while** $n \leq L$ **do**
- 3: Sample x_{new} from the proposal distribution with PDF $q(\cdot|x_n)$;
- 4: Compute

$$A = \min \left\{ 1, \frac{f(x_{new}|\theta)q(x_n|x_{new})}{f(x_n|\theta)q(x_{new}|x_n)} \right\};$$

- 5: Generate $u \sim \mathcal{U}(0, 1)$;
 - 6: **if** $u < A$ **then**
 - 7: Set $n \leftarrow n + 1$ and $x_n \leftarrow x_{new}$;
 - 8: **else**
 - 9: $y \leftarrow x_n$, $n \leftarrow n + 1$, and $x_n \leftarrow y$;
 - 10: **end**
 - 11: **end**
 - 12: Accept x_L as a generation from target distribution f .
-

Example 2-1

Let $f(x|\theta) \propto (x+1)^{-2}x^\theta \exp\{-x^\theta\}$ for $x > 0$ denote the PDF of target distribution in which $\theta > 0$ is the family parameter. Considering the Weibull distribution with shape parameter θ and scale unity as the proposal distribution whose PDF is denoted as $\mathcal{W}(\cdot|\theta, 1)$, we represent the acceptance rate (transition probability) as

$$\begin{aligned} A &= \min \left\{ 1, \frac{(x_{n+1}+1)^{-2}x_{n+1}^\theta \exp\{-x_{n+1}^\theta\} \mathcal{W}(x_n|\theta, 1)}{(x_n+1)^{-2}x_n^\theta \exp\{-x_n^\theta\} \mathcal{W}(x_{n+1}|\theta, 1)} \right\}. \\ &= \min \left\{ 1, \frac{(x_n+1)^2 x_{n+1}}{(x_{n+1}+1)^2 x_n} \right\}. \end{aligned} \quad (2.9)$$

Fortunately, though using an asymmetric proposal, the acceptance rate has a simple form. Setting $\theta = 5$, we follow the steps given by the following for simulating 800 realizations from $f(x|\theta = 5)$. It is worth to highlight that we run Algorithm 2 for $Ls = 1000$ times of which first $Lb = 200$ (burn-in point) generations are removed.

- Set $n = 0$, $Lb = 200$, $Ls = 1000$, and the initial state (generation) $x_0 = 1$;
 1. Generate $x_{new} \sim \mathcal{W}(\theta = 5, 1)$
 2. Compute $A = \min \left\{ 1, \frac{(x_n+1)^2 x_{new}}{(x_{new}+1)^2 x_n} \right\}$;
 3. Generate $u \sim \mathcal{U}(0, 1)$;
 4. If $u < A$, then $n \leftarrow n + 1$ and $x_n \leftarrow x_{new}$;
 5. Repeat algorithm from step 1;
- Stop algorithm if $n = Lb$ and then accept x_{Lb} as a generation from $f(x|\theta = 5)$.

■

The R code, for generating from PDF $f(x|\theta = 5)$ given in Example 2-1 is given as follows.

```

1 R> set.seed(20240412)
2 R> theta <- 5
3 R> Ls <- 1000 # total number of iterations
4 R> Lb <- 200 # burn-in point
5 R> x <- rep(NA, Ls)
6 R> x0 <- 1; x[1]<- x0 # initial value
7 R> for(n in 1:Ls)
8 + {
9 +   x.new <- rweibull(1, shape = theta, scale = 1)
10 +   A <- 2*log(x[n] + 1) + log(x.new) -
11 +     2*log(x.new + 1) - log(x[n])
12 +   if( runif(1) < exp( A ) )
13 +   {
14 +     x[n+1] <- x.new # x.new is accepted
15 +   }else{
16 +     x[n+1] <- x[n] # x.new is rejected
17 +   }
18 + }
19 R> hist(x[(Lb+1):Ls], prob=T, br=20)

```

As it is seen in example above, the Weibull choice of candidate yields a simple form of transition probability. Figure 2.1(b) shows the chain's motion across $Ls = 1000$ iterations while the initial state is $x_0 = 4$. As it is seen, the convergence occurs very soon. There have been introduced some extensions of the MH algorithm in the literature. We refer reader to for more details about extensions such as [Robert et al., 2004, Roberts and Rosenthal, 2009, Robert et al., 2010]. There have been introduced some extensions of the MH algorithm in the literature.

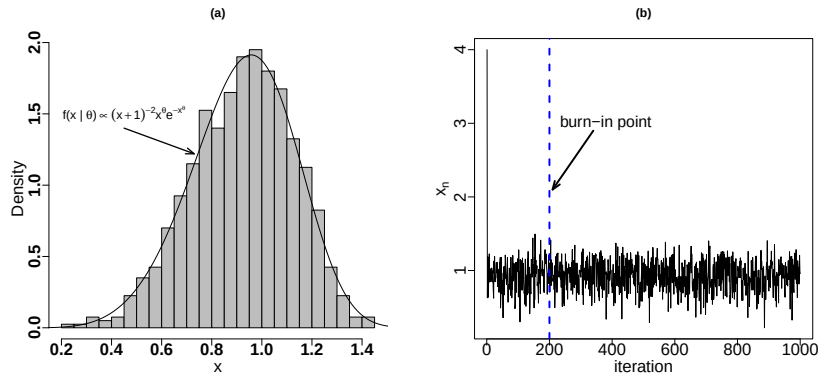


Figure 2.1: (a): Histogram (constructed based on chain's output between 201th up to 1000th iterations); superimposed is $f(x|\theta = 5)$ and (b): Chain's motion across iterations for sampling from $f(x|\theta = 5)$ with starting value $x_0 = 4$.

We refer reader to for more details about extensions such as [Robert et al., 2004, Roberts and Rosenthal, 2009, Robert et al., 2010].

2.2 Inverse transform method

Throughout this chapter let random variable X from which drawing sample is of interest has closed form cumulative distribution function (CDF), denoted herein by $F(\cdot)$, or is computed using accurate computer algorithms such as Gaussian distribution. In what follows $X \sim F(\cdot)$ or $X \sim f(\cdot)$ will denote the random variable X follows CDF $F(\cdot)$ or PDF $f(\cdot)$. The following proposition plays main role for simulating from X through the Inverse transform method.

Proposition 2.2.1. Let $X \sim F(\cdot)$ and $U \sim \mathcal{U}(0, 1)$. Then,

$$X \stackrel{d}{=} F^{-1}(U),$$

where “ $\stackrel{d}{=}$ ” denotes the identity in distribution and $F^{-1}(\cdot)$ is inverse of CDF or the quantile function.

Based on Proposition (2.2.1) in above, for simulating from random variable with CDF $F(\cdot)$ (or PDF $f(\cdot)$), first, one can simulate a generation such as u from $\mathcal{U}(0, 1)$ and then a sample such as x is produced by computing $F^{-1}(u)$. In what follows, we give an example that accomplishes this method simply for drawing sample from exponential distribution.

Example 2-2

Let $X \sim \mathcal{EXP}(\lambda)$ with CDF given by

$$F(x) = 1 - \exp\{-\lambda x\}, \quad (2.10)$$

where $\lambda > 0$ is the family rate parameter. It easy to see that

$$F^{-1}(u) = -\frac{1}{\lambda} \log(1 - u), 0 < u < 1. \quad (2.11)$$

Hence, a generation from exponential family with CDF in (2.10) becomes $x = -\log(1 - u)/\lambda$ in which u is drawn from $\mathcal{U}(0, 1)$. The R function `rexp0` given by

```
1 R> rexp0 <- function(n, lambda = 1)
2 +{
3 +x <- -1/lambda * log( 1- runif(n) )
4 +return(x)
5 +}
```

can be used for generating n independent realizations form $\mathcal{EXP}(\lambda)$. But, it should be noted that R has its own function `rexp(n, rate=lambda)` for generating from this family. So, we would prefer to use the latter in simulation setting rather than the former R code. ■

In order to simulate from some well-known statistical families of distributions in R, we may run the commands given in Table 2.1. When the CDF is a piecewise function, then the quantile function can be exploited by inverting the CDF on each piece for which $0 < F(\cdot) < 1$. Assuming that random variable X has a m -piece CDF represented by

$$F(x) = \begin{cases} 0, & \text{if } x < a_0, \\ F_1(x), & \text{if } a_0 \leq x < a_1, \\ F_2(x), & \text{if } a_1 \leq x < a_2, \\ \vdots & \vdots \\ F_m(x), & \text{if } a_{m-1} \leq x < a_m, \\ 1, & \text{if } a_m \leq x. \end{cases} \quad (2.12)$$

For example, schematic of a three-piece CDF

$$F(x) = \begin{cases} 0, & \text{if } x < 0, \\ \frac{1}{6} \times \frac{1 - \exp\{-2x\}}{1 - \exp\{-2\}}, & \text{if } 0 \leq x < 1, \\ \frac{1}{6} + \frac{x-1}{6}, & \text{if } 1 \leq x < 2, \\ \frac{1}{3} + \frac{2}{3} \times \frac{\exp\{-x\} - \exp\{-2\}}{\exp\{-8\} - \exp\{-2\}}, & \text{if } 2 \leq x < 8, \\ 1, & \text{if } 8 \leq x, \end{cases} \quad (2.13)$$

is displayed in Figure 2.2, We emphasize that $F_i(x)$ (for $i = 1, \dots, m$) are not CDF. Algorithm 3 describes how to simulate from a random variable that possesses a piecewise CDF.

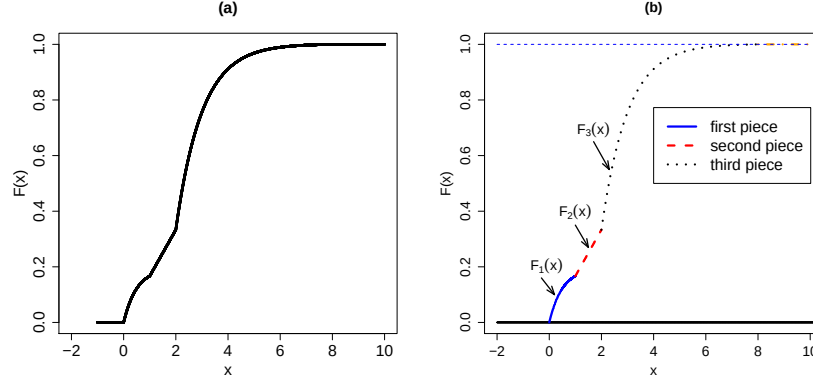


Figure 2.2: (a): a schematic of three-piece CDF and (b): the corresponding pieces.

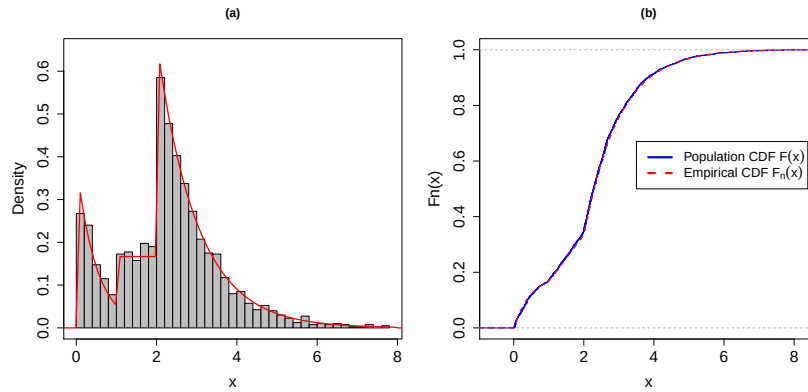


Figure 2.3: (a): Histogram of $n = 2000$ simulated data from family with CDF (2.13). The red-colored superimposed curve is the corresponding PDF. (b): The population CDF and empirical CDF.

2.3 Distributional identity

Herein, we present some examples in which generating from the distribution of interest is accomplished through its relation with other distribution(s) from which sampling is an easy or easier task. One of the most widely used distribution, that is the Gaussian, can be simulated through this approach. Although the quantile function of the Gaussian distribution can be computed numerically in **R** and then the inverse transform method can be applied to sample from, alternatively an efficient method known as the Box-Muller transform proposed by Box and Muller [1958] is suggested for this purpose as follows. It needs to generate independent random variables $U_1, U_2 \sim \mathcal{U}(0, 1)$ and then it can be proved that

$$\begin{aligned} Z_1 &= \sqrt{-2 \log U_1} \cos(2\pi U_2), \\ Z_2 &= \sqrt{-2 \log U_1} \sin(2\pi U_2), \end{aligned}$$

independently follow a standard Gaussian distribution. Consequently, either $\mu + \sigma Z_1$ or $\mu + \sigma Z_2$ follows $\mathcal{N}(\mu, \sigma^2)$. If evaluating the sine or cosine functions wastes the computational time, one can use the method proposed by Marsaglia and Bray [1964] that is described below by Algorithm 4. The pertinent **R** function **rnorm0** given below can be used for generating Gaussian realizations based on Marsaglia-Bray method.

```
1 R> rnorm0 <- function(n, mu, sigma)
2   +{ # n is supposed to be an even number
```

Algorithm 3 Simulation from random variable with piecewise CDF

```
1: Read  $n$  and sequence  $\{F(a_0), F(a_1), \dots, F(a_m)\}$ ;  
2: Generate  $u$  from  $\mathcal{U}(0, 1)$ ;  
3: if  $F(a_0) < u < F(a_1)$  then  
4:    $x = F_1^{-1}(u)$ ;  
5: if  $F(a_1) < u < F(a_2)$  then  
6:    $x = F_2^{-1}(u)$ ;  
7:    $\vdots$   
8: if  $F(a_{m-2}) < u < F(a_{m-1})$  then  
9:    $x = F_{m-1}^{-1}(u)$ ;  
10: if  $F(a_{m-1}) < u < F(a_m)$  then  
11:    $x = F_m^{-1}(u)$ ;  
12: return  $x$  as a generation from CDF  $F(\cdot)$ .
```

Algorithm 4 Marsaglia-Bray method for simulating Gaussian random variable

```
1: Generate  $U_1$  and  $U_2$  independently from  $\mathcal{U}(-1, 1)$ ;  
2: Compute  $W = U_1^2 + U_2^2$ ;  
3: if  $W < 1$  then  
4:    $Y = W^{-1/2} \times \sqrt{-2 \log W}$ ;  
5:    $Z_1 = YU_1$  and  $Z_2 = YU_2$ ;  
6: else  
7:   Come back to step 1 and repeat algorithm;  
8: end  
9: Return  $\mu + \sigma Z_1$  and  $\mu + \sigma Z_2$  as two independent generations from  $\mathcal{N}(\mu, \sigma^2)$ .
```

```

3 +i <- 1
4 +Z <- rep(NA, n)
5 + while (i <= n/2)
6 + {
7 + U1 <- 2*(runif(1) - 0.5); U2 <- 2*(runif(1) - 0.5)
8 + W <- U1^2 + U2^2
9 + if(W < 1)
10 + {
11 + Y <- sqrt(-2*log(W)/W)
12 + Z1 <- Y*U1; Z2 <- Y*U2;
13 + Z[ (2*i - 1):(2*i) ] <- c(Z1, Z2)
14 + i <- i + 1
15 + }
16 + }
17 + return( mu + sigma*Z )
18 +}

```

2.3.1 Multinomial distribution

If random vector $\mathbf{X} = (X_1, \dots, X_K)^\top$ follows a multinomial distribution, then its PDF is given by

$$\mathcal{MUL}(\mathbf{x}|N, \boldsymbol{\omega}) = \frac{\Gamma(N+1)}{\prod_{k=1}^K \Gamma(N_k+1)} \omega_1^{x_1} \omega_2^{x_2} \cdots \omega_K^{x_K}, \quad (2.14)$$

where $N = \sum_{k=1}^K x_k$ in which $x_k \geq 0$. The elements of family parameter $\boldsymbol{\omega} = (\omega_1, \dots, \omega_K)^\top$ are constrained to be nonnegative and sum up to one, that is $\sum_{k=1}^K \omega_k = 1$. We write $\mathbf{X} \sim \mathcal{MUL}(n, \boldsymbol{\omega})$ to indicate that random vector $\mathbf{X}$ follows a multinomial distribution with PDF given by (2.14). In order to simulate from $\mathcal{MUL}(\mathbf{x}|n, \boldsymbol{\omega})$ in R, we can use command `rmultinom(n, size, prob)` in which `n` is the sample size, `size` = N , and `prob` = $\boldsymbol{\omega}$. If `size` = $N = 1$, and $K = 2$ (or `prob` = $(\omega_1, \omega_2)^\top$), then multinomial distribution turns into a Bernoulli distribution with success probability ω_1 .

2.3.2 Finite mixture model

The PDF of a K-component finite mixture model is given by

$$g(y|\boldsymbol{\Psi}) = \sum_{k=1}^K \omega_k f(y|\boldsymbol{\theta}_k), \quad (2.15)$$

where $\boldsymbol{\Psi} = (\boldsymbol{\omega}^\top, \boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_K)$ is the parameter space in which $\boldsymbol{\omega} = (\omega_1, \dots, \omega_K)^\top$ is vector of mixing parameters such that $\sum_{k=1}^K \omega_k = 1$. In (6.55), $f(y|\boldsymbol{\theta}_k)$ is the PDF of k -th component and hence the PDF $g(y|\boldsymbol{\Psi})$ is proper since

$$\int_{\mathbb{R}^p} g(\mathbf{y}|\boldsymbol{\Psi}) d\mathbf{y} = \sum_{k=1}^K \omega_k \int_{\mathbb{R}^p} f(\mathbf{y}|\boldsymbol{\theta}_k) d\mathbf{y} = \sum_{k=1}^K \omega_k = 1.$$

The weight parameter $\boldsymbol{\omega}$ provides the chance of observing sample from components. Sometimes, representation (6.55) is called the *finite mixture model* since the quantity K or number of clusters is assumed to be finite. Herein, we suggest two approaches for generating from finite mixture model. Under the first method, for generating n realizations from a K-component mixture model, one can use the multinomial distribution. In what follows, Algorithm 5 gives a pseudo code describing how to draw a sample of size n from a K-component Gaussian mixture model under the first method. Under the second method, we assume that the number of simulated

Algorithm 5 Simulating from K-component Gaussian mixture model: first method

```
1: Read  $n$ ,  $K$ , and  $\Psi = (\omega, \theta_1^\top, \dots, \theta_K^\top)^\top$ ;
2: Set  $Y$  a vector of length  $n$ ;
3: Set  $i = 1$ ;
4: while  $i \leq n$  do
5:   Sample  $\mathbf{u} \sim \text{MUL}(1, \omega)$ ;
6:   Set  $k_{\max} = \{k | \mathbf{u}[k] = 1\}$  where  $\mathbf{u}[k]$  is the  $k$ th element of vector  $\mathbf{u}$  for
7:    $k = 1, \dots, K$ ;
8:   Sample  $y$  from PDF  $f(\cdot | \theta_{k_{\max}})$ 
9:   Set  $Y[i] \leftarrow y$ ;
10:  Set  $i \leftarrow i + 1$ ;
11: end
12: Accept  $Y$  as sample of size  $n$  from K-component finite mixture model.
```

realization under each component, that is n_k , is known or is determined as $n_k = \lceil n \times \omega_k \rceil$ (for $k = 1, \dots, K - 1$) provided that $n_K = n - \sum_{k=1}^{K-1} n_k$. Herein, the generic symbol $\lfloor n \times \omega_k \rfloor$ denotes the greatest integer value equal or less than $n \times \omega_k$. A sample of size n from K-component mixture model is obtained by merging all K simulated vectors each of size n_K from PDF $f(y|\theta_k)$. Algorithm 6 gives a pseudo code describing how to accomplish this task.

Algorithm 6 Simulating from K-component Gaussian mixture model: second method

```
1: Read  $K$ ,  $n_1, \dots, n_K$ , and  $\Psi = (\omega, \theta_1^\top, \dots, \theta_K^\top)^\top$ ;
2: Set  $Y$  as a vector of length  $n$ ;
3: Set  $k = 1$ ;
4: while  $k \leq K$  do
5:   Draw sample  $\mathbf{y}_1$  of size  $n_k$  from PDF  $f(\cdot | \theta_k)$ ;
6:   Set  $k \leftarrow k + 1$ ;
7: end
8: Accept vector  $(\mathbf{y}_1, \dots, \mathbf{y}_K)^\top$  as sample of size  $n$  from K-component finite mixture model.
```

Example 2-3

Suppose we are interested in generating a sample of $n = 300$ observations from two-component Gaussian mixture model with PDF given by

$$g(y|\Psi) = \omega_1 \times \mathcal{N}(y|\mu_1, \sigma_1^2) + \omega_2 \times \mathcal{N}(y|\mu_2, \sigma_2^2) \quad (2.16)$$

where $\Psi = (\omega_1, \omega_2, \theta_1^\top, \theta_2^\top)^\top$ is whole parameter vector whose elements are

$$\begin{aligned} \omega_1 &= \frac{6}{10}, \theta_1 = (\mu_1, \sigma_1)^\top = (-3, \sqrt{2})^\top, \\ \omega_2 &= \frac{4}{10}, \theta_2 = (\mu_2, \sigma_2)^\top = (0, 1)^\top. \end{aligned} \quad (2.17)$$

The R functions `rmixnorm1` and `rmixnorm2` have been developed based on Algorithm 5 and Algorithm 6, respectively, to generate from a two-component Gaussian mixture model (2.16) with whole parameter given in (2.17). Replacing one of the commands given in Table 2.1 with `rnorm(n=1, mu=mu[[k]], sigma=sigma[[k]])` in line 17 of function `rmixnorm1` to simulate from other finite mixture models.

For example, the command `rgamma(n=1, shape=shape[[k]], rate=rate[[k]])` should be replaced with `rnorm(n=1, mu=mu[[k]], sigma=sigma[[k]])` for simulating from a two-component gamma mixture model with PDF given by

$$g(y|\Psi) = \frac{6}{10} \times \mathcal{G}(y|a_1, b_1) + \frac{4}{10} \times \mathcal{G}(y|a_2, b_2), \quad (2.18)$$

if demanded. The corresponding whole parameter vector becomes

$$\begin{aligned} \omega_1 &= \frac{6}{10}, \theta_1 = (a_1, b_1)^\top = (6, 0.6)^\top, \\ \omega_2 &= \frac{4}{10}, \theta_2 = (a_2, b_2)^\top = (2, 1)^\top. \end{aligned} \quad (2.19)$$

Figure 2.4 displays the histogram of $n = 5000$ simulated data from two-component Gaussian (2.16) and gamma (2.18) mixture models. For producing Figure 2.4(a), we used function `rmixnorm1`. ■

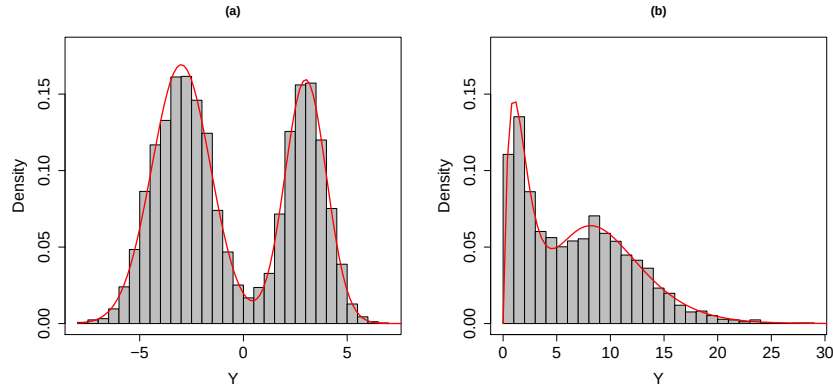


Figure 2.4: (a): histogram of $n = 5000$ simulated data from: (a) two-component Gaussian mixture model (2.16) and (b): two-component gamma mixture model (2.18). The red-colored superimposed curve is the corresponding PDFs given in (2.16) and (2.19), respectively.

```

1 R> set.seed(20240713)
2 R> n <- 5000; K <- 2; omega1 <- 0.6; omega2 <- 0.4; mu1 <- -3; mu2 <- 0; sigma1 <- 1; sigma2
  <- 2
3 R> omega <- c(omega1, omega2); mu <- c(mu1, mu2); sigma <- c(sigma1, sigma2)
4 R> rmixnorm1 <- function(n, omega, mu, sigma)
5 +{
6 + Y <- label <- rep(0, n)
7 + r_MUL <- rmultinom(n = n, size = 1, omega)
8 + label <- apply(r_MUL, 2, which.max)
9 + for(i in 1:n)
10 + {
11 + k <- label[i]
12 + Y[i] <- rnorm(n = 1, mean = mu[k], sd = sigma[k])
13 + }
14 + ls <- list( "sample" = Y, "label" = label )
15 + return(ls)
16 +}
17 R> Y <- rmixnorm1(n, mu, sigma, omega)
18 R> plot( Y$sample, col = Y$label )

1 R> set.seed(20240713)
2 R> n <- 5000; K <- 2; omega1 <- 0.6; omega2 <- 0.4; mu1 <- -3; mu2 <- 3; sigma1 <- sqrt(2);
  sigma2 <- 1
3 R> omega <- c(omega1, omega2); mu <- c(mu1, mu2); sigma <- c(sigma1, sigma2)
4 R> rmixnorm2 <- function(n, omega, mu, sigma)

```

```

5 + {
6 + Y <- rep(0, n)
7 + n_k <- rep(0, K)
8 + n_k <- floor(n*omega)
9 + cn_k <- cumsum(n_k)
10 + n_k[K] <- n - sum(n_k[1:(K-1)])
11 + Y[ 1: cn_k[1] ] <- rnorm(n = n_k[1], mean = mu[1], sd = sigma[1])
12 + for(k in 2:K) Y[ ( (k - 1)*cn_k[k-1] + 1): cn_k[k] ] <-
13 +   rnorm(n = n_k[k], mean = mu[k], sd = sigma[k])
14 + ls <- list( "sample" = Y, "label" = rep(1:K, n_k) )
15 + return(ls)
16 + }

```

2.3.3 Dirichlet distribution

If random vector $\mathbf{X} = (X_1, \dots, X_K)^\top$ follows a Dirichlet distribution with parameter vector $\boldsymbol{\rho}$, say $\mathbf{X} \sim \text{DIR}(\boldsymbol{\rho})$, then the PDF of $\mathbf{X}$ is

$$\text{DIR}(\mathbf{x}|\boldsymbol{\rho}) = \frac{\Gamma(\rho_1 + \dots + \rho_K)}{\prod_{k=1}^K \Gamma(\rho_k)} x_1^{\rho_1-1} \times x_2^{\rho_2-1} \times \dots \times x_K^{\rho_K-1}, \quad (2.20)$$

where $0 < x_k < 1$ and $\rho_k > 0$ (for $k = 1, \dots, K$), provided that $\sum_{k=1}^K x_k = 1$. If $K = 2$, then Dirichlet distribution turns into Beta distribution with shape parameters $\rho_1 > 0$ and $\rho_2 > 0$, denoted by $\text{BET}(\rho_1, \rho_2)$, whose PDF is

$$\text{BET}(x|\rho_1, \rho_2) = \frac{\Gamma(\rho_1 + \rho_2)}{\Gamma(\rho_1)\Gamma(\rho_2)} x^{\rho_1-1} (1-x)^{\rho_2-1}, \quad (2.21)$$

where $0 < x < 1$. Furthermore, it is worth to note that the i th marginal of random vector $\text{DIR}(\mathbf{x}|\boldsymbol{\rho})$ follows $\text{BET}(\rho_i, \sum_{k=1}^K \rho_k - \rho_i)$.

For generating form $\text{BET}(\rho_1, \rho_2)$ in R, we can use command `rbeta(n, shape1, shape2, ncp = 0)` in which `n` is sample size, `shape1` = ρ_1 , and `shape2` = ρ_2 . Let $X_k \sim \mathcal{G}(\rho_k, \beta)$ (for $k = 1, \dots, K$), then random vector $(X_1/Y, \dots, X_K/Y)^\top$ follows a Dirichlet distribution with parameter vector $\boldsymbol{\rho} = (\rho_1, \dots, \rho_K)^\top$ in which $Y = \sum_{k=1}^K X_k$.

```

1 R> rdirichlet <- function(n, omega)
2 +{
3 + K <- length(omega); beta <- 1
4 + X <- matrix(0, nrow = n, ncol = K)
5 + for(k in 1:K) X[, k] <- rgamma(n, shape = omega[k], rate = beta)
6 +return( X/rowSums(X) )
7 +}

```

2.3.4 Chi-squared distribution

The random variable X is said to have chi-squared distribution with ν degrees of freedom if its PDF is given by

$$f(x|\nu) = \frac{2^{-\frac{\nu}{2}}}{\Gamma(\nu/2)} x^{\frac{\nu}{2}-1} \exp\left\{-\frac{x}{2}\right\}, \quad (2.22)$$

where $\nu \in \mathbb{N}$ is the family parameter. The chi-squared distribution is a special case of gamma distribution. This means that if $X \sim \mathcal{G}(k/2, 1/2)$, then $X \sim \chi(k)$. For Simulating random variable X following a chi-squared distribution in R, we may use command `rchisq(n, df, ncp=0)` in which `n` is the sample size, `df` is degrees of freedom (here is parameter ν), and, by default, the non-centrality parameter `ncp` is set to be zero. In fact, X can be represented as the sum

of ν squared standard Gaussian random variables as $X = \sum_{i=1}^{\nu} Z_i^2$. Hence, as an elementary way, one can simulate a chi-squared random variable X through adding up ν number(s) independent standard Gaussian random variables. Of course, herein, we suggest to use either `rgamma(n, shape = nu/2, rate = 1/2)` or `rchisq(n, df = nu, ncp = 0)`. When the non-centrality parameter is nonzero, then the PDF of a chi-squared distribution can be written as

$$f(x|\nu, \lambda) = \frac{1}{2} \left(\frac{x}{2}\right)^{\frac{\nu}{4} - \frac{1}{2}} \exp\left\{-\frac{x + \lambda}{2}\right\} I_{\nu/2-1}(\sqrt{\lambda x}), \quad (2.23)$$

where $I_u(\cdot)$ is the modified Bessel function of the first kind that is given by

$$I_i(y) = \left(\frac{y}{2}\right)^i \sum_{j=0}^{\infty} \left(\frac{y^2}{4}\right)^j \frac{1}{\Gamma(j+1)\Gamma(i+j+1)}. \quad (2.24)$$

Alternatively, the PDF of each chi-squared distribution can be represented as

$$f(x|\nu, \lambda) = \sum_{i=0}^{\infty} \frac{\exp\{-\lambda/2\}}{\Gamma(i+1)} \left(\frac{\lambda}{2}\right)^i f(x|\nu + 2i). \quad (2.25)$$

As it is seen from (2.25), when $\lambda \neq 0$, then PDF of a chi-squared can be expressed in terms a mixture model whose weights are equal to a Poisson mass function. Therefore, for simulating from a chi-squared distribution with nonzero non-centrality parameter, we adopt the following R code.

```
1 R> rchisq0 <- function(n, nu = 1, lambda = 0)
2 + {
3 + X <- rep(NA, n)
4 + P <- rpois(n, lambda = lambda/2)
5 + for(i in 1:n) X[i] <- rchisq(1, df = nu + 2*P[i])
6 + return(X)
7 + }
```

2.3.5 Inverse Gaussian distribution

For simulating from Inverse Gaussian distribution, denoted as $\mathcal{IG}(\lambda, \mu)$, we follow the method proposed by Michael et al. [1976]. The positive random variable X is said to follow $\mathcal{IG}(\lambda, \mu)$ if its PDF is given by

$$f(x|\boldsymbol{\theta}) = \sqrt{\frac{\lambda}{2\pi x^3}} \exp\left\{-\frac{\lambda(x - \mu)^2}{2\mu^2 x}\right\}, \quad (2.26)$$

where $\boldsymbol{\theta} = (\lambda > 0, \mu > 0)^\top$. Comparing by the RHS of (2.51) and (2.26), it is evident that generalized Inverse Gaussian distribution specializes to Inverse Gaussian distribution when $a = -1/2$ and $b = 0$. In words, $\mathcal{IG}(\lambda, \mu) = \mathcal{GIG}(a = -1/2, b = \lambda/\mu^2, c = \lambda)$. For simulating from $\mathcal{IG}(\lambda, \mu)$, first we note that Shuster [1968]:

$$Y = g(X) = \frac{\lambda(X - \mu)^2}{\mu^2 X} \sim \chi_{(1)}, \quad (2.27)$$

where $\chi_{(n)}$ accounts for a chi-square distribution with n degrees of freedom. So, we may think that generating from X is accomplished straightforwardly by drawing a sample from $\chi_{(1)}$ and then X is recovered by a simple inverse transformation. But, care must be taken into the fact that transformation in (2.28) may have two roots and the problem is here to choose among these two distinct roots x_1 and x_2 . Overall, based on method of Michael et al. [1976], for a given

y in (2.28) there are k distinct roots $\{x_1, \dots, x_k\}$ (here, we have $k = 2$). The problem turns into determining the multinomial probabilities associated with each of x_i s (for $i = 1, \dots, k$). Here, we investigate the case in which X is a continuous random variable. Let $f(\cdot)$ and $F(\cdot)$, respectively, denote the PDF and CDF of random variable X that we are interested in sampling from and $p_i(y)$ is the conditional (or multinomial) probability with which the i th root must be chosen. It is shown that Michael et al. [1976]:

$$p_i(y) = \left[1 + \sum_{j=1, j \neq i}^k \left| \frac{g'(x_i)}{g'(x_j)} \right| \times \frac{f(x_j)}{f(x_i)} \right]^{-1}, \quad (2.28)$$

where $g'(x)$ denotes the first derivative of transformation $g(x)$ with respect to x . Computing $p_i(y)$ in (2.28) for $i = 1, \dots, k$, the i th root is chosen as a generation from X with probability $p_i(y)$. In what follows, we describe this method when $X \sim \mathcal{IG}(\lambda, \mu)$. Evidently, $g(x)$ given by (2.28) has two roots, namely x_1 and x_2 . Then more algebra shows $f(x_2)/f(x_1) = (x_1/\mu)^3$ and $g'(x_i)/g'(x_j) = -(\mu/x_1)^2$. Hence, for a given $y \sim \chi_{(1)}$, the roots x_1 and x_2 are

$$x_1 = \mu + \frac{\mu^2 y}{2\lambda} - \frac{\mu}{2\lambda} \sqrt{4\mu\lambda y + \mu^2 y^2},$$

and $x_2 = \mu^2/x_1$ with probabilities

$$p_1(y) = \left[1 + \left| \frac{g'(x_1)}{g'(x_2)} \right| \times \frac{f(x_2)}{f(x_1)} \right]^{-1} = \frac{\mu}{\mu + x_1},$$

and $p_2(y) = 1 - p_1(y) = x_1/(\mu + x_1)$, respectively. We adopt the Algorithm 7 for simulating from $X \sim \mathcal{IG}(\lambda, \mu)$ as follows. The corresponding R function `rig(n, lambda, mu)`, based on

Algorithm 7 Simulating from Inverse Gaussian distribution

- 1: Read parameters λ and μ ;
 - 2: Generate u from $\mathcal{U}(0, 1)$;
 - 3: Generate y from $\chi_{(1)}$;
 - 4: Set $x_1 = \mu + \frac{\mu^2 y}{2\lambda} - \frac{\mu}{2\lambda} \sqrt{4\mu\lambda y + \mu^2 y^2}$;
 - 5: **if** $u < \mu/(\mu + x_1)$ **then**
 - 6: $x = x_1$;
 - 7: **else**
 - 8: $x = \mu^2/x_1$;
 - 9: **end**
 - 10: Accept x as a generation form $\mathcal{IG}(\lambda, \mu)$.
-

Algorithm 7, for generating n realizations from $\mathcal{IG}(\lambda, \mu)$ is given by the following.

```

1  R> rig <- function(n, lambda, mu)
2  + {
3  +   y <- rchisq(n, df = 1)
4  +   u <- runif(n)
5  +   x_1 <- mu + mu^2*y/(2*lambda) - mu/(2*lambda)*sqrt(4*mu*lambda*y + mu^2*y^2)
6  +   x_2 <- mu^2/x_1
7  +   x <- ifelse( u < mu/(mu + x_1), x_1, x_2 )
8  +   return(x)
9  + }
```

2.3.6 α -Stable distribution

Herein, we briefly describe the family of α -stable distributions. This family of distributions was introduced by Paul Pierre Lévy, a French mathematician, in his work on sums of independent identically distributed random variables around 1924. Almost all of members of this

family either have not closed form for PDF or are only expressed in terms of the special functions [Zolotarev, 1986, Samorodnitsky and Taqqu, 1994]. So, this family of distributions are represented in terms of their characteristic function. The characteristic function of α -stable distribution has different parameterizations (forms). The most commonly used parameterizations for is given as follows [Nolan, 1998].

$$E(\exp\{jtX\}) = \begin{cases} \exp\left\{-|\sigma t|^\alpha \left[1 - j\beta \operatorname{sign}(t) \tan\left(\frac{\pi\alpha}{2}\right)\right] + jt\mu\right\}, & \text{if } \alpha \neq 1, \\ \exp\left\{-|\sigma t| \left[1 + j\beta \operatorname{sign}(t) \frac{2}{\pi} \log|t|\right] + jt\mu\right\}, & \text{if } \alpha = 1, \end{cases} \quad (2.29)$$

where $j^2 = -1$ and $\operatorname{sign}(u)$ is the sign function that takes values -1 and +1 for $u < 0$ and $u \geq 0$, respectively. Other forms for characteristic function of α -stable distributions are A, M, B, C, and E [Zolotarev, 1986]. The characteristic function given in (2.29) is known in the literature as S_1 parameterization that is a slightly different version of A parameterization. We write $X \sim S(\alpha, \beta, \sigma, \mu)$ to denote that random variable X follows an α -stable distribution with parameters α , β , σ , and μ in S_1 parameterization. Here, parameter $\alpha \in (0, 2]$ is the index of stability, which determines the tail thickness of the density function. When $\alpha = 2$, the tail thickness is at its thinnest while smaller values of α would yield thicker tails for the distribution. Similarly, the parameter $\beta \in [-1, 1]$ is the degree of skewness (-1 for totally skewed to the left, zero for symmetrical, and +1 for totally skewed to the right), $\sigma \in \mathbb{R}^+$, is the dispersion (scale) around the location parameter μ and $\mu \in \mathbb{R}$, is the horizontal shift of the density function. It is important to note that the parameters σ and μ , in general, are not synonymous with standard deviation and location parameter, as they are in the special case of Gaussian distribution. The only members of this class with closed form PDF are Lévy ($\alpha = 0.5$ and $\beta = 1$), Cauchy ($\alpha = 1$ and $\beta = 0$) and Gaussian ($\alpha = 2$). If $\beta = 0$, and $\mu = 0$, then the class of zero-location symmetric α -stable (S α S) distribution can be identified with the following characteristic function

$$\phi(t) = \exp\{-\sigma^\alpha |t|^\alpha\}. \quad (2.30)$$

The known members of S α S distribution with closed form density function are Cauchy ($\alpha = 1$) and Gaussian ($\alpha = 2$). Here, it is noteworthy to mention that the k th moment of each α -stable distribution is finite if $k < \alpha$. This means that variance of non-Gaussian α -stable distribution is not finite and its first moment is finite if $\alpha > 1$. For $\alpha > 1$ then μ becomes the expected value of distribution. For $\alpha < 1$, the support of class $S(\alpha, 1, \sigma, 0)$ only covers the positive semi-axis. For this reason, if $\alpha < 1$, $\beta = 1$, and $\mu = 0$, then the α -stable distribution family is also called positive α -stable (P α S) distribution.

An important member of P α S family is $S(\alpha/2, 1, [\cos(\pi\alpha/4)]^{2/\alpha}, 0)$ whose CDF is given by [Chernin and Ibragimov, 1959]:

$$F_P(p|\alpha) = \frac{1}{\pi} \int_0^\pi \exp\left\{-p^{-\frac{\alpha}{2-\alpha}} A(u)\right\} du, \quad (2.31)$$

where, for $0 < \alpha < 2$,

$$A(u) = \frac{\left\{\sin\left[\left(\frac{\alpha}{2}\right)u\right]\right\}^{\frac{\alpha}{2}} \left\{\sin\left[\left(1 - \frac{\alpha}{2}\right)u\right]\right\}^{1-\frac{\alpha}{2}}}{\sin(u)}. \quad (2.32)$$

It is worth to note that $[A(u)]^{2/(2-\alpha)}$ is called the Zolotarev's function Devroye [2009]. Let $W \sim \mathcal{EX}\mathcal{P}(1)$ and are $U \sim \mathcal{U}(0, \pi)$ independent. It follows from (2.31) that

$$F_P(p|\alpha) = P\left\{W > p^{-\frac{\alpha}{2-\alpha}} A(U)\right\}. \quad (2.33)$$

A straightforward method was suggested by Kanter [1975] for simulating $P \sim S(\alpha/2, 1, [\cos(\pi\alpha/4)]^{2/\alpha}, 0)$ as follows.

$$P \stackrel{d}{=} \left(\frac{A(U)}{W} \right)^{\frac{2-\alpha}{\alpha}}. \quad (2.34)$$

The PDF of polynomially tilted P α S family, represented as $PT(\alpha, \lambda)$ is given by Devroye [2009]:

$$f_{PT}(x|\alpha, \lambda) = \frac{\Gamma(1+\lambda)}{\Gamma(1+\frac{2\lambda}{\alpha})} x^{-\lambda} f_P(x|\alpha), \quad (2.35)$$

where $\lambda > 0$ (tilting parameter) and $f_P(x|\alpha)$ is the PDF of distribution (2.31). It can be shown that [Devroye, 2009]:

$$PT(\alpha, \lambda) \stackrel{d}{=} \left[\frac{A(R_{\alpha/2, 2\lambda/\alpha})}{G} \right]^{\frac{2-\alpha}{\alpha}}, \quad (2.36)$$

where $G \sim \mathcal{G}(1 + \lambda(2 - \alpha)/\alpha, 1)$ and $R_{a,b}$ on $(0, \pi)$ is the Zolotarev's random variable with PDF

$$f_R(r|a, b) = \frac{\Gamma(1+ab)\Gamma(1+(1-a)b)A(r)^{(1-a)b}}{\pi\Gamma(1+b)}, \quad 0 < a < 1, b > 0. \quad (2.37)$$

[Devroye, 2009] introduces a method for simulating from U with PDF (2.37), and then uses it to generate from PT distribution using (2.36). Let $B(\alpha) = 1 - |1 - \alpha|$ and

$$\begin{aligned} a^\alpha &= \sin\left(\frac{\pi B(\alpha)(1+\beta)}{2}\right) / \sin(\pi B(\alpha)), \\ b^\alpha &= \sin\left(\frac{\pi B(\alpha)(1-\beta)}{2}\right) / \sin(\pi B(\alpha)). \end{aligned} \quad (2.38)$$

Furthermore, suppose $Z \sim S(\alpha, \beta, 1, 0)$ and $P_1, P_2 \sim S(\alpha/2, 1, [\cos(\pi\alpha/4)]^{2/\alpha}, 0)$ are independent. A method for simulating Z in standard case ($\sigma = 1$ and $\mu = 0$) was proposed by Chambers et al. [1976] in B parameterization based on the fact that $Z \stackrel{d}{=} aP_1 + bP_2$. This method was elaborated to generate from Z in commonly used parameterization (2.29) with more detailed information by Weron [1996]. We refer reader to Nolan [2003] for a corrected version of the method for simulating $Z \sim S(\alpha, \beta, 1, 0)$. Let $\theta_0 = \arctan(\beta \tan(\pi\alpha/2))$. We have

$$Z \stackrel{d}{=} \begin{cases} \frac{2}{\pi} \left[\left(\frac{\pi}{2} + \beta U_1 \right) \tan(U_1) - \beta \log \left(\frac{\frac{\pi}{2} W \cos(U_1)}{\frac{\pi}{2} + \beta U_1} \right) \right], & \text{if } \alpha = 1, \\ \frac{\sin[\alpha(\theta_0 + U_1)]}{[\cos(\alpha\theta_0) \cos(U_1)]^{1/\alpha}} \left[\frac{\cos(\alpha\theta_0 + (1-\alpha)U_1)}{W} \right]^{(1-\alpha)/\alpha}, & \text{if } \alpha \neq 1, \end{cases} \quad (2.39)$$

where $W \sim \mathcal{E}\mathcal{H}\mathcal{P}(1)$ is independent of $U_1 \sim \mathcal{U}(-\pi/2, \pi/2)$. For general case ($\sigma \neq 1$ and $\mu \neq 0$), we use the following transformation for generating from $X \sim S(\alpha, \beta, \sigma, \mu)$.

$$X \stackrel{d}{=} \begin{cases} \sigma Z + \mu + \beta \frac{2}{\pi} \sigma \log \sigma, & \text{if } \alpha = 1, \\ \sigma Z + \mu, & \text{if } \alpha \neq 1, \end{cases} \quad (2.40)$$

where $Z \sim S(\alpha, \beta, 1, 0)$ is defined as (2.39). The class of α -stable distributions possesses several nice properties. For instance, suppose $X_1, X_2, X_3 \sim S(\alpha, \beta, \sigma, \mu)$ are independent. Then, for any given positive constants such as a and b , we have

$$aX_1 + bX_2 \stackrel{d}{=} (a^\alpha + b^\alpha)^{1/\alpha} X_3.$$

Furthermore, if $X_1 \sim S(\alpha, \beta, \sigma, \mu)$ and $X_2 \sim S(\alpha, -\beta, \sigma, \mu)$, then $X_1 \stackrel{d}{=} -X_2$. The latter is called reflection property [Nolan, 2003]. The R function `stable(n, alpha, beta, sigma, mu)`, given below, can be used for simulating n realizations of $X \sim S(\alpha, \beta, \sigma, \mu)$ in S_1 parameterization.

```

1 R<- rstable <- function(n, alpha, beta, sigma, mu)
2 +{
3 + u1 <- pi*( runif(n) - 1/2 )
4 + theta0 <- atan( beta*tan(pi*alpha/2) )/alpha
5 + w <- -log( runif(n) )
6 + if (alpha == 1)
7 + {
8 + z <- 2/pi*( (pi/2 + beta*u1)*tan(u1) - beta*log( ( pi/2*w*cos(u1) )/(pi/2 + +beta*u1) ) )
9 + x <- sigma*z + 2/pi*beta*sigma*log(sigma) + mu
10 + }else{
11 + z <- sin(alpha*(theta0 + u1))/(cos(alpha*theta0)*cos(u1))^(1/+alpha)*(cos(alpha*theta0 +
      (alpha - 1)*u1)/w)^( (1 - alpha)/alpha )
12 + x <- sigma*z + mu
13 + }
14 +return(x)
15 +}

```

2.3.7 Birnbaum-Saunders distribution

The Birnbaum-Saunders (BS) distribution has attracted much attention in life-testing and survival analysis. One interesting feature of the BS distribution is the role of the central limit theorem for constructing this distribution. Suppose a sequence of periodic loading are applied to a segment of metal, thereby a sequence of cracks with non-zero length in metal is produced. The total time T , until failure is called BS random variable. The CDF of the BS random variable T , is given by

$$F(t|\boldsymbol{\theta}) = \Phi\left(\frac{1}{\alpha}\left[\sqrt{\frac{t}{\beta}} - \sqrt{\frac{\beta}{t}}\right]\right), \quad (2.41)$$

where $t > 0$, $\Phi(\cdot)$ is the standard Gaussian CDF, and $\boldsymbol{\theta} = (\alpha, \beta)^\top$. Here, $\alpha > 0$ and $\beta > 0$ are the shape and scale parameters of the BS distribution, respectively. The BS distribution has received much attention in a wide range of fields. A review of its applications can be found in Ng et al. [2003], Teimouri [2023] and references therein. Differentiating the CDF given in (2.41), then the PDF of the BS distribution is given by

$$f(t|\boldsymbol{\theta}) = \frac{\sqrt{\frac{t}{\beta}} + \sqrt{\frac{\beta}{t}}}{2\sqrt{2\pi}\alpha t} \exp\left\{-\frac{1}{2\alpha^2}\left(\sqrt{\frac{t}{\beta}} - \sqrt{\frac{\beta}{t}}\right)^2\right\}. \quad (2.42)$$

We write $T \sim \mathcal{BS}(\alpha, \beta)$ to indicate that random Variable T follows a BS distribution with CDF (2.41). If random variable T follows the BS distribution with PDF given by (2.42) can be represented as a mixture of two GIG distributions as

$$f(t|\boldsymbol{\theta}) = \frac{1}{2}g(t|\boldsymbol{\theta}_1) + \frac{1}{2}g(t|\boldsymbol{\theta}_2), \quad (2.43)$$

where $g(\cdot|\boldsymbol{\theta}_i)$ (for $i = 1, 2$) is the PDF of GIG distribution[Hörmann and Leydold, 2014], $\boldsymbol{\theta}_1 = (1/2, 1/(\alpha^2\beta), \beta/\alpha^2)^\top$, and $\boldsymbol{\theta}_2 = (-1/2, 1/(\alpha^2\beta), \beta/\alpha^2)^\top$. This means that each BS random variable can be simulated through a two-component GIG mixture model with weight (or mixing) vector $\boldsymbol{\omega} = (0.5, 0.5)^\top$. It is not hard to check that

$$X \stackrel{d}{=} \frac{1}{2}\left(\sqrt{\frac{T}{\beta}} - \sqrt{\frac{\beta}{T}}\right), \quad (2.44)$$

where $X \sim \mathcal{N}(0, \alpha^2/4)$ or equivalently $T = \beta[1 + 2X^2 + 2X(1 + X^2)^{1/2}]$. Among distributional properties of BS distribution, we mention that if $T \sim \mathcal{BS}(\alpha, \beta)$, then $1/T \sim \mathcal{BS}(\alpha, 1/\beta)$. The R function `rbs(n, alpha, beta)`, given by the following, can be used for simulating n realizations of $T \sim \mathcal{BS}(\alpha, \beta)$.

```

1 R> rbs <- function(n, alpha, beta)
2 +{
3 + X <- alpha/2*rnorm(n)
4 + T <- beta*( 1 + 2*X^2 + 2*X*sqrt( 1 + X^2 ) )
5 + return(T)
6 +}

```

2.4 Rejection sampling

Suppose sampling from PDF $f(\cdot|\theta)$ is not easy but it is dominated by another PDF such as $g(\cdot|\theta)$ so that

$$\sup_{x \in \mathcal{S}_f} \frac{f(x|\theta)}{g(x|\theta)} \leq M,$$

where $M \geq 1$ may depend on θ . A proposal with PDF $g(x|\theta)$ which is as possible as close to $f(x|\theta)$ and easy-to-sample from would be the key objective in each rejection sampling scheme. The following four-step algorithm shows how a rejection sampling scheme works [Devroye, 1986]. It is proved that average numbers of trials for generating a sample in each rejection sampling scheme is M^{-1} , see Ross [2022]. Figure 2.5(b) shows the generated samples across 1000 iterations.

Algorithm 8 Rejection sampling technique

- 1: Choose suitable proposal with PDF $g(\cdot|\theta)$ and determine M ;
 - 2: Generate $y \sim g(\cdot|\theta)$;
 - 3: Generate $u \sim \mathcal{U}(0, 1)$;
 - 4: Squeezing test: If $u < \frac{f(y|\theta)}{M \times g(y|\theta)}$, then accept y as a generation from $f(\cdot|\theta)$, otherwise return to step 2 and repeat algorithm.
-

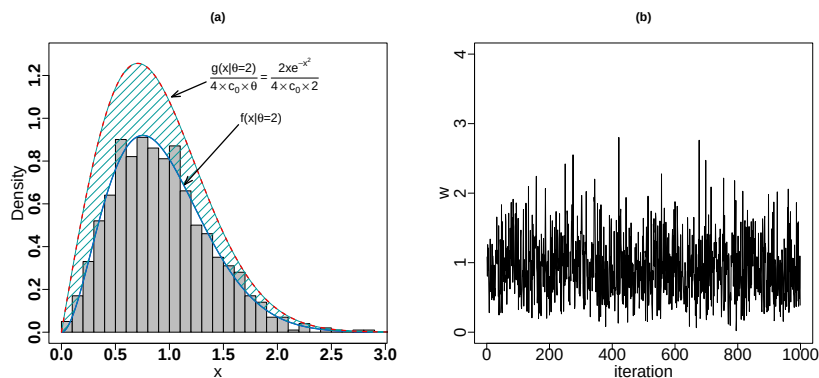


Figure 2.5: (a): Histogram constructed based on 1000 samples generated from $\mathcal{W}(\theta = 2, 1)$. Superimposed are $Mg(x|\theta)$ (red line) and $\mathcal{W}(x|\theta = 2, 1)$ (blue line). (b): Generated samples across iterations for sampling from $f(x|\theta = 2) = \mathcal{W}(x|\theta = 2, 1)$.

Example 2-4

Suppose $f(x|\theta)$ is defined as in Example 2-1. Herein, we can rewrite $f(x|\theta) = (x+1)^{-2}x^\theta \exp\{-x^\theta\}/c_0$ where c_0 is normalizing constant guarantees $\int_{\mathcal{S}_f} f(x|\theta)dx = 1$. Considering $g(x|\theta) = \mathcal{W}(x|\theta, 1)$ as the proposal, it is easy to check that

$$\frac{f(x|\theta)}{g(x|\theta)} = \frac{x}{c_0\theta(x+1)^2}. \quad (2.45)$$

Differentiating the RHS of (2.45) and setting the resultant to 0 reveals that the maximal value of RHS of (2.45) is $1/(4c_0\theta)$ that is obtained for $x = 1$. That is

$$\frac{f(x|\theta)}{g(x|\theta)} \leq \frac{1}{4c_0\theta} = M.$$

It follows that

$$\frac{f(x|\theta)}{M \times g(x|\theta)} = \frac{4x}{(x+1)^2}.$$

Based on Algorithm 8, the rejection sampling scheme for generating realization from $f(\cdot|\theta = 2)$ is described as follows.

1. Generate $y \sim \mathcal{W}(\theta = 2, 1)$;
2. Generate $u \sim \mathcal{U}(0, 1)$;
3. If $u < 4y/(y+1)^2$, then accept y as a generation from $f(\cdot|\theta = 2)$; otherwise go to step 1 and repeat algorithm.

■

In Example 2-4 above, the usage time study for generating $n = 1000$ realizations is around 0.01 second. The pertaining R code, for producing Figure 2.5(b), is given as follows.

```

1 R > set.seed(20240519)
2 R > n <- 10000 # size of generations
3 R > theta <- 2
4 R > w <- rep(0, n) # vector of generated realizations
5 R > j <- 1
6 R > while(j <= n)
7 + {
8 +   y <- rweibull(1, shape = theta, scale = 1)
9 +   u <- runif(1)
10 +   if( u < 4*y/(y+1)^2 )
11 +   {
12 +     w[j] <- y # y is accepted
13 +     j <- j + 1
14 +   }
15 + }
16 R > plot(w)
```

Example 2-5

Let random variable X follow a distribution with PDF given by Azzalini [1985]:

$$f(x|\mu, \sigma, \lambda) = \frac{2}{\sigma} \phi\left(\frac{x-\mu}{\sigma}\right) \Phi\left(\lambda \frac{x-\mu}{\sigma}\right), \quad (2.46)$$

where $\phi(\cdot)$ and $\Phi(\cdot)$ represent accordingly the PDF and CDF of the standard Gaussian distribution. Herein, $\lambda \in \mathbb{R}$ is the skewness parameter (the cases $\lambda \rightarrow -\infty$, $\lambda = 0$, and $\lambda \rightarrow +\infty$ yield totally skewed to the left, symmetric (ordinary Gaussian), and totally skewed to the right distributions, respectively), $\mu \in \mathbb{R}$ is the location parameter, and $\sigma \in \mathbb{R}^+$ denotes the scale parameter. Hereafter, we write $X \sim \mathcal{SG}(\mu, \sigma, \lambda)$ to denote that X follows the distribution with PDF given by (2.46). Without loss of generality, let $\mu = 0$ and $\sigma = 1$. A direct method proposed by Henze [1986] for simulating from $\mathcal{SG}(0, 1, \lambda)$ as follows. Let Z_1 and Z_2 denote two independent copies of the standard Gaussian random variate. Then, for random variable $X \sim \mathcal{SG}(0, 1, \lambda)$ we can write

$$X \stackrel{d}{=} \frac{\lambda|Z_1| + Z_2}{\sqrt{1 + \lambda^2}}. \quad (2.47)$$

For simulating from $\mathcal{SG}(0, 1, \lambda)$ using the rejection method, let $f(x|0, 1, \lambda = 4)$ to be the PDF of $\mathcal{SG}(x|0, 1, \lambda = 4)$. We proceed by considering the fact that

$$f(x|0, 1, \lambda) \leq 2g(x|\theta) = 2\phi(x). \quad (2.48)$$

Hence, simulating from $\mathcal{SG}(0, 1, \lambda)$, through the rejection sampling, is possible by considering the proposal PDF to be $\phi(x)$ and $M = 2$. Figure 2.6(a) shows histogram of generated realizations as well as the superimposed target and proposal PDFs for $\lambda = 4$. The sampler's motion across 500 iterations are displayed in Figure 2.5(b). ■

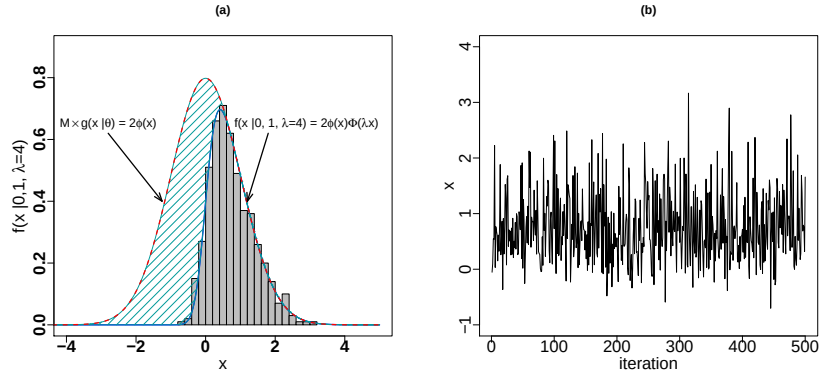


Figure 2.6: (a): Histogram constructed based on 500 samples generated from $\mathcal{SG}(0, 1, \lambda = 4)$. Superimposed are $Mg(x|\theta) = 2\phi(x)$ (red line) and $f(x|0, 1, \lambda = 4)$ (blue line). (b): Generations across iterations for sampling from $f(x|0, 1, \lambda = 4) \sim \mathcal{SG}(0, 1, \lambda = 4)$.

The pertaining R code, for producing Figure 2.6(b) is given as follows.

```

1 R> set.seed(20240225)
2 R> n <- 500
3 R> i <- 1; x <- rep(0, n); lambda <- 4
4 R> while(i <= n)
5 + {
6 +   z <- rnorm(1);
7 +   if( runif(1) < pnorm(lambda*z) )
8 +   {
9 +     x[i] <- z; i <- i + 1
10 +   }
11 + }
12 R> plot(x)
```

2.5 Adaptive rejection sampling

Suppose $f(x|\theta)$ is a log-concave PDF over $\mathcal{S}_f$ that equivalently means $\mathcal{H}(x) = \log f(x|\theta)$ is a concave function of x . A method was proposed by ? for generating realization from log-concave distribution provided that the mode (a point at which derivative of $f(x|\theta)$ with respect to x is zero) of $f(x|\theta)$ to be known. More precisely, the proposed method is a rejection sampling scheme using a three-piece exponential proposal in which the center piece touches the target PDF $f(x|\theta)$ at its mode. While this method is not adaptive, the adaptive rejection sampling (ARS) scheme [Gilks and Wild, 1992] develops this method by improving the proposal distribution in each iteration such that the updated proposal proposes generation that passes the *squeezing* test with higher probability than the previous iteration. In each ARS scheme, $\mathcal{H}(x)$ can be bounded by piecewise linear upper hull in which each tangent line is defined as

$$U_k(x) = (x - a)\mathcal{H}'(a) + \mathcal{H}(a), \quad (2.49)$$

for $x \in [z_{i-1}, z_i]$ and $i = 1, \dots, k$ where

$$z_i = \frac{\mathcal{H}(x_{i+1}) - \mathcal{H}(x_i) - x_{i+1}\mathcal{H}'(x_{i+1}) + x_i\mathcal{H}'(x_i)}{\mathcal{H}'(x_i) - \mathcal{H}'(x_{i+1})}.$$

We note that the tangent lines are constructed at abscissa (points) $T_k = \{x_1 \leq x_2 \leq \dots \leq x_k \in \mathcal{S}_f\}$ and intersect at z_i where z_0 and z_k are the lower ($-\infty$ if $\mathcal{S}_f$ is not bounded from below) and the upper (∞ if $\mathcal{S}_f$ is not bounded from above) bounds of $\mathcal{S}_f$, accordingly. Likewise, we can define a piecewise linear lower hull constructed based on chords between adjacent abscissae in T_k as

$$L_k(x) = \frac{\mathcal{H}(x_{i+1} - x)\mathcal{H}(x_i) - \mathcal{H}(x - x_i)\mathcal{H}(x_{i+1})}{x_{i+1}x_i}, \quad (2.50)$$

where $x \in [x_i, x_{i+1}]$ for $i = 1, \dots, k-1$. If $x < x_1$ (or $x > x_k$), then we set $L_k(x) = -\infty$. Notice that log-concavity of $f(x|\theta)$ ensures that $L_k(x) \leq \mathcal{H}(x) \leq U_k(x)$ for $x \in \mathcal{S}_f$. Each ARS algorithm proceeds three *initialization*, *sampling*, and *updating* steps for generating n realizations from PDF $f(x|\theta)$ given by the following.

- *initialization step*: Construct the abscissae in T_k . If $\mathcal{S}_f$ is left-unbounded, then set x_1 such that $\mathcal{H}'(x_1) < 0$ and if $\mathcal{S}_f$ is right-unbounded, then set x_k such that $\mathcal{H}'(x_k) < 0$. Furthermore, compute $L_k(x)$, $U_k(x)$, and $S_k(x)$ where

$$s_k(x) = \frac{\exp\{U_k(x)\}}{\int \exp\{U_k(y)\}dy},$$

and functions $L_k(x)$ and $U_k(x)$ as defined in (2.50) and (2.49), accordingly.

- *sampling step*: Generate x^* from $s_k(x)$ and $u \sim \text{Unif}(0, 1)$ independently; and then perform squeezing test that accepts x^* if $w \leq \exp\{L_k(x^*) - U_k(x^*)\}$; otherwise compute the rejection test that accepts x^* if $w \leq \exp\{\mathcal{H}(x^*) - U_k(x^*)\}$; otherwise reject x^* .
- *updating step*: If evaluating $\mathcal{H}(x^*)$ and $\mathcal{H}'(x^*)$ is necessary, then add x^* to T_k in order to form updated T_k as $T_k^* = \{\{x_1 \leq x_2 \leq \dots \leq x_k\} \cup x^* \in \mathcal{S}_f\}$. Return to the *initialization* step while the set of $k+1$ abscissae in T_k^* is relabeled in ascending order and repeat these three steps if n realizations have been not yet obtained. We note that for initialization, a set of two starting abscissae ($k \geq 2$) is necessary.

We note that a given PDF $f(\cdot|\theta)$ is unimodal if and only if $f(\cdot|\theta)$ is log-concave. So, for simulating from a distribution with unimodal PDF $f(\cdot|\theta)$, the ARS algorithm would be a suitable candidate provided that $\mathcal{H}(\cdot)$ has closed form.

Example 2-6

Let $f(x|\boldsymbol{\theta})$ represent the PDF of generalized inverse Gaussian (GIG), represented as $\mathcal{GIG}(\psi, \chi, \lambda)$, distribution defined as [Johnson et al., 1995, Jorgensen, 2012]:

$$f(x|\boldsymbol{\theta}) = \left(\frac{\psi}{\chi}\right)^{\frac{\lambda}{2}} \frac{x^{\lambda-1}}{2\mathcal{K}_{\lambda}(\sqrt{\psi\chi})} \exp\left\{-\frac{\psi x}{2} - \frac{\chi}{2x}\right\}, \quad (2.51)$$

where $\boldsymbol{\theta} = (\psi, \chi, \lambda)^{\top}$. It is easy to check that $f(x|\boldsymbol{\theta})$ is log-concave for $\lambda \geq 1$. Implementing the ARS algorithm is executable through the computer code. The package `ars` developed for this purpose that is available at <https://CRAN.R-project.org/package=ars>. ■

2.5.1 Two-dimensional single rejection sampling

Let $f(x|\theta)$, that may be known only up to a proportionality constant z , is an unimodal PDF of random variable $X \in \mathbb{R}$ from which sampling is of interest. If it is possible to consider an extra auxiliary variable Y , thereby the marginal PDF of X can be marginalized by integrating out Y as

$$f(x|\theta) = \int f(x, y|\theta) dy,$$

then one can use the *two-dimensional rejection sampling* scheme in order to generate sample x with PDF $f(x|\theta)$. For this purpose, let $l(x, y|\theta)$ denote the joint PDF of the proposal (candidate) such that

$$\sup_{(x,y) \in \mathcal{S}_{f(x,y|\theta)}} \frac{f(x, y|\theta)}{l(x, y|\theta)} = c.$$

We use the following algorithm for simulating from $f(x|\theta)$ through the *two-dimensional single rejection* sampling scheme.

1. Generate pair (X, Y) or the joint PDF $l(x, y|\theta)$;
2. Generate $u \sim \text{Unif}(0, 1)$;
3. If $u < f(x, y|\theta)/[c \times l(x, y|\theta)]$, then accept X as a generation from $f(x|\theta)$; otherwise return to step 1 and repeat the algorithm.

The efficiency of a *two-dimensional single rejection* sampling scheme depends on quantity c and the complexity of sampling from two-dimensional candidate $l(x, y|\theta)$. If $l(x, y|\theta)$ is easy to sample from and c is as small as possible, then this algorithm works efficiently.

Example 2-7

Let X denotes a random variable following an exponentially tilted α -stable distribution with tilting parameter $\lambda > 0$ if it follows a distribution whose PDF is given by

$$f(x|\boldsymbol{\theta}) = \exp\{-x\lambda + \lambda^{\frac{\alpha}{2}}\} d(x|\alpha), \quad (2.52)$$

where $\boldsymbol{\theta} = (\alpha, \lambda)^{\top}$ for $0 < \alpha \leq 2$ is the family parameter vector and $d(x|\alpha)$ is the PDF of a positive α -stable distribution, see Devroye [2009]. The PDF $d(x|\alpha)$ has not closed form, but can be represented as

$$d(x|\alpha) = \frac{1}{\pi} \int_0^{\pi} \exp\left\{-x^{-\frac{\alpha}{2-\alpha}} A(y)\right\} dy, \quad (2.53)$$

where

$$A(y) = \frac{\left\{ \sin\left[\left(\frac{\alpha}{2}\right)y\right] \right\}^{\frac{\alpha}{2}} \left\{ \sin\left[\left(1 - \frac{\alpha}{2}\right)y\right] \right\}^{1-\frac{\alpha}{2}}}{\sin(y)}.$$

We note that $A(y)^{2/(2-\alpha)}$ is called the Zolotarev's function [Devroye, 2009]. From (2.52) and (2.53), it can be easily seen that $f(x|\boldsymbol{\theta})$ is the marginal of a joint PDF. Herein, we do not aim to discuss this example in details and just mention that for simulating from $f(x, y|\boldsymbol{\theta})$, a bivariate distribution with PDF

$$l(x, y|\boldsymbol{\theta}) = \frac{1}{\pi} \times \frac{x^{m-1}}{\Gamma(m)} \exp\{-x\}, \quad (2.54)$$

is chosen as candidate. Depending on values of α and λ , the gamma PDF with shape parameter $m = \alpha\lambda^\alpha$ in RHS of (2.54) can be replaced with a gamma PDF with shape parameter $m = (1 - \alpha)\lambda^\alpha + 1$ or truncated Gaussian PDF. The reader is referred to [Hofert, 2011, Qu et al., 2021] for a thorough treatment of this example. For simulating from PDF (2.52), there is an open source code called `copula` has been developed for R language that is available at <https://CRAN.R-project.org/package=copula>. ■

Example 2-8

Let $\mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$ denote a p -dimensional random vector $\mathbf{X} = (X_1, \dots, X_p)^\top$ following a Gaussian distribution with mean vector $\boldsymbol{\mu}$ and covariance matrix Σ whose PDF and CDF are shown by $\phi_p(\cdot|\boldsymbol{\mu}, \Sigma)$ (relation 2.67) and $\Phi_p(\cdot|\boldsymbol{\mu}, \Sigma)$, respectively. Furthermore, write $\mathcal{N}_p(\boldsymbol{\mu}, \Sigma, \mathcal{R}^p)$ to denote a p -dimensional random vector $\mathbf{X} = (X_1, \dots, X_p)^\top$ following a Gaussian distribution truncated on positive Lebesgue measure region $\mathcal{R}^p$ denoted as

$$\mathcal{R}^p = \{\mathbf{x} \in \mathbb{R}^p | \mathbf{a} \leq \mathbf{A}\mathbf{x} \leq \mathbf{b}\}, \quad (2.55)$$

where $\mathbf{a} = (a_1, \dots, a_p)^\top$ and $\mathbf{b} = (b_1, \dots, b_p)^\top$ are vectors of finite or infinite constants. Moreover, matrix $\mathbf{A}$ is a $p \times p$ full rank matrix that constructs a set of p independent linear constraints. As pointed out by [Geweke, 1991] the marginals of $\mathbf{X}$ with distribution given by (2.55) are not truncated Gaussian, but the full conditionals, that is distribution of X_i (for $i = 1, \dots, p$) *conditional* on all of other members of $\mathbf{X}$ are truncated Gaussian. To verify this claim, we shall confine ourselves here to suppose $p = 2$ and $\mathbf{A} = \mathbf{I}_2$ in which $\mathbf{I}_2$ accounts for a 2×2 identity matrix. Therefore (2.55) can be rewritten as

$$\mathcal{N}_2(\boldsymbol{\mu}, \Sigma, \mathcal{R}^2), \quad (2.56)$$

where $\mathcal{R}^2 = \{\mathbf{x} = (x_1, x_2)^\top \in \mathbb{R}^2 | a_1 \leq x_1 \leq b_1, a_2 \leq x_2 \leq b_2\}$, $\boldsymbol{\mu} = (\mu_1, \mu_2)^\top$, and $\Sigma = [(\sigma_{11}, \sigma_{21})^\top, (\sigma_{12}, \sigma_{22})^\top]$. In univariate case, we may write $\mathcal{N}(\mu, \sigma, \mathcal{R})$ to denote the family of univariate Gaussian distributions with mean μ and variance σ truncated on interval $\mathcal{R} = \{x \in \mathbb{R} | a \leq x \leq b\}$. For computing the marginal PDF of (4.398), e.g. PDF of X_1 , we have

$$\begin{aligned} f_{X_1}(x_1|\boldsymbol{\mu}, \Sigma) &= \frac{1}{\mathcal{P}_G} \int_{a_2}^{b_2} \phi_2(\mathbf{x}|\boldsymbol{\mu}, \Sigma) dx_2, \\ &= \frac{1}{\mathcal{P}_G} \phi(x_1|\mu_1, \sigma_{11}) \int_{a_2}^{b_2} \phi(x_2|\mu_{2.1}, \sigma_{2.1}) dx_2, \\ &= \frac{1}{\mathcal{P}_G} \phi(x_1|\mu_1, \sigma_{11}) \left[\Phi(b_2|\mu_{2.1}, \sigma_{2.1}) - \Phi(a_2|\mu_{2.1}, \sigma_{2.1}) \right], \end{aligned} \quad (2.57)$$

where $\mathcal{P}_G = \Phi_2(\mathbf{b}|\boldsymbol{\mu}, \Sigma) - \Phi_2(\mathbf{a}|\boldsymbol{\mu}, \Sigma)$, $x_1 \in \mathcal{R}_1 = \{x_1 \in \mathbb{R} | a_1 \leq x_1 \leq b_1\}$, and

$$\mu_{2.1} = \mu_2 + \sigma_{21}\sigma_{11}^{-1}(x_1 - \mu_1), \quad (2.58)$$

$$\sigma_{2.1} = \sigma_{22} - \sigma_{21}\sigma_{11}^{-1}\sigma_{12}. \quad (2.59)$$

Obviously, since both terms within the square bracket in the RHS of (4.239) depend on x_1 through $\mu_{2.1}$ given by (2.58), hence $f_{X_1}(x_1|\boldsymbol{\mu}, \Sigma)$ is no longer the PDF of a truncated Gaussian distribution unless $\mu_{2.1}$ is independent of x_1 that occurs only when $\sigma_{12} = \sigma_{21} = 0$. This is while both of the full conditionals $X_1|X_2$ and $X_2|X_1$ correspond to the joint PDF in (4.398) are Gaussian. For example $X_2|X_1 \sim \mathcal{N}(\mu_{2.1}, \sigma_{2.1}, \mathcal{R}_2)$ where $\mathcal{R}_2 = \{x_2 \in \mathbb{R} | a_2 \leq x_2 \leq b_2\}$. Herein, we are willing to apply the two-dimensional single rejection scheme for sampling from distribution (4.398) whose PDF may be rewritten as

$$\begin{aligned} f_{X_1, X_2}(x_1, x_2 | \boldsymbol{\mu}, \Sigma) &= \frac{1}{\mathcal{P}_G} \phi\left(\frac{x_1 - \mu_1}{\sqrt{\sigma_{11}}}\right) \phi\left(\frac{x_2 - \mu_{2.1}}{\sqrt{\sigma_{2.1}}}\right) \times \mathbb{I}_{\mathcal{R}^2}(x_1, x_2) \\ &= c_1 \phi\left(\frac{x_1 - \mu_1}{\sqrt{\sigma_{11}}}\right) \phi\left(\frac{x_2 - \mu_{2.1}}{\sqrt{\sigma_{2.1}}}\right) \times \mathbb{I}_{\mathcal{R}^2}(x_1, x_2), \end{aligned}$$

where c_1 is a constant independent of $(x_1, x_2)^\top$ and $\mu_{2.1}$ and $\sigma_{2.1}$ are defined in (2.58) and (2.59), respectively. Furthermore, $\mathbb{I}_{\mathcal{R}^2}(x, y)$ is the well-known indicator function defined for the set $\mathcal{R}^2$ defined as

$$\mathbb{I}_{\mathcal{R}^2}(x_1, x_2) = \begin{cases} 1, & \text{if } (x_1, x_2) \in \mathcal{R}^2, \\ 0, & \text{if } (x_1, x_2) \notin \mathcal{R}^2. \end{cases} \quad (2.60)$$

We may consider the distribution of proposal as the product of $\mathcal{N}_{R_1}(\mu_1, \sigma_{11}, \mathcal{R}_1)$ and $\mathcal{U}(a_2, b_2)$ with PDF

$$\begin{aligned} l_{X_1, X_2}(x_1, x_2 | \boldsymbol{\mu}, \Sigma) &= \frac{\phi\left(\frac{x_1 - \mu_1}{\sqrt{\sigma_{11}}}\right) \times \mathbb{I}_{\mathcal{R}_1}(x_1)}{\Phi(b_1 | \mu_1, \sqrt{\sigma_{11}}) - \Phi(a_1 | \mu_1, \sqrt{\sigma_{11}})} \times \frac{\mathbb{I}_{\mathcal{R}_2}(x_2)}{b_2 - a_2}, \\ &= c_2 \times \phi\left(\frac{x_1 - \mu_1}{\sqrt{\sigma_{11}}}\right) \times \mathbb{I}_{\mathcal{R}_1}(x_1) \times \mathbb{I}_{\mathcal{R}_2}(x_2), \end{aligned}$$

where c_2 is independent of $(x_1, x_2)^\top$. It follows that

$$\begin{aligned} f_{X_1, X_2}(x_1, x_2 | \boldsymbol{\mu}, \Sigma) &\leq \frac{c_1}{\sqrt{2\pi\sigma_{2.1}}} \phi\left(\frac{x_1 - \mu_1}{\sqrt{\sigma_{11}}}\right) \times \mathbb{I}_{\mathcal{R}^2}(x_1, x_2) \\ &\leq c_1 \frac{b_2 - a_2}{\sqrt{2\pi} \times \sigma_{2.1}} l_{X_1, X_2}(x_1, x_2 | \boldsymbol{\mu}, \Sigma) \\ &= c \times l_{X_1, X_2}(x_1, x_2 | \boldsymbol{\mu}, \Sigma). \end{aligned}$$

Hence,

$$\frac{f_{X_1, X_2}(x_1, x_2 | \boldsymbol{\mu}, \Sigma)}{c \times l_{X_1, X_2}(x_1, x_2 | \boldsymbol{\mu}, \Sigma)} = \exp\left\{-\frac{(x_2 - \mu_{2.1})^2}{2\sigma_{2.1}}\right\}.$$

In what follows Algorithm 9 describes how to sample from truncated bivariate Gaussian distribution. In general where A is not necessarily an identical full rank matrix, we suggest the

use of method proposed by Geweke [1991] for simulating from truncated bivariate Gaussian distribution that will be discussed in Section 6.7. Here we give a brief discussion of this method. For simulating from PDF given by (2.55), first one needs to simulate from $\mathbf{Z} \sim \mathcal{N}_{\mathbf{T}}(\mathbf{0}, A\Sigma A^\top, \mathcal{R}^p)$ where $\mathcal{R}^p = \{\mathbf{z} \in \mathbb{R}^p | \mathbf{a} - A\boldsymbol{\mu} \leq \mathbf{z} \leq \mathbf{b} - A\boldsymbol{\mu}\}$, and then accept $\mathbf{X} = \boldsymbol{\mu} + A^{-1}\mathbf{Z}$ as a generation from PDF given by (2.55). The R function below is provided for generating n sample from truncated bivariate Gaussian distribution through two-dimensional single rejection scheme.

```
1 R> rtbinorm_rejection <- function(n, Mu, Sigma, a, b, A)
2 + {
```

Algorithm 9 Two-dimensional single rejection scheme for sampling from truncated bivariate Gaussian distribution

- 1: Read $\mathbf{a}$, $\mathbf{b}$, $\boldsymbol{\mu}$, and Σ given by (4.398);
 - 2: Generate $u \sim \mathcal{U}(0, 1)$;
 - 3: Generate $x_1 \sim \mathcal{TN}_{R_1}(\mu_1, \sigma_{11})$;
 - 4: Generate $x_2 \sim \mathcal{U}(a_2, b_2)$;
 - 5: Squeezing test: If $-2\sigma_{2.1} \log u > (x_2 - \mu_{2.1})^2$ where $\mu_{2.1}$ and $\sigma_{2.1}$ are given by (2.58) and (2.59), respectively, then accept $(x_1, x_2)^\top$ as a generation from PDF given by (4.398), otherwise return to step 2 and repeat algorithm;
-

```

3 + V <- A%*%Sigma%*t(A)
4 + T1 <- matrix( cbind(a - A%*%Mu, b - A%*%Mu), nrow = 2, ncol = 2)
5 + X <- Z <- matrix(0, nrow = n, ncol = 2)
6 + sigma2.1 <- V[2, 2] - V[1, 2]^2/V[1, 1]
7 + sigma1 <- sqrt( V[1, 1])
8 + j <- 1
9 + while(j < n)
10 + {
11 +   u <- runif(3)
12 +   cdf.a <- pnorm(T1[1, 1], 0, sigma1)
13 +   cdf.b <- pnorm(T1[1, 2], 0, sigma1)
14 +   x <- qnorm( u[1]*(cdf.b - cdf.a) + cdf.a,
15 +             mean = 0, sd = sigma1 )
16 +   mu2.1 <- V[1, 2]/V[1, 1]*x
17 +   y <- (T1[2, 2] - T1[2, 1])*u[2] + T1[2, 1]
18 +   if( -2*log(u[3]) > (y - mu2.1)^2/sigma2.1 )
19 +   {
20 +     Z[j, ] <- c(x, y)
21 +     j <- j + 1
22 +   }
23 + }
24 + X <- matrix(Mu, nrow = n, ncol = 2, byrow = T) + Z%*%solve(A)
25 + return(X)
26 + }

```



2.6 Ratio of uniforms method

The Ratio of uniforms method is in fact a rejection method that introduced by Kinderman and Monahan [1977]. Suppose we are interested in generating random variable X with PDF is $f(x)$ that may be known only up to a proportionality constant $k > 0$, that is $f(x) = k \times h(x)$ ². If random vector $(U, V)^\top$ is distributed uniformly on $C_h = \{(u, v) \in \mathbb{R}^2 \mid 0 \leq u \leq \sqrt{h(v/u)}\}$ with PDF $g(u, v)$ given by

$$g(u, v) = \begin{cases} 2k, & \text{if } 0 \leq u \leq \sqrt{h(v/u)}, \\ 0, & \text{otherwise.} \end{cases} \quad (2.61)$$

Consider two transformations $X = V/U$ and $Y = U$, it can be shown that the distribution of random X follows a distribution with PDF $f(x)$. To show this claim, we notice that Jacobian of inverted transformations, that is $U = Y$ and $V = XY$, is $-Y$. Therefore

$$f(x, y) = |-y| \times g(v/u, u) = y \times 2k, \quad 0 < y < \sqrt{h(x)}.$$

²The function $h(x)$ sometimes is called quasi PDF.

An expression for the marginal PDF of X is obtained by integrating out y as

$$\int_0^{\sqrt{h(x)}} f(x, y) dy = \int_0^{\sqrt{h(x)}} 2ky dy = kh(x) = f(x).$$

For generating X , first we generate two realizations from $\mathcal{U}(0, a)$ and $\mathcal{U}(b, c)$ in which

$$a = \sup_x \sqrt{h(x)}, \quad b = -\sup_x x\sqrt{h(x)} = \inf_x x\sqrt{h(x)}, \quad c = \sup_x x\sqrt{h(x)}.$$

The support of random vector $(U, V)^\top$, that is $\{(u, v) | u \in (0, a) \text{ and } v \in (b, c)\}$, is a bounding rectangle that encloses the region C_h . If simulated vector $(U, V)^\top$ lies inside region C_h , then we accept V/U as a generation of X . Hence, if $h(x)$ and $x^2h(x)$ are finite, then ratio of uniforms method proceeds as follows for simulating X with PDF $f(x)$.

- i. Generate $U \sim \mathcal{U}(0, a)$ and $V \sim \mathcal{U}(b, c)$ independently;
- ii. If $U^2 < h(V/U)$, then accept $X = V/U$ from PDF $f(x)$, otherwise go back step (i).

Evidently, the ratio of uniforms method is a rejection method. The acceptance rate is

$$\int h(x) dx \times [2a(c - b)]^{-1}.$$

As pointed out by [Wakefield et al., 1991, Theorem 2], for all location families of distributions that are unimodal, the acceptance probability for the ratio of uniforms method maximizes when the location parameter is zero. Therefore, for example, to generate $X \sim \mathcal{N}(\mu, \sigma^2)$, first we simulate from $Y = \mathcal{N}(0, \sigma^2)$ and then we have $Y + \mu \sim \mathcal{N}(\mu, \sigma^2)$.

Example 2-9

Suppose we are interested in simulating from $X \sim \mathcal{N}(0, 1)$ through the ratio of uniforms method. Let $h(x) = \exp\{-x^2/2\}$, we can see that

$$\begin{aligned} C_h &= \left\{ (u, v) \in \mathbb{R}^2 \mid 0 \leq u \leq \exp\{-1/2 \times v^2/u^2\} \right\} \\ &= \left\{ (u, v) \in \mathbb{R}^2 \mid v^2 \leq -4u^2 \log u \right\}, \end{aligned}$$

and

$$a = \sup_x \sqrt{h(x)} = 1, \quad b = -\sup_x x\sqrt{h(x)} = -\sqrt{2e}, \quad c = \sup_x x\sqrt{h(x)} = \sqrt{2e},$$

where $e = \exp\{1\} = 2.718281\dots$. We follow a two-step procedure given below for simulating X .

- i. Generate $U \sim \mathcal{U}(0, 1)$ and $V \sim \mathcal{U}(-\sqrt{2/e}, \sqrt{2/e})$ independently;
- ii. If $U^2 < \exp\{-1/2 \times V^2/U^2\}(V/U)$, then accept $X = V/U$ from PDF standard Gaussian distribution, otherwise go back to step (i).

The acceptance rate for this method is

$$\int h(x) dx \times [2a(c - b)]^{-1} = \frac{\sqrt{2\pi}}{4\sqrt{2} \exp\{-1\}} \approx 0.731.$$

The pertaining R function called `rnorm_ru`, for generating n realizations from $X \sim \mathcal{N}(0, 1)$ based on the ratio of uniforms method is given as follows.

```

1 R> rnorm_ru <- function(n)
2 +{
3 + X <- rep(NA, n)
4 + j <- 1
5 + while( j <= n )
6 + {
7 + U <- runif(1)
8 + V <- (2*runif(1) - 1)*sqrt( 2*exp(-1) )
9 + if( V^2 < -4*U^2*log( U ) )
10 + {
11 + X[j] <- V/U
12 + j <- j + 1
13 + }
14 + }
15 +return(X)
16 +}

```



Example 2-10

Let $X \sim \mathcal{G}(\alpha, \beta)$ with $\alpha > 1$. We know that $X \stackrel{d}{=} G/\beta$ in which $G \sim \mathcal{G}(\alpha, 1)$, hence without loss of generality, for simulating X through the ratio of uniforms method, we assume that $\beta = 1$. Once we have simulated G , the quantity X is simply produced as $X = G/\beta$. Setting $h(x) = x^{\alpha-1} \exp\{-x\}$, we have

$$\begin{aligned}
C_h &= \left\{ (u, v) \in \mathbb{R}^2 \mid u < \sqrt{(v/u)^{\alpha-1} \exp\{-v/u\}} \right\} \\
&= \left\{ (u, v) \in \mathbb{R}^2 \mid 2 \log u < (\alpha - 1) \log(v/u) - v/u \right\},
\end{aligned}$$

and

$$a = \sup_x \sqrt{h(x)} = \left(\frac{\alpha - 1}{e} \right)^{(\alpha-1)/2}, \quad b = -\sup_x x \sqrt{h(x)} = 0, \quad c = \sup_x x \sqrt{h(x)} = \left(\frac{\alpha + 1}{e} \right)^{(\alpha+1)/2},$$

where $e = \exp\{1\} = 2.718281\dots$. We follow a three-step procedure given below for simulating X .

- i. Generate $U \sim \mathcal{U}(0, 1)$ and $V \sim \mathcal{U}(-\sqrt{2/e}, \sqrt{2/e})$ independently;
- ii. If $U^2 < (V/U)^{\alpha-1} \exp\{-V/U\}$, then go next step, otherwise go back to step (i).
- iii. Accept $X = G/\beta$ as a generation from $\mathcal{G}(\alpha, \beta)$.

```

1 R> rgamma_ru <- function(n, alpha, beta = 1)
2 +{
3 + if (alpha <= 1) stop( "shape_parameter_must_be_greater_than_one." )
4 + a <- ( (alpha - 1)*exp(-1) )^( (alpha - 1)/2 )
5 + b <- 0
6 + c0 <- ( (alpha + 1)*exp(-1) )^( (alpha + 1)/2 )
7 + X <- rep(NA, n)
8 + j <- 1
9 + while( j <= n )
10 + {
11 + U <- runif(1, 0, a); V <- runif(1, 0, c0)
12 + upper <- 1/2*( (alpha - 1)*log(V/U) - V/U )
13 + if( log(U) < upper )
14 + {
15 + X[j] <- 1/beta*V/U
16 + j <- j + 1

```

```

17 + }
18 + }
19 +return(X)
20 +}

```

When $\alpha < 1$, then $h(x)$ appears to be unbounded and hence the ratio of uniforms method cannot be applied for simulating from gamma distribution. ■

2.7 Generalized ratio of uniforms method

The ratio of uniforms method discussed earlier has been extended by Wakefield et al. [1991]. Let $g(\cdot) : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ denote a strictly increasing function such that $g(0) = 0$. If random vector $(U, V)^\top$ is distributed uniformly on region

$$C_{h,g} = \left\{ (u, v) \in \mathbb{R}^2 \mid 0 < u \leq g^{-1} \left[ch \left(\frac{v}{g'(u)} \right) \right] \right\}, \quad (2.62)$$

where $c > 0$ is a constant, then $X = V/g'(U)$ follows a distribution with PDF $f(x) = h(x)/\int h(x)$ in which $h(x)$ is the quasi PDF. We note that $g'(\cdot)$ denotes the first derivative of $g(\cdot)$. The bounding rectangle $(0, a_g) \times (b_g, c_g)$ corresponds to the region $C_{h,g}$ is constructed by determining

$$\begin{aligned} a_g &= \sup_x g^{-1}[ch(x)] \\ b_g &= \inf_{x \leq 0} xg'[g^{-1}(ch(x))] \leq 0 \\ c_g &= \sup_{x \geq 0} xg'[g^{-1}(ch(x))] \geq 0. \end{aligned} \quad (2.63)$$

A simple choice for $g(\cdot)$ is then the power function for which $g'(\cdot)$ is also strictly increasing is $g(x) = u^{r+1}/(r+1)$ (for $r \geq 0$) and hence $c = (1+r)^{-1}$. The original method is recovered for $r = 1$ for which the region $C_{h,g}$ given in (2.62) turns into

$$C_h(r) = \left\{ (u, v) \in \mathbb{R}^2 \mid 0 < u \leq \left[h \left(\frac{v}{u^r} \right) \right]^{1/(r+1)} \right\}. \quad (2.64)$$

The area of $C_h(r)$ given in (2.64) becomes $(1+r)^{-1} \int h(x) dx$. If $h(x)$ and $x^{r+1}[h(x)]^r$ are bounded, then the bounds of rectangle (2.64) are

$$\begin{aligned} a(r) &= \sup_x [h(x)]^{1/(r+1)} \\ b(r) &= \inf_{x \leq 0} x [h(x)]^{r/(r+1)} \leq 0 \\ c(r) &= \sup_{x \geq 0} x [h(x)]^{r/(r+1)} \geq 0. \end{aligned} \quad (2.65)$$

For a pair $(u, v)^\top$ generated from rectangle (2.64), the acceptance probability is

$$p(r) = \frac{\int h(x)}{(r+1) \times a(r) \times [c(r) - b(r)]}. \quad (2.66)$$

Example 2-11

Following the Example 2-9, we would like to simulate from $X \sim \mathcal{N}(0, 1)$ through the generalized ratio of uniforms method. Let $h(x) = \exp\{-x^2/2\}$ for which $\int h(x)dx = \sqrt{2\pi}$. Furthermore, we can see $a(r) = 1$ and $b(r) = -c(r) = -\sqrt{1 + 1/r} \exp\{-1/2\}$. Substituting these quantities into (2.66) yields

$$p(r) = \frac{\sqrt{2\pi r \exp\{1\}}}{2(1 + r)^{3/2}}.$$

The maximum value of $p(r)$ is obtained when $r = 0.5$, that is $p(0.5) = 0.795$. Comparing with the acceptance rate of the original method discussed in Example 2-9 that is $p(1) = 0.731$, the generalized ratio of uniforms method is more efficient. ■

Table 2.1 lists some R commands for simulating from widely used univariate distributions. For other distributions, user may use the approaches discussed earlier in this chapter or those discussed in ?Ross [2022].

Table 2.1: Commands for simulating from some statistical families in R.

Family	command	PDF
$\mathcal{U}(a, b)$	<code>runif(n,min=a,max=b)</code>	$\frac{1}{b-a}$
$\mathcal{BET}(a, b)$	<code>rbeta(n,shape1=a,shape2=b,ncp=0)</code>	$\frac{1}{B(a,b)} x^{a-1} (1-x)^{b-1}$
$\mathcal{G}(a, b)$	<code>rgamma(n,shape=a,rate=b)</code>	$\frac{b^a}{\Gamma(a)} x^{a-1} \exp\{-bx\}$
$\chi(\nu)$	<code>rchisq(n,df=nu,ncp=0)</code>	$\frac{2^{-\frac{\nu}{2}}}{\Gamma(\nu/2)} x^{\frac{\nu}{2}-1} \exp\{-\frac{x}{2}\}$
$\mathcal{N}(\mu, \sigma^2)$	<code>rnorm(n,mean=mu,sd=sigma)</code>	$\frac{1}{\sqrt{2\pi}\sigma} \exp\{-\frac{(x-\mu)^2}{2\sigma^2}\}$
$\mathcal{W}(a, b)$	<code>rweibull(n,shape=a,scale=b)</code>	$\frac{a}{b} \left(\frac{x}{b}\right)^{a-1} \exp\{-\left(\frac{x}{b}\right)^a\}$
$\mathcal{LN}(\mu, \sigma)$	<code>rlognorm(n,meanlog=mu,sdlog=sigma)</code>	$\frac{1}{x\sigma\sqrt{2\pi}} \exp\{-\frac{(\log x - \mu)^2}{2\sigma^2}\}$
$\mathcal{T}(\nu)$	<code>rt(n,df=nu,ncp=0)</code>	$\frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\nu/2)} [1 + \frac{x^2}{\nu}]^{-\frac{\nu+1}{2}}$

2.8 Simulating multivariate distributions

This part dealing with simulating from multivariate distributions with widespread use in applications. These include Gaussian, Wishart, and Students t .

2.8.1 Gaussian distribution

Let random vector $\mathbf{X} = (X_1, \dots, X_p)^\top$ follows $\mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$. Each Gaussian distribution is characterized by its mean (location) vector $\boldsymbol{\mu} = (\mu_1, \dots, \mu_p)^\top$ and covariance (scale) matrix Σ . The PDF of $\mathbf{X}$ is shown by $\mathcal{N}_p(\cdot | \boldsymbol{\mu}, \Sigma)$ that is

$$\mathcal{N}_p(\mathbf{x} | \boldsymbol{\mu}, \Sigma) = \frac{1}{C_G} \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\}, \quad (2.67)$$

where

$$C_G = (2\pi)^{\frac{p}{2}} |\Sigma|^{\frac{1}{2}}, \quad (2.68)$$

$$\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma) = (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}). \quad (2.69)$$

The quantity C_G is the normalizing constant and expression (4.29) is the well-known *Mahalanobis* distance. We write $\mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$ to show that random vector $\mathbf{X}$ follows a p -dimensional

Gaussian distribution with location vector $\boldsymbol{\mu}$ and covariance matrix Σ whose PDF is given by (2.67). The structure of Σ may be shown as

$$\Sigma = \begin{pmatrix} \Sigma_{1,1} & \Sigma_{1,2} & \cdots & \Sigma_{1,p} \\ \Sigma_{2,1} & \Sigma_{2,2} & \cdots & \Sigma_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{p,1} & \Sigma_{p,2} & \cdots & \Sigma_{p,p} \end{pmatrix} = \begin{pmatrix} \text{var}(X_1) & \text{cov}(X_1, X_2) & \cdots & \text{cov}(X_1, X_p) \\ \text{cov}(X_2, X_1) & \text{var}(X_2) & \cdots & \text{cov}(X_2, X_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(X_p, X_1) & \text{cov}(X_p, X_2) & \cdots & \text{var}(X_p) \end{pmatrix}. \quad (2.70)$$

The matrix Σ is a symmetric and positive definite matrix and sometimes is called the variance-covariance matrix. This is because the diagonal elements of Σ are variances, that is $\text{var}(X_i) = \Sigma_{i,i}$, and its (i, j) th off-diagonal element represents the linear association between marginal variables X_i and X_j , that is $\text{cov}(X_i, X_j) = \Sigma_{i,j}$.

2.8.2 Properties of Gaussian distribution

Let $\mathbf{X} \sim \mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$, herein, we mention three interesting properties of Gaussian distribution.

- i. Both of expected value and variance of $\mathbf{X}$ are finite and given by $E(\mathbf{X}) = \boldsymbol{\mu}$ and $\text{var}(\mathbf{X}) = \Sigma$. The ML estimator of $\boldsymbol{\mu}$ and Σ , based on sample $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ is obtained readily as

$$\begin{aligned} \hat{\boldsymbol{\mu}}_{ML} &= \bar{\mathbf{X}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i, \\ \hat{\Sigma}_{ML} &= S = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top, \end{aligned} \quad (2.71)$$

respectively.

- ii. For a given $d \times p$ constant matrix such as A and constant vector $\mathbf{c}$ of length d , we have

$$\mathbf{c} + A\mathbf{X} \sim \mathcal{N}_d(\mathbf{c} + A\boldsymbol{\mu}, A\Sigma A^\top). \quad (2.72)$$

- iii. Let for given constant $1 \leq q < p$, the random vector $\mathbf{X}$ is represented as

$$\mathbf{X} = \begin{pmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \end{pmatrix}, \quad (2.73)$$

in which $\mathbf{X}_1 = (X_1, \dots, X_q)^\top$ and $\mathbf{X}_2 = (X_{q+1}, \dots, X_p)^\top$. The mean and scale parameters correspondingly can be represented as

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{pmatrix} = \begin{pmatrix} E(\mathbf{X}_1) \\ E(\mathbf{X}_2) \end{pmatrix}, \quad (2.74)$$

and

$$\begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix} = \begin{pmatrix} \Sigma_{1,1} & \cdots & \Sigma_{1,q} & \Sigma_{1,q+1} & \cdots & \Sigma_{1,p} \\ \Sigma_{2,1} & \cdots & \Sigma_{2,q} & \Sigma_{2,q+1} & \cdots & \Sigma_{2,p} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \Sigma_{q,1} & \cdots & \Sigma_{q,q} & \Sigma_{q,q+1} & \cdots & \Sigma_{q,p} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \Sigma_{p,1} & \cdots & \Sigma_{p,q} & \Sigma_{p,q+1} & \cdots & \Sigma_{p,p} \end{pmatrix}, \quad (2.75)$$

where $\Sigma_{11} = \text{var}(\mathbf{X}_1)$, $\Sigma_{12} = \Sigma_{21} = \text{cov}(\mathbf{X}_1, \mathbf{X}_2)$ and $\Sigma_{22} = \text{var}(\mathbf{X}_2)$. It can be shown that the conditional distribution $\mathbf{X}_1$ given $\mathbf{X}_2 = \mathbf{x}_2$ is $\mathcal{N}_q(\boldsymbol{\mu}_{\mathbf{X}_1|\mathbf{X}_2}, \Sigma_{\mathbf{X}_1|\mathbf{X}_2})$ where

$$\begin{aligned} \boldsymbol{\mu}_{\mathbf{X}_1|\mathbf{X}_2} &= \boldsymbol{\mu}_1 + \Sigma_{12}(\Sigma_{22})^{-1}(\mathbf{x}_2 - \boldsymbol{\mu}_2), \\ \Sigma_{\mathbf{X}_1|\mathbf{X}_2} &= \Sigma_{11} - \Sigma_{12}(\Sigma_{22})^{-1}\Sigma_{12}^\top. \end{aligned} \quad (2.76)$$

- iv. Sub-vector $\mathbf{X}_1 = (X_1, \dots, X_q)^\top$, for $q = 1, \dots, p$, follows $\mathcal{N}_q(\boldsymbol{\mu}_1, \Sigma_{11})$.

- v. If $\text{cov}(X_i, X_j) = 0$, for $i \neq j = 1, \dots, p$, then X_i and X_j are independent.

2.8.3 Generation from Gaussian distribution

Several methods have been introduced for sampling from Gaussian distribution in the literature. Herein, we represent the most three commonly used approaches that are based on decomposition (factorization) of scale matrix Σ .

- i. **Cholesky decomposition:** The first approach is based on the Cholesky decomposition of scale matrix Σ . If lower triangular matrix L denotes the Cholesky decomposition of Σ , then we can write $\Sigma = LL^\top$. Furthermore, suppose $\mathbf{Z} = (Z_1, \dots, Z_p)^\top$ is a vector p independent univariate standard Gaussian variates. Using property (2.72) while setting $\mathbf{c} = \boldsymbol{\mu}$, $A = L$, and considering the fact that $\mathbf{Z} \sim \mathcal{N}_p(\mathbf{0}, \mathbf{I}_p)$ in which $\mathbf{I}_p$ denotes a $p \times p$ identity matrix, we can easily see that

$$\boldsymbol{\mu} + L\mathbf{Z} \sim \mathcal{N}_p(\boldsymbol{\mu}, \Sigma). \quad (2.77)$$

- ii. **spectral decomposition:** The second way relies on the spectral or eigenvalue decomposition of Σ . Let $\{\lambda_1, \dots, \lambda_p\}$ is sequence of eigenvalues of Σ and columns of $p \times p$ matrix V are associated eigenvectors. It is well known that Σ can be represented as

$$\begin{aligned} \Sigma &= V \begin{pmatrix} \lambda_1 & 0 & 0 & \cdots & 0 \\ 0 & \lambda_2 & 0 & \cdots & 0 \\ 0 & 0 & \lambda_3 & & 0 \\ \vdots & \vdots & & \ddots & \\ 0 & 0 & 0 & & \lambda_p \end{pmatrix} V^{-1} \\ &= V D V^\top, \end{aligned} \quad (2.78)$$

where D is a $p \times p$ diagonal matrix whose positive diagonal elements are eigenvalues of Σ . Furthermore, it should be noted that

- (a) Since V is orthonormal then we have $V^{-1} = V^\top$.
- (b) Usually the software represents the eigenvalues in ascending order meaning that $\lambda_1 > \lambda_2 > \dots > \lambda_p > 0$.
- (c) $\Sigma^n = V D^n V^\top$, for $n \in \mathbb{N}$.
- (d) $\Sigma^{-1} = V D^{-1} V^\top$.
- (e)

$$D^{-1} = \begin{pmatrix} \frac{1}{\lambda_1} & 0 & 0 & \cdots & 0 \\ 0 & \frac{1}{\lambda_2} & 0 & \cdots & 0 \\ 0 & 0 & \frac{1}{\lambda_3} & & 0 \\ \vdots & \vdots & & \ddots & \\ 0 & 0 & 0 & & \frac{1}{\lambda_p} \end{pmatrix}.$$

Using first property given above that states $V^{-1} = V^\top$, it is easy to check that $\Sigma = U U^\top$ where $U = V D^{1/2}$. So recalling property (ii) given in Sub-section 2.72 while choosing $\mathbf{c} = \boldsymbol{\mu}$, $A = U$, and $\mathbf{Z} \sim \mathcal{N}_p(\mathbf{0}, \mathbf{I}_p)$, it turns out that

$$\boldsymbol{\mu} + V D^{\frac{1}{2}} \mathbf{Z} \sim \mathcal{N}_p(\boldsymbol{\mu}, \Sigma). \quad (2.79)$$

In what follows, a simple the R code is given for simulating from p -dimensional Gaussian distribution through the Cholesky decomposition.

```

1 R> rmvnorm <- function(n, mean = Mu, scale = Sigma)
2 +{
3 + p <- length(Mu)
4 + L <- t( chol(Sigma) )
5 + Z <- matrix( rnorm(n*p), nrow = p, ncol = n )
6 + L <- t( chol(Sigma) )
7 + X <- matrix(Mu, nrow = p, ncol = 1)%*%rep(1, n) + L %*%Z
8 + t(X)
9 +}

```

There are several R packages that can be used for simulating from multivariate Gaussian distribution.

3

Importance Sampling

The Monte Carlo techniques essentially can be seen as a tool for computing single or multiple integrals. This feature of the Monte Carlo techniques that focuses on computing integrals is called *importance* sampling. It is worth noting that the *importance* sampling aims to compute the expected value of an estimator that is typically computing an integral. This concern of this chapter is placed on computing the single and multiple integrals that are widely used in statistical analyses.

3.1 Importance sampling

Let $h(x)$ to be a continuous real function and $f(x)$ is a given PDF. Suppose we would like to compute the expectation

$$E_f(h(X)) = \int_{\mathbb{R}} h(x)f(x)dx, \quad (3.1)$$

where the generic symbol $E_f(h(X))$ indicates that the expectation of random variable $h(X)$ is computed when X follows the PDF f . Usually, computing (3.1) directly is not a simple task and must be evaluated numerically. The importance sampling is a tool to get round this difficulty. To this end, notice that the integral (3.1) can be rewritten as

$$E_f(h(X)) = \int_{\mathbb{R}} h(y) \frac{f(y)}{g(y)} g(y) dy = E_g\left(h(X) \frac{g(X)}{f(X)}\right) = E_g(h(X)w(X)), \quad (3.2)$$

where $w(u) = f(u)/g(u)$. Herein, the PDF $g(\cdot)$ is called the importance or instrumental PDF and its support dominates that of the PDF $f(x)$, that is $\mathcal{S}_f \subset \mathcal{S}_g$. By the law of large numbers, the RHS of (3.2) is estimated as

$$\hat{\mu} = E_f(\widehat{h(X)}) = \frac{1}{n} \sum_{i=1}^n w(x_i)h(x_i), \quad (3.3)$$

where $x_1, \dots, x_n$ come independently from PDF $g(\cdot)$. Hence, by generating a large number of realizations from PDF $g(\cdot)$ the preceding expectation is approximated well. for a wide ranges of applications the target PDF $f(\cdot)$ is just known up to a normalization constant. In such cases, estimator of $E_f(h(X))$ is given by

$$\tilde{\mu} = E_f(\widetilde{h(X)}) = \frac{\sum_{i=1}^n w(x_i)h(x_i)}{\sum_{i=1}^n w(x_i)}, \quad (3.4)$$

that is self-normalized. The bias and variance of both estimators $\hat{\mu}$ and $\tilde{\mu}$ can be evaluated [Liu and Liu, 2001]. We have

$$E_g(\hat{\mu}) = \mu \quad (3.5)$$

$$E_g(\tilde{\mu}) \approx \mu + \frac{\mu \times \text{var}_g(w(X)) - \text{cov}_g(w(X), w(X)h(X))}{n} \quad (3.6)$$

and

$$\text{var}_g(\hat{\mu}) = \frac{\text{var}_g(w(X)h(X))}{n} \quad (3.7)$$

$$\text{var}_g(\tilde{\mu}) \approx \frac{\text{var}_g(w(X)h(X)) - 2\mu \times \text{cov}_g(w(X), w(X)h(X)) + \mu^2 \times \text{var}_g(w(X))}{n} \quad (3.8)$$

It is clear from (3.5) and (3.6) that $\hat{\mu}$ is unbiased estimator for μ while the estimator $\tilde{\mu}$ is biased. Of course, in some situations the variance (standard error) of the self-normalized estimator $\tilde{\mu}$, as it is seen from RHS of (3.7), can be smaller than that of $\hat{\mu}$ that is given by (3.8). The main feature in each importance sampling problem is to determine the instrumental PDF $g(\cdot)$. Although, theoretically, there are many choices of $g(\cdot)$, but we have to find the best choice that yields as small as possible standard error for $\hat{\mu}$ (or $\tilde{\mu}$). Fortunately, there is a specific rule available for this purpose. It is proved that the optimal choice $g_o(x)$ becomes [Johansen et al., 2010]:

$$g_o(x) = \frac{|h(x)|f(x)}{\int_{\mathbb{R}} |h(u)|f(u)du}. \quad (3.9)$$

Example 3-1

Suppose we are interested in computing $\Omega = E(\log(1+X))$ where $X \sim \mathcal{G}(\alpha, \beta)$. We may choose three instrumentals as: 1- exponential distribution with rate parameter β , say $\mathcal{E}(\beta)$, 2- gamma distribution, say $\mathcal{G}(\alpha, \beta)$, and the optimal one with PDF $g_0(x) = h(x)\mathcal{G}(x|\alpha, \beta)$. Under the first choice, we have

$$\hat{\Omega} = \frac{\beta^{\alpha-1}}{\Gamma(\alpha)} \frac{1}{N} \sum_{i=1}^N x_i^{\alpha-1} \log(x_i + 1) \quad (3.10)$$

where x_i s come independently from $\mathcal{E}(\beta)$. In the case of second choice of instrumental, we have

$$\hat{\Omega} = \frac{1}{N} \sum_{i=1}^N x_i, \quad (3.11)$$

where x_i s come independently $\mathcal{G}(\alpha, \beta)$. In the case of $g_o(x)$ and $\mathcal{SG}(0, 1, \lambda)$, respectively. and second choices of instrumental, we have

$$\hat{\Omega} = N \left[\sum_{i=1}^N \frac{1}{\log(x_i + 1)} \right]^{-1}, \quad (3.12)$$

where x_i s come independently from PDF given by (3.13).

$$g_o(x) = \frac{\log(x+1)\mathcal{G}(x|\alpha, \beta)}{\int_0^\infty \log(u+1)\mathcal{G}(u|\alpha, \beta)du}, \quad (3.13)$$

For sampling from $g_o(x)$ given above, we designate a rejection scheme given by the following steps.

1. Sample realization y from $\mathcal{G}(\alpha + 1, \beta)$ as the proposal distribution;
2. Sample independent realization u from $\mathcal{U}(0, 1)$;
3. if $y < \log(y+1)/y$, then accept y as a generation from PDF $g_o(x)$ given by (3.13); otherwise return to step (1) and repeat this procedure until having acceptance.

The R function `rlgamma`, given below, generates sample from $g_o(x)$ based on above rejection scheme.

```

1  R> rlgamma <- function(N, alpha, beta)
2  + {
3  +   g <- rep(0, N) # vector of generated realizations
4  +   j <- 1
5  +   while(j <= N)
6  +   {
7  +     y <- rgamma(1, shape = alpha + 1, rate = beta)
8  +     u <- runif(1)
9  +     A <- log(y+1)/(y)
10 +     if( u < A )
11 +     {
12 +       g[j] <- y # y is accepted
13 +       j <- j + 1
14 +     }
15 +   }
16 +   return(g)
17 + }

```

Table 3.1 represents the results of simulation study based on $M = 5000$ runs of samples each of size $N = 5000$. It follows from this table that the second instrumental, that is $\mathcal{SG}(0, 1, \lambda)$, gives the best summary. Not surprisingly, the optimal instrumental does not work as well as or better than others since $\text{var}_{g_o}(W(X))$ is inflated by the small values in the generated data from this instrumental. It is worth noting that the $g_o(\cdot)$ works well if $W(x) = f(x)/g_o(x)$ is bounded. For our case, this condition may fail since $W(x) = f(x)/g_o(x) = 1/|x|$ becomes large in presence of even a few number of small realizations. ■

Table 3.1: Summary statistics for computing $\Omega = E(\log(1 + X))$ where $X \sim \mathcal{G}(\alpha, \beta)$ through three instrumentals.

(α, β)	Ω	instrumental	summary			
			bias	SE	minimum	maximum
(0.5, 0.5)	0.6695	$\mathcal{E}(\beta)$	1.1e-04	5.4e-05	0.5047	0.5611
		$\mathcal{G}(\alpha, \beta)$	-4.9e-05	3.4e-06	0.5271	0.5404
		g_o	6.2e-03	1.9e-03	0.0649	0.6249
(0.5, 5)	0.0884	$\mathcal{E}(\beta)$	-4.8e-05	-4.9e-05	0.0830	0.0948
		$\mathcal{G}(\alpha, \beta)$	-5.9e-07	-5.9e-07	0.0864	0.0902
		g_o	1.4e-03	1.5e-03	0.0066	0.1076
(5, 0.5)	2.3152	$\mathcal{E}(\beta)$	1.2e-04	3.4e-05	2.2940	2.3356
		$\mathcal{G}(\alpha, \beta)$	-4.1e-03	1.2e-01	1.4422	6.0980
		g_o	-2.9e-05	-4.1e-05	2.2926	2.3363
(5, 5)	0.6695	$\mathcal{E}(\beta)$	-1.8e-05	3.4e-05	0.6586	0.6814
		$\mathcal{G}(\alpha, \beta)$	1.1e-03	1.1e-01	0.4109	1.5190
		g_o	1.2e-05	4.1e-05	0.6562	0.6848

Next example shows that if above conditions are not met, then the importance sampling using the optimal choice $g_o(\cdot)$ may fail to show the best performance.

Example 3-2

Suppose we would like to compute $\Omega = E(|X|)$ where $X \sim \mathcal{SG}(0, 1, \lambda)$ and expression of $\mathcal{SG}(x|0, 1, \lambda)$ is given by (2.46). To this end, we consider three instrumentals including: $\mathcal{N}(0, 1)$, $\mathcal{SG}(0, 1, \lambda)$, and $g_o(x)$. The PDF of the latter is given by

$$g_o(x) = \frac{2|x|\phi(x)\Phi(\lambda x)}{\int_{-\infty}^{\infty} 2|u|\phi(u)\Phi(\lambda u)du}, \quad (3.14)$$

where quantity c plays the role of normalizing constant. But, first, we need to simulate from $g_o(x)$ given by (3.14). In what follows we give two methods for this purpose without proof.

1. Sample independent realizations z_1 and z_2 from $\mathcal{N}(0, 1)$;
2. Sample independent realization $u \sim \mathcal{U}(0, 1)$;
3. if $z_1 < \lambda z_2$ and $4u < |z_2|$, then accept z_2 as a generation from PDF (3.14); otherwise return to step (1) and repeat this procedure until having acceptance.

The second method is as follows.

1. Sample realization $w \sim \mathcal{W}(2, 1/\sqrt{2})$;
2. Sample independent realization $u \sim \mathcal{U}(0, 1)$;
3. If $u < \Phi(\lambda w)$, then accept w as a generation from PDF (3.14); otherwise accept $-w$ as a generation from PDF (3.14).

Evidently, the second algorithm is more efficient than the first one in terms of the time of implementing. For computing $\hat{\Omega}$, we consider under the first choice of instrumental, we have

$$\hat{\Omega} = \frac{2}{N} \sum_{i=1}^N |x_i| \Phi(\lambda x_i), \quad (3.15)$$

where x_i s come independently from $\mathcal{N}(0, 1)$. In the case of second choice of instrumental, we have

$$\hat{\Omega} = \frac{1}{N} \sum_{i=1}^N x_i, \quad (3.16)$$

where x_i s come independently $\mathcal{SG}(0, 1, \lambda)$, respectively. In the case of $g_o(x)$ and $\mathcal{SG}(0, 1, \lambda)$, respectively. and second choices of instrumental, we have

$$\hat{\Omega} = N \left[\sum_{i=1}^N \frac{1}{|x_i|} \right]^{-1}, \quad (3.17)$$

where x_i s come independently from PDF given by (3.14). Table represents the results of simulation study based on 5000 runs of samples each of size $N = 5000$. As it may be seen from Table 3.2, overall, the second instrumental, that is $\mathcal{SG}(0, 1, \lambda)$, gives the best summary. Not surprisingly, the optimal instrumental does not work as well as or better than others since $\text{var}_{g_o}(W(X))$ is inflated by the small values in the generated data from this instrumental. It is worth noting that the $g_o(\cdot)$ works well if $W(x) = f(x)/g_o(x)$ is bounded. For our case, this condition may fail since $W(x) = f(x)/g_o(x) = 1/|x|$ becomes large in presence of even a few number of small realizations. ■

3.2 Variance reduction

Suppose $\hat{\theta}$ is an unbiased candidate (estimator) for estimating unknown parameter θ . This means that $E(\hat{\theta}) = \theta$. For a better unbiased estimator $\hat{\theta}^*$, if there exists, we have $\text{var}(\hat{\theta}^*) < \text{var}(\hat{\theta})$. Here, we are interested in finding $\hat{\theta}^*$. To this end, we recall the well-known Rao-Blackwellization approach that states the standard error of an given biased (or unbiased) estimator $\hat{\theta}$ can be reduced by conditioning on a *sufficient* statistics (estimator). Overall, we can write

$$\text{var}(\hat{\theta}) = E[\text{var}(\hat{\theta}|T)] + \text{var}[E(\hat{\theta}|T)]. \quad (3.18)$$

Table 3.2: Summary statistics for computing $\Omega = E(|X|)$ where $X \sim \mathcal{SG}(0, 1, \lambda)$ through three instrumentals.

λ	instrumental	summary			
		bias	SE	minimum	maximum
$\lambda = 0$	$\mathcal{N}(0, 1)$	5.1e-05	0.0083	0.7688	0.8250
	$\mathcal{SG}(0, 1, \lambda = 0)$	3.8e-05	0.0084	0.7663	0.8259
	$g_o(\cdot)$	1.0e-03	0.0218	0.4925	0.8574
$\lambda = 5$	$\mathcal{N}(0, 1)$	4.3e-04	0.0168	0.7267	0.8560
	$\mathcal{SG}(0, 1, \lambda = 5)$	3.5e-05	0.0084	0.7693	0.8290
	$g_o(\cdot)$	1.1e-03	0.0214	0.5341	0.8573
$\lambda = 10$	$\mathcal{N}(0, 1)$	7.2e-05	0.0165	0.7367	0.8553
	$\mathcal{SG}(0, 1, \lambda = 10)$	2.9e-05	0.0084	0.7683	0.8311
	$g_o(\cdot)$	6.9e-04	0.0221	0.2998	0.8533

This implies

$$\text{var}[E(\hat{\theta}|T)] < \text{var}(\hat{\theta}). \quad (3.19)$$

Hence, $\hat{\theta}^* = E(\hat{\theta}|T)$ is an unbiased estimator whose standard error is smaller than that of the $\hat{\theta}$.

Example 3-3

Suppose $X_1, X_2 \sim \mathcal{N}(\mu, 1)$ are independent random variables. It is known that $T = X_1 + X_2 \sim \mathcal{N}(2\mu, 2)$ is a sufficient statistic for unknown parameter μ . Moreover, we know that $\hat{\theta} = X_1^2 - 1$ is an unbiased estimator for μ^2 with variance 1. But, an estimator of μ^2 with smaller standard error, called here $\hat{\theta}^*$, can be found using Rao-Blackwellization procedure as follows. First, we can see that the PDF of X_1 given $T = X_1 + X_2$ is

$$\begin{aligned} f_{X_1|T}(x_1|T=t) &= \frac{f_{X_1, X_2}(x_1, t-x_1)}{f_T(t)} \\ &= \frac{f_{X_1}(x_1)f_{X_2}(x_1, t-x_1)}{f_T(t)} \\ &= \frac{\sqrt{2}}{\sqrt{2\pi}} \exp\{-(x_1 - t/2)^2\}. \end{aligned}$$

So, $X_1|X_1 + X_2 = t \sim \mathcal{N}(t/2, 1/2)$. It follows that

$$\begin{aligned} \hat{\theta}^* &= E[\hat{\theta}|T=t] \\ &= E[X_1^2|X_1 + X_2 = t] - 1 \\ &= \text{var}[X_1|X_1 + X_2 = t] + \{E[X_1|X_1 + X_2 = t]\}^2 - 1 \\ &= \frac{1}{2} + \frac{t^2}{4} - 1 \\ &= \bar{X}^2 - \frac{1}{2}. \end{aligned}$$

It is evident that both estimators $\hat{\theta}$ and $\hat{\theta}^*$ are unbiased for μ^2 , that is $E(\hat{\theta}) = E(\hat{\theta}^*) = \mu^2$, but $\text{var}(\hat{\theta}^*) < \text{var}(\hat{\theta})$. ■

3.3 Estimators with negative covariance

Suppose both of estimators $\hat{\theta}_1$ and $\hat{\theta}_2$ are unbiased for unknown parameter θ . If $cov(\hat{\theta}_1, \hat{\theta}_2) < 0$, then $\hat{\theta}^* = [\hat{\theta}_1 + \hat{\theta}_2]/2$ is also unbiased estimator for θ with smaller standard error than either $\hat{\theta}_1$ or $\hat{\theta}_2$. This because

$$\begin{aligned} var(\hat{\theta}^*) &= \frac{var(\hat{\theta}_1) + var(\hat{\theta}_2)}{4} + \frac{cov(\hat{\theta}_1, \hat{\theta}_2)}{2} \\ &\leq \frac{var(\hat{\theta}_1) + var(\hat{\theta}_2)}{4} \\ &< \frac{1}{2} \max\{var(\hat{\theta}_1), var(\hat{\theta}_2)\}. \end{aligned}$$

Example 3-4

Let random variable X follows a BS distribution with PDF given by (2.41) in which parameter β is known. It can be shown that Ng et al. [2003]:

$$E(X) = \beta \left(1 + \frac{\alpha^2}{2}\right).$$

Since $1/X \sim \mathcal{BS}(\alpha, 1/\beta)$, we can write

$$E\left(\frac{1}{X}\right) = \frac{1}{\beta} \left(1 + \frac{\alpha^2}{2}\right).$$

Based on a random sample of size n denoted by $x_1, \dots, x_n$ drawn from $\mathcal{BS}(\alpha, 1)$, one can see that $\hat{\theta}_1 = 2(S - 1)$ and $\hat{\theta}_2 = 2(R - 1)$ are unbiased estimators for α^2 in which $S = 1/n \sum_{i=1}^n x_i$ and $R = 1/n \sum_{i=1}^n 1/x_i$. Intuitively, the statistics S and R have negative correlation, and hence, we can construct a new estimator represented as

$$\hat{\theta}^* = \frac{\hat{\theta}_1 + \hat{\theta}_2}{2},$$

whose standard error is less than that of $\hat{\theta}_1$ and $\hat{\theta}_2$. We performed a small comparison study in which four estimators including $\hat{\theta}_{MLE}$ (ML estimator of α^2), $\hat{\theta}^*$, $\hat{\theta}_1$, and $\hat{\theta}_2$ take part for estimating α^2 when β is assumed to be known. To this end, aforementioned four competitors have been computed for based on random sample of size $n = 100$ for $N = 1000$ runs. It is worth to note that for simulating for BS distribution we used the R function `rbs` given in 2.3.7 and the ML estimator of parameter α^2 is computed using package `bibs` available at <https://cran.r-project.org/web/packages/bibs/index.html>, then the $\hat{\theta}_{MLE}$ is easily obtained as the squared ML estimator of α by the general nice invariance property of the ML estimator. Figure 3.1 shows the average and standard error of these competitors. As it is seen, in terms of average and standard error, both estimators $\hat{\theta}_{MLE}$ and $\hat{\theta}^*$ shows the same performance and simultaneously outperform $\hat{\theta}_1$ and $\hat{\theta}_2$. The corresponding R code is given by the following.

```
1 R> library("bibs")
2 R> N <- 1000; n <- 100; alpha <- seq(0.2, 5, length = 10); beta <- 1
3 R> r <- s <- alpha2_mle <- alpha2_star <- alpha2_hat1 <- alpha2_hat2 <- rep(0, N)
4 R> bias <- rmse <- matrix(0, nrow = length(alpha), ncol = 4 )
5 R> for(j in 1:length(alpha)) {
6 +   for(i in 1:N) {
7 +     x <- rbs(n, alpha[j], beta); s[i] <- mean(x); r[i] <- mean(1/x)
```

```

8 + alpha2_mle[i] <- mlebs(x, c(alpha[j], beta), method = "Nelder-Mead", CI = 0.95)$Estimates
  [[1]]
9 + alpha2_hat1[i] <- ( 2*(r[i]*beta - 1) ); alpha2_hat2[i] <- ( 2*(s[i]/beta - 1) )
10 + alpha2_star[i] <- ( alpha2_hat1[i] + alpha2_hat2[i] )/2
11 + }
12 + hat <- as.matrix( cbind(alpha2_mle^2, alpha2_star, alpha2_hat1, alpha2_hat2), nrow = N,
  ncol = 4 )
13 + shifted <- sweep(hat, 2, alpha[j]^2, FUN = '-')
14 + bias[j, ] <- apply( shifted, 2, mean )
15 + rmse[j, ] <- sqrt( apply( shifted, 2, var ) )
16 + }

```

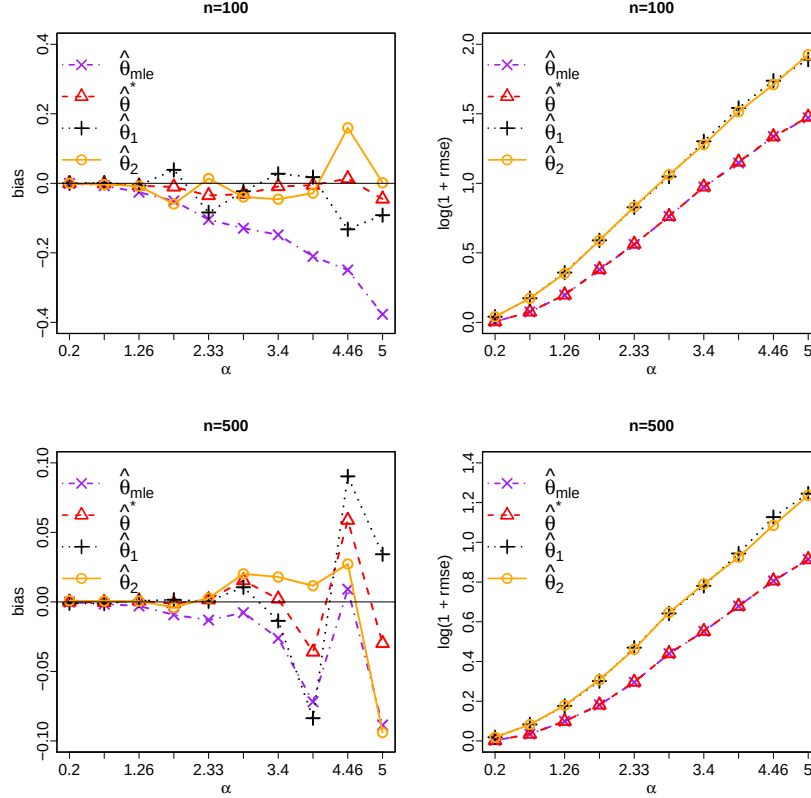


Figure 3.1: The top row shows the bias and $\log(1+\text{rmse})$ for $\hat{\theta}_{MLE}$, $\hat{\theta}^*$, $\hat{\theta}_1$, and $\hat{\theta}_2$ computed based on $N = 1000$ independent sample each of size $n = 100$ generated from $\mathcal{BS}(\alpha, \beta = 1)$. The bottom row displays the same graph with the same parameters except $n = 500$.

Example 3-5

Let $U, V \sim \mathcal{U}(-1, 1)$ are independent random variables distributed jointly on $(-1, 1) \times (-1, 1)$ (shown by a square in Figure 3.2). It may be seen that the probability of falling a point inside the circle in Figure 3.2 is $\pi/4$. Hence, the constant π is approximately as

$$\hat{\pi} = 4 \times \frac{\text{number of pairs lie inside the circle}}{\text{number of pairs do not lie inside the circle}}. \quad (3.20)$$

Therefore, the constant π can be estimated by simulating a sufficiently large (n , say) numbers of point distributed uniformly on $(-1, 1) \times (-1, 1)$ and then computing the ratio 3.20. To this end, we define

$$\mathbb{I}_C(u_i, v_i) = \begin{cases} 1, & \text{if } i\text{th pair } (u_i, v_i)^\top \text{ lies inside the circle,} \\ 0, & \text{if } i\text{th pair } (u_i, v_i)^\top \text{ lies outside the circle,} \end{cases} \quad (3.21)$$

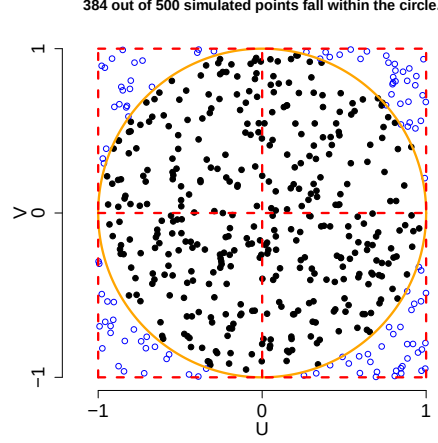


Figure 3.2: Components of random vector $(U, V)^\top$ come independently from $\mathcal{U}(-1, 1)$.

and then, based on the law of large numbers, we have

$$\hat{\pi} = 4 \times \frac{\sum_{i=1}^n \mathbb{I}_{\mathcal{C}}(u_i, v_i)}{n}. \quad (3.22)$$

It is easy to check that $E(\hat{\pi}) = \pi$ and $\text{var}(\hat{\pi}) = 4\pi(1 - \pi/4)/n = 2.69676/n$. Here, we are interested in improving estimator $\hat{\pi}$ by conditioning on marginal variable U as follows.

$$\begin{aligned} \hat{\pi}_1 &= \frac{4}{n} \sum_{i=1}^n E[\mathbb{I}_{\mathcal{C}}(U_i, V_i) | U_i = u_i] \\ &= \frac{4}{n} \sum_{i=1}^n P[U_i^2 + V_i^2 \leq 1 | U_i = u_i] \\ &= \frac{4}{n} \sum_{i=1}^n P[V_i^2 \leq 1 - u_i^2 | U_i = u_i] \\ &= \frac{4}{n} \sum_{i=1}^n P[V_i^2 \leq 1 - u_i^2] \\ &= \frac{4}{n} \sum_{i=1}^n \sqrt{1 - u_i^2}, \end{aligned}$$

where $U_i \sim \mathcal{U}(-1, 1)$. This yields

$$\hat{\pi}_1 = \frac{4}{n} \sum_{i=1}^n \sqrt{1 - U_i^2}.$$

Therefore, $\hat{\pi}_1$ is obtained, first by generating $u_1, \dots, u_n \sim \mathcal{U}(-1, 1)$, and then computing $\hat{\pi}_1 = 4/n \sum_{i=1}^n \sqrt{1 - u_i^2}$. It is easy to check that

$$\begin{aligned} E(\hat{\pi}_1) &= \frac{4}{n} \sum_{i=1}^n E(\sqrt{1 - U_i^2}) & \text{var}(\hat{\pi}_1) &= \frac{16}{n^2} \sum_{i=1}^n \left[E(1 - Z_i^2) - \left(\frac{\pi}{4}\right)^2 \right], \\ &= \frac{4}{n} \sum_{i=1}^n \int_{-1}^1 \sqrt{1 - y^2} \frac{1}{2} dy & &= \frac{16}{n} \left[\frac{2}{3} - \left(\frac{\pi}{4}\right)^2 \right] \\ &= \frac{4}{n} \sum_{i=1}^n E(\sqrt{1 - Z_i^2}) & &= \frac{0.79706}{n}. \\ &= \pi, \end{aligned}$$

where $Z_1, \dots, Z_n \sim \mathcal{U}(0, 1)$. As it is seen, $\hat{\pi}_1$ yields around 70.44% reduction in variance for estimating π compared to $\hat{\pi}$. Yet the performance of estimator $\hat{\pi}_1$ can be improved using the method introduced in Section 3.3. It follows from above that quantity

$$\frac{4}{n} \sum_{i=1}^n \sqrt{1 - Z_i^2}$$

with variance $0.79706/n$. We know that $Z \sim 1 - Z \sim \mathcal{U}(0, 1)$ and so the quantity

$$\frac{4}{n} \sum_{i=1}^n \sqrt{1 - (1 - Z_i)^2}$$

is also unbiased estimator for π with the same variance. But, evidently Z and $1 - Z$ are correlated negatively and hence

$$\hat{\pi}_2 = \frac{2}{n} \sum_{i=1}^n \sqrt{1 - Z_i^2} + \frac{2}{n} \sum_{i=1}^n \sqrt{1 - (1 - Z_i)^2},$$

is an unbiased estimator for π whose variance is smaller than that of $\hat{\pi}_1$. As it may seen from

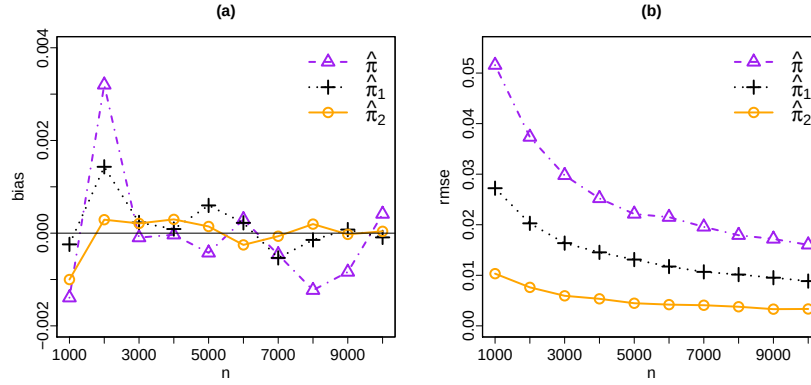


Figure 3.3: The bias (a) and rmse (b) of $\hat{\pi}$, $\hat{\pi}_1$, and $\hat{\pi}_2$ computed based on $N = 1000$ independent samples of size n .

Figure 3.3, the $\hat{\pi}_2$ outperforms $\hat{\pi}_1$ and $\hat{\pi}$, in terms of both bias and rmse criteria. The pertinent R code for computing the bias and rmse of $\hat{\pi}$, $\hat{\pi}_1$, and $\hat{\pi}_2$ is given by the following.

```

1  R> set.seed(20241012)
2  R> N <- 1000; n <- 100; sample<- seq(1000, 10000, 1000)
3  R> pi_hat <- pi_hat_star1 <- pi_hat_star2 <- rep(0, N)
4  R> bias <- rmse <- matrix(0, nrow = length(sample), ncol = 3 )
5  R> for(j in 1:length(sample))
6  + {
7  +   for(i in 1:N)
8  +   {
9  +     u <- runif(sample[j], -1, 1); v <- runif(sample[j], -1, 1); Z <- runif(sample[j], 0, 1)
10 +     pi_hat[i] <- 4*mean( ifelse(u^2 + v^2 <= 1, 1, 0) )
11 +     pi_hat_star1[i] <- 4*mean( sqrt(1 - u^2) )
12 +     pi_hat_star2[i] <- ( 4*mean( sqrt(1 - Z^2) ) + 4*mean( sqrt(1 - (1 - Z)^2) ) )/2
13 +   }
14 +   hat <- as.matrix( cbind(pi_hat, pi_hat_star1, pi_hat_star2), nrow = N, ncol = 3 )
15 +   colnames(hat) <- NULL
16 +   shifted <- sweep(hat, 2, pi, FUN = '-')
17 +   bias[j, ] <- apply( shifted, 2, mean )
18 +   rmse[j, ] <- sqrt( apply( shifted, 2, var ) )
19 + }

```



3.3.1 Computing the CDF of univariate Gaussian distribution

here, we proceed to provide the compute the Monte Carlo approximation for the CDF of the Gaussian distribution. Indeed computing the CDF of a Gaussian distribution is an importance sampling problem. Following example gives more detailed information.

Example 3-6

Let $Z \sim \mathcal{N}(0, 1)$ and we are interested in computing $P(Z < 2)$. Evidently the given probability can be represented as

$$\begin{aligned} P(Z < 2) &= \frac{1}{\sqrt{2\pi}} \int_{-\infty}^2 \exp\left\{-\frac{z^2}{2}\right\} dz \\ &= \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} \mathbb{I}_{(-\infty, 2)}(z) \times \exp\left\{-\frac{z^2}{2}\right\} dz \\ &= E\left(\mathbb{I}_{(-\infty, 2)}(Z)\right), \end{aligned} \tag{3.23}$$

where $\mathbb{I}_{\mathcal{S}}(x)$ is the well-known indicator function defined as

$$\mathbb{I}_{\mathcal{S}}(x) = \begin{cases} 1, & \text{if } x \in \mathcal{S}, \\ 0, & \text{if } x \notin \mathcal{S}. \end{cases} \tag{3.24}$$

Hence, the quantity can be readily approximated by considering the standard univariate Gaussian as the instrumental distribution. It follows, based on the law of large numbers, that

$$P(Z \leq 2) \approx \frac{1}{N} \sum_{i=1}^N \mathbb{I}_{(-\infty, 2)}(z_i) = \frac{\text{number of } z_i\text{s less or equal 2 in a sample of size } N}{N}, \tag{3.25}$$

where $z_1, \dots, z_N$ independently follow $\mathcal{N}(0, 1)$ and N is a sufficiently large integer value. This means that the Monte Carlo approximation of $P(Z < 2)$ given in (3.25) is the average of samples that are less than two. The Central limit theorem guarantees that estimator (3.25) is unbiased for $P(Z < 2)$ with variance

$$\frac{\Phi(2|0, 1)[1 - \Phi(2|0, 1)]}{N}.$$

■

The routines `integrate(f, lower, upper, ...)` (when `f` is the PDF of Gaussian distribution) and `pnorm(q, mean=0, sd=1, ...)` developed for R environment can be used for computing the CDF of Gaussian distribution. However, one could use the well known *Gaussian quadrature* rules discussed in next Chapter.

4

Gaussian Scale mixture models

In this chapter, we introduce the most commonly used family of univariate and multivariate distributions known in the literature as *Gaussian scale (variance) mixture* (GSM) distribution and *Gaussian variance-mean mixture* (GSVM) distribution. These two families incorporate a wide range of well-known distributions that are expressed in terms of the Gaussian distribution. Throughout this chapter, we will assume that $\eta(\cdot)$ is a positive real-valued function, $\Phi_p(\cdot|\boldsymbol{\mu}, \Sigma)$ denotes the CDF of a p -dimensional Gaussian distribution with location vector $\boldsymbol{\mu}$ and scale (covariance) matrix Σ denoted by $\mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$. Furthermore, $\phi_p(\cdot|\boldsymbol{\mu}, \Sigma)$ (or $\mathcal{N}_p(\cdot|\boldsymbol{\mu}, \Sigma)$) denotes the PDF of a Gaussian distribution with $\boldsymbol{\mu}$ and Σ described as above. In the univariate case, $\mathcal{N}(\mu, \sigma^2)$ accounts for family of Gaussian distributions with location parameter μ and variance σ^2 for which the CDF and PDF are denoted as $\Phi(\cdot|\mu, \Sigma^2)$ and $\phi(\cdot|\mu, \Sigma^2)$ (or $\mathcal{N}(\cdot|\mu, \sigma^2)$), respectively. Finally, for a positive definite scale matrix Σ , the lower triangular matrix Cholesky decomposition thereof is shown by $\Sigma^{1/2}$, that means $\Sigma = (\Sigma^{1/2})(\Sigma^{1/2})^\top$. In next subsection, we review three classes of the skewed Gaussian models.

4.0.1 Skewed Gaussian models

The class of skewed Gaussian distributions attracted much attention in many statistical applications. Notable among them are those of The SDB¹ Sahu et al. [2003], classical skewed Gaussian (CSG) Pyne et al. [2009], and canonical fundamental skewed Gaussian (CFUSG) Arellano-Valle and Genton [2005]. In what follows, we introduce these three well-known classes of skewed Gaussian distributions.

SDB model

Definition 4.0.1. (SDB model) A p -dimensional random vector $\mathbf{Y}$ is said to follow a SDB distribution, denoted as $\mathbf{Y} \sim \text{SDB}_p(\boldsymbol{\mu}, \Sigma, \Lambda)$, if its PDF is given by Sahu et al. [2003]:

$$f_{\mathbf{Y}}(\mathbf{y}|\boldsymbol{\mu}, \Sigma, \Lambda) = 2^p \phi_p(\mathbf{y}|\boldsymbol{\mu}, \Omega) \Phi_p(\Lambda \Omega^{-1}(\mathbf{y} - \boldsymbol{\mu})|\mathbf{0}, \Delta), \quad (4.1)$$

where $\Omega = \Sigma + \Lambda^2$ and $\Delta = \mathbf{I}_p - \Lambda \Omega^{-1} \Lambda$. Herein, $\boldsymbol{\mu}$ is location parameter, Σ is dispersion matrix, and Λ is $p \times p$ diagonal skewness matrix. We have

$$\mathbf{Y} \stackrel{d}{=} \boldsymbol{\mu} + \Lambda |\mathbf{Z}_0| + \Sigma^{\frac{1}{2}} \mathbf{Z}_1, \quad (4.2)$$

where $|\mathbf{Z}|^\top = (|Z_1|, \dots, |Z_p|)$ and $\mathbf{Z}_0 \sim \mathcal{N}_q(\mathbf{0}, \mathbf{I}_q)$ and $\mathbf{Z}_1 \sim \mathcal{N}_p(\mathbf{0}, \mathbf{I}_p)$ are independent.

¹The SDB refers to the author's initials

Classical skewed Gaussian (CSG) model

Definition 4.0.2. A p -dimensional random vector $\mathbf{Y}$ is said to follow a CSG distribution, denoted as $\mathbf{Y} \sim \text{CSG}_p(\boldsymbol{\mu}, \Sigma, \boldsymbol{\lambda})$, if its PDF is given by Pyne et al. [2009]:

$$f_{\mathbf{Y}}(\mathbf{y}|\boldsymbol{\mu}, \Sigma, \boldsymbol{\lambda}) = 2\phi_p(\mathbf{y}|\boldsymbol{\mu}, \Omega)\Phi(\boldsymbol{\lambda}^\top \Omega^{-1}(\mathbf{y} - \boldsymbol{\mu})|\mathbf{0}, \delta), \quad (4.3)$$

where $\Omega = \Sigma + \boldsymbol{\lambda}\boldsymbol{\lambda}^\top$ and $\delta = 1 - \boldsymbol{\lambda}^\top \Omega^{-1} \boldsymbol{\lambda}$. Herein, $\boldsymbol{\mu}$ is location parameter, Σ is dispersion matrix, and $\boldsymbol{\lambda}$ is the skewness vector of length p . We have

$$\mathbf{Y} \stackrel{d}{=} \boldsymbol{\mu} + \boldsymbol{\lambda}|Z_0| + \Sigma^{\frac{1}{2}}\mathbf{Z}_1, \quad (4.4)$$

where $Z_0 \sim \mathcal{N}(0, 1)$ and $\mathbf{Z}_1 \sim \mathcal{N}_p(\mathbf{0}, \mathbf{I}_p)$ are independent.

Canonical fundamental skewed Gaussian (CFUSG) model

Definition 4.0.3. A p -dimensional random vector $\mathbf{Y}$ follows a CFUSG distribution, denoted as $\mathbf{Y} \sim \text{CFUSG}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda)$, if its PDF is given by [Arellano-Valle and Genton, 2005]:

$$f(\mathbf{y}|\boldsymbol{\mu}, \Sigma, \Lambda) = 2^q \phi_p(\mathbf{y}|\boldsymbol{\mu}, \Omega) \Phi_q(\Lambda^\top \Omega^{-1}(\mathbf{y} - \boldsymbol{\mu})|\mathbf{0}, \Delta), \quad (4.5)$$

where $\Omega = \Sigma + \Lambda\Lambda^\top$, $\Delta = \mathbf{I}_q - \Lambda^\top \Omega^{-1} \Lambda$, and $\mathbf{I}_q$ denotes the $q \times q$ identity matrix. Moreover, random vector $\mathbf{Y}$ admits stochastic representation as

$$\mathbf{Y} \stackrel{d}{=} \boldsymbol{\mu} + \Lambda|\mathbf{Z}_0| + \Sigma^{\frac{1}{2}}\mathbf{Z}_1, \quad (4.6)$$

where $\mathbf{Z}_0 \sim \mathcal{N}_q(\mathbf{0}, \mathbf{I}_q)$ and $\mathbf{Z}_1 \sim \mathcal{N}_p(\mathbf{0}, \mathbf{I}_p)$ are independent.

The following R code can be used for simulating n realizations from CFUSG distribution.

```

1 R> rsfn <- function(n, mean = Mu, scale = Sigma, shape = Lambda)
2 + {
3 +   p <- length(Mu)
4 +   q <- length(Lambda[1, ])
5 +   Y <- matrix(NA, nrow = p, ncol = n)
6 +   Z1 <- rmvnorm( n, mean = rep(0, p), scale = Sigma )
7 +   Z0 <- abs( rmvnorm(n, mean = rep(0, q), scale = diag(q)) )
8 +   Y <- matrix(Mu, nrow = p, ncol = 1)%*%rep(1, n) + Lambda%*%t(Z0) + t(Z1)
9 +   t(Y)
10 + }
```

4.0.2 Gaussian scale/location-scale mixture models

In next two subsections, we introduce the general classes of Gaussian scale mixture models and Gaussian location-scale mixture models.

Definition 4.0.4. (univariate GSM) Let random variable Z follows a standard Gaussian distribution. For a given positive mixing random variable G with PDF $h(\cdot|\boldsymbol{\theta})$ in which $\boldsymbol{\theta}$ is the family parameter, the random variable Y with CDF $F(\cdot|\Psi)$ is said to follow a Gaussian scale mixture model if

$$Y = \mu + \sigma \frac{Z}{\sqrt{\eta(G)}}, \quad (4.7)$$

where $\Psi^\top = (\mu, \sigma, \boldsymbol{\theta}^\top)$ in which $\mu \in \mathbb{R}$ and $\sigma \in \mathbb{R}^+$ are the location and scale parameters of $F(\cdot|\Psi)$, respectively. Obviously identity (4.7) admits the following hierarchy

$$\begin{aligned} \mathbf{Y}|G = g &\sim \mathcal{N}\left(\mu, \frac{\sigma^2}{\eta(g)}\right), \\ G &\sim h(g|\boldsymbol{\theta}), \end{aligned} \quad (4.8)$$

or equivalently

$$F(y|\Psi) = \int_0^\infty \Phi\left[y|\mu, \frac{\sigma^2}{\eta(g)}\right] h(g|\boldsymbol{\theta}) dg, \quad (4.9)$$

Table 4.1 shows four known classes of the GSM models including the Student's t [Peel and McLachlan, 2000], Generalized Hyperbolic (GH) [Barndorff-Nielsen, 1978, McNeil et al., 2005], Exponential Power (EP) [Dang et al., 2015, Gómez et al., 1998, Teimouri and Nadarajah, 2022], and SGS distributions for which the mixing distributions are gamma, GIG [Good, 1953], polynomially tilted positive stable (PT) [Devroye, 2009], and positive stable, respectively.

Table 4.1: Examples of the GSM models.

class	$\boldsymbol{\theta}^\top$	mixing distribution	$\eta(g)$	GSM model
1	ν	$\mathcal{G}(\nu/2, \nu/2)$	g	Student's t
2	(ψ, χ, λ)	$\mathcal{GIG}(\psi, \chi, \lambda)$	g^{-1}	GH
3	α	$PT(\alpha, 1/2)$	$2g$	EP
4	α	PaS with CDF (2.31)	$2^{-1}g^{-1}$	SaS

Table 4.2: PDF of the GSM model in Table 4.1.

class	univariate case	multivariate case
1	$\frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\sigma\Gamma(\frac{\nu}{2})} \left[1 + \frac{(x-\mu)^2}{\nu\sigma^2}\right]^{-\frac{\nu+1}{2}}$	$\frac{\Gamma(\frac{\nu+p}{2})}{(\nu\pi)^{\frac{p}{2}} \Sigma ^{\frac{1}{2}}\Gamma(\frac{\nu}{2})} \left[1 + \frac{\delta(\mathbf{x}-\boldsymbol{\mu}, \Sigma)}{\nu}\right]^{-\frac{\nu+p}{2}}$
2	$\frac{1}{\sqrt{2\pi}\sigma} \left[\frac{\chi\sigma^2 + (x-\mu)^2}{\psi\sigma^2}\right]^{\frac{\lambda-1}{2}} \left(\frac{\psi}{\chi}\right)^{\frac{\lambda}{2}} \times \frac{\mathcal{K}_{\lambda-\frac{1}{2}}\left(\sqrt{\psi\left[\chi + \frac{(x-\mu)^2}{\sigma^2}\right]}\right)}{\mathcal{K}_\lambda(\sqrt{\psi\chi})}$	$\frac{1}{(2\pi)^{\frac{p}{2}} \Sigma ^{\frac{1}{2}}} \left[\frac{\chi + \delta(\mathbf{x}-\boldsymbol{\mu}, \Sigma)}{\psi}\right]^{\frac{\lambda-\frac{p}{2}}{2}} \left(\frac{\psi}{\chi}\right)^{\frac{\lambda}{2}} \times \frac{\mathcal{K}_{\lambda-\frac{p}{2}}\left(\sqrt{\psi[\chi + \delta(\mathbf{x}-\boldsymbol{\mu}, \Sigma)]}\right)}{\mathcal{K}_\lambda(\sqrt{\psi\chi})}$
3	$\frac{1}{2\sigma\Gamma(1+\frac{1}{\alpha})} \exp\left\{-\left \frac{x-\mu}{\sigma}\right ^\alpha\right\}$	$\frac{\Gamma(1+\frac{p}{2})}{(2\pi)^{\frac{p}{2}}\Gamma(1+\frac{p}{\alpha}) \Sigma ^{\frac{1}{2}}} \exp\left\{-\left[\frac{\delta(\mathbf{x}-\boldsymbol{\mu}, \Sigma)}{2}\right]^{\frac{\alpha}{2}}\right\}$
4	has not close form	has not close form

$\delta(\cdot, \cdot)$ is the Mahalanobis distance defined as (4.29).

Definition 4.0.5. (multivariate GSM) Let random vector $\mathbf{Z}$ follows a p -dimensional standard Gaussian distribution. For a given positive mixing random variable G with PDF $h(\cdot|\boldsymbol{\theta})$ in which $\boldsymbol{\theta}$ is the family parameter, the p -dimensional random vector $\mathbf{Y}$ with CDF $F(\cdot|\Psi)$ is said to follow a Gaussian scale mixture model if

$$\mathbf{Y} = \boldsymbol{\mu} + \Sigma^{\frac{1}{2}} \frac{\mathbf{Z}}{\sqrt{\eta(G)}}, \quad (4.10)$$

where $\Psi = (\boldsymbol{\mu}, \Sigma, \boldsymbol{\theta})$ in which $\boldsymbol{\mu} \in \mathbb{R}^p$ and $p \times p$ positive definite matrix Σ are the location and scale parameters of $F(\cdot|\Psi)$, respectively. Obviously identity (4.10) admits the following

hierarchy

$$\begin{aligned} \mathbf{Y}|G=g &\sim \mathcal{N}_p\left(\boldsymbol{\mu}, \frac{\boldsymbol{\Sigma}}{\eta(g)}\right), \\ G &\sim h(g|\boldsymbol{\theta}), \end{aligned} \quad (4.11)$$

or equivalently

$$F(\mathbf{y}|\Psi) = \int_0^\infty \Phi_p\left[\mathbf{y}|\boldsymbol{\mu}, \frac{\boldsymbol{\Sigma}}{\eta(g)}\right] h(g|\boldsymbol{\theta}) dg. \quad (4.12)$$

Corollary 4.0.1. *The i th, for $i = 1, \dots, p$, marginal of each GSM distribution in Definition 4.0.5 with $\boldsymbol{\Sigma} = (\sigma_{i,j})$ follows a univariate GSM distribution as given in Definition 4.0.4 for which $\sigma = \sqrt{\sigma_{i,i}}$.*

4.0.3 Gaussian mixture location-scale model

Definition 4.0.6. (univariate GLSM) *Let random variable Z follows a standard Gaussian distribution. For a given positive mixing random variable G with PDF $h(g|\boldsymbol{\theta})$ in which $\boldsymbol{\theta}$ is the family parameter, the random variable Y with CDF $F(\cdot|\Psi)$ is said to follow a Gaussian scale mixture model if*

$$Y = \mu + \frac{\lambda}{\sqrt{\eta(G)}} + \sigma \frac{Z}{\sqrt{\eta(G)}}, \quad (4.13)$$

where $\Psi^\top = (\mu, \lambda, \sigma, \boldsymbol{\theta}^\top)$ in which $\mu \in \mathbb{R}$, $\lambda \in \mathbb{R}$, and $\sigma \in \mathbb{R}^+$ are the location, skewness, and scale parameters of $F(\cdot|\Psi)$, respectively.

Likewise, the multivariate version of the GSMM distribution is given as follows.

Definition 4.0.7. (multivariate GLSM) *Let random vector $\mathbf{Z}$ follows a p -dimensional standard Gaussian distribution. For a given positive mixing random variable G with PDF $h(\cdot|\boldsymbol{\theta})$ in which $\boldsymbol{\theta}$ is the family parameter, the p -dimensional random vector $\mathbf{Y}$ with CDF $F(\cdot|\Psi)$ is said to follow a Gaussian scale mixture model if*

$$\mathbf{Y} = \boldsymbol{\mu} + \frac{\boldsymbol{\lambda}}{\eta(G)} + \boldsymbol{\Sigma}^{\frac{1}{2}} \frac{\mathbf{Z}}{\sqrt{\eta(G)}}, \quad (4.14)$$

where $\Psi = (\boldsymbol{\mu}, \boldsymbol{\lambda}, \boldsymbol{\Sigma}, \boldsymbol{\theta})$ in which $\boldsymbol{\mu} \in \mathbb{R}^p$, $\boldsymbol{\lambda} \in \mathbb{R}^p$, and $p \times p$ positive definite matrix $\boldsymbol{\Sigma}$ are the location, skewness, and scale parameters of $F(\cdot|\Psi)$, respectively. Obviously identity (4.14) admits the following hierarchy

$$\begin{aligned} \mathbf{Y}|G=g &\sim \mathcal{N}_p\left(\boldsymbol{\mu} + \frac{\boldsymbol{\lambda}}{\eta(g)}, \frac{\boldsymbol{\Sigma}}{\eta(g)}\right), \\ G &\sim h(g|\boldsymbol{\theta}), \end{aligned} \quad (4.15)$$

or equivalently the CDF of $\mathbf{Y}$ becomes

$$F(\mathbf{x}|\Psi) = \int_0^\infty \Phi_p\left[\mathbf{x} - \boldsymbol{\mu} - \frac{\boldsymbol{\alpha}}{\eta(g)} \middle| \mathbf{0}, \frac{\boldsymbol{\Sigma}}{\eta(g)}\right] h(g|\boldsymbol{\theta}) dg. \quad (4.16)$$

Example 4-1

If we let $\eta(G) = G^{-1}$, $\boldsymbol{\mu} = \mathbf{0}$, and $G \sim \mathcal{G}(\alpha, 1)$ in Definition 4.0.7, then the established GLSM model is a generalized p -dimensional asymmetric Laplace (GMAL) distribution with characteristic function

$$\psi(\mathbf{t}; \alpha, \boldsymbol{\Sigma}, \boldsymbol{\lambda}) = \left(1 + \frac{\mathbf{t}^T \boldsymbol{\Sigma} \mathbf{t}}{2} - i \boldsymbol{\lambda}^T \mathbf{t}\right)^{-\alpha},$$

where $i^2 = -1$ and $\mathbf{t} \in \mathbb{R}^p$. The class of GMAL distributions arises frequently in modeling data exhibiting heavier tails and higher peak than the Gaussian distribution Park and Lee [2004], Eltoft et al. [2006]. ■

Example 4-2

If we set in Definition 4.0.7 $\eta(G) = G^{-1}$ and $G \sim \mathcal{GIG}(\psi, \chi, \lambda)$ with PDF given in (2.51), then the established GLSM model is a generalized hyperbolic (GH) distribution with PDF given by [McNeil et al., 2005]:

$$f(\mathbf{x}|\Psi) = \frac{\exp\{(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} \boldsymbol{\alpha}\} \left[\frac{\chi + \delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{\psi + \delta(\boldsymbol{\alpha}, \Sigma)} \right]^{\frac{\lambda-p/2}{2}} \left(\frac{\psi}{\chi} \right)^{\frac{\lambda}{2}}}{(2\pi)^{p/2} |\Sigma|^{1/2}} \times \frac{\mathcal{K}_{\lambda-p/2}(\sqrt{[\psi + \delta(\boldsymbol{\alpha}, \Sigma)][\chi + \delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)]})}{\mathcal{K}_\lambda(\sqrt{\psi\chi})}, \quad (4.17)$$

where $\Psi = (\boldsymbol{\mu}, \boldsymbol{\alpha}, \Sigma, \boldsymbol{\theta})$ in which $\boldsymbol{\theta} = (\psi, \chi, \lambda)^\top$ and $\delta(\cdot, \cdot)$ is defined as (4.29). The GH distribution with PDF in (4.17) has been introduced briefly by Barndorff-Nielsen [1977a] and then its properties discussed in several works Barndorff-Nielsen [1977b, 1978]. As an application, this distribution was used for modelling the distribution of the Australian personal income [Barndorff-Nielsen, 1978]. For useful accounts of GH distribution, the reader is referred to Weibel et al. [2020] and referenced therein. One issue that may arise with the GH distribution is its non-identifiability [Murray et al., 2017]. For any $m > 0$, let $\Psi = (\boldsymbol{\mu}, \boldsymbol{\alpha}, \Sigma, \boldsymbol{\theta})$ and $\Psi^* = (\boldsymbol{\mu}, m\boldsymbol{\alpha}, m\Sigma, \boldsymbol{\theta})$ in which $\boldsymbol{\theta} = (m\psi, \chi/m, \lambda)^\top$ denote two different choices of the family parameter vector. It is easy to check from (4.17) that $f(\mathbf{x}|\Psi) = f(\mathbf{x}|\Psi^*)$. Hence, the GH distribution with PDF in (4.17) is not *identifiable*. In order to ensure the identifiability, we may set $\psi = \chi = a$, for $a > 0$, and $\boldsymbol{\alpha} = \mathbf{0}$. The result is the symmetric hyperbolic (SH) distribution with PDF

$$f(\mathbf{x}|\Psi) = \left[\frac{a + \delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{a} \right]^{\frac{\lambda-p/2}{2}} \frac{\mathcal{K}_{\lambda-p/2}(\sqrt{a[a + \delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)]})}{(2\pi)^{p/2} |\Sigma|^{1/2} \mathcal{K}_\lambda(a)}, \quad (4.18)$$

where $\Psi = (\boldsymbol{\mu}, \Sigma, \boldsymbol{\theta})$ in which $\boldsymbol{\theta} = (a, \lambda)^\top$. Evidently, each SH distribution is a GSM distribution that follows identity (??) in which $\eta(G) = G-1$ and $G \sim \mathcal{GIG}(a, a, \lambda)$. The SH distribution is a heavy tailed distribution and is commonly used in finance for modelling the distribution of return's rate Eberlein and Keller [1995] and also for computing the value-at-risk parameter for a certain portfolio Bauer [2000]. ■

4.1 Skewed Gaussian scale mixture models

The GSM models introduced earlier can be further developed to construct skewed models with widespread application in model-based clustering [Lee and McLachlan, 2016b] and factor analysis [Tortora et al., 2016], classification, and discriminant Analysis [Lee and McLachlan, 2013, Tortora et al., 2021], mixture regression [Kim and Browne, 2024], etc. In what follows, first, we give a definition of the general class of skewed Gaussian scale mixture models (SGSM) in multivariate case, and then three important members are represented.

Definition 4.1.1. Suppose $\mathbf{X} \sim \text{SDB}_p(\boldsymbol{\mu}, \Sigma, \Lambda)$ as Definition 4.0.1). For a given positive mixing random variable G with PDF $h(\cdot|\boldsymbol{\theta})$ in which $\boldsymbol{\theta}$ is the family parameter, the p -dimensional random vector $\mathbf{Y}$ is said to follow a SDB scale mixture model if

$$\mathbf{Y} \stackrel{d}{=} \boldsymbol{\mu} + \frac{\mathbf{X}}{\sqrt{h(G)}}, \quad (4.19)$$

or equivalently

$$\mathbf{Y} \stackrel{d}{=} \boldsymbol{\mu} + \frac{\Lambda |\mathbf{Z}_0|}{\sqrt{h(G)}} + \Sigma^{\frac{1}{2}} \frac{\mathbf{Z}_1}{\sqrt{h(G)}}, \quad (4.20)$$

where $\mathbf{Z}_0 \sim \mathcal{N}_p(\mathbf{0}, \mathbf{I}_q)$ and $\mathbf{Z}_1 \sim \mathcal{N}_p(\mathbf{0}, \mathbf{I}_p)$ are independent.

Corollary 4.1.1. *If $\mathbf{Y}$ follows a SDB scale mixture model (4.19) or (4.20), then we have*

$$\begin{aligned} \mathbf{Y}|U, G &\sim \mathcal{N}_p\left(\boldsymbol{\mu} + \Lambda U, \frac{\Sigma}{h(G)}\right), \\ U|G &\sim \mathcal{HN}_p\left(\mathbf{0}, \frac{\mathbf{I}_p}{h(G)}\right), \\ G &\sim f(g|\boldsymbol{\theta}), \end{aligned} \quad (4.21)$$

where the short form $\mathcal{HN}_p(\mathbf{0}, \Sigma)$ accounts for the half-normal distribution truncated on $\mathbb{R}^+$ with a zero-location vector and scale matrix Σ .

Definition 4.1.2. *Suppose $\mathbf{X} \sim \text{CSG}_p(\boldsymbol{\mu}, \Sigma, \boldsymbol{\lambda})$ as Definition 4.0.2). For a given positive mixing random variable G with PDF $h(\cdot|\boldsymbol{\theta})$ in which $\boldsymbol{\theta}$ is the family parameter, the p -dimensional random vector $\mathbf{Y}$ is said to follow a CSG scale mixture model if*

$$\mathbf{Y} \stackrel{d}{=} \boldsymbol{\mu} + \frac{\mathbf{X}}{\sqrt{h(G)}}, \quad (4.22)$$

or equivalently

$$\mathbf{Y} \stackrel{d}{=} \boldsymbol{\mu} + \frac{\lambda |Z_0|}{\sqrt{h(G)}} + \Sigma^{\frac{1}{2}} \frac{\mathbf{Z}_1}{\sqrt{h(G)}}, \quad (4.23)$$

where $Z_0 \sim \mathcal{N}(0, 1)$ and $\mathbf{Z}_1 \sim \mathcal{N}_p(\mathbf{0}, \mathbf{I}_p)$ are independent.

Corollary 4.1.2. *If $\mathbf{Y}$ follows a CSG scale mixture model (4.22) or (4.23), then we have*

$$\begin{aligned} \mathbf{Y}|U, G &\sim \mathcal{N}_p\left(\boldsymbol{\mu} + \lambda U, \frac{\Sigma}{h(G)}\right), \\ U|G &\sim \mathcal{HN}\left(0, \frac{1}{h(G)}\right), \\ G &\sim f(g|\boldsymbol{\theta}), \end{aligned} \quad (4.24)$$

where the short form $\mathcal{HN}(0, \sigma^2)$ accounts for the univariate half-normal distribution truncated on $\mathbb{R}^+$ with a zero-location and variance σ^2 .

Definition 4.1.3. *Suppose $\mathbf{X} \sim \text{CFUSG}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda)$ as Definition 4.0.3). For a given positive mixing random variable G with PDF $h(\cdot|\boldsymbol{\theta})$ in which $\boldsymbol{\theta}$ is the family parameter, the p -dimensional random vector $\mathbf{Y}$ is said to follow a CFUSG scale mixture model if*

$$\mathbf{Y} \stackrel{d}{=} \boldsymbol{\mu} + \frac{\mathbf{X}}{\sqrt{h(G)}}, \quad (4.25)$$

or equivalently

$$\mathbf{Y} \stackrel{d}{=} \boldsymbol{\mu} + \frac{\Lambda |\mathbf{Z}_0|}{\sqrt{h(G)}} + \Sigma^{\frac{1}{2}} \frac{\mathbf{Z}_1}{\sqrt{h(G)}}, \quad (4.26)$$

where $\mathbf{Z}_0 \sim \mathcal{N}_q(\mathbf{0}, \mathbf{I}_q)$ and $\mathbf{Z}_1 \sim \mathcal{N}_p(\mathbf{0}, \mathbf{I}_p)$ are independent.

Corollary 4.1.3. *If $\mathbf{Y}$ follows a CFUSG scale mixture model (4.25) or (4.26), then we have*

$$\begin{aligned}\mathbf{Y}|\mathbf{U}, G &\sim \mathcal{N}_p\left(\boldsymbol{\mu} + \Lambda\mathbf{U}, \frac{\Sigma}{h(G)}\right), \\ \mathbf{U}|G &\sim \mathcal{HN}_q\left(\mathbf{0}, \frac{\mathbf{I}_q}{h(G)}\right), \\ G &\sim f(g|\boldsymbol{\theta}),\end{aligned}\tag{4.27}$$

where the short form $\mathcal{HN}_q(\mathbf{0}, \Sigma)$ accounts for the half-normal distribution truncated on $\mathbb{R}^{q+}$ with a zero-location vector and scale matrix Σ .

4.1.1 Special models

Table 4.3 shows the most commonly used three CFUSG scale mixture models in model-based clustering. These models are constructed by assuming that G in (4.25) follows generalized Lindely, denoted by $\mathcal{GL}(\omega, \beta, \gamma)$, $\mathcal{G}(\eta, \zeta)$, and $\mathcal{GIG}(\tau, \chi, \chi)$. The models thereof are denoted as $\text{SGGL}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$, $\text{SGG}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$, and $\text{SGGIG}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$, respectively. We start with expressions for PDF and CDF of Student's t distribution. Let random vector $\mathbf{Y} = (Y_1, \dots, Y_p)^\top$ follow a p -dimensional Student's t distribution with $\nu > 0$ degrees of freedom. The PDF of $\mathbf{Y}$ is then given by [Kotz and Nadarajah, 2004]:

$$t_\nu(\mathbf{y}|\boldsymbol{\mu}, \Sigma) = \frac{\Gamma(\frac{\nu+p}{2})}{(\nu\pi)^{\frac{p}{2}}\Gamma(\frac{\nu}{2})|\Sigma|^{\frac{1}{2}}} \left[1 + \frac{\delta(\mathbf{y} - \boldsymbol{\mu}, \Sigma)}{\nu}\right]^{-\frac{\nu+p}{2}},\tag{4.28}$$

where $\delta(\cdot)$ is the well-known Mahalanobis distance defined as

$$\delta(\mathbf{x}, \Sigma) = \mathbf{x}^\top \Sigma^{-1} \mathbf{x},\tag{4.29}$$

and $\boldsymbol{\mu}$, Σ , and ν denote the location, scale, and degrees of freedom parameters, respectively. The CDF associated with the PDF (4.28) is

$$T_\nu(\mathbf{x}|\boldsymbol{\mu}, \Sigma) = \int_{-\infty}^{x_1} \cdots \int_{-\infty}^{x_p} \frac{\Gamma(\frac{\nu+p}{2})}{(\nu\pi)^{\frac{p}{2}}\Gamma(\frac{\nu}{2})|\Sigma|^{\frac{1}{2}}} \left[1 + \frac{\delta(\mathbf{y} - \boldsymbol{\mu}, \Sigma)}{\nu}\right]^{-\frac{\nu+p}{2}} dy_1 \cdots dy_p,\tag{4.30}$$

where $\mathbf{x} \in \mathbb{R}^p$. We write $\mathbf{Y} \sim t_\nu(\boldsymbol{\mu}, \Sigma)$ to indicate that $\mathbf{Y}$ follows a Student's t distribution with PDF (4.28). In univariate case, the PDF of Student's t distribution with ν degrees of freedom given by

$$t_\nu(y|\mu, \sigma^2) = \frac{\Gamma(\frac{\nu+1}{2})}{\sigma\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left[1 + \frac{z^2}{\nu}\right]^{-\frac{\nu+1}{2}}.\tag{4.31}$$

where $z = (y - \mu)/\sigma$. The CDF corresponds to (4.31) is represented as

$$T_\nu(x|\mu, \sigma^2) = \int_{-\infty}^x \frac{\Gamma(\frac{\nu+1}{2})}{\sigma\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left[1 + \frac{z^2}{\nu}\right]^{-\frac{\nu+1}{2}} dy,\tag{4.32}$$

where $x \in \mathbb{R}$ and $z = (y - \mu)/\sigma$. We write $Y \sim t_\nu(\mu, \sigma^2)$ to indicate that Y follows a Student's t distribution with PDF (4.31). Moreover, in the univariate case, the quantile function (inverse of CDF) of the Student's t distribution is represented as

$$T_\nu^{-1}(u) = \min\{x \mid T_\nu(x|\mu, \sigma^2) \geq u\}, 0 < u < 1.$$

Theorem 4.1.1. *Let $\mathbf{Y} \sim SGGIG_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$. The PDF of $\mathbf{Y}$ is given by*

$$f(\mathbf{y}|\boldsymbol{\Psi}) = 2^q \mathbf{h}(\mathbf{y}|\boldsymbol{\mu}, \Omega, \tau, \chi, \chi) \mathbf{H}\left(\mathbf{m}\left[\frac{\chi}{\chi + d(\mathbf{y})}\right]^{\frac{1}{4}} \middle| \mathbf{0}_q, \Delta, \tau - \frac{p}{2}, \kappa, \kappa\right),$$

where $\boldsymbol{\Psi} = (\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$, $\boldsymbol{\theta} = (\tau, \chi)^\top$, $\kappa = \chi\sqrt{1 + d(\mathbf{y})/\chi}$, $\mathbf{h}(\cdot|\boldsymbol{\mu}, \Omega, \tau, \chi, \chi)$ and $\mathbf{H}(\cdot|\mathbf{0}_q, \Delta, \tau - p/2, \kappa, \kappa)$ are the PDF and CDF of a p - and q -dimensional symmetric hyperbolic distribution, respectively.

Proof: See Murray et al. [2017].

In what follows, for example, Theorem 4.1.2 shows how PDF of CFUSG scale mixture models with generalized Lindley as the mixing distribution that is denoted by $SGGL_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$.

Table 4.3: Three members of CFUSG scale mixture models.

Characteristic	$\text{SGL}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$	$\text{SG}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$	$\text{SGIG}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$
family of G	$\mathcal{GL}(\omega, \beta, \gamma)$	$\mathcal{G}(\eta, \zeta)$	generalized inverse Gaussian ($\mathcal{GIG}(\tau, \chi, \chi)$)
parameter	$\boldsymbol{\theta} = (\omega, \beta, \gamma)^\top$	$\boldsymbol{\theta} = (\eta, \zeta)^\top$	$\boldsymbol{\theta} = (\tau, \chi)^\top$
PDF of G	$f(g \boldsymbol{\theta}) = \frac{\beta^{\omega+1} g^{\omega-1} (\omega+\gamma g)}{(\beta+\gamma) \Gamma(\omega+1)} \exp\{-\beta g\}$	$f(g \boldsymbol{\theta}) = \frac{\zeta^\eta}{\Gamma(\eta)} g^{\eta-1} \exp\{-\zeta g\}$	$f(g \boldsymbol{\theta}) = \frac{g^{\tau-1}}{2\kappa_\tau(\chi)} \exp\left\{-\frac{\chi g}{2} - \frac{\chi}{2g}\right\}$
PDF	$f(\mathbf{y} \boldsymbol{\Psi}) = \frac{2^q \beta^{1-\frac{p}{2}} \omega^{\frac{p}{2}}}{(\beta+\gamma)} t_{2\omega} \left(\sqrt{\frac{\omega}{\beta}} (\mathbf{y} - \boldsymbol{\mu}) \middle \mathbf{0}_p, \Omega \right)$ $\times \mathbf{T}_{p+2\omega} \left(\mathbf{m} \sqrt{\frac{p+2\omega}{2\beta+d(\mathbf{y})}} \middle \mathbf{0}_q, \Delta \right)$ $+ \frac{2^q \gamma \beta^{-\frac{p}{2}} (\omega+1)^{\frac{p}{2}}}{(\beta+\gamma)} t_{2\omega+2} \left(\sqrt{\frac{\omega+1}{\beta}} (\mathbf{y} - \boldsymbol{\mu}) \middle \mathbf{0}_p, \Omega \right)$ $\times \mathbf{T}_{p+2\omega+2} \left(\mathbf{m} \sqrt{\frac{p+2\omega+2}{2\beta+d(\mathbf{y})}} \middle \mathbf{0}_q, \Delta \right)$	$f(\mathbf{y} \boldsymbol{\Psi}) = 2^q \left(\frac{\eta}{\zeta} \right)^{\frac{p}{2}} \times t_{2\eta} \left(\sqrt{\frac{\eta}{\zeta}} (\mathbf{y} - \boldsymbol{\mu}) \middle \mathbf{0}_p, \Omega \right)$ $\times \mathbf{T}_{p+2\eta} \left(\mathbf{m} \sqrt{\frac{p+2\eta}{2\zeta+d(\mathbf{y})}} \middle \mathbf{0}_q, \Delta \right)$	$f(\mathbf{y} \boldsymbol{\Psi}) = 2^q h_p(\mathbf{y} \boldsymbol{\mu}, \Omega, \tau, \chi, \chi)$ $\times \mathbf{H}_q \left(\mathbf{m} \left[\frac{\chi}{\chi+d(\mathbf{y})} \right]^{\frac{1}{4}} \middle \mathbf{0}, \Delta, \tau - \frac{p}{2}, \chi \sqrt{1 + \frac{d(\mathbf{y})}{\chi}} \right)$ $\chi \sqrt{1 + \frac{d(\mathbf{y})}{\chi}}$
Representation	$\mathbf{Y}^d = \boldsymbol{\mu} + \frac{\Lambda \mathbf{Z}_0 }{\sqrt{GL}} + \Sigma^{\frac{1}{2}} \frac{\mathbf{Z}_1}{\sqrt{GL}}$	$\mathbf{Y}^d = \boldsymbol{\mu} + \frac{\Lambda \mathbf{Z}_0 }{\sqrt{G}} + \Sigma^{\frac{1}{2}} \frac{\mathbf{Z}_1}{\sqrt{G}}$	$\mathbf{Y}^d = \boldsymbol{\mu} + \frac{\Lambda}{\sqrt{GIG}} \mathbf{Z}_0 + \Sigma^{\frac{1}{2}} \frac{\mathbf{Z}_1}{\sqrt{GIG}}$

Theorem 4.1.2. Let $\mathbf{Y} \sim SGGL_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$. The PDF of $\mathbf{Y}$ is given by

$$\begin{aligned} f(\mathbf{y}|\Psi) &= \frac{2^q \beta^{1-\frac{p}{2}} \omega^{\frac{p}{2}}}{(\beta + \gamma)} \mathbf{t}_{2\omega} \left(\sqrt{\frac{\omega}{\beta}} (\mathbf{y} - \boldsymbol{\mu}) \middle| \mathbf{0}_p, \Omega \right) \mathbf{T}_{p+2\omega} \left(\mathbf{m} \sqrt{\frac{p+2\omega}{2\beta+d(\mathbf{y})}} \middle| \mathbf{0}_q, \Delta \right) \\ &\quad + \frac{2^q \gamma \beta^{-\frac{p}{2}} (\omega+1)^{\frac{p}{2}}}{(\beta + \gamma)} \mathbf{t}_{2\omega+2} \left(\sqrt{\frac{\omega+1}{\beta}} (\mathbf{y} - \boldsymbol{\mu}) \middle| \mathbf{0}_p, \Omega \right) \\ &\quad \times \mathbf{T}_{p+2\omega+2} \left(\mathbf{m} \sqrt{\frac{p+2\omega+2}{2\beta+d(\mathbf{y})}} \middle| \mathbf{0}_q, \Delta \right), \end{aligned}$$

where $\Psi = (\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$, $\boldsymbol{\theta} = (\omega, \beta, \gamma)^\top$, $d(\mathbf{y}) = (\mathbf{y} - \boldsymbol{\mu})^\top \Omega^{-1} (\mathbf{y} - \boldsymbol{\mu})$, $\mathbf{m} = \Lambda^\top \Omega^{-1} (\mathbf{y} - \boldsymbol{\mu})$, $\Omega = \Sigma + \Lambda \Lambda^\top$, and $\Delta = \mathbf{I}_q - \Lambda^\top \Omega^{-1} \Lambda$.

Proof: By definition, based on (4.27), we have

$$\begin{aligned} f_{\mathbf{Y}}(\mathbf{y}|\Psi) &= \int_{\mathbb{R}^{q+}} \int_0^\infty f(\mathbf{y}|\mathbf{t}, g, \Psi) f(\mathbf{t}|g) f(g|\boldsymbol{\theta}) dg d\mathbf{t}, \\ &= \frac{2^q |\Sigma|^{-\frac{1}{2}}}{(2\pi)^{\frac{p+q}{2}}} \int_{\mathbb{R}^{q+}} \int_0^\infty g^{\frac{p+q}{2}} \exp \left\{ -\frac{g}{2} (\mathbf{y} - \boldsymbol{\mu} - \Lambda \mathbf{t})^\top \Sigma^{-1} (\mathbf{y} - \boldsymbol{\mu} - \Lambda \mathbf{t}) \right\} \\ &\quad \times \exp \left\{ -\frac{g}{2} \mathbf{t}^\top \mathbf{I}_q \mathbf{t} \right\} f(g|\boldsymbol{\theta}) dg d\mathbf{t} \\ &= \frac{2^q}{(2\pi)^{\frac{p+q}{2}} |\Sigma|^{\frac{1}{2}}} \int_{\mathbb{R}^{q+}} \int_0^\infty g^{\frac{p+1}{2}} \exp \left\{ -\frac{g}{2} [d(\mathbf{y}) + (\mathbf{t} - \mathbf{m})^\top \Delta^{-1} (\mathbf{t} - \mathbf{m})] \right\} \\ &\quad \times f(g|\boldsymbol{\theta}) dg d\mathbf{t} \\ &= \frac{2^q \beta^{\alpha+1}}{(\beta + \gamma) \Gamma(\alpha) (2\pi)^{\frac{p+q}{2}} |\Sigma|^{\frac{1}{2}}} \int_{\mathbb{R}^{q+}} \int_0^\infty g^{\frac{p+q+2\alpha}{2}-1} \\ &\quad \times \exp \left\{ -\frac{g}{2} [d(\mathbf{y}) + (\mathbf{t} - \mathbf{m})^\top \Delta^{-1} (\mathbf{t} - \mathbf{m}) + 2\beta] \right\} dg d\mathbf{t} \\ &\quad + \frac{2^q \gamma \beta^{\alpha+1}}{\alpha(\beta + \gamma) \Gamma(\alpha) (2\pi)^{\frac{p+q}{2}} |\Sigma|^{\frac{1}{2}}} \int_{\mathbb{R}^{q+}} \int_0^\infty g^{\frac{p+q+2\alpha+2}{2}-1} \\ &\quad \times \exp \left\{ -\frac{g}{2} [d(\mathbf{y}) + (\mathbf{t} - \mathbf{m})^\top \Delta^{-1} (\mathbf{t} - \mathbf{m}) + 2\beta] \right\} dg d\mathbf{t}. \end{aligned}$$

It follows that

$$\begin{aligned} f_{\mathbf{Y}}(\mathbf{y}|\Psi) &= \frac{2^q \beta^{\alpha+1} \Gamma(\frac{p+q+2\alpha}{2}) |\Sigma|^{-\frac{1}{2}}}{(\beta + \gamma) \Gamma(\alpha) (2\pi)^{\frac{p+q}{2}}} \int_{\mathbb{R}^{q+}} \frac{d\mathbf{t}}{\left[\frac{2\beta+d(\mathbf{y})+(\mathbf{t}-\mathbf{m})^\top \Delta^{-1} (\mathbf{t}-\mathbf{m})}{2} \right]^{\frac{p+q+2\alpha}{2}}} \\ &\quad + \frac{2^q \gamma \beta^{\alpha+1} \Gamma(\frac{p+q+2\alpha+2}{2}) |\Sigma|^{-\frac{1}{2}}}{\alpha(\beta + \gamma) \Gamma(\alpha) (2\pi)^{\frac{p+q}{2}}} \int_{\mathbb{R}^{q+}} \frac{d\mathbf{t}}{\left[\frac{2\beta+d(\mathbf{y})+(\mathbf{t}-\mathbf{m})^\top \Delta^{-1} (\mathbf{t}-\mathbf{m})}{2} \right]^{\frac{p+q+2\alpha+2}{2}}} \\ &= I_1 + I_2. \end{aligned} \tag{4.33}$$

Using a change of variable $\mathbf{v} = (\mathbf{t} - \mathbf{m}) \sqrt{p+2\alpha} / \sqrt{2\beta+d(\mathbf{y})}$, we can write

$$\begin{aligned} I_1 &= \frac{2^q \beta^{\alpha+1} \Gamma(\frac{p+q+2\alpha}{2}) |\Sigma|^{-\frac{1}{2}}}{(\beta + \gamma) \Gamma(\alpha) (2\pi)^{\frac{p+q}{2}}} \frac{2^{\frac{p+q+2\alpha}{2}} (p+2\alpha)^{-\frac{q}{2}}}{\left[2\beta+d(\mathbf{y}) \right]^{\frac{p+2\alpha}{2}}} \int_{-\infty}^{\mathbf{m}_0} \frac{d\mathbf{v}}{\left[1 + \frac{\mathbf{v}^\top \Delta^{-1} \mathbf{v}}{p+2\alpha} \right]^{\frac{p+q+2\alpha}{2}}}, \\ &= \frac{2^q \beta^{\alpha+1} \Gamma(\frac{p+2\alpha}{2}) |\Sigma|^{-\frac{1}{2}}}{(\beta + \gamma) \Gamma(\alpha) (2\pi)^{\frac{p}{2}}} \frac{2^{\frac{p+2\alpha}{2}} |\Delta|^{\frac{1}{2}}}{\left[2\beta+d(\mathbf{y}) \right]^{\frac{p+2\alpha}{2}}} \mathbf{T}_{p+2\alpha} \left(\mathbf{m} \sqrt{\frac{p+2\alpha}{2\beta+d(\mathbf{y})}} \middle| \mathbf{0}_q, \Delta \right), \end{aligned}$$

where $\mathbf{m}_0 = \mathbf{m}\sqrt{p+2\alpha}/\sqrt{2\beta+d(\mathbf{y})}$. It follows that

$$\begin{aligned}
I_1 &= \frac{2^q \beta^{\alpha+1} 2^{\frac{p+2\alpha}{2}} (2\alpha)^{\frac{p}{2}} |\Omega|^{\frac{1}{2}} |\Delta|^{\frac{1}{2}}}{(\beta + \gamma) 2^{\frac{p}{2}} |\Sigma|^{\frac{1}{2}}} \frac{\Gamma(\frac{p+2\alpha}{2})}{\pi^{\frac{p}{2}} (2\alpha)^{\frac{p}{2}} \Gamma(\alpha) |\Omega|^{\frac{1}{2}} [2\beta + d(\mathbf{y})]^{\frac{p+2\alpha}{2}}} \\
&\quad \times \mathbf{T}_{p+2\alpha} \left(\mathbf{m} \sqrt{\frac{p+2\alpha}{2\beta + d(\mathbf{y})}} \middle| \mathbf{0}_q, \Delta \right) \\
&= \frac{2^q \beta^{1-\frac{p}{2}} \alpha^{\frac{p}{2}}}{(\beta + \gamma)} \mathbf{t}_{2\alpha} \left(\sqrt{\frac{\alpha}{\beta}} (\mathbf{y} - \boldsymbol{\mu}) \middle| \mathbf{0}_p, \Omega \right) \mathbf{T}_{p+2\alpha} \left(\mathbf{m} \sqrt{\frac{p+2\alpha}{2\beta + d(\mathbf{y})}} \middle| \mathbf{0}_q, \Delta \right). \tag{4.34}
\end{aligned}$$

It should be noted that for obtaining the term in the RHS of (4.34), we use this fact that $|\Delta|^{\frac{1}{2}} |\Omega|^{\frac{1}{2}} / |\Sigma|^{\frac{1}{2}} = 1$. Likewise, we can obtain I_2 as

$$\begin{aligned}
I_2 &= \frac{2^q \gamma \beta^{\alpha+1} \Gamma(\frac{p+2\alpha+2}{2})}{\alpha(\beta + \gamma) \Gamma(\alpha) (2\pi)^{\frac{p}{2}} |\Sigma|^{\frac{1}{2}}} \frac{2^{\frac{p+2\alpha+2}{2}} |\Delta|^{\frac{1}{2}}}{[2\beta + d(\mathbf{y})]^{\frac{p+2\alpha+2}{2}}} \\
&\quad \times \mathbf{T}_{p+2\alpha} \left(\mathbf{m} \sqrt{\frac{p+2\alpha+2}{2\beta + d(\mathbf{y})}} \middle| \mathbf{0}_q, \Delta \right), \\
&= \frac{2^q \gamma \beta^{\alpha+1} 2^{\frac{p+2\alpha+2}{2}} (2\alpha+2)^{\frac{p}{2}} |\Omega|^{\frac{1}{2}} |\Delta|^{\frac{1}{2}} \Gamma(\frac{p+2\alpha+2}{2}) [2\beta + d(\mathbf{y})]^{-\frac{p+2\alpha+2}{2}}}{(\beta + \gamma) 2^{\frac{p}{2}} |\Sigma|^{\frac{1}{2}} \pi^{\frac{p}{2}} (2\alpha+2)^{\frac{p}{2}} \alpha \Gamma(\alpha) |\Omega|^{\frac{1}{2}}} \\
&\quad \times \mathbf{T}_{p+2\alpha+2} \left(\mathbf{m} \sqrt{\frac{p+2\alpha+2}{2\beta + d(\mathbf{y})}} \middle| \mathbf{0}_q, \Delta \right) \\
&= \frac{2^q \gamma \beta^{-\frac{p}{2}} (\alpha+1)^{\frac{p}{2}}}{(\beta + \gamma)} \mathbf{t}_{2\alpha+2} \left(\sqrt{\frac{\alpha+1}{\beta}} (\mathbf{y} - \boldsymbol{\mu}) \middle| \mathbf{0}_p, \Omega \right) \\
&\quad \times \mathbf{T}_{p+2\alpha+2} \left(\mathbf{m} \sqrt{\frac{p+2\alpha+2}{2\beta + d(\mathbf{y})}} \middle| \mathbf{0}_q, \Delta \right).
\end{aligned}$$

The proof is complete.

It is worth to note that model $\text{SGGL}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$, for $\boldsymbol{\theta} = \nu/2$, specializes to the canonical fundamental skewed student's t distribution [Lee and McLachlan, 2016a, Table 1]. We have

Corollary 4.1.4. *If in (4.27) $G \sim \mathcal{G}(\nu/2, \nu/2)$, then $\mathbf{Y} \sim \text{SNG}_{p,q}(\nu, \boldsymbol{\mu}, \Sigma, \Lambda)$ follows skew Student's t distribution with PDF given by*

$$f(\mathbf{y}|\boldsymbol{\Psi}) = 2^q \mathbf{t}_{\nu}(\mathbf{y}|\boldsymbol{\mu}, \Omega) \mathbf{T}_{p+\nu} \left(\mathbf{m} \sqrt{\frac{\nu+p}{\nu+d(\mathbf{y})}} \middle| \mathbf{0}_q, \Delta \right),$$

where $\boldsymbol{\Psi} = (\nu, \boldsymbol{\mu}, \Sigma, \Lambda)$.

4.1.2 Computing the CDF of multivariate Gaussian distribution

Due to significant importance of the multivariate Gaussian distribution in a wide range of study fields, computing its CDF has a long history. Several efforts have been made for computing the CDF of multivariate Gaussian distribution. For more detailed information, we refer the reader to Botev [2017] and references therein. One of the pioneers in this context was Genz [1992] who suggested the separation of variables (SOV) method. Let $\mathbf{X} \sim \mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$ and L denote the Cholesky decomposition of Σ , that is $\Sigma = LL^{\top}$ in which L is a $p \times p$ lower triangular matrix.

For computing $\mathcal{P}_G = P(\mathbf{a} < \mathbf{X} < \mathbf{b})$, in which $\mathbf{a}$ and $\mathbf{b}$ are vectors of real constants, the SOV method can be described as follows. Consider transformation $\mathbf{x} = L\mathbf{y}$, we have

$$\mathbf{x}^\top \Sigma^{-1} \mathbf{x} = \mathbf{y}^\top L^\top L^{-\top} L^{-1} L \mathbf{y} = \mathbf{y}^\top \mathbf{y} = \sum_{i=1}^p y_i^2,$$

and $d\mathbf{x} = |L|d\mathbf{y} = \sqrt{|\Sigma|}d\mathbf{y}$. Hence, $\mathbf{a} < L\mathbf{y} < \mathbf{b}$ or equivalently

$$\begin{aligned} \frac{a_1}{L_{1,1}} &\leq y_1 \leq \frac{b_1}{L_{1,1}} \\ \frac{a_2 - L_{2,1}y_1}{L_{2,2}} &\leq y_2 \leq \frac{b_2 - L_{2,1}y_1}{L_{2,2}} \\ &\vdots \\ \frac{a_p - \sum_{i=1}^{p-1} L_{p,i}y_i}{L_{p,p}} &\leq y_p \leq \frac{b_p - \sum_{i=1}^{p-1} L_{p,i}y_i}{L_{p,p}}. \end{aligned}$$

This type of change of variable turns the problem of computing $\mathcal{P}$ into the product of p univariate integrals as

$$\mathcal{P}_G = (2\pi)^{-\frac{p}{2}} \int_{a'_1}^{b'_1} \exp\left\{-\frac{y_1^2}{2}\right\} \int_{a'_2}^{b'_2} \exp\left\{-\frac{y_2^2}{2}\right\} \cdots \int_{a'_p}^{b'_p} \exp\left\{-\frac{y_p^2}{2}\right\} d\mathbf{y},$$

where $a'_i = (a_i - \sum_{j=1}^{i-1} L_{i,j}y_j)/L_{i,i}$ and $b'_i = (b_i - \sum_{j=1}^{i-1} L_{i,j}y_j)/L_{i,i}$ in which, by convention, we assume $\sum_{j=1}^0 L_{i,j} = 0$. Using transformation of the form $z_i = \Phi(y_i)$ (for $i = 1, \dots, p$), we obtain

$$\mathcal{P}_G = \int_{d_1}^{e_1} \int_{d_2}^{e_2} \cdots \int_{d_p}^{e_p} d\mathbf{z}, \quad (4.35)$$

where

$$\begin{aligned} d_i &= \Phi\left(\frac{a_i - \sum_{j=1}^{i-1} L_{i,j}\Phi^{-1}(z_j)}{L_{i,i}}\right), \\ e_i &= \Phi\left(\frac{b_i - \sum_{j=1}^{i-1} L_{i,j}\Phi^{-1}(z_j)}{L_{i,i}}\right). \end{aligned}$$

For computational purposes, we apply an extra transformation $w_i = (z_i - d_i)/(e_i - d_i)$ by which $\mathcal{P}_G$ in (4.35) becomes a $(p-1)$ -dimensional integral in unit hypercube $[0, 1]^{p-1}$. It follows that

$$\begin{aligned} \mathcal{P}_G &= \int_0^1 (e'_1 - d'_1)dw_1 \int_0^1 (e'_2 - d'_2)dw_2 \cdots \int_0^1 (e'_{p-1} - d'_{p-1})dw_{p-1} \int_0^1 (e'_p - d'_p)dw_p, \\ &= (e'_1 - d'_1) \int_0^1 (e'_2 - d'_2)dw_1 \int_0^1 (e'_3 - d'_3)dw_2 \cdots \int_0^1 (e'_p - d'_p)dw_{p-1} \int_0^1 dw_p, \\ &= (e'_1 - d'_1) \int_0^1 (e'_2 - d'_2)dw_1 \int_0^1 (e'_3 - d'_3)dw_2 \cdots \int_0^1 (e'_p - d'_p)dw_{p-1}, \end{aligned} \quad (4.36)$$

where

$$\begin{aligned} d'_i &= \Phi\left(\frac{a_i - \sum_{j=1}^{i-1} L_{i,j}\Phi^{-1}(d_j + w_j(e_j - d_j))}{L_{i,i}}\right), \\ e'_i &= \Phi\left(\frac{b_i - \sum_{j=1}^{i-1} L_{i,j}\Phi^{-1}(d_j + w_j(e_j - d_j))}{L_{i,i}}\right). \end{aligned}$$

It is worth to note that in the RHS of (4.36) the quantity $e'_1 - d'_1$ is independent of w . For computing $\mathcal{P}_G$ Genz [1992] suggests a Monte Carlo approach in which w_i (for $i = 1, \dots, p-1$) is drawn from $\mathcal{U}(0, 1)$ as the instrumental distribution. This method as originally developed for zero-mean Gaussian distribution. In general, when $\boldsymbol{\mu}$ of, the constants $\mathbf{a}$ and $\mathbf{b}$ are replaced with $\mathbf{a} - \boldsymbol{\mu}$ and $\mathbf{b} - \boldsymbol{\mu}$, respectively. The number of Monte Carlo iterations N , typically is large and hence, the central limit theorem (CLT) guarantees that the standard error (se) for estimator of $\mathcal{P}_G$ is asymptotically

$$\text{se} = \alpha \frac{\sigma}{\sqrt{N}},$$

where σ is the standard deviation of the Monte Carlo approximation. Naturally σ is unknown and therefore is estimated based on N Monte Carlo iterations. The constant α is the Gaussian upper quantile such as $z_{0.995} \approx 2.576$ by which one expects the absolute error in approximating $\mathcal{P}_G$ to be less than given error in 99 percent of the time. Genz [1992] suggests the use of $\alpha = 2.5$. Algorithm 10 given below describes the method of Genz [1992] for computing $\mathcal{P}_G$ in more details.

Algorithm 10 Computing CDF of the multivariate Gaussian distribution

- 1: Read $\mathbf{a}$, $\mathbf{b}$, $\boldsymbol{\mu}$, Σ , ϵ , and N_{max} ;
 - 2: Compute the Cholesky decomposition L of Σ ;
 - 3: Set s to be a vector of zeros of length N_{max} and $\alpha = 2.5$;
 - 4: Set $\mathbf{b} = \mathbf{b} - \boldsymbol{\mu}$ and $\mathbf{a} = \mathbf{a} - \boldsymbol{\mu}$;
 - 5: Set $e_1 = \Phi(b_1/L_{1,1})$, $d_1 = \Phi(a_1/L_{1,1})$, and $f_1 = e_1 - d_1$;
 - 6: Set $N = 0$ and $\text{se} = 10$ (or any value greater than ϵ);
 - 7: **while** $\text{se} > \epsilon$ and $N < N_{max}$ **do**
 - 8: Generate realizations $w_1, w_2, \dots, w_{p-1}$ from $\mathcal{U}(0, 1)$;
 - 9: Set $i = 2$;
 - 10: **while** $i \leq p$ **do**
 - 11: Set $j = i - 1$;
 - 12: $y_j = \Phi^{-1}(d_j + w_j(e_j - d_j))$;
 - 13: $L_y = \sum_{k=1}^j L_{i,k} \times y_k$;
 - 14: $d_i = \Phi\left(\frac{a_i - L_y}{L_{i,i}}\right)$;
 - 15: $e_i = \Phi\left(\frac{b_i - L_y}{L_{i,i}}\right)$;
 - 16: $f_i = (e_i - d_i)f_j$;
 - 17: Set $i = i + 1$;
 - 18: **end**
 - 19: $N = N + 1$;
 - 20: $s[N] = f_p$ where $s[N]$ denotes the N th element of vector s ;
 - 21: $\sigma = \sqrt{\text{Var}(s[1 : N])}$;
 - 22: $\text{se} = \alpha \frac{\sigma}{\sqrt{N}}$;
 - 23: **end**
 - 24: Return $\sum_{r=1}^N s[r]/N$ as an estimator for the CDF of multivariate Gaussian distribution with standard error se .
-

In the following, the R function `pmvnorm_Genz` is written for implementing the Genz [1992] algorithm.

```

1 R> pmvnorm_Genz <- function(a, b, Mu, Sigma, epsilon = 10e-4, N_max = 200)
2   +{
3   + if(any(a >= b) == TRUE) stop(message("vector_a_must_be_smaller_than_vector_b."))

```

```

4 + p <- length(Mu)
5 + L <- t( chol(Sigma) )
6 + s <- rep(0, N_max)
7 + alpha <- 2.5
8 + a0 <- b0 <- d <- e <- f <- w <- y <- rep(0, p)
9 + a0 <- a - Mu
10 + b0 <- b - Mu
11 + k <- 5.32
12 + threshold <- k*sqrt( diag(Sigma) )
13 + for(i in 1:p)
14 + {
15 +   if( is.infinite( a0[i] ) ) a0[i] <- Mu[i] - threshold[i]
16 +   if( is.infinite( b0[i] ) ) b0[i] <- Mu[i] + threshold[i]
17 +   if( is.infinite( a[i] ) & a0[i] >= b0[i] ) a0[i] <- b0[i] - threshold[i]
18 +   if( is.infinite( b[i] ) & a0[i] >= b0[i] ) b0[i] <- a0[i] + threshold[i]
19 + }
20 + d[1] <- pnorm( a0[1]/L[1, 1] )
21 + e[1] <- pnorm( b0[1]/L[1, 1] )
22 + f[1] <- e[1] - d[1]
23 + se <- 10
24 + N <- 0
25 + while( se > epsilon | N < N_max )
26 + {
27 +   w <- runif(p - 1)
28 +   i <- 2
29 +   while( i <= p )
30 +   {
31 +     j <- i - 1
32 +     y[j] <- qnorm( d[j] + w[j]*(e[j] - d[j]) )
33 +     L_y <- sum( L[i, (1:j)]*y[1:j] )
34 +     d[i] <- pnorm( (a0[i] - L_y)/L[i, i] )
35 +     e[i] <- pnorm( (b0[i] - L_y)/L[i, i] )
36 +     f[i] <- (e[i] - d[i]) * f[j]
37 +     i <- i + 1
38 +   }
39 +   N <- N + 1
40 +   s[N] <- f[p]
41 +   sigma <- sqrt( var( s[1:N] ) )
42 +   se <- alpha*sigma/sqrt(N)
43 +   se <- ifelse( is.na(se), 10, se )
44 + }
45 + ls <- list("cdf" = mean(s), "se" = se)
46 + return(ls)
47 +}

```

4.1.3 Computing CDF of the multivariate Student's t distribution

Let $\mathbf{X} = (X_1, \dots, X_p)^\top \sim t_\nu(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ as defined in (4.28). The focus of this part concerns with computing $\mathcal{P}_t = P(\mathbf{a} < \mathbf{X} < \mathbf{b})$ based on method proposed by Genz and Bretz [1999]. Without loss of generality, we assume that $\boldsymbol{\mu} = \mathbf{0}$. If $\boldsymbol{\mu} \neq \mathbf{0}$, the constants $\mathbf{a}$ and $\mathbf{b}$ are replaced with $\mathbf{a} - \boldsymbol{\mu}$ and $\mathbf{b} - \boldsymbol{\mu}$, respectively. By definition we have

$$\mathcal{P}_t = \int_{\mathbf{a}}^{\mathbf{b}} t_\nu(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) d\mathbf{x} \quad (4.37)$$

$$= \frac{\Gamma(\frac{\nu+p}{2})}{(\nu\pi)^{\frac{p}{2}} \Gamma(\frac{\nu}{2}) |\boldsymbol{\Sigma}|^{\frac{1}{2}}} \int_{\mathbf{a}}^{\mathbf{b}} \left[1 + \frac{\delta(\mathbf{x}, \boldsymbol{\Sigma})}{\nu} \right]^{-\frac{\nu+p}{2}} d\mathbf{x}, \quad (4.38)$$

where $\mathbf{a}$ and $\mathbf{b}$ are two real constant vectors of size p such that $\mathbf{a} < \mathbf{b}$. Based on the SOV method, described earlier in Section 4.1.2 for computing the CDF of multivariate Gaussian

distribution, we have

$$\begin{aligned}\mathcal{P}_t &= \frac{\Gamma(\frac{\nu+p}{2})}{(\nu\pi)^{\frac{p}{2}}\Gamma(\frac{\nu}{2})} \int_{\mathbf{a}'}^{\mathbf{b}'} \left[1 + \frac{\sum_{i=1}^p y_i^2}{\nu}\right]^{-\frac{\nu+p}{2}} d\mathbf{y} \\ &= C_t(\nu, p) \int_{\mathbf{a}'}^{\mathbf{b}'} \left[1 + \frac{\sum_{i=1}^p y_i^2}{\nu}\right]^{-\frac{\nu+p}{2}} d\mathbf{y},\end{aligned}$$

where $a'_i = (a_i - \sum_{j=1}^{i-1} L_{i,j} y_j)/L_{i,i}$ and $b'_i = (b_i - \sum_{j=1}^{i-1} L_{i,j} y_j)/L_{i,i}$, for $i = 1, \dots, p$. Taking into account the fact that

$$\left[1 + \frac{\sum_{i=1}^p y_i^2}{\nu}\right]^{-\frac{\nu+p}{2}} = \left[1 + \frac{y_1^2}{\nu}\right]^{-\frac{\nu+p}{2}} \left[1 + \frac{y_2^2}{\nu + y_1^2}\right]^{-\frac{\nu+p}{2}} \cdots \left[1 + \frac{y_p^2}{\nu + \sum_{i=1}^{p-1} y_i^2}\right]^{-\frac{\nu+p}{2}},$$

it follows that

$$\mathcal{P}_t = C_t(\nu, p) \int_{\mathbf{a}'}^{\mathbf{b}'} \left[1 + \frac{y_1^2}{\nu}\right]^{-\frac{\nu+p}{2}} \left[1 + \frac{y_2^2}{\nu + y_1^2}\right]^{-\frac{\nu+p}{2}} \cdots \left[1 + \frac{y_p^2}{\nu + \sum_{i=1}^{p-1} y_i^2}\right]^{-\frac{\nu+p}{2}} d\mathbf{y}.$$

Applying, for $i = 1, \dots, p$, a change of variable of the form

$$u_i = y_i \frac{\sqrt{\nu + i - 1}}{\sqrt{v + \sum_{j=1}^{i-1} y_j^2}} \quad (4.39)$$

for which we have

$$dy_i = du_i \frac{\sqrt{v + \sum_{j=1}^{i-1} y_j^2}}{\sqrt{\nu + i - 1}}$$

Hence,

$$\begin{aligned}\mathcal{P}_t &= C_t(\nu, p) \int_{\mathbf{a}''}^{\mathbf{b}''} \left[1 + \frac{u_1^2}{\nu}\right]^{-\frac{\nu+p}{2}} \left[1 + \frac{u_2^2}{\nu + 1}\right]^{-\frac{\nu+p}{2}} \cdots \left[1 + \frac{u_p^2}{\nu + p - 1}\right]^{-\frac{\nu+p}{2}} d\mathbf{y} \\ &= C_t(\nu, p) \prod_{i=1}^p \int_{a_i''}^{b_i''} \left[1 + \frac{u_i^2}{\nu + i - 1}\right]^{-\frac{\nu+p}{2}} \times \prod_{i=1}^p du_i \frac{\sqrt{v + \sum_{j=1}^{i-1} y_j^2}}{\sqrt{\nu + i - 1}}\end{aligned} \quad (4.40)$$

where by convention $\sum_{j=1}^0 x_j^2 = 0$ and

$$a_i'' = a'_i \frac{\sqrt{\nu + i - 1}}{\sqrt{v + \sum_{j=1}^{i-1} y_j^2}} = \frac{a_i - \sum_{j=1}^{i-1} L_{i,j} y_j}{L_{i,i}} \frac{\sqrt{\nu + i - 1}}{\sqrt{v + \sum_{j=1}^{i-1} y_j^2}} \quad (4.41)$$

$$b_i'' = b'_i \frac{\sqrt{\nu + i - 1}}{\sqrt{v + \sum_{j=1}^{i-1} y_j^2}} = \frac{b_i - \sum_{j=1}^{i-1} L_{i,j} y_j}{L_{i,i}} \frac{\sqrt{\nu + i - 1}}{\sqrt{v + \sum_{j=1}^{i-1} y_j^2}}. \quad (4.42)$$

For a given integer $1 \leq i \leq p$, let $h(i) = v + \sum_{j=1}^i y_j^2$. Evidently $h(0) = \nu$ and it is easy to check, based on (4.40), that the recurrence relation

$$h(i-1) = h(i-2) \left[1 + \frac{u_{i-1}^2}{\nu + i - 2}\right], i = 2, \dots, p,$$

holds true and therefore

$$\begin{aligned}\prod_{i=1}^p du_i \frac{\sqrt{h(i-1)}}{\sqrt{\nu + i - 1}} &= du_1 \times \prod_{i=2}^p du_i \frac{\sqrt{h(i-1)}}{\sqrt{\nu + i - 1}} \\ &= du_1 \times \prod_{i=1}^{p-1} du_{i+1} \frac{\sqrt{\nu}}{\sqrt{\nu + i}} \left[1 + \frac{u_i^2}{\nu + i - 1}\right]^{\frac{p-i}{2}}.\end{aligned} \quad (4.43)$$

Substituting the RHS of (4.43) into the RHS of (4.40) yields

$$\begin{aligned}
\mathcal{P}_t &= C_t(\nu, p) \times \prod_{i=1}^{p-1} \frac{\sqrt{\nu}}{\sqrt{\nu+i}} \times \int_{a_1''}^{b_1''} \left[1 + \frac{u_1^2}{\nu}\right]^{-\frac{\nu+1}{2}} \times \int_{a_2''}^{b_2''} \left[1 + \frac{u_2^2}{\nu+1}\right]^{-\frac{\nu+2}{2}} \\
&\quad \times \cdots \times \int_{a_p''}^{b_p''} \left[1 + \frac{u_p^2}{\nu+p-1}\right]^{-\frac{\nu+p}{2}} du_p \cdots du_2 du_1 \\
&= C_t(\nu, p) \times \prod_{i=0}^{p-1} \frac{\sqrt{\nu}}{\sqrt{\nu+i} \times C_t(\nu+i, 1)} \times \int_{a_1''}^{b_1''} t_\nu(u_1) \times \int_{a_2''}^{b_2''} t_{\nu+1}(u_2) \\
&\quad \times \cdots \times \int_{a_p''}^{b_p''} t_{\nu+p-1}(u_p) du_p \cdots du_2 du_1.
\end{aligned}$$

Moreover, it is easy to check that

$$\begin{aligned}
C_t(\nu, p) \times \prod_{i=0}^{p-1} \frac{\sqrt{\nu}}{\sqrt{\nu+i} \times C_t(\nu+i, 1)} &= \frac{\Gamma(\frac{\nu+p}{2})}{(\nu\pi)^{\frac{p}{2}} \Gamma(\frac{\nu}{2})} \times \prod_{i=0}^{p-1} \frac{\sqrt{\nu}}{\sqrt{\nu+i}} \frac{[(\nu+i)\pi]^{\frac{1}{2}} \Gamma(\frac{\nu+i}{2})}{\Gamma(\frac{\nu+i+1}{2})} \\
&= \frac{\Gamma(\frac{\nu+p}{2})}{\Gamma(\frac{\nu}{2})} \times \prod_{i=0}^{p-1} \frac{\Gamma(\frac{\nu+i}{2})}{\Gamma(\frac{\nu+i+1}{2})} \\
&= 1.
\end{aligned}$$

Hence,

$$\mathcal{P}_t = \int_{a_1''}^{b_1''} t_\nu(u_1) \times \int_{a_2''}^{b_2''} t_{\nu+1}(u_2) \times \cdots \times \int_{a_p''}^{b_p''} t_{\nu+p-1}(u_p) du_p \cdots du_2 du_1.$$

Using transformation $z_i = T_{\nu+i-1}(u_i)$ in which u_i (for $i = 1, \dots, p$) is given by (4.39), it follows that

$$\mathcal{P}_t = \int_{d_1}^{e_1} \int_{d_2}^{e_2} \cdots \int_{d_p}^{e_p} dz_p \cdots dz_1, \quad (4.44)$$

where $d_i = T_{\nu+i-1}(a_i'')$ and $e_i = T_{\nu+i-1}(b_i'')$ in which constants a_i'' and b_i'' are given by (4.41) and (4.42), respectively. For computational purposes we may apply another change of variable $w_i = (z_i - d_i) / (e_i - d_i)$ by which $\mathcal{P}_t$ in (4.44) would be computed within a $(p-1)$ -dimensional unit hypercube $[0, 1]^{p-1}$. We have

$$\begin{aligned}
\mathcal{P}_t &= \int_0^1 (e_1 - d_1) dw_1 \cdots \int_0^1 (e_{p-1} - d_{p-1}) dw_{p-1} \int_0^1 (e_p - d_p) dw_p \\
&= (e_1 - d_1) \int_0^1 (e_2 - d_2) dw_1 \cdots \int_0^1 (e_p - d_p) dw_{p-1}.
\end{aligned} \quad (4.45)$$

Accomplishing $\mathcal{P}_t$ in (4.45) needs computing quantities $\{d_1, \dots, d_p\}$ and $\{e_1, \dots, e_p\}$ at each iteration of the Monte Carlo computation. By (4.41) and (4.42), the initial values d_1 and e_1 are $d_1 = T_\nu(a_1/L_{1,1})$ and $e_1 = T_\nu(b_1/L_{1,1})$. The reminders are recovered using a recurrence relation. Algorithm 11 given below explains how to compute the CDF of a Student's t distribution. In the following, the R function `pmvt_Genz` is given for implementing the algorithm proposed by Genz and Bretz [1999].

```

1 R> pmvt_Genz <- function(a, b, Mu, Sigma, nu, epsilon = 10e-4, N_max = 200)
2   + {
3   +   p <- length(Mu)
4   +   L <- t( chol(Sigma) )
5   +   alpha <- 2.5

```

Algorithm 11 Computing CDF of the Multivariate Student's t distribution

```

1: Read  $\mathbf{a}$ ,  $\mathbf{b}$ ,  $\boldsymbol{\mu}$ ,  $\Sigma$ ,  $\nu$ ,  $\epsilon$ , and  $N_{max}$ ;
2: Compute the Cholesky decomposition  $L$  of  $\Sigma$ ;
3: Set  $\mathbf{s}$  to be a vector of zeros of length  $N_{max}$  and  $\alpha = 2.5$ ;
4: Set  $\mathbf{b} = \mathbf{b} - \boldsymbol{\mu}$  and  $\mathbf{a} = \mathbf{a} - \boldsymbol{\mu}$ ;
5: Set  $e_1 = T_\nu(b_1/L_{1,1})$ ,  $d_1 = T_\nu(a_1/L_{1,1})$ , and  $f_1 = e_1 - d_1$ ;
6: Set  $N = 0$  and  $se = 10$ ;
7: while  $se > \epsilon$  and  $N < N_{max}$  do
8:   Generate realizations  $w_1, w_2, \dots, w_{p-1}$  from  $\mathcal{U}(0, 1)$ ;
9:   Set  $u_1 = T_\nu^{-1}(d_1 + w_1 f_1)$ ,  $u_1 = y_1$ , and  $ssy = 0$ ;
10:  Set  $ssy = ssy + y_1^2$ ;
11:  Set  $i = 2$ ;
12:  while  $i \leq p$  do
13:    Set  $j = i - 1$ ;
14:     $L_\nu = (\nu + j)^{\frac{1}{2}}(\nu + \sum_{k=1}^j y_k^2)^{-\frac{1}{2}}$ ;
15:     $a_i'' = \frac{a_i - \sum_{k=1}^j y_k \times L_{i,k}}{L_{i,i}} \times L_\nu$ ;
16:     $b_i'' = \frac{b_i - \sum_{k=1}^j y_k \times L_{i,k}}{L_{i,i}} \times L_\nu$ ;
17:     $d_i = T_{\nu+j}(a_i'')$ ;
18:     $e_i = T_{\nu+j}(b_i'')$ ;
19:     $f_i = (e_i - d_i) f_j$ ;
20:     $u_i = T_{\nu+j}^{-1}[d_i + w_i(e_i - d_i)]$ ;
21:     $y_i = u_i / L_\nu$ ;
22:     $ssy = ssy + y_i^2$ ;
23:    Set  $i = i + 1$ ;
24:  end
25:   $N = N + 1$ ;
26:   $s[N] = f_p$  where  $s[N]$  denotes the  $N$ th element of vector  $\mathbf{s}$ ;
27:   $\sigma = \sqrt{\text{Var}(s[1 : N])}$ ;
28:   $se = \alpha \frac{\sigma}{\sqrt{N}}$ ;
29: end
30: Return  $\sum_{r=1}^N s[r]/N$  as an estimator for the CDF of multivariate Student's  $t$  distribution
    with standard error  $se$ .

```

```

6 + s <- rep(0, N_max)
7 + a0 <- b0 <- d <- e <- f <- w <- y <- u <- rep(0, p)
8 + a0 <- a- Mu
9 + b0 <- b- Mu
10 + nu <- ifelse(nu >= 1000, 1000, nu)
11 + d[1] <- pt( q = a0[1]/L[1, 1], df = nu )
12 + e[1] <- pt( q = b0[1]/L[1, 1], df = nu )
13 + f[1] <- e[1] - d[1]
14 + se <- 10
15 + N <- 0
16 + while( se > epsilon | N < N_max )
17 + {
18 +   w <- runif(p- 1)
19 +   u[1] <- qt( p = d[1] + w[1]*(e[1] - d[1]), df = nu )
20 +   y[1] <- u[1]
21 +   ss_y <- 0;
22 +   ss_y <- ss_y + y[1]^2
23 +   i <- 2
24 +   while( i <= p )
25 +   {
26 +     j <- i - 1
27 +     L_nu <- sqrt( (nu + j)/(nu + ss_y) )
28 +     L_y <- sum( L[i, 1:j]*y[1:j] )
29 +     a0_second <- (a0[i] - L_y)/L[i, i]*L_nu
30 +     b0_second <- (b0[i] - L_y)/L[i, i]*L_nu
31 +     d[i] <- pt( q = a0_second, df = nu + j )
32 +     e[i] <- pt( q = b0_second, df = nu + j )
33 +     d[i] <- ifelse( d[i] == 0, .Machine$double.xmin, d[i] )
34 +     e[i] <- ifelse( e[i] == 1, 0.9999999, e[i] )
35 +     f[i] <- (e[i] - d[i])*f[j]
36 +     u[i] <- qt( p = d[i] + w[i]*(e[i] - d[i]), df = nu + j )
37 +     y[i] <- u[i]/L_nu
38 +     ss_y <- ss_y + y[i]^2
39 +     i <- i + 1
40 +   }
41 +   N <- N + 1
42 +   s[N] <- f[p]
43 +   sigma <- sqrt( var( s[1:N] ) )
44 +   se <- alpha*sigma/sqrt(N)
45 +   se <- ifelse( is.na(se), 10, se )
46 + }
47 + ls <- list("cdf" = mean(s), "se" = se)
48 + return(ls)
49 +}

```

4.2 Univariate Gaussian quadrature

The *quadrature rules* involve methods for approximating an integral

$$I = \int_a^b f(x)dx \approx \mathcal{Q}(f) = \sum_{i=1}^k \omega_i f(x_i) \quad (4.46)$$

where quantities ω_i and x_i are the i th *weight* and *node* of the quadrature rule, respectively. The idea behind a quadrature rule is to approximate the integrand $f(x)$ through a simple function. Reasonably, an appropriate choice is a polynomial $P(x)$, of degree not greater than $k - 1$ that interpolate $f(x)$ at x_i (for $i = 1, \dots, k$). Suppose, for a given function $f(x)$, we are interested in approximating the integral of the form $f(x)$ over interval (a, b) using a two-point ($k = 2$)

quadrature rule. We have

$$\int_a^b f(x)dx \approx \omega_1 f(x_1) + \omega_2 f(x_2), \quad (4.47)$$

where the value of above integral is known when all four unknown constants including ω_1 (first weight), ω_2 (second weight), x_1 (first node), and x_2 (second node) are determined. To this end, we assume $f(x)$ is a polynomial of degree three represented as $f(x) = a_0 + a_1x + a_2x^2 + a_3x^3$. Although we can let for larger values of m , but we set $m = 2$ for simplicity. So, substituting the third-order polynomial $f(x) = a_0 + a_1x + a_2x^2 + a_3x^3$ into the LHS of (4.47), it follows that

$$\begin{aligned} \int_a^b f(x) dx &= \int_a^b a_0 + a_1x + a_2x^2 + a_3x^3 dx \\ &= \left[a_0x + a_1\frac{x^2}{2} + a_2\frac{x^3}{3} + a_3\frac{x^4}{4} \right]_a^b \\ &= a_0(b-a) + a_1\left(\frac{b^2-a^2}{2}\right) + a_2\left(\frac{b^3-a^3}{3}\right) + a_3\left(\frac{b^4-a^4}{4}\right). \end{aligned} \quad (4.48)$$

Representing the RHS of (4.47) in terms of third-order polynomial yields

$$\omega_1 f(x_1) + \omega_2 f(x_2) = \omega_1(a_0 + a_1x_1 + a_2x_1^2 + a_3x_1^3) + \omega_2(a_0 + a_1x_2 + a_2x_2^2 + a_3x_2^3). \quad (4.49)$$

Equating the RHSs of (4.48) and (4.49) yields a set of four equations as follows.

$$\begin{cases} b-a = \omega_1 + \omega_2, \\ \frac{b^2-a^2}{2} = \omega_1x_1 + \omega_2x_2, \\ \frac{b^3-a^3}{3} = \omega_1x_1^2 + \omega_2x_2^2, \\ \frac{b^4-a^4}{4} = \omega_1x_1^3 + \omega_2x_2^3. \end{cases} \quad (4.50)$$

Solving the set of four equations given in (4.50), we have

$$\begin{aligned} \omega_1 &= \frac{b-a}{2}, \\ \omega_2 &= \frac{b-a}{2}, \\ x_1 &= -\frac{1}{\sqrt{3}}\left(\frac{b-a}{2}\right) + \frac{b+a}{2}, \\ x_2 &= \frac{1}{\sqrt{3}}\left(\frac{b-a}{2}\right) + \frac{b+a}{2}. \end{aligned} \quad (4.51)$$

Substitute the RHS of (4.51) into the RHS of (4.47) to see that

$$\begin{aligned} \int_a^b f(x)dx &\approx \omega_1 f(x_1) + \omega_2 f(x_2) \\ &= \frac{b-a}{2} f\left(-\frac{1}{\sqrt{3}}\frac{b-a}{2} + \frac{b+a}{2}\right) + \frac{b-a}{2} f\left(\frac{1}{\sqrt{3}}\frac{b-a}{2} + \frac{b+a}{2}\right). \end{aligned} \quad (4.52)$$

Following example shows how much a quadrature can be reliable.

Example 4-3

Let $f(x) = x \exp\{-x\}$ and we would like to approximate

$$I = \int_0^3 x \exp\{-x\} dx, \quad (4.53)$$

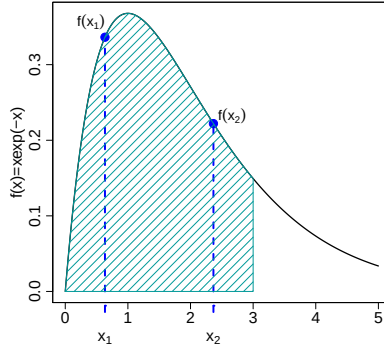


Figure 4.1: Plot of integrand $f(x) = x \exp\{-x\}$ and nodes x_1 and x_2 . The area under the integrand is shown by shaded area.

based on Gaussian quadrature of $m = 2$ points. We compute two weights and nodes through (4.50). It is easy to check that

$$I \approx 1.5 \times f(0.63397) + 1.5 \times f(2.36602) = 0.83755, \quad (4.54)$$

where the exact value of above integral is 0.80085. Figure 4.1 displays the integrand and the corresponding nodes. ■

The above reasoning can be extended for $k > 2$ that yields more accurate approximation of integral (4.47). In general, using the quadrature as described above, one can approximate the given definite integral using a quadrature rule with an arbitrary k numbers of nodes and weights. In general, we can write

$$I = \int_a^b f(x) dx = \int_a^b P(x) dx = \int_a^b \sum_{i=1}^k L_i(x) f(x_i) dx = \sum_{i=1}^k \omega_i f(x_i),$$

where

$$\omega_i = \int_a^b L_i(x) dx,$$

in which $L_i(x)$ is the well-known Lagrange interpolating polynomial of order $k - 1$. defined as

$$L_i(x) = \prod_{j=1, j \neq i}^n \frac{x - x_j}{x_j - x_i}.$$

In what follows, among several quadrature rules introduced in the literature, we briefly describe the *Gaussian quadrature* rules for computing a definite integral.

4.2.1 Gaussian quadrature rules

The quadrature rule presented in the previous sub-section works based on a general polynomial. Herein, we describe the well-known *Gaussian quadrature* that works based on a special class of polynomials known as the *orthogonal polynomials* and can be applied for approximating accurately a wide range of definite integrals [Dahlquist and Björck, 2003]. For presentation and theoretical purposes, the following definitions are useful.

Definition 4.2.1. [Foupouagnigni and Koepf, 2018] The function $w(\cdot) : (a, b) \rightarrow \mathbb{R}^+$ that satisfies the following three conditions is said to be a weight function.

i. $w(x)$ is measurable.

ii. The k th moment of $w(x)$ must be finite, that is $\int_a^b |x|^k w(x) dx < \infty$ for $k = 0, 1, 2, \dots$.

iii. For a given polynomial such as $g(x)$ if $\int_a^b g(x)w(x)dx = 0$ then we must have $g(x) = 0$.

Definition 4.2.2. [Foupouagnigni and Koepf, 2018] The sequence of polynomials $\{f_0(x), f_1(x), f_2(x), \dots\}$ constitutes a sequence of orthogonal polynomials with respect to weight function $w(\cdot) : (a, b) \rightarrow \mathbb{R}^+$ if we have

$$\langle f_n, f_m \rangle = \int_a^b f_n(x) f_m(x) w(x) dx = C_n \times \mathcal{D}_{m,n},$$

where $\langle \cdot, \cdot \rangle$ represents the inner product operator, C_n is a constant, and $\mathcal{D}_{m,n}$ is the Kronecker delta defined as

$$\mathcal{D}_{m,n} = \begin{cases} 1, & \text{if } m = n, \\ 0, & \text{if } m \neq n. \end{cases} \quad (4.55)$$

There are several types of the orthogonal polynomials such as *Chebyshev of first kind* ($T_{1,n}$), *Chebyshev of second kind* ($T_{2,n}$), *generalized Laguerre* ($\mathcal{GL}_n^\alpha$), *Hermite* (H_n), *Jacobi* ($J_n^{\alpha,\beta}$), *Laguerre* ($\mathcal{L}_n$), and *Legendre* (L_n), to name a few. In general, the orthogonal polynomials are solution of differential equation and obtained through a recurrence formula. In what follows, we describe briefly the *Hermite* and *Laguerre* classes of orthogonal polynomials with widespread applications in statistics.

- **Hermite polynomial of first kind:** the Hermite polynomial of degree n , shown by $H_{1,n}(x)$, is orthogonal with respect to weight function $w(x) = \exp\{-x^2\}$ over the range $(-\infty, \infty)$. This means that

$$\langle H_{1,n}, H_{1,m} \rangle = \int_{-\infty}^{\infty} H_{1,n}(x) H_{1,m}(x) \exp\{-x^2\} dx = 2^n \sqrt{\pi} \Gamma(n+1) \times \mathcal{D}_{m,n}. \quad (4.56)$$

The expansion representation for $H_{1,n}(x)$ is

$$H_{1,n}(x) = \Gamma(n+1) \sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} \frac{(-1)^i (2x)^{n-2i}}{\Gamma(i+1) \Gamma(n-2i+1)}, \quad (4.57)$$

where $\lfloor n/2 \rfloor$ denotes the greatest integer less or equal $n/2$ (for $n = 0, 1, 2, \dots$). This class of orthogonal polynomials satisfies with the recurrence relations

$$\frac{dH_{1,n}(x)}{dx} - 2xH_{1,n}(x) + H_{1,n+1}(x) = 0,$$

for $n \geq 0$ and

$$H_{1,n+1}(x) - 2xH_{1,n}(x) + 2nH_{1,n-1}(x) = 0, \quad (4.58)$$

for $n \geq 1$. Using expression (4.58), the first three members of Hermite polynomials are as follows [Arfken et al., 2011].

$$\begin{aligned} H_{1,0}(x) &= 1, \\ H_{1,1}(x) &= 2x, \\ H_{1,2}(x) &= 4x^2 - 2, \end{aligned}$$

Figure 4.2 displays the graph of Hermite (first kind) and Legendre polynomials of orders $n = 2, 3, 4, 5$.

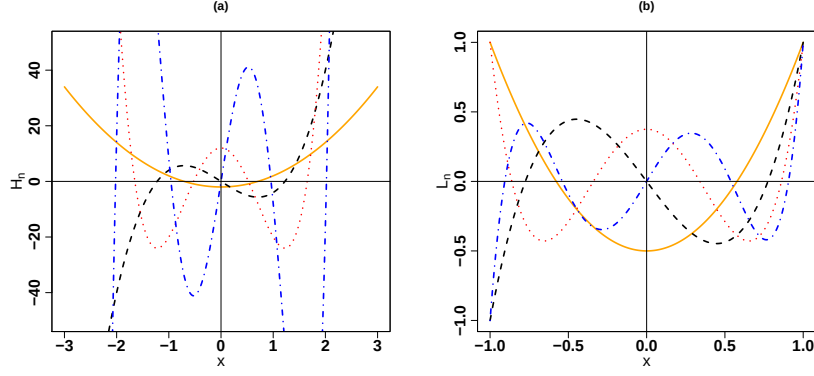


Figure 4.2: Graph of (a): Hermite and (b): Legendre polynomials of orders $n = 2$ (solid orange line), $n = 3$ (dashed black line), $n = 4$ (dotted red line), and $n = 5$ (dotted-dashed blue line).

- **Hermite polynomial of second kind:** the Hermite polynomial of second kind, shown by $H_{2,n}(x)$, is orthogonal with respect to weight function $w(x) = \exp\{-x^2\}$ over the range $(0, b)$ where b is a real positive value. This means that

$$\langle H_{2,n}, H_{2,m} \rangle = \int_0^b H_{2,n}(x) H_{2,m}(x) \exp\{-x^2\} dx = \mu_n \times \mathcal{D}_{m,n}, \quad (4.59)$$

where μ_n , for $n \geq 1$, meets the recurrence relation

$$2\mu_n = \mu_{n-1} - \left[\exp\{-x^2\} H_{2,n}(x) H_{2,n-1}(x) \right]_0^b, \quad (4.60)$$

with

$$\mu_0 = \frac{\sqrt{\pi}}{2} \text{erf}(b), \quad (4.61)$$

in which $\text{erf}(\cdot)$ is the well-known *error function* that can be expressed in terms of the standard Gaussian CDF as

$$\text{erf}(b) = \frac{2}{\sqrt{\pi}} \int_0^b \exp\{-u^2\} du = 2\Phi(\sqrt{2}b) - 1. \quad (4.62)$$

This class of orthogonal polynomials satisfies, for $n \geq 1$, with the recurrence relation

$$xH_{2,n}(x) - H_{2,n+1}(x) + \alpha_n H_{2,n}(x) + \beta_n H_{2,n-1}(x) = 0, \quad (4.63)$$

where three first polynomials are

$$\begin{aligned} H_{2,0}(x) &= 1, \\ H_{2,1}(x) &= x - \eta(b), \\ H_{2,2}(x) &= \left[x - \frac{\exp\{-b^2\} [b - \eta(b)]^2 - \eta^2(b)}{\beta_1 \sqrt{\pi} \text{erf}(b)} \right] [x - \eta(b)] + \beta_1, \end{aligned} \quad (4.64)$$

where

$$\beta_1 = -\frac{1}{2} + \frac{b}{\sqrt{\pi} \text{erf}(b)} - b\eta(b) + \eta^2(b) \quad (4.65)$$

in which

$$\eta(b) = \frac{1 - \exp\{-b^2\}}{\sqrt{\pi} \text{erf}(b)}. \quad (4.66)$$

Details for determining coefficients α_n and β_n in (4.63) will be discussed later.

- **Hermite polynomial of third kind:** the Hermite polynomial of third kind $H_{3,n}(x)$, is orthogonal with respect to weight function $w(x) = \exp\{-x^2\}$ over the range $(0, \infty)$. This means that

$$\langle H_{3,n}, H_{3,m} \rangle = \int_0^\infty H_{3,n}(x) H_{3,m}(x) \exp\{-x^2\} dx = \mu_n \times \mathcal{D}_{m,n}, \quad (4.67)$$

where μ_n , for $n \geq 1$, meets the recurrence relation

$$2\mu_n = n\mu_{n-1} + H_{3,n}(0)H_{3,n-1}(0), \quad (4.68)$$

with

$$\mu_0 = \frac{\sqrt{\pi}}{2}. \quad (4.69)$$

The class $H_{3,n}(x)$ satisfies, for $n \geq 1$, with the recurrence relation

$$xH_{3,n}(x) - H_{3,n+1}(x) + \alpha_n H_{3,n}(x) + \beta_n H_{3,n-1}(x) = 0, \quad (4.70)$$

where three first polynomials are

$$\begin{aligned} H_{3,0}(x) &= 1, \\ H_{3,1}(x) &= x - \frac{1}{\sqrt{\pi}}, \\ H_{3,2}(x) &= \left[x + \frac{2}{\sqrt{\pi}(2-\pi)} \right] \left[x - \frac{1}{\sqrt{\pi}} \right] + \frac{1}{\pi} - \frac{1}{2}. \end{aligned} \quad (4.71)$$

Details for determining coefficients α_n and β_n in (4.70) will be discussed later.

- **Laguerre polynomial:** The Laguerre polynomial of degree n , shown by $\mathcal{L}_n(x)$, is orthogonal with respect to weight function $w(x) = \exp\{-x\}$ over the range $(0, \infty)$. This means that

$$\langle \mathcal{L}_n, \mathcal{L}_m \rangle = \int_0^\infty \mathcal{L}_n(x) \mathcal{L}_m(x) \exp\{-x\} dx = \mathcal{D}_{m,n}. \quad (4.72)$$

This class of orthogonal polynomials is solution of the differential equation

$$x \frac{d^2 \mathcal{L}_n(x)}{dx^2} + (1-x) \frac{d \mathcal{L}_n(x)}{dx} + n \mathcal{L}_n(x) = 0, \quad (4.73)$$

whose solution becomes

$$\mathcal{L}_n(x) = \Gamma(n+1) \sum_{i=0}^n \frac{(-1)^i x^i}{\Gamma(n-i+1) [\Gamma(i+1)]^2}. \quad (4.74)$$

Using expression (4.74), the first three members of Laguerre polynomials are as follows.

$$\begin{aligned} \mathcal{L}_0(x) &= 1, \\ \mathcal{L}_1(x) &= 1 - x, \\ \mathcal{L}_2(x) &= \frac{x^2 - 4x + 2}{2}. \end{aligned}$$

Table 4.4 shows a list of most commonly used orthogonal polynomials in statistics. There

are different types of the Gaussian quadrature rules have been developed in the literature for

Table 4.4: Most commonly used orthogonal polynomials.

$P_n(\cdot)$	symbol	C_n	$[a, b]$	$w(x)$
Chebyshev (first kind)	$T_{1,n}$	$\begin{cases} \pi & \text{if } n = 0, \\ \frac{\pi}{2} & \text{if } n \neq 0. \end{cases}$	$(-1, 1)$	$(1 - x^2)^{-\frac{1}{2}}$
Chebyshev (second kind)	$T_{2,n}$	$\frac{\pi}{2}$	$[-1, 1]$	$(1 - x^2)^{\frac{1}{2}}$
Generalized Laguerre	$\mathcal{L}_n^\alpha$	$\frac{\Gamma(n+\alpha+1)}{\Gamma(n+1)}$	$[0, \infty)$	$x^\alpha \exp\{-x\},$ $\alpha > -1$
Hermite (first kind)	$H_{1,n}$	$2^n \sqrt{\pi} \Gamma(n+1)$	$(-\infty, \infty)$	$\exp\{-x^2\}$
Hermite (second kind)	$H_{2,n}$	no closed form, see (4.59)	$(0, b)$	$\exp\{-x^2\}$
Hermite (third kind)	$H_{3,n}$	no closed form, see (4.67)	$(0, \infty)$	$\exp\{-x^2\}$
Jacobi	$J_n^{\alpha,\beta}$	$\frac{2^{\alpha+\beta+1}}{2n+\alpha+\beta+1} \times \frac{\Gamma(n+\alpha+1)\Gamma(n+\beta+1)}{\Gamma(n+\alpha+\beta+1)\Gamma(n+1)}$	$(-1, 1)$	$(1-x)^\alpha(1+x)^\beta,$ $\alpha, \beta > -1$
Laguerre	$\mathcal{L}_n$	1	$[0, \infty)$	$\exp\{-x\}$
Legendre	L_n	$\frac{2}{2n+1}$	$[-1, 1]$	1

approximating the integral (4.75). Particular Gauss quadrature rule arises from particular choice of the interval and the form of the weight function $w(x)$. For example, the Gauss-Legendre quadrature rule is applied when $[a, b] = [-1, 1]$ and $w(x) = 1$, and so is employed for approximating integral of the form

$$\int_{-1}^1 f(x) dx.$$

Moreover, for approximating integral of the form

$$\int_0^\infty \exp\{-x\} f(x) dx,$$

we may use the Gauss-Laguerre quadrature rule since as it is seen the range of integral is $[a, b] = [0, \infty)$ and $w(x) = \exp\{-x\}$, and finally, a k -point Gauss-Chebyshev rule is useful when $[a, b] = [-1, 1]$. The bounds of integral (4.75) can be changed from (a, b) to $[-1, 1]$ through a simple transformation when $a \neq -1$ and $b \neq 1$. For integrand of the form $w(x)f(x)$, in which $w(x)$ is given in Table 4.4, an orthogonal polynomial up to degree $k+1$ is fitted to integrand $w(x)f(x)$ that intercepts integrand $w(x)f(x)$ at k nodes $\{x_n\}_{n=1}^k$. Once we have found k pairs of nodes and associated weights, shown by $\{(x_n, \omega_n)\}_{n=1}^k$, the integral can be approximated based on a k -point Gaussian quadrature rule as

$$\int_a^b w(x)f(x) dx \approx \sum_{n=1}^k \omega_n f(x_n). \quad (4.75)$$

- **weight:** Once we have computed the eigenvalues of A in (4.81), the n th weight ω_n is obtained as

$$\omega_n = \mu_0 \frac{(\mathbf{v}_n[1])^2}{\|\mathbf{v}_n\|}, \quad (4.82)$$

in which

$$\mu_0 = \int_a^b w(x)dx,$$

$\|\cdot\|$ denotes the Euclidean norm, $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ are the eigenvectors correspond to $\{e_1, \dots, e_k\}$, and $\mathbf{v}_n[1]$ is the first element of eigenvector $\mathbf{v}_n$.

In what follows, we compute the Gaussian quadrature rule for orthogonal polynomials presented in Table 4.4.

- **nodes and weights for generalized Laguerre polynomial:** The class $\mathcal{L}_n^\alpha(\cdot)$ of orthogonal polynomials admits the recurrence relation

$$(n+1)\mathcal{L}_{n+1}^\alpha(x) = (2n+\alpha+1-x)\mathcal{L}_n^\alpha(x) - (n+\alpha)\mathcal{L}_{n-1}^\alpha(x), \quad (4.83)$$

where $\mathcal{L}_0^\alpha(x) = 1$. Comparing the recurrence relations given in (4.80) and (4.83), we obtain $b_n = 0$, for $n = 0, 1, 2, \dots$, and

$$\begin{cases} a_n = -(n+1) \text{ and } b_n = (2n+\alpha+1), & \text{if } n \geq 0, \\ c_n = -(n+\alpha), & \text{if } n \geq 1. \end{cases} \quad (4.84)$$

In this case, matrix A in (4.81) can be reconstructed by using the constants a_n , b_n , and c_n provided in (4.84) for computing eigenvalues of the reconstructed matrix A as the nodes. For computing weights, it follows from Table 4.4 that

$$\mu_0 = \int_a^b w(x)dx = \int_0^\infty x^\alpha \exp\{-x\}dx = \Gamma(\alpha+1).$$

Hence the weights ω_n (for $n = 1, \dots, k$) are obtained by considering $\mu_0 = \Gamma(\alpha+1)$ in the RHS of (4.82) in which $\mathbf{v}_n$ (for $n = 1, \dots, k$) is the eigenvector corresponds to i th eigenvalue of matrix A constructed based on information given by (4.84).

- **nodes and weights for Hermite polynomial of first kind:** The class $H_{1,n}(\cdot)$ of orthogonal polynomials admits the recurrence relation given by (4.58) for $n \geq 1$. Comparing the recurrence relations given in (4.80) and (4.58), we obtain

$$\begin{cases} a_n = \frac{1}{2} \text{ and } b_n = 0, & \text{if } n \geq 0, \\ c_n = n, & \text{if } n \geq 1. \end{cases} \quad (4.85)$$

In this case, matrix A in (4.81) can be reconstructed by using the constants a_n , b_n , and c_n provided in (4.85) for computing eigenvalues of the reconstructed matrix A as the nodes. For computing weights, it follows from Table 4.4 that

$$\mu_0 = \int_a^b w(x)dx = \int_{-\infty}^\infty \exp\{-x^2\}dx = \sqrt{2\pi}.$$

Hence the weights ω_n (for $n = 1, \dots, k$) are obtained by considering $\mu_0 = \sqrt{2\pi}$ in the RHS of (4.82) in which $\mathbf{v}_n$ (for $n = 1, \dots, k$) is the eigenvector corresponds to i th eigenvalue of matrix A constructed based on information given by (4.85).

- **nodes and weights for Hermite polynomial of second kind:** The class $H_{2,n}(\cdot)$ of orthogonal polynomials admits the recurrence relation given by (4.60) for $n \geq 1$. Comparing the recurrence relations given in (4.80) and (4.63), we obtain $a_n = 1$ (for $n = 0, \dots, k-2$), $b_n = -\alpha_n$ (for $n = 0, \dots, k-1$), and $c_n = -\beta_n$ (for $n = 1, \dots, k-1$). Coefficients α_n and β_n are given by

$$\alpha_n = -\frac{1}{\mu_n} \int_0^b x \exp\{-x^2\} H_{2,n}^2(x) dx, \quad (4.86)$$

and

$$\beta_n = -\frac{\mu_n}{\mu_{n-1}}, \quad (4.87)$$

in which

$$\mu_n = \int_0^b \exp\{-x^2\} H_{2,n}^2(x) dx. \quad (4.88)$$

Under the orthogonality assumption of $H_{2,n}(\cdot)$, by multiplying both sides of (4.63) by $\exp\{-x^2\} H_{2,n-1}(x)$ and then integrating, we can see that μ_n in (4.88) can be represented as

$$\mu_n = \int_0^b x \exp\{-x^2\} H_{2,n}(x) H_{2,n-1}(x) dx. \quad (4.89)$$

Using integrating by parts and then applying the well-known Christoffel–Darboux [Darboux, 1878] formula, Steen et al. [1969] showed that μ_n in (4.89) admits the recursion relation (4.60). Using integration by parts, then relation (4.86) equivalently becomes

$$\alpha_n = \frac{1}{2\gamma_n} \left[\exp\{-x^2\} H_{2,n}^2(x) \right]_0^b. \quad (4.90)$$

In general, computing coefficients β_n in (4.87) and α_n in (4.90) requires $H_{2,n}$ to be known for $n \geq 2$. To this end, Algorithm 12 describes how to compute coefficients α_n and β_n or which the initial values are given by the following.

$$\begin{cases} H_{2,0}(0) = 1, & H_{2,1}(0) = -\eta(b), \\ H_{2,0}(b) = 1, & H_{2,1}(b) = b - \eta(b), \\ \mu_0 = \frac{\sqrt{\pi}}{2} \operatorname{erf}(b), & \mu_1 = \frac{\sqrt{\pi}}{4} \operatorname{erf}(b) - \frac{1}{2} \left[\exp\{-b^2\} [b - \eta(b)] + \eta(b) \right], \\ \alpha_1 = \frac{1}{2\mu_1} \left[\exp\{-b^2\} [b - \eta(b)]^2 - \eta^2(b) \right], & \beta_1 = -\frac{1}{2} + \frac{1}{\sqrt{\pi} \operatorname{erf}(b)} \left[\exp\{-b^2\} [b - \eta(b)] + \eta(b) \right]. \end{cases} \quad (4.91)$$

Using computed coefficients α_n and β_n based on Algorithm 12, the elements of matrix A in (4.81) are:

$$\begin{cases} a_n = 1 & \text{for } n = 0, \dots, k-2, \\ b_n = -\alpha_n & \text{for } n = 0, \dots, k-1, \text{ in which } \alpha_0 = -\eta(b), \\ c_n = -\beta_n & \text{for } n = 1, \dots, k-1. \end{cases}$$

Finally, taking into account the fact that

$$\mu_0 = \int_a^b w(x) dx = \int_0^b \exp\{-x^2\} dx = \frac{\sqrt{\pi}}{2} \operatorname{erf}(b),$$

the weights ω_n (for $n = 1, \dots, k$) are obtained using (4.82).

Algorithm 12 Computing first k coefficients of class $H_{2,n}(\cdot)$

```

1: Read  $k$  and  $b$ ;
2: Set  $h0$ ,  $hb$ ,  $\alpha$ ,  $\beta$ , and  $\mu$  to be vectors of length  $k$ ;
3: Set  $\alpha_1$ ,  $\beta_1$ , and  $\mu_1$  as given by (4.91);
4: Set  $H00 = H_{2,0}(0)$ , and  $H10 = H_{2,1}(0)$ ,  $H0b = H_{2,0}(b)$ , and  $H1b = H_{2,1}(b)$  in which
    $H_{2,0}(0)$ ,  $H_{2,1}(0)$ ,  $H_{2,0}(b)$ , and  $H_{2,1}(b)$  are given by (4.91);
5: Set  $n = 1$ 
6: while  $n \leq k - 1$  do
7:    $h0_{n+1} = \alpha_n \times H10 + \beta_n \times H00$ ;
8:    $hb_{n+1} = (b + \alpha_n) \times H1b + \beta_n \times H0b$ ;
9:    $\mu_{n+1} = \frac{n+1}{2}\mu_n - \frac{1}{2}[\exp\{-b^2\} \times hb_{n+1} \times H1b - h0_{n+1} \times H10]$ ;
10:   $\alpha_{n+1} = \frac{1}{2\mu_{n+1}}[\exp\{-b^2\} \times (hb_{n+1})^2 - (h0_{n+1})^2]$ ;
11:   $\beta_{n+1} = -\frac{\mu_{n+1}}{\mu_n}$ ;
12:   $H00 \leftarrow H10$  and  $H0b \leftarrow H1b$ ;
13:   $H10 \leftarrow h0_{n+1}$  and  $H1b \leftarrow hb_{n+1}$ ;
14:   $n \leftarrow n + 1$ ;
15: end
16: Return  $\{\alpha\}_{n=1}^k$  and  $\{\beta\}_{n=1}^k$ .

```

- **nodes and weighs for Hermite polynomial of third kind:** If we let $b \rightarrow +\infty$ in (4.59), then the class of Hermite polynomials of second kind turns into the class of Hermite polynomials of third kind denoted as $H_{3,n}(\cdot)$. Hence, the coefficients α_n , β_n , and μ_n in class $H_{3,n}(\cdot)$ are those of class $H_{2,n}(\cdot)$ when $b \rightarrow +\infty$. We have

$$\alpha_n = -\frac{1}{\mu_n} \int_0^\infty x \exp\{-x^2\} H_{2,n}^2(x) dx, \quad (4.92)$$

and

$$\beta_n = -\frac{\mu_n}{\mu_{n-1}}, \quad (4.93)$$

in which

$$\mu_n = \int_0^\infty \exp\{-x^2\} H_{2,n}^2(x) dx. \quad (4.94)$$

Using integration by parts for (4.92) or taking limit in (4.90) when $b \rightarrow +\infty$, one can see

$$\alpha_n = -\frac{H_{2,n}^2(0)}{2\mu_n}. \quad (4.95)$$

Likewise, when $b \rightarrow +\infty$ in (4.60), then μ_n in admits the recursion relation

$$\mu_n = \frac{n}{2}\mu_{n-1} + \frac{H_{2,n}(0)H_{2,n-1}(0)}{2}. \quad (4.96)$$

For $n \geq 2$, coefficients α_n in (4.95) and β_n in (4.93) are determined using the recurrence relation (4.60) and the initial values

$$\begin{cases} H_{2,0}(0) = 1, & H_{2,1}(0) = -\frac{1}{\sqrt{\pi}}, \\ \mu_0 = \frac{\sqrt{\pi}}{2}, & \mu_1 = \frac{\mu_0}{2} - \frac{1}{2\sqrt{\pi}}, \\ \alpha_1 = \frac{2}{\sqrt{\pi}(2-\pi)}, & \beta_1 = \frac{2-\pi}{2\pi}, \\ \alpha_0 = -\frac{1}{\sqrt{\pi}}. \end{cases} \quad (4.97)$$

To this end, Algorithm 13 describes how to compute coefficients α_n and β_n for $n = 2, \dots, k$. Using computed coefficients α_n and β_n , the elements of matrix A in (4.81) are:

Algorithm 13 Computing first k coefficients of class $H_{3,n}(\cdot)$

- 1: Read k and set h , α , β , and μ to be vectors of length k ;
 - 2: Set $\alpha_1 = 2/[\sqrt{\pi}(2 - \pi)]$, $\beta_1 = (2 - \pi)/(2\pi)$, and $\mu_1 = \sqrt{\pi}/4 - 1/(2\sqrt{\pi})$ using (4.97);
 - 3: Set $n = 1$, $H0 = H_{2,0}(0) = 1$, and $H1 = H_{2,1}(0) = -1/\sqrt{\pi}$ using (4.97);
 - 4: **while** $n \leq k - 1$ **do**
 - 5: $h_{n+1} = \alpha_n H1 + \beta_n H0$;
 - 6: $\mu_{n+1} = \frac{n+1}{2}\mu_n + \frac{1}{2}h_{n+1}H1$;
 - 7: $\alpha_{n+1} = -\frac{1}{2\mu_{n+1}}h_{n+1}^2$;
 - 8: $\beta_{n+1} = -\frac{\mu_{n+1}}{\mu_n}$;
 - 9: $H0 \leftarrow H1$;
 - 10: $H1 \leftarrow h_{n+1}$;
 - 11: Set $n = n + 1$;
 - 12: **end**
 - 13: Return $\{\alpha\}_{n=1}^k$ and $\{\beta\}_{n=1}^k$.
-

$$\begin{cases} a_n = 1 & \text{for } n = 0, \dots, k-2, \\ b_n = -\alpha_n & \text{for } n = 0, \dots, k-1, \text{ in which } \alpha_0 = -\frac{1}{\sqrt{\pi}}, \\ c_n = -\beta_n & \text{for } n = 1, \dots, k-1. \end{cases}$$

Finally, taking into account the fact that

$$\mu_0 = \int_a^b w(x)dx = \int_0^\infty \exp\{-x^2\}dx = \frac{\sqrt{\pi}}{2},$$

the weights ω_n (for $n = 1, \dots, k$) are obtained using (4.82). Since **R** does not have enough binary digits in order to represent results whose digits are more than 22, hence we cannot guarantee the precision of the computed nodes and weights for large k , > 10 say. Of course one can use the **Maple** framework for computing the nodes and weights of the Gauss-Hermite rule. The input parameters of the corresponding procedure, called **GH2**, consist of the number of nodes and b . The output consists of vector of weights, vector of nodes, and EC criterion, respectively.

Table 4.5: The nodes and weights associated with the Gauss-Hermite rule of third kind.

n	$k = 10$			$k = 12$			$k = 14$		
	ω_n	x_n	x_n	ω_n	x_n	x_n	ω_n	x_n	x_n
1	9.85520975191087E-02	3.87385243257289E-02	7.62461468014692E-02	2.98897007730461E-02	6.12109822716413E-02	2.39567896629936E-02	6.12109822716413E-02	2.39567896629936E-02	2.39567896629936E-02
2	2.08678066608185E-01	1.98233304013083E-01	1.66446068894088E-01	1.54204878281815E-01	1.66446068894088E-01	1.24240346144723E-01	1.36062060620609E-01	1.24240346144723E-01	1.24240346144723E-01
3	2.52051688403761E-01	4.65201111814767E-01	2.19394898138567E-01	3.66143963007318E-01	2.19394898138567E-01	2.97338573288085E-01	1.88856803527084E-01	2.97338573288085E-01	2.97338573288085E-01
4	1.98684340038387E-01	8.16861885592273E-01	2.07016508675540E-01	6.50881015894534E-01	2.07016508675540E-01	5.33329221273305E-01	1.98577829123488E-01	5.33329221273305E-01	5.33329221273305E-01
5	9.71984227600620E-02	1.23454132402818E+00	1.37264362783550E-01	9.94366869943082E-01	1.37264362783550E-01	8.21873198117369E-01	1.58617337872050E-01	8.21873198117369E-01	8.21873198117369E-01
6	2.70244164355446E-02	1.70679814968913E+00	6.05056743380072E-02	1.38589120372088E+00	6.05056743380072E-02	1.15406708458062E+00	9.28167828948399E-02	1.15406708458062E+00	1.15406708458062E+00
7	3.80464962249537E-03	2.22994008892494E+00	1.65538019519272E-02	1.81884860850511E+00	1.65538019519272E-02	1.52327480337614E+00	3.79316390125047E-02	1.52327480337614E+00	1.52327480337614E+00
8	2.2886243044656E-04	2.80910374689875E+00	2.58608378742107E-03	2.29084273875259E+00	2.58608378742107E-03	1.92533822320942E+00	1.02563910691812E-02	1.92533822320942E+00	1.92533822320942E+00
9	4.34534479844469E-06	3.46387241949586E+00	2.06237540974292E-04	2.80409679347421E+00	2.06237540974292E-04	2.35860077983781E+00	1.72277180701059E-03	2.35860077983781E+00	2.35860077983781E+00
10	1.24773714817825E-08	4.25536180636608E+00	7.06650986370700E-06	3.36727070424289E+00	7.06650986370700E-06	2.82409376823402E+00	1.65956340534487E-04	2.82409376823402E+00	2.82409376823402E+00
11			7.59131546779026E-08	4.00168347575167E+00	7.59131546779026E-08	3.32626937208736E+00	8.19589322531928E-06	3.32626937208736E+00	3.32626937208736E+00
12			1.18195417081408E-10	4.76821628806517E+00	1.18195417081408E-10	3.87510500420455E+00	1.738766608495078E-07	3.87510500420455E+00	3.87510500420455E+00
13						4.49243808452490E+00	1.14293978310768E-09	4.49243808452490E+00	4.49243808452490E+00
14						5.23843137515097E+00	1.04120010017399E-12	5.23843137515097E+00	5.23843137515097E+00
EC = 25043×10^{-19}			EC = 36052×10^{-24}			EC = 37125×10^{-29}			

¹ The CE is a short form of the error coefficient given by (4.104).

```

> restart: with(LinearAlgebra)[`<`,`>`(Eigenvalues, Eigenvectors)]:
> GH2 := proc (k::integer, b)
  local alpha, A, beta, bn, cn, d, e, EC, H0, H1,
    H2,i, j, l, mu0, mu, n, node, u, V, weight;
  interface(rtablesize = k):
  node := Vector(k): weight := Vector(k): H2 := Vector(k):
  mu := Vector(k): alpha := Vector(k): beta := Vector(k):
  cn := Vector(k-1): bn := Vector(k):
  A := Matrix(k, k, fill = 0):
  H0 := 1: H1 := x-(1-exp(-b^2))/(sqrt(Pi)*erf(b)):
  mu0 := evalf( (1/2)*sqrt(Pi)*erf(b) ):
  bn[1] := evalf( -(1/2)*(exp(-b^2)-1)/mu0 ):
  mu[1] := evalf( int(exp(-x^2)*H1^2, x=0 .. b) ):
  beta[1] := evalf( -mu[1]/mu0 ):
  alpha[1] := evalf( -(int(x*exp(-x^2)*H1^2, x = 0 .. b))/mu[1] ):
  for n to k-1 do:
    bn[n+1] := -alpha[n]:
    cn[n] := sqrt(-beta[n]):
    H2 := (x + alpha[n])*H1 + beta[n]*H0:
    mu[n+1] := evalf( int(exp(-x^2)*H2^2, x = 0 .. b) ):
    beta[n+1] := evalf( -mu[n+1]/mu[n] ):
    alpha[n+1] := evalf( -(int(x*exp(-x^2)*H2^2, x = 0 .. b))/mu[n+1] ):
    H0 := H1:
    H1 := H2:
  end do:
  d := Vector(bn): u := Vector(cn): l := Vector(cn):
  A := Matrix(k, proc (i, j) options operator, arrow:
    if i = j then d[i] elif i = j-1 then u[j-1]
    elif i = j+1 then l[i-1] else 0 end if end proc):
  e := Eigenvectors(A):
  node := Re(e[1]):
  V := e[2]:
  for n to k do:
    weight[n] := evalf( mu0*Re(V[1, n])^2/Norm(V[..,n], Euclidean) ):
  end do:
  EC := evalf( mu[k]/GAMMA(2*k+1) ):
  weight, node, EC:
end proc

```

- **nodes and weighs for Jacobi polynomial:** The class $J_i^{\alpha,\beta}(\cdot)$ of orthogonal polynomials admits the recurrence relation

$$J_{n+1}^{\alpha,\beta}(x) = (xa_n^{\alpha,\beta} - b_n^{\alpha,\beta})J_n^{\alpha,\beta}(x) - c_n^{\alpha,\beta}J_{n-1}^{\alpha,\beta}(x), \quad (4.98)$$

with $J_0^{\alpha,\beta}(x) = 1$ and $J_1^{\alpha,\beta}(x) = x(\alpha + \beta + 2)/2 + (\alpha - \beta)/2$, for $n = 1, 2, \dots$. Furthermore,

$$\begin{aligned} a_n^{\alpha,\beta} &= \frac{(2n + \alpha + \beta + 1)(2n + \alpha + \beta + 2)}{2(n + 1)(n + \alpha + \beta + 1)}, \\ b_n^{\alpha,\beta} &= \frac{(\beta^2 - \alpha^2)(2n + \alpha + \beta + 1)}{2(n + 1)(n + \alpha + \beta + 1)(2n + \alpha + \beta)}, \\ c_n^{\alpha,\beta} &= \frac{(n + \alpha)(n + \beta)(2n + \alpha + \beta + 2)}{(n + 1)(n + \alpha + \beta + 1)(2n + \alpha + \beta)}. \end{aligned} \quad (4.99)$$

Rearranging the RHS of (4.98) and comparing it with the recurrence relation given in

(4.80), we obtain

$$\begin{cases} a_0 = \frac{2}{\alpha+\beta+2} \text{ and } b_0 = \frac{\beta-\alpha}{\alpha+\beta+2}, & \text{if } n = 0, \\ a_n = \frac{2(n+1)(n+\alpha+\beta+1)}{(2n+\alpha+\beta+1)(2n+\alpha+\beta+2)}, & \text{if } n \geq 1, \\ b_n = \frac{\beta^2-\alpha^2}{(2n+\alpha+\beta)(2n+\alpha+\beta+2)}, & \text{if } n \geq 1, \\ c_n = \frac{2(n+\alpha)(n+\beta)}{(2n+\alpha+\beta)(2n+\alpha+\beta+1)}, & \text{if } n \geq 1. \end{cases} \quad (4.100)$$

In this case, matrix A in (4.81) can be reconstructed by using the constants a_n , b_n , and c_n provided in (4.100). As before, the nodes are eigenvalues of the reconstructed matrix A and for computing weights, it follows from Table 4.4 that

$$\mu_0 = \int_a^b w(x)dx = \int_{-1}^1 (1-x)^\alpha(1+x)^\beta dx = \frac{2^{\alpha+\beta+1}\Gamma(\alpha+1)\Gamma(\beta+1)}{\Gamma(\alpha+\beta+2)}.$$

Hence the weights ω_n (for $n = 1, \dots, k$) are obtained by considering $\mu_0 = 2$ in the RHS of (4.82) in which $\mathbf{v}_n$ (for $n = 1, \dots, k$) is the eigenvector corresponds to n th eigenvalue of matrix A constructed based on information given by (4.100).

- **nodes and weights for Legendre polynomial:** The class $L_n(\cdot)$ of orthogonal polynomials admits the recurrence relation

$$xL_n(x) = \frac{n+1}{2n+1}L_{n+1}(x) + \frac{n}{2n+1}L_{n-1}(x), \quad (4.101)$$

with $L_0(x) = 1$ and $L_1(x) = x$, for $n = 1, 2, \dots$. Comparing the recurrence relations given in (4.80) and (4.101), we obtain $b_n = 0$, for $n = 0, 1, 2, \dots$, and

$$\begin{cases} a_n = \frac{n+1}{2n+1}, & \text{if } n \geq 0, \\ c_n = \frac{n}{2n+1}, & \text{if } n \geq 1. \end{cases} \quad (4.102)$$

In this case, matrix A in (4.81) can be reconstructed by setting $\sqrt{a_{n-1}c_n} = n/\sqrt{4n^2-1}$ (for $n = 1, \dots, k$) in the RHS of (4.81). The nodes are eigenvalues of the reconstructed matrix A . For computing weights, it follows from Table 4.4 that

$$\mu_0 = \int_a^b w(x)dx = \int_{-1}^1 dx = 2.$$

Hence the weights ω_n (for $n = 1, \dots, k$) are obtained by considering $\mu_0 = 2$ in the RHS of (4.82) in which $\mathbf{v}_n$ (for $n = 1, \dots, k$) is the eigenvector corresponds to n th eigenvalue of matrix A constructed based on information given by (4.102).

It should be noted that the accuracy of a Gaussian quadrature depends on the structure of the orthogonal polynomial. In general, if $f(x)$ in (4.75) has continuous derivatives of order $2k$, then the error in approximating integral (4.75) becomes [Kahaner et al., 1989]:

$$\begin{aligned} \text{Error} &= \int_a^b w(x)f(x)dx - \sum_{i=1}^k \omega_i f(x_i) \\ &= \frac{1}{\Gamma(2k+1)} \int_a^b w(x)P_k^2(x)dx \times f^{(2k)}(y) \\ &= \frac{\mu_n}{\Gamma(2k+1)} f^{(2k)}(y), \end{aligned} \quad (4.103)$$

where $f^{(2k)}(y)$ denotes the $(2k)$ th derivative of $f(x)$ for some $a < y < b$. Regardless of $f^{(2k)}(y)$ in RHS of (4.103), the multiplicative term

$$\text{EC} = \frac{\mu_n}{\Gamma(2k+1)}, \quad (4.104)$$

is called error coefficient (EC). It should be noted that, in special case, when $w(x) = 1$, we have

$$\text{Error} = \frac{(b-a)^{2k+1} [\Gamma(k+1)]^4}{(2k+1) [\Gamma(2k+1)]^3} f^{(2k)}(y).$$

If the exact value of integral I is known, then a useful criterion, the goodness of approximation $\hat{I}$ for I is reflected by absolute relative error (ARE) criteria defined as

$$\text{ARE} = \left| \frac{I - \hat{I}}{I} \right|. \quad (4.105)$$

The following R function `quad_rule` given below computes the nodes and weights for implementing the Gaussian quadrature rules discussed earlier. We note that the argument `type` includes: CH1 (for Chebyshev of first kind), CH2 (for Chebyshev of second kind), HE1 (for Hermite of first kind), HE2 (for Hermite of second kind), HE3 (for Hermite of third kind), GL (for generalized Laguerre), JA (for Jacobi), and LE (for Legendre). The arguments `alpha` and `beta` are parameters of Gaussian-generalized Laguerre (or Gauss-Jacobi) rule and argument `b` is the parameter of Gauss-Hermite of third kind rule.

```

1  R> quad_rule <- function(k, type = "HE2", alpha = 0, beta = 0, b = Inf)
2  + {
3  +   if ( any( c(alpha, beta) <= -1 ) ) stop("_alpha_and_beta_must_be_greater_than_-1.")
4  +   if ( type == "HE3" & is.infinite(b) == T ) stop("_b_must_be_finite.")
5  +   A <- diag(0, k)
6  +   if ( type == "CH1" )
7  +   {
8  +     ia <- 1:k
9  +     node <- cos( (2*ia - 1)/(2*k)*pi )
10 +     weight <- pi/k
11 +   }
12 +   if ( type == "CH2" )
13 +   {
14 +     ia <- 1:k
15 +     node <- cos( ia/(k+1)*pi )
16 +     weight <- pi/(k+1)*( sin( ia/(k+1)*pi ) )^2
17 +   }
18 +   if ( type == "HE1" )
19 +   {
20 +     a_nc_n <- 1:(k-1)/2
21 +     A[row(A) - col(A) == 1] <- A[row(A) - col(A) == -1] <- sqrt( a_nc_n )
22 +     A_eigen <- eigen(A)
23 +     node <- A_eigen$values
24 +     weight <- sqrt(pi)*( A_eigen$vector[1, ] )^2
25 +   }
26 +   if ( type == "HE2" )
27 +   {
28 +     alpha <- beta <- mu <- h_0 <- h_b <- rep(0, k-1)
29 +     erf <- 2*pnorm( sqrt(2)*b ) - 1
30 +     eta <- ( 1 - exp(-b^2) )/( sqrt(pi)*erf )
31 +     H0_0 <- H0_b <- 1
32 +     H1_0 <- -eta
33 +     H1_b <- b - eta
34 +     mu_0 <- sqrt(pi)*erf/2
35 +     mu[1] <- mu_0/2 - 1/2*( exp(-b^2)*H1_b*H0_b - H1_0*H0_0 )
36 +     beta[1] <- -mu[1]/mu_0
37 +     alpha[1] <- 1/(2*mu[1])*( exp(-b^2)*(H1_b)^2 - (H1_0)^2 )
38 +     for(n in 1:(k-1) )
39 +     {
40 +       h_0[n+1] <- alpha[n]*H1_0 + beta[n]*H0_0
41 +       h_b[n+1] <- (b + alpha[n])*H1_b + beta[n]*H0_b
42 +       mu[n+1] <- (n+1)/2*mu[n] - 1/2*( exp(-b^2)*h_b[n+1]*H1_b - h_0[n+1]*H1_0 )

```

```

43 + alpha[n + 1] <- 1/(2*mu[n + 1])*( exp(-b^2)*( h_b[n + 1] )^2 - ( h_0[n + 1] )^2 )
44 + beta[n + 1] <- -mu[n + 1]/mu[n]
45 + H0_0 <- H1_0
46 + H0_b <- H1_b
47 + H1_0 <- h_0[n + 1]
48 + H1_b <- h_b[n + 1]
49 + }
50 + b_n <- abs( c( eta, -alpha[1:(k - 1)] ) )
51 + a_nc_n <- -beta[1:(k - 1)]
52 + A[row(A) - col(A) == 1] <- A[row(A) - col(A) == -1] <- sqrt( abs(a_nc_n) )
53 + diag(A) <- b_n
54 + A_eigen <- eigen(A)
55 + node <- A_eigen$values
56 + weight <- mu_0*( A_eigen$vector[1, ] )^2
57 + }
58 + if ( type == "HE3" )
59 + {
60 + alpha <- beta <- mu <- h <- rep(0, k - 1)
61 + H0 <- 1
62 + H1 <- -1/sqrt(pi)
63 + alpha_1 <- 2/( sqrt(pi)*(2 - pi) )
64 + alpha[1] <- alpha_1
65 + beta[1] <- (2 - pi)/(2*pi)
66 + mu_1 <- sqrt(pi)/4 - 1/( 2*sqrt(pi) )
67 + mu[1] <- mu_1
68 + for(n in 1:(k - 1) )
69 + {
70 + h[n + 1] <- alpha[n]*H1 + beta[n]*H0
71 + mu[n + 1] <- (n + 1)/2*mu[n] + h[n + 1]*H1/2
72 + alpha[n + 1] <- -1/(2*mu[n + 1])*( h[n + 1] )^2
73 + beta[n + 1] <- -mu[n + 1]/mu[n]
74 + H0 <- H1
75 + H1 <- h[n + 1]
76 + }
77 + b_n <- abs( c( 1/sqrt(pi), -alpha[1:(k - 1)] ) )
78 + a_nc_n <- -beta[1:(k - 1)]
79 + A[row(A) - col(A) == 1] <- A[row(A) - col(A) == -1] <- sqrt( abs( a_nc_n ) )
80 + diag(A) <- b_n
81 + A_eigen <- eigen(A)
82 + node <- A_eigen$values
83 + weight <- sqrt(pi)/2*( A_eigen$vector[1, ] )^2
84 + }
85 + if ( type == "GL" )
86 + {
87 + ib <- seq(0, k - 1)
88 + b_n <- 2*ib + alpha + 1
89 + ia <- seq(0, k - 2)
90 + diag(A) <- b_n
91 + A[row(A) - col(A) == 1] <- A[row(A) - col(A) == -1] <- sqrt( (ia + 1)*(ia + 1 + alpha) )
92 + A_eigen <- eigen(A)
93 + node <- A_eigen$values
94 + weight <- gamma(alpha + 1)*( A_eigen$vector[1, ] )^2
95 + }
96 + if ( type == "JA" )
97 + {
98 + ib <- seq(1, k - 1)
99 + b_n <- (beta^2 - alpha^2)/( (2*ib + alpha + beta)*(2*ib + alpha + beta + 2) )
100 + ia <- seq(1, k - 2)
101 + a_n <- 2*(ia+1)*(ia+alpha+beta+1)/( (2*ia+alpha+beta+1)*(2*ia+ alpha+beta+2) )
102 + ic <- ia + 1
103 + c_n <- 2*(ic + alpha)*(ic + beta)/( (2*ic + alpha + beta)*(2*ic + alpha + beta + 1) )
104 + diag(A) <- c(0, b_n )
105 + A[row(A) - col(A) == 1] <- A[row(A) - col(A) == -1] <- c(0, sqrt( a_n*c_n ) )

```

```

106 + A[1, 1] <- (beta - alpha)/( alpha + beta + 2 )
107 + A[1, 2] <- sqrt( 4*(1 + alpha)*(1 + beta )/( (2 + alpha + beta)^2*(3 + alpha + beta) ) )
108 + A[2, 1] <- A[1, 2]
109 + A_eigen <- eigen(A)
110 + node <- A_eigen$values
111 + weight <- 2^(alpha + beta + 1)*gamma(alpha + 1)*gamma(beta + 1)/
112 + gamma(alpha + beta + 2)*( A_eigen$vector[1, ] )^2
113 + }
114 + if ( type == "LE" )
115 + {
116 + ia <- seq(0, k - 2)
117 + a_n <- (ia + 1)/(2*ia + 1)
118 + ic <- ia + 1
119 + c_n <- ic/(2*ic + 1)
120 + A[row(A) - col(A) == 1] <- A[row(A) - col(A) == -1] <- sqrt( a_n*c_n )
121 + A_eigen <- eigen(A)
122 + node <- A_eigen$values
123 + weight <- 2*( A_eigen$vector[1, ] )^2
124 + }
125 + ls <- list( "node" = node , "weight" = weight)
126 + return(ls)
127 + }

```

Example 4-4

Let's to approximate integral (4.53) of Example 4-3 through the Gaussian-Legendre rule. To this end, first, we need to convert the range of integration from $[0, 3]$ to $[-1, 1]$. Using a change of variable $z = 2u/3 - 1$ it turns out that

$$\begin{aligned}
I &= \int_0^3 x \exp\{-x\} dx = \frac{9}{4} \exp\left\{-\frac{3}{2}\right\} \int_{-1}^1 (z + 1) \exp\left\{-\frac{3}{2}z\right\} dz \\
&= \frac{9}{4} \exp\left\{-\frac{3}{2}\right\} \int_{-1}^1 w(z) f(z) dz,
\end{aligned} \tag{4.106}$$

where $w(z) = 1$ and $f(z) = (z + 1) \exp\{-3z/2\}$. Hence, the sequence of nodes and weights can be computed using function `quad_rule`, when `type="LE"`. Computing the sequence $\{x_i, \omega_i\}_{i=1}^k$, we can write

$$I \approx \frac{9}{4} \exp\left\{-\frac{3}{2}\right\} \sum_{i=1}^k \omega_i \times (x_i + 1) \exp\left\{-\frac{3}{2}x_i\right\}. \tag{4.107}$$

The pertaining AREs for $k = 2, 3, 5$, and 10 are $4.58\text{e-}02$, $1.50\text{e-}03$, $1.75\text{e-}07$, and $3.18\text{e-}15$, respectively. We remind that the general quadrature rule applied in Example 4-3 yields 0.83755 with $\text{ARE} = 0.045$. Evidently, the Gaussian quadrature shows superior performance with $\text{ARE} = 0.045$ just by adding one more node. ■

Example 4-5

Suppose we are interested in approximating

$$I(p) = \int_0^\infty x^p \log x \exp\{-x\} dx, \tag{4.108}$$

where $p > 0$. Since the range of integrand I is $(0, \infty)$ and involves the term $\exp\{-x\}$, using class $\mathcal{L}_k^p$ of orthogonal polynomials, a Gaussian quadrature rule is applied for approximating $I(p)$ for different settings of k and p . To this end, we compute the sequence of nodes and weights using function `quad_rule`, when `type="GL"` and `alpha=p`. Computing sequence $\{x_i, \omega_i\}_{i=1}^k$, we approximate integral $I(p)$ as follows.

$$I(p) \approx \sum_{i=1}^k \omega_i \times (x_i)^p \log x_i. \quad (4.109)$$

Table 4.6 shows the ARE results for computing integral $I(p)$ in (4.108). It turns out from Table 4.6 that for larger p , smaller k provides the accuracy at desired level. Figure 4.3 shows the

p	Exact value of $I(p)$	ARE			
		$k = 5$	$k = 10$	$k = 15$	$k = 20$
0	-0.5772156	0.214375	0.108274	7.24e-02	5.44e-02
1	0.42278433	4.50e-01	0.012404	5.70e-03	3.26e-03
2	1.84556867	5.69e-03	9.12e-04	2.96e-04	1.30e-04
3	7.53670601	1.53e-03	1.51e-04	3.54e-05	1.22e-05
4	36.1468240	5.64e-04	3.55e-05	6.14e-06	1.68e-06

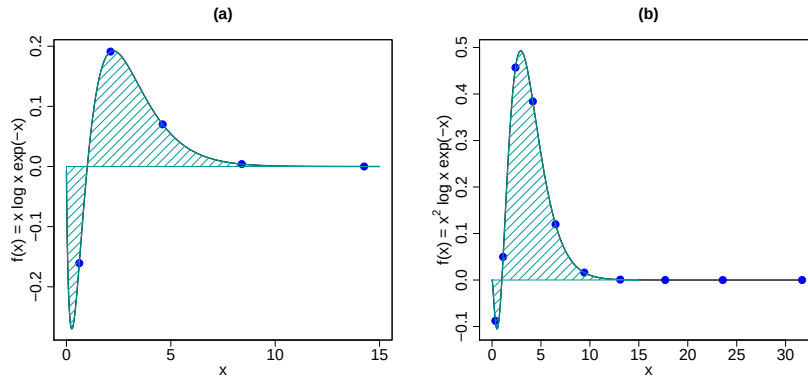


Figure 4.3: Graph of integrand (4.108). (a): $p = 1$ and $k = 5$, (b): $p = 2$ and $k = 10$. integrand of integral $I(p)$ in (4.108) on which x-coordinates of bullets are the position of the nodes. ■

Example 4-6

Suppose we are interested in approximating

$$I(z, \alpha) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp\{-u^\alpha\} \cos(zu) du, \quad (4.110)$$

where $0 < \alpha \leq 2$. In fact, integral $I(z, \alpha)$ is the PDF of symmetric standard α -stable distribution at point z [Nolan, 2020]. For $\alpha = 2$, each standard symmetric α -stable distribution turns into a zero-mean Gaussian distribution with scale $1/\sqrt{2}$. Hence, we have $I(z, 2) = \mathcal{N}(z|0, 1/2)$. Herein, using a Gauss-Hermite rule, we proceed to approximate integral $I(z, \alpha)$. Table 4.7 shows the ARE for different settings of k and z . As it is seen, the ARE for large z decreases and a larger k needs to provide the accuracy of desired level. Figure 4.4 displays the pertaining

integrand on which x-coordinates of bullets are the position of the nodes that are symmetrically distributed around origin. It is worth to note that integrand in (4.110) is an oscillatory function and when z is large the number of oscillations is large [Nolan, 2020, pp. 68]. ■

Table 4.7: ARE for approximating integral in (4.110) for $\alpha = 2$ using Gauss-Hermite rule.

z	Exact value	ARE			
		$k = 5$	$k = 10$	$k = 15$	$k = 20$
0	0.28209479	1.96e-16	1.96e-16	1.96e-16	5.90e-16
1	0.21969564	1.18e-06	2.65e-15	8.84e-16	1.51e-15
2	0.10377687	1.80e-03	2.56e-09	3.61e-15	5.08e-15
3	0.02973257	2.02e-01	1.63e-05	9.84e-11	1.08e-14
4	0.00516674	9.18174	1.27e-02	1.35e-06	2.64e-11
5	0.00054457	284.672	3.568332	3.47e-03	6.29e-07

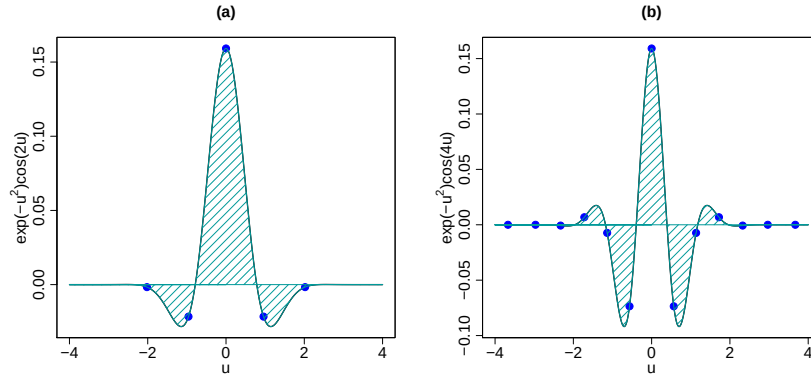


Figure 4.4: Graph of integrand (4.110) for (a): $z = 2$ and $k = 5$, (b): $z = 4$ and $k = 15$.

Example 4-7

The Owen's $T(h, a)$ function is very useful for computing the CDF of skew Gaussian [Azzalini, 2022] and bivariate Gaussian probabilities [Gupta, 1963, Young and Minder, 1974, Thomas, 1986]. This function has an integral representation as [Owen, 1956]:

$$T(h, a) = \frac{1}{2\pi} \int_0^a \frac{\exp\{-\frac{1}{2}h^2(1+z^2)\}}{1+z^2} dz, \quad (4.111)$$

where $h, a \in \mathbb{R}$. Patefield [2000] proposed six methods for evaluating $T(h, a)$. A listing of interesting properties of this function is given by [Owen, 1956]:

$$\begin{cases} T(h, -a) = -T(h, a), & T(-h, a) = T(h, a), \\ T(0, a) = \frac{1}{2\pi} \arctan(a), & T(h, 1) = \frac{1}{2} \Phi(h) [1 - \Phi(h)], \\ T(h, a) = \frac{1}{2} [\Phi(h) + \Phi(ah)] - \Phi(h) \Phi(ah) - T(ah, \frac{1}{a}), & \text{if } h > 0, \end{cases} \quad (4.112)$$

and

$$T(h, \infty) = \begin{cases} \frac{1}{2} \bar{\Phi}(h), & \text{if } h \geq 0, \\ \frac{1}{2} \Phi(h), & \text{if } h \leq 0, \end{cases} \quad (4.113)$$

where $\Phi(\cdot)$ is the CDF of a standard univariate Gaussian. For instance, based on the last property in (4.112), computing $T(h, a)$ can be reduced simply for a smaller interval $0 \leq a \leq 1$, rather than the whole half line when $h \geq 0$. However, based on their procedure, each of six approaches has been applied on one or more parts of the region $\mathcal{R} = \{(a, h) \in \mathbb{R}^2 | a > 0, h > 0\}$ consisting of 18 partitions. Among those, their fifth method, *T5* say, uses a $2k$ -point Gauss-Legendre rule for which the nodes and weights can be obtained, for example, from Abramowitz and Stegun [1948] as suggested. Taking into account the fact that integrand in $T(h, a)$ is an even function, they suggest to compute $T(h, a)$ in a simpler form that allows to reduce the number of node (or weight) points by half. This yields

$$\begin{aligned} T(h, a) &= \frac{a}{4\pi} \int_{-1}^1 \frac{\exp\{-\frac{1}{2}h^2(1 + (az)^2)\}}{1 + (az)^2} dz \\ &\approx \frac{a}{4\pi} \sum_{i=1}^k \omega_i \frac{\exp\{-\frac{1}{2}h^2(1 + (ax_i)^2)\}}{1 + (ax_i)^2}, \end{aligned} \quad (4.114)$$

where $\{x_i, \omega_i\}_{i=1}^k$ in RHS of (4.114) are the set of first k nodes and weights obtained from $2k$ -point Gauss-Legendre rule. To this end, we may use command `quad_rule(2k, type="LE")`. We further can approximate the Owen's $T(h, a)$ function using a k -point Gauss-Hermite (second kind) rule. To this end, from (4.111), it may be easily seen that

$$\begin{aligned} T(h, a) &= \frac{\exp\{-\frac{1}{2}h^2\}}{h\sqrt{2\pi}} \int_0^{\frac{ha}{\sqrt{2}}} \frac{\exp\{-z^2\}}{1 + 2h^{-2}z^2} dz \\ &\approx \frac{\exp\{-\frac{1}{2}h^2\}}{h\sqrt{2\pi}} \sum_{i=1}^k \omega_i [1 + 2h^{-2}x_i^2]^{-1}, \end{aligned} \quad (4.115)$$

where $\{x_i, \omega_i\}_{i=1}^k$ in RHS of (4.115) is the set of nodes and weights obtained from k -point Gauss-Hermite (second kind) rule for which $b = ha/\sqrt{2}$. It should be noted that for the pairs $(h, a) = (\sqrt{2}, 1)$ and $(h, a) = (\sqrt{2}, 2)$ we have $b = 1$ and $b = 2$, respectively. Hence, we may use function `quad_rule` using commands `quad_rule(k, type="HE2", b=1)` and `quad_rule(k, type="HE2", b=2)` for computing $\{x_i, \omega_i\}_{i=1}^k$ in above two case. In what follows, we compare the performance of Gauss-Hermite (second kind) and Gauss-Legendre rules for computing Owen's $T(\sqrt{2}, a)$. We note that the exact value of $T(h, 1)$ can be simply computed in terms of $\Phi(\cdot)$ as shown by (4.112). The resultants are given in Table 4.8. As it is seen from Table 4.8, for $a = 1$, the Gauss-Legendre rule outperforms the Gauss-Hermite rule while for $a = 2$, the reverse holds true. ■

4.3 Multivariate Gaussian quadrature

Each univariate k -point Gaussian quadrature rule represented as (4.75) can be straightforwardly extended to the p -dimensional case with k^p points. Let $f(\mathbf{x})$ denote real function with domain $\mathbf{x} \in \mathbb{R}^p$ and we are willing to approximate the multiple integral of the form

$$\int_{\mathbf{a}}^{\mathbf{b}} w(\mathbf{x}) f(\mathbf{x}) d\mathbf{x} = \int_{a_1}^{b_1} \cdots \int_{a_p}^{b_p} w(x_1, \dots, x_p) f(x_1, \dots, x_p) dx_1 \cdots dx_p, \quad (4.116)$$

in which elements of either $\mathbf{a} = (a_1, \dots, a_p)^\top$ or $\mathbf{b} = (b_1, \dots, b_p)^\top$ may be finite or infinite. It is assumed that the weight function is decomposed as the product of p marginal weight functions as $w(\mathbf{x}) = w_1(x_1)w_2(x_2) \cdots w_p(x_p)$. Hence, the integral (4.116) can be represented as

$$\int_{\mathbf{a}}^{\mathbf{b}} w(\mathbf{x}) f(\mathbf{x}) d\mathbf{x} = \int_{a_1}^{b_1} \cdots \int_{a_p}^{b_p} w_1(x_1)w_2(x_2) \cdots w_p(x_p) f(x_1, \dots, x_p) dx_1 \cdots dx_p.$$

Table 4.8: Absolute error of the Gaussian quadrature rules in approximating $T(\sqrt{2}, a)$.

a	k	exact value	absolute error	
			Gauss-Hermite	Gauss-Legendre
$a = 1$	2	0.036231921695240241	5.26e-05	1.70e-04
	4		2.18e-07	1.52e-07
	6		4.07e-10	1.32e-10
	8		8.33e-13	1.15e-13
	10		6.30e-14	2.01e-16
$a = 2$	2	0.039282968506234197 ¹	5.39e-04	5.86e-03
	4		6.17e-06	1.38e-04
	6		4.02e-08	2.95e-06
	8		2.72e-10	6.30e-08
	10		1.53e-11	1.34e-09

¹ Hint: The exact value of $T(\sqrt{2}, 2)$ has been computed using **Maple**.

The Gaussian quadrature in multivariate case allows to approximate the integral (4.116) as

$$\int_{\mathbf{a}}^{\mathbf{b}} w(\mathbf{x}) f(\mathbf{x}) d\mathbf{x} \approx \sum_{i_1=1}^{k_1} \sum_{i_2=1}^{k_2} \cdots \sum_{i_p=1}^{k_p} \omega_{1i_1} \omega_{2i_2} \cdots \omega_{pi_p} f(x_{1i_1}, x_{2i_2}, \dots, x_{pi_p}), \quad (4.117)$$

where $\{(x_{ij}, \omega_{ij})\}_{j=1}^{k_i}$ is the set of k_i nodes and weights associated with the i th coordinate under one of seven Gauss quadrature rules given by Table 4.4, for $i = 1, \dots, p$. It is worthwhile to note that the Gauss quadrature rule applied to two distinct coordinates may be different. For example, for approximating a double integral, we may compute k_1 pairs of nodes and weights for the first coordinate using the Gauss-Laguerre rule while the set of k_2 nodes and weights for the second coordinate are obtained using the Gauss-Legendre rule.

Example 4-8

Let the PDF of random vector $\mathbf{X} = (X_1, X_2)^\top$ is given by

$$g(x_1, x_2 | \boldsymbol{\theta}) = \frac{\beta^\alpha x_1^{\alpha+\lambda-1} x_2^{\lambda-1}}{\Gamma(\lambda)\Gamma(\alpha)} \exp\{-\beta x_1 - x_1 x_2\}, \quad (4.118)$$

where $x_1 > 0$, $x_2 > 0$, and $\boldsymbol{\theta} = (\alpha > 0, \beta > 0, \lambda > 0)^\top$. For computing the associated CDF that can be expressed in terms of integral representation as

$$G(q_1, q_2 | \boldsymbol{\theta}) = \frac{\beta^\alpha}{\Gamma(\lambda)\Gamma(\alpha)} \int_0^{q_1} \int_0^{q_2} x_1^{\alpha+\lambda-1} x_2^{\lambda-1} \exp\{-\beta x_1 - x_1 x_2\} dx_2 dx_1, \quad (4.119)$$

where $q_1 \in \mathbb{R}^+$ and $q_2 \in \mathbb{R}^+$, we would like to use the Gaussian quadrature rule. To this end, we apply the Gauss-Legendre rule to both coordinates of integral in the RHS of (4.119). Hence, using a simple change of variable, we have

$$\begin{aligned} G(q_1, q_2 | \boldsymbol{\theta}) &= \int_{-1}^1 \int_{-1}^1 \frac{\beta^\alpha}{\Gamma(\lambda)\Gamma(\alpha)} \left[\frac{x_1+1}{2} q_1 \right]^{\alpha+\lambda-1} \left[\frac{x_2+1}{2} q_2 \right]^{\lambda-1} \\ &\quad \times \exp\left\{ -\frac{x_1+1}{2} q_1 \left[\beta + \frac{x_2+1}{2} q_2 \right] \right\} dx_2 dx_1 \\ &= \frac{q_1 q_2}{4} \int_{-1}^1 \int_{-1}^1 w(\mathbf{x}) f(x_1, x_2) dx_2 dx_1, \end{aligned} \quad (4.120)$$

in which

$$w(\mathbf{x}) = w(x_1)w(x_2) = 1 \times 1 = 1, \quad (4.121)$$

$$f(u, v) = \frac{\beta^\alpha}{\Gamma(\lambda)\Gamma(\alpha)} \left[\frac{u+1}{2} q_1 \right]^{\alpha+\lambda-1} \left[\frac{v+1}{2} q_2 \right]^{\lambda-1} \exp \left\{ -\frac{u+1}{2} q_1 \left[\beta + \frac{v+1}{2} q_2 \right] \right\}. \quad (4.122)$$

It follows from (4.135) that

$$G(q_1, q_2 | \boldsymbol{\theta}) \approx \frac{q_1 q_2}{4} \sum_{i=1}^{k_1} \sum_{j=1}^{k_2} \omega_{1i} \omega_{2j} f(x_{1i}, x_{2j}), \quad (4.123)$$

where ω_{pi} and x_{pi} denote, accordingly, the i th weight and node associated with the p th coordinate, for $p = 1, 2$, and $f(\cdot, \cdot)$ is given by (4.139). Furthermore, constants k_1 and k_2 are the number of nodes (or weights) corresponds to quadrature rule considered for the first and second coordinates, respectively. Evidently, herein, we should use the Gauss-Legendre rule. In what follows, we give R code for approximating $G(q_1, q_2 | \boldsymbol{\theta})$.

```

1  R > G_quad <- function(q1, q2, k, theta)
2  + {
3  +   alpha <- theta[1]; beta <- theta[2]; lambda <- theta[3]
4  +   out <- quad_rule(k, type = "LE", alpha = 0, beta = 0)
5  +   weight_grid <- x_grid <- matrix(0, nrow = k^2, ncol = 2)
6  +   weight_grid <- apply( expand.grid(out$weight, out$weight), 1, prod)
7  +   x_grid <- expand.grid(out$node, out$node)
8  +   x1 <- (x_grid[, 1] + 1)*q1/2
9  +   x2 <- (x_grid[, 2] + 1)*q2/2
10 +   f <- x1^(alpha + lambda - 1)*x2^(lambda - 1)*exp(-beta*x1 - x1*x2)
11 +   I1 <- sum( weight_grid*f )
12 +   q1*q2/4*beta^alpha/(gamma(lambda)*gamma(alpha))*I1
13 + }

```

It should be noted that the function `G_quad(...)` depends on `quad_rule(...)` that is given after (4.105). We end this example with a brief investigation on performance of `G_quad(...)` for approximating CDF given by (4.119). Table 4.9 shows the pertinent results.

As it is seen from Table 4.9, when λ is small or both elements of pair (α, λ) are small, then the performance of Gaussian quadrature significantly lessened for $k = 10$ compared to other settings of k . Figure 4.5 (a)-(d) display the PDF (4.137) for two settings of parameter vector $\boldsymbol{\theta}$. Specifications of the Gauss-Legendre rule with $k_1 = k_2 = 5$ points for approximating $G(0.5, 0.5 | (0.5, 1, 0.5)^\top)$ and $G(10, 10 | (5, 1, 5)^\top)$ are shown by Figure 4.5 (e)-(f). ■

Example 4-9

Recall the univariate BS distribution whose CDF is given by (2.41). We write $\mathbf{T} \sim \mathcal{BS}(\boldsymbol{\theta})$ to denote random vector $\mathbf{T} = (T_1, T_2)^\top$ follows a bivariate BS distribution with PDF given by [Kundu et al., 2010]:

$$f(t_1, t_2 | \boldsymbol{\theta}) = \phi_2(\mathbf{z} | \mathbf{0}, \Sigma) \prod_{i=1}^2 \frac{1}{2\alpha_i \beta_i} \left[\left(\frac{\beta_i}{t_i} \right)^{\frac{1}{2}} + \left(\frac{\beta_i}{t_i} \right)^{\frac{3}{2}} \right], \quad (4.124)$$

where $\boldsymbol{\theta} = (\alpha_1, \beta_1, \alpha_2, \beta_2, \rho)^\top$ is the family parameter vector, $\Sigma = [(1, \rho)^\top, (\rho, 1)^\top]$, and $\mathbf{z} = (z_1, z_2)^\top$ with

$$z_i = \frac{1}{\alpha_i} \left[\sqrt{\frac{t_i}{\beta_i}} - \sqrt{\frac{\beta_i}{t_i}} \right], \quad i = 1, 2. \quad (4.125)$$

Table 4.9: ARE for approximating CDF (4.119) for $\beta = 1$ using Gauss-Legendre rule.

$q_1 = 0.5 \quad q_2 = 0.5$		ARE				
		Exact value ¹	$k = 10$	$k = 40$	$k = 80$	$k = 160$
$\alpha = 0.5$	$\lambda = 0.5$	0.17066603	0.04304235	0.01115619	0.00561266	0.00281506
$\alpha = 0.5$	$\lambda = 5$	3.2578e-07	4.5364e-07	4.53441e-07	4.5341e-07	4.5341e-07
$\alpha = 5$	$\lambda = 0.5$	8.1943e-05	0.04438234	0.01150114	0.00578534	0.00290082
$\alpha = 5$	$\lambda = 5$	5.5782e-10	5.1988e-10	5.1988e-10	5.1988e-10	5.1988e-10
$q_1 = 0.5 \quad q_2 = 10$						
$\alpha = 0.5$	$\lambda = 0.5$	0.48776719	0.06751127	0.01745956	0.00878285	0.00440495
$\alpha = 0.5$	$\lambda = 5$	0.05560379	7.0189e-09	5.0508e-10	5.0508e-10	5.0508e-10
$\alpha = 5$	$\lambda = 0.5$	1.7098e-04	0.09553933	0.02465960	0.01240306	0.00622006
$\alpha = 5$	$\lambda = 5$	6.8194e-05	9.2085e-10	9.2843e-10	9.2843e-10	9.2843e-10
$q_1 = 10 \quad q_2 = 0.5$						
$\alpha = 0.5$	$\lambda = 0.5$	0.39181881	0.04766594	0.01234977	0.00621302	0.00311616
$\alpha = 0.5$	$\lambda = 5$	0.00118855	5.0360e-06	1.9364e-11	1.9364e-11	1.9364e-11
$\alpha = 5$	$\lambda = 0.5$	0.92144044	0.07497320	0.01938347	0.00975048	0.00489024
$\alpha = 5$	$\lambda = 5$	0.12532847	3.4583e-07	2.5685e-10	2.5685e-10	2.5685e-10
$q_1 = 10 \quad q_2 = 10$						
$\alpha = 0.5$	$\lambda = 0.5$	0.80501002	0.10896581	0.02690028	0.01352624	0.00678325
$\alpha = 0.5$	$\lambda = 5$	0.34088538	1.7070e-02	5.3893e-09	2.5295e-10	2.5120e-10
$\alpha = 5$	$\lambda = 0.5$	0.97074572	0.34194629	0.08258106	0.04142828	0.02076378
$\alpha = 5$	$\lambda = 5$	0.97017575	1.3694e-02	2.0986e-11	2.0992e-11	2.0983e-11

¹ The exact value has been computed using **Maple** up to eight decimal places.

We further note that

$$P(T_1 \leq t_1, T_2 \leq t_2 | \boldsymbol{\theta}) = \Phi_2(\mathbf{z} | \mathbf{0}, \Sigma). \quad (4.126)$$

where z_i is given by (4.125). Let $\xi_i = \xi_i(\alpha_i, \rho) = 2(1 - \rho^2)\alpha_i^2$ for $i = 1, 2$. Then more algebra shows

$$\begin{aligned} E(T_1^m T_2^n) &= \int_0^\infty \int_0^\infty t_1^m t_2^n f(t_1, t_2 | \boldsymbol{\theta}) dt_1 dt_2 \\ &\approx \int_{L_1}^{U_1} \int_{L_2}^{U_2} t_1^m t_2^n \phi_2(\mathbf{z} | \mathbf{0}, \Sigma) \prod_{i=1}^2 \frac{1}{2\alpha_i \beta_i} \left[\left(\frac{\beta_i}{t_i} \right)^{\frac{1}{2}} + \left(\frac{\beta_i}{t_i} \right)^{\frac{3}{2}} \right] dt_i, \end{aligned} \quad (4.127)$$

where the quantities L_i and U_i are chosen in such a way as to constitute on a region beyond which integrand in (4.127) is sufficiently small. To this end, we set

$$L_i = \left\{ t_i \in \mathbb{R}^+ \mid \frac{1}{\alpha_i} \left[\sqrt{\frac{t_i}{\beta_i}} - \sqrt{\frac{\beta_i}{t_i}} \right] = -\eta \right\}, \quad (4.128)$$

$$U_i = \left\{ t_i \in \mathbb{R}^+ \mid \frac{1}{\alpha_i} \left[\sqrt{\frac{t_i}{\beta_i}} - \sqrt{\frac{\beta_i}{t_i}} \right] = \eta \right\}, \quad (4.129)$$

where η is a positive constant. Since z_i is an increasing function of t_i , then by solving non-linear equations in (4.128)-(4.129), we obtain

$$L_i = -\eta \alpha_i \beta_i \left[\frac{\eta}{2} \alpha_i + \frac{1}{2} \sqrt{\eta^2 \alpha_i^2 + 4} \right] + \beta_i (\eta^2 \alpha_i^2 + 1), \quad (4.130)$$

$$U_i = -\eta \alpha_i \beta_i \left[\frac{\eta}{2} \alpha_i - \frac{1}{2} \sqrt{\eta^2 \alpha_i^2 + 4} \right] + \beta_i (\eta^2 \alpha_i^2 + 1). \quad (4.131)$$

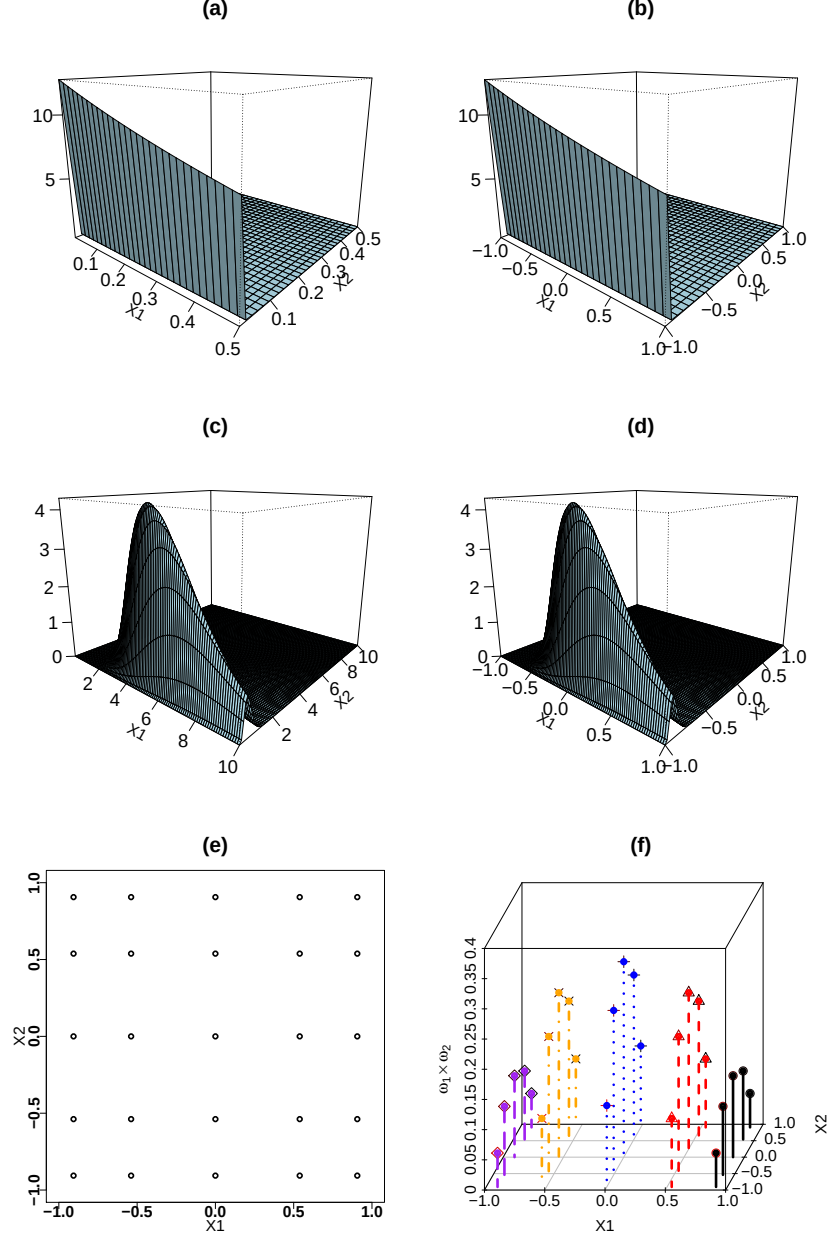


Figure 4.5: (a): Graph of $g(x_1, x_2 | \theta = (0.5, 1, 0.5)^\top)$ on region $[0, 0.5] \times [0, 0.5]$, (b): Graph of $g(x_1, x_2 | \theta = (0.5, 1, 0.5)^\top)$ transformed to region $[-1, 1] \times [-1, 1]$, (c): Graph of $g(x_1, x_2 | \theta = (5, 1, 5)^\top)$ on region $[0, 10] \times [0, 10]$, (d): Graph of $g(x_1, x_2 | \theta = (5, 1, 5)^\top)$ transformed to region $[-1, 1] \times [-1, 1]$, (e): Coordinates of nodes computed using Gauss-Legendre rule with $k_1 = k_2 = 5$ points, and (f): Product of weights computed using Gauss-Legendre rule with $k_1 = k_2 = 5$ points.

Herein, we set $\eta = 6$ that guarantees the joint Gaussian PDF, and hence integrand in the RHS of (4.127), is sufficiently small. Using an appropriate transformation of the form $y_i = 2(t_i - L_i)/(U_i - L_i) - 1$, for $i = 1, 2$, then the RHS of (4.127) becomes

$$E(T_1^m T_2^n) \approx \prod_{i=1}^2 \frac{U_i - L_i}{4\alpha_i \beta_i} \times \int_{-1}^1 \int_{-1}^1 \eta_1^m \eta_2^n \phi_2(\mathbf{y} | \mathbf{0}, \Sigma) \left[\left(\frac{\beta_i}{y_i} \right)^{\frac{1}{2}} + \left(\frac{\beta_i}{y_i} \right)^{\frac{3}{2}} \right] dy_1 dy_2, \quad (4.132)$$

where $\eta_i = (y_i + 1)(U_i - L_i)/2 + L_i$ for $i = 1, 2$. The RHS of (4.132) can be approximated through a k -point Gauss-Legendre rule. Therefore

$$\begin{aligned} E(T_1^m T_2^n) &\approx \prod_{r=1}^2 \frac{U_r - L_r}{4\alpha_r \beta_r} \times \sum_{i=1}^k \sum_{j=1}^k \omega_{1i} \omega_{2j} \left[\frac{1}{2} (y_{1i} + 1)(U_1 - L_1) + L_1 \right]^m \\ &\quad \times \left[\frac{1}{2} (y_{2j} + 1)(U_2 - L_2) + L_2 \right]^n \phi_2 \left((y_{1i}, y_{2j})^\top \middle| \mathbf{0}, \Sigma \right) \\ &\quad \times \left[\left(\frac{\beta_1}{y_{1i}} \right)^{\frac{1}{2}} + \left(\frac{\beta_1}{y_{1i}} \right)^{\frac{3}{2}} \right] \left[\left(\frac{\beta_2}{y_{2j}} \right)^{\frac{1}{2}} + \left(\frac{\beta_2}{y_{2j}} \right)^{\frac{3}{2}} \right], \end{aligned} \quad (4.133)$$

where ω_{rs} and y_{rs} are, accordingly, the s th (for $s = 1, \dots, k$) weight and node for the r th coordinate (for $r = 1, 2$) computed based on a k -point Gauss-Legendre rule. A simulation study have been carried out to check the performance of approximation (4.133). The results are shown in Table 4.10. We notice that when $\rho \neq 0$, then the Gauss-Legendre rule works well only when k is large. ■

Table 4.10: Results for approximating $E(T_1^2 T_2^2)$ using k -point Gauss-Legendre rule.

$\rho = 0.99$		Approximation			
		Exact value ¹	$k = 30$	$k = 60$	$k = 120$
$\alpha_1 = 0.2$	$\alpha_2 = 0.2$	1.36981507	2.19135117	1.39966739	1.36981553
$\alpha_1 = 2.0$	$\alpha_2 = 0.2$	74.3634327	74.53853374	74.35577417	74.36313027
$\alpha_1 = 2.0$	$\alpha_2 = 2.0$	17422.2127	17724.84078171	17421.54059427	17421.52544734
$\rho = 0.50$					
$\alpha_1 = 0.2$	$\alpha_2 = 0.2$	1.26793909	1.26793903	1.26793903	1.26793903
$\alpha_1 = 2.0$	$\alpha_2 = 0.2$	52.2184563	52.21790783	52.21827096	52.21835271
$\alpha_1 = 2.0$	$\alpha_2 = 2.0$	5412.76146	5412.62822259	5412.65315429	5412.65234609
$\rho = 0.00$					
$\alpha_1 = 0.2$	$\alpha_2 = 0.2$	1.17158976	1.17158974	1.17158974	1.17158974
$\alpha_1 = 2.0$	$\alpha_2 = 0.2$	35.7191999	35.71896967	35.71904935	35.71917926
$\alpha_1 = 2.0$	$\alpha_2 = 2.0$	1088.99999	1088.98597622	1088.99083451	1088.99875603

¹ The exact value of $E(T_1^2 T_2^2)$ has been computed using **Maple**.

Example 4-10

Let $\mathcal{G}(\alpha, \beta)$ denote the family of gamma distributions with shape parameter α and rate parameter β . The PDF of $\mathcal{G}(\alpha, \beta)$ is then given by

$$f(x|\boldsymbol{\theta}) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} \exp\{-\beta x\}, \quad (4.134)$$

where $\boldsymbol{\theta} = (\alpha, \beta)^\top$. Based on a sample of n realizations such as $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$, the likelihood function becomes

$$f(\mathbf{x}|\boldsymbol{\theta}) = \left[\frac{\beta^\alpha}{\Gamma(\alpha)} \right]^n \left[\prod_{i=1}^n x_i \right]^{\alpha-1} \exp \left\{ -\beta \sum_{i=1}^n x_i \right\}.$$

It is not hard to check that the Fisher information matrix corresponds to a gamma distribution $\mathcal{G}(\alpha, \beta)$ becomes

$$I(\boldsymbol{\theta}) = \begin{bmatrix} \Psi(1, \alpha) & -\frac{1}{\beta} \\ -\frac{1}{\beta} & \frac{\alpha}{\beta^2} \end{bmatrix}, \quad (4.135)$$

where

$$\Psi(n, x) = \frac{\partial^{n+1}}{\partial x^{n+1}} \log \Gamma(x), \quad (4.136)$$

is the well-known polygamma function. The most commonly used polygamma functions are digamma and trigamma denoted as $\Psi(0, x) = \Psi(x)$ and $\Psi(1, x)$, respectively. Simple argument shows that

$$\pi_J(\boldsymbol{\theta}) \propto \sqrt{\det I(\boldsymbol{\theta})} = \frac{1}{\beta} \sqrt{1 + \alpha \Psi(1, \alpha)}. \quad (4.137)$$

Therefore the posterior PDF is then given by

$$\begin{aligned} \pi(\boldsymbol{\theta} | \mathbf{x}) &= \frac{f(\mathbf{x} | \boldsymbol{\theta}) \pi_J(\boldsymbol{\theta})}{\int_0^\infty \int_0^\infty f(\mathbf{x} | \boldsymbol{\theta}) \pi_J(\boldsymbol{\theta}) d\alpha d\beta} \\ &= \frac{h(\alpha, \beta | \mathbf{x})}{\int_0^\infty \int_0^\infty h(\alpha, \beta | \mathbf{x}) d\alpha d\beta}, \end{aligned}$$

where the real function $h(\alpha, \beta | \mathbf{x}) = f(\mathbf{x} | \boldsymbol{\theta}) \pi_J(\boldsymbol{\theta})$ is given by

$$h(\alpha, \beta | \mathbf{x}) = \sqrt{1 + \alpha \Psi(1, \alpha)} \frac{\beta^{n\alpha-1}}{[\Gamma(\alpha)]^n} \left[\prod_{i=1}^n x_i \right]^{\alpha-1} \exp \left\{ -\beta \sum_{i=1}^n x_i \right\}. \quad (4.138)$$

It is well known that under squared loss function the Bayesian estimator $\hat{\boldsymbol{\theta}}_B$, of $\boldsymbol{\theta}$ is the average of the posterior distribution. This implies

$$\hat{\boldsymbol{\theta}}_B = E_{\pi(\boldsymbol{\theta} | \mathbf{x})}(\boldsymbol{\theta}) = \left(\frac{I_{10}}{I_{00}}, \frac{I_{01}}{I_{00}} \right)^\top,$$

where

$$I_{rs} = \int_0^\infty \int_0^\infty \alpha^r \beta^s h(\alpha, \beta | \mathbf{x}) d\alpha d\beta, \quad (4.139)$$

for $r, s = \{0, 1\}$. Using a k -point Gauss-Legendre rule, then for approximating I_{10}, I_{01} , and I_{00} , we proceed as follows. Let $\hat{\boldsymbol{\theta}}_{ML} = (\hat{\alpha}_{ML}, \hat{\beta}_{ML})^\top$ denote the ML estimator of the unknown parameter vector $\boldsymbol{\theta} = (\alpha, \beta)^\top$, then based on sample evidence $\mathbf{x}$, we define

$$\sigma_{\hat{\alpha}_{ML}} = \frac{3}{\sqrt{n}} \sqrt{I_{1,1}^{-1}(\hat{\boldsymbol{\theta}}_{ML})},$$

where $I_{i,j}^{-1}(\boldsymbol{\theta})$ denotes the (i, j) th entry of the inverse of Fisher information matrix given by (4.135). More algebra shows

$$\sigma_{\hat{\alpha}_{ML}} = \frac{3}{\sqrt{n}} \frac{\sqrt{\hat{\alpha}_{ML}}}{\sqrt{\Psi(1, \hat{\alpha}_{ML}) \hat{\alpha}_{ML} - 1}}.$$

For our purposes, we set $L_{\hat{\alpha}} = \max\{0, \hat{\alpha}_{ML} - \sigma_{\hat{\alpha}_{ML}}\}$, and $U_{\hat{\alpha}} = \hat{\alpha}_{ML} + \sigma_{\hat{\alpha}_{ML}}$. It is worth some discussion on the posterior distribution of β given α and $\mathbf{x}$. It follows from (4.138) that $\pi^* = \pi(\beta | \alpha, \mathbf{x}) \sim \mathcal{G}(n\alpha, n\bar{x})$, then we let

$$\sigma_{\hat{\beta}_{ML}} = 3 \times \sqrt{\text{var}_{\pi^*}(\beta)} = \frac{3}{\sqrt{n}} \frac{\sqrt{\hat{\alpha}_{ML}}}{\bar{x}}.$$

Using the fact that $E_{\pi^*}(\beta) = \alpha/\bar{x}$, we define $L_{\hat{\beta}} = \max \{0, E_{\pi^*}(\beta) - \sigma_{\hat{\beta}_{ML}}\}$ and $U_{\hat{\beta}} = E_{\pi^*}(\beta) + \sigma_{\hat{\beta}_{ML}}$. If $n \times \hat{\alpha}_{ML} > 1$, then we set $U_{\hat{\beta}}$ to be the 0.9999th quantile of π^* . Thus, I_{rs} is approximated as

$$I_{rs} = \int_0^\infty \int_0^\infty \alpha^r \beta^s h(\alpha, \beta | \mathbf{x}) d\alpha d\beta \\ \approx \int_{L_{\hat{\beta}}}^{U_{\hat{\beta}}} \int_{L_{\hat{\alpha}}}^{U_{\hat{\alpha}}} \alpha^r \beta^s h(\alpha, \beta | \mathbf{x}) d\alpha d\beta.$$

Define $R_{\hat{\alpha}} = U_{\hat{\alpha}} - L_{\hat{\alpha}}$ and $R_{\hat{\beta}} = U_{\hat{\beta}} - L_{\hat{\beta}}$, then using two suitable change of variables of the form $z_1 = 2(\alpha - L_{\hat{\alpha}})/R_{\hat{\alpha}} - 1$ and $z_2 = 2(\beta - L_{\hat{\beta}})/R_{\hat{\beta}} - 1$, it follows that

$$I_{rs} \approx \frac{R_{\hat{\alpha}} R_{\hat{\beta}}}{4} \int_{-1}^1 \int_{-1}^1 \left[\frac{1}{2}(z_1 + 1)R_{\hat{\alpha}} + L_{\hat{\alpha}} \right]^r \left[\frac{1}{2}(z_2 + 1)R_{\hat{\beta}} + L_{\hat{\beta}} \right]^s \\ \times h \left[\frac{1}{2}(z_1 + 1)R_{\hat{\alpha}} + L_{\hat{\alpha}}, \frac{1}{2}(z_2 + 1)R_{\hat{\beta}} + L_{\hat{\beta}} \middle| \mathbf{x} \right] dz_1 dz_2. \quad (4.140)$$

Based on a k -point Gauss-Legendre rule, an approximation of I_{rs} in (4.140), is then given by

$$I_{rs} \approx \frac{R_{\hat{\alpha}} R_{\hat{\beta}}}{4} \sum_{i=1}^k \sum_{j=1}^k \omega_{1i} \omega_{2j} \left[\frac{1}{2}(y_{1i} + 1)R_{\hat{\alpha}} + L_{\hat{\alpha}} \right]^i \left[\frac{1}{2}(y_{2j} + 1)R_{\hat{\beta}} + L_{\hat{\beta}} \right]^j \\ \times h \left[\frac{1}{2}(y_{1i} + 1)R_{\hat{\alpha}} + L_{\hat{\alpha}}, \frac{1}{2}(y_{2j} + 1)R_{\hat{\beta}} + L_{\hat{\beta}} \middle| \mathbf{x} \right], \quad (4.141)$$

where ω_{ij} and y_{ij} for $i = 1, \dots, k$ and $j = 1, 2$ are, respectively, the weight and node associated with the s th coordinate based on a k -point Gauss-Legendre rule. It is worth to note that when sample size is large, then both of the ML and Bayesian paradigms yield the same estimations. The R code below computes $\hat{\theta}_B$ under a squared loss function and based on the Jeffreys prior (4.137) is given below. It should be noted that arguments `alpha0` and `beta0` in function `Bayes_gamma_quad` are the initial (ML) estimators of α and β for constructing $L_{\hat{\alpha}}, U_{\hat{\alpha}}, L_{\hat{\beta}}$, and $U_{\hat{\beta}}$.

```

1  R> Bayes_gamma_quad <- function(x, k, alpha0, beta0)
2  +{
3  + n <- length(x)
4  + y <- matrix(0, nrow = k^2, ncol = 2)
5  + x_bar <- mean(x)
6  + sigma_alpha <- 3/sqrt(n)*sqrt(alpha0)/sqrt(alpha0*trigamma(alpha0)- 1)
7  + L_alpha <- max(0, alpha0- sigma_alpha[1] )
8  + U_alpha <- alpha0 + sigma_alpha[1]
9  + sigma_beta <- 3/sqrt(n)*sqrt(alpha0)/x_bar
10 + L_beta <- max(0, alpha0/x_bar- sigma_beta )
11 + if(n*alpha0 > 1)
12 + {
13 +   U_beta <- alpha0/x_bar + sigma_beta
14 + }else{
15 +   U_beta <- qgamma(0.9999, shape = n*alpha0, rate = n*x_bar )
16 + }
17 + R_alpha <- U_alpha- L_alpha
18 + R_beta <- U_beta- L_beta
19 + out <- quad_rule(k, type = "LE", alpha = 0, beta = 0)
20 + node <- out$node
21 + weight <- out$weight
22 + node_grid <- as.data.frame( expand.grid(node, node) )
23 + weight_grid <- as.data.frame( expand.grid(weight, weight) )
24 + omega <- rowSums( weight_grid )
25 + y[, 1] <- (node_grid[, 1] + 1)*R_alpha/2 + L_alpha

```

```

26 + y[, 2] <- (node_grid[, 2] + 1)*R_beta /2 + L_beta
27 + h_theta <- log(omega) + 1/2*log( 1 + y[, 1]*trigamma(y[, 1]) ) +
28 + (n*y[, 1]- 1)*log(y[, 2])- n*lgamma(y[, 1])
29 + n*y[, 2]*x_bar + (y[, 1]- 1)*sum( log( x ) )
30 + I00 <- sum( exp( h_theta- mean( h_theta) ) )
31 + if( I00 == 0 | I00 == Inf )
32 + {
33 +   h_theta <- 0
34 +   I00 <- k^2
35 + }
36 + I10 <- sum( exp( log(y[, 1]) + h_theta- mean( h_theta) ) )
37 + I01 <- sum( exp( log(y[, 2]) + h_theta- mean( h_theta) ) )
38 + return( ls = list( "alpha_bayes" = I10/I00, "beta_bayes" = I01/I00 ) )
39 +}

```

To assess how well a gamma distribution works in modelling a data set, we provide an R function called `GOF`² for computing some goodness-of-fit measures including Anderson Darling (AD) Anderson and Darling [Anderson and Darling, 1954], Akaike information criterion (AIC) [Akaike, 2003], Bayesian information criterion (BIC) [Schwarz et al., 1978], Cramér-von Misses (CVM) [Cramér, 1928], log-likelihood, and the Kolmogorov-Smirnov (KS) statistics with corresponding p-value. Some of these statistics have been used in Sub-section 1.6.3 for testing normality. If $x_1, \dots, x_n$ independently and identically come from the CDF $F(\cdot|\boldsymbol{\theta})$ whose PDF is $f(\cdot|\boldsymbol{\theta})$, then above statistics are defined as follows.

$$AD = -n - \sum_{i=1}^n \left(\frac{2i-1}{n} \right) \left\{ \log[F(x_{(i)}|\boldsymbol{\theta})] + \log[1 - F(x_{(n-i+1)}|\boldsymbol{\theta})] \right\}, \quad (4.142)$$

$$AIC = -2 \sum_{i=1}^n \log[f(x_i|\boldsymbol{\theta})] + 2p, \quad (4.143)$$

$$BIC = -2 \sum_{i=1}^n \log[f(x_i|\boldsymbol{\theta})] + p \log(n), \quad (4.144)$$

$$CVM = \frac{1}{12n} + \sum_{i=1}^n \left[F(x_{(i)}|\boldsymbol{\theta}) - \frac{2i-1}{2n} \right]^2, \quad (4.145)$$

$$KS = \sup_{1 \leq i \leq n} \left| F(x_i|\boldsymbol{\theta}) - F_n(x_i) \right|, \quad (4.146)$$

where p is the number of family parameters (number of unknown elements of $\boldsymbol{\theta}$), $x_{(i)}$ denotes the i th ordered observation within sample $\mathbf{x}$, and $F_n(\cdot)$ is the empirical CDF of sample $\mathbf{x}$, that is, $F_n(x) = r/n$ if exactly r scores are less or equal than x (for $r = 0, \dots, n$). We note that the smaller value for statistics given in (4.142)-(4.146) indicates a better model and, in practice, $\boldsymbol{\theta}$ is substituted with its estimator.³

```

1 R> GOF <- function(x, theta)
2 +{
3 +   n <- length(x)
4 +   index <- 1:n
5 +   sort_x <- sort(x)
6 +   n_theta <- length(theta)
7 +   log_pdf <- dgamma(sort_x, shape = theta[1], rate = theta[2], log = T)
8 +   cdf <- pgamma(sort_x, shape = theta[1], rate = theta[2])
9 +   log_cdf <- pgamma(sort_x, shape = theta[1], rate = theta[2], log = T)

```

²The function `GOF` can be modified for computing the GOF statistics when other well-known distributions are fitted to data set. For example, if one would rather have a Weibull model instead of gamma, it just needs to change expression `pgamma` in lines 7, 8, 9, 10, and 17 to `pweibull`.

³If a GOF statistic involves a hypothesis testing, such as KS test, replacing $\boldsymbol{\theta}$ with $\hat{\boldsymbol{\theta}}$ would affect the corresponding critical values specially when n is small [Lilliefors, 1967, Shorack and Wellner, 2009]. Although the ML estimator is biased for small n , but in the case of substituting, it is suggested.

```

10 + log_survival <- pgamma(sort_x, shape = theta[1], rate = theta[2], log = T, lower.tail = F)
11 + log.like <- sum( log_pdf )
12 + cdf <- ifelse(cdf == 1, .Machine$double.eps, cdf)
13 + AD <- -n - mean( (2*index - 1)*log_cdf + (2*n + 1 - 2*index)*log_survival )
14 + AIC <- -2*log.like + 2*n_theta
15 + BIC <- -2*log.like + n_theta*log(n)
16 + CVM <- sum( ( cdf - (2*index - 1)/(2*n) )^2 ) + 1/(12*n)
17 + KS <- suppressWarnings( ks.test(sort_x, "pgamma", theta[1], theta[2]) )
18 + out1 <- cbind(AD, AIC, BIC, CVM, log.like)
19 + colnames(out1) <- c("AD", "AIC", "BIC", "CVM", "log.likelihood")
20 + rownames(out1) <- c("")
21 + out2 <- cbind(KS$statistic, KS$p.value)
22 + colnames(out2) <- c("statistics", "p-value")
23 + rownames(out2) <- c("")
24 + list( "GOF_statistics" = out1, "Kolmogorov-Smirnov_test" = out2 )
25 +}

```

As an illustration, we focus on fatigue life of 10 bearings of a certain type reported by McCool [1974] that is presented in Table 4.11. Different distributions such as BS [Ng et al., 2003] and three-parameter Weibull [Cohen et al., 1984] were fitted to this data set. Herein, we would like to fit a gamma distribution to these observations and then estimate its pa-

Table 4.11: Fatigue life of 10 bearings of a certain type (in hours).

152.7	204.7	172.0	216.5	172.5	234.9	173.3	262.6	193.0	422.6
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

rameters using the ML method and Bayesian paradigm. To this end, first, we use command `fitdist(x, "gamma", method = "mle")` existing in R package `fitdistrplus` to compute the ML estimator $\hat{\theta}_{ML}$, of θ and then compute $\hat{\theta}_B$ using a 20-point Gauss-Legendre rule for which $\hat{\theta}_{ML}$ is taken as the initial estimate. The results are shown in Table 4.12. Indeed if we accept that observations in Table 4.11 independently follow a gamma distribution, then the $\hat{\theta}_{ML}$ slightly outperforms $\hat{\theta}_B$ in terms of all GOF statistics given in Table 4.12. ■

Table 4.12: Estimators and associated GOF statistics for gamma distribution fitted to to fatigue life data given in Table 4.11.

	parameter		GOF statistic					
	α	β	AD	AIC	BIC	CVM	KS	p -value
$\hat{\theta}_{ML}$	11.5619	0.0524	0.6813	115.2275	115.8327	0.0987	0.1853	0.8222
$\hat{\theta}_B$	11.4344	0.0516	0.6967	115.2300	115.8352	0.1041	0.1904	0.7971

Example 4-11

Suppose we are interested in computing double integral

$$I(\theta) = \int_a^b \int_{c(x_1)}^{d(x_1)} f(x_1, x_2 | \theta) dx_2 dx_1. \quad (4.147)$$

For approximating $I(\boldsymbol{\theta})$, we may proceed based on a Gauss-Legendre rule as follows. First we apply a k_1 -point Gauss-Legendre rule for inner integral as

$$\begin{aligned} \int_{c(x_1)}^{d(x_1)} f(x_1, x_2 | \boldsymbol{\theta}) dx_2 &= \int_{-1}^1 f\left[x_1, \frac{(x_2 + 1)(d(x_1) - c(x_1))}{2} + c(x_1) \middle| \boldsymbol{\theta}\right] dx_2 \\ &\approx \frac{d(x_1) - c(x_1)}{2} \sum_{j=1}^{k_2} \omega_{2j} f\left[x_1, \frac{(x_{2j} + 1)(d(x_1) - c(x_1))}{2} + c(x_1) \middle| \boldsymbol{\theta}\right], \end{aligned} \quad (4.148)$$

where ω_{2j} and x_{2j} , for $j = 1, \dots, k_2$ are, accordingly, the weights and nodes constructed based on a k_2 -point Gauss-Legendre rule. Then, we complete computing $I(\boldsymbol{\theta})$ by approximating the outer integral as follows.

$$\begin{aligned} I(\boldsymbol{\theta}) &\approx \frac{b-a}{4} \sum_{i=1}^{k_1} \sum_{j=1}^{k_2} \omega_{1i} \omega_{2j} [d(x_{1i}) - c(x_{1i})] \\ &\quad \times f\left[\frac{(x_{1i} + 1)(b-a)}{2} + a, \frac{(x_{2j} + 1)(d(x_{1i}) - c(x_{1i}))}{2} + c(x_{1i}) \middle| \boldsymbol{\theta}\right], \end{aligned} \quad (4.149)$$

where ω_{1i} and x_{1i} , for $i = 1, \dots, k_1$ are, accordingly, the weights and nodes constructed based on a k_1 -point Gauss-Legendre rule. ■

Example 4-12

Indeed, computing the probability of a bivariate Gaussian distribution is simply expressed in terms of $I(\boldsymbol{\theta})$ given in (4.147). Let $\mathbf{X} = (X_1, X_2)^\top$ follow a Gaussian distribution with mean $\boldsymbol{\mu} = (\mu_1, \mu_2)^\top$ and covariance matrix $\Sigma = [(\Sigma_{1,1}, \Sigma_{1,2})^\top, (\Sigma_{2,1}, \Sigma_{2,2})^\top]$. Recall from Subsection 4.1.2 that discusses computing $\mathcal{P}_G = P(\mathbf{a} \leq \mathbf{X} \leq \mathbf{b})$ using a transformation $\mathbf{y} = L(\mathbf{x} - \boldsymbol{\mu})$ in which L denotes the Cholesky decomposition of Σ . We have

$$\mathcal{P}_G = \frac{1}{2\pi} \int_{\frac{a_1 - \mu_1}{L_{1,1}}}^{\frac{b_1 - \mu_1}{L_{1,1}}} \int_{c(y_1)}^{d(y_1)} \exp\left\{-\frac{y_1^2 + y_2^2}{2}\right\} dy_2 dy_1, \quad (4.150)$$

where $\mathbf{a} = (a_1, a_2)^\top$, $\mathbf{b} = (b_1, b_2)^\top$, and

$$\begin{aligned} c(y_1) &= \frac{a_2 - \mu_2 - L_{2,1}y_1}{L_{2,2}}, \\ d(y_1) &= \frac{b_2 - \mu_2 - L_{2,1}y_1}{L_{2,2}}. \end{aligned}$$

Following (4-11) for approximating the RHS of (4.150), it turns out that

$$\begin{aligned} \mathcal{P}_G &\approx \frac{(b_1 - a_1)(b_2 - a_2)}{4L_{1,1}L_{2,2}} \sum_{i=1}^{k_1} \sum_{j=1}^{k_2} \omega_{1i} \omega_{2j} \exp\left\{-\frac{z_{1i}^2 + z_{2j}^2}{2}\right\}, \\ &= \sum_{i=1}^{k_1} \sum_{j=1}^{k_2} \omega_{1i} \omega_{2j} g(z_{1i}, z_{2i}), \end{aligned} \quad (4.151)$$

where

$$\begin{aligned} z_{1i} &= \frac{(x_{1i} + 1)(b_1 - a_1)}{2L_{1,1}} + \frac{a_1 - \mu_1}{L_{1,1}}, \\ z_{2j} &= \frac{(x_{2j} + 1)(b_2 - a_2)}{2L_{2,2}} + \frac{a_2 - \mu_2 - L_{2,1}z_{1i}}{L_{2,2}}. \end{aligned}$$

The R code below approximates $\mathcal{P}_G$ given in (4.150) using a k -point Gauss-Legendre rule using Cholesky decomposition of covariance matrix.

```

1  R> Chol_quad <- function(a, b, Mu, Sigma, k)
2  +{
3  + x <- matrix(0, nrow = k^2, ncol = 2)
4  + ch <- t( chol(Sigma) )
5  + threshold <- 5*sqrt( diag(Sigma) )
6  + a0 <- a - Mu
7  + b0 <- b - Mu
8  + a0 <- ifelse( a0 < -threshold, -threshold, a0 )
9  + b0 <- ifelse( b0 > threshold, threshold, b0 )
10 + f1 <- function(x) ( x + 1 )*(b0[1] - a0[1])/(2*ch[1,1]) + a0[1]/ch[1,1]
11 + f2 <- function(x, y) 1/(2*pi)*exp( -x^2/2 - y^2/2 )
12 + out <- quad_rule(k, type = "LE", alpha = 0, beta = 0)
13 + weight <- apply( expand.grid(out$weight, out$weight), 1, prod )
14 + x1 <- f1( out$node )
15 + x2 <- out$node
16 + x[, 1] <- rep(x1, each = k)
17 + x[, 2] <- as.vector( t( sapply(1:k, function(j) (x2[j] + 1)*(b0[2] - a0[2])/
18 + (2*ch[2,2]) + (a0[2] - ch[2,1]*x1)/ch[2,2] ) ) )
19 + I1 <- (b0[1] - a0[1])*(b0[2] - a0[2])/( 4*ch[1,1]*ch[2,2] ) *
20 + sum( weight*f2(x[, 1], x[, 2]) )
21 + min( abs(I1), 1)
22 +}

```



4.4 Computing the CDF of multivariate Gaussian distribution using Gaussian Quadrature

Recall from Subsection 4.1.2 that computes the CDF of Gaussian distribution using SOV technique proposed by Genz [1992]. Herein, we are willing to compute $\mathcal{P}_G$ through the Gaussian quadrature based on spectral decomposition (2.78) of covariance matrix.

4.4.1 Computing the CDF of bivariate Gaussian distribution

Computing bivariate Gaussian probability has a long history. Herein, we describe briefly methods proposed by Drezner and Wesolowsky [1990]. First note that [Sheppard and Pearson, 1900] shows that each bivariate Gaussian probability on region $[x, \infty) \times [y, \infty)$ represented as

$$L(x, y|\rho) = \frac{1}{2\pi\sqrt{1-\rho^2}} \int_x^\infty \int_y^\infty \exp\left\{-\frac{u^2 + v^2 - 2uv\rho}{2(1-\rho^2)}\right\} dv du, \quad (4.152)$$

where $|\rho| \leq 1$ reduces to one-dimensional integral as

$$L(x, y|\rho) = \frac{1}{2\pi} \int_{\cos^{-1}(\rho)}^\pi \exp\left\{-\frac{x^2 + y^2 - 2xy \cos(z)}{2 \sin^2(z)}\right\} dz. \quad (4.153)$$

Differentiating (4.153) with respect to $\rho > 0$ yields [Plackett, 1954]:

$$\frac{\partial L(x, y|\rho)}{\partial \rho} = \frac{1}{2\pi\sqrt{1-\rho^2}} \exp\left\{-\frac{x^2 + y^2 - 2xy\rho}{2(1-\rho^2)}\right\}. \quad (4.154)$$

Based on the Plackett's formula (4.154), a two-step procedure for computing (4.152) has been introduced by Drezner and Wesolowsky [1990] as follows.

- **First step** ($|\rho| < 0.7$): Suppose $\rho > 0$. Using the fact that

$$\int_0^\rho \frac{\partial L(x, y|r)}{\partial r} dr = L(x, y|\rho) - L(x, y|0),$$

or

$$L(x, y|\rho) = L(x, y|0) + \int_0^\rho \frac{\partial L(x, y|r)}{\partial r} dr \quad (4.155)$$

It is immediate from (4.152) that $L(x, y|0) = \Phi(-x)\Phi(-y)$. Substituting $\partial L(x, y|\rho)/\partial \rho$ from (4.154) into the RHS of (4.155) it follows that

$$L(x, y|\rho) = \Phi(-x)\Phi(-y) + \frac{1}{2\pi} \int_0^\rho \frac{1}{\sqrt{1-r^2}} \exp\left\{-\frac{x^2+y^2-2xyr}{2(1-r^2)}\right\} dr.$$

If ρ is negative, then a simple change of variable shows

$$L(x, y|\rho) = \Phi(-x)\Phi(-y) + \frac{s}{2\pi} \int_0^{|\rho|} \frac{1}{\sqrt{1-r^2}} \exp\left\{-\frac{x^2+y^2-2xsyr}{2(1-r^2)}\right\} dr, \quad (4.156)$$

where $s = \text{sign}(\rho)$. Drezner and Wesolowsky [1990] suggest to approximate (4.156) using a 5-point Gauss-Legendre rule. They further used k -point Gauss-Legendre rule (for $k = \{2, 3, 5, 10\}$) and demonstrated that the results are more accurate than that of [Mee and Owen, 1983] for $|\rho| \leq 0.9$.

- **Second step** ($|\rho| \geq 0.7$): Even though the first step works well, but it can be modified when ρ is large ($|\rho| \geq 0.7$, say). For computational purpose, we consider the fact that

$$\int_\rho^s \frac{\partial L(x, y|r)}{\partial r} dr = L(x, y|s) - L(x, y|\rho), \quad (4.157)$$

Rearranging (4.157) and then substituting integrand $\partial L(x, y|r)/\partial r$ from (4.154) we get

$$\begin{aligned} L(x, y|\rho) &= L(x, y|s) - \int_\rho^s \frac{\partial L(x, y|r)}{\partial r} dr \\ &= L(x, y|s) - \frac{1}{2\pi} \int_\rho^s \frac{1}{\sqrt{1-r^2}} \exp\left\{-\frac{x^2+y^2-2xyr}{2(1-r^2)}\right\} dr. \end{aligned} \quad (4.158)$$

The RHS of (4.158) can be evidently represented as

$$L(x, y|\rho) = L(x, y|s) - \frac{s}{2\pi} \int_{|\rho|}^1 \frac{1}{\sqrt{1-r^2}} \exp\left\{-\frac{x^2+y^2-2xsyr}{2(1-r^2)}\right\} dr, \quad (4.159)$$

and a change of variable $z = \sqrt{1-r^2}$ turns out that

$$L(x, y|\rho) = L(x, y|s) - \frac{s}{2\pi} \int_0^{\sqrt{1-\rho^2}} \frac{1}{\sqrt{1-z^2}} \exp\left\{-\frac{x^2+y^2-2xsy\sqrt{1-z^2}}{2z^2}\right\} dz. \quad (4.160)$$

A 5-point Gauss-Legendre rule for computing integral in (4.160) yields error, in most cases, less than 10^{-7} when $|\rho| \geq 0.7$. As pointed out by Drezner and Wesolowsky [1990], more refinements is obtained when $|\rho|$ is large and x is close to sy (but not equal to sy). To this end, we rewrite (4.160) as

$$L(x, y|\rho) = L(x, y|s) - \frac{s}{2\pi} \int_0^\tau \exp\left\{-\frac{\eta^2}{2z^2}\right\} \frac{1}{\sqrt{1-z^2}} \exp\left\{-\frac{xsy}{1+\sqrt{1-z^2}}\right\} dz, \quad (4.161)$$

where $\eta = |x - sy|$ and $\tau = \sqrt{1 - \rho^2}$. If x is close to sy (but not equal to sy) and $|\rho|$ is large (τ is small), then the term $\exp\{-(x - sy)^2/(2z^2)\}$ in RHS of (4.161) is the source of large errors. As it is seen from (4.161) if τ is small, then z is near zero. Using the Taylor expansion

$$\frac{1}{\sqrt{1 - z^2}} \exp\left\{-\frac{xsy}{1 + \sqrt{1 - z^2}}\right\} = \exp\left\{-\frac{xsy}{2}\right\} [1 + \lambda z^2] + \mathcal{O}(z^4), \quad (4.162)$$

in which $\lambda = (4 - xsy)/8$, the integral in RHS of (4.161) can be written as

$$\begin{aligned} & \int_0^\tau \exp\left\{-\frac{\eta^2}{2z^2} - \frac{xsy}{2}\right\} [1 + \lambda z^2] dz + \int_0^\tau \exp\left\{-\frac{\eta^2}{2z^2}\right\} \mathcal{O}(z^4) dz \\ &= \int_0^\tau \exp\left\{-\frac{\eta^2}{2z^2} - \frac{xsy}{2}\right\} [1 + \lambda z^2] dz \\ &+ \int_0^\tau \frac{1}{\sqrt{1 - z^2}} \exp\left\{-\frac{\eta^2}{2z^2} - \frac{xsy}{1 + \sqrt{1 - z^2}}\right\} - \exp\left\{-\frac{\eta^2}{2z^2} - \frac{xsy}{2}\right\} (1 + \lambda z^2) dz. \end{aligned} \quad (4.163)$$

For computing the first integral in RHS of (4.163), we take into account the two following identities.

$$\int z^n \exp\left\{-\frac{\eta^2}{2z^2}\right\} dz = \frac{z^{n+1}}{n+1} \exp\left\{-\frac{\eta^2}{2z^2}\right\} - \frac{\eta^2}{n+1} \int z^{n-2} \exp\left\{-\frac{\eta^2}{2z^2}\right\} dz, \quad (4.164)$$

$$\begin{aligned} \int \exp\left\{-\frac{\eta^2}{2z^2}\right\} dz &= z \exp\left\{-\frac{\eta^2}{2z^2}\right\} - b \int_{\eta/z}^\infty \exp\left\{-\frac{u^2}{2}\right\} du + C \\ &= z \exp\left\{-\frac{\eta^2}{2z^2}\right\} dz - \sqrt{2\pi}\eta \Phi\left(-\frac{\eta}{z}\right) + C, \end{aligned} \quad (4.165)$$

where C is a real constant. Therefore, for $n = 2$, it follows from (4.164) and (4.165) that

$$\begin{aligned} I_1 &= \int_0^\tau \exp\left\{-\frac{xsy}{2} - \frac{\eta^2}{2z^2}\right\} [1 + \lambda z^2] dz \\ &= \exp\left\{-\frac{xsy}{2}\right\} \left[\tau \left(1 - \lambda \frac{\eta^2 - \tau^2}{3}\right) \exp\left\{-\frac{\eta^2}{2\tau^2}\right\} - \sqrt{2\pi}\eta \left[1 - \lambda \frac{\eta^2}{3}\right] \Phi\left(-\frac{\eta}{\tau}\right) \right]. \end{aligned} \quad (4.166)$$

The second integral in RHS of (4.163) is

$$I_2 = \int_0^\tau \frac{1}{\sqrt{1 - z^2}} \exp\left\{-\frac{\eta^2}{2z^2} - \frac{xsy}{1 + \sqrt{1 - z^2}}\right\} - \exp\left\{-\frac{\eta^2}{2z^2} - \frac{xsy}{2}\right\} (1 + \lambda z^2) dz. \quad (4.167)$$

Hence,

$$L(x, y|\rho) = L(x, y|s) - \frac{s}{2\pi} [I_1 + I_2]. \quad (4.168)$$

Splitting integral in RHS of (4.161) into I_1 and I_2 yields more accurate result since by computing I_1 analytically though (4.166) the rate of error (when $|\rho|$ is large and η is small) is diminished. The integral of reminder term, that is I_2 in (4.167), would be unaffected by error source and computed using a k -point Gauss-Legendre rule. For computing $L(x, y|s)$ in RHS of (4.168), we take into account the fact that

$$\begin{aligned} L(x, y|\rho) &= \int_x^\infty \int_y^\infty \phi(u) \phi\left[\frac{v - \rho u}{\sqrt{1 - \rho^2}}\right] dv du \\ &= \int_x^\infty \phi(u) \bar{\Phi}\left[\frac{y - \rho u}{\sqrt{1 - \rho^2}}\right] du, \end{aligned} \quad (4.169)$$

where $\bar{\Phi}(x) = 1 - \Phi(x)$. Taking limit from both sides of (4.169) when $\rho \rightarrow 1$, it turns out that

$$L(x, y|1) = \begin{cases} \int_x^\infty \phi(u) \bar{\Phi}\left[\frac{y-u}{0^+}\right] du = \bar{\Phi}(x), & \text{if } y < x, \\ \int_x^y \phi(u) \bar{\Phi}\left[\frac{y-u}{0^+}\right] du + \int_y^\infty \phi(u) \bar{\Phi}\left[\frac{y-u}{0^+}\right] du & \text{if } y > x. \\ = 0 + \int_y^\infty \phi(u) \bar{\Phi}\left[\frac{y-u}{0^+}\right] du \\ = \bar{\Phi}(y), \end{cases} \quad (4.170)$$

From (4.170) we conclude that

$$L(x, y|1) = \bar{\Phi}(\max\{x, y\}) = \Phi(-\max\{x, y\}). \quad (4.171)$$

Similar arguments as above in (4.170) show that

$$\begin{aligned} L(x, y|-1) &= \begin{cases} 0, & \text{if } x + y > 0, \\ \Phi(-y) - \Phi(x), & \text{if } x + y \leq 0. \end{cases} \\ &= \max\{0, \Phi(-y) - \Phi(x)\}. \end{aligned} \quad (4.172)$$

Hence,

$$L(x, y|s) = \begin{cases} \Phi(-\max\{x, y\}), & \text{if } s = 1, \\ \max\{0, \Phi(-y) - \Phi(x)\}, & \text{if } s = -1. \end{cases} \quad (4.173)$$

Implementing the two-step procedure of Drezner and Wesolowsky [1990] described above based on 5-point Gauss-Legendre rule yields a maximum error of 2×10^{-7} . The original work of Drezner and Wesolowsky [1990] incorporates a FORTRAN code of 36 lines for computing $L(x, y|\rho)$. In what follows, we give the corresponding R code.

```

1  R> BVN_Drez <- function(a, Mu, Sigma)
2  +{
3  + rho <- Sigma[1, 2]/sqrt( Sigma[1, 1]*Sigma[2, 2] )
4  + lb <- c(a - Mu)/sqrt( diag(Sigma) )
5  + x <- lb[1]
6  + y <- lb[2]
7  + s <- sign(rho)
8  + w <- c(0.2369268851, 0.4786286705, 0.5688888889, 0.4786286705, 0.2369268851)
9  + node <- c(0.9061798459, 0.5384693101, 0, -0.5384693101, -0.9061798459)
10 + x0 <- rho*(1 + node)/(2*s)
11 + p1 <- sqrt(1 - x0^2)
12 + if( abs(rho) < 0.7)
13 + {
14 +   out <- pnorm(-x)*pnorm(-y) +
15 +     rho/(4*pi)*sum( w/p1*exp(-(x^2 + y^2 - 2*x*s*y*x0)/( 2*p1^2 ) ) )
16 + }else{
17 +   L_s <- ifelse( rho < 0, max(0, pnorm(-y)-pnorm(x)), pnorm( -max(x, y) ) )
18 +   y <- s*y
19 +   tau <- sqrt(1 - rho^2)
20 +   x0 <- tau*(node + 1)/2
21 +   p2 <- sqrt(1 - x0^2)
22 +   lambda <- (4 - x*y)/8
23 +   eta <- abs(x - y)
24 +   p3 <- exp(-x*y/2)

```

```

25 +   p4 <- eta/tau
26 +   p5 <- exp(- p4^2/2 )
27 +   I1 <- p3*( tau*(1 - lambda*(eta^2 - tau^2)/3)*p5 -
28 +         sqrt(2*pi)*eta*(1 - lambda*eta^2/3)*pnorm(-p4) )
29 +   p6 <- exp(-eta^2/(2*x0^2))
30 +   p7 <- exp(- x*y/( 1 + p2 ) )/p2 - p3*(1 + lambda*x0^2)
31 +   I2 <- tau/2*sum( w*p6*p7 )
32 +   out <- L_s - s/(2*pi)*(I1 + I2)
33 + }
34 +return( abs(out) )
35 +}

```

Genz [2004] pointed out that the hybrid (two-step) method of Drezner and Wesolowsky [1990] described earlier is yet a single precision algorithm⁴ so as can yield a maximum absolute error of 2.5×10^{-7} if the threshold for ρ changes to 0.80 rather than 0.70, and further claimed if one uses a three-term Taylor expansion in (4.162) as

$$\frac{1}{\sqrt{1-z^2}} \exp\left\{-\frac{xy}{1+\sqrt{1-z^2}}\right\} = \exp\left\{-\frac{xy}{2}\right\} \left[1 + \lambda z^2 + \left(\frac{3}{8} - \frac{1}{8}xy + \frac{1}{128}xy^2\right)z^4\right] + \mathcal{O}(z^6), \quad (4.174)$$

then, based on a 20-point Gauss-Legendre rule, method of Drezner and Wesolowsky [1990] would be a double precision algorithm⁵ with a maximum error of 5×10^{-16} if the threshold for ρ changes to 0.925 rather than 0.70.

Example 4-13

Based on Definition 4.0.5, for $\Psi = (0, \sigma, \boldsymbol{\theta})^\top$, the CDF of GSM model can be represented as

$$F(x|\Psi) = \begin{cases} \frac{1}{2} + \frac{\text{sign}(x)}{\sqrt{\pi}} \int_0^\infty H\left(\frac{x^2}{2\sigma^2 z^2} \middle| \boldsymbol{\theta}\right) \exp\{-z^2\} dz, & \text{if } \eta(g) = g, \\ \frac{1}{2} + \frac{\text{sign}(x)}{\sqrt{\pi}} \int_0^\infty \bar{H}\left(\frac{2\sigma^2 z^2}{x^2} \middle| \boldsymbol{\theta}\right) \exp\{-z^2\} dz, & \text{if } \eta(g) = \frac{1}{g}, \end{cases} \quad (4.175)$$

where $H(u|\boldsymbol{\theta})$ is the CDF of mixing variable G in Definition 4.0.5 and $\bar{H}(u|\boldsymbol{\theta}) = 1 - H(u|\boldsymbol{\theta})$. Hence, $F(x|\Psi)$ in (4.175) can be approximated using the Gauss-Hermite rule of third (or first) kind. As the second method, the CDF $F(x|\Psi)$ can be obtained directly by integration the corresponding PDF. For example, suppose in Definition 4.0.5 we have $G \sim \mathcal{G}(\nu/2, \nu/2)$ and $\eta(G) = G^{-1}$. Then, clearly $F(\cdot|\Psi)$ in (4.175) becomes the CDF of Student's t distribution with ν degrees of freedom. That is

$$F(x|\Psi) = \frac{1}{2} + \frac{\text{sign}(x)}{2\sqrt{\pi}} \frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})\sqrt{\nu\pi}\sigma} \int_0^{|x|} \left(1 + \frac{z^2}{\nu\sigma^2}\right)^{-\frac{\nu+1}{2}} dz, \quad (4.176)$$

The R function `pt(x,df,...)` computes the CDF of Student's t distribution with degrees of freedom `df`. Table 4.13 shows the results for approximating $F(\cdot|\Psi)$ in which $\Psi = (0, 1, \nu)^\top$ using a k -point Gauss-Hermite rule of third kind and a k -point Gaussian-Legendre rule applied, accordingly, on RHS of (4.175), when $\eta(G) = G^{-1}$, and (4.176). Our investigation reveals that using a hybrid process, described as follow, the maximum ARE would be less or equal than 8×10^{-10} . Based on the hybrid process, a 10-point Gauss-Hermite rule is applied when $\nu \leq 8$ and $x \geq 3.5\sigma$ and a 10-point Gaussian-Legendre rule otherwise. ■

⁴In single precision algorithm, of a total of 32 bits, machine assigns one bit for sign, 8 bits for exponent, and 23 bits for fraction part of a real number. Therefore, a single precision format approximately provides a $\log_{10}(2^{24}) = 7.22471 \dots \approx 7$ digits of precision.

⁵In double precision algorithm, of a total of 64 bits, machine assigns one bit for sign, 11 bits for exponent, and 52 bits for fraction part of a real number. Therefore, a double precision format approximately provides a $\log_{10}(2^{53}) = 15.9545 \dots \approx 16$ digits of precision.

Table 4.13: Computed ARF for approximating the CDF of Student's t using k -point Gaussian-Legendre and Gauss-Hermite rules.

k	ν	ARE	
		Gauss-Hermite	Gauss-Legendre
2	2	0.02424	0.00013
2	10	0.03742	0.00012
2	40	0.02748	0.00014
5	2	0.00018	1.81e-08
5	10	0.01700	5.73e-10
5	40	0.02720	6.11e-11
10	2	6.31e-08	2.81e-16
10	10	9.22e-04	1.33e-16
10	40	0.01706	3.97e-16

4.4.2 Computing the CDF of bivariate Student's t distribution

Herein, we describe the method of Genz [2004] for computing the CDF of a bivariate Student's t distribution. The CDF of a p -dimensional Student's t distribution with ν degrees of freedom is

$$T_\nu(\mathbf{b}|\boldsymbol{\mu}, R) = \frac{(\nu\pi)^{-\frac{p}{2}} \Gamma(\frac{\nu+p}{2})}{\sqrt{|R|} \Gamma(\frac{\nu}{2})} \int_{-\infty}^{b_1} \cdots \int_{-\infty}^{b_p} \left[1 + \frac{(\mathbf{x} - \boldsymbol{\mu})^\top R^{-1}(\mathbf{x} - \boldsymbol{\mu})}{\nu} \right]^{-\frac{\nu+p}{2}} dx_p \cdots dx_1, \quad (4.177)$$

where $R = \{\rho_{i,j}\}$ denotes the correlation matrix. In bivariate case, the CDF of Student's t is given by

$$T_\nu(\mathbf{b}|\mathbf{0}, \rho) = \frac{1}{2\pi\sqrt{1-\rho^2}} \int_{-\infty}^{b_1} \int_{-\infty}^{b_2} \left[1 + \frac{x_1^2 + x_2^2 - 2\rho x_1 x_2}{\nu(1-\rho^2)} \right]^{-\frac{\nu+2}{2}} dx_2 dx_1, \quad (4.178)$$

where $\mathbf{b} = (b_1, b_2)^\top$ denotes a vector of real constants. Recall from Definition 4.0.5 that each Student's t is a Gaussian scale mixture model. Hence, setting $\boldsymbol{\mu} = \mathbf{0}$, $\Sigma = R = [(1, \rho)^\top, (\rho, 1)^\top]$, $\eta(G) = G$, and $G \sim \mathcal{G}(\nu/2, \nu/2)$, and then applying relation (??), the CDF of bivariate Student's t distribution given in (4.178) can be represented as

$$\begin{aligned} T_\nu(\mathbf{b}|\mathbf{0}, \rho) &= F(\mathbf{b}|\Psi = (\rho, \nu)^\top) = \int_0^\infty \Phi_2\left(\mathbf{b}|\mathbf{0}, \frac{\Sigma}{g}\right) \mathcal{G}\left(g|\frac{\nu}{2}, \frac{\nu}{2}\right) dg \\ &= \int_0^\infty \Phi_2(\sqrt{g}\mathbf{b}|\mathbf{0}, R = \rho) \mathcal{G}\left(g|\frac{\nu}{2}, \frac{\nu}{2}\right) dg. \end{aligned}$$

For computational purpose, using transformation $g\nu = u^2$, we obtain another representation of (4.178) given by [Cornish, 1954] as

$$T_\nu(\mathbf{b}|\mathbf{0}, \rho) = \frac{2^{1-\nu/2}}{\Gamma(\frac{\nu}{2})} \int_0^\infty \Phi_2\left(\frac{u\mathbf{b}}{\sqrt{\nu}}|\mathbf{0}, \rho\right) u^{\nu-1} \exp\left\{-\frac{u^2}{2}\right\} du, \quad (4.179)$$

By applying the Plackett's formula (4.154) on the term $\Phi_2(\cdot|\mathbf{0}, \rho)$ in RHS of (4.179), we obtain

$$\frac{\partial}{\partial \rho} T_\nu(\mathbf{b}|\mathbf{0}, \rho) = \frac{2^{1-\nu/2}}{\Gamma(\frac{\nu}{2})} \int_0^\infty \frac{u^{\nu-1}}{2\pi\sqrt{1-\rho^2}} \exp\left\{-\frac{u^2}{2}\left[1 + \frac{\delta(\mathbf{b}, \rho)}{\nu}\right]\right\} du, \quad (4.180)$$

where $\delta(\mathbf{b}, \rho)$ is given by

$$\begin{aligned}\delta(\mathbf{b}, \Sigma) &= \mathbf{b}^\top \Sigma^{-1} \mathbf{b} \\ &= \frac{b_1^2 + b_2^2 - 2\rho b_1 b_2}{(1 - \rho^2)} \\ &= \delta(\mathbf{b}, \rho).\end{aligned}\tag{4.181}$$

Applying the change of variable $z = u\sqrt{1 + \delta(\mathbf{b}, \rho)/\nu}$ on RHS of (4.180), it turns out that

$$\begin{aligned}\frac{\partial}{\partial \rho} T_\nu(\mathbf{b}|\mathbf{0}, \rho) &= \frac{2^{1-\nu/2}}{\Gamma(\frac{\nu}{2})2\pi\sqrt{1-\rho^2}} \left[1 + \frac{\delta(\mathbf{b}, \rho)}{\nu}\right]^{-\frac{\nu}{2}} \int_0^\infty z^{\nu-1} \exp\left\{-\frac{z^2}{2}\right\} dz, \\ &= \frac{1}{2\pi\sqrt{1-\rho^2}} \left[1 + \frac{\delta(\mathbf{b}, \rho)}{\nu}\right]^{-\frac{\nu}{2}}.\end{aligned}\tag{4.182}$$

Integrating both sides of (4.182) over the range (s, ρ) shows

$$T_\nu(\mathbf{b}|\mathbf{0}, \rho) = T_\nu(\mathbf{b}|\mathbf{0}, s) + \frac{1}{2\pi} \int_s^\rho \frac{\left[1 + \frac{\delta(\mathbf{b}, r)}{\nu}\right]^{-\frac{\nu}{2}}}{\sqrt{1-r^2}} dr.\tag{4.183}$$

where $s = \text{sign}(\rho)$ and

$$T_\nu(\mathbf{b}|\mathbf{0}, s) = \begin{cases} T_\nu(\min\{b_1, b_2\}), & \text{if } s = +1, \\ \max\{0, T_\nu(b_1) - T_\nu(-b_2)\}, & \text{if } s = -1. \end{cases}$$

Applying a change of variable $r = \sin(\theta)$ on (4.183), it turns out that

$$T_\nu(\mathbf{b}|\mathbf{0}, \rho) = T_\nu(\mathbf{b}|\mathbf{0}, s) + \frac{1}{2\pi} \int_{s\frac{\pi}{2}}^{\sin^{-1}(\rho)} \left[1 + \frac{b_1^2 + b_2^2 - 2\sin(\theta)b_1 b_2}{\nu \cos(\theta)^2}\right]^{-\frac{\nu}{2}} d\theta.\tag{4.184}$$

It is worthwhile to mention that if $\rho = 0$, then the marginals of multivariate Student's t are not independent and hence, $T_\nu(\mathbf{b}|\mathbf{0}, 0)$ cannot be expressed as the product of $T_\nu(b_1|0, 1)$ and $T_\nu(b_2|0, 1)$. Furthermore, for any positive small constant such as ϵ , if $|\rho| \leq \epsilon$, then based on the continuity property of $T_\nu(\mathbf{b}|\mathbf{0}, \rho)$, formula (4.184) can be applied for computing $T_\nu(\mathbf{b}|\mathbf{0}, \rho)$ if ρ and s are, accordingly, replaced with ϵ and $-\epsilon$ where ϵ is sufficiently small, $\epsilon = 10e-8$ say. Genz [2004] uses a k -point Gauss-Legendre rule for approximating bivariate Student's t probabilities using (4.184). The maximum recorded of absolute errors are 10^{-3} , 10^{-4} , 10^{-6} , and 6×10^{-11} when $k = 6, 12, 24$, and 48 , respectively. In what follows, we give the pertinent R function `BVT_quad` for computing $T_\nu(\mathbf{b}|\mathbf{0}, R)$.

Example 4-14

```
1 R> BVT_Genz <- function(b, rho, nu, k)
2 +{
3 + T1 <- max(0, pt( q = b[1], df = nu ) - pt( q = -b[2], df = nu ))
4 + T2 <- pt( q = min(b), df = nu )
5 + rho <- ifelse( abs(rho) < 10e-8, 10e-8, rho)
6 + s <- sign(rho)
7 + p1 <- ifelse(s < 0, T1, T2)
8 + out <- quad_rule(k, type = "LE", alpha = 0, beta = 0)
9 + node <- out$node; w <- out$weight;
10 + x <- (node + 1)*( asin(rho) - s*pi/2 )/2 + s*pi/2
```

```

11 + delta <- ( b[1]^2 + b[2]^2 - 2*sin(x)*b[1]*b[2] )/cos(x)^2
12 + p2 <- sum( w*( 1 + delta/nu )^(-nu/2) )
13 + CDF <- p1 + ( asin(rho) - s*pi/2 )/(4*pi)*p2
14 + return(CDF)
15 +}

```

It is worth to note that the built-in function `pt(.,.)` in the fourth line of function `BVT_Genz` can be replaced with that of has been just suggested in Example 4-13. ■

4.4.3 Computing the CDF of bivariate Gaussian scale mixture distribution

Herein, Theorem 4.4.1 gives a general framework for computing the CDF of bivariate Gaussian scale mixture distribution.

Theorem 4.4.1. *Let $\mathcal{M}_G(t|\boldsymbol{\theta}) = E(\exp\{tG\})$ denote the MGF of the mixing variable G in Definition 4.0.5. Set $\eta(G) = G$ and $\Psi = (\mathbf{0}, R, \boldsymbol{\theta})$ in which $R = [(1, \rho)^\top, (\rho, 1)^\top]$. Then*

$$F(\mathbf{x}|\Psi) = \begin{cases} F(\min\{x_1, x_2\}) + \frac{1}{2\pi} \int_{\frac{\pi}{2}}^{\sin^{-1}(\rho)} \mathcal{M}_G\left[-\frac{\delta(\mathbf{x}, \sin(u))}{2}\right] du, & \text{if } \rho > 0, \\ \max\{0, F(x_2) - F(-x_1)\} + \frac{1}{2\pi} \int_{-\frac{\pi}{2}}^{\sin^{-1}(\rho)} \mathcal{M}_G\left[-\frac{\delta(\mathbf{x}, \sin(u))}{2}\right] du, & \text{if } \rho < 0, \end{cases} \quad (4.185)$$

where $|\rho| < 1$ and $\delta(\mathbf{x}, u)$, for $|u| < 1$, is defined as (4.181).

Proof: Suppose in Definition 4.0.5 that $\boldsymbol{\mu} = \mathbf{0}$, $\eta(G) = G$, $p = 2$, and $\Sigma^{1/2} = [(1, 0)^\top, (\rho, \sqrt{1-\rho^2})^\top]$ for which the scale (correlation) matrix is $\Sigma = R = [(1, \rho)^\top, (\rho, 1)^\top]$. Then $\Psi = (\mathbf{0}, R, \boldsymbol{\theta})$ and the CDF of $\mathbf{X}$ becomes

$$F(\mathbf{x}|\Psi) = \int_0^\infty \Phi_2(\sqrt{g}\mathbf{x}|\mathbf{0}, \rho) h(g|\boldsymbol{\theta}) dg, \quad (4.186)$$

where $\mathbf{x}^\top = (x_1, x_2)$. Taking into account the fact

$$\Phi_2(\sqrt{g}\mathbf{x}|\mathbf{0}, \rho) = L(-\sqrt{g}x_1, -\sqrt{g}x_2|\rho), \quad (4.187)$$

where $L(x, y|\rho)$ is given in (4.152) and then applying the Plackett's formula (4.154) on the both sides of (4.186), we have

$$\begin{aligned} \frac{\partial}{\partial \rho} F[\mathbf{x}|\Psi = (\mathbf{0}, \rho, \boldsymbol{\theta})] &= \int_0^\infty \frac{\partial}{\partial \rho} \Phi_2(\sqrt{g}\mathbf{x}|\mathbf{0}, \rho) h(g|\boldsymbol{\theta}) dg \\ &= \int_0^\infty \frac{1}{2\pi\sqrt{1-\rho^2}} \exp\left\{-g\frac{x_1^2 + x_2^2 - 2x_1x_2\rho}{2(1-\rho^2)}\right\} h(g|\boldsymbol{\theta}) dg \\ &= \frac{1}{2\pi\sqrt{1-\rho^2}} \int_0^\infty \exp\left\{-g\frac{\delta(\mathbf{x}, \rho)}{2}\right\} h(g|\boldsymbol{\theta}) dg \\ &= \frac{1}{2\pi\sqrt{1-\rho^2}} \mathcal{M}_G\left(-\frac{1}{2}\delta(\mathbf{x}, \rho)|\boldsymbol{\theta}\right), \end{aligned} \quad (4.188)$$

where $\delta(\mathbf{b}, \rho)$ is given in (4.181) and $\mathcal{M}_G(\cdot)$ denotes the moment generating function (MGF) of random variable G . Integrating both sides of (4.188) from $s = \text{sign}(\rho)$ to ρ yields

$$F[\mathbf{x}|\Psi = (\mathbf{0}, \rho, \boldsymbol{\theta})] = F[\mathbf{x}|\Psi = (\mathbf{0}, s, \boldsymbol{\theta})] + \frac{1}{2\pi} \int_s^\rho \frac{1}{\sqrt{1-r^2}} \mathcal{M}_G\left(-\frac{1}{2}\delta(\mathbf{x}, r)|\boldsymbol{\theta}\right) dr.$$

Using a change of variable of the form $r = \sin(u)$, we obtain

$$F[\mathbf{x}|\Psi = (\mathbf{0}, \rho, \boldsymbol{\theta})] = F[\mathbf{x}|\Psi = (\mathbf{0}, s, \boldsymbol{\theta})] + \frac{1}{2\pi} \int_{s\frac{\pi}{2}}^{\sin^{-1}(\rho)} \mathcal{M}_G\left(-\frac{1}{2}\delta(\mathbf{x}, \sin(u))|\boldsymbol{\theta}\right) du. \quad (4.189)$$

For computing $F(\mathbf{x}|\Psi)$ in RHS of (4.189), it follows from (4.186) that

$$F[\mathbf{x}|\Psi = (\mathbf{0}, s, \boldsymbol{\theta})] = \int_0^\infty \Phi_2(\sqrt{g}\mathbf{x}|\mathbf{0}, s)h(g|\boldsymbol{\theta})dg. \quad (4.190)$$

From (4.187) it turns out that

$$F[\mathbf{x}|\Psi = (\mathbf{0}, s, \boldsymbol{\theta})] = \int_0^\infty L(-\sqrt{g}x_1, -\sqrt{g}x_2|s)h(g|\boldsymbol{\theta})dg, \quad (4.191)$$

and based on (4.173), we can write

$$\begin{aligned} F[\mathbf{x}|\Psi = (\mathbf{0}, s, \boldsymbol{\theta})] &= \begin{cases} \int_0^\infty \Phi(-\max\{-\sqrt{g}x_1, -\sqrt{g}x_2\})h(g|\boldsymbol{\theta})dg, & \text{if } s = +1, \\ \int_0^\infty \max\{0, \Phi(\sqrt{g}x_2) - \Phi(-\sqrt{g}x_1)\}h(g|\boldsymbol{\theta})dg, & \text{if } s = -1. \end{cases} \\ &= \begin{cases} \int_0^\infty \Phi(\sqrt{g}\min\{x_1, x_2\})h(g|\boldsymbol{\theta})dg, & \text{if } s = +1, \\ \max\left\{0, \int_0^\infty \Phi(\sqrt{g}x_2)h(g|\boldsymbol{\theta})dg - \int_0^\infty \Phi(-\sqrt{g}x_1)h(g|\boldsymbol{\theta})dg\right\}, & \text{if } s = -1. \end{cases} \\ &= \begin{cases} F(\min\{x_1, x_2\}), & \text{if } s = +1, \\ \max\{0, F(x_2) - F(-x_1)\}, & \text{if } s = -1. \end{cases} \end{aligned} \quad (4.192)$$

Proof is complete by substituting RHS of (4.192) into (4.189).

We note that when $\rho = 0$, then Theorem 4.4.1 can be used if $\Psi = (\mathbf{0}, R = \{0\}, \boldsymbol{\theta})$ is replaced with $\Psi' = (\mathbf{0}, R = \{\epsilon\}, \boldsymbol{\theta})$ in which where ϵ is sufficiently small positive constant, $\epsilon = 10e - 8$ say. Corollary 4.4.1 given below is useful when the MGF of mixing distribution has not closed form (or does not exist), but its characteristic function (CF) does exist.

Corollary 4.4.1. *Let $\varphi_G(t|\boldsymbol{\theta}) = E(\exp\{itG\})$ denote the CF of the mixing variable G in Definition 4.0.5. Set $\eta(G) = G$ and $\Psi = (\mathbf{0}, R, \boldsymbol{\theta})$ in which $R = [(1, \rho)^\top, (\rho, 1)^\top]$. Then*

$$F(\mathbf{x}|\Psi) = \begin{cases} F(\min\{x_1, x_2\}) + \frac{1}{2\pi} \int_{\frac{\pi}{2}}^{\sin^{-1}(\rho)} \varphi_G\left[i \frac{\delta(\mathbf{x}, \sin(u))}{2}\right] du, & \text{if } \rho > 0, \\ \max\{0, F(x_2) - F(-x_1)\} + \frac{1}{2\pi} \int_{-\frac{\pi}{2}}^{\sin^{-1}(\rho)} \varphi_G\left[i \frac{\delta(\mathbf{x}, \sin(u))}{2}\right] du, & \text{if } \rho < 0, \end{cases}$$

where $i^2 = -1$, $|\rho| < 1$ and $\delta(\mathbf{x}, u)$ is defined as (4.181).

Corollary 4.4.2. *Let $\mathcal{M}_{G^{-1}}(t|\boldsymbol{\theta}) = E(\exp\{tG^{-1}\})$ denote the MGF of the mixing variable G^{-1} in Definition 4.0.5. Set $\eta(G) = G^{-1}$ and $\Psi = (\mathbf{0}, R, \boldsymbol{\theta})$ in which $R = [(1, \rho)^\top, (\rho, 1)^\top]$. Then*

$$F(\mathbf{x}|\Psi) = \begin{cases} F(\min\{x_1, x_2\}) + \frac{1}{2\pi} \int_{\frac{\pi}{2}}^{\sin^{-1}(\rho)} \mathcal{M}_{G^{-1}}\left(-\frac{\delta(\mathbf{x}, \sin(u))}{2}\right) du, & \text{if } \rho > 0, \\ \max\{0, F(x_2) - F(-x_1)\} + \frac{1}{2\pi} \int_{-\frac{\pi}{2}}^{\sin^{-1}(\rho)} \mathcal{M}_{G^{-1}}\left(-\frac{\delta(\mathbf{x}, \sin(u))}{2}\right) du, & \text{if } \rho < 0, \end{cases}$$

where $i^2 = -1$, $|\rho| < 1$ and $\delta(\mathbf{x}, u)$, for $|u| < 1$, is defined as (4.181).

Theorem 4.4.1, Corollary 4.4.1, and Corollary 4.4.2 are also applicable for computing the CDF of a GSM distribution when the parameter vector Ψ involves the scale matrix $\Sigma = [(\sigma_{1,1}, \sigma_{1,2})^\top, (\sigma_{2,1}, \sigma_{2,2})^\top]$ rather than the correlation one. Corollary (4.4.3) given by the following is useful in this case.

Corollary 4.4.3. Let $\Psi' = (\boldsymbol{\mu}, \Sigma, \boldsymbol{\theta})$ and $\Psi = (\mathbf{0}, R, \boldsymbol{\theta})$. Then

$$F(\mathbf{x}|\Psi') = F\left(\frac{x_1 - \mu_1}{\sqrt{\sigma_{1,1}}}, \frac{x_2 - \mu_2}{\sqrt{\sigma_{2,2}}} \middle| \Psi\right),$$

where $R = \{\rho_{i,j}\}$ and $\Sigma = \{\sigma_{i,j}\}$, for $i, j = 1, 2$, are the correlation and covariance matrices, respectively.

Theorem 4.4.1 and Corollary 4.4.3 are useful for computing the CDF of bivariate Student's t [Kotz and Nadarajah, 2004] and SH [McNeil et al., 2005] distributions, respectively. To this end, we consider two cases given by the following.

- **Student's t :** Theorem 4.4.1 specializes to the method of Genz [2004] given in (4.184) for computing the CDF of bivariate Student's t distribution if $G \sim \mathcal{G}(\nu/2, \nu/2)$, for which $\mathcal{M}_G(t) = (1 - 2t/\nu)^{\nu/2}$, $\eta(g) = g$, and $\Psi = (\mathbf{0}, R = \{\rho\}, \boldsymbol{\theta} = \nu)^\top$. Here, $F(\cdot)$ is the CDF of univariate Student's t with ν degrees of freedom.
- **SH:** For computing the CDF $F[\mathbf{x}|\Psi = (\mathbf{0}, R = \{\rho\}, \boldsymbol{\theta} = (a, \lambda)^\top)]$ whose PDF is given in (4.18), we may use Corollary 4.4.2 when $F(\cdot)$ is the CDF of univariate SH distribution and $\mathcal{M}_{G^{-1}}[t|\boldsymbol{\theta} = (a, a, \lambda)^\top]$ where

$$\mathcal{M}_{G^{-1}}[t|\boldsymbol{\theta} = (\psi, \chi, \lambda)^\top] = \mathcal{M}_G[t|\boldsymbol{\theta} = (\psi, \chi, -\lambda)^\top], \quad (4.193)$$

in which $G \sim \mathcal{GIG}(\psi, \chi, \lambda)$ with PDF (2.51) and CF

$$\mathcal{M}_G[t|\boldsymbol{\theta} = (\psi, \chi, \lambda)^\top] = \left(\frac{\psi}{\psi - 2t}\right)^{\frac{\lambda}{2}} \frac{\mathcal{K}_\lambda(\sqrt{\chi(\psi - 2t)})}{\mathcal{K}_\lambda(\sqrt{\chi\psi})}. \quad (4.194)$$

4.4.4 Computing the CDF of trivariate Gaussian distribution

- **Owen's method:** The efforts for numerical evaluation in three-dimensional case dates back to [Owen, 1956] that suggests the use of

$$\Phi_3(\mathbf{x}|\mathbf{0}, R) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x_1} \exp\left\{-\frac{y^2}{2}\right\} \Phi_2(\mathbf{z}(y)|\mathbf{0}, R^* = \{\rho^*\}) dy, \quad (4.195)$$

where $\mathbf{x}^\top = (x_1, x_2, x_3)$, $R = \{\rho_{i,j}\}$ denotes the associated 3×3 correlation matrix, $\mathbf{z}(y) = [z_1(y), z_2(y)]^\top$ in which

$$z_i(y) = \frac{x_{i+1} - \rho_{i+1,1} \times y}{\sqrt{1 - \rho_{i+1,1}^2}}, \quad i = 1, 2, \quad (4.196)$$

and

$$\rho^* = \frac{\rho_{2,3} - \rho_{2,1}\rho_{1,3}}{\prod_{i=1}^2 (1 - \rho_{i+1,1}^2)^{\frac{1}{2}}}. \quad (4.197)$$

Genz [2004] tested two methods for implementing the Owen's idea in (4.195) as follows. As the first method, using transformation $y = \Phi^{-1}(u)$, it follows that

$$\Phi_3(\mathbf{x}|\mathbf{0}, R) = \int_0^{\Phi(x_1)} \Phi_2\left\{\mathbf{z}[\Phi^{-1}(u)] \middle| \mathbf{0}, \rho^*\right\} du, \quad (4.198)$$

where $z_i(\cdot)$ and ρ^* , accordingly, are given by (4.196) and (4.197). The second method, that relies on the method of Drezner [1992], uses the Gauss-Hermite rule of third kind. To this end, applying the transformation $u = y - x_1$, then more algebra shows that

$$\Phi_3(\mathbf{x}|\mathbf{0}, R) = \frac{\exp\left\{-\frac{x_1^2}{2}\right\}}{\sqrt{2\pi}} \int_0^\infty \exp\left\{-\frac{u^2}{2} + ux_1\right\} \Phi_2(\mathbf{z}(u)|\mathbf{0}, \rho^*) du, \quad (4.199)$$

where $z_i(u) = x_1 - u$, for $i = 1, 2$, is given in (4.196). Since the shortest integration interval typically yields more accurate result, hence one can choose the smallest x_i , among the components of $\mathbf{x}$, as the upper bound of integration in RHS of (4.195) .

- **Plackett's method:** Despite its elegance, as pointed out by [Gassmann, 2003], the Plackett's approach seems to have been largely forgotten. Plackett [1954] derives the partial differential equation

$$\frac{\partial \phi_p(x_1, x_2, \dots, x_p | \mathbf{0}, R)}{\partial \rho_{i,j}} = \frac{\partial^2 \phi_p(x_1, x_2, \dots, x_p | \mathbf{0}, R)}{\partial x_i \partial x_j}, \quad (4.200)$$

and employed it for computing $\partial L_p(\mathbf{a}|R)/\partial \rho_{1,2}$ as follows.

$$\begin{aligned} \frac{\partial L_p(\mathbf{a}|R)}{\partial \rho_{1,2}} &= \int_{a_1}^{\infty} \int_{a_2}^{\infty} \dots \int_{a_p}^{\infty} \frac{\partial \phi_p(\mathbf{x} | \mathbf{0}, R)}{\partial \rho_{1,2}} dx_1 dx_2 \dots dx_p \\ &= \int_{a_3}^{\infty} \int_{a_4}^{\infty} \dots \int_{a_p}^{\infty} \left[\int_{a_1}^{\infty} \int_{a_2}^{\infty} \frac{\partial \phi_p(\mathbf{x} | \mathbf{0}, R)}{\partial x_1 x_2} dx_1 dx_2 \right] dx_3 dx_4 \dots dx_p. \end{aligned} \quad (4.201)$$

Using property (4.200), the RHS of (4.201) becomes

$$\frac{\partial L_p(\mathbf{a}|R)}{\partial \rho_{1,2}} = \int_{a_3}^{\infty} \int_{a_4}^{\infty} \dots \int_{a_p}^{\infty} \phi_p \left[(a_1, a_2, x_3, x_4, \dots, x_p)^\top | \mathbf{0}, R \right] dx_1 dx_2 dx_3 dx_4 \dots dx_p. \quad (4.202)$$

Let

$$R = \begin{pmatrix} R_{11} & R_{12} \\ R_{21} & R_{22} \end{pmatrix}, \quad (4.203)$$

be a decomposition of matrix R in which

$$R_{11} = [(1, \rho_{2,1})^\top, (\rho_{1,2}, 1)^\top]. \quad (4.204)$$

Furthermore, set

$$\tilde{R} = R_{22} - R_{21} R_{11}^{-1} R_{12}, \quad (4.205)$$

and $\mathbf{x}_{[-1,-2]}(\rho_{1,2})$ is vector of length $p-2$ defined as

$$\mathbf{x}_{[-1,-2]}(\rho_{1,2}) = (x_3, x_4, \dots, x_p)^\top - R_{21} R_{11}^{-1} \times (a_1, a_2)^\top, \quad (4.206)$$

whose elements, for $k = 3, 4, \dots, p$, are

$$x_k - \frac{(\rho_{1,k} - \rho_{1,2} \rho_{2,k}) a_1 + (\rho_{2,k} - \rho_{2,1} \rho_{1,k}) a_2}{1 - \rho_{1,2}^2}. \quad (4.207)$$

Based on decomposition (4.203), the integrand in the RHS of (4.202) can be expressed as the product of a 2-dimensional unconditional and a $(p-2)$ -dimensional conditional Gaussian distributions. It follows that

$$\begin{aligned} \frac{\partial L_p(\mathbf{a}|R)}{\partial \rho_{1,2}} &= \phi_2[(a_1, a_2)^\top | \mathbf{0}, R_{11}] \int_{a_3}^{\infty} \int_{a_4}^{\infty} \dots \int_{a_p}^{\infty} \phi_{p-2}[\mathbf{x}_{[-1,-2]}(\rho_{1,2}) | \mathbf{0}, \tilde{R}] dx_3 dx_4 \dots dx_p, \\ &= \phi_2[(a_1, a_2)^\top | \mathbf{0}, R_{11}] \times L_{p-2}[\mathbf{a}_{[-1,-2]}(\rho_{1,2}) | \tilde{R}], \end{aligned} \quad (4.208)$$

where R_{11} , $\tilde{R} = \{\tilde{\rho}_{i,j}\}$, and $\mathbf{a}_{[-1,-2]}(\rho_{1,2})$ are given, accordingly, in (4.204), (4.205), and (4.206). For any choice of R such as $R^* = \{\rho_{i,j}^*\}$ for which computing $L_p(\mathbf{a}|R^*)$ is feasible easily, consider the positive definite matrix

$$\Omega(t) = \{\omega_{i,j}(t)\} = tR + (1-t)R^*, \quad (4.209)$$

for $t \in (0, 1]$. It can be shown that

$$L_p(\mathbf{a}|R) = L_p(\mathbf{a}|R^*) + \sum_{i < j}^p \sum_{j=1}^p \int_0^1 (\rho_{i,j} - \rho_{i,j}^*) \frac{\partial L_p(\mathbf{a}|\Omega(t))}{\partial \rho_{i,j}} dt. \quad (4.210)$$

The quantities $L_p(\mathbf{a}|R)$ and $L_p(\mathbf{a}|R^*)$ are known in the literature as the *target* and *reference* probabilities, respectively [Gassmann, 2003]. The Plackett's recursion formula is then, by combining (4.208) and (4.210), given by

$$\begin{aligned} L_p(\mathbf{a}|R) = & L_p(\mathbf{a}|R^*) + \sum_{i < j}^p \sum_{j=1}^p \int_0^1 (\rho_{i,j} - \rho_{i,j}^*) \\ & \times \phi_2[(a_i, a_j)^\top | \mathbf{0}, R_{11}(t)] \times L_{p-2}[\mathbf{a}_{[-i,-j]}(\omega_{i,j}(t)) | \tilde{\Omega}(t)] dt, \end{aligned} \quad (4.211)$$

where $R_{11}(t) = [(1, \omega_{i,j}(t))^\top, (\omega_{j,i}(t), 1)^\top]$, $\tilde{\Omega}(t) = t\tilde{R} + (1-t)R^*$ in which $\tilde{R}$ is given in (4.205), and elements of $\mathbf{a}_{[-i,-j]}(\omega_{i,j}(t))$ are given, for $k \neq \{i, j\} = 1, \dots, p$, by

$$a_k - \frac{[\omega_{i,k}(t) - \omega_{i,j}(t)\omega_{j,k}(t)]a_i + [\omega_{j,k}(t) - \omega_{j,i}(t)\omega_{i,k}(t)]a_j}{1 - \omega_{i,j}^2(t)}. \quad (4.212)$$

It is immediate from (4.211) that, if $\rho_{i,j} \geq \rho_{i,j}^*$ for all $i < j$, then $L_p(\mathbf{a}|R) \geq L_p(\mathbf{a}|R^*)$ Slepian [1962].

Drezner [1994] introduces two approaches for computing $L_3(\mathbf{a}|R)$ based on the Plackett's method as follows.

I. First method: it follows from (4.208) that

$$\frac{\partial L_3(\mathbf{a}|R)}{\partial \rho_{1,2}} = \phi_2[(a_1, a_2)^\top | \mathbf{0}, R_{11}] \times L_1(a_{[-1,-2]}(\rho_{1,2}) | 1), \quad (4.213)$$

where $\mathbf{a} = (a_1, a_2, a_3)^\top$, $R_{11} = [(1, \rho_{1,2})^\top, (\rho_{2,1}, 1)^\top]$, and

$$\begin{aligned} a_{[-1,-2]}(\rho_{1,2}) = & a_3 - R_{21}R_{11}^{-1} \times (a_1, a_2)^\top \\ = & a_3 - \frac{a_2[\rho_{2,3} - \rho_{2,1}\rho_{1,3}] + a_1[\rho_{1,3} - \rho_{1,2}\rho_{2,3}]}{(1 - \rho_{1,2}^2)}. \end{aligned}$$

Using the correlation matrix

$$R_1^* = \mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad (4.214)$$

for the reference probability it follows that

$$L_3(\mathbf{a}|R_1^\star) = \prod_{m=1}^3 L_1(a_m|1) = \prod_{m=1}^3 \bar{\Phi}(a_m), \quad (4.215)$$

$$\Omega(t) = \begin{bmatrix} 1 & \omega_{1,2}(t) & \omega_{1,3}(t) \\ \omega_{2,1}(t) & 1 & \omega_{2,3}(t) \\ \omega_{3,1}(t) & \omega_{3,2}(t) & 1 \end{bmatrix} = \begin{bmatrix} 1 & t\rho_{1,2} & t\rho_{1,3} \\ t\rho_{2,1} & 1 & t\rho_{2,3} \\ t\rho_{3,1} & t\rho_{3,2} & 1 \end{bmatrix}, \quad (4.216)$$

$$R_{11} = R_{11}(t) = [(1, t\rho_{i,j})^\top, (t\rho_{j,i}, 1)^\top], \quad (4.217)$$

$$\tilde{\Omega}(t) = 1 - \frac{t^2\rho_{i,k}^2 + t^2\rho_{j,k}^2 - 2t^3\rho_{i,j}\rho_{i,k}\rho_{j,k}}{1 - t^2\rho_{i,j}^2}, \quad (4.218)$$

$$a_{[-i,-j]}(\rho_{i,j}(t)) = a_k - \frac{[t\rho_{i,k} - t^2\rho_{i,j}\rho_{j,k}]a_i + [t\rho_{j,k} - t^2\rho_{j,i}\rho_{i,k}]a_j}{1 - t^2\rho_{i,j}^2}, \quad (4.219)$$

where $i < j$ and $k \neq \{i, j\} = 1, \dots, 3$. Applying information given in (4.215)-(4.219) on the RHS of (4.211) for $p = 3$, then more algebra shows

$$L_3(\mathbf{a}|R) = \prod_{i=1}^3 L_1(a_i|1) + \sum_{i < j}^3 \int_0^1 \rho_{i,j} \phi_2[(a_i, a_j)^\top | \mathbf{0}, R_{11}(t)] \bar{\Phi}\left[\frac{a_{[-i,-j]}(\rho_{i,j}(t))}{\sqrt{\tilde{\Omega}(t)}}\right] dt,$$

where $\bar{\Phi}(\cdot) = 1 - \Phi(\cdot)$.

II. **Second method:** under the second approach, the correlation matrix of the reference probability is

$$R_2^\star = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & \rho_{2,3} \\ 0 & \rho_{3,2} & 1 \end{bmatrix}. \quad (4.220)$$

It follows that

$$L_3(\mathbf{a}|R_2^\star) = L_1(a_1|1) \times L_2[(a_2, a_3)^\top | R = \{\rho_{2,3}\}], \quad (4.221)$$

$$\Omega(t) = \begin{bmatrix} 1 & \omega_{1,2}(t) & \omega_{1,3}(t) \\ \omega_{2,1}(t) & 1 & \omega_{2,3}(t) \\ \omega_{3,1}(t) & \omega_{3,2}(t) & 1 \end{bmatrix} = \begin{bmatrix} 1 & t\rho_{1,2} & t\rho_{1,3} \\ t\rho_{2,1} & 1 & \rho_{2,3} \\ t\rho_{3,1} & \rho_{3,2} & 1 \end{bmatrix}, \quad (4.222)$$

$$R_{11}(t) = [(1, t\rho_{1,j})^\top, (t\rho_{j,1}, 1)^\top], \quad (4.223)$$

$$\tilde{\Omega}(t) = 1 - \frac{t^2\rho_{1,k}^2 + \rho_{k,j}^2 - 2t^2\rho_{1,j}\rho_{j,k}\rho_{1,k}}{1 - t^2\rho_{1,j}^2}, \quad (4.224)$$

$$a_{[-1,-j]}(\rho_{i,j}(t)) = a_k - \frac{[t\rho_{1,k} - t\rho_{1,j}\rho_{j,k}]a_1 + [\rho_{j,k} - t^2\rho_{1,j}\rho_{1,k}]a_j}{1 - t^2\rho_{1,j}^2}, \quad (4.225)$$

for $k \neq j = 2, 3$. Applying information given in (4.221)-(4.225) on the RHS of (4.211) for $p = 3$, then more algebra shows

$$\begin{aligned} L_3(\mathbf{a}|R) &= L_1(a_1|1) \times L_2[(a_2, a_3)^\top | R = \{\rho_{2,3}\}] \\ &+ \sum_{j=2}^3 \int_0^1 \rho_{1,j} \phi_2[(a_1, a_j)^\top | \mathbf{0}, R_{11}(t) = \{t\rho_{1,j}\}] \bar{\Phi}\left[\frac{a_{[-1,-j]}(\rho_{i,j}(t))}{\sqrt{\tilde{\Omega}(t)}}\right] dt, \end{aligned}$$

where $\bar{\Phi}(\cdot) = 1 - \Phi(\cdot)$.

Based on a 5-point Gauss-Legendre rule, as pointed out by Drezner [1994], both methods described above work well with maximum error less than 10^{-7} when $\rho_{\max} \leq 0.5$. If $\rho_{\max} = 0.9$, then in most cases the maximum error is limited by 10^{-7} , but attains 10^{-5} in very few cases. Moreover, it is shown that the error is more dependent on the determinant of R than it is on the magnitude of $\rho_{\max}$. It is shown, by simulation, that if $|R| > 0.15$, then the error is limited by 10^{-7} .

In what follows, the R functions TVN1_Drez and TVN2_Drez are given for implementing, accordingly, the first and second methods of Drezner [1994].

```

1 R> TVN1_Drez <- function(a, Mu, R)
2 +{
3 + p <- length(a); threshold <- rep(5, p); a <- a - Mu
4 + a <- ifelse( a < -threshold, -threshold, a )
5 + w <- c(0.2369268851, 0.4786286705, 0.5688888889, 0.4786286705, 0.2369268851)
6 + node <- c(0.9061798459, 0.5384693101, 0, -0.5384693101, -0.9061798459)
7 + Sum <- 0
8 + for( i in 1:(p - 1) )
9 + {
10 + for( j in (i + 1):p )
11 + {
12 + index <- c( R[ c(-i, -j), c(i, j) ] )
13 + r <- c(1:p)[ c(-i, -j) ]; t <- (node + 1)/2
14 + R11 <- matrix( c(1, R[i, j], R[i, j], 1), nrow = 2, ncol = 2)
15 + p1 <- sqrt( 1 - t^2*R[i, j]^2 )
16 + p2 <- exp( -1/2*( a[i]^2 + a[j]^2 - 2*t*a[i]*a[j]*R[i, j] )/p1^2 )/( 2*pi*p1 )
17 + Omega <- 1 - ( (t*R[r, i])^2 + (t*R[r, j])^2 - 2*t^3*R[r, i]*R[r, j]*R[i, j] )/p1^2
18 + a_tilde <- a[r] - ( ( t*R[r, i] - t^2*R[r, j]*R[i, j] )*a[i] +
19 + ( t*R[r, j] - t^2*R[r, i]*R[i, j] )*a[j] )/p1^2
20 + p3 <- pnorm( a_tilde/sqrt(Omega), lower.tail = F )
21 + Sum <- Sum + R[i, j]*sum( w*p2*p3 )/2
22 + }
23 + }
24 + prod( pnorm(a, lower.tail = F) ) + Sum
25 +}

```

```

1 R> TVN2_Drez <- function(a, Mu, R)
2 +{
3 + p <- length(a); threshold <- rep(5, p); a <- a - Mu
4 + a <- ifelse( a < -threshold, -threshold, a )
5 + w <- c(0.2369268851, 0.4786286705, 0.5688888889, 0.4786286705, 0.2369268851)
6 + node <- c(0.9061798459, 0.5384693101, 0, -0.5384693101, -0.9061798459)
7 + Sum <- 0
8 + for( j in 2:3 )
9 + {
10 + i <- 1
11 + index <- c( R[ c(-i, -j), c(i, j) ] )
12 + r <- c(1:p)[ c(-i, -j) ]
13 + t <- (node + 1)/2
14 + R11 <- matrix( c(1, R[i, j], R[i, j], 1), nrow = 2, ncol = 2)
15 + p1 <- sqrt( 1 - t^2*R[i, j]^2 )
16 + p2 <- exp( -1/2*( a[i]^2 + a[j]^2 - 2*t*a[i]*a[j]*R[i, j] )/p1^2 )/( 2*pi*p1 )
17 + Omega <- 1 - ( (t*R[r, i])^2 + (R[r, j])^2 - 2*t^2*R[r, i]*R[r, j]*R[i, j] )/p1^2
18 + a_tilde <- a[r] - ( (t*R[r, i] - t*R[r, j]*R[i, j]) *a[i] +
19 + ( R[r, j] - t^2*R[r, i]*R[i, j] )*a[j] )/p1^2
20 + p3 <- pnorm( a_tilde/sqrt(Omega), lower.tail = F )
21 + Sum <- Sum + R[i, j]*sum( w*p2*p3 )/2
22 + }
23 + R23 <- matrix( c(1, R[2,3], R[3,2], 1), nrow = 2, ncol = 2)
24 + pnorm(a[1], lower.tail = F)*BVN_Drez(a[-1], c(0, 0), R23) + Sum

```

Example 4-15

We are willing to compute $\Phi_3(\mathbf{a}|\mathbf{0}, R) = L_3(-\mathbf{a}|R)$ using the first method of Drezner [1994] for which $\mathbf{a} = (-1.2, -1.0, 0.5)^\top$ and $R = [(1, 0.7, -0.2)^\top, (0.7, 1, -0.4)^\top, (-0.2, -0.4, 1)^\top]$. Hence, command `TVN1_Drez(-c(-1.2, -1.0, 0.5), rep(0, 3), R)` gives

$$\Phi_3(\mathbf{a}|\mathbf{0}, R) \approx 0.208019949666041.$$

■

4.4.5 Computing the CDF of trivariate Student's t distribution

Herein, we describe four methods for computing the CDF of Student's t distribution with ν degrees of freedom proposed by Genz [2004]. The first three methods are based on the Plackett's formula for the multivariate Gaussian distribution. The last one relies on the method of [Owen, 1956] for computing the CDF of trivariate Gaussian distribution given in (4.195). First, we introduce those methods that are based on the Plackett's formula. To this end, we consider the CDF of the Student's t distribution, proposed by Cornish [1954], as follows.

$$T_\nu(\mathbf{b}|\mathbf{0}, R) = \frac{2^{1-\nu/2}}{\Gamma(\nu/2)} \int_0^\infty u^{\nu-1} \exp\left\{-\frac{u^2}{2}\right\} \Phi_3\left(\frac{u\mathbf{b}}{\sqrt{\nu}}|\mathbf{0}, R\right) du. \quad (4.226)$$

In order to generalize the Plackett's formula to the Student's t distribution, one needs to compute $\partial T_\nu(\mathbf{b}|\mathbf{0}, R) \partial \rho_{1,2}$. Using (4.226), it turns out that

$$\frac{\partial T_\nu(\mathbf{b}|\mathbf{0}, R)}{\partial \rho_{1,2}} = \frac{2^{1-\nu/2}}{\Gamma(\nu/2)} \int_0^\infty u^{\nu-1} \exp\left\{-\frac{u^2}{2}\right\} \frac{\partial}{\partial \rho_{1,2}} \Phi_3\left(\frac{u\mathbf{b}}{\sqrt{\nu}}|\mathbf{0}, R\right) du. \quad (4.227)$$

We notice that the Plackett's formula (4.208) can be represented as

$$\frac{\partial \Phi_3(\mathbf{b}|\mathbf{0}, R)}{\partial \rho_{i,j}} = \frac{\exp\left\{-\frac{1}{2}\delta[(b_i, b_j)^\top, \rho_{i,j}]\right\}}{2\pi\sqrt{1-\rho_{i,j}^2}} \Phi[h(\rho_{i,j})], \quad (4.228)$$

where $\mathbf{b} = (b_1, b_2, b_3)^\top$, $\delta(\cdot, \cdot)$ is given in (4.181), and

$$h(\rho_{i,j}, \rho_{i,k}) = \frac{b_k(1-\rho_{i,j}^2) - b_j(\rho_{j,k} - \rho_{i,j}\rho_{i,k}) - b_i(\rho_{i,k} - \rho_{i,j}\rho_{j,k})}{[(1-\rho_{i,j}^2)(1-\rho_{i,j}^2 - \rho_{i,k}^2 - \rho_{j,k}^2 + 2\rho_{i,j}\rho_{i,k}\rho_{j,k})]^{1/2}}, \quad (4.229)$$

with $k \neq i \neq j = \{1, 2, 3\}$. Substituting (4.228) into the RHS of (4.227) gives

$$\frac{\partial T_\nu(\mathbf{b}|\mathbf{0}, R)}{\partial \rho_{i,j}} = \frac{2^{1-\frac{\nu}{2}}}{\Gamma(\frac{\nu}{2})} \int_0^\infty \frac{u^{\nu-1} \exp\left\{-\frac{u^2}{2}\left(1 + \frac{1}{\nu}\delta[(b_i, b_j)^\top, \rho_{i,j}]\right)\right\}}{(2\pi)^{\frac{3}{2}}\sqrt{1-\rho_{i,j}^2}} \int_{-\infty}^{\frac{uh(\rho_{i,j}, \rho_{i,k})}{\sqrt{\nu}}} \exp\left\{-\frac{x^2}{2}\right\} dx du.$$

Applying change of variable $y = \sqrt{\nu}x/u$ yields

$$\frac{\partial T_\nu(\mathbf{b}|\mathbf{0}, R)}{\partial \rho_{i,j}} = \frac{2^{1-\frac{\nu}{2}}}{\sqrt{\nu}\Gamma(\frac{\nu}{2})} \int_{-\infty}^{h(\rho_{i,j}, \rho_{i,k})} \int_0^\infty \frac{u^\nu \exp\left\{-\frac{u^2}{2}\left[1 + \frac{\delta[(b_i, b_j)^\top, \rho_{i,j}] + y^2}{\nu}\right]\right\}}{(2\pi)^{\frac{3}{2}}\sqrt{1-\rho_{i,j}^2}} du dy.$$

Using another transformation of the form

$$r = u \left[1 + \frac{\delta[(b_i, b_j)^\top, \rho_{i,j}] + y^2}{\nu} \right]^{\frac{1}{2}},$$

giving

$$\begin{aligned} \frac{\partial T_\nu(\mathbf{b}|\mathbf{0}, R)}{\partial \rho_{i,j}} &= \frac{2^{1-\frac{\nu}{2}}}{\sqrt{\nu}\Gamma(\frac{\nu}{2})} \int_{-\infty}^{h(\rho_{i,j}, \rho_{i,k})} \frac{\left[1 + \frac{\delta[(b_i, b_j)^\top, \rho_{i,j}] + y^2}{\nu} \right]^{-\frac{\nu+1}{2}}}{(2\pi)^{\frac{3}{2}} \sqrt{1 - \rho_{i,j}^2}} \int_0^\infty r^\nu \exp\left\{-\frac{r^2}{2}\right\} dr dy \\ &= \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \int_{-\infty}^{h(\rho_{i,j}, \rho_{i,k})} \frac{\left[1 + \frac{\delta[(b_i, b_j)^\top, \rho_{i,j}] + y^2}{\nu} \right]^{-\frac{\nu+1}{2}}}{2\pi \sqrt{1 - \rho_{i,j}^2}} dy. \end{aligned}$$

Taking into account the fact that

$$\left[1 + \frac{\delta[(b_i, b_j)^\top, \rho_{i,j}] + y^2}{\nu} \right] = \left[1 + \frac{\delta[(b_i, b_j)^\top, \rho_{i,j}]}{\nu} \right] \times \left[1 + \frac{y^2}{\nu + \delta[(b_i, b_j)^\top, \rho_{i,j}]} \right],$$

it follows that

$$\frac{\partial T_\nu(\mathbf{b}|\mathbf{0}, R)}{\partial \rho_{i,j}} = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \frac{\left[1 + \frac{\delta[(b_i, b_j)^\top, \rho_{i,j}]}{\nu} \right]^{-\frac{\nu+1}{2}}}{2\pi \sqrt{1 - \rho_{i,j}^2}} \int_{-\infty}^{h(\rho_{i,j}, \rho_{i,k})} \left[1 + \frac{y^2}{\nu + \delta[(b_i, b_j)^\top, \rho_{i,j}]} \right]^{-\frac{\nu+1}{2}} dy.$$

We need the final transformation $z = y(1 + \nu^{-1}\delta[(b_i, b_j)^\top, \rho_{i,j}])^{-1/2}$ to see that

$$\frac{\partial T_\nu(\mathbf{b}|\mathbf{0}, R)}{\partial \rho_{i,j}} = \frac{\left[1 + \nu^{-1}\delta[(b_i, b_j)^\top, \rho_{i,j}] \right]^{-\frac{\nu}{2}}}{2\pi \sqrt{1 - \rho_{i,j}^2}} \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \int_{-\infty}^\eta \left[1 + \frac{z^2}{\nu} \right]^{-\frac{\nu+1}{2}} dz,$$

where $\eta = h(\rho_{i,j}, \rho_{i,k})(1 + \nu^{-1}\delta[(b_i, b_j)^\top, \rho_{i,j}])^{-1/2}$ or equivalently

$$\frac{\partial T_\nu(\mathbf{b}|\mathbf{0}, R)}{\partial \rho_{i,j}} = \frac{\left[1 + \nu^{-1}\delta[(b_i, b_j)^\top, \rho_{i,j}] \right]^{-\frac{\nu}{2}}}{2\pi \sqrt{1 - \rho_{i,j}^2}} T_\nu \left[\frac{h(\rho_{i,j}, \rho_{i,k})}{\sqrt{1 + \nu^{-1}\delta[(b_i, b_j)^\top, \rho_{i,j}]}} \right]. \quad (4.230)$$

Relation in (4.230) can be regarded as the Plackett's formula for the bivariate Student's t distribution. The first and second methods of Let $T_\nu(\mathbf{b}|\mathbf{0}, R_1^*)$ and $T_\nu(\mathbf{b}|\mathbf{0}, R_2^*)$ denote two reference probabilities in which

$$R_1^* = \begin{bmatrix} 1 & \rho_{1,2}^* & \rho_{1,3} \\ \rho_{2,1}^* & 1 & \rho_{2,3} \\ \rho_{3,1} & \rho_{3,2} & 1 \end{bmatrix} \quad \text{and} \quad R_2^* = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & \rho_{2,3} \\ 0 & \rho_{3,2} & 1 \end{bmatrix}. \quad (4.231)$$

Regarding $T_\nu(\mathbf{b}|\mathbf{0}, R_1^*)$ and $T_\nu(\mathbf{b}|\mathbf{0}, R_2^*)$ as the two reference probabilities, by integrating both sides of (4.230), the first and second method of Genz [2004] are given by the following.

I. **First method:** using the reference probability $T_\nu(\mathbf{b}|\mathbf{0}, R_1^*)$ for which R_1^* is given in (4.231), the first method suggests to compute $T_\nu(\mathbf{b}|\mathbf{0}, R)$ as follows.

$$\begin{aligned} T_\nu(\mathbf{b}|\mathbf{0}, R) &= T_\nu(\mathbf{b}|\mathbf{0}, R_1^*) \\ &+ \frac{1}{2\pi} \int_{\rho_{1,2}^*}^{\rho_{1,2}} \frac{\left(1 + \nu^{-1}\delta[(b_1, b_2)^\top, r] \right)^{-\frac{\nu}{2}}}{\sqrt{1 - r^2}} T_\nu \left[\frac{h(r\rho_{1,2}, r\rho_{1,3})}{\sqrt{1 + \nu^{-1}\delta[(b_1, b_2)^\top, r]}} \right] dr, \end{aligned} \quad (4.232)$$

where $h(\cdot, \cdot)$ is given in (4.229).

II. **Second method:** using the reference probability $T_\nu(\mathbf{b}|\mathbf{0}, R_1^*)$ for which R_2^* is given in (4.231), the second method suggests to compute $T_\nu(\mathbf{b}|\mathbf{0}, R)$ as follows.

$$\begin{aligned}
T_\nu(\mathbf{b}|\mathbf{0}, R) = & T_\nu(\mathbf{b}|\mathbf{0}, R_2^*) \\
& + \int_0^1 \frac{\left(1 + \nu^{-1}\delta[(b_1, b_2)^\top, t\rho_{1,2}]\right)^{-\frac{\nu}{2}}}{2\pi\rho_{1,2}^{-1}\sqrt{1-t^2\rho_{1,2}^2}} T_\nu\left[\frac{h(t\rho_{1,2}, t\rho_{1,3})}{\sqrt{1 + \nu^{-1}\delta[(b_1, b_2)^\top, t\rho_{1,2}]}}\right] dt \\
& + \int_0^1 \frac{\left(1 + \nu^{-1}\delta[(b_1, b_3)^\top, t\rho_{1,3}]\right)^{-\frac{\nu}{2}}}{2\pi\rho_{1,3}^{-1}\sqrt{1-t^2\rho_{1,3}^2}} T_\nu\left[\frac{h(t\rho_{1,3}, t\rho_{1,2})}{\sqrt{1 + \nu^{-1}\delta[(b_1, b_3)^\top, t\rho_{1,3}]}}\right] dt.
\end{aligned} \tag{4.233}$$

We note that the second method in (4.233) is computationally superior to the first one in (4.232).

III. **Third method:** efficient computing of (4.233) is possible if the reference probability $T_\nu(\mathbf{b}|\mathbf{0}, R_2^*)$ be replaced with

$$\begin{aligned}
T_\nu(\mathbf{b}|\mathbf{0}, R_2^*) = & T_\nu(\mathbf{b}|\mathbf{0}, R_3^*) \\
& + \frac{1}{2\pi} \int_s^{\rho_{2,3}} \frac{\left(1 + \nu^{-1}\delta[(b_2, b_3)^\top, r]\right)^{-\frac{\nu}{2}}}{\sqrt{1-r^2}} T_\nu\left[\frac{b_1}{\sqrt{1 + \nu^{-1}\delta[(b_2, b_3)^\top, r]}}\right] dr,
\end{aligned} \tag{4.234}$$

where

$$R_3^* = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & s \\ 0 & s & 1 \end{bmatrix},$$

with $s = \text{sign}(\rho_{2,3})$. Hence, by combining (4.233) and (4.234), then the third method suggests to compute $T_\nu(\mathbf{b}|\mathbf{0}, R)$ as follows.

$$\begin{aligned}
T_\nu(\mathbf{b}|\mathbf{0}, R) = & T_\nu(\mathbf{b}|\mathbf{0}, R_3^*) \\
& + \frac{1}{2\pi} \int_s^{\rho_{2,3}} \frac{\left(1 + \nu^{-1}\delta[(b_2, b_3)^\top, r]\right)^{-\frac{\nu}{2}}}{\sqrt{1-r^2}} T_\nu\left[\frac{b_1}{\sqrt{1 + \nu^{-1}\delta[(b_2, b_3)^\top, r]}}\right] dr \\
& + \int_0^1 \frac{\left(1 + \nu^{-1}\delta[(b_1, b_2)^\top, t\rho_{1,2}]\right)^{-\frac{\nu}{2}}}{2\pi\rho_{1,2}^{-1}\sqrt{1-t^2\rho_{1,2}^2}} T_\nu\left[\frac{h(t\rho_{1,2}, t\rho_{1,3})}{\sqrt{1 + \nu^{-1}\delta[(b_1, b_2)^\top, t\rho_{1,2}]}}\right] dt \\
& + \int_0^1 \frac{\left(1 + \nu^{-1}\delta[(b_1, b_3)^\top, t\rho_{1,3}]\right)^{-\frac{\nu}{2}}}{2\pi\rho_{1,3}^{-1}\sqrt{1-t^2\rho_{1,3}^2}} T_\nu\left[\frac{h(t\rho_{1,3}, t\rho_{1,2})}{\sqrt{1 + \nu^{-1}\delta[(b_1, b_3)^\top, t\rho_{1,3}]}}\right] dt,
\end{aligned} \tag{4.235}$$

where

$$T_\nu(\mathbf{b}|\mathbf{0}, R_3^*) = \begin{cases} T_\nu\left[(b_1, \min\{b_2, b_3\})|\mathbf{0}, R=0\right], & \text{if } s = +1, \\ \max\left\{0, T_\nu[(b_1, b_2)|\mathbf{0}, R=0] - T_\nu[(b_1, -b_3)|\mathbf{0}, R=0]\right\}, & \text{if } s = -1. \end{cases}$$

It should be noted that for computing the second term in RHS of (4.235) the use of transformation $r = \sin(\theta)$ may be useful.

IV. **Fourth method:** based on representation (4.195), proposed by [Owen, 1956] for the CDF of trivariate Gaussian distribution, it can be shown that the CDF of Student's t distribution with ν degrees of freedom is

$$T_\nu(\mathbf{b}|\mathbf{0}, R) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \int_{-\infty}^{b_1} \left[1 + \frac{y^2}{\nu}\right]^{-\frac{\nu+1}{2}} T_\nu(\mathbf{z}(y)|\mathbf{0}, R_4^* = \{\rho^*\}) dy, \quad (4.236)$$

where $\mathbf{b} = (b_1, b_2, b_3)^\top$, $R = \{\rho_{i,j}\}$ denotes the associated 3×3 correlation matrix, $\mathbf{z}(y) = [z_1(y), z_2(y)]^\top$ in which

$$z_i(y) = \frac{(\nu+1)^{\frac{1}{2}} \times (b_{i+1} - \rho_{i+1,1} \times y)}{\sqrt{(\nu+y^2)(1-\rho_{i+1,1}^2)}}, \quad i = 1, 2,$$

and

$$\rho^* = \frac{\rho_{2,3} - \rho_{1,2}\rho_{1,3}}{\prod_{i=1}^2 (1 - \rho_{i+1,1}^2)^{\frac{1}{2}}}.$$

Applying transformation $y = T_\nu^{-1}(u)$ on the RHS of (4.236) giving

$$T_\nu(\mathbf{b}|\mathbf{0}, R) = \int_0^{T_\nu(b_1)} T_\nu[\mathbf{z}(T_\nu^{-1}(t))|\mathbf{0}, R^* = \{\rho^*\}] dt, \quad (4.237)$$

where $T_\nu(\cdot)$ and $T_\nu^{-1}(\cdot)$ denote, accordingly, the CDF and quantile function of a standard Student's t distribution with ν degrees of freedom.

Genz [2004] shows by simulation that the third method in (4.235) outperforms, in almost all of cases, the other three methods both in terms of maximum (average) error and taken time for implementation [Genz, 2004, Tables 3-5, pp. 258-259]. For instance, when $\nu = 1$, using 6-point Gauss-Legendre and adaptive algorithm (with $\epsilon = 0.1$) the maximum (average) errors becomes $10^{-5}(10^{-7})$ and $10^{-13}(10^{-17})$, respectively. In what follows, the R functions TTV_Genz is given for implementing (4.235) when computing the CDF of 3-dimensional Student's t distribution.

```

1  R> TTV_Genz <- function(b, R, nu, k)
2  +{
3  + out <- quad_rule(k, type = "LE", alpha = 0, beta = 0)
4  + node <- out$node; w <- out$weight; eps <- 10e-8
5  + R[2, 3] <- ifelse( abs(R[2, 3]) < eps, eps, R[2, 3])
6  + s <- sign(R[2, 3])
7  + T1 <- BVT_Genz( c( b[1], min(b[-1]) ), eps, nu, k )
8  + T2 <- BVT_Genz( c( b[1], b[2] ), eps, nu, k )
9  + T3 <- BVT_Genz( c( b[1], -b[3] ), eps, nu, k )
10 + T_star <- ifelse(s < 0, max(0, T2 - T3 ), T1)
11 + L <- ( asin(R[2, 3]) - s*pi/2 )/2
12 + x <- (node + 1)*L + s*pi/2
13 + delta <- ( b[2]^2 + b[3]^2 - 2*sin(x)*b[2]*b[3] )/cos(x)^2
14 + p1 <- pt( b[1]/sqrt( 1 + delta/nu ), df = nu )
15 + CDF_23 <- T_star + L/(2*pi)*sum( w*( 1 + delta/nu )^(-nu/2)*p1 )
16 + x <- (node + 1)/2
17 + J <- c(2, 3)
18 + CDF <- 0
19 + for( i in 1:2 )
20 + {
21 + r <- x*R[J[i], 1]
22 + p2 <- sqrt( 1 - r^2 )
23 + delta <- ( b[1]^2 + b[J[i]]^2 - 2*r*b[1]*b[J[i]] )/p2^2
24 + p3 <- ( 1 + delta/nu )^(-nu/2)/p2

```

```

25 + p4 <- 1 - R[1, 2]^2*x^2 - R[1, 3]^2*x^2 - R[2, 3]^2 +
26 + 2*x^2*R[1, 2]*R[1, 3]*R[2, 3]
27 + p5 <- b[J[-i]]*(1 - r^2) - b[1]*x*(R[J[-i], 1] - R[J[i], 1]*R[2, 3]) -
28 + b[J[i]]*(R[3, 2] - x^2*R[3, 1]*R[2, 1])
29 + p6 <- p2*sqrt(p4)
30 + h <- p5/p6
31 + p5 <- pt(h/sqrt(1 + delta/nu), df = nu)
32 + CDF <- CDF + R[J[i], 1]/(4*pi)*sum(w*p3*p5)
33 + }
34 +return( CDF_23 + CDF )
35 +}

```

4.4.6 Rectangular Gaussian probability with positive correlation matrix

The rectangular Gaussian probability may be represented as

$$\Phi_p(\mathcal{R}^p | \mathbf{0}, R) = \int_{a_1}^{b_1} \int_{a_2}^{b_2} \cdots \int_{a_p}^{b_p} \phi_p(\mathbf{x} | \mathbf{0}, R) dx_1 \cdots dx_p, \quad (4.238)$$

where region $\mathcal{R}^p$ is a hypercube defined as (4.398). Let $R^+ = \{\rho_{i,j}^+\}$ denote the absolute value of elements of correlation matrix $R = \{\rho_{i,j}\}$, herein we are willing to show how the Gaussian probability in (4.238) can be expressed in terms of $\Phi_p(\mathcal{R}_*^p | \mathbf{0}, R^+)$ where

$$\mathcal{R}_*^p = \{\mathbf{x} \in \mathbb{R}^p | \mathbf{a}^* \leq \mathbf{x} \leq \mathbf{b}^*\}, \quad (4.239)$$

where $\mathbf{a}^* = (a_1^*, \dots, a_p^*)^\top$ and $\mathbf{b}^* = (b_1^*, \dots, b_p^*)^\top$. Suppose $s_{i,j} = \text{sign}(\rho_{i,j})$, $i, j = 1, \dots, p$, in which $\text{sign}(u)$ is the well-known sign function defined as $\text{sign}(u) = +1$ if $u \geq 0$ and $\text{sign}(u) = -1$ otherwise. Each normal distribution benefit from the well-known property of changeability in the sense that sign of each coordinate and corresponding elements of correlation matrix R can be interchanged. For example, taking $p = 2$, we can write

$$\phi_2(s_{1,2} \times x_1, x_2 | \mathbf{0}, \rho_{1,2}^+) = \phi_2(x_1, s_{1,2} \times x_2 | \mathbf{0}, \rho_{1,2}^+). \quad (4.240)$$

Evidently, for each set of three arbitrary random variables with second finite moment such as Y_1 , Y_2 , and Y_3 , if we have $\rho_{2,1} = \text{corr}(Y_2, Y_1) \leq 0$ and $\rho_{1,3} = \text{corr}(Y_1, Y_3) \leq 0$, then $\rho_{2,3} = \text{corr}(Y_2, Y_3) > 0$. In general, for $i, j = 1, 2, \dots, p$, we have $s_{i,j} = s_{i,1} \times s_{1,j}$ that implies $s_{i,j} < 0$ if and only if $s_{1,i} = s_{i,1} < 0$ or $s_{1,j} = s_{j,1} < 0$. Hence, if the j th element, $j = 1, \dots, p$, of vector $\mathbf{s}_1 = (s_{1,1}, \dots, s_{1,p})^\top$ is negative, then the j th coordinate of $\mathbf{x}$ within $\phi_p(\mathbf{x} | \mathbf{0}, R)$ is multiplied by $s_{1,j}$ while the corresponding column in correlation matrix, that is $R[:, j] = \boldsymbol{\rho}_j = \{\rho_{1,j}, \rho_{2,j}, \dots, \rho_{p,j}\}$, is replaced with its absolute value $R^+[:, j] = \boldsymbol{\rho}_j^+$. Taking into account the above fact, we start with a Lemma that is important for establishing the main result of this work.

Lemma 4.4.1. *Let $\phi_p(x_1, x_2, \dots, x_p | \mathbf{0}, R)$ be the PDF of a p -variate zero-mean Gaussian distribution with correlation matrix R and $s_{i,j} = \text{sign}(\rho_{i,j})$, $i, j = 1, 2, \dots, p$. Then*

$$\phi_p(x_1, x_2, \dots, x_p | \mathbf{0}, R) = \phi_p(s_{1,1} \times x_1, s_{1,2} \times x_2, \dots, s_{1,p} \times x_p | \mathbf{0}, R^+), \quad (4.241)$$

where R^+ is the absolute value of correlation matrix R .

Using Lemma 4.4.1 while applying a change of variable $x_i = s_{1,i} \times z_i$, $i = 1, 2, \dots, p$, on the RHS of (4.238), then Lemma 4.4.2 given below, shows how the CDF of two Gaussian distributions one with correlation matrix R and the other with R^+ are connected to each other.

Lemma 4.4.2. Let $\Phi_p(\mathcal{R}^p|\mathbf{0}, R)$ be the CDF of a p -variate zero-mean normal distribution with correlation matrix $R = \{\rho_{i,j}\}$ computed on hypercube defined in (4.240) and further assume $s_{i,j} = \text{sign}(\rho_{i,j})$, $i, j = 1, 2, \dots, p$. Then

$$\Phi_p(\mathcal{R}^p|\mathbf{0}, R) = \Phi_p(\mathcal{R}_*^p|\mathbf{0}, R^+), \quad (4.242)$$

where $\mathcal{R}_*^p$ is defined as (4.239) for which elements of $\mathbf{a}^*$ and $\mathbf{b}^*$ are as

$$\begin{cases} a_j^* = a_j & \text{and } b_j^* = b_j, & \text{if } s_{1,j} = +1, \\ a_j^* = -b_j & \text{and } b_j^* = -a_j, & \text{if } s_{1,j} = -1, \end{cases}$$

for $j = 1, 2, \dots, p$.

Example 4-16

Recall from Example 4-15, suppose $\mathbf{a} = (-1, -1, -\infty)^\top$ and $\mathbf{b} = (\infty, \infty, 2)^\top$. Based on Lemma 4.4.2, it is not hard to check that

$$\Phi_3(\mathcal{R}^3|\mathbf{0}, R) = \Phi_3(\mathcal{R}_*^3|\mathbf{0}, R^+) \approx 0.755968249358648,$$

where $\mathbf{a}^* = (-1, -1, -2)$ and $\mathbf{b}^* = (\infty, \infty, \infty)$. Not surprisingly, it can be seen easily that this probability is equal to $L_3(\mathbf{a}^*|R^+)$. ■

4.4.7 Computing the CDF of multivariate Gaussian distribution: General case

[Forbes and Wraith, 2014] show that the PDF of each p -dimensional Gaussian distribution can be represented as the product of p univariate Gaussian PDFs as

$$\phi_p(\mathbf{x}|\boldsymbol{\mu}, \Sigma) = \phi_p(\mathbf{x}|\boldsymbol{\mu}, VDV^\top) = \prod_{i=1}^p \phi(\mathbf{x}|[V^\top \mathbf{z}]_i, d_{i,i}) = \prod_{i=1}^p \phi([V^\top \mathbf{x}]_i|[V^\top \boldsymbol{\mu}]_i, d_{i,i}), \quad (4.243)$$

where $\mathbf{z} = \mathbf{x} - \boldsymbol{\mu}$, $[V^\top \mathbf{z}]_i$ denotes the i th element of vector $V^\top \mathbf{z}$, and $d_{i,i}$ is the i th diagonal entry of the diagonal matrix $D = (d_{i,j})$ whose main diagonal elements are eigen values of Σ . Let p -dimensional random vector $\mathbf{X}$ follows $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$. For computing $\mathcal{P}_G = P(\mathbf{a} \leq \mathbf{X} \leq \mathbf{b})$, based on spectral decomposition, we have

$$\begin{aligned} \mathcal{P}_G &= \int_{a_1}^{b_1} \cdots \int_{a_p}^{b_p} \prod_{i=1}^p \phi([V^\top \mathbf{x}]_i|[V^\top \boldsymbol{\mu}]_i, d_{i,i}) dx_1 \cdots dx_p \\ &= \int_{a_1 - \mu_1}^{b_1 - \mu_1} \cdots \int_{a_p - \mu_p}^{b_p - \mu_p} \prod_{i=1}^p \phi([V^\top \mathbf{x}]_i|0, d_{i,i}) dx_1 \cdots dx_p \\ &= \int_{-1}^1 \cdots \int_{-1}^1 \prod_{i=1}^p \left[\frac{b_i - a_i}{2} \right] \phi([V^\top \mathbf{y}]_i|0, d_{i,i}) dx_1 \cdots dx_p, \end{aligned} \quad (4.244)$$

where $\mathbf{y} = (\mathbf{x} + 1)(\mathbf{b} - \mathbf{a})/2 + \mathbf{a} - \boldsymbol{\mu}$. Using the Gauss-Legendre rule the quantity $\mathcal{P}_G$ can be approximated as

$$\mathcal{P}_G \approx \prod_{i=1}^p \left[\frac{b_i - a_i}{2\sqrt{2\pi d_{i,i}}} \right] \times \sum_{i_1=1}^{k_1} \cdots \sum_{i_p=1}^{k_p} \omega_{1i_1} \cdots \omega_{pi_p} \exp \left\{ -\frac{1}{2} \sum_{r=1}^p \frac{(\sum_{c=1}^p v_{r,c} \times z_{ci_c})^2}{d_{r,r}} \right\}, \quad (4.245)$$

where $z_{ci_c} = (x_{ci_c} + 1)(b_c - a_c)/2 + a_c - \mu_c$ in which x_{ci_c} is the i_c th node computed based on a k_c -point Gauss-Legendre rule for the c th coordinate and furthermore $v_{r,c}$ denotes the (r, c) th entry of matrix $V^\top$.

Example 4-17

Herein, we want to compute $\mathcal{P}_G$ given in Example 4-12 based on spectral decomposition discussed earlier. For $p = 2$ and $k_1 = k_2 = k$, it follows from (4.245) that

$$\begin{aligned}\mathcal{P}_G &\approx \prod_{i=1}^2 \left[\frac{b_i - a_i}{2\sqrt{2\pi d_{i,i}}} \right] \times \sum_{i=1}^k \sum_{j=1}^k \omega_{1i} \omega_{2j} \exp \left\{ -\frac{(v_{1,1}z_{1i} + v_{1,2}z_{2j})^2}{2d_{1,1}} - \frac{(v_{2,1}z_{1i} + v_{2,2}z_{2j})^2}{2d_{2,2}} \right\}, \\ &= \sum_{i=1}^k \sum_{j=1}^k \omega_{1i} \omega_{2j} g(z_{1i}, z_{2i}),\end{aligned}\tag{4.246}$$

where $z_{ij} = (x_{ij} + 1)(b_i - a_i)/2 + a_i - \mu_i$ in which x_{ij} (or ω_{ij}) for $i = 1, 2$ and $j = 1, \dots, k$ is the j th node (or weight) computed based on a k -point Gauss-Laguerre rule for the i th coordinate. Table 4.14 shows the performance of the k -point Gauss-Legendre rule for approximating $\mathcal{P}_G$ when $\boldsymbol{\mu} = (0, 0)^\top$ and covariance matrix $\Sigma = [(1, \rho)^\top, (\rho, 1)^\top]$. It is worth to note that both of the Cholesky and spectral decompositions show the same results in cases discussed in Table 4.14.

Table 4.14: ARE for approximating $\mathcal{P} = P(\mathbf{a} \leq \mathbf{X} \leq \mathbf{b})$ using k -point Gauss-Legendre rule based on Cholesky and spectral decompositions.

$\rho = 0.99$		ARE			
		Exact value ¹	$k = 10$	$k = 20$	$k = 40$
$a = (-\infty, -\infty)^\top$	$b = (2, 2)^\top$	0.97421138	1.245136942	0.20878929	5.673800e-04
$a = (2, 2)^\top$	$b = (\infty, \infty)^\top$	0.01971164	0.003664231	0.00198487	1.984895e-03
$a = (-2, -2)^\top$	$b = (2, 2)^\top$	0.94842275	0.520591497	0.01685183	7.154119e-08
$\rho = 0.50$			$k = 10$	$k = 20$	$k = 40$
$a = (-\infty, -\infty)^\top$	$b = (2, 2)^\top$	0.95855268	6.519226e-05	6.557075e-05	6.557075e-05
$a = (2, 2)^\top$	$b = (\infty, \infty)^\top$	0.00405294	8.493075e-03	8.493075e-03	8.493075e-03
$a = (-2, -2)^\top$	$b = (2, 2)^\top$	0.91711185	2.453014e-09	2.856401e-09	2.856405e-09
$\rho = 0.00$			$k = 10$	$k = 20$	$k = 40$
$a = (-\infty, -\infty)^\top$	$b = (2, 2)^\top$	0.95501730	6.438875e-05	6.481121e-05	6.481121e-05
$a = (2, 2)^\top$	$b = (\infty, \infty)^\top$	0.00051756	2.765946e-03	2.765946e-03	2.765946e-03
$a = (-2, -2)^\top$	$b = (2, 2)^\top$	0.91106974	6.285136e-09	6.829245e-09	6.829248e-09

¹ The exact value has been computed using Maple software up to eight decimal places.

Figure 4.6 displays the address of nodes and product of weights using the Gauss-Legendre rule with $k_1 = k_2 = 5$ points for approximating $\mathcal{P}_G$ when $\boldsymbol{\mu} = (0, 0)^\top$ and $\Sigma = [(1, \rho)^\top, (\rho, 1)^\top]$. More investigations reveal that a k -point Gauss-Legendre rule based on spectral decomposition shows superior performance than the Cholesky decomposition since in the latter case the integration region is rectangular. In general, for approximating a p -dimensional integral one needs to compute k^p nodes and weights that computationally is expensive when p becomes large. In such case, the Gaussian quadrature rules are not applicable. Alternatively, we suggest the use of R package `TruncatedNormal` that has been developed by [Botev, 2017] and is available at CRAN. Our proposed function `spec_quad` works well for small p ($p \leq 5$, say).

```
1 R> Spec_quad <- function(a, b, Mu, Sigma, k)
2 + {
3 + p <- length(Mu)
```

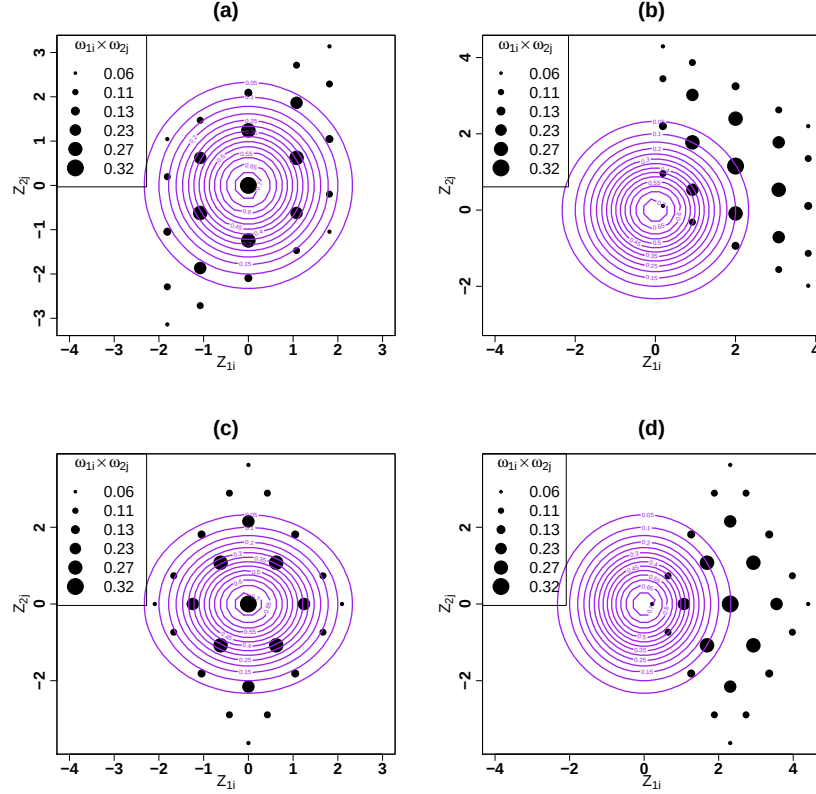


Figure 4.6: First row: contour plot of $g(z_{1i}, z_{2j})$ given in the RHS of (4.151). The filled black points show the address of nodes based on the Cholesky decomposition for (a): $\mathbf{a} = (-2, -2)^\top$, $\mathbf{b} = (2, 2)^\top$, and $\rho = -0.50$ and (b): $\mathbf{a} = (0, 0)^\top$, $\mathbf{b} = (\infty, \infty)^\top$, and $\rho = 0.50$. Second row: contour plot of $g(z_{1i}, z_{2j})$ given in the RHS of (4.246). The filled black points show the address of nodes based on the spectral decomposition for (c): $\mathbf{a} = (-2, -2)^\top$, $\mathbf{b} = (2, 2)^\top$, and $\rho = -0.50$ and (d): $\mathbf{a} = (0, 0)^\top$, $\mathbf{b} = (\infty, \infty)^\top$, and $\rho = 0.50$.

```

4 + n <- k^p
5 + threshold <- 5*sqrt( diag(Sigma) )
6 + a0 <- a - Mu
7 + b0 <- b - Mu
8 + a0 <- ifelse( a0 < -threshold, -threshold, a0 )
9 + b0 <- ifelse( b0 > threshold, threshold, b0 )
10 + e <- eigen(Sigma)
11 + ev <- e$vectors
12 + out <- quad_rule(k, type = "LE", alpha = 0, beta = 0)
13 + weight <- out$weight
14 + node <- out$node
15 + weight_matrix <- as.data.frame( matrix(rep(weight, p), nrow = k, ncol = p) )
16 + node_matrix <- as.data.frame( matrix(rep(node, p), nrow = k, ncol = p) )
17 + weight_grid <- as.matrix( do.call( expand.grid, weight_matrix ), nrow = n, ncol = p )
18 + omega <- sapply( 1:n, function(i) prod( weight_grid[i, ] ) )
19 + node_grid <- as.matrix( do.call( expand.grid, node_matrix ), nrow = n, ncol = p )
20 + zij <- node_grid %*% diag((b0 - a0)/2) + matrix( (b0 + a0)/2, n, p, byrow = T )
21 + p1 <- apply( ( t( zij %*% sweep(ev, 2, sqrt(e$values), "/" ) ) )^2, 2, sum )
22 + l1 <- 1/sqrt( (2*pi)^p * prod(e$values) ) * prod( (b0 - a0)/2 ) * sum( omega * exp( -p1/2 ) )
23 + min( abs(l1), 1 )
24 + }

```



4.5 The EM algorithm

In practice, often we are working with a family of distributions for which computing the ML estimator could be difficult due to the complicate structure of the likelihood or log-likelihood function. The notable among them are finding the ML estimator for the models fitted to grouped data, censored data, multivariate data, univariate multi-modal data (mixture models), and multivariate multi-modal data (model-based clustering), etc. For a comprehensive survey of applications of E algorithm, we refer reader to McLachlan and Krishnan [1997]. Fortunately, for aforementioned applications, it is possible to formulate the associated problem (known as “incomplete-data” or observed data) in a new form as “complete-data” or “augmented-data” problem when the observed data are accompanied by a set of new variables. Within a framework referred to as the expectation-maximization (EM) algorithm introduced by Dempster et al. [1977], one can find the ML estimator of the statistical model provided that the model could be formulate as the complete-data problem. To this end, in what follows, we give some necessary preliminaries.

Definition 4.5.1. Let $f(\cdot)$ denote a real-valued function on $\mathbb{R}$. Then $f(\cdot)$ is a convex function on $\mathbb{R}$ if

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y), \quad 0 \leq \alpha \leq 1; x, y \in \mathbb{R}. \quad (4.247)$$

Definition 4.5.2. For a given random variable X , suppose $E(X) < \infty$ and $E[f(\cdot)] < \infty$ in which $f(\cdot)$ is a convex function. Then

$$f[E(X)] \leq E[f(X)]. \quad (4.248)$$

Remark 4.5.1. If $f(\cdot)$ is a concave function, then

$$f[E(X)] \geq E[f(X)]. \quad (4.249)$$

It is worth to note that if $f(\cdot)$ is concave then the inequality (4.248) is reversed. Let

$$\ell(\Psi|\mathbf{y}) = \sum_{i=1}^n \log f(y_i|\Psi), \quad (4.250)$$

to be the log-likelihood function, based on n observed data of family of distributions with PDF $f(y_i|\Psi)$ in which $\Psi \in \mathbb{R}^p$, for $p \geq 1$, is the vector of unknown parameters. Sometimes $\mathbf{y}$ is called *incomplete-data* and therefore $\ell(\Psi|\mathbf{y})$ in (4.250) becomes the log-likelihood function of incomplete-data. Within the EM algorithm framework, for each observed value y_i , we need to evaluate the joint PDF of y and some random variable (vector), denoted by z_i (for $i = 1, \dots, n$). The random variate z_i is referred to as *latent*, *missing*, or *unobservable* variable⁶. Each of latter three terms may be used for z_i depending the nature of problem under study. Herein, for the sake of simplicity, we assume that z_i is univariate, but in general, it can be multivariate. The joint PDF of pair (y_i, z_i) must have close form, even proportional to a constant, and mathematically tractable for the sake of computation. It is worth to note that union of observed data $\mathbf{y}^\top = (y_1, \dots, y_n)$ and latent variable $\mathbf{z}^\top = (z_1, \dots, z_n)$, represented by $\mathbf{x} = \{\mathbf{y}, \mathbf{z}\}$, is known as the *complete-data*. Furthermore, $L_c(\Psi|\mathbf{y}, \mathbf{z}) = L_c(\Psi|\mathbf{x})$ and $\ell_c(\Psi|\mathbf{y}, \mathbf{z}) = \ell_c(\Psi|\mathbf{x})$ are the complete-data likelihood and log-likelihood functions, respectively. It is worth to mention that subscript letter “c” for the likelihood or log-likelihood function denotes the “complete” data.

⁶Herein, we assume that the latent variable z is continuous random variate. However, it can be either continuous or discrete.

4.5.1 How does the EM algorithm work?

Simply we can write

$$\begin{aligned}
\ell(\Psi|\mathbf{y}) &= \log \int_{\mathbb{R}^n} f(\mathbf{y}, \mathbf{z}|\Psi) d\mathbf{z} \\
&= \log \int_{\mathbb{R}^n} \frac{f(\mathbf{y}, \mathbf{z}|\Psi)}{f(\mathbf{z}|\mathbf{y}, \Psi^{\text{old}})} f(\mathbf{z}|\mathbf{y}, \Psi^{\text{old}}) d\mathbf{z} \\
&= \log E \left[\frac{f(\mathbf{y}, \mathbf{z}|\Psi)}{f(\mathbf{z}|\mathbf{y}, \Psi^{\text{old}})} \middle| \mathbf{y}, \Psi^{\text{old}} \right] \\
&\geq E \left[\log \frac{f(\mathbf{y}, \mathbf{z}|\Psi)}{f(\mathbf{z}|\mathbf{y}, \Psi^{\text{old}})} \middle| \mathbf{y}, \Psi^{\text{old}} \right] \tag{4.251}
\end{aligned}$$

$$\begin{aligned}
&= E[\log f(\mathbf{y}, \mathbf{z}|\Psi) | \mathbf{y}, \Psi^{\text{old}}] - E[\log f(\mathbf{z}|\mathbf{y}, \Psi^{\text{old}}) | \mathbf{y}, \Psi^{\text{old}}] \\
&= Q(\Psi|\Psi^{\text{old}}) - E[\log f(\mathbf{z}|\mathbf{y}, \Psi^{\text{old}}) | \mathbf{y}, \Psi^{\text{old}}], \tag{4.252}
\end{aligned}$$

where to attain (4.251) we apply Remark 4.5.1 since $\log(\cdot)$ is a concave mapping. It follows that

$$\ell(\Psi|\mathbf{y}) \geq Q(\Psi|\Psi^{\text{old}}) - E[\log f(\mathbf{z}|\mathbf{y}, \Psi^{\text{old}}) | \mathbf{y}, \Psi^{\text{old}}]. \tag{4.253}$$

Evidently, only the first term in RHS of (4.253) depends on unknown parameter Ψ . As we know, the ML approach find $\hat{\Psi}$ by maximizing incomplete-data log-likelihood (4.250), but the EM algorithm finds $\hat{\Psi}$ by following Algorithm 14 as follows.

Algorithm 14 Concise description of the EM algorithm

- 1: Set $k = 0$, read small constant $\epsilon > 0$, an appropriate norm $\|\cdot\|$, vector of observed data $\mathbf{y}^\top = (y_1, \dots, y_n)$, and starting value for Ψ denoted as $\Psi^{(k)}$;
- 2: **while** $\|\Psi^{(k+1)} - \Psi^{(k)}\| > \epsilon$ **do**
- 3: **E-step:** At the $(k+1)$ th iteration of the EM algorithm, having $\Psi^{(k)}$, the E-step is complete by computing $Q(\Psi|\Psi^{(k)})$ as

$$Q(\Psi|\Psi^{(k)}) = E[\ell(\Psi|\mathbf{y}, \mathbf{z}) | \mathbf{y}, \Psi^{(k)}].$$

- 4: **M-step:** At the $(k+1)$ th iteration of the EM algorithm, having $\Psi^{(k)}$, the M-step is complete by finding $\Psi^{(k+1)}$ such that

$$\Psi^{(k+1)} = \arg \max_{\Psi} Q(\Psi|\Psi^{(k)}) \tag{4.254}$$

- 5: **end**
 - 6: Accept $\Psi^{(k+1)}$ as the ML estimator of Ψ .
 - 7: **end**
-

Algorithm 14 iterates alternatively between E-step and M-step until convergence occurs in sequence $\{\Psi^{(k)}\}_{k \geq 1}$.

4.5.2 Convergence of the EM Algorithm

It can be argued [McLachlan et al., 2004] that each iteration of the EM algorithm yields $\Psi^{(k+1)}$ such that:

$$L(\Psi^{(k+1)}|\mathbf{y}) \geq L(\Psi^{(k)}|\mathbf{y}), \quad k = 0, 1, 2, \dots, \tag{4.255}$$

meaning that sequence $L(\Psi^{(k)}|\mathbf{y})$ converges monotonically to some stationary point, L^* say, provided that $L(\Psi|\mathbf{y})$ is bounded. But, there is no guarantee that the EM algorithm converges

to the ML estimator $\hat{\Psi}$, that yields the *global maximum* for $L(\Psi|\mathbf{y})$. The EM algorithm may converge to either *local maximum*, *local minimum*, or a *saddle point* of $L(\Psi|\mathbf{y})$, see [McLachlan and Krishnan, 1997, Section 3.6] for known examples of such cases. If $L(\Psi|\mathbf{y})$ has different stationary points, then convergence of the EM algorithm to the local maximum, global maximum, or a saddle point depends on the starting value $\Psi^{(0)}$. If $L(\Psi|\mathbf{y})$ is unimodal and satisfies a certain mild differentiability condition, then the sequence $\{\Psi^{(k)}\}$ converges to the ML estimator, regardless of the starting value [Wu, 1983]. In what follows, Figure 4.7 shows the log-likelihood function of the Student's t distribution for $\Psi^\top = (\mu, \sigma, \nu) = (0, 0.1, 1)$ when μ is assumed to be the only unknown parameter of the model. As it is seen, the EM algorithm finds the true ML estimator $\hat{\mu}$, when two starting values $\mu^{(0)} = \{-4, 1\}$ and two sample sizes $n = \{10, 40\}$ are used. As it is seen the EM algorithm, regardless of the $\mu^{(0)}$, attains convergence after $k = 20$ th iteration.

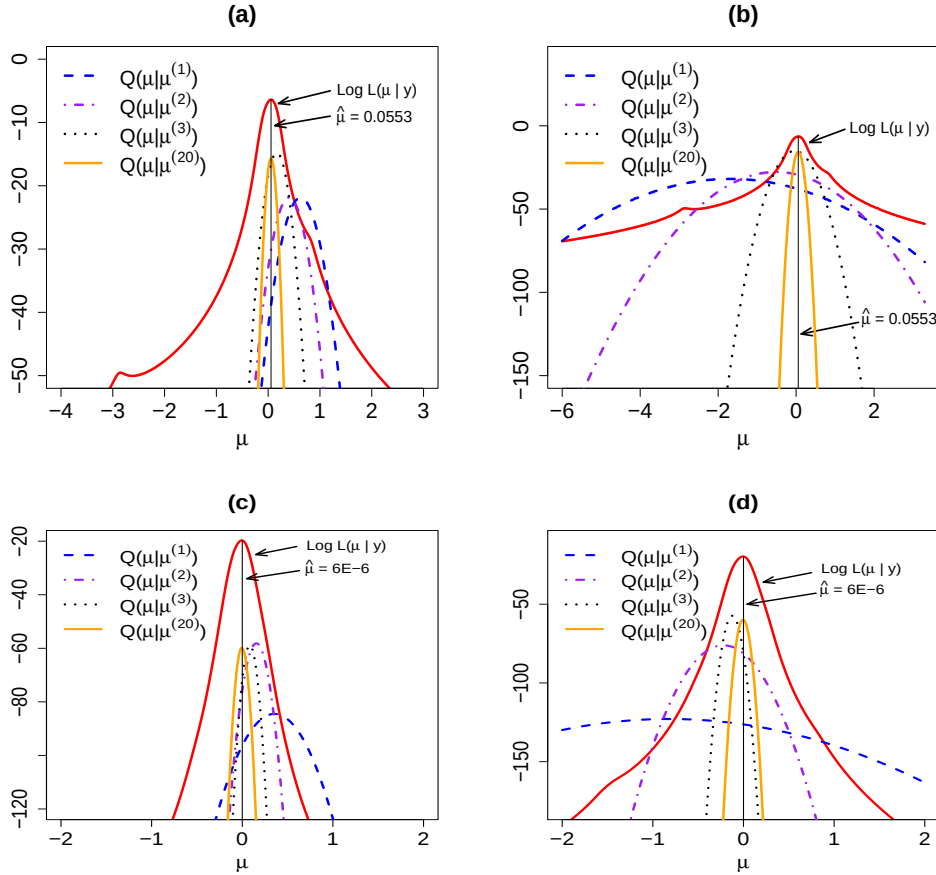


Figure 4.7: The red-colored solid curve is $\ell(\mu|\mathbf{y}) = \log L(\mu|\mathbf{y})$. Superimposed is $Q(\mu|\mu^{(k)})$ at iterations $k = \{1, 2, 3, 20\}$ for four combinations of sample size and starting value as (a): $n = 10, \mu^{(0)} = -4$, (b): $n = 10, \mu^{(0)} = 1$, (c): $n = 40, \mu^{(0)} = -4$, and (d): $n = 40, \mu^{(0)} = 1$.

4.5.3 Extensions of the EM algorithm

The EM algorithm described earlier is known as the ordinary EM algorithm. In practice, however, there are situations for which implementing the ordinary EM algorithm is not feasible or tedious computationally. Notable among them are alternating ECM (AECM) [Meng and Van Dyk, 1997], ECM algorithm [Meng and Rubin, 1993], ECME algorithm Liu and Rubin [1994], Monte Carlo EM (MCEM) algorithm [Wei and Tanner, 1990], parameter expanded EM (PX-EM) algorithm [Liu et al., 1998], space-alternating generalized EM (SAGE) algorithm [Fessler and Hero, 1994], stochastic EM algorithm (Celeux [1986]), and supplementary EM algorithm (Meng and Rubin [1991]). Next, we are willing to describe MCEM, ECM, and ECME

algorithms. For a comprehensive survey on other extensions mentioned above, we refer reader to McLachlan and Krishnan [1997], McLachlan et al. [2004].

Monte Carlo EM algorithm

At $(k + 1)$ iteration of the EM algorithm, any function of latent variable $g(z_i)$ say, is replaced by its posterior expectation given by McLachlan et al. [2004]:

$$E_i^{(k+1)} = E(g(z_i)|y_i, \Psi^{(k)}), \quad i = 1, \dots, n \quad (4.256)$$

provided that RHS of (4.256) is finite. The law of iterated expectation guarantees that LHS of (4.256) is finite, on the average, if we have $E[g(z_i)] < \infty$. In some applications evaluating RHS of (4.256) is not possible either theoretically or computationally. In such cases, we may use the Monte Carlo EM (MCEM) algorithm in order to compute the E-step of the EM algorithm as follows [Wei and Tanner, 1990].

1. At $(k + 1)$ th iteration of the MCEM algorithm, generate B realizations $\{z_{i,1}^{(k)} \dots, z_{i,B}^{(k)}\}$ from posterior distribution with PDF $f(z_i|y_i, \Psi^{(k)})$, for $i = 1, \dots, n$.
2. Approximate $Q(\Psi|\Psi^{(k)})$ as

$$\begin{aligned} \tilde{Q}(\Psi|\Psi^{(k)}) &\approx Q_B(\Psi|\Psi^{(k)}) = \frac{1}{B} \sum_{b=1}^B E \left[\sum_{i=1}^n \log f(z_i|y_i, \Psi^{(k)}) \right] \\ &= \frac{1}{B} \sum_{b=1}^B E [\log f(\mathbf{z}|\mathbf{y}, \Psi^{(k)})]. \end{aligned} \quad (4.257)$$

The M-step if follows as usual by maximizing $\tilde{Q}_B(\Psi|\Psi^{(k)})$ with respect to Ψ .

One issue that arises with the MCEM algorithm is how to assess the convergence. Since the E-step within the MCEM algorithm is subject to random fluctuations, hence the monotonicity property no longer holds. But, in certain cases, the algorithm gives results that are most likely close to the true ML estimator [Booth and Hobert, 1999]. As recommended by Wei and Tanner [1990], the constant B is small for the initial iterations and then as algorithm gets closer to true ML estimator, then B becomes larger. The diagnostics for convergence, that is assessing stability in $\Psi^{(k)}$, is carried out by plotting $\Psi^{(k)}$ versus k .

Example 4-18

Let random variable Y follow the generalized exponential, denoted by $\mathcal{GE}(\alpha, \lambda)$, with CDF and PDF given, respectively by

$$F_Y(y|\Psi) = (1 - e^{-\lambda y})^\alpha, \quad (4.258)$$

$$f_Y(y|\Psi) = \alpha(1 - e^{-\lambda y})^{\alpha-1} e^{-\lambda y}, \quad (4.259)$$

where $\Psi^\top = (\alpha, \lambda)$ in which α is the shape parameter and λ is the rate parameter [DK, 1993]. Suppose n subjects are placed on a life testing simultaneously at time $t_0 = 0$ and under inspection at m pre-determined times $t_1 < t_2 < \dots < t_m$ in which t_m is the time to terminate the life testing. At the i -th inspection time t_i , the x_i numbers of failures within $(t_i, t_{i+1}]$ are recorded and r_i alive items are randomly removed from the life testing, for $i = 1, \dots, m$. Such a life testing is referred to as progressively type-I interval censoring (PTIC) plan [Chen and Lio, 2010]. The number s_i , of surviving items is a random variable and the exact number of items withdrawn subjects can never be greater than s_i at time t_i . Equivalently, r_i can be represented as $r_i = \lfloor p_i s_i \rfloor$ in which $p_1, \dots, p_{m-1}$ and $p_m = 1$ are a set of pre-specified percentage for withdrawing remaining surviving units at time t_i , for $i = 1, \dots, m$. We write

$$\{u_i, r_i, t_i\}_{i=1}^m, \quad n = \sum_{i=1}^m u_i + r_i, \quad (4.260)$$

to show a PTIC plan. If $r_i = 0$, for $i = 1, \dots, m-1$, then the PTIC plan turns into a type-I interval censoring scheme with sample $u_1, u_2, \dots, u_m, u_{m+1} = r_m$ [Aggarwala*, 2001]. Suppose n subjects that independently follow a distribution with PDF $f_Y(\cdot|\Psi)$ take part in plan (4.260). It can be shown that the (observed) likelihood function is [Chen and Lio, 2010]:

$$L(\Psi) \propto \prod_{i=1}^m [F_Y(t_i|\Psi) - F_Y(t_{i-1}|\Psi)]^{x_i} [1 - F_Y(t_i|\Psi)]^{r_i}. \quad (4.261)$$

For $i = 1, \dots, m$ and $j = 1, \dots, x_i$, let t_{ij} denote the failure time of subject within subinterval $(t_{i-1}, t_i]$. Further, for $i = 1, \dots, m$ and $j = 1, \dots, r_i$, suppose t_{ij}^* the lifetime of alive item that is randomly removed at the endpoint of subinterval $(t_{i-1}, t_i]$. As to the complete-data log-likelihood function, we can see [Chen and Lio, 2010]:

$$\ell_c(\Psi) \propto \sum_{i=1}^m \sum_{j=1}^{x_i} \log f_Y(t_{ij}|\Psi) + \sum_{i=1}^m \sum_{j=1}^{r_i} \log f_Y(t_{ij}^*|\Psi). \quad (4.262)$$

In the EM algorithm framework, the vector of observed data $\mathbf{y}$, consists of number of recorded failure times x_i s, and number of removed alive items r_i , during subinterval $(t_{i-1}, t_i]$, for $i = 1, \dots, m$. As to observed data, missing data, and unobservable, respectively, we can represent as

$$\begin{aligned} \underline{\mathbf{y}} &= (\mathbf{y}_1^\top, \dots, \mathbf{y}_m^\top) = \{(x_1, r_1), \dots, (x_m, r_m)\}, \\ \underline{\mathbf{z}} &= (\mathbf{z}_1^\top, \dots, \mathbf{z}_m^\top), \\ \underline{\mathbf{z}}^* &= (\mathbf{z}_1^{*\top}, \dots, \mathbf{z}_m^{*\top}), \end{aligned} \quad (4.263)$$

where $\mathbf{z}_i^\top = (t_{i1}, \dots, t_{ix_i})$ consists of missing independent and identically failure times during subinterval $(t_{i-1}, t_i]$ and $\mathbf{z}_i^{*\top} = (t_{i1}^*, \dots, t_{ir_i}^*)$ consists of unobservable survival times withdrawn during subinterval $(t_{i-1}, t_i]$, for $i = 1, \dots, m$. Therefore the complete-data is $\mathbf{x} = \{\underline{\mathbf{y}}, \underline{\mathbf{z}}, \underline{\mathbf{z}}^*\}$. At $(k+1)$ th iteration, two steps of the EM algorithm are given as follows.

1. E-step:

$$\begin{aligned} Q(\Psi|\Psi^{(k)}) &= C + \sum_{i=1}^m \sum_{j=1}^{x_i} E[\log f_Y(t_{ij}|\Psi) | \mathbf{y}_i, \Psi^{(k)}] + \sum_{i=1}^m \sum_{j=1}^{r_i} E[\log f_Y(t_{ij}^*|\Psi) | \mathbf{y}_i, \Psi^{(k)}] \\ &= [\log(\alpha) + \log(\lambda)] \sum_{i=1}^m (x_i + r_i) - \lambda \sum_{i=1}^m \sum_{j=1}^{x_i} E(t_{ij} | \mathbf{y}_i, \Psi^{(k)}) \\ &\quad - \lambda \sum_{i=1}^m \sum_{j=1}^{r_i} E(t_{ij}^* | \mathbf{y}_i, \Psi^{(k)}) + (\alpha - 1) \sum_{i=1}^m \sum_{j=1}^{x_i} E\{\log[1 - \exp\{-\lambda t_{ij}\}] | \mathbf{y}_i, \Psi^{(k)}\} \\ &\quad + (\alpha - 1) \sum_{i=1}^m \sum_{j=1}^{r_i} E\{\log[1 - \exp\{-\lambda t_{ij}^*\}] | \mathbf{y}_i, \Psi^{(k)}\}, \end{aligned} \quad (4.264)$$

where C is a constant independent of Ψ . It is worth to note that the unobserved lifetimes of the r_i alive items removed during subinterval $(t_{i-1}, t_i]$ are conditionally independent and follow a common truncated $\mathcal{GE}(\alpha, \lambda)$ distribution on $[t_i, \infty)$. Furthermore, missing lifetimes of the x_i subjects during subinterval $(t_{i-1}, t_i]$ are conditionally independent and follow the a common double-truncated $\mathcal{GE}(\alpha, \lambda)$ distribution on $(t_{i-1}, t_i]$, for $i = 1, \dots, m$. Therefore, the required posterior expectations to complete the E-step at $(k+1)$ th iteration are:

$$E_{i1}^{(k)} = E(t_{ij} | \mathbf{y}_i, \Psi^{(k)}) = \frac{\int_{t_{i-1}}^{t_i} u f(u | \Psi^{(k)}) du}{F(t_i | \Psi^{(k)}) - F(t_{i-1} | \Psi^{(k)})}, \quad (4.265)$$

$$E_{i2}^{(k)} = E[\log(1 - \exp\{-\lambda t_{ij}\}) | \mathbf{y}_i, \Psi^{(k)}] = \frac{\int_{t_{i-1}}^{t_i} \log(1 - \exp\{-\lambda u\}) f(u | \Psi^{(k)}) du}{F(t_i | \Psi^{(k)}) - F(t_{i-1} | \Psi^{(k)})}, \quad (4.266)$$

$$E_{i3}^{(k)} = E(t_{ij}^* | \mathbf{y}_i, \Psi^{(k)}) = \frac{\int_{t_i}^{\infty} u f(u | \Psi^{(k)}) du}{1 - F(t_i | \Psi^{(k)})}, \quad (4.267)$$

$$E_{i4}^{(k)} = E[\log(1 - \exp\{-\lambda t_{ij}^*\}) | \mathbf{y}_i, \Psi^{(k)}] = \frac{\int_{t_i}^{\infty} \log(1 - \exp\{-\lambda u\}) f(u | \Psi^{(k)}) du}{1 - F(t_i | \Psi^{(k)})}, \quad (4.268)$$

where $i = 1, \dots, m$ and $t_0 = 0$.

Though posterior expectations (4.265)-(4.268) can be computed numerically, but we are willing to hire the MCEM when these quantities are approximated as

$$\tilde{E}_{i1}^{(k)} = \frac{1}{B} \sum_{b=1}^B u_{i,b}, \quad (4.269)$$

$$\tilde{E}_{i2}^{(k)} = \frac{1}{B} \sum_{b=1}^B \log(1 - \exp\{-\lambda^{(k)} u_{i,b}\}), \quad (4.270)$$

$$\tilde{E}_{i3}^{(k)} = \frac{1}{B} \sum_{b=1}^B u_{i,b}^*, \quad (4.271)$$

$$\tilde{E}_{i4}^{(k)} = \frac{1}{B} \sum_{b=1}^B \log(1 - \exp\{-\lambda^{(k)} u_{i,b}^*\}), \quad (4.272)$$

where $u_{i,1}, \dots, u_{i,B}$ come independently from $\mathcal{GE}(\alpha^{(k)}, \lambda^{(k)})$ truncated on $(t_{i-1}, t_i]$ and $u_{i,1}^*, \dots, u_{i,B}^*$ follow independently $\mathcal{GE}(\alpha^{(k)}, \lambda^{(k)})$ truncated on $[t_i, \infty)$, for $i = 1, \dots, m$.

2. **M-step:** Substituting evaluated posterior expectations (4.269)-(4.272) into RHS of (4.264) and then differentiating with respect to α and λ yields

$$\begin{aligned} \frac{\partial \tilde{Q}(\Psi | \Psi^{(k)})}{\partial \alpha} &= \frac{\sum_{i=1}^m (x_i + r_i)}{\alpha} + \sum_{i=1}^m x_i \tilde{E}_{i2}^{(k)} + \sum_{i=1}^m r_i \tilde{E}_{i4}^{(k)}, \\ \frac{\partial \tilde{Q}(\Psi | \Psi^{(k)})}{\partial \lambda} &= \frac{\sum_{i=1}^m (x_i + r_i)}{\lambda} - \sum_{i=1}^m x_i \tilde{E}_{i1}^{(k)} - \sum_{i=1}^m r_i \tilde{E}_{i3}^{(k)}. \end{aligned}$$

It follows that

$$\tilde{\alpha}^{(k+1)} = - \frac{n}{\sum_{i=1}^m x_i \tilde{E}_{i2}^{(k)} + \sum_{i=1}^m r_i \tilde{E}_{i4}^{(k)}}, \quad (4.273)$$

and

$$\tilde{\lambda}^{(k+1)} = \frac{n}{\sum_{i=1}^m x_i \tilde{E}_{i1}^{(k)} + \sum_{i=1}^m r_i \tilde{E}_{i3}^{(k)}}. \quad (4.274)$$

The M-step is complete.

We use the EM and MCEM algorithms for estimating parameters of $\mathcal{GE}$ fitted to 112 patients with plasma cell myeloma treated at the National Cancer Institute [Carbone et al., 1967] that supposed to be under a PTIC plan. To this end, for $m = 9$ pre-specified inspection times including: $t_1 = 5.5$, $t_2 = 10.5$, $t_3 = 15.5$, $t_4 = 20.5$, $t_5 = 25.5$, $t_6 = 30.5$, $t_7 = 40.5$, $t_8 = 50.5$, and termination time $t_9 = 60.5$, the set of missing lifetimes and number dropped (alive) patients are $x = (18, 16, 18, 10, 11, 8, 13, 4, 1)$, and $r = (1, 1, 3, 0, 0, 1, 2, 3, 2)$, respective. The R code for implementing the EM algorithm thereof is given below.

```

1  R> rtrunc_ge <- function(n, alpha, lambda, a, b)
2  +{
3  + F_U <- (1 - exp(-b*lambda))^alpha
4  + F_L <- (1 - exp(-a*lambda))^alpha
5  + u <- runif(n)*(F_U - F_L) + F_L
6  + out <- -log( 1 - u^(1/alpha) )/lambda
7  +return(out)
8  +}
9  R> pge <- function(x, a, b, log.p = FALSE, lower.tail = TRUE)
10 +{
11 + n <- length(x)
12 + CDF <- rep(NA, n)
13 + let <- !( is.na(x) | abs(x) == Inf )
14 + y <- x[let]
15 + CDF <- pexp(y, rate = b)^a
16 + if(log.p == TRUE & lower.tail == FALSE) CDF <- log(1 - CDF)
17 + if(log.p == TRUE & lower.tail == TRUE ) CDF <- log(CDF)
18 + if(log.p == FALSE & lower.tail == FALSE) CDF <- 1 - CDF
19 + return(CDF)
20 +}
21 R> dge <- function(x, a, b, log = FALSE)
22 +{
23 + n <- length(x)
24 + PDF <- rep(NA, n)
25 + let <- !( is.na(x) | abs(x) == Inf )
26 + y <- x[let]
27 + PDF <- a*pexp(x, rate = b)^(a - 1)*dexp(x, rate = b)
28 + if(log == TRUE) PDF <- log( PDF )
29 + return(PDF)
30 +}
31 R> x <- c(18, 16, 18, 10, 11, 8, 13, 4, 1);
32 R> r <- c(1 , 1, 3, 0, 0, 1, 2, 3, 2)
33 R> t <- c(5.5,10.5,15.5,20.5,25.5,30.5,40.5,50.5,60.5)
34 R> t <- c(10e-8, t); m <- 9; n <- 112
35 R> ##### implementing EM algorithm #####
36 R> Ei1 <- Ei2 <- Ei3 <- Ei4 <- rep(0, m)
37 R> K <- 100; Psi_iter <- matrix(NA, nrow = K, ncol = 2)
38 R> k <- 1; dif <- 1; tol <- 10e-8;
39 R> Psi_iter[k, ] <- c(1, 0.1);
40 R> f1 <- function(u, par)u*dge(u, par[1], par[2])
41 R> f2 <- function(u, par)pexp(u, rate = par[2], log.p = T)*dge(u, par[1], par[2])
42 R> v2 <- Vectorize(function(u, par) pge(u, par[1], par[2], lower.tail = FALSE), "u")
43 R> v1 <- Vectorize(function(u, par) pge(u, par[1], par[2]), "u")
44 R> while(dif > tol)
45 +{
46 + d2 <- v2(t[-1], Psi_iter[k, ])
47 + d1 <- diff( v1(t, Psi_iter[k, ]) )
48 + #### E-step of the EM algorithm
49 + for(i in 2:(m+1))
50 + {
51 + Ei1[i-1]<-integrate(f1,lower=t[i-1],upper=t[i],par=Psi_iter[k,])$value/d1[i-1]
52 + Ei2[i-1]<-integrate(f2,lower=t[i-1],upper=t[i],par=Psi_iter[k,])$value/d1[i-1]
53 + Ei3[i-1]<-integrate(f1,lower=t[i ],upper=Inf ,par=Psi_iter[k,])$value/d2[i-1]

```

```

54 + Ei4[i-1]<-integrate(f2,lower=t[i ],upper=Inf ,par=Psi_iter[k,])$value/d2[i-1]
55 + }
56 + ##### M-step of the EM algorithm
57 + Psi_iter[k + 1, ] <- c( -n/sum(x*Ei2 + r*Ei4), n/sum(x*Ei1 + r*Ei3) )
58 + dif <- sum( abs( Psi_iter[k + 1, ] - Psi_iter[k, ] ) )
59 + cat( "The EM output at iteration", k,"is:", Psi_iter[k, ], "\n" )
60 + k <- k + 1
61 +}
62 R> ##### implementing MCEM algorithm #####
63 R> set.seed(20260710);
64 R> tilde_Ei1 <- tilde_Ei2 <- tilde_Ei3 <- tilde_Ei4 <- rep(0, m)
65 R> K <- 30; tilde_Psi_iter <- matrix(NA,nrow=K+1,ncol=2);# K=30 is number of MCEM runs
66 R> B <- 100; tilde_Psi_iter[1, ] <- c(1, 0.1);
67 R> for(k in 1:K)
68 + {
69 + ##### E-step of the MCEM algorithm
70 + for (i in 2:(m+1))
71 + {
72 + u_i <- rtrunc_ge(B, tilde_Psi_iter[k, 1], tilde_Psi_iter[k, 2], t[i-1], t[i])
73 + u_i_star <- rtrunc_ge(B, tilde_Psi_iter[k, 1], tilde_Psi_iter[k, 2], t[i], Inf)
74 + tilde_Ei1[i-1] <- mean(u_i)
75 + tilde_Ei2[i-1] <- mean(pexp(u_i, rate = tilde_Psi_iter[k, 2], log.p = T))
76 + tilde_Ei3[i-1] <- mean(u_i_star)
77 + tilde_Ei4[i-1] <- mean(pexp(u_i_star, rate = tilde_Psi_iter[k, 2], log.p = T))
78 + }
79 + ##### M-step of the MCEM algorithm
80 + tilde_Psi_iter[k + 1, ] <- c( -n/sum(x*tilde_Ei2 + r*tilde_Ei4),
81 + n/sum(x*tilde_Ei1 + r*tilde_Ei3) )
82 + cat( "The EM output at iteration", k,"is:", tilde_Psi_iter[k, ], "\n" )
83 + }

```

Using starting value $\hat{\Psi}^{(0)} = (1, 0.1)^\top$, the EM algorithm converges to $\hat{\Psi}_{EM}^\top = (1.1532, 0.0448)$ after $k = 12$ th iteration, but not to the ML estimator $\hat{\Psi}_{ML}^\top = (1.4328, 0.0571)$. As it is seen, the MCEM algorithm moves smoothly after $k = 3$ th iteration. We mention that the MCEM algorithm has been terminated at $k = 30$ th iteration and number $B = 100$, of generations within the E-step has been fixed across iterations. As to the ML estimator, the R code is given as follows.

```

1 R> log_like <- function(par)
2 + {
3 + loglike <- rep(0, m); a = par[1]; b = par[2]
4 + for(i in 1:m)
5 + {
6 + loglike[i] <- x[i]*log( pge(t[i+1], a, b) - pge(t[i], a, b) ) +
7 + r[i]*pge(t[i+1], a, b, log.p = TRUE, lower.tail = FALSE)
8 + }
9 + -sum(loglike)
10 + }
11 R> optim(par=c(1, 0.1), fn=log_like, method="L-BFGS-B", lower=10e-5, upper=Inf)$par

```



Example 4-19

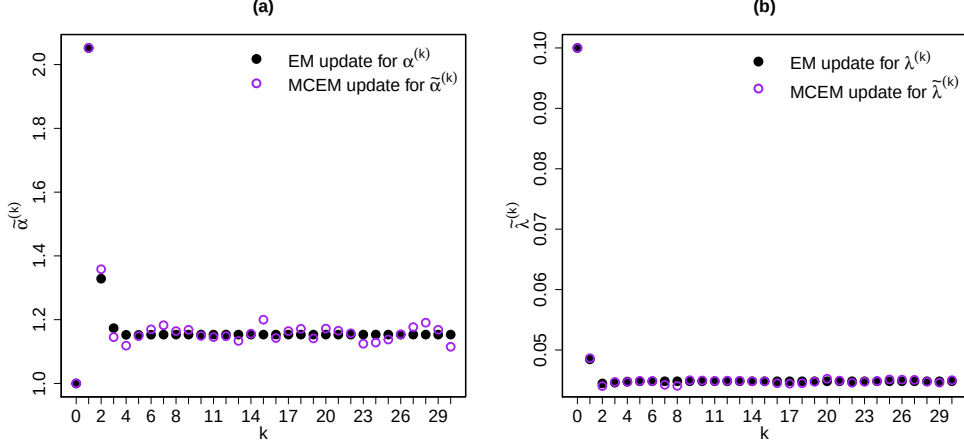


Figure 4.8: The output of EM and MCEM algorithms for estimating (a): α and (b): λ , when $\mathcal{GE}$ distribution is fitted to 112 patients with plasma cell myeloma.

Recall that the GSM model (4.7) follows a S α S distribution when $\eta(Z) = 2^{-1}Z^{-1}$ with $Z \sim S(\alpha/2, 1, [\cos(\pi\alpha/4)]^{2/\alpha}, 0)$ as shown by Table 4.1. Let $\mathbf{y}^\top = (y_1, y_2, \dots, y_n)$ denotes a sample of n independent observations from S α S distribution or equivalently $y_i \stackrel{\text{i.i.d.}}{\sim} S(\alpha, 0, \gamma, \delta)$ where $\Psi^\top = (\gamma, \delta)$. Herein, we assume that the tail thickness parameter is known α . In the EM algorithm framework, regarding $\mathbf{y}^\top = (y_1, y_2, \dots, y_n)$, $\mathbf{z}^\top = (z_1, z_2, \dots, z_n)$, and $\mathbf{x}^\top = (\mathbf{y}^\top, \mathbf{z}^\top)$ as the vector of observed data, vector of latent variables, and complete-data. It turns out from hierarchy (4.8) that

$$\begin{aligned} y_i | z_i &\sim \mathcal{N}(\delta, 2z_i\gamma^2), \\ z_i &\sim S(\alpha/2, 1, [\cos(\pi\alpha/4)]^{2/\alpha}, 0), \quad i = 1, \dots, n. \end{aligned} \quad (4.275)$$

Based on hierarchy (4.275) the complete-data likelihood function is

$$\begin{aligned} L_c(\Psi | \mathbf{x}) &= \prod_{i=1}^n z_i^{-\frac{1}{2}} \mathcal{N}(y_i | \delta, 2z_i\gamma^2) f_Z(z_i | \alpha), \\ &= \prod_{i=1}^n \frac{z_i^{-\frac{1}{2}}}{2\sqrt{\pi}\gamma} \exp\left\{-\frac{(y_i - \delta)^2}{4z_i\gamma^2}\right\} f_Z(z_i | \alpha), \end{aligned}$$

where $f_Z(\cdot | \alpha)$ is the PDF of $S(\alpha/2, 1, [\cos(\pi\alpha/4)]^{2/\alpha}, 0)$ whose CDF given by (2.31). The complete-data log-likelihood function is then becomes

$$\ell_c(\Psi | \mathbf{x}) = C - n \log(\gamma) - \frac{1}{4} \sum_{i=1}^n \frac{(y_i - \delta)^2}{\gamma^2} z_i^{-1}, \quad (4.276)$$

where C is a constant independent of Ψ . The PDF $f(z_i | y_i, \Psi)$ say, of $Z_i | (y_i, \Psi)$ has not close form and can be expression in terms of $f_Z(\cdot | \alpha)$ as

$$f(z_i | y_i, \Psi) = \frac{\mathcal{N}(y_i | \delta, 2z_i\gamma^2)}{f(y_i | \Psi)} f_Z(z_i | \alpha), \quad (4.277)$$

where $f(y_i | \Psi)$ is the PDF of observed data $y_i \sim S(\alpha, 0, \gamma, \delta)$. At $(k+1)$ iteration, two steps of the EM algorithm are as follows.

1. **E-step:** Computing $Q(\Psi|\Psi^{(k)})$ by taking expectation from (4.276) with respect to z_i given $\mathbf{y}$ and $\Psi^{(k)}$. We have

$$\begin{aligned} Q(\Psi|\Psi^{(k)}) &= E[\ell_c(\Psi|\mathbf{x})|\mathbf{y}, \Psi^{(k)}], \\ &= C - n \log(\gamma) - \frac{1}{4} \sum_{i=1}^n \frac{(y_i - \delta)^2}{\gamma^2} E_{i1}^{(k)}(\alpha), \end{aligned}$$

where $E_{i1}^{(k)}(\alpha) = E[z_i^{-1}|\mathbf{y}, \Psi^{(k)}]$. It follows from (4.277) that

$$\begin{aligned} E_{i1}^{(k)}(\alpha) &= \frac{\int_0^\infty z^{-1} \mathcal{N}(y_i|\delta, 2z \times \gamma^2) f_Z(z|\alpha) dz}{f(y_i|\Psi)} \\ &= \frac{\int_0^\infty z^{-1} \mathcal{N}(y_i|\delta, 2z \times \gamma^2) f_Z(z|\alpha) dz}{\int_0^\infty \mathcal{N}(y_i|\delta, 2z \times \gamma^2) f_Z(z|\alpha) dz}. \end{aligned} \quad (4.278)$$

Since $E_{i1}^{(k)}(\alpha)$ in (4.278) has not closed form, we prefer to follow the MCEM algorithm rather than the ordinary EM algorithm. To this end, we approximate $E_{i1}^{(k)}(\alpha)$ in (4.278) as

$$\begin{aligned} \tilde{E}_{i1}^{(k)}(\alpha) &= \frac{B^{-1} \sum_{b=1}^B u_{i,b}^{-1} \mathcal{N}(y_i|\delta^{(k)}, 2u_{i,b} \times \gamma^{2(k)})}{B^{-1} \sum_{b=1}^B \mathcal{N}(y_i|\delta^{(k)}, 2u_{i,b} \times \gamma^{2(k)})}, \\ &= \frac{\sum_{b=1}^B u_{i,b}^{-\frac{3}{2}} \exp\left\{-\frac{1}{4} \sum_{i=1}^n \frac{(y_i - \delta^{(k)})^2}{u_{i,b} \times \gamma^{2(k)}}\right\}}{\sum_{b=1}^B u_{i,b}^{-\frac{1}{2}} \exp\left\{-\frac{1}{4} \sum_{i=1}^n \frac{(y_i - \delta^{(k)})^2}{u_{i,b} \times \gamma^{2(k)}}\right\}}, \end{aligned} \quad (4.279)$$

where $u_{i,1}, \dots, u_{i,B}$ come independently from $S(\alpha/2, 1, [\cos(\pi\alpha/4)]^{2/\alpha}, 0)$. Therefore,

$$\begin{aligned} \tilde{Q}(\Psi|\Psi^{(k)}) &= E[\ell_c(\Psi|\mathbf{x})|\mathbf{y}, \Psi^{(k)}], \\ &= C - n \log(\gamma) - \frac{1}{4} \sum_{i=1}^n \frac{(y_i - \delta)^2}{\gamma^2} \tilde{E}_{i1}^{(k)}(\alpha), \end{aligned} \quad (4.280)$$

2. **M-step:** Maximizing RHS of (4.281) with respect to δ and γ for finding $\Psi^{(k+1)}$. This can be accomplished by differentiating RHS of (4.281) with respect to δ and γ and then solving the equations when is set to zero. We have

$$\frac{\partial \tilde{Q}(\Psi|\Psi^{(k)})}{\partial \delta} = \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \delta)}{\gamma^2} \tilde{E}_{i1}^{(k)}(\alpha) = 0. \quad (4.281)$$

$$\frac{\partial \tilde{Q}(\Psi|\Psi^{(k)})}{\partial \gamma} = -\frac{n}{\gamma} + \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \delta)^2}{\gamma^3} \tilde{E}_{i1}^{(k)}(\alpha) = 0. \quad (4.282)$$

It follows from (4.281) that

$$\delta^{(k+1)} = \frac{\sum_{i=1}^n y_i \tilde{E}_{i1}^{(k)}(\alpha)}{\sum_{i=1}^n \tilde{E}_{i1}^{(k)}(\alpha)}. \quad (4.283)$$

Once we have obtained $\delta^{(k+1)}$, then by (4.282), we have

$$[\tilde{\gamma}^{(k+1)}]^2 = \frac{\sum_{i=1}^n [y_i - \delta^{(k+1)}]^2 \tilde{E}_{i1}^{(k)}(\alpha)}{2n}. \quad (4.284)$$

To assess the performance of the MCEM algorithm, we simulate $n = 100$ realizations from SaS distribution using function `rstab(.)` (with arguments $\alpha = 1.2$, $\beta = 0$, $\gamma = 1$, and $\delta = 0$) given in Subsection 2.3.6. Based on starting value $\hat{\Psi}^{(0)} = (1, 0)^\top$, the MCEM algorithm, as it is seen from Figure 4.9, moves smoothly after $k = 10$ th iteration. We mention that the MCEM algorithm has been terminated at $k = 30$ th iteration when a number of $B = 1000$ realizations for approximating the posterior expectation in E-step. The corresponding R code is given by the following.

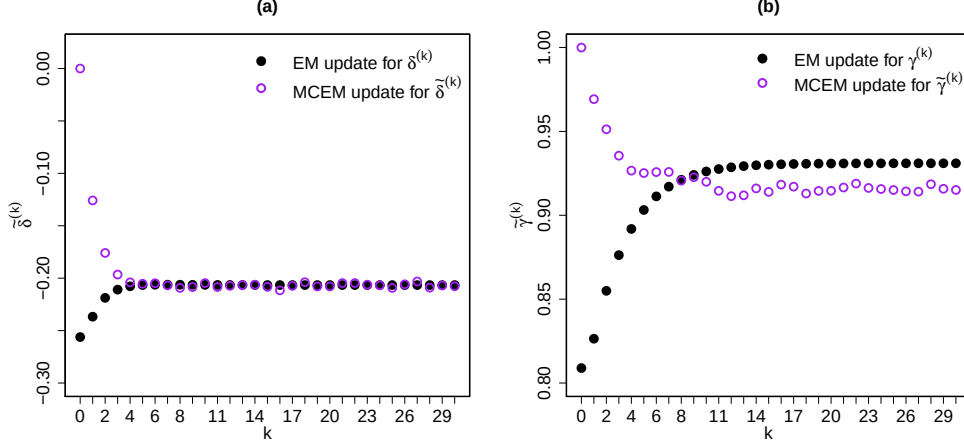


Figure 4.9: The output of EM and MCEM algorithms for estimating (a): δ and (b): γ , when SaS distribution is fitted to $n = 100$ simulated realizations.

```

1 R> set.seed(20260714)
2 R> n <- 100; alpha <- 1.2; beta <- 0; gamma <- 1; delta <- 0
3 R> y <- rstab(n, alpha, beta, gamma, delta)
4 R> M <- 1000; K <- 30; tilde_Psi_iter <- matrix(NA, nrow= K+1, ncol = 2);
5 R> tilde_Psi_iter[1, ] <- c(delta, gamma); tilde_Ei1 <- rep(NA, n);
6 R> for(k in 1:K)
7 +{
8 + ### E-step of the MCEM algorithm
9 + for(i in 1:n)
10 + {
11 + u <- rstable(M,alpha/2,1,(cos(pi*alpha/4))^(2/alpha),0)
12 + p1 <- dnorm(y[i],mean=tilde_Psi_iter[k,1],sd=sqrt(2*u)*tilde_Psi_iter[k,2])
13 + num <- sum(1/u*p1 )
14 + dnum <- sum( p1 )
15 + tilde_Ei1[i] <- num/dnum
16 + }
17 + ### M-step of the MCEM algorithm
18 + tilde_Psi_iter[k+1,1]<-sum(y*tilde_Ei1)/sum(tilde_Ei1)
19 + tilde_Psi_iter[k+1,2]<-sqrt(sum((y-tilde_Psi_iter[k+1,1])^2*tilde_Ei1)/(2*n))
20 + dif <- sum( abs( tilde_Psi_iter[k+1, ] - tilde_Psi_iter[k, ] ) )
21 + cat( "The EM output at iteration", k,"is:", tilde_Psi_iter[k, ], "\n" )
22 +}

```

As to the ordinary EM algorithm, using the starting values that obtained based on the log-moment method [Kuruoglu, 2001], we have $\hat{\Psi}_{EM}^\top = (0.9310, -0.2065)$. The R code for implementing the ordinary EM algorithm is given by the following.

```

1 R> dstab <- function(x, par, log = FALSE) # computes the PDF of S-alpha-S dist.
2 +{
3 + w <- c(

```

```

4 + 0.002908622553, 0.006759799195, 0.010590548383, 0.014380822761, 0.018115560713,
5 + 0.021780243170, 0.025360673570, 0.028842993580, 0.032213728223, 0.035459835615,
6 + 0.038568756612, 0.041528463090, 0.044327504338, 0.046955051303, 0.049400938449,
7 + 0.051655703069, 0.053710621888, 0.055557744806, 0.057189925647, 0.058600849813,
8 + 0.059785058704, 0.060737970841, 0.061455899590, 0.061936067420, 0.062176616655,
9 + 0.062176616655, 0.061936067420, 0.061455899590, 0.060737970841, 0.059785058704,
10 + 0.058600849813, 0.057189925647, 0.055557744806, 0.053710621888, 0.051655703069,
11 + 0.049400938449, 0.046955051303, 0.044327504338, 0.041528463090, 0.038568756612,
12 + 0.035459835615, 0.032213728223, 0.028842993580, 0.025360673570, 0.021780243170,
13 + 0.018115560713, 0.014380822761, 0.010590548383, 0.006759799195, 0.002908622553)
14 + node <- c(
15 + 0.99886640442, 0.99403196943, 0.98535408404, 0.97286438510, 0.95661095524,
16 + 0.93665661894, 0.91307855665, 0.88596797952, 0.85542976942, 0.82158207085,
17 + 0.78455583290, 0.74449430222, 0.70155246870, 0.65589646568, 0.60770292718,
18 + 0.55715830451, 0.50445814490, 0.44980633497, 0.39341431189, 0.33550024541,
19 + 0.27628819377, 0.21600723687, 0.15489058999, 0.09317470156, 0.03109833832,
20 + -0.03109833832, -0.09317470156, -0.15489058999, -0.21600723687, -0.27628819377,
21 + -0.33550024541, -0.39341431189, -0.44980633497, -0.50445814490, -0.55715830451,
22 + -0.60770292718, -0.65589646568, -0.70155246870, -0.74449430222, -0.78455583290,
23 + -0.82158207085, -0.85542976942, -0.88596797952, -0.91307855665, -0.93665661894,
24 + -0.95661095524, -0.97286438510, -0.98535408404, -0.99403196943, -0.99886640442)
25 + a <- par[1]; s <- par[2]; m <- par[3]; n <- length(x)
26 + PDF <- rep(NA, n); let <- !( is.na(x) | abs(x) == Inf )
27 + y <- x[let]; z <- (y - m)/s; az <- abs(y - m)/s; n <- length(z)
28 + PDF <- rep(NA, n)
29 + if( abs(a - 1) < 0.005 )
30 + {
31 + PDF <- dt(z, df = 1)/s
32 + }else{
33 + up_tol <- 50
34 + low_tol <- 4e-08
35 + j <- ifelse( abs(a - 1) < 0.03, 80, 55 )
36 + k <- seq(1, j)
37 + j2 <- ifelse(a < 0.5, 5, 150)
38 + k2 <- seq(1, j2, 2)
39 + up <- if( (a > 1.1 | | a < 0.3) ){ up <- pi/2 }else{ up <- 1.55 }
40 + ex1 <- ifelse( a > 1.02, 2*a, a)
41 + ex2 <- 0.8*a
42 + L <- max(0, (a*exp(lgamma(a*j + a) - lgamma(a*j + 1)))^(1/a) + ex1)
43 + U <- max(0, a*exp(lgamma(j/a + 1) - lgamma(j/a + 1/a)) - ex2)
44 + if( (a > 1.005 & a <= 1.02) )
45 + {
46 + L <- 3; U <- 0.05
47 + }else if( (a < 0.995 & a >= 0.98) )
48 + {
49 + L <- 0.94; U <- 0.31
50 + }
51 + let_L <- which(az > L)
52 + z_L <- z[ let_L ]
53 + let_U <- which(az < U)
54 + z_U <- z[ let_U ]
55 + let_0 <- which( (az >= U & az <= L) )
56 + z_0 <- z[ let_0 ]
57 + n_U <- length(z_U)
58 + n_L <- length(z_L)
59 + n_0 <- n - n_U - n_L
60 + if( length(let_0) >= 1 )
61 + {
62 + for(i in let_0)
63 + {
64 + low <- ifelse(a < 0.99, 0, min( 10e-16, up_tol*az[i]^(a/(1-a)),
65 + low_tol*az[i]^(a/(1-a)) ) )
66 + low <- ifelse( (a > 0.99 & a <1), 0.05, low)

```

```

67 + v1 <- function(u) (1/sin(a*u))^(a/(a-1))*cos((a-1)*u)*
68 +   cos(u)^(1/(a-1)) - up_tol*az[i]^(a/(1-a))
69 + v2 <- function(u) (1/sin(a*u))^(a/(a-1))*cos((a-1)*u)*
70 +   cos(u)^(1/(a-1)) - low_tol*az[i]^(a/(1-a))
71 + a0 <- uniroot(v1, lower = low, upper = up)$root
72 + b0 <- uniroot(v2, lower = low, upper = up)$root
73 + d0 <- abs(b0 - a0)
74 + x <- d0 *(node + 1)/2 + min(a0, b0)
75 + out <- d0/2*sum(a/(pi*abs(a-1)*az[i]*s)*w*az[i]^(a/(a-1))*
76 +   (1/sin(a*x))^(a/(a-1))* cos((a-1)*x)*cos(x)^(1/(a-1))*
77 +   exp(-az[i]^(a/(a-1))*(1/sin(a*x))^(a/(a-1))*
78 +   cos((a-1)*x)*cos(x)^(1/(a-1))))
79 + PDF[i] <- ifelse(d0 == 0, gamma(1 + 1/a)/pi, out)
80 + }
81 + }
82 + if( length(let_L) >= 1 )
83 + {
84 +   PDF[let_L] <- sapply(let_L,function(i) 1/(pi*az[i]*s)*
85 +     sum( (-1)^(k-1)*exp(lgamma(a*k+1)-lgamma(k+1))*
86 +     az[i]^(-a*k)*sin(k*pi/2*a) ) )
87 + }
88 + if( length(let_U) >= 1 )
89 + {
90 +   PDF[let_U] <- sapply(let_U,function(i) 1/(pi*az[i]*s)*
91 +     sum( (-1)^(k2-1)*exp(lgamma(k2/a+1)-lgamma(k2+1))*
92 +     az[i]^k2*sin(k2*pi/2) ) )
93 + }
94 + }
95 + PDF[az == 0] <- gamma(1 + 1/a)/pi
96 + if( log == TRUE ) PDF[let] <- log( PDF[let] )
97 + return(PDF)
98 +}
99 R> s_alpha_s_EM <- function(x) # compute the EM estimator for S-alpha-S dist.
100 +{
101 + n <- length(x)
102 + let <- !( is.na(x) | abs(x) == Inf )
103 + y <- x[let]
104 + ##### starting values
105 + delta <- median(y)
106 + y0 <- abs(y - delta)
107 + mean_log <- mean( log(y0) )
108 + var_log <- var( log(y0) )
109 + alpha <- min(1/sqrt( abs(6*var_log/pi^2 - 1/2) ), 1)
110 + gamma <- exp( mean_log - 0.57721*(1/alpha - 1) )
111 + K <- 200; Psi_iter <- matrix(NA, nrow = K, ncol = 4);
112 + k <- 1; dif <- 1; tol <- 10e-8;
113 + Psi_iter[1, ] <- c(alpha, gamma, delta, 0)
114 + min_loglike <- 10e+8
115 + h <- 0.01
116 + while( dif > tol )
117 + {
118 +   y_s <- (y - delta)/gamma
119 +   ##### E-step of the EM algorithm
120 +   E11 <- -2/y_s*(dstab(y_s+h,c(alpha,1,0),log=TRUE)-
121 +     dstab(y_s-h,c(alpha,1,0),log=TRUE))/(2*h)
122 +   ##### M-step of the EM algorithm
123 +   delta <- sum( y*E11 )/sum( E11 )
124 +   gamma <- sqrt( sum( (y - delta)^2*E11, na.rm = TRUE )/(2*n) )
125 +   ##### CML-step of the EM algorithm
126 +   margin_log_like <- function(x) -sum(dstab(y,c(x,gamma,delta),log=TRUE))
127 +   out <- suppressWarnings( optimize( margin_log_like, c(0.2, 1.999) ) )
128 +   alpha <- out$minimum
129 +   if(out$objective <= min_loglike)

```

```

130 + Psi_iter[k + 1, ] <- c(alpha, gamma, delta, out$objective)
131 + dif <- sum( abs( Psi_iter[k + 1, 1:3] - Psi_iter[k, 1:3] ) )
132 + cat( "The EM output at iteration", k, "is:", Psi_iter[k, 1:3], "\n" )
133 + k <- k + 1
134 + }
135 + EM <- list( alpha = Psi_iter[k, 1], gamma = Psi_iter[k, 2],
136 + delta = Psi_iter[k, 3], log_likelihood = Psi_iter[k, 4])
137 +return( EM )
138 +}

```

■

Example 4-20

Let $\mathbf{U} = (U_1, U_2, U_3, U_4)^\top \sim \mathcal{MUL}(n, \boldsymbol{\omega})$ in which $\boldsymbol{\omega} = (1/2 + \theta/4, (1 - \theta)/4, (1 - \theta)/4, \theta/4)^\top$. The ML estimator of θ can be computed directly by applying non-linear optimization tools available in R on logarithm of observed likelihood function

$$L(\theta|\mathbf{u}) = \frac{n!}{u_1!u_2!u_3!u_4!} \left(\frac{1}{2} + \frac{\theta}{4}\right)^{u_1} \left(\frac{1 - \theta}{4}\right)^{u_2+u_3} \left(\frac{\theta}{4}\right)^{u_4}, \quad (4.285)$$

where $\mathbf{u} = (u_1, u_2, u_3, u_4)^\top$ is vector of observed data such that $\sum_{i=1}^4 u_i = n$. Herein, instead of computing the ML estimator $\hat{\theta}$, through maximizing (4.285), we are willing to use the EM algorithm. To this end, suppose $\mathbf{T} = (T_1, T_2, T_3, T_4, T_5)^\top \sim \mathcal{MUL}(n, \boldsymbol{\pi})$ in which $\boldsymbol{\pi} = (1/2, \theta/4, (1 - \theta)/4, (1 - \theta)/4, \theta/4)^\top$. In this setting, we only observe

$$\mathbf{t} = (t_1 + t_2, t_3, t_4, t_5)^\top = (u_1, u_2, u_3, u_4)^\top \quad (4.286)$$

rather than $(t_1, t_2, t_3, t_4, t_5)^\top$, define $Z = t_2$ as the latent variable, and set $\mathbf{X} = (t_1, t_2, t_3, t_4, t_5)^\top$ to be the vector of complete-data. Evidently,

$$L_c(\theta|\mathbf{X}) = L_c(\theta|\mathbf{t}, Z) = \frac{n!}{t_1!t_2!t_3!t_4!t_5!} \left(\frac{1}{2}\right)^{t_1} \left(\frac{\theta}{4}\right)^{t_2} \left(\frac{1 - \theta}{4}\right)^{t_3+t_4} \left(\frac{\theta}{4}\right)^{t_5}, \quad (4.287)$$

and

$$P(Z = z|\mathbf{t}, \theta) = \frac{L_c(\theta|\mathbf{t}, z)}{L(\theta|\mathbf{t})}, \quad (4.288)$$

where $\mathbf{t}$ is given in (4.286), numerator and denominator in RHS of (4.288) are given in (4.287) and (4.286), respectively. It easy to see that

$$Z|(\mathbf{t}, \theta) \sim \mathcal{BIN}\left(t_1 + t_2, \frac{\theta}{2 + \theta}\right). \quad (4.289)$$

Two steps of the EM algorithm are as follows.

1. **E-step:** Computing $Q(\theta|\theta^{(t)})$. Taking logarithm of (4.287), we have

$$\begin{aligned}
Q(\theta|\theta^{(t)}) &= E[\ell_c(\theta|\mathbf{X})|\mathbf{t}, \theta^{(t)}] \\
&= C + E[T_2|\mathbf{t}, \theta^{(t)}] \log(\theta) + (t_3 + t_4) \log(1 - \theta) + t_5 \log(\theta) \\
&= C + \frac{(t_1 + t_2)\theta}{2 + \theta} \log(\theta) + (t_3 + t_4) \log(1 - \theta) + t_5 \log(\theta).
\end{aligned} \quad (4.290)$$

2. **M-step:** Maximizing RHS of (4.288) with respect to θ for finding $\theta^{(t+1)}$. We have

$$\frac{\partial Q(\theta|\theta^{(t)})}{\partial \theta} = \frac{(t_1 + t_2)}{\theta} \frac{\theta^{(t)}}{2 + \theta^{(t)}} - \frac{(t_3 + t_4)}{1 - \theta} + \frac{t_5}{\theta} = 0. \quad (4.291)$$

It turns out that

$$\theta^{(t+1)} = \frac{(t_1 + t_2)\theta^{(t)} + t_5[2 + \theta^{(t)}]}{(t_1 + t_2)\theta^{(t)} + (t_5 + t_4 + t_3)[2 + \theta^{(t)}]}. \quad (4.292)$$

Equivalently, based on (4.286), RHS of (4.292) can be rewritten as

$$\theta^{(t+1)} = \frac{u_1\theta^{(t)} + u_4[2 + \theta^{(t)}]}{u_1\theta^{(t)} + (u_4 + u_3 + u_2)[2 + \theta^{(t)}]}. \quad (4.293)$$

In what follows, for $\theta = \{0.25, 0.50, 0.75\}$ and $n = 50$, we run a simulation study in which the $\hat{\theta}$ is found both directly by maximizing (4.285) and the EM algorithm. Table 4.15 shows the results of simulation. We note that the true value of θ is set to be the starting value, that is $\theta^{(0)} = \theta$, for the EM algorithm. In what follows, we give R code for implementing the EM

Table 4.15: Output of the EM algorithm for multinomial distribution.

iteration	EM output		
	$\theta = 0.25$	$\theta = 0.50$	$\theta = 0.75$
0	0.25	0.50	0.75
1	0.1923077	0.4230769	0.6538462
2	0.1721992	0.4056604	0.6418685
3	0.1646911	0.4014085	0.6402544
4	0.1618164	0.4003517	0.6400347
5	0.1607052	0.4000879	0.6400047
6	0.1602741	0.4000220	0.6400006
7	0.1601066	0.4000055	0.6400001
8	0.1600414	0.4000014	0.6400000
9	0.1600161	0.4000003	0.6400000
10	0.1600063	0.4000001	0.6400000
11	0.1600024	0.4000000	0.6400000
12	0.1600009	0.4000000	0.6400000
13	0.1600004	0.4000000	0.6400000
14	0.1600001	0.4000000	0.6400000
MLE	0.1600193	0.3999997	0.6400203

algorithm for this example.

```

1 R> set.seed(20260607)
2 R> n <- 50; theta <- 0.25; N <- 100; theta_iter <- rep(0, N);
3 R> theta_iter[1] <- theta; j <- 1; dif <- 1; tol <- 10e-8;
4 R> success <- c(1/2 + theta/4, (1 - theta)/4, (1 - theta)/4, theta/4)
5 R> y <- apply( rmultinom(1, n, prob = success), 1, sum )
6 R> y0 <- sum(y[-1])
7 R> observed_loglik <- function(x) -sum( dmultinom(y, size = n,
8 + prob = c(1/2 + x/4, (1 - x)/4, (1 - x)/4, x/4), log = TRUE) )
9 R> ML <- optimize(observed_loglik, lower = 0, upper = 1)$minimum
10 R> while( dif > tol )
11 + {
12 +   theta_iter[j + 1] <- (theta_iter[j]*y[1] + y[4]*(2 + theta_iter[j])) /
13 +     (theta_iter[j]*y[1] + y0*(2 + theta_iter[j]))
14 +   dif <- abs( theta_iter[j + 1] - theta_iter[j] )
15 +   cat( "The_ML_estimator_is:", ML, "and_the_EM_at_iteration", j,"is:", theta_iter[j], "\n" )
16 +   j <- j + 1
17 + }

```

Example 4-21

Suppose $\mathbf{y}^\top = (y_1, y_2, \dots, y_n)$ denotes a sample of n independent observations from Student's t distribution with known ν degrees of freedom. The ML estimator of ν can be directly found through maximizing

$$\hat{\Psi} = \arg \max_{\mu \in \mathbb{R}, \sigma \in \mathbb{R}^+} \sum_{i=1}^n \log \left\{ \frac{1}{\sigma} t_\nu \left(\frac{y_i - \mu}{\sigma} \middle| 0, 1 \right) \right\}, \quad (4.294)$$

where $\Psi^\top = (\mu, \sigma)$. In the EM algorithm framework, regarding $\mathbf{y}^\top = (y_1, y_2, \dots, y_n)$, $\mathbf{z}^\top = (z_1, z_2, \dots, z_n)$, and $\mathbf{x}^\top = (\mathbf{y}^\top, \mathbf{z}^\top)$ as the vector of observed data, vector of latent variables, and complete-data. Recall from (4.7) that each Student's t random variable can be represented as a GSM model for $\eta(Z) = Z \sim \mathcal{G}(\nu/2, \nu/2)$. Hence, using hierarchy (4.8), we have

$$\begin{aligned} y_i | z_i &\sim \mathcal{N} \left(\mu, \frac{\sigma^2}{z_i} \right), \\ z_i &\sim \mathcal{G} \left(\frac{\nu}{2}, \frac{\nu}{2} \right), \quad i = 1, \dots, n. \end{aligned} \quad (4.295)$$

Following hierarchy (4.295) the complete-data likelihood function as

$$L_c(\Psi | \mathbf{x}) = \prod_{i=1}^n \frac{z_i^{\frac{1}{2}}}{\sqrt{2\pi}\sigma} \exp \left\{ -\frac{z_i (y_i - \mu)^2}{2\sigma^2} \right\} \frac{(\frac{\nu}{2})^{\frac{\nu}{2}}}{\Gamma(\frac{\nu}{2})} z_i^{\frac{\nu}{2}-1} \exp \left\{ -z_i \frac{\nu}{2} \right\}.$$

The complete-data log-likelihood function is then becomes

$$\ell_c(\Psi | \mathbf{x}) = C - n \log(\sigma) - \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \mu)^2}{\sigma^2} z_i + \frac{\nu}{2} \sum_{i=1}^n \log(z_i) - \frac{\nu}{2} \sum_{i=1}^n z_i, \quad (4.296)$$

where C is a constant independent of Ψ . It is not hard to check that (see problem 19):

$$Z_i | (y_i, \Psi) \sim \mathcal{G} \left(\frac{\nu+1}{2}, \frac{1}{2} \frac{(y_i - \mu)^2}{\sigma^2} + \frac{\nu}{2} \right). \quad (4.297)$$

Two steps of the EM algorithm are as follows.

1. **E-step:** Computing $Q(\Psi | \Psi^{(t)})$ by taking expectation with respect to z_i given $\mathbf{y}$ and $\Psi^{(k)}$. We have

$$\begin{aligned} Q(\Psi | \Psi^{(k)}) &= E[\ell_c(\Psi | \mathbf{x}) | \mathbf{y}, \Psi^{(k)}], \\ &= C - n \log(\sigma) - \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \mu)^2}{\sigma^2} E[z_i | \mathbf{y}, \Psi^{(k)}] + \frac{\nu}{2} \sum_{i=1}^n E[\log(z_i) | \mathbf{y}, \Psi^{(k)}] \\ &\quad - \frac{\nu}{2} E[z_i | \mathbf{y}, \Psi^{(k)}]. \end{aligned}$$

It follows from (4.297) that

$$E[z_i | \mathbf{y}, \Psi^{(k)}] = E_{i1}^{(k)}(\nu) = \frac{\nu+1}{\nu + [\sigma^{(k)}]^{-2} [y_i - \mu^{(k)}]^2}, \quad (4.298)$$

$$E[\log(z_i) | \mathbf{y}, \Psi^{(k)}] = E_{i2}^{(k)}(\nu) = \Psi \left(0, \frac{\nu+1}{2} \right) - \log \left(\frac{1}{2} \left[\frac{y_i - \mu^{(k)}}{\sigma^{(k)}} \right]^2 + \frac{\nu}{2} \right), \quad (4.299)$$

where $\Psi(0, x)$ is digamma function defined as (4.136). So

$$Q(\Psi | \Psi^{(k)}) = C - n \log(\sigma) - \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \mu)^2}{\sigma^2} E_{i1}^{(k)}(\nu) + \frac{\nu}{2} \sum_{i=1}^n E_{i2}^{(k)}(\nu) - \frac{\nu}{2} \sum_{i=1}^n E_{i1}^{(k)}(\nu). \quad (4.300)$$

Moreover, since quantities $E_{i2}^{(k)}(\nu)$ and $E_{i1}^{(k)}(\nu)$ in (4.298) and (4.299) are independent of Ψ , then RHS of (4.300) can be rewritten in more concise form as follows.

$$Q(\Psi|\Psi^{(k)}) = C - n \log(\sigma) - \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \mu)^2}{\sigma^2} E_{i1}^{(k)}(\nu). \quad (4.301)$$

2. **M-step:** Maximizing RHS of (4.301) with respect to μ and σ for finding $\Psi^{(k+1)}$. This can be accomplished by differentiating RHS of (4.301) with respect to μ and σ and then solving the equations when is set to zero. We have

$$\begin{aligned} \frac{\partial Q(\Psi|\Psi^{(k)})}{\partial \mu} &= \sum_{i=1}^n \frac{(y_i - \mu)}{\sigma^2} E_{i1}^{(k)}(\nu) = 0. \\ \frac{\partial Q(\Psi|\Psi^{(k)})}{\partial \sigma} &= -\frac{n}{\sigma} + \sum_{i=1}^n \frac{(y_i - \mu)^2}{\sigma^3} E_{i1}^{(k)}(\nu) = 0. \end{aligned} \quad (4.302)$$

Substituting $E_{i1}^{(k)}(\nu)$ from (4.298) into RHS of (4.302) yields

$$\mu^{(k+1)} = \frac{\sum_{i=1}^n y_i E_{i1}^{(k)}(\nu)}{\sum_{i=1}^n E_{i1}^{(k)}(\nu)}. \quad (4.303)$$

Once we have obtained $\mu^{(k+1)}$, then we have

$$[\sigma^{(k+1)}]^2 = \frac{\sum_{i=1}^n [y_i - \mu^{(k+1)}]^2 E_{i1}^{(k)}(\nu)}{n}. \quad (4.304)$$

By (4.298), if $\nu \rightarrow \infty$ then $E_{i1}^{(k)}(\nu) \rightarrow 1$ indicating the fact that the EM updates in (4.303) and (4.304) are the ML estimator of mean and variance of Gaussian as the limiting distribution of Student's t . Figure 4.10 displays the result of simulation for estimating $\Psi^\top = (\mu = 0, \sigma = 1)$ when a set of $n = 50$ observation are independently generated from Student's t distribution with $\nu = \{5, 30\}$. Herein, the sample mean and standard deviation are used as the starting values for implementing the EM algorithm. As it is seen, the EM algorithm converges before 10th iteration. The R code for computing the EM estimator for $\Psi^\top = (\mu = 0, \sigma = 1)$ when

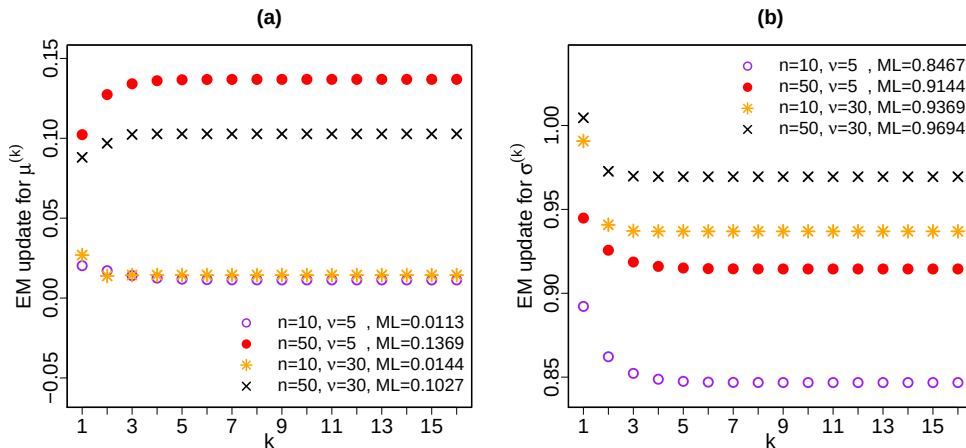


Figure 4.10: The output of EM algorithm for estimating (a): $\mu = 0$ and (b): $\sigma = 1$. $\nu = 5$ is given below.

```

1 R> set.seed(20260608)
2 R> n <- 20; mu <- 0; sigma <- 1; nu <- 5
3 R> y <- sigma*rt(n, df = nu) + mu
4 R> start <- c( median(y), sd(y) )
5 R> observed_loglik <- function(theta)
6 + {
7 +   mu = theta[1]; sigma = theta[2];
8 +   -sum( dt( (y - mu)/sigma, df = nu, log = TRUE ) - log(sigma) )
9 + }
10 R> ML <- optim(par = start, fn = observed_loglik, method = "L-BFGS-B",
11 +   lower = c(-Inf, 0), upper = rep(Inf, 2) )$par
12 R> N <- 100; theta_iter <- matrix(0, nrow = N, ncol = 2);
13 R> theta_iter[1, ] <- start
14 R> j <- 1; dif <- 1; tol <- 10e-8
15 R> while( dif > tol)
16 + {
17 +   E_i1 <- (nu + 2)/( nu + (y - mu)^2/sigma^2 )
18 +   mu <- sum(y*E_i1)/sum(E_i1)
19 +   sigma <- sqrt( sum( (y - mu)^2*E_i1 )/n )
20 +   theta_iter[j + 1, ] <- c(mu, sigma)
21 +   dif <- sum( abs( theta_iter[j + 1, ] - theta_iter[j, ] ) )
22 +   cat( "The EM at iteration", j,"is:", mu, ",", sigma, "\n" )
23 +   j <- j + 1
24 + }

```

■

In next example, we proceed to find the ML estimator of the GSM model and then find the ML estimator for $\Psi^\top = (\mu, \sigma, \boldsymbol{\theta} = \nu)$ as the Student's t distribution parameter vector.

Example 4-22

For a given positive real-valued function $\eta(\cdot)$, it follows from 4.8 that

$$\begin{aligned}
 y_i | z_i &\sim \mathcal{N}\left(\mu, \frac{\sigma^2}{\eta(z_i)}\right), \\
 z_i &\sim h(\cdot | \boldsymbol{\theta}), \quad i = 1, \dots, n.
 \end{aligned} \tag{4.305}$$

Based on hierarchy (4.305) above and considering $\mathbf{y}^\top = (y_1, y_2, \dots, y_n)$, $\mathbf{z}^\top = (z_1, z_2, \dots, z_n)$, and $\mathbf{x}^\top = (\mathbf{y}^\top, \mathbf{z}^\top)$ as the vector of observed data, vector of latent variables, and complete-data, the complete-data likelihood function is

$$L_c(\Psi | \mathbf{x}) = \prod_{i=1}^n \frac{\sqrt{\eta(z_i)}}{\sqrt{2\pi}\sigma} \exp\left\{-\frac{\eta(z_i)}{2} \frac{(y_i - \mu)^2}{\sigma^2}\right\} h(z_i | \boldsymbol{\theta}),$$

where $\Psi^\top = (\mu, \sigma, \boldsymbol{\theta}^\top)$ and $h(\cdot | \boldsymbol{\theta})$ is the PDF of mixing random variable. The complete-data log-likelihood function is then becomes

$$\ell_c(\Psi | \mathbf{x}) = C - n \log(\sigma) - \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \mu)^2}{\sigma^2} \eta(z_i) + \sum_{i=1}^n \log[h(z_i | \boldsymbol{\theta})], \tag{4.306}$$

where C is a constant independent of Ψ . Two steps of the EM algorithm are as follows.

1. **E-step:** Computing $Q(\Psi | \Psi^{(k)})$ by taking expectation with respect to z_i given $\mathbf{y}$ and $\Psi^{(k)}$. At $(k + 1)$ th iteration of the EM algorithm, we have

$$\begin{aligned}
 Q(\Psi | \Psi^{(k)}) &= E[\ell_c(\Psi | \mathbf{x}) | \mathbf{y}, \Psi^{(k)}], \\
 &= C - n \log(\sigma) - \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \mu)^2}{\sigma^2} E[\eta(z_i) | \mathbf{y}, \Psi^{(k)}] + \sum_{i=1}^n E[\log h(z_i | \boldsymbol{\theta}) | \mathbf{y}, \Psi^{(k)}],
 \end{aligned}$$

or equivalently

$$Q(\Psi|\Psi^{(k)}) = C - n \log(\sigma) - \frac{1}{2} \sum_{i=1}^n \frac{(y_i - \mu)^2}{\sigma^2} E_{i1}^{(k)}(\boldsymbol{\theta}) + \sum_{i=1}^n E[\log h(z_i|\boldsymbol{\theta})|\mathbf{y}, \Psi^{(k)}], \quad (4.307)$$

where

$$\begin{aligned} E_{i1}^{(k)}(\boldsymbol{\theta}) &= \int_0^\infty \eta(z_i) f(z_i|y_i, \Psi^{(k)}) dz_i \\ &= \int_0^\infty \eta(z_i) \frac{\mathcal{N}\left(y_i \middle| \mu^{(k)}, \frac{\sigma^{2(k)}}{\eta(z_i)}\right) h(z_i|\boldsymbol{\theta})}{f(y_i|\Psi^{(k)})} dz_i \\ &= \frac{\int_0^\infty [\eta(z_i)]^{\frac{3}{2}} \exp\left\{-\frac{\eta(z_i)}{2} \left(\frac{y_i - \mu^{(k)}}{\sigma^{(k)}}\right)^2\right\} h(z_i|\boldsymbol{\theta}) dz_i}{\int_0^\infty [\eta(z_i)]^{\frac{1}{2}} \exp\left\{-\frac{\eta(z_i)}{2} \left(\frac{y_i - \mu^{(k)}}{\sigma^{2(k)}}\right)^2\right\} h(z_i|\boldsymbol{\theta}) dz_i}. \end{aligned} \quad (4.308)$$

To avoid computing two integrals in RHS of (4.308), one can easily check that

$$\begin{aligned} \frac{\partial}{\partial y_i} \int_0^\infty [\eta(z_i)]^{\frac{1}{2}} \exp\left\{-\frac{\eta(z_i)}{2} \left(\frac{y_i - \mu^{(k)}}{\sigma^{(k)}}\right)^2\right\} h(z_i|\boldsymbol{\theta}) dz_i = \\ - \frac{y_i - \mu^{(k)}}{[\sigma^{(k)}]^2} \int_0^\infty [\eta(z_i)]^{\frac{3}{2}} \exp\left\{-\frac{\eta(z_i)}{2} \left(\frac{y_i - \mu^{(k)}}{\sigma^{(k)}}\right)^2\right\} h(z_i|\boldsymbol{\theta}) dz_i. \end{aligned} \quad (4.309)$$

Hence, (4.308) reduces to

$$\begin{aligned} E_{i1}^{(k)}(\boldsymbol{\theta}) &= - \frac{[\sigma^{(k)}]^2}{y_i - \mu^{(k)}} \frac{\partial}{\partial y_i} \log \int_0^\infty [\eta(z_i)]^{\frac{1}{2}} \exp\left\{-\frac{\eta(z_i)}{2} \left(\frac{y_i - \mu^{(k)}}{\sigma^{(k)}}\right)^2\right\} h(z_i|\boldsymbol{\theta}) dz_i \\ &= - \frac{[\sigma^{(k)}]^2}{y_i - \mu^{(k)}} \times \frac{\partial}{\partial y_i} \log f(y_i|\Psi^{(k)}), \quad y_i \neq \mu^{(k)}. \end{aligned} \quad (4.310)$$

This means that the E-step is complete simply by differentiating PDF of observed data. Table 4.2 shows the PDF $f(\cdot|\Psi^{(k)})$ of three well-known GSM models in univariate and multivariate cases. In this case, we have the Student's t distribution with PDF $t_\nu(\cdot|\mu, \sigma)$. More algebra yields

$$E_{i1}^{(k)}(\nu) = - \frac{\pi(\nu+1)}{\nu} \times \frac{y_i - \mu^{(k)}}{[\sigma^{(k)}]^3} \times t_\nu\left(\frac{y_i - \mu^{(k)}}{\sigma^{(k)}} \middle| 0, 1\right) \times t_1\left(\frac{y_i - \mu^{(k)}}{\sqrt{\nu}\sigma^{(k)}} \middle| 0, 1\right). \quad (4.311)$$

In Example 4-21 assumes $\boldsymbol{\theta} = \nu$ is unknown. Herein, we let $\boldsymbol{\theta}$ to be unknown and shall estimate it using the CML-steps of the within ECME algorithm as follows.

2. **CM-step:** At $(k+1)$ th iteration of ECME algorithm, we maximize RHS of (4.307) with respect to μ and σ for finding $\mu^{(k+1)}$ and $\sigma^{(k+1)}$. This yields Substituting $E_{i1}^{(k)}(\nu)$ from (4.298) into RHS of (4.302) yields

$$\mu^{(k+1)} = \frac{\sum_{i=1}^n y_i E_{i1}^{(k)}(\nu)}{\sum_{i=1}^n E_{i1}^{(k)}(\nu)}. \quad (4.312)$$

Once we have obtained $\mu^{(k+1)}$, then we have

$$[\sigma^{(k+1)}]^2 = \frac{\sum_{i=1}^n [y_i - \mu^{(k+1)}]^2 E_{i1}^{(k)}(\nu)}{n}, \quad (4.313)$$

where $E_{i1}^{(k)}(\nu)$ is given by (4.311).

3. **CML-step:** Having $\mu^{(k+1)}$ and $\sigma^{(k+1)}$, at $(k+1)$ th iteration of ECME algorithm, we update $\nu^{(k)}$ by maximizing the marginal log-likelihood of observed data as

$$\nu^{(k+1)} = \arg \max_{\nu \in \mathbb{R}^+} \sum_{i=1}^n \log \left\{ \frac{1}{\sigma^{(k+1)}} t_{\nu} \left(\frac{y_i - \mu^{(k+1)}}{\sigma^{(k+1)}} \middle| 0, 1 \right) \right\}. \quad (4.314)$$

In practice, we may use nonlinear optimization routines in R such as `optimize(.)` to obtain $\nu^{(k+1)}$ in (4.314).

Above E-, CM-, and CML-steps are alternatively repeated until convergence occurs for maximizer $\Psi^{(k+1)}$ within the ECME algorithm framework. On the other hand, one may accomplish the M-step of the ordinary EM algorithm, rather than CM- and CML-steps as above, for computing the ML estimators of Student's t distribution. To this end, for the RHS of (4.307), we can write

$$\begin{aligned} \sum_{i=1}^n E[\log h(z_i | \theta) | \mathbf{y}, \Psi^{(k)}] &= \sum_{i=1}^n E \left[\log \left(\frac{\nu}{2} \right)^{\frac{\nu}{2}} z_i^{\frac{\nu}{2}-1} \exp \left\{ -z_i \frac{\nu}{2} \right\} \middle| \mathbf{y}, \Psi^{(k)} \right] \\ &= n \frac{\nu}{2} \log \left(\frac{\nu}{2} \right) - n \log \Gamma \left(\frac{\nu}{2} \right) + \left(\frac{\nu}{2} - 1 \right) \sum_{i=1}^n E_{i2}^{(k)}(\nu^{(k)}) - \frac{\nu}{2} \sum_{i=1}^n E_{i1}^{(k)}(\nu^{(k)}). \end{aligned} \quad (4.315)$$

where, by (4.299), we have

$$E_{i2}^{(k)}(\nu^{(k)}) = E[\log z_i | \mathbf{y}, \Psi^{(k)}] = \Psi \left(0, \frac{\nu^{(k)}}{2} + 1 \right) - \log \left(\frac{1}{2} \left[\frac{y_i - \mu^{(k)}}{\sigma^{(k)}} \right]^2 + \frac{\nu^{(k)}}{2} \right). \quad (4.316)$$

Substituting RHS of (4.315) into RHS of (4.307), differentiating the result with respect to ν , and then equating to zero yields

$$\frac{\partial Q(\Psi | \Psi^{(k)})}{\partial \nu} = \frac{n}{2} \log \left(\frac{\nu}{2} \right) + \frac{n}{2} - \frac{n}{2} \Psi \left(0, \frac{\nu}{2} \right) + \frac{1}{2} \sum_{i=1}^n E_{i2}^{(k)}(\nu^{(k)}) - \frac{1}{2} \sum_{i=1}^n E_{i1}^{(k)}(\nu^{(k)}) = 0. \quad (4.317)$$

In practice, since RHS of (4.317) is strictly monotonic in $\nu > 0$, one may use routine `uniroot(.)` to solve RHS of (4.317). The M-step is complete by solving RHS of (4.317) in terms of ν . The E- and M-steps are alternatively repeated till convergence occurs for maximizer $\Psi^{(k+1)}$ within the EM algorithm framework. The R code below provided for computing the ML estimator of Student's t distribution parameter $\Psi^T = (\mu, \sigma, \nu)$ using both of the EM and ECME algorithms, and then verifying that the ECME algorithm is faster than the EM algorithm.

```

1 R> EM_t <- function(x, method = c("EM", "ECME"))
2 +{
3 + n <- length(x)
4 + let <- !( is.na(x) | abs(x) == Inf )
5 + y <- x[let]
6 + #### starting values
7 + mu <- median(y); sigma <- sd(y); nu <- 5
8 + K <- 5000; Psi_iter <- matrix(NA, nrow = K, ncol = 4);
9 + k <- 1; dif <- 1; tol <- 10e-6;
10 + Psi_iter[1, ] <- c(nu, sigma, mu, 0)
11 + while( (dif > tol & k < 5000) )
12 + {
13 + y_s <- (y - mu)/sigma
14 + #### E-step of the EM algorithm
15 + E11 <- (nu + 1)/( nu + y_s^2)
16 + E12 <- digamma((nu + 1)/2) - log( y_s^2/2 + nu/2 )
17 + #### M-step of the EM algorithm

```

```

18 + mu <- sum(y*Ei1)/sum(Ei1)
19 + sigma <- sqrt( sum( (y - mu)^2*Ei1 )/n )
20 + if(method == "EM")
21 + {
22 + g <- function(x) sum( log(x/2) + 1 - digamma(x/2) + Ei2 - Ei1 )
23 + nu <- uniroot(g, c(0.1, 200) )$root
24 + neg_log_like <- -sum( dt((y-mu)/sigma,df=nu,log=TRUE)-log(sigma) )
25 + ##### CML-step of the EM algorithm
26 + }else if(method == "ECME")
27 + {
28 + margin_log_like <- function(x)-sum(dt((y-mu)/sigma,df=x,log=T)-log(sigma))
29 + out <- suppressWarnings( optimize( margin_log_like, c(0.1, 200) ) )
30 + nu <- out$minimum; neg_log_like <- out$objective
31 + }
32 + Psi_iter[k + 1, ] <- c(nu, sigma, mu, -neg_log_like)
33 + dif <- ( abs( Psi_iter[k + 1, 4] - Psi_iter[k, 4] ) )
34 + cat( "The_log_likelihood_at_iteration", k,"is:", Psi_iter[k, 4], "\n" )
35 + k <- k + 1
36 + }
37 + EM <- list( nu = Psi_iter[k, 1], sigma = Psi_iter[k, 2],
38 + mu = Psi_iter[k, 3], log_likelihood = -Psi_iter[k, 4])
39 +return( EM )
40 +}

```

■

Finding the ML estimator of a K-component mixture model (6.55) is one of the most commonly used application of the EM algorithm. If the starting values are selected suitably, then the EM algorithm converges without any difficulty.

Example 4-23

Let $g(\cdot|\Psi)$ as defined in (6.55) is the PDF of a K-component Gaussian mixture model in which each component follows $\mathcal{N}(\mu_j, \sigma_j^2)$, for $j = 1, \dots, K$. Let $\mathbf{u}^\top = (u_1, u_2, \dots, u_n)$ represent the observed random sample from model $g(\cdot|\Psi)$ where $\Psi^\top = (\theta_1^\top, \theta_2^\top, \theta_3^\top)$ in which $\theta_1^\top = (\omega_1, \dots, \omega_K)$, $\theta_2^\top = (\mu_1, \dots, \mu_K)$, and $\theta_3^\top = (\sigma_1, \dots, \sigma_K)$ such that $\sum_{j=1}^K \omega_j = 1$. The ML estimator $\hat{\Psi}$, of Ψ can be directly found as

$$\hat{\Psi} = \arg \max_{\Psi} \prod_{i=1}^n \sum_{j=1}^K \frac{\omega_j}{\sqrt{2\pi}\sigma_j} \exp\left\{-\frac{1}{2}\left(\frac{u_i - \mu_j}{\sigma_j}\right)^2\right\}. \quad (4.318)$$

Finding $\hat{\Psi}$ through (4.318) is not computationally straightforward. But, the EM algorithm enable us to represent the complete-data likelihood function in a simple form by which $\hat{\Psi}$ can be found using the optimization methods easily. To this end, we regard the vector of complete-data as

$$\mathbf{x} = \{\mathbf{x}_1^\top, \dots, \mathbf{x}_n^\top\} = \{(y_1, z_1), \dots, (y_n, z_n)\}, \quad (4.319)$$

in which $\mathbf{u}^\top = \mathbf{y}^\top = (y_1, y_2, \dots, y_n)$ is vector of observed data $\mathbf{z} = \{z_1^\top, \dots, z_n^\top\}$ is vector of *missing variables*. Herein, $\mathbf{z}_i^\top = (z_{i1}, \dots, z_{iK})$ defines the origin component of observed value y_i , for $i = 1, \dots, n$. By convention, the j th component, say z_{ij} (for $j = 1, \dots, K$) is one with probability ω_j and the others are zero. This means that $\mathbf{z}_i \sim \mathcal{MUL}(1, \boldsymbol{\omega})$ or equivalently

$$P(\mathbf{Z}_i = \mathbf{z}_i) = \frac{\omega_1^{z_{i1}} \times \dots \times \omega_K^{z_{iK}}}{z_{i1}! \times \dots \times z_{iK}!}, \quad z_{ij} \in \{0, 1\}, \quad \sum_{j=1}^K z_{ij} = 1. \quad (4.320)$$

Sometimes $\mathbf{z}$ is called vector of *missing labels* due to widespread application of the finite mixture models in model-based clustering. In fact, in model-based clustering, it is assumed that each observation accompanied by a missing label, herein $\mathbf{z}_i$, indicating the true number of cluster that observation u_i is coming from. In what follows, as the key step for implementing the EM algorithm, we represent the complete-data log-likelihood $\ell_c(\Psi|\mathbf{x})$ function. We have

$$\ell_c(\Psi|\mathbf{x}) = \prod_{i=1}^n f(y_i, \mathbf{z}_i|\Psi), \quad (4.321)$$

where

$$\begin{aligned} f(y_i, \mathbf{z}_i|\Psi) &= P(\mathbf{Z}_i = \mathbf{z}_i) f_{Y_i|\mathbf{Z}_i}(y_i|\mathbf{z}_i) \\ &= \frac{\omega_1^{z_{i1}} \times \cdots \times \omega_K^{z_{iK}}}{z_{i1}! \cdots z_{iK}!} \\ &\quad \times \left\{ f_{Y_i|Z_{i1}}(y_i|z_{i1}, \mu_j, \sigma_j) \right\}^{z_{i1}} \times \cdots \times \left\{ f_{Y_i|Z_{iK}}(y_i|z_{iK}, \mu_K, \sigma_K) \right\}^{z_{iK}} \\ &= \prod_{j=1}^K \left\{ \omega_j f_{Y_i|Z_{ij}}(y_i|z_{ij}, \mu_j, \sigma_j) \right\}^{z_{ij}}, \end{aligned} \quad (4.322)$$

where $Z_{ij} \in \{0, 1\}$, $\sum_{j=1}^K Z_{ij} = 1$, and $f_{Y_i|Z_{i1}}(y_i|z_{i1}, \mu_j, \sigma_j)$ is PDF of $\mathcal{N}(\mu_j, \sigma_j)$, that is

$$f_{Y_i|Z_{i1}}(y_i|z_{i1}, \mu_j, \sigma_j) = \frac{1}{\sqrt{2\pi}\sigma_j} \exp\left\{-\frac{1}{2}\left(\frac{y_i - \mu_j}{\sigma_j}\right)^2\right\}. \quad (4.323)$$

Substituting RHS of (4.323) into RHS of (4.322), the complete-data log-likelihood is

$$\ell_c(\Psi|\mathbf{x}) = C + \sum_{i=1}^n \sum_{j=1}^K z_{ij} \log(\omega_j) - \sum_{i=1}^n \sum_{j=1}^K z_{ij} \left[\log(\sigma_j) + \frac{(y_i - \mu_j)^2}{2\sigma_j^2} \right], \quad (4.324)$$

where C is a constant independent of Ψ . It is not hard to check that (see problem 19):

$$\mathbf{Z}_i|\mathbf{y}_i, \Psi \sim \mathcal{MUL}(1, \boldsymbol{\tau}_i). \quad (4.325)$$

where $\boldsymbol{\tau}_i^\top = (\tau_{i1}, \dots, \tau_{iK})$ with

$$\tau_{ij} = \frac{\omega_j \times \mathcal{N}(y_i|\mu_j, \sigma_j^2)}{\sum_{j=1}^K \omega_j \times \mathcal{N}(y_i|\mu_j, \sigma_j^2)}, \quad j = 1, \dots, K, i = 1, \dots, n. \quad (4.326)$$

At $(k+1)$ th iteration of the EM algorithm, the E- and M-steps of the algorithm are as follows.

1. **E-step:** Computing $Q(\Psi|\Psi^{(k)})$ by taking expectation with respect to $\mathbf{z}_i$ given $\mathbf{y}$ and $\Psi^{(k)}$. We have

$$\begin{aligned} Q(\Psi|\Psi^{(k)}) &= E[\ell_c(\Psi|\mathbf{x})|\mathbf{y}, \Psi^{(k)}], \\ &= C + \sum_{i=1}^n \sum_{j=1}^K E[Z_{ij}|\mathbf{y}, \Psi^{(k)}] \log(\omega_j) \\ &\quad + \sum_{i=1}^n \sum_{j=1}^K E[Z_{ij}|\mathbf{y}, \Psi^{(k)}] \left[\log(\sigma_j) + \frac{(y_i - \mu_j)^2}{2\sigma_j^2} \right] \end{aligned} \quad (4.327)$$

where

$$E[Z_{ij}|\mathbf{y}, \Psi^{(k)}] = \tau_{ij}^{(k)} = \frac{\omega_j^{(k)} \times \mathcal{N}(y_i|\mu_j^{(k)}, [\sigma_j^{(k)}]^2)}{\sum_{j=1}^K \omega_j^{(k)} \times \mathcal{N}(y_i|\mu_j^{(k)}, [\sigma_j^{(k)}]^2)}. \quad (4.328)$$

When y_i is far from any components, then the RHS of (4.328) can be rewritten as

$$E[Z_{ij}|\mathbf{y}, \mathbf{\Psi}^{(k)}] = \tau_{ij}^{(k)} = \left[1 + \frac{\sum_{j' \neq j}^K \omega_{j'}^{(k)} \times \mathcal{N}(y_i|\mu_{j'}^{(k)}, [\sigma_{j'}^{(k)}]^2)}{\omega_j^{(k)} \times \mathcal{N}(y_i|\mu_j^{(k)}, [\sigma_j^{(k)}]^2)}\right]^{-1}, \quad (4.329)$$

to avoid the indeterminant case 0/0 [Tatiana et al., 2009, pp. 5].

2. **M-step:** Maximizing RHS of (4.302) with respect to $\mathbf{\Psi}$ for finding $\mathbf{\Psi}^{(k+1)}$. We have

$$\frac{\partial Q(\mathbf{\Psi}|\mathbf{\Psi}^{(k)})}{\partial \omega_j} = \frac{\sum_{i=1}^n \tau_{ij}^{(k)}}{\omega_j}. \quad (4.330)$$

As it is seen, RHS of (4.329) is always positive. For solving $\partial Q(\mathbf{\Psi}|\mathbf{\Psi}^{(k)})/\partial \omega_j = 0$, we use the fact that $\sum_{j=1}^K \omega_j = 1$. So we maximize RHS of (4.327) by enforcing the Lagrange multiplier as follows.

$$Q(\mathbf{\Psi}|\mathbf{\Psi}^{(k)}) + \eta \left(\sum_{j=1}^K \omega_j - 1 \right). \quad (4.331)$$

Hence

$$\begin{aligned} \frac{\partial Q(\mathbf{\Psi}|\mathbf{\Psi}^{(k)})}{\partial \omega_j} &= \frac{\partial}{\partial \omega_j} \left\{ \sum_{i=1}^n \sum_{j=1}^K \tau_{ij}^{(k)} \log \omega_j + \sum_{i=1}^n \sum_{j=1}^K \tau_{ij}^{(k)} \left[\log(\sigma_j) + \frac{(y_i - \mu_j)^2}{2\sigma_j^2} \right] \right. \\ &\quad \left. + \eta \left(\sum_{j=1}^K \omega_j - 1 \right) \right\} = 0. \end{aligned} \quad (4.332)$$

Differentiating RHS of (4.331) with respect to ω_j , we have

$$\frac{1}{\omega_j} \sum_{i=1}^n \tau_{ij}^{(k)} + \eta = 0. \quad (4.333)$$

Rearranging (4.333) we have

$$\sum_{i=1}^n \tau_{ij}^{(k)} = -\eta \omega_j. \quad (4.334)$$

Keeping in mind that

$$\sum_{j=1}^K \sum_{i=1}^n \tau_{ij}^{(k)} = n, \quad (4.335)$$

it follows from (4.334)

$$n = -\eta \sum_{j=1}^K \omega_j = -\eta. \quad (4.336)$$

Substituting $\eta = -n$ into RHS of (4.334) yields $\omega = 1/n \sum_{i=1}^n \tau_{ij}^{(k)}$ or equivalently

$$\omega_j^{(k+1)} = \frac{1}{n} \sum_{i=1}^n \tau_{ij}^{(k)}. \quad (4.337)$$

Furthermore, differentiating RHS of (4.327) with respect to μ_j we obtain

$$\mu_j^{(k+1)} = \frac{\sum_{i=1}^n y_i \tau_{ij}^{(k)}}{\sum_{i=1}^n \tau_{ij}^{(k)}}. \quad (4.338)$$

Once we have obtained $\mu_j^{(k+1)}$, then we have

$$[\sigma_j^{(k+1)}]^2 = \frac{\sum_{i=1}^n [y_i - \mu_j^{(k+1)}]^2 \tau_{ij}^{(k)}}{\sum_{i=1}^n \tau_{ij}^{(k)}}. \quad (4.339)$$

Using the vector of starting values $\Psi^{(0)}$ and iterating alternatively equations (4.336)-(4.339), the EM algorithm attains at the ML estimator $\hat{\Psi}$. Herein, we use function `rmixnorm1()` for simulating $n = 100$ observations from 3-component Gaussian mixture model with parameter vector $\Psi^\top = (\theta_1^\top, \theta_2^\top, \theta_3^\top)$ in which $\theta_1^\top = (0.3, 0.3, 0.4)$, $\theta_2^\top = (-3, 0, 3)$, and $\theta_3^\top = (1, 0.5, 1)$.

```

1  R> mixnorm_fit <- function(y, K, start)
2  +{
3  + #start includes the starting values in form
4  + #c(omega_1,...,omega_K,mu_1,...,mu_K,sigma_1,...,sigma_K)
5  + n <- length(y)
6  + omega <- start[1:K]; mu <- start[(K+1):(2*K)]; sigma <- start[(2*K+1):(3*K)]
7  + stopifnot("sum of mixing proportions must be one." = sum(omega) == 1 )
8  + N <- 1000; n_par <- 3*K - 1
9  + psi_iter <- matrix(0, nrow = N, ncol = n_par + 1);
10 + psi_iter[1, ] <- c(omega, mu, sigma)
11 + j <- 1; dif <- 1; tol <- 10e-7;
12 + while( dif > tol )
13 + {
14 + p1 <- sapply(1:K, function(k) omega[k]*dnorm(y, mean = mu[k], sd = sigma[k]) )
15 + tau_ik <- sapply(1:K, function(j) 1/( 1 + p1[, -j]/p1[, j] ) )
16 + #tau_ik <- p1/apply(p1, 1, sum)
17 + colsum_tau_ik <- apply(tau_ik, 2, sum)
18 + omega <- colsum_tau_ik/n
19 + mu <- c( as.vector(y)%*%tau_ik )/colsum_tau_ik
20 + y_shift <- sweep(matrix( y, nrow = n, ncol = K ), 2, mu, FUN = "-" )
21 + sigma <- sqrt( apply( y_shift^2*tau_ik, 2, sum)/colsum_tau_ik )
22 + psi_iter[j + 1, ] <- c(omega, mu, sigma)
23 + dif <- sum( abs( psi_iter[j + 1, ] - psi_iter[j, ] ) )
24 + label <- apply( tau_ik, 1, which.max )
25 + #cat( "The EM at iteration", j,"is: (", mu,",",sigma,")", "\n" )
26 + j <- j + 1
27 + }
28 + out <- list( label = label, omega = omega, mu = mu, sigma = sigma)
29 + return(out)
30 +}

```



In what follows, Example 4-24 illustrates how to utilize the EM algorithm for estimating parameters of the K-components skewed Student's t distribution. Each components of this mixture model is supposed to follow a skewed Student's t distribution constructed based on the CSG scale mixture mode as Definition 4.1.2 with gamma random variable as mixing distribution ⁷.

⁷This type of Student's t distribution is referred to as *restricted* skewed Student's t distribution but, as pointed out by Murray et al. [2017], such terminology is in contrast to the stochastic representation of skew-normal distribution in (4.4) since λ is vector, but random variate Z_0 is scalar. We refer reader to Murray et al. [2017] for more information.

Example 4-24

Let $\mathbf{y} = (\mathbf{y}_1^\top, \dots, \mathbf{y}_n^\top)$ denote a sample of n realizations observed from a K -component p -dimensional skewed Student's t distribution with PDF

$$g(\mathbf{y}|\Psi) = \sum_{j=1}^K \omega_j f_{\mathbf{Y}}(\mathbf{y}|\Psi_j), \quad (4.340)$$

in which, for i th observation $\mathbf{y}_i^\top = (y_{i1}, \dots, y_{ip})$, the PDF of j -th component is represented as

$$f_{\mathbf{Y}}(\mathbf{y}_i|\Psi_j) = 2t_{\nu_j}(\mathbf{y}_i|\boldsymbol{\mu}_j, \Omega_j) T_{p+\nu_j}\left(m_{ij} \sqrt{\frac{\nu_j + p}{\nu_j + d_j(\mathbf{y}_i)}} \middle| 0, \Delta_j\right), \quad (4.341)$$

with

$$d_j(\mathbf{y}_i) = (\mathbf{y}_i - \boldsymbol{\mu}_j)^\top \Omega_j^{-1} (\mathbf{y}_i - \boldsymbol{\mu}_j), \quad (4.342)$$

$$m_{ij} = \boldsymbol{\lambda}_j^\top \Omega_j^{-1} (\mathbf{y}_i - \boldsymbol{\mu}_j), \quad (4.343)$$

$$\Omega_j = \Sigma_j + \boldsymbol{\lambda}_j \boldsymbol{\lambda}_j^\top, \quad (4.344)$$

$$\delta_j = 1 - \boldsymbol{\lambda}_j^\top \Omega_j^{-1} \boldsymbol{\lambda}_j. \quad (4.345)$$

We denote the parameter vector of model (4.340) by $\Psi = \{\boldsymbol{\omega}, \Psi_1, \dots, \Psi_K\}$ in which $\Psi_j = \{\boldsymbol{\mu}_j, \boldsymbol{\lambda}_j, \Sigma_j, \nu_j\}$ (for $j = 1, \dots, K$) and $\boldsymbol{\omega}^\top = (\omega_1, \dots, \omega_K)$ is the vector of mixing parameters (proportions) such that $\sum_{j=1}^K \omega_j = 1$.

For model (4.340), the complete-data consists of a vector of missing variables (component's labels), two vectors of latent variables, and vector of observed data. As to the missing variable, it is worth to mention that, for each observation $\mathbf{y}_i$ we consider a missing label showing the component that $\mathbf{y}_i$ is coming from and represented as $\mathbf{z}_i^\top = (z_{i1}, \dots, z_{iK})$ (for $i = 1, \dots, n$). As usual, if $\mathbf{y}_i$ comes with probability ω_j from j th component (for $j = 1, \dots, K$), then j th element of $\mathbf{z}_i^\top$ is one (and by convention the others are zero). The complete-data is represented as $\mathbf{x} = \{\mathbf{y}, \mathbf{u}, \mathbf{g}, \mathbf{z}\}$ that consists the observed data, first vector of unobservable variables, second vector of unobservable variables, and vector of missing labels. We have

$$\begin{aligned} \mathbf{y} &= (\mathbf{y}_1^\top, \dots, \mathbf{y}_n^\top), & \text{vector of observed data,} \\ \mathbf{u} &= (\mathbf{u}_1^\top, \dots, \mathbf{u}_1^\top)^\top, & \text{vector of first unobservable variables.} \\ \mathbf{g} &= (\mathbf{g}_1^\top, \dots, \mathbf{g}_1^\top)^\top, & \text{vector of second unobservable variables.} \\ \mathbf{z} &= (\mathbf{z}_1^\top, \dots, \mathbf{z}_n^\top), & \text{vector of missing variables.} \end{aligned}$$

The best way for realizing the number of missing and unobservable variables involved in the model is to constructing the model's hierarchy. As to the K -component p -dimensional skewed Student's t model with PDF (4.340), based on Corollary 4.1.2, the hierarchy for constituting the complete-data likelihood takes the following form.

$$\begin{aligned} \mathbf{Y}_i | U_{ij} = u_{ij}, G_{ij} = g_{ij}, Z_{ij} = 1 &\sim \mathcal{N}_p\left(\boldsymbol{\mu}_j + \boldsymbol{\lambda}_j u_{ij}, \frac{\Sigma_j}{g_{ij}}\right), \\ U_{ij} | G_{ij} = g_{ij}, Z_{ij} = 1 &\sim \mathcal{HN}\left(0, \frac{1}{g_{ij}}\right), \\ G_{ij} | Z_{ij} = 1 &\sim \mathcal{G}\left(\frac{\nu_j}{2}, \frac{\nu_j}{2}\right), \\ \mathbf{Z}_i &\sim \mathcal{MUL}(1, \boldsymbol{\omega}), \quad i = 1, \dots, n, j = 1, \dots, K. \end{aligned} \quad (4.346)$$

So, as it is seen from (4.346), associated with the mixture model (4.340), there are one missing and two unobservable (or latent) variables for constructing the complete-data log-likelihood $\ell_c(\Psi)$. We have

$$\ell_c(\Psi|\mathbf{x}) = \prod_{i=1}^n f(\mathbf{y}_i|\Psi) = \prod_{i=1}^n \sum_{j=1}^K \omega_j f_{\mathbf{Y}}(\mathbf{y}_i|\Psi_j), \quad (4.347)$$

Based on arguments similar to those given in (4.322), it follows that

$$\begin{aligned} \ell_c(\Psi|\mathbf{x}) &= \prod_{i=1}^n \prod_{j=1}^K \left[\omega_j f_{\mathbf{Y}}|_{U_{ij}, G_{ij}, Z_{ij}}(\mathbf{y}_i|u_{ij}, g_{ij}, z_{ij}, \Psi) \right. \\ &\quad \left. f_{U_{ij}|G_{ij}, Z_{ij}}(u_{ij}|g_{ij}, z_{ij}, \Psi) f_{G_{ij}|Z_{ij}}(g_{ij}|z_{ij}, \Psi) \right]^{z_{ij}}, \end{aligned} \quad (4.348)$$

where conditional PDFs in RHS of (4.348) are known from hierarchy (4.346). That is

$$f_{\mathbf{Y}}|_{U_{ij}, G_{ij}, Z_{ij}}(\mathbf{y}_i|u_{ij}, g_{ij}, z_{ij}, \Psi) = \frac{(2\pi)^{-\frac{p}{2}}}{|\Sigma_j|^{\frac{1}{2}}} \exp\left\{-\frac{g_{ij}}{2}(\mathbf{y}_i - \mu_j - \lambda_j u_{ij})^\top \Sigma_j^{-1}(\mathbf{y}_i - \mu_j - \lambda_j u_{ij})\right\}, \quad (4.349)$$

$$f_{U_{ij}|G_{ij}, Z_{ij}}(u_{ij}|g_{ij}, z_{ij}, \Psi) = \frac{2}{\sqrt{2\pi}} \exp\left\{-\frac{u_{ij}^2}{2}\right\} \times \mathbb{I}_{\mathbb{R}^+}(u_{ij}), \quad (4.350)$$

$$f_{G_{ij}|Z_{ij}}(g_{ij}|z_{ij}, \Psi) = \left(\frac{\nu_j}{2}\right)^{\frac{\nu_j}{2}} \frac{g_{ij}^{\frac{\nu_j}{2}-1}}{\Gamma(\frac{\nu_j}{2})} \exp\left\{-\frac{\nu_j}{2}g_{ij}\right\}. \quad (4.351)$$

Replacing PDFs in RHSs of (4.349)- (4.353) with those in RHS of (4.348), and then taking logarithm, the complete-data log-likelihood becomes

$$\ell_c(\Psi|\mathbf{x}) = \ell_{1c}(\omega|\mathbf{x}) + \ell_{2c}(\Psi|\mathbf{x}) + \ell_{3c}(\Psi|\mathbf{x}),$$

where

$$\ell_{1c}(\omega) = \sum_{i=1}^n \sum_{j=1}^K z_{ij} \log \omega_j, \quad (4.352)$$

$$\ell_{2c}(\Psi) = \sum_{i=1}^n \sum_{j=1}^K z_{ij} \left[\frac{\nu_j}{2} \log\left(\frac{\nu_j}{2}\right) + \left(\frac{\nu_j}{2} - 1\right) \log(g_{ij}) - \log\left[\Gamma\left(\frac{\nu_j}{2}\right)\right] - \frac{\nu_j}{2} g_{ij} \right], \quad (4.353)$$

$$\ell_{3c}(\Psi) = C - \sum_{i=1}^n \sum_{j=1}^K \frac{z_{ij}}{2} \left[\log |\Sigma_j| + g_{ij}(\mathbf{y}_i - \mu_j - \lambda_j u_{ij})^\top \Sigma_j^{-1}(\mathbf{y}_i - \mu_j - \lambda_j u_{ij}) \right], \quad (4.354)$$

where C is a constant independent of Ψ and

$$d_j(\mathbf{y}_i) = (\mathbf{y}_i - \mu_j)^\top \Omega_j^{-1}(\mathbf{y}_i - \mu_j), \quad (4.355)$$

$$m_{ij} = \lambda_j^\top \Omega_j^{-1}(\mathbf{y}_i - \mu_j), \quad (4.356)$$

$$\Omega_j = \Sigma_j + \lambda_j \lambda_j^\top, \quad (4.357)$$

$$\delta_j = 1 - \lambda_j^\top \Omega_j^{-1} \lambda_j. \quad (4.358)$$

Matrix algebra using Sherman-Morrison identity ([Petersen et al., 2008, pp. 18, Eq. 160]) shows

$$(\mathbf{y}_i - \mu_j - \lambda_j u_{ij})^\top \Sigma_j^{-1}(\mathbf{y}_i - \mu_j - \lambda_j u_{ij}) = d_j(\mathbf{y}_i) + \frac{(u_{ij} - m_{ij})^2}{\delta_{ij}}. \quad (4.359)$$

We use (4.359) in place of second term within bracket in RHS of (4.354) for the computational purposes. So

$$\ell_{3c}(\boldsymbol{\mu}, \Sigma, \boldsymbol{\lambda}) = C - \sum_{i=1}^n \sum_{j=1}^K \frac{z_{ij}}{2} \left[\log |\Sigma_j| + g_{ij} d_j(\mathbf{y}_i) + g_{ij} (u_{ij} - m_{ij})^2 \delta_{ij}^{-1} \right]. \quad (4.360)$$

The conditional expectation of complete-data log-likelihood $Q(\boldsymbol{\Psi}|\boldsymbol{\Psi}^{(k)}) = E(\ell_c(\boldsymbol{\Psi})|\underline{\mathbf{y}}, \boldsymbol{\Psi}^{(k)})$ is represented as

$$\begin{aligned} Q(\boldsymbol{\Psi}|\boldsymbol{\Psi}^{(k)}) &= E(\ell_{1c}(\boldsymbol{\Psi})|\underline{\mathbf{y}}, \boldsymbol{\Psi}^{(k)}) + E(\ell_{2c}(\boldsymbol{\Psi})|\underline{\mathbf{y}}, \boldsymbol{\Psi}^{(k)}) + E(\ell_{3c}(\boldsymbol{\Psi})|\underline{\mathbf{y}}, \boldsymbol{\Psi}^{(k)}) \\ &= Q_1(\boldsymbol{\Psi}|\boldsymbol{\Psi}^{(k)}) + Q_2(\boldsymbol{\Psi}|\boldsymbol{\Psi}^{(k)}) + Q_3(\boldsymbol{\Psi}|\boldsymbol{\Psi}^{(k)}). \end{aligned} \quad (4.361)$$

It is easy to check that, at $(k+1)$ th iteration, the E-step of the EM algorithm is complete by computing the following posterior expectations.

$$\tau_{ij}^{(k)} = E(z_{ij}|\mathbf{y}_i, \boldsymbol{\Psi}^{(k)}), \quad (4.362)$$

$$E_{1ij}^{(k)} = E(z_{ij} g_{ij}|\mathbf{y}_i, z_{ij} = 1, \boldsymbol{\Psi}^{(k)}) = \tau_{ij}^{(k)} E(g_{ij}|\mathbf{y}_i, z_{ij} = 1, \boldsymbol{\Psi}^{(k)}), \quad (4.363)$$

$$E_{2ij}^{(k)} = E(z_{ij} g_{ij} u_{ij}|\mathbf{y}_i, z_{ij} = 1, \boldsymbol{\Psi}^{(k)}) = \tau_{ij}^{(k)} E(g_{ij} u_{ij}|\mathbf{y}_i, z_{ij} = 1, \boldsymbol{\Psi}^{(k)}), \quad (4.364)$$

$$E_{3ij}^{(k)} = E(z_{ij} g_{ij} u_{ij}^2|\mathbf{y}_i, z_{ij} = 1, \boldsymbol{\Psi}^{(k)}) = \tau_{ij}^{(k)} E(g_{ij} u_{ij}^2|\mathbf{y}_i, z_{ij} = 1, \boldsymbol{\Psi}^{(k)}), \quad (4.365)$$

$$E_{4ij}^{(k)} = E(z_{ij} \log g_{ij}|\mathbf{y}_i, z_{ij} = 1, \boldsymbol{\Psi}^{(k)}) = \tau_{ij}^{(k)} E(\log g_{ij}|\mathbf{y}_i, z_{ij} = 1, \boldsymbol{\Psi}^{(k)}), \quad (4.366)$$

where $\boldsymbol{\Psi}^{(k)} = (\boldsymbol{\omega}, \Psi_1^{(k)}, \Psi_2^{(k)}, \Psi_3^{(k)})$. Therefore, three parts of $Q(\boldsymbol{\Psi}|\boldsymbol{\Psi}^{(k)})$ function in (4.361) can be represented as follows.

$$Q_1(\boldsymbol{\Psi}|\boldsymbol{\Psi}^{(k)}) = \sum_{i=1}^n \sum_{j=1}^K \tau_{ij}^{(k)} \log \omega_j, \quad (4.367)$$

$$Q_2(\boldsymbol{\Psi}|\boldsymbol{\Psi}^{(k)}) = \sum_{i=1}^n \sum_{j=1}^K \left[\tau_{ij}^{(k)} \frac{\nu_j}{2} \log \left(\frac{\nu_j}{2} \right) + \left(\frac{\nu_j}{2} - 1 \right) E_{4ij}^{(k)} - \tau_{ij}^{(k)} \log \left[\Gamma \left(\frac{\nu_j}{2} \right) \right] - \frac{\nu_j}{2} E_{1ij}^{(k)} \right], \quad (4.368)$$

$$Q_3(\boldsymbol{\Psi}|\boldsymbol{\Psi}^{(k)}) = C - \sum_{i=1}^n \sum_{j=1}^K \left[\frac{\tau_{ij}^{(k)}}{2} \log |\Sigma_j| + \frac{E_{1ij}^{(k)}}{2} d_j(\mathbf{y}_i) + \frac{E_{3ij}^{(k)}}{2} \delta_{ij}^{-1} + \frac{E_{2ij}^{(k)}}{2} m_{ij}^2 \delta_{ij}^{-1} - E_{2ij}^{(k)} m_{ij} \delta_{ij}^{-1} \right]. \quad (4.369)$$

For computing posterior expectations in (4.362)-(4.366), we mention that $\tau_{ij}^{(k)}$ can be easily evaluated as

$$\tau_{ij}^{(k)} = \frac{\omega_j^{(k)} f_{\mathbf{Y}}(\mathbf{y}_i|\boldsymbol{\Psi}^{(k)})}{\sum_{j=1}^K \omega_j^{(k)} f_{\mathbf{Y}}(\mathbf{y}_i|\boldsymbol{\Psi}^{(k)})}. \quad (4.370)$$

Other expectations are computed as follows. ■

4.6 Robust linear regression

Let $\mathbf{y}^\top = (y_1, y_2, \dots, y_n)$ denote a set of n observations of response variable and $X = \{x_{ij}\}$ (for $i = 1, \dots, n$ and $j = 1, \dots, k+1$) is the associated design matrix of k independent (explanatory) variables. The ordinary least squares (OLS) is a mathematical principle that aims to find the vector of coefficients $\hat{\boldsymbol{\beta}}^\top = (\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_k)$ in linear relationship

$$y_i = \mathbf{x}_i^\top \boldsymbol{\beta}, \quad i = 1, 2, \dots, n, \quad (4.371)$$

where $\mathbf{x}_i^\top = (1, x_{i1}, x_{i2}, \dots, x_{ik})$ such that

$$\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta} \in \mathbb{R}^{k+1}} \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2. \quad (4.372)$$

The statistical logic goes further by adding term ϵ_i , on the right-hand side of (4.371). This yields

$$y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + \epsilon_i, \quad i = 1, 2, \dots, n. \quad (4.373)$$

Herein, the term ϵ_i is meant to explain the recording or sampling error that occurs during the planning stage of statistical design. Naturally, it is assumed that ϵ_i s are *i.i.d* quantities each following a zero-mean Gaussian distribution with common variance, that is $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. In practice, these three assumptions may be violated and departure from each of these underlying assumptions can make the OLS estimator deteriorate. If violated, there are some treatments in such cases, among them, we mention the weighted regression and transformation. Unfortunately, applying latter two methods is subject to some restrictions. For example, for weighted method, finding suitable weights is not in general an easy task since determining the weights may need knowing the variance of the response variable and if the distribution of ϵ_i is not known due to departure from normality, then finding suitable weights would be impossible. One way that relaxes the restrictions of weighted and transformation methods is to consider a more flexible family for the distribution for ϵ_i . For instance, we may assume ϵ_i follows a GSM model (??) (for $\mu = 0$) in which the mixing random variable G is given by Table 4.1. Equivalently,

$$\epsilon_i = \frac{\mathcal{N}(0, \sigma^2)}{\sqrt{\eta(G)}}, \quad i = 1, 2, \dots, n. \quad (4.374)$$

Computing $\hat{\boldsymbol{\beta}}$ in (4.372) using the principle of OLS is computationally intensive. The Bayesian approach looks a good candidate but, as shown in Table 4.1, in such cases as 3 and 4 the posterior distribution has not close form making this approach less useful computationally. Herein, we are willing to compute $\hat{\boldsymbol{\beta}}$ through the ML method based on hierarchy (??). Based on (4.371), we have

$$\begin{aligned} Y_i | G = g &\sim \mathcal{N}(\mathbf{x}_i^\top \boldsymbol{\beta}, \sigma^2 \eta(g)), \\ G &\sim h(g | \boldsymbol{\theta}), \end{aligned} \quad (4.375)$$

where $h(\cdot | \boldsymbol{\theta})$ is the PDF of mixing random variable G with parameter vector $\boldsymbol{\theta}$. Then, the joint PDF of (Y, G) is

$$f_{Y_i, G}(y_i, g) = \frac{1}{\sqrt{2\pi\eta(g)}\sigma} \exp\left\{-\frac{(y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2}{2\sigma^2\eta(g)}\right\} h(g | \boldsymbol{\theta}). \quad (4.376)$$

4.7 Model and notation

Definition 4.7.1. Let random variate Y follow a skewed Student's t distribution. The probability density function (pdf) of Y is then given by [Azzalini and Capitanio, 2003]:

$$f(y, \boldsymbol{\theta}) = \frac{2}{\sigma} t_\nu(z) T_{\nu+1}\left(\lambda z \sqrt{\frac{\nu+1}{\nu+z^2}}\right), \quad (4.377)$$

where $z = (y - \mu)/\sigma$. Herein, $\boldsymbol{\theta}^\top = (\mu, \sigma, \lambda, \nu)$ is the family parameter vector in which $\mu \in \mathbb{R}$ is the location parameter, $\sigma > 0$ is the scale parameter, $\lambda \in \mathbb{R}$ is the skewness parameter, $\nu > 0$ is the degrees of freedom, and the pdf and cumulative distribution function (cdf) of the standard Student's t family with ν degrees of freedom are represented as $t_\nu(\cdot)$ and $T_\nu(\cdot)$, respectively.

It is worth to mention that the negative, zero, and positive values of λ in Definition 4.7.1 yields skewed to the left, symmetric, and skewed to the right pdf, respectively. The family of distributions in (4.377) specializes to the skewed Gaussian [Azzalini and Capitanio, 2003] for $\nu \rightarrow \infty$ and the ordinary Gaussian for $\lambda \rightarrow 0$ and $\nu \rightarrow \infty$. We write $Y \sim \text{ST}(\mu, \sigma, \lambda, \nu)$ to denote that random variate Y follows a distribution with pdf (4.377). Let y_i be the response associated with the covariate row $\mathbf{x}_i^\top = (1, x_{i1}, \dots, x_{ik})$ and regression coefficients $\boldsymbol{\beta}^\top = (\beta_0, \beta_1, \dots, \beta_k)$. The linear model is

$$y_i = \mathbf{x}_i^\top \boldsymbol{\beta} + \varepsilon_i, \quad i = 1, \dots, n, \quad (4.378)$$

where $\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \text{ST}(0, \sigma, \lambda, \nu)$ or equivalently $Y_i \mid \mathbf{x}_i \sim \text{ST}(\mathbf{x}_i^\top \boldsymbol{\beta}, \sigma, \lambda, \nu)$.

4.7.1 Coefficient estimator

Problems

1. The Simpson's 1/3 rule rule for numerically computation of an integral over interval $[a, b]$ works by computing the integrand $f(\cdot)$ at two end points and the midpoint of interval as

$$\int_a^b f(x)dx \approx \frac{h}{3} [f(a) + f(b) + 4f(a+h)],$$

where $h = (b - a)/2$. The error in approximating above integral becomes $-f^{(4)} \times (b - a)^5/2880$ in which $f^{(\xi)}$, for $a < \xi < b$, is the fourth order derivative of $f(\cdot)$ over $[a, b]$. Approximate the CDF of a standard Gaussian distribution at $z = \{-3, 0, 3\}$ using the Simpson's rule and compare the results with those of the **R** built-in function `pnorm(z)`.

2. When the correlation matrix $R = \{\rho_{i,j}\}$ of a zero-mean p -variate Gaussian distribution is tridiagonal so that $\rho_{i,i-1} = \rho$ for $2 \leq i \leq p$ and $\rho_{i,j} = 0$, for $|i - j| > 1$, then the exact value of orthant probability, that is $L(\mathbf{0}|R)$, would be known for some special cases [Miwa et al., 2003, pp. 225-226]. For instance, if $\rho = -1/2$, then $L(\mathbf{0}|R) = 1/(1 + p)!$. Check this fact numerically for $p = 2$ and $p = 3$ using functions `BVN_Drez` and `TVN1_Drez`, respectively.
3. Let $F(x|\Psi)$ is the CDF of a GSM distribution. Show that (4.175) can be approximated as follows.

$$F(x|\Psi) \approx \begin{cases} \frac{1}{2} + \frac{(b-a)\text{sign}(x)}{2\sqrt{\pi}} \int_{-1}^1 H\left(\frac{x^2}{2\sigma^2 y^2} \mid \boldsymbol{\theta}\right) \exp\{-y^2\} dz, & \text{if } \eta(g) = g, \\ \frac{1}{2} + \frac{(b-a)\text{sign}(x)}{2\sqrt{\pi}} \int_{-1}^1 \bar{H}\left(\frac{2\sigma^2 y^2}{x^2} \mid \boldsymbol{\theta}\right) \exp\{-y^2\} dz, & \text{if } \eta(g) = \frac{1}{g}, \end{cases}$$

where $H(\cdot|\boldsymbol{\theta})$ is the CDF of mixing distribution and $\bar{H}(\cdot|\boldsymbol{\theta}) = 1 - H(\cdot|\boldsymbol{\theta})$.

4. Explain that how $F(x|\Psi)$ in (4.175) can be approximated by a k -point Gauss-Hermite rule of the first kind.
5. Let $f(x, y) = \max\{0, F(x|\nu) - F(-y|\nu)\}$ in which $F(x|\nu)$ is the CDF of the Student's t distribution with ν degrees of freedom. Prove that $f(x, y) = f(y, x)$ for any paired $(x, y) \in \mathbb{R}^2$. Is this result valid for any symmetric distribution around origin?
6. Prove the continuity property of $F(\mathbf{x}|\Psi)$ in Theorem 4.4.1 at $\rho = 0$. Hint: integrate both sides of (4.188) over interval $(-\epsilon, \epsilon)$ and then take limit when $\epsilon \downarrow 0$ to show that

$$\lim_{\epsilon \rightarrow 0} |F(\mathbf{x}|\Psi) - F(\mathbf{x}|\Psi')| \approx \lim_{\epsilon \rightarrow 0} \frac{|\epsilon|}{2\pi} \mathcal{M}_G\left[-\frac{1}{2}\|\mathbf{x}\|^2\right] = 0,$$

where $\Psi = (\mathbf{0}, R = \{\epsilon\}, \boldsymbol{\theta})$, $\Psi' = (\mathbf{0}, R = \{-\epsilon\}, \boldsymbol{\theta})$, and $\|\cdot\|$ denotes the Euclidean norm.

7. Let $\mathbf{X} \sim \mathcal{N}_p(\boldsymbol{\mu}, R = \{\rho\})$ and P_i denote the i th set among all $p!$ permutations of the set $\mathbf{x} = \{x_1, \dots, x_p\}$.

- (a) Prove that $\Phi_p(P_i|\mathbf{0}, R = \{\rho\}) = \Phi_p(P_j|\mathbf{0}, R = \{\rho\})$ for $i = j = 1, \dots, p!$. In special case, when $p = 2$, we have

$$\Phi_2((x_1, x_2)^\top | \mathbf{0}, R = \{\rho\}) = \Phi_2((x_2, x_1)^\top | \mathbf{0}, R = \{\rho\}). \quad (4.379)$$

- (b) Does identity (4.379) hold true if we have $\boldsymbol{\mu} = (\mu_1, \mu_2)^\top = (\mu, \mu)^\top$?

8. Prove property (4.113) of Owen's T function.

9. Write $B(x, y|\rho)$ to denote the function proposed by Owen [1956] as

$$B(x, y|\rho) = \frac{\Phi(x)}{2} + \frac{\Phi(y)}{2} - T(x, a_x) - T(y, a_y) - \begin{cases} 0, & \text{if } xy > 0, \\ \frac{1}{2}, & \text{if } xy = 0 \text{ and } x + y \geq 0 \end{cases}, \quad (4.380)$$

for computing $\Phi_2(\mathbf{x}|\mathbf{0}, \rho)$ where $\mathbf{x} \in \mathbb{R}^2$. Use the following R code for approximating the Owen's $B(x, y|\rho)$ function in (4.380) and compare its performance with functions `BVN_Drez` and `Spec_quad` given in this section for this purpose.

```
1 B_owen <- function(x, y, rho, k)
2 {
3   if( x*y > 0 | (x*y == 0 & x + y >= 0) ){ p1 <- 0 }else{ p1 <- 1/2 }
4   a_x <- y/( x*sqrt(1 - rho^2) ) - rho/sqrt(1 - rho^2)
5   a_y <- x/( y*sqrt(1 - rho^2) ) - rho/sqrt(1 - rho^2)
6   out <- quad_rule(k, type = "LE"); x0 <- out$node; w <- out$weight;
7   T_x <- a_x/(4*pi)*sum( w*exp(-x^2/2*(1 + (a_x*x0)^2))/(1 + (a_x*x0)^2) )
8   T_y <- a_y/(4*pi)*sum( w*exp(-y^2/2*(1 + (a_y*x0)^2))/(1 + (a_y*x0)^2) )
9   pnorm(x)/2 + pnorm(y)/2 - T_x - T_y - p1
10 }
```

10. Let $\mathbf{X} \sim \mathcal{N}_p(\mathbf{0}, \Sigma)$ with PDF $\phi_p(\mathbf{x}|\mathbf{0}, \Sigma)$ that can be represented, using the well-known *inversion formula*, as

$$\phi_p(\mathbf{x}|\mathbf{0}, \Sigma) = \frac{1}{(2\pi)^p} \int \cdots \int \exp\{-i\mathbf{t}^\top \mathbf{x} - \frac{1}{2}\mathbf{t}^\top \Sigma \mathbf{t}\} dt_1 \cdots dt_p, \quad (4.381)$$

where $i^2 = -1$ and $\mathbf{t} \in \mathbb{R}^p$. Verify identity (4.200) by taking derivatives, first with respect to $\rho_{r,c}$ and, second with respect to x_r and x_c from RHS of (4.381) for $r < c = 1, \dots, p$.

11. The random vector $\mathbf{Y} = (Y_1, \dots, Y_p)^\top$ with CF

$$\psi(\mathbf{t}|\Sigma, \boldsymbol{\alpha}, \lambda) = \left(1 + \frac{\mathbf{t}^\top \Sigma \mathbf{t}}{2} - i\boldsymbol{\alpha}^\top \mathbf{t}\right)^{-\lambda},$$

where $i^2 = -1$, $\mathbf{t} \in \mathbb{R}^p$, $\boldsymbol{\alpha} \in \mathbb{R}^p$, $a > 0$, and Σ is $p \times p$ non-negative definite symmetric matrix is called generalized asymmetric Laplace (GAL) distribution Kotz et al. [2012]. In fact each GAL distribution is a GSMM model with representation

$$\mathbf{Y} = \boldsymbol{\mu} + \boldsymbol{\alpha}G + \sqrt{G}\mathbf{X}, \quad (4.382)$$

where $\mathbf{X} \sim \mathcal{N}_p(\mathbf{0}, \Sigma)$ and $G \sim \mathcal{G}(\lambda, 1)$ are independent. Prove that the PDF of $\mathbf{Y}$ becomes

$$f(\mathbf{y}|\Psi) = \frac{2 \exp\left\{(\mathbf{y} - \boldsymbol{\mu})^\top \Sigma^{-1} \boldsymbol{\alpha}\right\}}{(2\pi)^{p/2} |\Sigma|^{1/2} \Gamma(\lambda)} \left(\frac{d_1(\mathbf{y})}{d_2(\mathbf{y})}\right)^{\frac{p}{4} - \frac{\lambda}{2}} K_{\lambda - \frac{p}{2}}(\sqrt{d_1(\mathbf{y})d_2(\mathbf{y})}), \quad (4.383)$$

where $\Psi = (\boldsymbol{\mu}, \boldsymbol{\alpha}, \Sigma, \lambda)$, $d_1(\mathbf{y}) = 2 + \delta(\boldsymbol{\alpha}, \Sigma)$, $d_2(\mathbf{y}) = \delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)$. It may be interesting if in (4.17) we let $\lambda = 1$, $\psi = 2$, and $\chi \rightarrow 0$, then PDFs in (4.383) and (4.17) are equal.

12. The PDF of a univariate SH distribution is

$$f(x|\Psi) = \left[\frac{a + \sigma^{-2}(x - \mu)^2}{a} \right]^{\frac{\lambda-1/2}{2}} \frac{\mathcal{K}_{\lambda-1/2}(\sqrt{a[a + \sigma^{-2}(x - \mu)^2]})}{\sqrt{2\pi}\sigma\mathcal{K}_\lambda(a)}, \quad (4.384)$$

where $\Psi = (\mu, \sigma, \boldsymbol{\theta})$ in which $\boldsymbol{\theta} = (a, \lambda)^\top$. Describe a method for computing its CDF numerically using the Gaussian quadrature rule and write down the pertinent R code.

13. If $G \sim \mathcal{GIG}(a, a, \lambda)$, then it follows from (4.193), (4.194), and the fact $\mathcal{K}_\lambda(x) = \mathcal{K}_{-\lambda}(x)$ that

$$\mathcal{M}_{G^{-1}}(t|\boldsymbol{\theta}) = \left(\frac{a}{a - 2t} \right)^{-\frac{\lambda}{2}} \frac{\mathcal{K}_\lambda(\sqrt{a(a - 2t)})}{\mathcal{K}_\lambda(a)},$$

where $\boldsymbol{\theta} = (a, a, \lambda)^\top$. Based on the above result, use the Corollary 4.4.2 for computing the CDF of bivariate SH distribution with PDF is given in (4.18) numerically using the Gaussian quadrature rule. Write the R code down similar to Example 4-15 for bivariate Student's t distribution.

14. It is well known that if $(X, Y)^\top \sim \mathcal{N}_2(\mathbf{0}, R = \{\rho\})$, then $Y|X = h$ is distributed normally. But, $Y_h = Y|X \leq h$, for $h \in \mathbb{R}$, does not follow a Gaussian distribution.

- (a) Prove that the PDF of Y_h is [Johnson et al., 1972, pp. 112-114]:

$$f(t) = \frac{\phi(t)}{\Phi(h)} \Phi\left[\frac{h - \rho t}{\sqrt{1 - \rho^2}}\right],$$

with mean $\mu_h = -\rho\phi(h)/\Phi(h)$ and variance $\sigma_h^2 = 1 + \rho h\mu - \mu^2$ in which $-1 \leq \rho \leq 1$.

- (b) Taking account into the fact that when ρ is small, $|\rho| \leq 0.5$ say, then the PDF's of Y_h and $Z \sim \mathcal{N}(\mu_h, \sigma_h^2)$ are virtually indistinguishable. Prove that

$$\Phi((x, y)^\top | \mathbf{0}, R = \{\rho\}) \approx \Phi(x)\Phi\left[\frac{y - \mu_h}{\sigma_h}\right]. \quad (4.385)$$

- (c) Mee and Owen [1983] proposes the following three-step algorithm with maximum error less than 0.0008 for approximating $\Phi((c, d) | \mathbf{0}, R = \{\rho\})$ when $|r| \leq 0.5$.

- i. Choose quantities c and d so that $|c| \geq |d|$. Such choices always exist since identity (4.379) states $\Phi((c, d) | \mathbf{0}, R = \{\rho\}) = \Phi((d, c) | \mathbf{0}, R = \{\rho\})$;
- ii. If $c \leq 0$, then set $x = c$, $y = d$, $\rho = r$, and approximate $\Phi((c, d) | \mathbf{0}, R = \{\rho\})$ by (4.385);
- iii. If $c > 0$, then set $x = -c$, $y = d$, $\rho = -r$, and approximate $\Phi((c, d)^\top | \mathbf{0}, R = \{\rho\}) = \Phi(d) - \Phi((-c, d)^\top | \mathbf{0}, R = \{-r\})$ by $\Phi(y) - \Phi((x, y)^\top | \mathbf{0}, R = \{\rho\})$.

Apply the above algorithm for approximating $\Phi((0.2, 1)^\top | \mathbf{0}, R = \{0.7\}) = 0.558416 \dots$ and check that its absolute error is around 0.000233.

15. Apply the first method of Drezner [1994] to show that the orthant probability $L_3(\mathbf{0}|R)$, becomes explicitly

$$L_3(\mathbf{0}|R) = \frac{1}{8} + \frac{1}{4\pi} \sum_{i < j} \sin^{-1}(\rho_{i,j}).$$

16. Let $R = \{\rho_{i,j}\}$ denote the correlation matrix of a 3-dimensional Gaussian distribution.

- (a) Show that $|R| = 1 - \rho_{1,2}^2 - \rho_{1,3}^2 - \rho_{2,3}^2 + 2\rho_{1,2}\rho_{1,3}\rho_{2,3}$.

- (b) Show that $|R| \leq 1$.
- (c) For reference matrix $R_2^* = \{\rho_{i,j}^*\}$ given in (4.220), show that $|\Omega(t)| = 1 - t^2 \rho_{1,2}^2 - t^2 \rho_{1,3}^2 - \rho_{2,3}^2 + 2t^2 \rho_{1,2} \times \rho_{1,3} \times \rho_{2,3}$ where $\Omega(t)$ is given in (4.209).
- (d) Show that if $|\Omega(t)|$ is positive for $t = 1$, then it is positive for $0 \leq t \leq 1$. Hint: show that $\rho_{1,2}^2 + \rho_{1,3}^2 - 2\rho_{1,2} \times \rho_{1,3} \times \rho_{2,3} \geq 0$.

17. Prove Proposition ?? using

- (a) The Jensen inequality.
- (b) The Lagrange multiplier enforcing on function $f(p, q)$ defined as

$$f(p, q, \lambda) = KL(p||q) - \lambda[1 - \int q(x)dx],$$

and then taking partial derivative with respect to q .

18. Does the Kullback-Leibler divergence satisfy three requirements called the axioms of metric space? Why?
19. Using hierarchy (4.295), prove that

$$Z_i|(y_i, \boldsymbol{\theta}) \sim \mathcal{G}\left(\frac{\nu}{2}, \frac{1}{2} \frac{(y_i - \mu)^2}{\sigma^2} + \frac{\nu}{2}\right).$$

4.8 Moments of truncated distributions

Truncated distribution is widely used in different fields of statistics such as, survival analysis, regression analysis, and model-based clustering. The truncated random variable may take any value on region defined by (2.55). If both the lower and upper bounds of region $\mathcal{R}$ are finite, then we have a doubly truncated of random variable. If the lower (upper) bound of region $\mathcal{R}$ is not finite, then we have a right (left) truncated random variable. In other word, the support of doubly, right, and left truncated random variable are, accordingly, (a, b) , $(-\infty, b)$, and (a, ∞) . Write $X_{\mathcal{R}}$ to denote random variable X truncated on region $\mathcal{R} = \{x \in \mathbb{R} | a \leq x \leq b\}$. The PDF of random variable $X_{\mathcal{R}}$ truncated on (a, b) is simply given by

$$f_{X_{\mathcal{R}}}(x) = \frac{f(x)}{\int_{\mathcal{R}} f(u)du}, \quad (4.386)$$

where $x \in \mathcal{R}$. Each truncated random variable $X_{\mathcal{R}}$ is in fact a conditional version of random variable X . In other words

$$X_{\mathcal{R}} = X|X \in \mathcal{R}.$$

Let $F(\cdot)$ denotes the CDF of random variable X . The CDF of $\mathcal{X}$ can be represented in terms of $F(\cdot)$ as follows.

$$F_{X_{\mathcal{R}}}(x) = \frac{F(x) - F(a)}{F(b) - F(a)},$$

where $x \in \mathcal{R}$. Hence, in univariate case, the r th moment of truncated random variable X on (a, b) , for $r = 1, 2, 3, \dots$ is

$$\Omega^r = E(X^r | a < X < b) = \frac{\int_a^b u^r f(u)du}{F(b) - F(a)}, \quad -\infty < a < x < b < \infty. \quad (4.387)$$

In general, let $\mathbf{X}_{\mathcal{R}^p}$ denote the truncated version of p -dimensional random vector $\mathbf{X}$ on region (hyper-cube) $\mathcal{R}^p$ given in (2.55). Then the PDF of $\mathbf{X}_{\mathcal{R}^p}$ is

$$\frac{f(\mathbf{x})}{\int_a^b f(\mathbf{u})d\mathbf{u}} = \frac{f(\mathbf{x})}{\mathcal{P}_F}, \quad (4.388)$$

where $\mathbf{x} \in \mathcal{R}^p$ and $F(\cdot)$ is the CDF of $\mathbf{X}$. Hence, the first and second moments of truncated random vector $\mathbf{X}$, respectively, are

$$\begin{aligned} \boldsymbol{\Omega} &= E(\mathbf{X}_{\mathcal{R}^p}) = E(\mathbf{X} | a_1 \leq X_1 \leq b_1, \dots, a_p \leq X_p \leq b_p), \\ \boldsymbol{\Omega}^2 &= E(\mathbf{X}_{\mathcal{R}^p} \mathbf{X}_{\mathcal{R}^p}^\top) = E(\mathbf{X} \mathbf{X}^\top | a_1 \leq X_1 \leq b_1, \dots, a_p \leq X_p \leq b_p). \end{aligned}$$

Herein, the concern of study mainly is devoted to computing the first and second moments given by since these quantities have received more attention than other moments in practice. If $\mathbf{X} \sim f(\cdot)$, by definition, the first moment of $\mathbf{X}$ is computed as

$$\boldsymbol{\Omega} = \frac{1}{\mathcal{P}_F} \int_a^b \mathbf{u} f(\mathbf{u}) d\mathbf{u}. \quad (4.389)$$

The Monte Carlo approximation of $\boldsymbol{\Omega}$ can be obtained by considering the instrumental distribution with PDFs $f(\cdot)/[F(a) - F(b)]$. Here, we can use the rejection approach for simulating from truncated random variable as follows. Suppose, $x_1, \dots, x_N$, for sufficiently large N , are realizations from instrumental with PDF f_1 , the Monte Carlo approximation of $\boldsymbol{\Omega}$ is obtained as

$$\hat{\boldsymbol{\Omega}} = \frac{1}{N[F(a) - F(b)]} \sum_{i=1}^N f(x_i). \quad (4.390)$$

The pseudo code for sampling from $f(\cdot)/[F(a) - F(b)]$ is given as follows.

1. Read a, b , and N ;
2. Set $j = 0$ and $y = 0$;
3. Simulate $x \sim f(\cdot)$
4. If $a < x < b$, then accept x as a generation from $f(\cdot)/[F(a) - F(b)]$, set $y = y + x$ and $j = j + 1$;
5. If $j = N$, then go to the next step, otherwise return to step 2;
6. Approximate $\boldsymbol{\Omega}$ as $\hat{\boldsymbol{\Omega}} = y/N$;
7. End.

The focus of this chapter is placed on compute the the first and second moments of truncated Gaussian distribution in univariate and multivariate cases. Herein, we proceed to compute the exact values of first and second moments of the truncated univariate and multivariate Gaussian distributions.

4.9 Moments of truncated Gaussian distribution in univariate case

Recall from Example 2-8 that deals with simulating from truncated Gaussian distribution on region given by (2.55), herein we are willing to compute the moments of this family.

4.9.1 First two moments of truncated Gaussian distribution

Let $X \sim \mathcal{N}(\mu, \sigma^2, \mathcal{R})$. By definition, from (4.387) for $r = 1$, it follows that

$$\Omega_1 = E(X) = \frac{1}{\mathcal{P}_G} \int_{\mathcal{R}} x \phi(x|\mu, \sigma^2) dx, \quad (4.391)$$

where $\mathcal{P}_G = \Phi(b|\mu, \sigma^2) - \Phi(a|\mu, \sigma^2)$ and $\mathcal{R} = \{x \in \mathbb{R} | a < x < b\}$. It is easy to check that

$$\frac{\partial}{\partial \mu} \phi(x|\mu, \sigma^2) = \left(\frac{x - \mu}{\sigma^2} \right) \phi(x|\mu, \sigma^2). \quad (4.392)$$

Integrating both sides of (4.392) and dividing by $\mathcal{P}_G$ yields

$$\frac{1}{\mathcal{P}_G} \int_a^b \frac{\partial}{\partial \mu} \phi(x|\mu, \sigma^2) dx = \frac{\Omega}{\sigma^2} - \frac{\mu}{\sigma^2}.$$

It follows that

$$\begin{aligned} \Omega &= \mu + \frac{\sigma^2}{\mathcal{P}_G} \frac{\partial}{\partial \mu} \int_{a-\mu}^{b-\mu} \phi(u|0, \sigma^2) du \\ &= \mu + \frac{\sigma^2}{\mathcal{P}_G} [\phi(a|\mu, \sigma^2) - \phi(b|\mu, \sigma^2)]. \end{aligned}$$

Hence,

$$\Omega = \mu + \sigma^2 \frac{\phi(a|\mu, \sigma^2) - \phi(b|\mu, \sigma^2)}{\Phi(b|\mu, \sigma^2) - \Phi(a|\mu, \sigma^2)}. \quad (4.393)$$

By definition, from (4.387) for $m = 2$, it follows that

$$\Omega^2 = E(X^2) = \frac{1}{\mathcal{P}_G} \int_a^b x^2 \phi(x|\mu, \sigma^2) dx,$$

Taking the second order derivative from (4.392) with respect to μ , we have

$$\frac{\partial^2}{\partial \mu^2} \phi(x|\mu, \sigma^2) = \left(\frac{x - \mu}{\sigma^2} \right)^2 \phi(x|\mu, \sigma^2) - \frac{\phi(x|\mu, \sigma^2)}{\sigma^2}. \quad (4.394)$$

Integrating both sides of (4.394), dividing by $\mathcal{P}_G$, and then rearranging yields

$$\frac{1}{\mathcal{P}_G} \frac{\partial^2}{\partial \mu^2} \int_{a-\mu}^{b-\mu} \phi(x|0, \sigma^2) dx = \frac{1}{\mathcal{P}_G} \int_a^b \left(\frac{x - \mu}{\sigma^2} \right)^2 \phi(x|\mu, \sigma^2) dx - \frac{1}{\sigma^2}. \quad (4.395)$$

It is easy to check that the LHS of (4.395) becomes

$$\frac{1}{\mathcal{P}_G} \frac{\partial^2}{\partial \mu^2} \int_{a-\mu}^{b-\mu} \phi(x|0, \sigma^2) dx = \frac{1}{\mathcal{P}_G} \left[\left(\frac{a - \mu}{\sigma^2} \right) \phi(a|\mu, \sigma^2) - \left(\frac{b - \mu}{\sigma^2} \right) \phi(b|\mu, \sigma^2) \right], \quad (4.396)$$

and further

$$\begin{aligned} \frac{1}{\mathcal{P}_G} \int_a^b \left(\frac{x - \mu}{\sigma^2} \right)^2 \phi(x|\mu, \sigma^2) dx &= \frac{1}{\sigma^2} E \left[\left(\frac{X - \mu}{\sigma} \right)^2 \middle| a < X < b \right] \\ &= \frac{1}{\sigma^4} E(X^2 | a < X < b) - 2 \frac{\mu}{\sigma^4} E(X | a < X < b) + \frac{\mu^2}{\sigma^4}. \end{aligned} \quad (4.397)$$

Hence, substituting the RHS of (4.396) and (4.397) into (4.395) and rearranging, we obtain

$$\Omega^2 = \sigma^2 \frac{(a - \mu)\phi(a|\mu, \sigma^2) - (b - \mu)\phi(b|\mu, \sigma^2)}{\Phi(b|\mu, \sigma^2) - \Phi(a|\mu, \sigma^2)} + \sigma^2 - \mu^2 + 2\mu\Omega.$$

In what follows R function `Ex` is given for computing the first and second moments of truncated univariate Gaussian distribution.

```

1 R> Ex <- function(mu, sigma, a, b)
2 + {
3 +   pb <- pnorm(b, mu, sigma)
4 +   pa <- pnorm(a, mu, sigma)
5 +   da <- dnorm(a, mu, sigma)
6 +   db <- dnorm(b, mu, sigma)
7 +   Omega <- mu + sigma^2*( da - db )/( pb - pa )
8 +   P1 <- pnorm(b, mu, sigma) - pnorm(a, mu, sigma)
9 +   ex <- mu + sigma^2*( da - db )/( pb - pa )
10 +   Omega2 <- sigma^2*( (a-mu)*da - (b-mu)*db )/( pb - pa ) + sigma^2 -mu^2 + 2*mu*ex
11 +   out2 <- list("Ex" = Omega, "Exx" = Omega2)
12 +   return( out2 )
13 + }

```

4.10 First moment of truncated multivariate Gaussian distribution

Recall that $\mathcal{N}_p(\boldsymbol{\mu}, \Sigma, \mathcal{R}^p)$ is a truncated Gaussian distribution truncated on region

$$\mathcal{R}^p = \{\mathbf{x} \in \mathbb{R}^p | \mathbf{a} \leq \mathbf{x} \leq \mathbf{b}\}, \quad (4.398)$$

where $\mathbf{a} = (a_1, \dots, a_p)^\top$ and $\mathbf{b} = (b_1, \dots, b_p)^\top$. Moreover, define

$$\mathbf{v}_i[x] = (v_1, \dots, v_{i-1}, x, v_{i+1}, \dots, v_p), \quad (4.399)$$

$$\mathbf{v}[-i] = (v_1, \dots, v_{i-1}, v_{i+1}, \dots, v_p), \quad (4.400)$$

in which $\mathbf{v} = (v_1, v_2, \dots, v_p)^\top$ is a vector of real values. By definition, we can write

$$\begin{aligned}
\boldsymbol{\Omega} &= \frac{1}{\mathcal{P}_G} \int_a^b \mathbf{x} \times \phi_p(\mathbf{x} | \boldsymbol{\mu}, \Sigma) d\mathbf{x} \\
&= \frac{1}{\mathbf{C}_G \mathcal{P}_G} \int_a^b \mathbf{x} \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\} d\mathbf{x} \\
&= \frac{1}{\mathbf{C}_G \mathcal{P}_G} \int_a^b (\mathbf{x} - \boldsymbol{\mu}) \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\} d\mathbf{x} \\
&\quad + \frac{\boldsymbol{\mu}}{\mathbf{C}_G \mathcal{P}_G} \int_a^b \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\} d\mathbf{x} \\
&= \frac{1}{\mathbf{C}_G \mathcal{P}_G} \int_a^b (\mathbf{x} - \boldsymbol{\mu}) \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\} d\mathbf{x} + \boldsymbol{\mu},
\end{aligned} \quad (4.401)$$

where

$$\mathbf{C}_G = (2\pi)^{\frac{p}{2}} |\Sigma|^{\frac{1}{2}}, \quad (4.402)$$

$$\mathcal{P}_G = P(\mathbf{X} \in \mathcal{R}^p) = \Phi_p(\mathcal{R}^p | \boldsymbol{\mu}, \Sigma) = \frac{1}{\mathbf{C}_G} \int_a^b \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\} d\mathbf{x}, \quad (4.403)$$

and $\delta(\cdot, \cdot)$ is the Mahalanobis distance defined as (4.29). Taking into account the fact that

$$\frac{\partial}{\partial \boldsymbol{\mu}} \frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2} = -\Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}),$$

then $\boldsymbol{\Omega}$ in RHS of (4.401) can be represented as

$$\begin{aligned}
\boldsymbol{\Omega} &= \frac{1}{\mathbf{C}_G \mathcal{P}_G} \int_a^b \Sigma \frac{\partial}{\partial \boldsymbol{\mu}} \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\} d\mathbf{x} + \boldsymbol{\mu} \\
&= \frac{\Sigma}{\mathbf{C}_G \mathcal{P}_G} \frac{\partial}{\partial \boldsymbol{\mu}} \int_{\mathbf{a}-\boldsymbol{\mu}}^{\mathbf{b}-\boldsymbol{\mu}} \exp\left\{-\frac{\delta(\mathbf{u}, \Sigma)}{2}\right\} d\mathbf{u} + \boldsymbol{\mu} \\
&= \Sigma \times \mathbf{I} + \boldsymbol{\mu},
\end{aligned}$$

where the i th element of vector $\mathbf{I} = (I_1, I_2, \dots, I_p)^\top$, for $i = 1, \dots, p$, becomes

$$I_i = \frac{1}{\mathbf{C}_G \mathcal{P}_G} \int_{\mathbf{a}^{[-i]} - \boldsymbol{\mu}^{[-i]}}^{\mathbf{b}^{[-i]} - \boldsymbol{\mu}^{[-i]}} \left[\exp\left\{-\frac{\delta(\mathbf{u}_i[a_i - \mu_i], \Sigma)}{2}\right\} - \exp\left\{-\frac{\delta(\mathbf{u}_i[b_i - \mu_i], \Sigma)}{2}\right\} \right] d\mathbf{u}^{[-i]}. \quad (4.404)$$

We further note that the quadratic form $\delta(\mathbf{u}_i[x], \Sigma)$ in RHS of (4.404) can be represented as

$$\delta(\mathbf{u}_i[x], \Sigma) = x^2 \times (\Sigma_{i,i})^{-1} + \delta(\mathbf{u}^{[-i]} - \boldsymbol{\xi}(x), \Delta_{ii}), \quad (4.405)$$

where

$$\boldsymbol{\xi}(x) = \Sigma_{i,-i} \times (\Sigma_{i,i})^{-1} \times x, \quad (4.406)$$

$$\Delta_{ii} = \Sigma_{-i,-i} - (\Sigma_{i,i})^{-1} \times (\Sigma_{i,-i})^\top \Sigma_{i,-i}, \quad (4.407)$$

in which $\Sigma_{i,j}$ denotes the (i, j) th element of Σ , $\Sigma_{-i,-j}$ denotes the matrix Σ when its i th row and j th column is eliminated, and $\Sigma_{i,-j}$ is the i th row of matrix Σ when its j th column is eliminated, for $i, j = 1, \dots, p$. Hence, using expression (4.405), the integrand in RHS of (4.404) can be rewritten as

$$\begin{aligned} I_i = & \frac{1}{\mathbf{C}_G \mathcal{P}_G} \exp\left\{-\frac{(a_i - \mu_i)^2}{2\Sigma_{i,i}}\right\} \int_{\mathbf{a}^{[-i]} - \boldsymbol{\mu}^{[-i]}}^{\mathbf{b}^{[-i]} - \boldsymbol{\mu}^{[-i]}} \exp\left\{-\frac{1}{2}\delta(\mathbf{u}^{[-i]} - \boldsymbol{\xi}(a_i - \mu_i), \Delta_{ii})\right\} d\mathbf{u}^{[-i]} \\ & - \frac{1}{\mathbf{C}_G \mathcal{P}_G} \exp\left\{-\frac{(b_i - \mu_i)^2}{2\Sigma_{i,i}}\right\} \int_{\mathbf{a}^{[-i]} - \boldsymbol{\mu}^{[-i]}}^{\mathbf{b}^{[-i]} - \boldsymbol{\mu}^{[-i]}} \exp\left\{-\frac{1}{2}\delta(\mathbf{u}^{[-i]} - \boldsymbol{\xi}(b_i - \mu_i), \Delta_{ii})\right\} d\mathbf{u}^{[-i]}, \end{aligned} \quad (4.408)$$

where quantity $\mathbf{C}_G$ in (4.402) can be rewritten as

$$\mathbf{C}_G = (2\pi)^{\frac{1}{2}} |\Sigma_{i,i}|^{\frac{1}{2}} (2\pi)^{\frac{p-1}{2}} |\Delta_{ii}|^{\frac{1}{2}}. \quad (4.409)$$

Once we have computed I_i in (4.408), for $i = 1, \dots, p$, the first moment of truncated Gaussian distribution is then obtained by substituting the constructed vector $\mathbf{I}$ into the RHS of (4.401).

Here, we carry out a small simulation study for computing the first moment of vector $\mathbf{X}_{\mathcal{R}^2} \sim \mathcal{N}_2(\mathbf{0}, \Sigma, \mathcal{R}^2)$ under two scenarios. Under the first scenario, we assume that $\sigma_1^2 = 2, \sigma_2^2 = 1, \rho = 0.3535$, $\mathbf{a} = (-1, -1)^\top$, and $\mathbf{b} = (2, 2)^\top$. Under the second scenario it is assumed that $\sigma_1^2 = 2, \sigma_2^2 = 2, \rho = 0.25$, $\mathbf{a} = (0, 0)^\top$, and $\mathbf{b} = (+\infty, +\infty)^\top$. For both scenarios, we compute $\boldsymbol{\Omega}$ by generating samples of sizes $N = 2000, 5000$, and 20000 based on 5000 runs. The instrumental distribution is set to be $\mathcal{N}_2(\boldsymbol{\mu}, \Sigma)$. For computing $\boldsymbol{\Omega}$ through the $\mathcal{N}_2(\boldsymbol{\mu}, \Sigma)$, one can follow two ways. In the first way the quantity $\boldsymbol{\Omega}$ is estimated as

$$\hat{\boldsymbol{\Omega}} = \frac{1}{N} \sum_{i=1}^N |\mathbf{x}_i| \quad (4.410)$$

where $|\mathbf{x}_i| = (|x_{1i}|, |x_{2i}|)^\top$ where $\mathbf{x}_i$ s come independently from $\mathcal{N}_2(\boldsymbol{\mu}, \Sigma)$, for sufficiently large N . The second method computes $\hat{\boldsymbol{\Omega}}$ based on sample of size N when the lower and upper truncation bounds are $\mathbf{a} = \mathbf{0}$ and $\mathbf{b} = +\infty$, respectively. Details for implementing the second method are given as follows.

1. Read $\mathbf{a} = \mathbf{0}, \mathbf{b} = +\infty, \boldsymbol{\mu}, \Sigma$, and N ;
2. Set $j = 0$ and $\mathbf{y} = (0, 0)^\top$;
3. Simulate $\mathbf{x} = (x_1, x_2)^\top \sim \mathcal{N}_2(\mathbf{0}, \Sigma)$

4. If $a_1 \leq x_1 \leq b_1$ and $a_2 \leq x_2 \leq b_2$, then accept $\mathbf{x}$ as a generation from $\mathcal{N}_2(\mathbf{0}, \Sigma, \mathcal{R}^2)$, set $\mathbf{y} = \mathbf{y} + (x_1, x_2)^\top$ and $j = j + 1$;
5. If $j = N$, then go to the next step, otherwise return to step 2;
6. Compute an approximation of $\mathbf{\Omega} = (\Omega_1, \Omega_2)^\top$ as $\hat{\mathbf{\Omega}} = (y_1/N, y_2/N)^\top$;
7. End.

While the exact value of $\mathbf{\Omega}$ under both scenarios is computed using (4.401), Table 4.16 shows the results of simulation study for approximating $\mathbf{\Omega}$ under both scenarios through the instrumental $\mathcal{N}_2(\boldsymbol{\mu}, \Sigma)$. The PDF of bivariate Gaussian distribution and the corresponding truncated version

Table 4.16: Summary statistics for computing $\mathbf{\Omega}$ through the exact and importance sampling with $\mathcal{N}_2(\mathbf{0}, \Sigma)$ as the instrumental.

Scenario	$\mathbf{a}^\top$	$\mathbf{b}^\top$	N	$\mathbf{\Omega}$	$\hat{\mathbf{\Omega}}$	summary			
						bias	SE	min.	max.
1	(-1,-1)	(2,2)	2000	0.3691	$\hat{\Omega}_1$	-1.7e-04	0.0174	0.3021	0.4440
				0.2603	$\hat{\Omega}_2$	5.9e-05	0.0158	0.2072	0.3158
			5000	0.3691	$\hat{\Omega}_1$	7.5e-06	0.0113	0.3268	0.4071
				0.2603	$\hat{\Omega}_2$	-1.8e-04	0.0101	0.2255	0.2971
2	(0,0)	(∞, ∞)	2000	1.2150	$\hat{\Omega}_1$	-7.8e-05	0.0197	1.1287	1.2802
				1.2150	$\hat{\Omega}_2$	-8.3e-04	0.0196	1.1462	1.2827
			5000	1.2150	$\hat{\Omega}_1$	-7.4e-04	0.0124	1.1690	1.2603
				1.2150	$\hat{\Omega}_2$	-4.7e-04	0.0125	1.1701	1.2774

under two scenarios mentioned above is depicted in Figure 4.11. The corresponding R function for computing $\mathbf{\Omega}$ is given by the following.

```

1 R> library(mvtnorm) # for computing multivariate Gaussian CDF
2 R> a <- c(-1, -1); b <- c(2, 2); Mu <- c(0, 0);
3 R> Sigma <- matrix( c(2, 0.5, 0.5, 2), nrow = length(Mu), ncol = length(Mu), byrow = TRUE)
4 R> EX <- function(Mu, Sigma, a, b)
5 + {
6 +   if( any(a >= b) == TRUE )
7 +     stop(message("lower_bound_must_be_smaller_than_the_upper_bound."))
8 +   if( length(a) != length(Mu) || length(b) != length(Mu) )
9 +     stop(message("truncation_bounds_and_location_parameter_must_be_of_the_same_size."))
10 +   p <- length(Mu)
11 +   threshold <- 6*sqrt( diag(Sigma) )
12 +   a0 <- a - Mu
13 +   b0 <- b - Mu
14 +   for(i in 1:p)
15 +   {
16 +     if( is.infinite( a0[i] ) ) a0[i] <- Mu[i] - threshold[i]
17 +     if( is.infinite( b0[i] ) ) b0[i] <- Mu[i] + threshold[i]
18 +     if( is.infinite( a[i] ) & a0[i] >= b0[i] ) a0[i] <- b0[i] - threshold[i]
19 +     if( is.infinite( b[i] ) & a0[i] >= b0[i] ) b0[i] <- a0[i] + threshold[i]
20 +   }
21 +   Omega <- numeric(p)
22 +   V <- numeric(p)
23 +   if(p == 2)
24 +   {
25 +     for(i in 1:p)
26 +     {
27 +       Mu_ia <- Sigma[i, -i]*a0[i]/Sigma[i, i]
28 +       Mu_ib <- Sigma[i, -i]*b0[i]/Sigma[i, i]

```

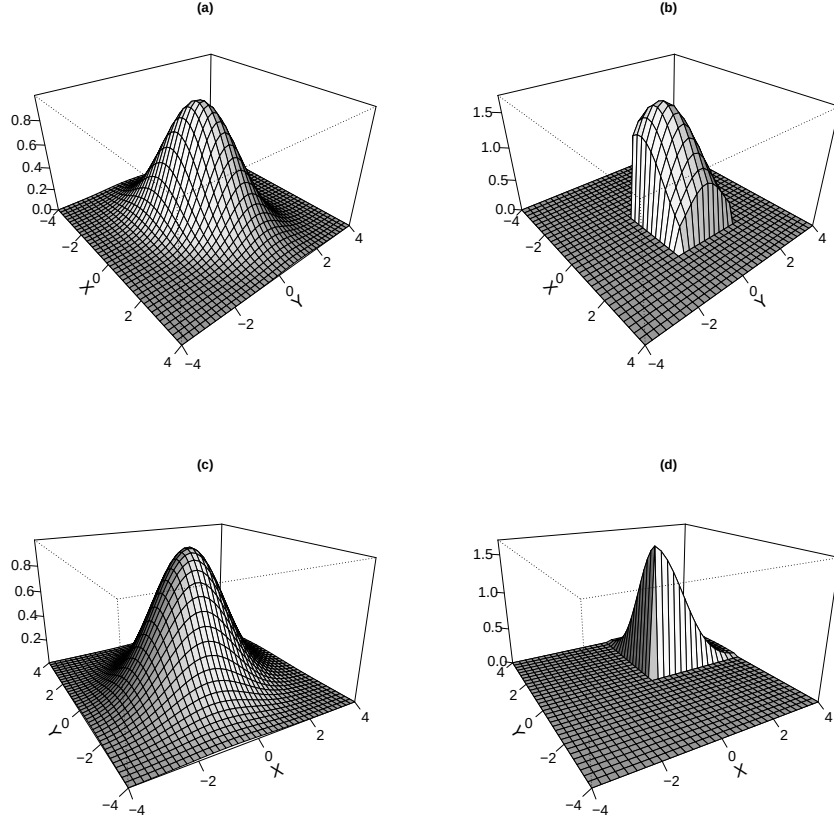


Figure 4.11: (a): The PDF of bivariate Gaussian distribution with parameters $\mu_1 = \mu_2 = 0$, $\sigma_1^2 = 1$, $\sigma_2^2 = 2$, and $\rho = 0.3535$. (b): The PDF of the corresponding truncated bivariate Gaussian distribution on area with lower limits $\mathbf{a} = (-1, -1)^\top$ and upper limits $\mathbf{b} = (2, 2)^\top$. (c) The PDF of bivariate Gaussian distribution with parameters $\mu_1 = \mu_2 = 0$, $\sigma_1^2 = 2$, $\sigma_2^2 = 2$, and $\rho = 0.25$. (b): The PDF of the corresponding truncated bivariate Gaussian distribution on area with lower limits $\mathbf{a} = (0, 0)^\top$ and upper limits $\mathbf{b} = (+\infty, +\infty)^\top$.

```

29 + Delta_i <- sqrt( Sigma[-i, -i] - Sigma[i, -i]^2/Sigma[i, i] )
30 + pai <- pnorm(a0[-i], mean = Mu_ia, sd = Delta_i)
31 + p_ai <- pnorm(a0[-i], mean = Mu_ib, sd = Delta_i)
32 + pbi <- pnorm(b0[-i], mean = Mu_ia, sd = Delta_i)
33 + p_bi <- pnorm(b0[-i], mean = Mu_ib, sd = Delta_i)
34 + V[i] <- dnorm(a0[i], 0, sd = sqrt(Sigma[i, i]))*( pbi - pai ) -
35 + dnorm(b0[i], 0, sd = sqrt(Sigma[i, i]))*( p_bi - p_ai )
36 + }
37 + }else{
38 + for(i in 1:p)
39 + {
40 + Mu_ia <- Sigma[i, -i]*a0[i]/Sigma[i, i]
41 + Mu_ib <- Sigma[i, -i]*b0[i]/Sigma[i, i]
42 + Delta_i <- Sigma[-i, -i] - c( Sigma[i, -i]*t(Sigma[i, -i])/Sigma[i, i] )
43 + p1 <- pmvnorm(a0[-i], b0[-i], mean = Mu_ia, sigma = Delta_i)[1]
44 + p2 <- pmvnorm(a0[-i], b0[-i], mean = Mu_ib, sigma = Delta_i)[1]
45 + V[i] <- dnorm(a0[i], 0, sqrt(Sigma[i, i]))*p1 -
46 + dnorm(b0[i], 0, sqrt(Sigma[i, i]))*p2
47 + }
48 + }
49 + Omega <- Sigma%*%V/pmvnorm(a, b, mean = Mu, sigma = Sigma)[1] + Mu
50 + return(Omega)

```

```

51 + }
52 R> EX(Mu, Sigma, a, b)

```

4.11 Second moment of truncated Gaussian distribution

Let $\mathbf{v} = (v_1, \dots, v_p)^\top$ represent a real-valued vector of length p . Based on vector $\mathbf{v}$, for $i < j$, we define the following notations.

$$\mathbf{v}_{i,j}[x, y] = (v_1, \dots, v_{i-1}, x, v_{i+1}, \dots, v_{j-1}, y, v_{j+1}, \dots, v_p), \quad (4.411)$$

$$\mathbf{v}[-i, -j] = (v_1, \dots, v_{i-1}, v_{i+1}, \dots, v_{j-1}, v_{j+1}, \dots, v_p), \quad (4.412)$$

$$\mathbf{v}[i, j] = (v_i, v_j). \quad (4.413)$$

Let $\mathbf{X} \sim \mathcal{N}_p(\boldsymbol{\mu}, \Sigma, \mathcal{R}^p)$. By definition, the second (raw) moment of truncated Gaussian distribution is given by

$$\Omega^2 = E(\mathbf{X}\mathbf{X}^\top) = \frac{1}{\mathbf{C}_G \mathcal{P}_G} \int_a^b \mathbf{x}\mathbf{x}^\top \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\} d\mathbf{x},$$

where $\mathbf{C}_G$ and $\mathcal{P}_G$ are given by (4.402) and (4.403), respectively. More matrix algebra shows

$$\begin{aligned} \frac{\partial^2}{\partial \boldsymbol{\mu} \partial \boldsymbol{\mu}^\top} \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\} &= \left[\Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} - \Sigma^{-1} \right] \\ &\quad \times \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\}. \end{aligned} \quad (4.414)$$

Integrating both sides of (4.414), then multiplying from left and right by Σ , and finally rearranging, we have

$$\begin{aligned} \Omega^2 &= \Omega \boldsymbol{\mu}^\top + \boldsymbol{\mu} \Omega^\top - \boldsymbol{\mu} \boldsymbol{\mu}^\top + \Sigma + \frac{\Sigma}{\mathbf{C}_G \mathcal{P}_G} \int_a^b \frac{\partial^2}{\partial \boldsymbol{\mu} \partial \boldsymbol{\mu}^\top} \exp\left\{-\frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)}{2}\right\} d\mathbf{x} \Sigma \\ &= \Omega \boldsymbol{\mu}^\top + \boldsymbol{\mu} \Omega^\top - \boldsymbol{\mu} \boldsymbol{\mu}^\top + \Sigma + \frac{\Sigma}{\mathbf{C}_G \mathcal{P}_G} \frac{\partial^2}{\partial \boldsymbol{\mu} \partial \boldsymbol{\mu}^\top} \int_{a-\boldsymbol{\mu}}^{b-\boldsymbol{\mu}} \exp\left\{-\frac{\delta(\mathbf{u}, \Sigma)}{2}\right\} d\mathbf{u} \Sigma \\ &= \Omega \boldsymbol{\mu}^\top + \boldsymbol{\mu} \Omega^\top - \boldsymbol{\mu} \boldsymbol{\mu}^\top + \Sigma + \Sigma \mathbf{I} \Sigma, \end{aligned} \quad (4.415)$$

where

$$\mathbf{I} = \frac{1}{\mathbf{C}_G \mathcal{P}_G} \frac{\partial^2}{\partial \boldsymbol{\mu} \partial \boldsymbol{\mu}^\top} \int_{a-\boldsymbol{\mu}}^{b-\boldsymbol{\mu}} \exp\left\{-\frac{\delta(\mathbf{u}, \Sigma)}{2}\right\} d\mathbf{u}. \quad (4.416)$$

As it may seen, $\mathbf{I}$ is a $p \times p$ square matrix whose (i, j) th element is shown by $I_{i,j}$. In order to compute $I_{i,j}$, we consider three scenarios given by the following.

- **Scenario 1:** $i \neq j = 1, \dots, p$ and $p = 2$
- **Scenario 2:** $i \neq j = 1, \dots, p$ and $p > 2$
- **Scenario 3:** $i = j = 1, \dots, p$

In what follows, we proceed to compute $I_{i,j}$ under three scenarios mentioned above.

- **Scenario 1** ($i \neq j = 1, \dots, p$, and $p = 2$): It follows from (4.416) that

$$I_{1,2} = \frac{1}{\mathbf{C}_G \mathcal{P}_G} \frac{\partial^2}{\partial \mu_1 \partial \mu_2} \int_{a_1 - \mu_1}^{b_1 - \mu_1} \int_{a_2 - \mu_2}^{b_2 - \mu_2} \exp\left\{-\frac{\delta(\mathbf{u}, \Sigma)}{2}\right\} du_1 du_2. \quad (4.417)$$

Applying the Leibniz integral rule for the RHS of (4.417), we have

$$I_{1,2} = \frac{1}{\mathcal{P}_G} \left[\phi_2(\mathbf{b} | \boldsymbol{\mu}, \Sigma) - \phi_2((b_1, a_2)^\top | \boldsymbol{\mu}, \Sigma) + \phi_2(\mathbf{a} | \boldsymbol{\mu}, \Sigma) - \phi_2((a_1, b_2)^\top | \boldsymbol{\mu}, \Sigma) \right].$$

Obviously $I_{2,1} = I_{1,2}$.

- **Scenario 2** ($i \neq j = 1, \dots, p$, and $p > 2$): For a given vector $(x, y)^\top$, it can be seen that the quadratic form $\delta(\mathbf{u}_{i,j}[x, y], \Sigma)$, for which $\mathbf{u}_{i,j}[x, y]$ is defined in (4.411), can be decomposed as

$$\delta(\mathbf{u}_{i,j}[x, y], \Sigma) = \delta_{ij}(x, y) + \delta(\mathbf{u}[-i, -j] - \boldsymbol{\xi}(x, y), \Delta_{ij}), \quad (4.418)$$

where $\delta_{ij}(x, y) = (x, y)^\top \times (\Sigma_{\mathcal{I}, \mathcal{I}})^{-1} \times (x, y)$ for $\mathcal{I} = \{i, j\}$ and

$$\boldsymbol{\xi}(x, y) = \Sigma_{-\mathcal{I}, \mathcal{I}} \times (\Sigma_{\mathcal{I}, \mathcal{I}})^{-1} \times (x, y), \quad (4.419)$$

$$\Delta_{ij} = \Sigma_{-\mathcal{I}, -\mathcal{I}} - (\Sigma_{-\mathcal{I}, \mathcal{I}} \times (\Sigma_{\mathcal{I}, \mathcal{I}})^{-1} \times (\Sigma_{\mathcal{I}, -\mathcal{I}}). \quad (4.420)$$

Herein, $\boldsymbol{\xi}(x, y)$ is a vector of length $p - 2$, Δ_{ij} is a $(p - 2) \times (p - 2)$ positive definite scale matrix, and 2×2 positive definite scale matrix $\Sigma_{\mathcal{I}, \mathcal{I}}$ is given by

$$\Sigma_{\mathcal{I}, \mathcal{I}} = \begin{bmatrix} \Sigma_{i,i} & \Sigma_{i,j} \\ \Sigma_{j,i} & \Sigma_{j,j} \end{bmatrix}.$$

For constructing $(p - 2) \times 2$ matrix $\Sigma_{-\mathcal{I}, \mathcal{I}}$, first construct a two-column matrix based on the i th and j th columns of matrix Σ and then remove i th and j th rows of the constructed two-column matrix. We apply the Leibniz integral rule twice to the RHS of (4.416), and simultaneously utilize the property (4.418), to see that

$$\begin{aligned} I_{i,j} &= \frac{1}{\mathbf{C}_G \mathcal{P}_G} \exp\left\{-\frac{\delta_{ij}(\mathbf{b}[i, j] - \boldsymbol{\mu}[i, j])}{2}\right\} \times I_1(\mathbf{b}[i, j]) \\ &\quad - \frac{1}{\mathbf{C}_G \mathcal{P}_G} \exp\left\{-\frac{\delta_{ij}((\mathbf{a}[i], \mathbf{b}[j]) - \boldsymbol{\mu}[i, j])}{2}\right\} \times I_1(\mathbf{a}[i], \mathbf{b}[j]) \\ &\quad - \frac{1}{\mathbf{C}_G \mathcal{P}_G} \exp\left\{-\frac{\delta_{ij}((\mathbf{b}[i], \mathbf{a}[j]) - \boldsymbol{\mu}[i, j])}{2}\right\} \times I_1(\mathbf{b}[i], \mathbf{a}[j]) \\ &\quad + \frac{1}{\mathbf{C}_G \mathcal{P}_G} \exp\left\{-\frac{\delta_{ij}(\mathbf{a}[i, j] - \boldsymbol{\mu}[i, j])}{2}\right\} \times I_1(\mathbf{a}[i, j]), \end{aligned} \quad (4.421)$$

where

$$I_1(\mathbf{z}) = \int_{\mathbf{a}[-i, -j] - \boldsymbol{\mu}[-i, -j]}^{\mathbf{b}[-i, -j] - \boldsymbol{\mu}[-i, -j]} \exp\left\{-\frac{1}{2} \delta(\mathbf{u}[-i, -j] - \boldsymbol{\xi}(\mathbf{z}), \Delta_{ij})\right\} d\mathbf{u}[-i, -j]. \quad (4.422)$$

We further notice that the quantity $\mathbf{C}_G$ can be represented as

$$\mathbf{C}_G = (2\pi)^{\frac{p}{2}} |\Sigma|^{\frac{1}{2}} = 2\pi |\Sigma_{\mathcal{I}, \mathcal{I}}|^{\frac{1}{2}} (2\pi)^{\frac{p-2}{2}} |\Delta_{ij}|^{\frac{1}{2}}. \quad (4.423)$$

Substituting (4.423) into the RHS of (4.421), it follows that

$$\begin{aligned} I_{i,j} &= \frac{1}{\mathcal{P}_G} \phi_2(\mathbf{b}[i, j] | \boldsymbol{\mu}[i, j], \Sigma_{\mathcal{I}, \mathcal{I}}) \times I_1(\mathbf{b}[i, j]) \\ &\quad - \frac{1}{\mathcal{P}_G} \phi_2((\mathbf{a}[i], \mathbf{b}[j]) | \boldsymbol{\mu}[i, j], \Sigma_{\mathcal{I}, \mathcal{I}}) \times I_1(\mathbf{a}[i], \mathbf{b}[j]) \\ &\quad - \frac{1}{\mathcal{P}_G} \phi_2((\mathbf{b}[i], \mathbf{a}[j]) | \boldsymbol{\mu}[i, j], \Sigma_{\mathcal{I}, \mathcal{I}}) \times I_1(\mathbf{b}[i], \mathbf{a}[j]) \\ &\quad + \frac{1}{\mathcal{P}_G} \phi_2(\mathbf{a}[i, j] | \boldsymbol{\mu}[i, j], \Sigma_{\mathcal{I}, \mathcal{I}}) \times I_1(\mathbf{a}[i, j]). \end{aligned} \quad (4.424)$$

The matrix $\mathbf{I}$ is constructed by computing $I_{i,j}$ in (4.424) for all $i \neq j = 1, \dots, p$. The second moment of truncated Gaussian distribution is then obtained by substituting $\mathbf{I}$ into the RHS of (4.415).

- **Scenario 3** ($i = j = 1, \dots, p$): Using the Leibniz integral rule, the first order derivative of the RHS of (4.416) is

$$\begin{aligned}\frac{\partial}{\partial \mu_i} \mathbf{I} &= \frac{1}{\mathbf{C}_G \mathcal{P}_G} \frac{\partial}{\partial \mu_i} \int_{\mathbf{a}-\boldsymbol{\mu}}^{\mathbf{b}-\boldsymbol{\mu}} \exp\left\{-\frac{\delta(\mathbf{u}, \Sigma)}{2}\right\} d\mathbf{u} \\ &= \frac{1}{\mathbf{C}_G \mathcal{P}_G} \int_{\mathbf{a}[-i]-\boldsymbol{\mu}[-i]}^{\mathbf{b}[-i]-\boldsymbol{\mu}[-i]} \left[\exp\left\{-\frac{\delta(\mathbf{u}_i[a_i - \mu_i], \Sigma)}{2}\right\} \right. \\ &\quad \left. - \exp\left\{-\frac{\delta(\mathbf{u}_i[b_i - \mu_i], \Sigma)}{2}\right\} \right] d\mathbf{u}[-i],\end{aligned}\quad (4.425)$$

for $i = 1, \dots, p$. We note that the quadratic form $\delta(\mathbf{u}_i[x], \Sigma)$, for which $\mathbf{u}_i[x]$ is defined in (4.411), can be rewritten as

$$\delta(\mathbf{u}_i[x], \Sigma) = \delta_{ii}(x, x) + \delta(\mathbf{u}[-i] - \boldsymbol{\xi}(x), \Delta_{ii}), \quad (4.426)$$

where $\delta_{ii}(x, x) = (x_i - \mu_i)^2 \times (\Sigma_{i,i})^{-1}$ is a scalar and quantities $\boldsymbol{\xi}(\cdot)$ and Δ_{ii} are given in (4.406) and (4.407), respectively. We note that $\boldsymbol{\xi}(x)$ is a vector of length $p-1$ and Δ_{ii} is a $(p-1) \times (p-1)$ positive definite scale matrix. We further notice that the quantity $\mathbf{C}_G$ is given by 4.409). Using information given in (4.426), (4.406), (4.407), and (4.402), the RHS of (4.425) can be represented as

$$\begin{aligned}\frac{\partial}{\partial \mu_i} \mathbf{I} &= \frac{1}{\mathcal{P}_G} \phi(a_i | \mu_i, \Sigma_{i,i}) \times \boldsymbol{\Phi}_{p-1}(\mathcal{R}^{p-1} | \boldsymbol{\xi}(a_i - \mu_i), \Delta_{ii}) \\ &\quad - \frac{1}{\mathcal{P}_G} \phi(b_i | \mu_i, \Sigma_{i,i}) \times \boldsymbol{\Phi}_{p-1}(\mathcal{R}^{p-1} | \boldsymbol{\xi}(b_i - \mu_i), \Delta_{ii}) \\ &= I_{ia} - I_{ib},\end{aligned}\quad (4.427)$$

where

$$\mathcal{R}^{p-1} = \{\mathbf{x}[-i] \in \mathbb{R}^{p-1} | \mathbf{a}[-i] - \boldsymbol{\mu}[-i] \leq \mathbf{x}[-i] \leq \mathbf{b}[-i] - \boldsymbol{\mu}[-i]\} \quad (4.428)$$

Now, taking the second partial derivative with respect to μ_i from RHS of (4.427), we obtain

$$\begin{aligned}\frac{\partial I_{ia}}{\partial \mu_i} &= \frac{(a_i - \mu_i)}{\Sigma_{i,i} \times \mathcal{P}_G} \phi(a_i | \mu_i, \Sigma_{i,i}) \times \boldsymbol{\Phi}_{p-1}(\mathcal{R}^{p-1} | \boldsymbol{\xi}(a_i - \mu_i), \Delta_{ii}) \\ &\quad - \frac{\phi(a_i | \mu_i, \Sigma_{i,i})}{\mathcal{P}_G} \times \frac{\Sigma_{-i,i}}{\Sigma_{i,i}} \times \int_{\mathbf{a}[-i]-\boldsymbol{\mu}[-i]}^{\mathbf{b}[-i]-\boldsymbol{\mu}[-i]} \Delta_{ii}^{-1} [\mathbf{u}[-i] - \boldsymbol{\xi}(a_i - \mu_i)] \\ &\quad \times \exp\left\{-\frac{\delta(\mathbf{u}[-i] - \boldsymbol{\xi}(a_i - \mu_i), \Delta_{ii})}{2}\right\} d\mathbf{u}[-i].\end{aligned}\quad (4.429)$$

Clearly, the RHS of (4.429) can be represented in terms of the first moment of a truncated Gaussian distribution. That means

$$\begin{aligned}\frac{\partial I_{ia}}{\partial \mu_i} &= \frac{1}{\Sigma_{i,i} \times \mathcal{P}_G} \phi(a_i | \mu_i, \Sigma_{i,i}) \times \boldsymbol{\Phi}_{p-1}(\mathcal{R}^{p-1} | \boldsymbol{\xi}(a_i - \mu_i), \Delta_{ii}) \\ &\quad \times \left[a_i - \mu_i - \Sigma_{-i,i} (\Delta_{ii})^{-1} \boldsymbol{\Omega}_a \right],\end{aligned}\quad (4.430)$$

where $\mathcal{R}^{p-1}$ is given by (4.428) and $\boldsymbol{\Omega}_a = E(\mathbf{Y}_a)$ where $\mathbf{Y}_a \sim \mathcal{N}_{p-1}(\mathbf{0}, \Delta_{ii}, \mathcal{R}_a^{p-1})$ in which

$$\mathcal{R}_a^{p-1} = \{\mathbf{y} \in \mathbb{R}^{p-1} | \mathbf{a}[-i] - \boldsymbol{\mu}[-i] - \boldsymbol{\xi}(a_i - \mu_i) \leq \mathbf{y} \leq \mathbf{b}[-i] - \boldsymbol{\mu}[-i] - \boldsymbol{\xi}(a_i - \mu_i)\}.$$

For computing $\partial I_{ib}/\partial \mu_i$, we likewise have

$$\begin{aligned} \frac{\partial I_{ib}}{\partial \mu_i} = & \frac{1}{\Sigma_{i,i} \times \mathcal{P}_G} \phi(b_i | \mu_i, \Sigma_{i,i}) \times \Phi_{p-1}(\mathcal{R}^{p-1} | \xi(b_i - \mu_i), \Delta_{ii}) \\ & \times [b_i - \mu_i - \Sigma_{-i,i}(\Delta_{ii})^{-1} \Omega_b], \end{aligned} \quad (4.431)$$

where $\mathcal{R}^{p-1}$ is given by (4.428) and $\Omega_b = E(\mathbf{Y}_b)$ where $\mathbf{Y}_b \sim \mathcal{N}_{p-1}(\mathbf{0}, \Delta_{ii}, \mathcal{R}_b^{p-1})$ in which

$$\mathcal{R}_b^{p-1} = \{\mathbf{y} \in \mathbb{R}^{p-1} | \mathbf{a}[-i] - \boldsymbol{\mu}[-i] - \boldsymbol{\xi}(b_i - \mu_i) \leq \mathbf{y} \leq \mathbf{b}[-i] - \boldsymbol{\mu}[-i] - \boldsymbol{\xi}(b_i - \mu_i)\}.$$

Once we have computed $\partial I_{ia}/\partial \mu_i$ in (4.430) and $\partial I_{ib}/\partial \mu_i$ in (4.431), for $i = 1, \dots, p$, then the diagonal elements of $\mathbf{I}$ given by (4.415) are available.

Once we have construct matrix $\mathbf{I}$, we can compute Ω^2 by substituting computed $\mathbf{I}$ at the RHS of (4.415).

Herein, we carry out a small simulation study for computing the first moment of vector $\mathbf{X} = (X_1, X_2)^\top$ under two scenarios given by the following.

- i. we suppose that $\mathbf{X} \sim \mathcal{N}_2(\boldsymbol{\mu}, \Sigma, \mathcal{R}^2)$ in which $\boldsymbol{\mu} = \mathbf{0}$, $\Sigma = [(2, 0.5)^\top, (0.5, 1)^\top]$ truncated on region $\mathcal{R}^2$ whose lower and upper bounds are $\mathbf{a} = (-1, -1)^\top$ and $\mathbf{b} = (2, 2)^\top$, respectively.
- ii. we suppose that $(X, Y)^\top$ follows truncated bivariate Gaussian distribution with parameters $\boldsymbol{\mu} = \mathbf{0}$, $\Sigma = [(2, 0.5)^\top, (0.5, 2)^\top]$ truncated on region $\mathcal{R}^2$ whose lower and upper bounds are $\mathbf{a} = (0, 0)^\top$ and $\mathbf{b} = (+\infty, +\infty)^\top$, respectively.

We note that the concern of the second scenario is placed on computing the absolute moment, that is $\Omega = E(\mathbf{X}_{\mathcal{R}^2}) = E(|\mathbf{X}|)$. For both scenarios, we would compute Ω by generating samples of sizes $N = 2000, 5000$, and 20000 based on 5000 runs. The instrumental distribution is set to be $\mathcal{N}_2(\boldsymbol{\mu}, \Sigma)$. To this end, one may follow on of two following ways. First, the quantity Ω is estimated as

$$\hat{\Omega} = \frac{1}{N} \sum_{i=1}^N |\mathbf{x}_i| \quad (4.432)$$

where $|\mathbf{x}_i| = (|x_{1i}|, |x_{2i}|)^\top$ where $\mathbf{x}_i$ s come independently from $\mathcal{N}_2(\boldsymbol{\mu}, \Sigma)$, for sufficiently large N . Second, Ω is estimated based on sample of size N when the lower and upper truncation bounds are $\mathbf{a} = \mathbf{0}$ and $\mathbf{b} = +\infty$, respectively. Details for implementing the second method are given as follows. The corresponding R function `EXX` for computing Ω is given by the following.

```

1 R> library(mvtnorm) # for computing multivariate Gaussian CDF
2 R> EXX <- function(Mu, Sigma, a, b)
3 +{
4 + if(any(a >= b) == TRUE) stop(message("vector a must be smaller than vector b."))
5 + p <- length(Mu)
6 + PG <- pmvnorm(lower = a, upper = b, mean = Mu, sigma = Sigma)[1]
7 + IO <- matrix(0, nrow = p, ncol = p)
8 + threshold <- 6*sqrt(diag(Sigma))
9 + a0 <- a - Mu
10 + b0 <- b - Mu
11 + for(i in 1:p)
12 + {
13 +   if(is.infinite(a0[i])) a0[i] <- Mu[i] - threshold[i]
14 +   if(is.infinite(b0[i])) b0[i] <- Mu[i] + threshold[i]
15 +   if(is.infinite(a[i]) & a0[i] >= b0[i]) a0[i] <- b0[i] - threshold[i]
16 +   if(is.infinite(b[i]) & a0[i] >= b0[i]) b0[i] <- a0[i] + threshold[i]
17 + }

```

```

18 + if( p == 2 )
19 + {
20 +   for(i in 1:p)
21 +   {
22 +     Eta_ia <- Sigma[i, -i]*a0[i]/Sigma[i, i]
23 +     Eta_ib <- Sigma[i, -i]*b0[i]/Sigma[i, i]
24 +     Delta_i <- Sigma[-i, -i] - Sigma[i, -i]*Sigma[i, -i]/Sigma[i, i]
25 +     La_i <- a0[-i] - Eta_ia
26 +     Ua_i <- b0[-i] - Eta_ia
27 +     Lb_i <- a0[-i] - Eta_ib
28 +     Ub_i <- b0[-i] - Eta_ib
29 +     PG_ia <- pnorm( Ua_i, mean = rep(0, p-1), sd = sqrt( Delta_i ) )-
30 +     pnorm( La_i, mean = rep(0, p-1), sd = sqrt( Delta_i ) )
31 +     PG_ib <- pnorm( Ub_i, mean = rep(0, p-1), sd = sqrt( Delta_i ) )-
32 +     pnorm( Lb_i, mean = rep(0, p-1), sd = sqrt( Delta_i ) )
33 +     Ex_ia <- Delta_i*( dnorm(La_i, 0, sqrt(Delta_i)) -
34 +     dnorm(Ua_i, 0, sqrt(Delta_i)) )/PG_ia
35 +     Ex_ib <- Delta_i*( dnorm(Lb_i, 0, sqrt(Delta_i)) -
36 +     dnorm(Ub_i, 0, sqrt(Delta_i)) )/PG_ib
37 +     p1 <- a0[i]*dnorm(a0[i], 0, sqrt(Sigma[i, i]))*PG_ia-
38 +     PG_ia*dnorm(a0[i], 0, sqrt(Sigma[i, i]))*Sigma[i, -i]*Ex_ia/Delta_i
39 +     p2 <- b0[i]*dnorm(b0[i], 0, sqrt(Sigma[i, i]))*PG_ib -
40 +     PG_ib*dnorm(b0[i], 0, sqrt(Sigma[i, i]))*Sigma[i, -i]*Ex_ib/Delta_i
41 +     p1 <- ifelse( is.na(p1) | is.nan(p1) | is.infinite(p1), 0, p1)
42 +     p2 <- ifelse( is.na(p2) | is.nan(p2) | is.infinite(p2), 0, p2)
43 +     IO[i, i] <- (p1 - p2)/(PG*Sigma[i, i])
44 +   }
45 +   IO[1, 2] <- ( dmvnorm(b0, mean = rep(0, p), sigma = Sigma )-
46 +   dmvnorm(c(a0[1], b0[2]), mean = rep(0, p), sigma = Sigma )-
47 +   dmvnorm(c(b0[1], a0[2]), mean = rep(0, p), sigma = Sigma )+
48 +   dmvnorm(a0, mean = rep(0, p), sigma = Sigma ) )/PG
49 +   IO[2, 1] <- IO[1, 2]
50 + }
51 + if( p == 3 )
52 + {
53 +   for(i in 1:(p - 1))
54 +   {
55 +     for(j in (i + 1):p)
56 +     {
57 +       I_ij <- c(i, j)
58 +       Sigma_ij <- Sigma[I_ij, I_ij]
59 +       V0 <- c( Sigma[I_ij, -I_ij]%solve(Sigma_ij) )
60 +       Mu_aa <- (V0%*c(a0[i], a0[j]))[1]
61 +       Mu_ab <- (V0%*c(a0[i], b0[j]))[1]
62 +       Mu_ba <- (V0%*c(b0[i], a0[j]))[1]
63 +       Mu_bb <- (V0%*c(b0[i], b0[j]))[1]
64 +       Delta_ij <- Sigma[-I_ij, -I_ij] - ( V0%*Sigma[-I_ij, I_ij] ) [1]
65 +       p1 <- dmvnorm(c(b0[i], b0[j]), mean = rep(0, 2), sigma = Sigma_ij)*(
66 + pnorm(b[-I_ij] - Mu[-I_ij], mean = Mu_bb, sd = sqrt( Delta_ij ) )-
67 + pnorm(a[-I_ij] - Mu[-I_ij], mean = Mu_bb, sd = sqrt( Delta_ij ) ) )
68 +       p2 <- dmvnorm(c(a0[i], b0[j]), mean = rep(0, 2), sigma = Sigma_ij)*(
69 + pnorm(b[-I_ij] - Mu[-I_ij], mean = Mu_ab, sd = sqrt( Delta_ij ) )-
70 + pnorm(a[-I_ij] - Mu[-I_ij], mean = Mu_ab, sd = sqrt( Delta_ij ) ) )
71 +       p3 <- dmvnorm(c(b0[i], a0[j]), mean = rep(0, 2), sigma = Sigma_ij)*(
72 + pnorm(b[-I_ij] - Mu[-I_ij], mean = Mu_ba, sd = sqrt( Delta_ij ) )-
73 + pnorm(a[-I_ij] - Mu[-I_ij], mean = Mu_ba, sd = sqrt( Delta_ij ) ) )
74 +       p4 <- dmvnorm(c(a0[i], a0[j]), mean = rep(0, 2), sigma = Sigma_ij)*(
75 + pnorm(b[-I_ij] - Mu[-I_ij], mean = Mu_aa, sd = sqrt( Delta_ij ) )-
76 + pnorm(a[-I_ij] - Mu[-I_ij], mean = Mu_aa, sd = sqrt( Delta_ij ) ) )
77 +       IO[i, j] <- (p1 - p2 - p3 + p4)/PG
78 +       IO[j, i] <- IO[i, j]
79 +     }
80 +   }

```

```

81 + for(i in 1:p)
82 + {
83 +   Mu_ia <- Sigma[i, -i]*a0[i]/Sigma[i, i]
84 +   Mu_ib <- Sigma[i, -i]*b0[i]/Sigma[i, i]
85 +   Delta_i <- Sigma[-i, -i] - Sigma[i, -i]**t(Sigma[i, -i])/Sigma[i, i]
86 +   La_i <- a0[-i] - Mu_ia
87 +   Ua_i <- b0[-i] - Mu_ia
88 +   Lb_i <- a0[-i] - Mu_ib
89 +   Ub_i <- b0[-i] - Mu_ib
90 +   PG_ia <- pmvnorm(lower=La_i, upper=Ua_i, mean=rep(0, p-1), sigma=Delta_i)[1]
91 +   PG_ib <- pmvnorm(lower=Lb_i, upper=Ub_i, mean=rep(0, p-1), sigma=Delta_i)[1]
92 +   p5 <- a0[i]*dnorm(a0[i],0,sqrt(Sigma[i, i]))*
93 +   pmvnorm(a0[-i], b0[-i], mean = Mu_ia, sigma = Delta_i)[1]-
94 +   PG_ia*dnorm(a0[i],0,sqrt(Sigma[i, i]))*(t(Sigma[i, -i])**
95 +   solve(Delta_i)**EX(rep(0,p-1), Delta_i, La_i, Ua_i))[1]
96 +   p6 <- b0[i]*dnorm(b[i]-Mu[i],0,sqrt(Sigma[i, i]))*
97 +   pmvnorm(a0[-i], b0[-i], mean = Mu_ib, sigma = Delta_i)[1]-
98 +   PG_ib*dnorm(b0[i],0, sqrt(Sigma[i, i]))*(t(Sigma[i, -i])**
99 +   solve(Delta_i)**EX(rep(0,p-1), Delta_i, Lb_i, Ub_i))[1]
100 +   p5 <- ifelse( is.nan(p5) | is.na(p5), 0, p5 )
101 +   p6 <- ifelse( is.nan(p6) | is.na(p6), 0, p6 )
102 +   IO[i, i] <- (p5 - p6)/(PG*Sigma[i, i])
103 + }
104 + }
105 + if ( p > 3 )
106 + {
107 +   for(i in 1:(p - 1))
108 +   {
109 +     for(j in (i + 1):p)
110 +     {
111 +       I_ij <- c(i, j)
112 +       Sigma_ij <- Sigma[I_ij, I_ij]
113 +       V0 <- Sigma[-I_ij, I_ij]**solve(Sigma_ij)
114 +       Mu_aa <- c( V0**c(a0[i], a0[j])) )
115 +       Mu_ab <- c( V0**c(a0[i], b0[j])) )
116 +       Mu_ba <- c( V0**c(b0[i], a0[j])) )
117 +       Mu_bb <- c( V0**c(b0[i], b0[j])) )
118 +       Delta_ij <- Sigma[-I_ij, -I_ij] - V0**Sigma[I_ij, -I_ij]
119 +       p1 <- dmnorm(c(b0[i], b0[j]), mean = rep(0, 2), sigma = Sigma_ij)*
120 +       pmvnorm(a0[-I_ij], b0[-I_ij], mean = Mu_bb, sigma = Delta_ij )
121 +       p2 <- dmnorm(c(a0[i], b0[j]), mean = rep(0, 2), sigma = Sigma_ij)*
122 +       pmvnorm(a0[-I_ij], b0[-I_ij], mean = Mu_ab, sigma = Delta_ij )
123 +       p3 <- dmnorm(c(b0[i], a0[j]), mean = rep(0, 2), sigma = Sigma_ij)*
124 +       pmvnorm(a0[-I_ij], b0[-I_ij], mean = Mu_ba, sigma = Delta_ij )
125 +       p4 <- dmnorm(c(a0[i], a0[j]), mean = rep(0, 2), sigma = Sigma_ij)*
126 +       pmvnorm(a0[-I_ij], b0[-I_ij], mean = Mu_aa, sigma = Delta_ij )
127 +       IO[i, j] <- (p1[1] - p2[1] - p3[1] + p4[1])/PG
128 +       IO[j, i] <- IO[i, j]
129 +     }
130 +   }
131 +   for(i in 1:p)
132 +   {
133 +     Mu_ia <- Sigma[i, -i]*a0[i]/Sigma[i, i]
134 +     Mu_ib <- Sigma[i, -i]*b0[i]/Sigma[i, i]
135 +     Delta_i <- Sigma[-i, -i] - Sigma[i, -i]**t(Sigma[i, -i])/Sigma[i, i]
136 +     La_i <- a0[-i] - Mu_ia
137 +     Ua_i <- b0[-i] - Mu_ia
138 +     Lb_i <- a0[-i] - Mu_ib
139 +     Ub_i <- b0[-i] - Mu_ib
140 +     PG_ia <- pmvnorm(lower=La_i, upper=Ua_i, mean=rep(0, p-1), sigma=Delta_i)[1]
141 +     PG_ib <- pmvnorm(lower=Lb_i, upper=Ub_i, mean=rep(0, p-1), sigma=Delta_i)[1]
142 +     p51 <- a0[i]*dnorm(a0[i], 0, sqrt(Sigma[i, i]))*
143 +     pmvnorm(a0[-i], b0[-i], mean=Mu_ia, sigma=Delta_i)[1]

```

```

144 + p52 <- PG_ia*dnorm(a0[i], 0, sqrt(Sigma[i, i]))*(t(Sigma[i, -i]))**%
145 + solve(Delta_i)**%EX( rep(0, p - 1), Delta_i, La_i, Ua_i ) [1]
146 + p61 <- b0[i]*dnorm(b0[i], 0, sqrt(Sigma[i, i]))*
147 + pmvnorm(a0[-i], b0[-i], mean=Mu_ib, sigma=Delta_i) [1]
148 + p62 <- PG_ib*dnorm(b0[i], 0, sqrt(Sigma[i, i]))*(t(Sigma[i, -i]))**%
149 + solve(Delta_i)**%EX(rep(0, p - 1), Delta_i, Lb_i, Ub_i ) [1]
150 + p51 <- ifelse( is.nan(p51) | is.na(p51), 0, p51 )
151 + p52 <- ifelse( is.nan(p52) | is.na(p52), 0, p52 )
152 + p61 <- ifelse( is.nan(p61) | is.na(p61), 0, p61 )
153 + p62 <- ifelse( is.nan(p62) | is.na(p62), 0, p62 )
154 + IO[i, i] <- (p51 - p52 - p61 + p62)/(PG*Sigma[i, i])
155 + }
156 + }
157 + Omega <- EX(Mu, Sigma, a, b)
158 + EXX <- Omega**%Mu + t(Omega**%Mu) - Mu**%t(Mu) + Sigma + Sigma**%IO**%Sigma
159 + return(EXX)
160 + }

```

4.12 Truncated skew Gaussian distribution

Recall representation (4.6) for $\mathbf{X} \sim \text{CFUSG}_{p,q}(\boldsymbol{\mu}, \Sigma, \Lambda)$. Alternatively, $\mathbf{X}$ can be represented as [Lee and McLachlan, 2016a, Maleki et al., 2019, Morales et al., 2022]:

$$\mathbf{X} \stackrel{d}{=} \boldsymbol{\mu} + (\mathbf{W} | \mathbf{W}_0 > \mathbf{0}), \quad (4.433)$$

where

$$\mathbf{Y} = \begin{bmatrix} \mathbf{W}_0 \\ \mathbf{W}_1 \end{bmatrix} \sim \mathcal{N}_{q+p} \left(\begin{bmatrix} \mathbf{0}_q \\ \mathbf{0}_p \end{bmatrix}, \begin{bmatrix} \mathbf{I}_q & \boldsymbol{\Lambda}^\top \\ \boldsymbol{\Lambda} & \Sigma + \boldsymbol{\Lambda} \boldsymbol{\Lambda}^\top \end{bmatrix} \right). \quad (4.434)$$

By the property (4.434), we can construct a method for simulating from truncated CFUSG distribution

$$\mathbf{X}_{\mathcal{R}^p} \stackrel{d}{=} \mathbf{X} | \mathbf{a} \leq \mathbf{X} \leq \mathbf{b},$$

where $\mathbf{X}$ admits relation (4.6). To this end, we need to simulate from $(p + q)$ -dimensional Gaussian random vector $\mathbf{Y}$ given by (4.434). Some algebra show

$$\mathbf{X}_{\mathcal{R}^p} \stackrel{d}{=} \boldsymbol{\mu}^* + \mathbf{Y} | (\mathbf{a}^* - \boldsymbol{\mu}^* \leq \mathbf{Y} \leq \mathbf{b}^* - \boldsymbol{\mu}^*),$$

where $\mathbf{a}^* = (\mathbf{0}_q^\top, \mathbf{a}^\top)^\top$, $\mathbf{b}^* = (\boldsymbol{\infty}_q^\top, \mathbf{b}^\top)^\top$, and $\boldsymbol{\mu}^* = (\mathbf{0}_q^\top, \boldsymbol{\mu}^\top)^\top$ in which $\boldsymbol{\infty}_q$ is a vector of length q of infinities. Hence, using the methodology given in Sections 4.10 and 4.11 for computing the first and second moments of multivariate Gaussian distribution, we can compute the first and second moments of truncated skew Gaussian distribution by computing $E(\mathbf{Y} | \mathbf{a}^* \leq \mathbf{Y} \leq \mathbf{b}^*)$ and $E(\mathbf{Y} \mathbf{Y}^\top | \mathbf{a}^* \leq \mathbf{Y} \leq \mathbf{b}^*)$, respectively. We note that an R package called MomTrunc uploaded at <https://cran.r-project.org/web/packages/MomTrunc/index.html> developed for this aim [Morales et al., 2022]. The following R code can be used for computing first two moments of the truncated random vector $\mathbf{X}$ where $\mathbf{X} \sim \text{SG}_{2,2}(\boldsymbol{\mu}, \Sigma, \Lambda)$ truncated on $(\mathbf{a}, \mathbf{b})$ in which $\boldsymbol{\mu} = (0, 0)^\top$, $\mathbf{a} = (-3, -3)^\top$, $\mathbf{b} = (2, 2)^\top$,

$$\Sigma = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 2 \end{bmatrix}, \Lambda = \begin{bmatrix} 4 & 2 \\ 2 & -5 \end{bmatrix}.$$

```

1 library("MomTrunc")
2 a <- c(-3, -3); b <- c(2, 2); Mu <- c(0, 0); p <- 2; q<- 2
3 a_star <- c(0, 0, a); b_star <- c(Inf, Inf, b); Mu_star <- c(0, 0, Mu)
4 Sigma <- matrix( c(2, 0.5, 0.5, 2), nrow = p , ncol = p)
5 ch <- t(chol(Sigma)); Lambda <- matrix( c(4, 2 ,2, -5 ), nrow = p , ncol = q)
6 p1 <- diag(q); p2 <- t(Lambda); p3 <- t(p2); p4 <- Sigma + p3%*%t(p3)
7 Sigma_star <- matrix(
8   c(p1[1,1],p1[1,2], p2[1,1], p2[1,2],
9     p1[2,1],p1[2,2], p2[2,1], p2[2,2],
10    p3[1,1],p3[1,2], p4[1,1], p4[1,2],
11    p3[2,1],p3[2,2], p4[2,1], p4[2,2]), nrow = p + q, ncol = p + q)
12 out <- MCmeanvarTMD(a_star, b_star, rep(0, p + q), Sigma_star, dist = "normal")
13 out$mean[(p+1):(p+q)]
14 out$EYY[(p+1):(p+q), (p+1):(p+q)]

```

4.13 Moment of truncated Student's t distribution

Let $\mathbf{X} = (X_1, \dots, X_p)^\top \sim t_\nu(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with PDF given by (4.28). Recall from Definition 4.0.5 that each Student's t is a Gaussian scale mixture model for $\eta(G) = G$ where $G \sim \mathcal{G}(\nu/2, \nu/2)$. By definition

$$E(\mathbf{X}) = \int_{-\infty}^{\infty} \frac{\mathbf{x} \Gamma(\frac{\nu+p}{2})}{(\nu\pi)^{\frac{p}{2}} \Gamma(\frac{\nu}{2}) |\boldsymbol{\Sigma}|^{\frac{1}{2}}} \left[1 + \frac{\delta(\mathbf{x} - \boldsymbol{\mu}, \boldsymbol{\Sigma})}{\nu} \right]^{-\frac{\nu+p}{2}} d\mathbf{x}, \quad \nu > 1. \quad (4.435)$$

Suppose $\mathbf{t}_\nu(\boldsymbol{\mu}, \boldsymbol{\Sigma}, \mathcal{R}^p)$ accounts for the family of the truncated Student's t distributions on region $\mathcal{R}^p$. The first moment of $\mathbf{t}_\nu(\boldsymbol{\mu}, \boldsymbol{\Sigma}, \mathcal{R}^p)$ can be evaluated through relation (4.389) if $f(\mathbf{u})$ is replaced with PDF given in (4.28) and $P(\mathbf{a} \leq \mathbf{X} \leq \mathbf{b}) = \int_{\mathbf{a}}^{\mathbf{b}} \mathbf{t}_\nu(\mathbf{x} | \boldsymbol{\mu}, \boldsymbol{\Sigma}) d\mathbf{x}$. However, herein, we proceed to compute $\boldsymbol{\Omega} = E(\mathbf{X}_{\mathcal{R}^p})$ based on the property of *Gaussian scale mixture model* introduced in Section 4.4.3.

4.13.1 First two moments of truncated Student's t distribution distribution in univariate case

Herein, we proceed to compute the first and second moments of an univariate Student's t distribution. The PDF and CDF of univariate Student's t distribution distribution are given by (4.31) and (4.32), respectively. Furthermore, we write $X \sim t_\nu(\mu, \sigma)$ to show that random variable X follows a Student's t distribution with location parameter μ , scale parameter σ , and ν degrees of freedom. The truncated version on interval $\mathcal{R} = \{x \in \mathbb{R} | a \leq x \leq b\}$ is then represented by $t_\nu(\mu, \sigma, \mathcal{R})$.

If $X \sim t_\nu(\mu, \sigma, \mathcal{R})$, it can be seen from (4.387) for $r = 1$ that

$$\begin{aligned} \Omega &= \frac{1}{\mathcal{P}_t} \int_a^b x \times t(x | \mu, \sigma^2, \nu) dx, \\ &= \frac{1}{\mathcal{P}_t} \int_a^b \frac{x \Gamma(\frac{\nu+1}{2})}{\sigma \sqrt{\nu\pi} \Gamma(\frac{\nu}{2})} \left[1 + \frac{(x - \mu)^2}{\nu \sigma^2} \right]^{-\frac{\nu+1}{2}} dx, \end{aligned} \quad (4.436)$$

where $\mathcal{P}_t = T_\nu(b | \mu, \sigma) - T_\nu(a | \mu, \sigma)$. Rather computing Ω through (4.436), we prefer to follow Definition 4.0.4 for $\eta(G) = G$ when $G \sim \mathcal{G}(\nu/2, \nu/2)$. It turns out that

$$\begin{aligned} X | G &\sim \mathcal{N}\left(\mu, \frac{\sigma^2}{G}\right), \\ G &\sim \mathcal{G}\left(\frac{\nu}{2}, \frac{\nu}{2}\right). \end{aligned}$$

Using above hierarchy, the quantity Ω can be computed as follows.

$$\begin{aligned}\Omega &= \frac{1}{\mathcal{P}_t} \int_a^b x \int_0^\infty \phi\left(x \middle| \mu, \frac{\sigma^2}{g}\right) \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg dx, \\ &= \frac{1}{C_G \mathcal{P}_t} \int_0^\infty g^{\frac{1}{2}} \int_a^b x \exp\left\{-\frac{g(x-\mu)^2}{2\sigma^2}\right\} dx \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg, \\ &= \frac{\sigma^2}{C_G \mathcal{P}_t} \int_0^\infty g^{-\frac{1}{2}} \int_a^b \frac{g(x-\mu)}{\sigma^2} \exp\left\{-\frac{g(x-\mu)^2}{2\sigma^2}\right\} dx \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg + \mu,\end{aligned}$$

where $C_G = \sqrt{2\pi}\sigma$. On the other hand, since

$$\frac{\partial}{\partial \mu} \exp\left\{-\frac{g(x-\mu)^2}{2\sigma^2}\right\} = g \times \left(\frac{x-\mu}{\sigma^2}\right) \exp\left\{-\frac{g(x-\mu)^2}{2\sigma^2}\right\}, \quad (4.437)$$

it turns out that

$$\begin{aligned}\Omega &= \frac{\sigma^2}{C_G \mathcal{P}_t} \int_0^\infty g^{-\frac{1}{2}} \frac{\partial}{\partial \mu} \int_a^b \exp\left\{-\frac{g(x-\mu)^2}{2\sigma^2}\right\} dx \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg + \mu, \\ &= \frac{\sigma^2}{C_G \mathcal{P}_t} \int_0^\infty g^{-\frac{1}{2}} \frac{\partial}{\partial \mu} \int_{a-\mu}^{b-\mu} \exp\left\{-\frac{gu^2}{2\sigma^2}\right\} du \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg + \mu, \\ &= \frac{\sigma^2}{C_G \mathcal{P}_t} \int_0^\infty g^{-\frac{1}{2}} \left[\exp\left\{-\frac{g(a-\mu)^2}{2\sigma^2}\right\} - \exp\left\{-\frac{g(b-\mu)^2}{2\sigma^2}\right\} \right] \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg + \mu, \\ &= \frac{\sigma^2}{C_G \mathcal{P}_t \Gamma(\frac{\nu}{2})} \left(\frac{\nu}{2}\right)^{\frac{\nu}{2}} \int_0^\infty g^{\frac{\nu-1}{2}-1} \exp\left\{-g\left[\frac{(a-\mu)^2}{2\sigma^2} + \frac{\nu}{2}\right]\right\} dg \\ &\quad - \frac{\sigma^2}{C_G \mathcal{P}_t \Gamma(\frac{\nu}{2})} \left(\frac{\nu}{2}\right)^{\frac{\nu}{2}} \int_0^\infty g^{\frac{\nu-1}{2}-1} \exp\left\{-g\left[\frac{(b-\mu)^2}{2\sigma^2} + \frac{\nu}{2}\right]\right\} dg + \mu \\ &= \frac{\sigma^2 \Gamma(\frac{\nu-1}{2})}{C_G \mathcal{P}_t \Gamma(\frac{\nu}{2})} \left(\frac{\nu}{2}\right)^{\frac{\nu}{2}} \left[\left[\frac{(a-\mu)^2}{2\sigma^2} + \frac{\nu}{2}\right]^{-\frac{\nu-1}{2}} + \left[\frac{(b-\mu)^2}{2\sigma^2} + \frac{\nu}{2}\right]^{-\frac{\nu-1}{2}} \right] + \mu.\end{aligned} \quad (4.438)$$

Rearranging the RHS of (4.438) in terms of Student's t PDF, we obtain

$$\Omega = \frac{\sigma(\nu)}{\mathcal{P}_t} \left[t_{\nu-2}\left(\frac{a-\mu}{\sigma(\nu)} \middle| 0, 1\right) - t_{\nu-2}\left(\frac{b-\mu}{\sigma(\nu)} \middle| 0, 1\right) \right] + \mu, \quad (4.439)$$

where $\sigma(\nu) = \sigma\sqrt{\nu}/\sqrt{\nu-2}$ for $\nu > 2$. Applying the above argument to computing Ω , we have

$$\Omega^2 = \frac{1}{\mathcal{P}_t} \int_a^b x^2 \int_0^\infty \phi\left(x \middle| \mu, \frac{\sigma^2}{g}\right) \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg dx.$$

For computing Ω^2 , we take into account the fact that

$$\frac{\partial^2}{\partial \mu^2} \exp\left\{-g\frac{(x-\mu)^2}{2\sigma^2}\right\} = \left[-\frac{g}{\sigma^2} + g^2 \times \left(\frac{x-\mu}{\sigma^2}\right)^2\right] \exp\left\{-g\frac{(x-\mu)^2}{2\sigma^2}\right\}. \quad (4.440)$$

Rearranging equation (4.440) yields

$$\begin{aligned}\frac{g^2 x^2}{\sigma^4} \exp\left\{-g\frac{(x-\mu)^2}{2\sigma^2}\right\} &= \frac{\partial^2}{\partial \mu^2} \exp\left\{-g\frac{(x-\mu)^2}{2\sigma^2}\right\} \\ &\quad + \left[\left(\frac{g}{\sigma^2} - \frac{\mu^2}{\sigma^4} g^2 + 2\frac{\mu}{\sigma^4} g^2 x\right) \exp\left\{-g\frac{(x-\mu)^2}{2\sigma^2}\right\} \right].\end{aligned} \quad (4.441)$$

Multiplying both sides of (4.441) by $\sigma^4 g^{-3/2} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right)$, then integrating over x and g , and finally dividing by $C_G \mathcal{P}_t$, we have

$$\begin{aligned}\Omega^2 &= \frac{\sigma^4}{C_G \mathcal{P}_t} \int_0^\infty g^{-\frac{3}{2}} \frac{\partial^2}{\partial \mu^2} \int_a^b \exp\left\{-g\frac{(x-\mu)^2}{2\sigma^2}\right\} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg dx \\ &\quad + \left[\frac{\sigma^2}{C_G \mathcal{P}_t} \mathcal{B}_{-\frac{1}{2}, 0} - \frac{\mu^2}{C_G \mathcal{P}_t} \mathcal{B}_{\frac{1}{2}, 0} + 2\frac{\mu}{C_G \mathcal{P}_t} \mathcal{B}_{\frac{1}{2}, 1} \right],\end{aligned} \quad (4.442)$$

where

$$\mathcal{B}_{i,j} = \int_a^b \int_0^\infty g^i x^j \exp\left\{-g \frac{(x-\mu)^2}{2\sigma^2}\right\} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg dx. \quad (4.443)$$

For integral in the RHS of (4.442), it is not hard to check that

$$\begin{aligned} & \frac{\sigma^4}{C_G \mathcal{P}_t} \int_0^\infty g^{-\frac{3}{2}} \frac{\partial^2}{\partial \mu^2} \int_a^b \exp\left\{-g \frac{(x-\mu)^2}{2\sigma^2}\right\} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg dx \\ &= \frac{\sigma(\nu)}{\mathcal{P}_t} (a-\mu) t_{\nu-2}\left(\frac{a-\mu}{\sigma(\nu)} \middle| 0, 1\right) - \frac{\sigma(\nu)}{\mathcal{P}_t} (b-\mu) t_{\nu-2}\left(\frac{b-\mu}{\sigma(\nu)} \middle| 0, 1\right), \end{aligned} \quad (4.444)$$

where $\sigma(\nu) = \sigma\sqrt{\nu}/\sqrt{\nu-2}$ for $\nu > 2$. Hence, replacing the RHS of (4.444) with integral in RHS of (4.442), we have

$$\begin{aligned} \Omega^2 &= \frac{\sigma(\nu)}{\mathcal{P}_t} (a-\mu) t_{\nu-2}\left(\frac{a-\mu}{\sigma(\nu)} \middle| 0, 1\right) - \frac{\sigma(\nu)}{\mathcal{P}_t} (b-\mu) t_{\nu-2}\left(\frac{b-\mu}{\sigma(\nu)} \middle| 0, 1\right) \\ &+ \frac{\sigma^2}{C_G \mathcal{P}_t} \mathcal{B}_{-\frac{1}{2},0} - \frac{\mu^2}{C_G \mathcal{P}_t} \mathcal{B}_{\frac{1}{2},0} + 2 \frac{\mu}{C_G \mathcal{P}_t} \mathcal{B}_{\frac{1}{2},1}. \end{aligned} \quad (4.445)$$

Moreover, it may seen, for $i > (1-\nu)/2$ and $j = \{0, 1\}$, that

$$\mathcal{B}_{i,j} = \begin{cases} \frac{2^i \Gamma(\frac{\nu(i)}{2}) \sigma(i) \nu^{-i}}{[\pi \nu(i)]^{-\frac{1}{2}} \Gamma(\frac{\nu}{2})} \left[T_{\nu(i)}\left(\frac{b-\mu}{\sigma(i)} \middle| 0, 1\right) - T_{\nu(i)}\left(\frac{a-\mu}{\sigma_i} \middle| 0, 1\right) \right], & \text{if } j = 0, \\ \frac{2^i \Gamma(\frac{\nu(i)}{2}) \sigma(i) \nu^{-i}}{[\pi \nu(i)]^{-\frac{1}{2}} \Gamma(\frac{\nu}{2})} \left[T_{\nu(i)}\left(\frac{b-\mu}{\sigma(i)} \middle| 0, 1\right) - T_{\nu(i)}\left(\frac{a-\mu}{\sigma_i} \middle| 0, 1\right) \right] \Omega_*^j, & \text{if } j > 0, \end{cases} \quad (4.446)$$

where $\sigma(i) = \sigma\sqrt{\nu}/\sqrt{\nu+2i-1}$ (for $i > (1-\nu)/2$), $\nu(i) = \nu+2i-1$, and $\Omega_*^j = E(Y^j | a \leq Y \leq b)$ is the j th moment of $Y \sim t_{\nu(i)}(\mu, \sigma_i, \mathcal{R})$ in which $\mathcal{R} = \{x \in \mathbb{R} | a \leq x \leq b\}$. Computing the quantity $\mathcal{B}_{i,j}$ represented in (4.445) using (4.446), for $i = \{-1/2, 1/2\}$ and $j = \{0, 1\}$, and then replacing the results in the RHS of (4.442), one can compute the quantity Ω^2 . The R function given by the following is developed for computing first two moments of the Student's t distribution truncated on $\mathcal{R}$.

```

1 R> Ex <- function(mu, sigma, nu, a, b)
2 + {
3 +   CG <- sqrt(2*pi*sigma^2)
4 +   za <- (a - mu)/sigma; zb <- (b - mu)/sigma
5 +   Pt <- pt(zb, df = nu) - pt(za, df = nu)
6 +   ex <- function(m, s, nu, a, b)
7 +   {
8 +     za <- (a - m)/s; zb <- (b - m)/s
9 +     Pt <- pt(zb, df = nu) - pt(za, df = nu)
10 +     s_nu <- s*sqrt(nu/(nu - 2))
11 +     za_nu <- (a - m)/s_nu
12 +     zb_nu <- (b - m)/s_nu
13 +     p1 <- s_nu/Pt*dt(za_nu, df = nu - 2)
14 +     p2 <- s_nu/Pt*dt(zb_nu, df = nu - 2)
15 +     return( list("p1" = p1, "p2" = p2, "Omega" = p1 - p2 + m) )
16 +   }
17 +   out1 <- ex(mu, sigma, nu, a, b)
18 +   p3 <- (a - mu)*out1$p1 - (b - mu)*out1$p2
19 +   Rij <- function(i, j)
20 +   {
21 +     nu_i <- nu + 2*i - 1
22 +     sigma_i <- sigma*sqrt(nu/nu_i)

```

```

23 +   zb_i <- (b - mu)/sigma_i
24 +   za_i <- (a - mu)/sigma_i
25 +   C_ij <- (2/nu)^(i)*sqrt( pi*nu_i ) * gamma(nu_i/2)/gamma(nu/2)*sigma_i
26 +   Pt_i <- pt(zb_i, df = nu_i ) - pt(za_i, df = nu_i )
27 +   if(j == 0)
28 +   {
29 +     R_ij <- C_ij*Pt_i
30 +   }else{
31 +     R_ij <- C_ij*Pt_i*ex(mu, sigma_i, nu_i, a, b)$Omega
32 +   }
33 +   R_ij
34 + }
35 + Omega2 <- p3 + sigma^2/(CG*Pt)*Rij(-1/2, 0) - mu^2/(CG*Pt)*Rij(1/2, 0) +
36 + 2*mu/(CG*Pt)*Rij(1/2, 1)
37 + out2 <- list("Ex" = out1$Omega, "Exx" = Omega2)
38 + return( out2 )
39 + }

```

4.14 First moment of truncated Student's t distribution in multivariate case

Recall hierarchy (4.11) for $\eta(G) = G$ when $G \sim \mathcal{G}(\nu/2, \nu/2)$. Then simple algebra show that $E(\mathbf{X})$ in (4.435) can be represented as

$$\begin{aligned}
E(\mathbf{X}) &= \int_{-\infty}^{\infty} \mathbf{x} \int_0^{\infty} \phi_p(\mathbf{x} | \boldsymbol{\mu}, \frac{\Sigma}{g}) \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg d\mathbf{x}. \\
&= \frac{1}{\mathbf{C}_G} \int_0^{\infty} g^{\frac{p}{2}} \int_{-\infty}^{\infty} \mathbf{x} \exp\left\{-\frac{g}{2}\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)\right\} d\mathbf{x} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg,
\end{aligned}$$

where $\mathbf{C}_G = (2\pi)^{p/2} |\Sigma|^{1/2}$. The first moment of truncated Student's t distribution $\mathbf{t}_{\nu}(\boldsymbol{\mu}, \Sigma, \mathcal{R}^p)$, is

$$\boldsymbol{\Omega} = E(\mathbf{X}_{\mathcal{R}^p}) = \frac{1}{\mathbf{C}_G \mathcal{P}_t} \int_0^{\infty} \int_a^b g^{\frac{p}{2}} \mathbf{x} \exp\left\{-\frac{g}{2}\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)\right\} d\mathbf{x} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg,$$

where $\mathcal{P}_t = P(\mathbf{a} \leq \mathbf{X} \leq \mathbf{b})$. We can write

$$\begin{aligned}
\boldsymbol{\Omega} &= \frac{1}{\mathbf{C}_G \mathcal{P}_t} \int_0^{\infty} \int_a^b g^{\frac{p}{2}} (\mathbf{x} - \boldsymbol{\mu}) \exp\left\{-\frac{g}{2}\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)\right\} d\mathbf{x} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg \\
&\quad + \frac{\boldsymbol{\mu}}{\mathbf{C}_G \mathcal{P}_t} \int_0^{\infty} \int_a^b g^{\frac{p}{2}} \exp\left\{-\frac{g}{2}\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)\right\} d\mathbf{x} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg \\
&= \frac{1}{\mathbf{C}_G \mathcal{P}_t} \int_0^{\infty} \int_a^b g^{\frac{p}{2}} (\mathbf{x} - \boldsymbol{\mu}) \exp\left\{-\frac{g}{2}\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)\right\} d\mathbf{x} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg + \boldsymbol{\mu} \\
&= \mathbf{I} + \boldsymbol{\mu}.
\end{aligned} \tag{4.447}$$

Since

$$\frac{\partial}{\partial \boldsymbol{\mu}} \frac{g}{2} \delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma) = -g \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}),$$

we have

$$\mathbf{I} = \frac{\Sigma}{\mathcal{P}_t} \int_0^{\infty} \frac{g^{\frac{p}{2}-1}}{\mathbf{C}_G} \frac{\partial}{\partial \boldsymbol{\mu}} \int_a^b \exp\left\{-\frac{g}{2}\delta(\mathbf{x} - \boldsymbol{\mu}, \Sigma)\right\} d\mathbf{x} \mathcal{G}\left(g \middle| \frac{\nu}{2}, \frac{\nu}{2}\right) dg.$$

Applying the change of variable $\mathbf{u} = \mathbf{x} - \boldsymbol{\mu}$, it follows that

$$\begin{aligned} \mathbf{I} &= \frac{\Sigma}{\mathcal{P}_t} \int_0^\infty \frac{g^{\frac{p}{2}-1}}{\mathbf{C}_G} \frac{\partial}{\partial \boldsymbol{\mu}} \int_{\mathbf{a}-\boldsymbol{\mu}}^{\mathbf{b}-\boldsymbol{\mu}} \exp\left\{-\frac{g}{2}\delta(\mathbf{u}, \Sigma)\right\} d\mathbf{u} \mathcal{G}\left(g\left|\frac{\nu}{2}, \frac{\nu}{2}\right.\right) dg \\ &= (I_1, I_2, \dots, I_p)^\top, \end{aligned}$$

where

$$\begin{aligned} I_i &= \frac{\Sigma}{\mathcal{P}_t} \int_0^\infty \frac{g^{\frac{p}{2}-1}}{\mathbf{C}_G} \int_{\mathbf{a}^{[-i]}-\boldsymbol{\mu}^{[-i]}}^{\mathbf{b}^{[-i]}-\boldsymbol{\mu}^{[-i]}} \left[\exp\left\{-\frac{g\delta(\mathbf{u}_i[a_i-\mu_i], \Sigma)}{2}\right\} \right. \\ &\quad \left. - \exp\left\{-\frac{g\delta(\mathbf{u}_i[b_i-\mu_i], \Sigma)}{2}\right\} \right] d\mathbf{u}^{[-i]} \mathcal{G}\left(g\left|\frac{\nu}{2}, \frac{\nu}{2}\right.\right) dg, \end{aligned} \quad (4.448)$$

for $i = 1, \dots, p$. Using identity (4.405) and the fact that

$$\frac{g^{\frac{p}{2}-1}}{\mathbf{C}_G} = \left[\frac{1}{\sqrt{2\pi\Sigma_{i,i}}} \frac{1}{(2\pi)^{\frac{p-1}{2}} |\Delta_{ii}|^{\frac{1}{2}}} \right] \times g^{\frac{p}{2}-1} = \frac{g^{-1}}{\sqrt{2\pi}} \left[\frac{\Sigma_{i,i}}{g} \right]^{-\frac{1}{2}} \frac{1}{(2\pi)^{\frac{p-1}{2}} \left| \frac{\Delta_{ii}}{g} \right|^{-\frac{1}{2}}},$$

the integrand in the RHS of (4.448) can be rewritten as

$$\begin{aligned} I_i &= \frac{\Sigma}{\mathcal{P}_t} \int_0^\infty \frac{g^{-1}}{\sqrt{2\pi}} \left[\frac{\Sigma_{i,i}}{g} \right]^{-\frac{1}{2}} \exp\left\{-\frac{g(a_i-\mu_i)^2}{2\Sigma_{i,i}}\right\} \mathcal{G}\left(g\left|\frac{\nu}{2}, \frac{\nu}{2}\right.\right) \\ &\quad \times \int_{\mathbf{a}^{[-i]}-\boldsymbol{\mu}^{[-i]}}^{\mathbf{b}^{[-i]}-\boldsymbol{\mu}^{[-i]}} \frac{1}{(2\pi)^{\frac{p-1}{2}} \left| \frac{\Delta_{ii}}{g} \right|^{-\frac{1}{2}}} \exp\left\{-\frac{g}{2}\delta(\mathbf{u}^{[-i]} - \boldsymbol{\xi}(a_i - \mu_i), \Delta_{ii})\right\} d\mathbf{u}^{[-i]} dg \\ &\quad - \frac{\Sigma}{\mathcal{P}_t} \int_0^\infty \frac{g^{-1}}{\sqrt{2\pi}} \left[\frac{\Sigma_{i,i}}{g} \right]^{-\frac{1}{2}} \exp\left\{-\frac{g(b_i-\mu_i)^2}{2\Sigma_{i,i}}\right\} \mathcal{G}\left(g\left|\frac{\nu}{2}, \frac{\nu}{2}\right.\right) \\ &\quad \times \int_{\mathbf{a}^{[-i]}-\boldsymbol{\mu}^{[-i]}}^{\mathbf{b}^{[-i]}-\boldsymbol{\mu}^{[-i]}} \frac{1}{(2\pi)^{\frac{p-1}{2}} \left| \frac{\Delta_{ii}}{g} \right|^{-\frac{1}{2}}} \exp\left\{-\frac{g}{2}\delta(\mathbf{u}^{[-i]} - \boldsymbol{\xi}(b_i - \mu_i), \Delta_{ii})\right\} d\mathbf{u}^{[-i]} dg, \end{aligned} \quad (4.449)$$

where $\boldsymbol{\xi}(x)$ and Δ_{ii} are defined in (4.406) and (4.407), respectively. Moreover, it is not hard to check that

$$\begin{aligned} &\frac{g^{-1}}{\sqrt{2\pi}} \left[\frac{\Sigma_{i,i}}{g} \right]^{-\frac{1}{2}} \exp\left\{-\frac{g(a_i-\mu_i)^2}{2\Sigma_{i,i}}\right\} \mathcal{G}\left(g\left|\frac{\nu}{2}, \frac{\nu}{2}\right.\right) \\ &= \sqrt{\frac{\nu_2}{\Sigma_{i,i}}} t_{\nu-2}(a_i | \mu_i, \nu_2 \Sigma_{i,i}) \mathcal{G}\left(g\left|\frac{\nu-1}{2}, \lambda_a\right.\right), \end{aligned} \quad (4.450)$$

where $\nu_2 = \nu/(\nu-2)$, $\lambda_a = (a_i - \mu_i)^2/(2\Sigma_{i,i}) + \nu/2$. Further, for given vectors $\mathbf{c}_1$ and $\mathbf{c}_2$ of length $p-1$ such that $\mathbf{c}_1 < \mathbf{c}_2$, we have

$$\begin{aligned} &\int_0^\infty \int_{\mathbf{c}_1}^{\mathbf{c}_2} \frac{1}{(2\pi)^{\frac{p-1}{2}} \left| \frac{\Delta_{ii}}{g} \right|^{-\frac{1}{2}}} \exp\left\{-\frac{g}{2}\delta(\mathbf{y} - \mathbf{m}, \Delta_{ii})\right\} d\mathbf{y} \mathcal{G}\left(g\left|\frac{\nu-1}{2}, \lambda_a\right.\right) dg \\ &= \int_{\mathbf{c}_1}^{\mathbf{c}_2} \frac{\lambda_a^{\frac{\nu-1}{2}}}{(2\pi)^{\frac{p-1}{2}} \left| \frac{\Delta_{ii}}{g} \right|^{\frac{1}{2}}} \frac{\Gamma(\frac{\nu+p-2}{2})}{\Gamma(\frac{\nu-1}{2})} \left[\frac{\delta(\mathbf{y} - \mathbf{m}, \Delta_{ii})}{2} + \lambda_a \right]^{-\frac{\nu+p-2}{2}} d\mathbf{y} \\ &= \mathbf{T}_{\nu-1}\left(\mathbf{c}_2 | \mathbf{m}, \frac{2\lambda_a}{\nu-1} \Delta_{ii}\right) - \mathbf{T}_{\nu-1}\left(\mathbf{c}_1 | \mathbf{m}, \frac{2\lambda_a}{\nu-1} \Delta_{ii}\right), \end{aligned} \quad (4.451)$$

where $\nu > 1$. Using the facts given in (4.450) and (4.451), the RHS of (4.449) becomes

$$\begin{aligned}
I_i = & \sqrt{\frac{\nu_2}{\Sigma_{i,i}}} \frac{\Sigma}{\mathcal{P}_t} t_{\nu-2}(a_i | \mu_i, \nu_2 \Sigma_{i,i}) \left[\mathbf{T}_{\nu-1}(\mathbf{b}[-i] - \boldsymbol{\mu}[-i] | \boldsymbol{\xi}(a_i - \mu_i), \frac{2\lambda_a}{\nu-1} \Delta_{ii}) \right. \\
& \left. - \mathbf{T}_{\nu-1}(\mathbf{a}[-i] - \boldsymbol{\mu}[-i] | \boldsymbol{\xi}(a_i - \mu_i), \frac{2\lambda_a}{\nu-1} \Delta_{ii}) \right] - \\
& \sqrt{\frac{\nu_2}{\Sigma_{i,i}}} \frac{\Sigma}{\mathcal{P}_t} t_{\nu-2}(b_i | \mu_i, \nu_2 \Sigma_{i,i}) \left[\mathbf{T}_{\nu-1}(\mathbf{b}[-i] - \boldsymbol{\mu}[-i] | \boldsymbol{\xi}(b_i - \mu_i), \frac{2\lambda_b}{\nu-1} \Delta_{ii}) \right. \\
& \left. - \mathbf{T}_{\nu-1}(\mathbf{a}[-i] - \boldsymbol{\mu}[-i] | \boldsymbol{\xi}(b_i - \mu_i), \frac{2\lambda_b}{\nu-1} \Delta_{ii}) \right], \tag{4.452}
\end{aligned}$$

where $\nu > 2$ and $\lambda_b = (b_i - \mu_i)^2 / (2\Sigma_{i,i}) + \nu/2$. Computing I_i in (4.452), for $i = 1, \dots, p$, the first moment of truncated Student's t distribution is then obtained by substituting the constructed vector $\mathbf{I}$ into the RHS of (4.447). In what follows, we perform a small simulation study for computing $\boldsymbol{\Omega}$ when $\mathbf{X} = (X_1, X_2, X_3)^\top$ follows three-dimensional Student's t distribution with parameters $\nu = 3$, $\boldsymbol{\mu} = (1, 1, 1)^\top$ and

$$\Sigma = \begin{bmatrix} 2 & 0.5 & 0.4 \\ 0.5 & 1.5 & 0.3 \\ 0.4 & 0.3 & 1 \end{bmatrix}. \tag{4.453}$$

The truncation bounds are $\mathbf{a} = (0, 0, 0)^\top$ and $\mathbf{b} = (10, 10, 10)^\top$. To this end, we compute $\boldsymbol{\Omega}$ using the exact method described above and the importance sampling in which the instrumental distribution is $\mathbf{t}_\nu(\boldsymbol{\mu}, \Sigma)$. For computing $\boldsymbol{\Omega}$ through the importance sampling, we generate a sample of size N through the rejection method. Table 4.17 shows the results of simulation study for approximating $\boldsymbol{\Omega}$ under both of exact and importance sampling scenarios.

Table 4.17: Summary statistics for computing $\boldsymbol{\Omega} = E(\mathbf{X}_{\mathcal{R}^p})$ through the exact and importance sampling methods.

	$\mathbf{a}^\top$	$\mathbf{b}^\top$	N	$\boldsymbol{\Omega}$	$\hat{\boldsymbol{\Omega}}$	summary			
						bias	SE	min.	max.
1	(-1,-1)	(2,2)	2000	0.3691	$\widehat{\Omega}_1$	-1.7e-04	0.0174	0.3021	0.4440
				0.2603	$\widehat{\Omega}_2$	5.9e-05	0.0158	0.2072	0.3158
			5000	0.3691	$\widehat{\Omega}_1$	7.5e-06	0.0113	0.3268	0.4071
				0.2603	$\widehat{\Omega}_2$	-1.8e-04	0.0101	0.2255	0.2971
2	(0,0)	(∞, ∞)	2000	1.2150	$\widehat{\Omega}_1$	-7.8e-05	0.0197	1.1287	1.2802
				1.2150	$\widehat{\Omega}_2$	-8.3e-04	0.0196	1.1462	1.2827
			5000	1.2150	$\widehat{\Omega}_1$	-7.4e-04	0.0124	1.1690	1.2603
				1.2150	$\widehat{\Omega}_2$	-4.7e-04	0.0125	1.1701	1.2774

```

1 R> library(MomTrunc) #for computing the CDF of multivariate Student's t
2 R> a <- rep(0, p); Mu <- rep(1, p); b <- rep(100, p); nu <- 3;
3 R> Sigma <- matrix( c(2,0.5,0.4,0.5,1.5,0.3,0.4,0.3,1), nrow = 3, ncol = 3)
4 R> EX <- function(Mu, Sigma, nu, a, b)
5 + {
6 + p <- length(Mu)
7 + V <- numeric(p)
8 + for(i in 1:p){
9 + p1 <- (a[i] - Mu[i])/sqrt(nu/(nu-2)*Sigma[i, i])
10 + p2 <- (b[i] - Mu[i])/sqrt(nu/(nu-2)*Sigma[i, i])
11 + lambda_a <- (a[i] - Mu[i])^2/(2*Sigma[i, i]) + nu/2

```

```

12 + lambda_b <- (b[i] - Mu[i])^2/(2*Sigma[i, i])+ nu/2
13 + da <- sqrt(nu)/sqrt( (nu-2)*Sigma[i, i] )*dt(p1, df = nu - 2)
14 + db <- sqrt(nu)/sqrt( (nu-2)*Sigma[i, i] )*dt(p2, df = nu - 2)
15 + xi <- function(x) Sigma[i, -i]*x/Sigma[i, i]
16 + if(p == 2)
17 + {
18 + Delta_i <- Sigma[-i, -i] - Sigma[-i, i]^2/Sigma[i, i]
19 + }else{
20 + Delta_i <- Sigma[-i, -i] - Sigma[-i, i]%*%t(Sigma[-i, i])/Sigma[i, i]
21 + }
22 + Delta_ia <- 2*lambda_a/(nu - 1)*Delta_i
23 + Delta_ib <- 2*lambda_b/(nu - 1)*Delta_i
24 + p3 <- da*pmvEST( a[-i] - Mu[-i], b[-i] - Mu[-i], mu = xi(a[i] - Mu[i]),
25 + Sigma = Delta_ia, lambda = rep(0, p-1), tau = 0, nu = nu - 1 )
26 + p4 <- db*pmvEST( a[-i] - Mu[-i], b[-i] - Mu[-i], mu = xi(b[i] - Mu[i]),
27 + Sigma = Delta_ib, lambda = rep(0, p-1), tau = 0, nu = nu - 1 )
28 + V[i] <- p3 - p4
29 + }
30 + EX <- Sigma%*%V/pmvEST(a, b, Mu, Sigma, rep(0, p), tau = 0, nu = nu) + Mu
31 + return(EX)
32 + }

```

Bayesian inference

*A Bayesian prior is an assumption of an infinite amount of past relevant experience.
But you cannot forget that you have just made up a whole bunch of data*¹
– Bradley Efron, Stanford University

In the framework of classical (or frequentist) statistics, the unknown parameter is assumed to be fixed and there is not any guess or belief about its true value. The only source to estimate the unknown parameter is the sample evidence (information). Contrary to this school of thinking, the Bayesian statistics permits to involve our *prior* knowledge or belief in process of estimating of unknown parameter. For this reason, the classical and Bayesian statistics are sometimes called *objective* and *subjective*² statistics, respectively. In other words, the subjectivity of belief about unknown parameter leads us to assume that unknown parameter is a random variable with a known distribution, called in the literature *prior* distribution. The Bayesian statistics would make a revision on the experimenter belief about the unknown parameter using the sample evidence. Let the PDF corresponds to the prior distribution of unknown parameter θ is shown by $\pi(\theta)$. Furthermore suppose the revised version of $\pi(\theta)$ given sample evidence $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$, is represented as $\pi(\theta|\mathbf{x})$ that is known in the literature as the *posterior* PDF. The elementary Bayesian formula links the *prior* with *posterior* as

$$\pi(\theta|\mathbf{x}) = \frac{f(\mathbf{x}|\theta)}{\int_{\theta} f(\mathbf{x}|\theta)\pi(\theta)d\theta}\pi(\theta) = \frac{f(\mathbf{x}|\theta)}{m(\mathbf{x})}\pi(\theta) \propto f(\mathbf{x}|\theta)\pi(\theta) = L(\theta|\mathbf{x})\pi(\theta), \quad (5.1)$$

where $L(\theta|\mathbf{x}) = f(\mathbf{x}|\theta) = f(x_1, x_2, \dots, x_n|\theta) = \prod_{i=1}^n f(x_i|\theta)$ is the likelihood function. Hence,

$$\pi(\theta|\mathbf{x}) \propto \prod_{i=1}^n f(x_i|\theta)\pi(\theta), \quad (5.2)$$

Notice that joint PDF of sample evidence in (5.2), is represented as the product of marginal PDFs since the sample evidence $x_1, x_2, \dots, x_n$ are assumed to be independent. Notice that in (6.30) the *prior* may be either *proper* or *improper*³. If the chosen *prior* $\pi(\theta)$ is *improper*, then the formal Bayes formula (6.30) is valid if we have $\pi(\theta|\mathbf{x})$ is *proper* or equivalently $m(\mathbf{x}) = \int_{\theta} f(\mathbf{x}|\theta)\pi(\theta)d\theta < \infty$. Furthermore, we note that the parameter space may be d -dimensional (multi-parameter case), represented as $\boldsymbol{\theta} = (\theta_1, \dots, \theta_d)^{\top}$.

5.1 Prior selection

Although the prior reflects experimenter's belief or *prior* knowledge on θ , but the term *subjectivity* does not mean that experimenter is free to choose any candidate for distribution of *prior*.

¹<https://rss.onlinelibrary.wiley.com/doi/pdf/10.1111/j.1740-9713.2010.00460.x>

²The belief about unknown parameter differs from individual to another one, so it is *subjective*.

³The *prior* $\pi(\theta)$ is called improper if $\int_{\theta} \pi(\theta)d\theta$ diverges; otherwise $\pi(\theta)$ is called *proper*.

In practice, there are a variety of rules for this mean that makes the process of *prior* selection a quite *objective*⁴ process that may need comprehensive study or knowledge about the unknown parameter. The structure of *prior* may also depend on some extra parameter(s) that is known in the literature as *hyperparameter*. In what follows, we cite some known classes of *objective prior*.

5.1.1 Uniform prior

The first class is the uniform prior $\pi_U(\theta)$, that is useful when the parameter space is compact interval and experimenter believes that all points in parameter space are equally likely outcomes. The famous example of this type may happen when one is interested in Bayesian inference for the success probability in a Bernoulli process for which we may have $\pi_U(\theta) \propto 1$. Considering a beta distribution for the law of *prior*, then $\pi_U(\theta|a, b)$, for which a and b play the role of *hyperparameters*, is no longer uniform. Doob [1949]

5.1.2 Reference prior

The first class of *priors* is known as the reference *prior* $\pi_R(\theta)$, proposed by Bernardo [1979]. For convenience let us to give some useful definitions as follows.

Definition 5.1.1. Let $F(x|\theta)$ and $f(x|\theta)$ denote, respectively, the CDF and PDF of the random variable X with unknown parameter $\theta \in \Theta$.

- (i) **location family:** The family of random variable X is a location family if $F(x - \theta|\theta)$ or $f(x - \theta|\theta)$ for $\theta \in \mathbb{R}$ no longer depends on θ . Statistically speaking, distribution of transformation $X - \theta$ does not depend on θ . Herein, θ is known in the literature as the location parameter.
- (ii) **scale family:** The family of random variable X is a scale family if $F(x/\theta|\theta)$ or $1/\theta \times f(x/\theta|\theta)$ for $\theta \in \mathbb{R}^+$ no longer depends on θ . Statistically speaking, distribution of transformation X/θ does not depend on θ . Herein, θ is known in the literature as the scale parameter.
- (iii) **sufficient statistic** Let $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$ denote a sequence of observations of random variables $X_1, X_2, \dots, X_n$ that follow independently a family of distributions with PDF $f(x|\theta)$. A function of random variables $\mathbf{X} = \{X_1, X_2, \dots, X_n\}$ such as $S(\mathbf{X})$ is called sufficient statistic if $S(\mathbf{X})$ provides information enough about unknown parameter $\theta \in \Theta$. Theoretically speaking, the conditional distribution of $\mathbf{X}$ given $S(\mathbf{X})$ does not depend on θ .

The reference *prior* is obtained by maximizing the expected value of the Kullback-Leibler (or logarithmic) divergence between the *prior* and the posterior. The logarithmic divergence of PDF $\pi(\theta)$ from PDF $\tilde{\pi}(\theta)$ is represented as

$$\mathcal{K}(\pi||\tilde{\pi}) = \int_{\Theta} \pi(\theta) \log \frac{\pi(\theta)}{\tilde{\pi}(\theta)} d\theta, \quad (5.3)$$

provided that integral (5.3) is finite. The reference *prior* must justify two convincing rationales including *permissibility* and *expected logarithmically convergence* [Berger et al., 2009, Definition 5]. There follows a listing of nice properties of the reference prior including:

- (i) Consistency under reparameterization of the unknown parameter θ

⁴The term *objective* means that experimenter has some plausible guideline for selecting the distribution of *prior*.

- (ii) Independence of sample size when observations are independent
- (iii) Compatibility with sufficient statistic
- (iiii) For a distribution for which nothing is known about unknown parameter θ , the reference prior follows the uniform distribution

For a proof of properties noted above, we refer the reader to Berger et al. [2009]. The property (i) states, for a location family, that the reference *prior* for location parameter is flat (equally likely over $\mathbb{R}$) and within a scale family, the reference *prior* of the scale parameter θ is proportional to $1/\theta$ that is the same of Jeffreys *prior*, that is $\pi_R(\theta) = \pi_J(\theta) \propto 1/\theta$ [Bernardo, 2005]. Although the reference prior is an objective prior and inherits nice properties among them some noted above, but it suffers from some drawbacks as follows.

- (i) The reference prior must be computed point-by-point and then approximated through interpolation techniques.
- (ii) The burden of computations grows when sample size increases and it may be critical if $f(x|\theta)$ lacks closed form such as α -stable distribution.
- (iii) Although the conditions for existence of reference prior for multi-parameter case is guaranteed, but obviously demands for more computational complexity.

In general, a reference prior is obtained by following the steps of the algorithm given below [Berger et al., 2009].

Algorithm 15 Computing the reference prior

- 1: Choose a moderate integer value for k ;
 - 2: Choose an arbitrary positive function $\pi^*(\theta)$ such as $\pi^*(\theta) = 1$;
 - 3: Consider the sequence $\theta_1, \dots, \theta_m$ to be m arbitrary distinct points that θ can take on;
 - 4: Set $j = 1$;
 - 5: **while** $j \leq m$ **do**
 - 6: Generate realizations $x_1, x_2, \dots, x_k$ from PDF $f(\cdot|\theta_j)$;
 - 7: Compute the integral $c_j = \int_{\mathbb{R}} \prod_{i=1}^k f(x_i|u) \pi^*(u) du$;
 - 8: Compute $r_j = \log\left(\prod_{i=1}^k f(x_i|\theta_j) \pi^*(\theta_j) / c_j\right)$;
 - 9: Compute $\pi(\theta_j) = \exp\{m^{-1} \sum_{j=1}^m r_j\}$ and store the pair $(\theta_j, \pi(\theta_j))$
 - 10: Set $j \leftarrow j + 1$;
 - 11: **end**
 - 12: The set $\{(\theta_j, \pi(\theta_j))\}_{j=1}^m$ is the sequence of m paired from which $\pi(\theta)$ can be obtained through interpolation.
-

5.1.3 Jeffreys prior

The second class of objective *prior* is known as the Jeffreys *prior* $\pi_J(\theta)$ that was proposed by Jeffreys [1946]. We have

$$\pi_J(\theta) \propto \sqrt{\det I(\theta)}, \quad (5.4)$$

where $I(\theta) = -E[\partial^2 / \partial \theta_i \partial \theta_j \log L(\theta|\mathbf{x})]$, for $i, j = 1, \dots, d$ is the Fisher information matrix.

5.1.4 Conjugate prior

The last class of priors is the conjugate prior $\pi_C(\theta)$. If prior and posterior follow the same family of distributions, then the corresponding *prior* is called a conjugate *prior*. For example,

let $x_1, x_2, \dots, x_n$ independently follow $\mathcal{N}(\mu, \sigma^2)$. Considering a *prior* with PDF $\mathcal{N}(\cdot | \mu_0, \sigma_0^2)$, it is easy to see that *posterior* of μ is

$$\pi(\mu | \mathbf{x}) \sim \mathcal{N}\left(\frac{\frac{\sigma^2}{n}\mu_0 + \sigma_0^2\bar{x}}{\frac{\sigma^2}{n} + \sigma_0^2}, \left(\frac{n}{\sigma^2} + \frac{1}{\sigma_0^2}\right)^{-1}\right), \quad (5.5)$$

where $\bar{x}$ is the sample mean and $\boldsymbol{\theta}_0 = (\mu_0, \sigma_0)^T$ is vector of *hyperparameters*. As it is seen from (5.5), the *posterior* follows a Gaussian distribution too. We note that the conjugate prior is an *subjective* prior since it is chosen by experimenter without rationale and just for the sake of simplicity in form and specially sampling from. The main advantage of the conjugate *prior* is that sampling from the corresponding posterior is an easy task.

If the PDF of family under study has not closed form, finding $\pi_J(\theta)$, $\pi_R(\theta)$, or even $\pi_C(\theta)$ may be difficult. As we will see in the next section, uniform and conjugate priors will be proposed for α and $\delta = \sigma^2$, respectively. These types were chosen by [Buckle, 1995, Lombardi, 2007, Godsill and Kuruoglu, 1999] for Bayesian estimation of tail thickness and scale parameters of α -stable distribution. However, some drawbacks of IG conjugate *prior* were addressed by [Gelman, 2006, Teimouri et al., 2024]. Indeed, investigating advantages and disadvantages of above-noted *priors* in more details needs more space that is out of scope of this work. We refer the reader to [Jeffreys, 1946, Jaynes, 1968, Gelman et al., 1995, Berger et al., 2009, Box and Tiao, 2011] for further specific information.

5.2 Empirical Bayes

As noted earlier, the prior itself may depend on hyperparameter(s). Hence, sometimes we use the notation $\pi(\theta | \boldsymbol{\theta}_0)$ rather than $\pi(\theta)$ to emphasize the fact that prior depends on hyperparameter $\boldsymbol{\theta}_0$. Evidently, for characterizing distribution of the unknown parameter θ through the posterior in (6.30), it may be seen

$$\pi(\theta | \mathbf{x}, \boldsymbol{\theta}_0) = \frac{f(\mathbf{x} | \theta)}{m(\mathbf{x} | \boldsymbol{\theta}_0)} \pi(\theta | \boldsymbol{\theta}_0), \quad (5.6)$$

where the unknown *hyperparameter* $\boldsymbol{\theta}_0$ plays the role of a *nuisance* parameter. The empirical Bayes is essentially an inferential approach, for estimating $\boldsymbol{\theta}_0$, that was introduced by Robbins [1956]. It takes two *non-parametric* and *parametric* types [Casella, 1985]. The empirical Bayes proposed by Robbins [1956] is a *non-parametric* approach under which experimenter has no any assumption about specification(s) of the prior's distribution while within a *parametric* framework the family of prior is fully specified by experimenter [Morris, 1983, Casella, 1985]. More precisely, the *parametric* empirical Bayes takes into account information provided by sample evidence $\mathbf{x}$ for estimating *hyperparameter* $\boldsymbol{\theta}_0$. Herein, we assume that $\boldsymbol{\theta}_0$ is fixed, but unknown. If $\boldsymbol{\theta}_0$ itself follows some distribution, then the procedure of estimating θ within Bayesian framework is called *hierarchical Bayesian inference*. For a single observation x_i , we recall from (5.6) that

$$\pi(\theta | x_i, \boldsymbol{\theta}_0) = \frac{f(x_i | \theta)}{m(x_i | \boldsymbol{\theta}_0)} \pi(\theta | \boldsymbol{\theta}_0). \quad (5.7)$$

If $\pi(\theta | x_i, \boldsymbol{\theta}_0)$ is proper, then $m(x_i | \boldsymbol{\theta}_0)$ in denominator of (5.7) itself is a PDF since $0 < m(x_i | \boldsymbol{\theta}_0) < \infty$ and $\int_{\mathbb{R}} m(x_i | \boldsymbol{\theta}_0) d\mathbf{x} = 1$. In fact, $m(x_i | \boldsymbol{\theta}_0)$ plays the role of *marginal* PDF (likelihood function) when θ is integrated out. The unknown hyperparameter $\boldsymbol{\theta}_0$ is then estimated through the maximum likelihood (ML) by maximizing $\prod_{i=1}^n m(x_i | \boldsymbol{\theta}_0)$ with respect to $\boldsymbol{\theta}_0$ or using the method of moments [Teimouri et al., 2024]. In what follows, we proceed to compute the *parametric* empirical Bayes through an example.

Example 5-1

Let $x_1, x_2, \dots, x_n$ independently follow $\mathcal{N}(\mu, \sigma^2)$ in which scale parameter $\sigma > 0$ is assumed to be known. Furthermore, a Gaussian prior $\pi(\mu) = \mathcal{N}(\mu|\mu_0, \sigma_0^2)$, is assumed for μ . For estimating $\theta_0 = (\mu_0, \sigma_0)^T$, we can write

$$m(x_i|\theta_0) = \int_{-\infty}^{\infty} f(x_i|\mu)\pi(\mu|\theta_0)d\mu = \int_{-\infty}^{\infty} f(x_i|\mu)\pi(\mu|\theta_0)d\mu,$$

where $f(x_i|\mu) = \mathcal{N}(x_i|\mu, \sigma^2)$. We have

$$m(x_i|\theta_0) = \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\sigma} \exp\left\{-\frac{(x_i - \mu)^2}{2\sigma^2}\right\} \frac{1}{\sqrt{2\pi}\sigma_0} \exp\left\{-\frac{(\mu - \mu_0)^2}{2\sigma_0^2}\right\} d\mu.$$

Simplifying and some algebra leads to

$$m(x_i|\theta_0) = \mathcal{N}(x_i|\mu_0, \sigma^2 + \sigma_0^2).$$

It is well known that the ML estimator of mean and variance of each Gaussian distribution are the sample counterparts. So, the empirical Bayesian estimator of μ_0 and σ_0 are $\hat{\mu}_{0EB} = \bar{x}$ and $\hat{\sigma}_{0EB} = \sqrt{|S^2 - \sigma^2|}$ where $\bar{x}$ and S^2 are the sample mean and variance, respectively. It is noteworthy that the quantity $S^2 - \sigma^2$ may be negative especially when sample size is small. Hence, the absolute sign is used to avoid negative value for $\hat{\sigma}_{0EB}$. ■

It is immediate from Example 5-1 above that the Bayesian estimator of μ , that is $\hat{\mu}_B$, becomes the average of the posterior (5.5). Hence, substituting $\hat{\mu}_{0EB}$ and $\hat{\sigma}_{0EB}$ obtained above in RHS of (5.5) yields $\hat{\mu}_B = \bar{x}$ with standard error

$$SE(\hat{\mu}_B) = \left(\frac{n}{\sigma^2} + \frac{1}{|S^2 - \sigma^2|}\right)^{-\frac{1}{2}}.$$

Evidently, $\hat{\mu}_B$ is more efficient than the ML counterpart $\hat{\mu}_{ML} = \bar{x}$ since

$$SE(\hat{\mu}_B) < SE(\hat{\mu}_{ML}) = \frac{\sigma}{\sqrt{n}}.$$

A different version of Example 5-1 in which x_i s independently, for $i = 1, \dots, n$, follow $\mathcal{N}(\mu_i, \sigma^2)$ is discussed by Casella [1985].

Example 5-2

Suppose $x_1, x_2, \dots, x_n$ follow independently a Bernoulli distribution with success probability y , that is $X_i \sim \mathcal{BER}(y)$ for $i = 1, \dots, n$ and $0 < y < 1$. For computing the Bayes estimation of y , that is $\hat{y}_B$, we may consider a Beta distribution with PDF $\pi(y|\theta_0) = 1/\mathcal{B}(a, b)y^{a-1}(1-y)^{b-1}$ for the prior of y where $\theta_0 = (a, b)^T$ is the vector of hyperparameters. We can write

$$\begin{aligned} \pi(y|\mathbf{x}, \theta_0) &= y^{\sum_{i=1}^n x_i} (1-y)^{n-\sum_{i=1}^n x_i} \pi(y|a, b) \\ &\propto y^{\sum_{i=1}^n x_i + a - 1} (1-y)^{n-\sum_{i=1}^n x_i + b - 1}, \end{aligned} \quad (5.8)$$

where $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$. Obviously, $y|\mathbf{x}$ follows a beta distribution. So, we can consider the mean of posterior, that is $(\sum_{i=1}^n x_i + a - 1)/(n + a + b - 2)$, as the Bayes estimator for the success probability y . Recalling from Subsection 5.2, for computing the vector of hyperparameters $\theta_0 = (a, b)^T$, we can write

$$\begin{aligned} m(x_i|\theta_0) &= \int_0^1 f(x_i|y)\pi(y|\theta_0)dy = \int_0^1 \frac{y^{x_i+a-1}(1-y)^{b-x_i}}{\mathcal{B}(a, b)} dy \\ &= \frac{\mathcal{B}(x_i + a, b - x_i + 1)}{\mathcal{B}(a, b)} \\ &= \frac{\Gamma(x_i + a)\Gamma(b - x_i + 1)}{(a + b)\Gamma(a)\Gamma(b)}. \end{aligned}$$

Hence the likelihood function becomes

$$m(\mathbf{x}|\boldsymbol{\theta}_0) = \frac{\prod_{i=1}^n \Gamma(x_i + a) \Gamma(b - x_i + 1)}{(a + b)^n [\Gamma(a) \Gamma(b)]^n}. \quad (5.9)$$

Differentiating the RHS of (5.9) with respect to $\boldsymbol{\theta}_0$ and some algebra leads to the empirical Bayes estimator of $\boldsymbol{\theta}_0$ as $\widehat{\boldsymbol{\theta}}_{0EB} = (\sum_{i=1}^n x_i/n, (n - \sum_{i=1}^n x_i)/n)^\top$. ■

If in Example 5-2, the Bayesian estimator $\hat{y}_B$, of success probability y is desired, one could assume $\hat{y}_B$ is equal to the expected value of posterior as usual. To obtain $\hat{y}_B$, we recall from (5.8) that

$$\pi(y|\mathbf{x}) = \frac{y^{\sum_{i=1}^n x_i + a - 1} (1 - y)^{n - \sum_{i=1}^n x_i + b - 1}}{\mathcal{B}(\sum_{i=1}^n x_i + a - 1, n - \sum_{i=1}^n x_i + b - 1)}. \quad (5.10)$$

It turns out that $y|\mathbf{x}$ follows a Beta distribution with parameters $\sum_{i=1}^n x_i + a$ and $n - \sum_{i=1}^n x_i + b$. Hence,

$$\hat{y}_B = E(y|\mathbf{x}) = \int_0^1 y \pi(y|\mathbf{x}) dy = \frac{\sum_{i=1}^n x_i + a}{n + a + b}. \quad (5.11)$$

Substituting $\boldsymbol{\theta}_0$ computed above into the RHS of (5.11) yields

$$\hat{y}_B = \frac{\sum_{i=1}^n x_i + \frac{\sum_{i=1}^n x_i}{n}}{n + 1}.$$

As it is seen, the Bayesian estimator $\hat{y}_B$ is asymptotically (that is, $n \rightarrow \infty$) equal to the maximum likelihood (ML) estimator $\hat{y}_{ML} = \sum_{i=1}^n x_i/n$ of success probability y .

Example 5-3

Let $x_1, x_2, \dots, x_n$ come independently from $\mathcal{G}(\alpha, \beta)$ with unknown α and known β . For computing the Bayesian estimator of α , we suppose that $\pi(\alpha|\boldsymbol{\theta}_0) = \mathcal{G}(\alpha|\alpha_0, \beta_0)$. But, first we need to determine hyperparameter $\boldsymbol{\theta}_0 = (\alpha_0, \beta_0)^\top$. To this end, we proceed through the empirical Bayes as follows.

$$\pi(\alpha|\mathbf{x}, \boldsymbol{\theta}_0) = \frac{f(\mathbf{x}|\alpha) \pi(\alpha|\boldsymbol{\theta}_0)}{m(\mathbf{x}|\boldsymbol{\theta}_0)},$$

where

$$\begin{aligned} f(\mathbf{x}|\alpha) \pi(\alpha|\boldsymbol{\theta}_0) &= \left[\prod_{i=1}^n \frac{\beta^\alpha x_i^{\alpha-1}}{\Gamma(\alpha)} \exp\{-\beta x_i\} \right] \frac{\beta_0^{\alpha_0} \alpha^{\alpha_0-1}}{\Gamma(\alpha_0)} \exp\{-\beta_0 \alpha\}, \\ &= \frac{\beta_0^{\alpha_0} \beta^{n\alpha} \alpha^{\alpha_0-1}}{\Gamma(\alpha_0) [\Gamma(\alpha)]^n} \left[\prod_{i=1}^n x_i \right]^{\alpha-1} \exp\{-\beta \sum_{i=1}^n x_i - \beta_0 \alpha\}, \end{aligned}$$

and

$$\begin{aligned} m(x_i|\boldsymbol{\theta}_0) &= \int_0^\infty f(x_i|\alpha) \pi(\alpha|\boldsymbol{\theta}_0) d\alpha \\ &= \int_0^\infty \beta^\alpha \frac{x_i^{\alpha-1}}{\Gamma(\alpha)} \exp\{-\beta x_i\} \beta_0^{\alpha_0} \frac{\alpha^{\alpha_0-1}}{\Gamma(\alpha_0)} \exp\{-\beta_0 \alpha\} d\alpha. \end{aligned} \quad (5.12)$$

Obviously, the integral in the RHS of (5.1) has no closed form and so computing $\widehat{\boldsymbol{\theta}}_{0EB}$ using the ML approach is computationally cumbersome. Herein, we proceed to find the moment-based estimator of $\boldsymbol{\theta}_0$. First note that

$$\begin{aligned}
E(X^s|\boldsymbol{\theta}_0) &= \int_0^\infty x^s \int_0^\infty f(x|\alpha)\pi(\alpha|\boldsymbol{\theta}_0)d\alpha dx \\
&= \int_0^\infty x^s \int_0^\infty \beta^\alpha \frac{x^{\alpha-1}}{\Gamma(\alpha)} \exp\{-\beta x\} \beta_0^{\alpha_0} \frac{\alpha^{\alpha_0-1}}{\Gamma(\alpha_0)} \exp\{-\beta_0 \alpha\} d\alpha dx \\
&= \int_0^\infty \frac{\beta^\alpha \beta_0^{\alpha_0} \alpha^{\alpha_0-1} \exp\{-\beta_0 \alpha\}}{\Gamma(\alpha)\Gamma(\alpha_0)} \int_0^\infty x^{s+\alpha-1} \exp\{-\beta x\} dx d\alpha \\
&= \int_0^\infty \frac{\beta^\alpha \beta_0^{\alpha_0} \alpha^{\alpha_0-1} \Gamma(s+\alpha) \exp\{-\beta_0 \alpha\}}{\Gamma(\alpha)\Gamma(\alpha_0)\beta^{s+\alpha}} d\alpha,
\end{aligned} \tag{5.13}$$

where $s \in \mathbb{N}$. It is easy to check that

$$E(X|\boldsymbol{\theta}_0) = \frac{\alpha_0}{\beta_0\beta}, \tag{5.14}$$

and

$$E(X^2|\boldsymbol{\theta}_0) = \frac{\alpha_0}{\beta_0\beta^2} + \frac{\alpha_0(\alpha_0+1)}{\beta_0^2\beta^2}. \tag{5.15}$$

Using (5.14) and (5.15), and some simplifications yields

$$\alpha_0 = \frac{\beta[E(X|\boldsymbol{\theta}_0)]^2}{\beta \text{var}(X|\boldsymbol{\theta}_0) - E(X|\boldsymbol{\theta}_0)}. \tag{5.16}$$

So, based on sample evidence $\{x_1, x_2, \dots, x_n\}$, the moment-based estimator $\widehat{\alpha}_{0EB}$ is given by

$$\widehat{\alpha}_{0EB} = \frac{\beta \bar{x}^2}{|\beta S^2 - \bar{x}|}. \tag{5.17}$$

Consequently, the moment-based estimator $\widehat{\beta}_{0EB}$ is then given by

$$\widehat{\beta}_{0EB} = \frac{\widehat{\alpha}_{0EB}}{\beta \bar{x}}. \tag{5.18}$$

As it is seen, theoretically, denominator in the RHS of (5.16) is zero. This means that $\widehat{\alpha}_{0EB}$ must be large. Moreover, the absolute sign is used in denominator of (5.17) to avoid negative value for $\widehat{\alpha}_{0EB}$. Figure 5.1 shows the visual comparison between prior and posterior distributions when sample evidence are generated from $\mathcal{G}(\alpha = 2, \beta = 2)$ as underlying distribution for two data sets of sizes $n = 50$ and $n = 100$. ■

5.3 Credible interval

One common type of inference in frequentist statistics is *confidence interval*. The Bayesian analogue of confidence interval is known as *credible interval* and defined as follows.

Definition 5.3.1. *Given the observed sample evidence $\mathbf{x} = \{x_1, \dots, x_n\}$, the interval (L, U) is a $100(1 - \gamma)\%$ credible interval for unknown parameter θ if*

$$P(L < \theta < U|\mathbf{x}) = \int_L^U \pi(\theta|\mathbf{x})d\theta \geq 1 - \gamma, \tag{5.19}$$

where $0 < \gamma < 1$.

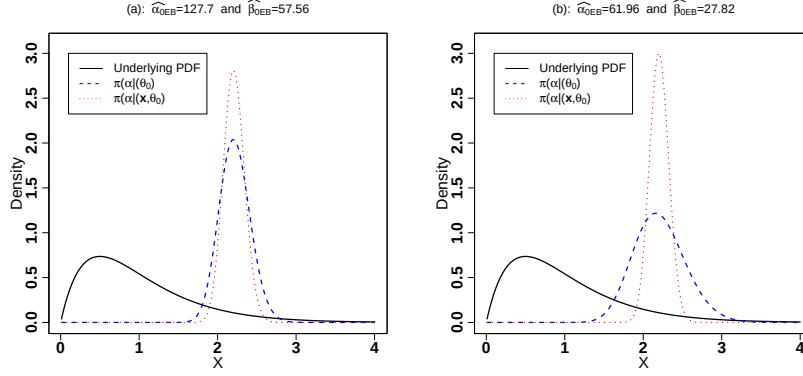


Figure 5.1: Schematic diagram for PDF of underlying, prior, and posterior when (a): $n = 50$ and (b): $n = 100$.

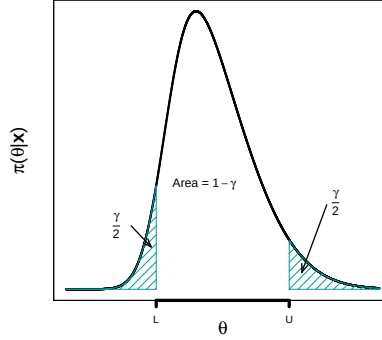


Figure 5.2: Schematic diagram for credible interval.

Indeed, Definition 5.2 states that there is at least $100(1 - \gamma)\%$ confidence that interval (L, U) includes unknown parameter θ . Figure 5.2 provides a schematic diagram for credible interval.

Example 5-4

Let $x_1, x_2, \dots, x_n$ come independently from $\mathcal{N}(\mu, \sigma^2)$ with known variance. By considering a conjugate prior $\mathcal{N}(\cdot | \mu_0, \sigma_0^2)$ for μ , we recall from (5.5) that $\mu | \mathbf{x}$ follows a Gaussian distribution. Hence

$$\sqrt{\frac{n}{\sigma^2} + \frac{1}{\sigma_0^2}} \left(\mu - \frac{n^{-1}\sigma^2\mu_0 + \sigma_0^2\bar{x}}{n^{-1}\sigma^2 + \sigma_0^2} \right) \sim \mathcal{N}(0, 1). \quad (5.20)$$

Therefore, $100(1 - \gamma)\%$ credible interval for μ is given by

$$\frac{n^{-1}\sigma^2\mu_0 + \sigma_0^2\bar{x}}{n^{-1}\sigma^2 + \sigma_0^2} \pm z_{\gamma/2} \sqrt{\frac{n}{\sigma^2} + \frac{1}{\sigma_0^2}}, \quad (5.21)$$

where $z_{\gamma/2}$ is the upper $\gamma/2$ -quantile of the standard Gaussian distribution, that is, $P(Z > z_{\gamma/2}) = \gamma/2$ in which $Z \sim \mathcal{N}(0, 1)$. ■

As it is seen from Example 5-4, for constructing credible interval, the upper and lower $\gamma/2$ of the posterior must be known. If the posterior gets complicated from the latter quantities cannot be found easily. In such cases, credible interval is constructed based on upper $\gamma/2$ -quantile of

the samples generated from the posterior. Let $\pi_{\gamma/2}$ denote the posterior's upper $\gamma/2$ -quantile, that is

$$\int_{\pi_{\gamma/2}}^{\infty} \pi(\theta|\mathbf{x})d\theta = \frac{\gamma}{2}. \quad (5.22)$$

Furthermore, suppose $\lceil u \rceil$ denotes the smallest integer value equal or greater than real value u and $\theta_{(r)}$ is r th order statistic of sample $\{\theta_1, \theta_2, \dots, \theta_N\}$ generated from posterior with PDF $\pi(\theta|\mathbf{x})$. When $N \rightarrow \infty$, then strong law of large numbers states that $\pi_{\gamma/2} \approx \theta_{(v)}$ where $v = \lceil (1 - \gamma/2)N \rceil$. The definition below gives the empirical credible interval.

Definition 5.3.2. Let $\{\theta_1, \theta_2, \dots, \theta_N\}$, for large N , denote generated samples form posterior with PDF $\pi(\theta|\mathbf{x})$. A $100(1 - \gamma)\%$ empirical credible interval for unknown parameter θ is then give by

$$(L_e, U_e) = (\theta_{(v)}, \theta_{(N-v)}), \quad (5.23)$$

where $v = \lceil (1 - \gamma/2)N \rceil$.

Example 5-5 (continued)

Recalling from Example 5-3, suppose we observed $n = 200$ observations, represented as $\mathbf{x} = \{x_1, x_2, \dots, x_{200}\}$ from $\mathcal{G}(\alpha = 2, \beta = 2)$. For known β , using the moment-based method described earlier in Example 5-3, we obtain $\boldsymbol{\theta}_0 = (\widehat{\alpha}_{0EB}, \widehat{\beta}_{0EB})^\top = (62.179, 30.083)^\top$. The corresponding posterior is given by

$$\begin{aligned} \pi(\alpha|\mathbf{x}, \boldsymbol{\theta}_0) &\propto \left[\prod_{i=1}^n \frac{\beta^\alpha x_i^{\alpha-1}}{\Gamma(\alpha)} \exp\{-\beta x_i\} \right] \frac{\beta_0^{\alpha_0} \alpha^{\alpha_0-1}}{\Gamma(\alpha_0)} \exp\{-\beta_0 \alpha\}, \\ &\propto \frac{\beta^{n\alpha} \alpha^{\alpha_0-1}}{[\Gamma(\alpha)]^n} \left[\prod_{i=1}^n x_i \right]^{\alpha-1} \exp\{-\beta_0 \alpha\}. \end{aligned} \quad (5.24)$$

In order to construct a 95% empirical credible interval for α , we need to sample from posterior (5.24) using methods such as MH or ARS. Taking into account into the fact that $\widehat{\alpha}_{0EB} = 62.179$ and it is simple to see that the posterior (5.24) is log-concave for $\alpha_0 \geq 1$. So, we proceed for simulating from this posterior using the ARS approach. Based on a sample of size 5000 generated from posterior (5.24), we obtain quantities $L_e = 1.778$, $\hat{\alpha}_B = 1.945$, and $U_e = 2.121$. The scatterplot of generated samples as well as the latter quantities are shown in Figure 5.3. The R code for constructing the 95% empirical credible interval for α is given below.

```

1 R> f.a <- function(x, beta, alpha0, beta0, y)
2 + {
3 +   n=length(y); n*x*log(beta)-n*lgamma(x)+(alpha0-1)*log(x)+
4 +   (x-1)*sum(y)-beta0*x
5 + }
6 R> fprim.a <- function(x, beta, alpha0, beta0, y)
7 + {
8 +   n=length(y); n*log(beta)-n*digamma(x)+(alpha0-1)/x+sum(y)-beta0
9 + }
10 R> library(ars)
11 R> set.seed(20240608)
12 R> n <- 200; alpha <-2; beta <- 2
13 R> y <- rgamma(n, shape = alpha, rate = beta)
14 R> alpha0 <- beta*mean(y)^2/abs( beta*var(y) - mean(y) )
15 R> beta0 <- alpha0/(beta*mean(y))
16 R> N <- 5000 # number of generations
17 R> M <- 0 # size of burn-in period
18 R> theta <- rep(0, N)

```

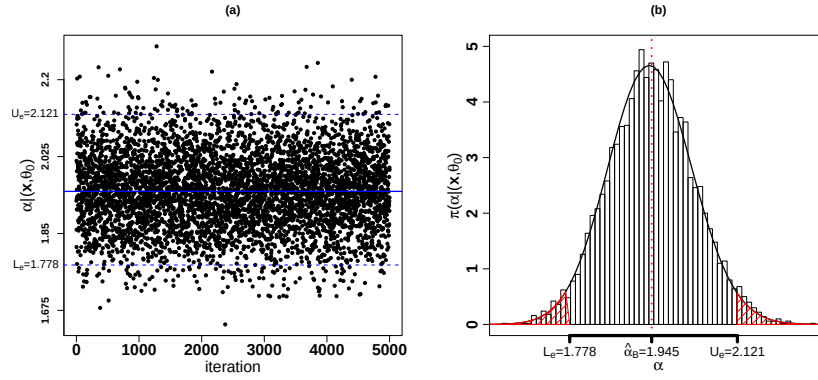


Figure 5.3: (a): the 95% empirical credible interval for parameter α based on a sample of $n = 200$ realizations generated from $\mathcal{G}(\alpha = 2, \beta = 2)$ distribution and (b): histogram of the generated samples with fitted posterior $\pi(\alpha|x, \theta_0)$.

```

19 R> for(j in 1:N)
20 + {
21 +   theta[j] <- ars(1, f.a, fprim.a, x = c(0.1, 4, 10, 30), m = 4,
22 +     lb = TRUE, xlb = 0, ub = FALSE, beta = beta, alpha0 = alpha0,
23 +     beta0 = beta0, y = log(y) )
24 + }
25 R> Le <- quantile(theta[(M+1):N], 0.025)[[1]]
26 R> Ue <- quantile(theta[(M+1):N], 0.975)[[1]]
27 R> alphahat_B <- mean(theta)
28 R> list("Lower_bound"=Le, "Bayes_estimator"=alphahat_B, "Upper_bound"=Ue)

```



6

Gibbs sampling

“Statistics is the science of information gathering, especially when the information arrives in little pieces instead of big ones.”

– Bradley Efron, Stanford University

This section has three parts. First, we describe the concept of Gibbs sampling as a general framework for generating independent samples from the given multivariate target distribution. Second, we discuss the situation that the Gibbs sampling is utilized to estimate the model’s unknown parameter(s) through the Bayesian paradigm.

6.1 Gibbs sampling in broad sense

The Gibbs sampling (or sampler) is an iterative Markov chain Monte Carlo (MCMC) sampling approach was developed by Geman and Geman [1984] for image reconstruction, but its application in statistical analyses were shown by Gelfand and Smith [1990]. It is noteworthy that sampling from a p -dimensional distribution using the methods such as MH or AMH discussed earlier may involve undesired computational burden. In such cases, the Gibbs sampling technique that works efficiently is becoming increasingly popular in a wide range of applications. Assuming that we are at t th iteration of the Gibbs sampling for generating $\mathbf{x} = (x_1, \dots, x_p)^\top$ from a p -dimensional target distribution with PDF $\pi(\cdot|\boldsymbol{\theta})$ ¹. For presentation purposes, we consider a set of generic symbols as follows.

$$\begin{aligned}\mathbf{x}^{(t)} &= (x_1^{(t)}, x_2^{(t)}, \dots, x_p^{(t)}), \\ \mathbf{x}_i^{(t)}[y] &= (x_1^{(t)}, \dots, x_{i-1}^{(t)}, y, x_{i+1}^{(t)}, \dots, x_p^{(t)}), \\ \mathbf{x}^{(t)}[-i] &= (x_1^{(t)}, \dots, x_{i-1}^{(t)}, x_{i+1}^{(t)}, \dots, x_p^{(t)}).\end{aligned}\tag{6.1}$$

Indeed, each Gibbs sampling scheme works by generating sample from conditional distribution of vector $\mathbf{x} = (x_1, \dots, x_p)^\top$ that is known in the literature as the *full conditional*. Hence, for generating $\mathbf{x}$ from target distribution with PDF $\pi(\cdot|\boldsymbol{\theta})$, we need to extract sample from p *full conditionals* each with PDF $\pi(x_i|\mathbf{x}[-i], \boldsymbol{\theta})$. It is proved [Geman and Geman, 1984] that, after *burn-in* period [Roberts and Smith, 1994], the generated sample comes from the target distribution whose PDF is $\pi(\cdot|\boldsymbol{\theta})$. The Gibbs sampling technique for simulating from a p -dimensional random vector with target distribution whose PDF is $\pi(\cdot|\boldsymbol{\theta})$ is summarized in Algorithm 16 algorithm.

Example 6-1

¹Herein it is assumed that the family parameter $\boldsymbol{\theta}$ is a vector of known constants. If elements of $\boldsymbol{\theta}$ are not known, then they can be regarded as a variable of interest to sample from within the Gibbs sampling scheme.

Algorithm 16 Gibbs sampling for simulating from multivariate Gaussian distribution

```
1: Set  $t = 0$ , read  $M$  (burn-in period),  $N$  (total number of iterations), and  $\boldsymbol{\theta}$ ;  
2: Set  $\mathbf{x}^{(t)} = (x_1^{(t)}, \dots, x_p^{(t)})^\top$  as the vector of initial sample;  
3: while  $t \leq N$  do  
4:   Set  $i = 1$ ;  
5:   while  $i \leq p$  do  
6:     Generate sample  $y_i$  from full conditional with PDF  $\pi(\cdot | \mathbf{x}^{(t)}[-i], \boldsymbol{\theta})$ ;  
7:     Set  $\mathbf{x}^{(t)}[i] \leftarrow y_i$ ;  
8:     Set  $i \leftarrow i + 1$ ;  
9:   end  
10:  Set  $\mathbf{x}^{(t)} \leftarrow \mathbf{x}^{(t)}$ ;  
11:  Set  $t \leftarrow t + 1$ ;  
12: end  
13: Sequence  $\{\mathbf{x}^{(M)}, \mathbf{x}^{(M+1)}, \dots, \mathbf{x}^{(N)}\}$  is a sample of size  $N - M + 1$  from target distribution  
14: with PDF  $\pi(\cdot | \boldsymbol{\theta})$ .
```

Let $\mathbf{X} = (X_1, X_2)^\top$ follows a bivariate Gaussian distribution with PDF

$$\pi(\mathbf{x} | \rho) = \frac{1}{2\pi\sqrt{1-\rho^2}} \exp\left\{-\frac{x_1^2 + x_2^2 - 2\rho x_1 x_2}{2(1-\rho^2)}\right\}. \quad (6.2)$$

where $\mathbf{x} = (x_1, x_2)^\top$ and $-1 < \rho < 1$. Since we have just two variables X_1 and X_2 , so for simulating from $\mathbf{X}$ using the Gibbs sampling technique, we need to simulate from full conditionals $X_1 | (X_2, \rho)$ and $X_2 | (X_1, \rho)$. It can be easily seen that

$$\pi(x_1 | x_2, \rho) \propto \exp\left\{-\frac{x_1^2 - 2\rho x_1 x_2}{2(1-\rho^2)}\right\}. \quad (6.3)$$

Since $x_1^2 - 2\rho x_1 x_2 = (x_1 - \rho x_2)^2 - (\rho x_2)^2$, then it follows that

$$\pi(x_1 | x_2, \rho) \propto \exp\left\{-\frac{(x_1 - \rho x_2)^2}{2(1-\rho^2)}\right\}. \quad (6.4)$$

This means that $X_1 | (X_2, \rho) \sim \mathcal{N}(\rho x_2, 1 - \rho^2)$. Likewise, we can write $X_2 | (X_1, \rho) \sim \mathcal{N}(\rho x_1, 1 - \rho^2)$. Hence, based on the Algorithm 16, for simulating a sample of $n = 3000$ observations we may proceed as follows.

1. Set $t = 0$ and generate $\mathbf{x}^{(t)} = (x_1^{(t)}, x_2^{(t)})^\top$ in which $x_1^{(t)} \sim \mathcal{N}(0, 1 - \rho^2)$ and $x_2^{(t)} \sim \mathcal{N}(0, 1 - \rho^2)$;
2. Simulate y_1 from the full conditional $\pi(x_1 | x_2^{(t)}, \rho) \sim \mathcal{N}(\rho x_2^{(t)}, 1 - \rho^2)$;
3. Having $x_1^{(t)} = y_1$, simulate y_2 from the full conditional $\pi(x_2 | y_1, \rho) \sim \mathcal{N}(\rho y_1, 1 - \rho^2)$;
4. $\mathbf{x}^{(t+1)} \leftarrow (y_1, y_2)^\top$ and $t \leftarrow t + 1$;
5. If $t = 5000$, then go to the next step, otherwise return to step 2;
6. Sequence $\{\mathbf{x}^{(2001)}, \mathbf{x}^{(2002)}, \dots, \mathbf{x}^{(5000)}\}$ constitutes a sample of size 3000 from target distribution with PDF given in (6.2).

The associated R code for generating 3000 realizations from $\mathcal{N}_2(\mu_1 = 0, \mu_2 = 0, \sigma_1^2 = 1, \sigma_2^2 = 1, \rho = 0.50)$ is given as follows.

```

1 R > set.seed(20240520)
2 R > N <- 5000 # number of generations
3 R > M <- 2000 # size of burn-in period
4 R > rho <- 0.5
5 R > X <- matrix(0, nrow = N, ncol = 2)
6 R > sigma1 <- sigma2 <- 1
7 R > Sigma <- matrix( c(1, rho*sigma1*sigma2, rho*sigma1*sigma2, 1), 2, 2)
8 R > X[1, ] <- rnorm(2, mean = 0, sd = 1 - rho^2) # initial generation
9 R > j <- 1
10 R > while(j < N)
11 + {
12 +   X[j + 1, 1] <- rnorm(1, mean = rho*sqrt(Sigma[1,1]/Sigma[2,2])*X[j, 2],
13 +   sd = 1 - rho^2)
14 +   X[j + 1, 2] <- rnorm(1, mean = rho*sqrt(Sigma[2,2]/Sigma[1,1])*X[j + 1, 1],
15 +   sd = 1 - rho^2)
16 +   j <- j + 1
17 + }
18 R > plot(X[(N-M+1):N,])

```

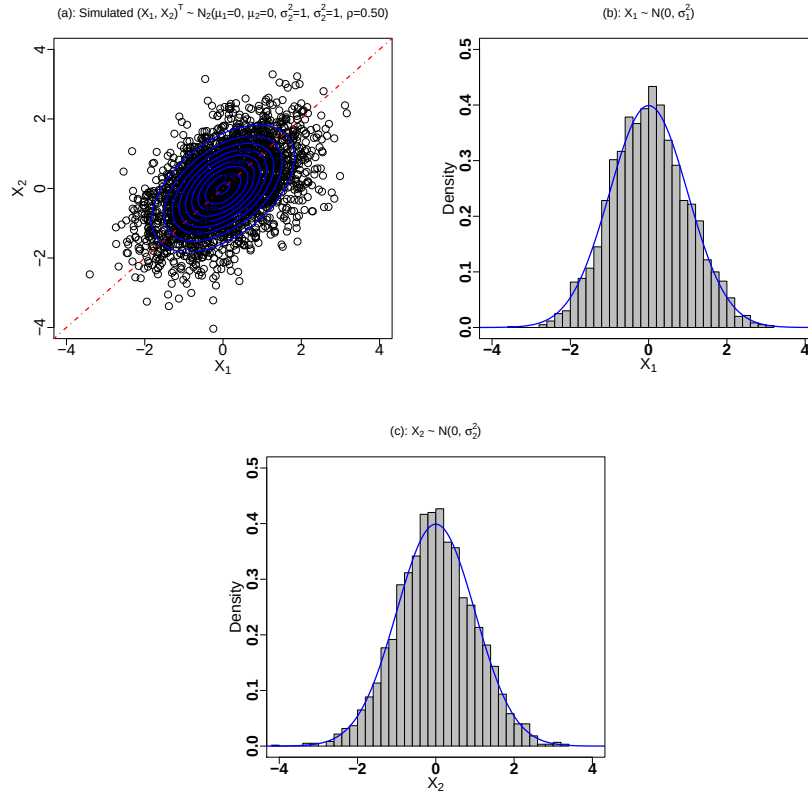


Figure 6.1: (a): Scatterplot with fitted contours based on 3000 samples generated from $\mathcal{N}_2(\mathbf{0}, \Sigma)$ in which $\Sigma = [(1, 0.5)^T, (0.5, 1)^T]$, (b): histogram of generated samples from marginal X_1 , and (c): histogram of generated samples from marginal X_2 .

6.2 Moments of truncated Gaussian distribution: Gibbs sampling scheme

The first two moments of the truncated multivariate Gaussian distribution, that is $\mathcal{N}_p(\boldsymbol{\mu}, \Sigma, \mathcal{R}^p)$, on region (4.398) has been discussed in Subsection 4.10. Herein, we are interested in dealing

with computing these quantities as well as the corresponding rectangular probability through the method of Gibbs sampling.

Recall from Example 2-8 that the marginals of $\mathcal{N}_p(\boldsymbol{\mu}, \Sigma, \mathcal{R}^p)$ are not necessarily truncated univariate Gaussian. In contrast, the full conditionals are truncated univariate Gaussian. Let $\mathbf{X} = (X_1, \dots, X_p)^\top \sim \mathcal{N}_p(\boldsymbol{\mu}, \Sigma, \mathcal{R}^p)$. It is not hard to check that

$$\pi(x_i | \mathbf{x}[-i], \boldsymbol{\mu}, \Sigma) \sim \mathcal{N}[\mu_{i|(-i)}(\mathbf{x})(\mathbf{x}[-i] - \boldsymbol{\mu}[-i]), \sigma_{i|(-i)}^2, \mathcal{R}_i], \quad (6.5)$$

where $\mathbf{x}[-i]$ is given in (4.400) and

$$\mu_{i|(-i)}(\mathbf{x}) = \Sigma_{i,-i}^\top \times \Sigma_{-i,-i}^{-1} \times \mathbf{x}, \quad (6.6)$$

$$\sigma_{i|(-i)}^2 = \Sigma_{i,i} - \Sigma_{i,-i}^\top \times \Sigma_{-i,-i}^{-1} \times \Sigma_{i,-i}, \quad (6.7)$$

$$\mathcal{R}_i = \{x \in \mathbb{R} | a_i \leq x \leq b_i\}, \quad (6.8)$$

in which a_i and b_i are the i th elements of vectors $\mathbf{a} = (a_1, \dots, a_p)^\top$ and $\mathbf{b} = (b_1, \dots, b_p)^\top$, respectively. In what follows, Algorithm 17 describes how to simulate from $\mathcal{N}(\boldsymbol{\mu}, \Sigma, \mathcal{R}^p)$.

Algorithm 17 Computing the moments of truncated multivariate Gaussian distribution

- 1: Set $t = 0$, read M (*burn-in* period), N (total number of iterations), $\boldsymbol{\mu}$, Σ , $\mathbf{a}$, and $\mathbf{b}$;
- 2: Set $\mathbf{x}^{(t)} = (x_1^{(t)}, \dots, x_p^{(t)})^\top$ as the vector of initial values in which $x_j^{(t)} = (a_j + b_j)/2$, for $j = 1, \dots, p$;
- 3: **while** $t \leq N$ **do**
- 4: Set $i = 1$;
- 5: **while** $i \leq p$ **do**
- 6: Generate sample y_i from full conditional with PDF

$$\pi(x_i | \mathbf{x}^{(t)}[-i], \boldsymbol{\mu}, \Sigma) \sim \mathcal{N}[\mu_{i|(-i)}(\mathbf{x}^{(t)}[-i] - \boldsymbol{\mu}[-i]), \sigma_{i|(-i)}^2, \mathcal{R}_i],$$

- 7: in which $\mu_{i|(-i)}(\mathbf{x})$, $\sigma_{i|(-i)}^2$, and $\mathcal{R}_i$ are given in (6.6), (6.7), and (6.8),
 - 8: respectively.
 - 9: Set $\mathbf{x}^{(t)}[i] \leftarrow y_i$;
 - 10: Set $i \leftarrow i + 1$;
 - 11: **end**
 - 12: Set $t \leftarrow t + 1$;
 - 13: **end**
 - 14: Sequence $\{\mathbf{x}^{(M)}, \mathbf{x}^{(M+1)}, \dots, \mathbf{x}^{(N)}\}$ is a sample of size $N - M + 1$ from $\mathcal{N}(\boldsymbol{\mu}, \Sigma, \mathcal{R}^p)$.
-

Example 6-2

Recall that Example 4-15 computes the CDF of trivariate Gaussian distribution. Herein, we are interested in computing $\Phi_3(\mathbf{a} | \mathbf{0}, R)$ where $R = [(1, 0.7, -0.2)^\top, (0.7, 1, -0.4)^\top, (-0.2, -0.4, 1)^\top]$ and $\mathbf{a} = (-1.2, -1, 0.5)^\top$. Using R function `rtnorm_gibbs(N, M, Mu, Sigma, a, b)` given as follows.

```

1 R>rtnorm_gibbs <- function(N, M, Mu, Sigma, a, b)
2 +{
3 + p <- length(Mu)
4 + k <- 5.32
5 + Z <- matrix(0, nrow = N, ncol = p)
6 + V <- matrix(0, nrow = p, ncol = p-1)
7 + threshold <- k*sqrt( diag(Sigma) )

```

```

8 + for(i in 1:p)
9 + {
10 +   if( is.infinite( a[i] ) ) a[i] <- Mu[i] - threshold[i]
11 +   if( is.infinite( b[i] ) ) b[i] <- Mu[i] + threshold[i]
12 +   if( is.infinite( a[i] ) & a[i] >= b[i] ) a[i] <- b[i] - threshold[i]
13 +   if( is.infinite( b[i] ) & a[i] >= b[i] ) b[i] <- a[i] + threshold[i]
14 +   V[i, ] <- as.numeric( Sigma[i, -i]%%solve( Sigma[-i, -i] ) )
15 + }
16 + U <- matrix( cbind(a - Mu, b - Mu), nrow = p, ncol = 2)
17 + Z[1, ] <- (a + b)/2
18 + j <- 1
19 + while(j < N)
20 + {
21 +   u <- runif(p)
22 +   for(i in 1:p)
23 +   {
24 +     mu_i_minus_i <- V[i, ]%%Z[j, -i]
25 +     sigma_i_minus_i <- sqrt( (Sigma[i, i] - V[i, ]%%Sigma[i, -i])[[1]] )
26 +     Phi_a <- pnorm(U[i, 1], mu_i_minus_i, sigma_i_minus_i)
27 +     Phi_b <- pnorm(U[i, 2], mu_i_minus_i, sigma_i_minus_i)
28 +     Z[j, i] <- qnorm( u[i]*(Phi_b - Phi_a) + Phi_a,
29 +       mu_i_minus_i, sigma_i_minus_i )
30 +     Z[j+1, i] <- Z[j, i]
31 +   }
32 +   j <- j + 1
33 + }
34 + Z <- matrix(Mu, nrow = N, ncol = p, byrow = T) + Z
35 + return( Z[(M+1):N], )
36 +}

```

Setting $R = \Sigma$ and $\mathbf{b} = (+\infty, +\infty, +\infty)^\top$, we can easily compute the first moment of truncated Gaussian distribution by running R command as

```

R> set.seed(20251020)
R> N <- 10000; M <- 1500; a <- c(-1.2, -1, 0.5); b <- rep(Inf, 3)
R> v <- c(1, 0.7, -0.2, 0.7, 1, -0.4, -0.2, -0.4, 1)
R> Mu <- rep(0, 3); Sigma <- matrix(v, nrow = 3, ncol = 3)
R> x <- rtnorm_gibbs(N, M, Mu, Sigma, a, b) # dim x is 8500 * 3
R> apply(x, 2, mean)
R> EX(Mu, Sigma, a, b)

```

The result, based on 8500 realizations, would be (0.20568623, 0.02342306, 1.08851900) that is close enough to the exact value (0.20067270, 0.02071517, 1.08395808) obtained through function `EX(Mu, Sigma, a, b)` given earlier. It is worth to mention that in the last line of function `EX(Mu, Sigma, a, b)`, we prefer to use function `Spec_quad(a, b, Mu, Sigma, 30)` rather than function `pmvnorm(a, b, mean=Mu, sigma=Sigma)` to avoid calling package `mvtnorm` that cannot yield an exact evaluation for $\mathcal{P}_G$ given in (4.403). ■

6.3 Gibbs sampling in Bayesian paradigm

Herein, we show how a Gibbs sampling technique described in the previous section can be used for computing the Bayesian estimator of the unknown parameter $\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)^\top$. Notice that, based on random sample $\{x_1, \dots, x_n\}$ following a distribution with PDF $f(\cdot|\boldsymbol{\theta})$, the Bayes

estimator of $\boldsymbol{\theta}$ usually becomes the mean² of the posterior defined as

$$E[\boldsymbol{\theta}|\mathbf{x}] = \int_{\mathbb{R}^p} \boldsymbol{\theta} \pi(\boldsymbol{\theta}|\mathbf{x}) d\boldsymbol{\theta}. \quad (6.9)$$

Computing integration (6.9) may involve huge computational burden especially when number of parameters p , is large. The law of large numbers guarantees that the sample mean converges with probability one to the population mean when sample size is sufficiently large, that is

$$\hat{\boldsymbol{\theta}} = \lim_{N \rightarrow \infty} \frac{\boldsymbol{\theta}^{(1)} + \dots + \boldsymbol{\theta}^{(N)}}{N} = E[\boldsymbol{\theta}|\mathbf{x}].$$

Hence, if we could generate sample enough from the posterior $\pi(\boldsymbol{\theta}|\mathbf{x})$, then $\boldsymbol{\theta}$ can be estimated via $\hat{\boldsymbol{\theta}}$ in which $\{\boldsymbol{\theta}^{(1)}, \dots, \boldsymbol{\theta}^{(N)}\}$ is generated from posterior $\pi(\boldsymbol{\theta}|\mathbf{x})$ when N is sufficiently large (here $\boldsymbol{\theta}^{(i)}$ denotes the i th sample generated from posterior $\pi(\boldsymbol{\theta}|\mathbf{x})$). Generating sample from the target distribution (that is here $\pi(\boldsymbol{\theta}|\mathbf{x})$) using the MCMC techniques described in the previous section appears to be challenging when p is large. In such cases, the Gibbs sampling is guaranteed to work well. It should be noted that since in each Bayesian framework the unknown parameter $\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)^\top$ itself is a random variable, in addition of the model's variables, herein we must consider an extra number of p additional full conditionals within a Gibbs sampler schemes. For instance, recalling from the Example (3), we must consider three full conditionals given by variables $X_1|X_2, \rho$, $X_2|X_1, \rho$, and $\rho|X_1, X_2$. In what follows, we consider the foregoing example when Bayesian inference on ρ is desired.

Example 6-3

Let $\underline{\mathbf{x}} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ account for a sequence of n independent observations in which $\mathbf{x}_i = (x_{i1}, x_{i2})^\top$. Furthermore, assume $\mathbf{x}_i$ (for $i = 1, 2, \dots, n$) follows a bivariate Gaussian distribution with PDF given by

$$\pi(\mathbf{x}_i|\boldsymbol{\theta}) = \frac{1}{2\pi\sigma_1\sigma_2\sqrt{1-\rho^2}} \exp\left\{-\frac{1}{2(1-\rho^2)}\left[\left(\frac{x_{i1}}{\sigma_1}\right)^2 + \left(\frac{x_{i2}}{\sigma_2}\right)^2 - 2\rho\frac{x_{i1}x_{i2}}{\sigma_1\sigma_2}\right]\right\}, \quad (6.10)$$

where $\boldsymbol{\theta} = (\sigma_1, \sigma_2, \rho)^\top$ is the unknown parameter vector. Using a reparameterization $\boldsymbol{\theta} = (\theta_1, \theta_2, \theta_3)^\top$ in which $\theta_1 = \sigma_1/\sigma_2$, $\theta_2 = \sigma_1\sigma_2\sqrt{1-\rho^2}$, and $\theta_3 = \rho$, the bivariate Gaussian PDF is rewritten as by

$$\pi(\mathbf{x}_i|\boldsymbol{\theta}) = \frac{1}{2\pi\theta_2} \exp\left\{-\frac{1}{2(1-\theta_3^2)^{1/2}\theta_2}\left[\frac{x_{i1}^2}{\theta_1} + \theta_1 x_{i2}^2 - 2\theta_3 x_{i1}x_{i2}\right]\right\}. \quad (6.11)$$

For computing the Bayesian estimator of $\boldsymbol{\theta}$, we first note that

$$\pi(\rho|\underline{\mathbf{x}}) \propto \prod_{i=1}^n \pi(\mathbf{x}_i|\boldsymbol{\theta}) \pi(\boldsymbol{\theta}). \quad (6.12)$$

The marginals of joint prior $\pi(\boldsymbol{\theta})$ are assumed to be independent, that is, $\pi(\boldsymbol{\theta}) = \pi(\theta_1)\pi(\theta_2)\pi(\theta_3)$. Also, we consider gamma prior for both of θ_1 , inverse gamma prior for θ_2 , and an uniform on $(-1,1)$ prior for θ represented, respectively, as follows.

$$\begin{aligned} \pi(\theta_1|a_1, b_1) &= \frac{b_1^{a_1} \theta_1^{a_1-1}}{\Gamma(a_1)} \exp\{-b_1 \theta_1\}, \quad \theta_1 > 0, \\ \pi(\theta_2|a_2, b_2) &= \frac{b_2^{a_2} \theta_2^{-a_2-1}}{\Gamma(a_2)} \exp\{-\frac{b_2}{\theta_2}\}, \quad \theta_2 > 0, \\ \pi(\theta_3) &= \frac{1}{2}, \quad -1 < \theta_3 < 1. \end{aligned} \quad (6.13)$$

²Statistically speaking, the form of Bayesian estimator depends on the type of the loss function. For example, if the loss function takes either *square* or *absolute* form, then the Bayesian estimator is *mean* or *median* of posterior, respectively. But, the most commonly used summary representing the central tendency such as mean, median, or mode of the posterior would be considered as the Bayesian estimator.

We note that, here we consider above priors just for the simplicity of computations. It follows that

$$\begin{aligned} \pi(\boldsymbol{\theta}|\underline{\mathbf{x}}) &\propto \frac{1}{\theta_2^n} \exp\left\{-\frac{1}{2\theta_2\sqrt{1-\theta_3^2}}\left[\frac{\sum_{i=1}^n x_{i1}^2}{\theta_1} + \theta_1 \sum_{i=1}^n x_{i2}^2 - 2\theta_3 \sum_{i=1}^n x_{i1}x_{i2}\right]\right\} \\ &\times \frac{b_1^{a_1}\theta_1^{a_1-1}}{\Gamma(a_1)} \exp\{-b_1\theta_1\} \times \frac{b_2^{a_2}\theta_2^{-a_2-1}}{\Gamma(a_2)} \exp\left\{-\frac{b_2}{\theta_2}\right\} \times \frac{1}{2} \end{aligned} \quad (6.14)$$

We note that, herein, the model's variables are θ_1 , θ_2 , and θ_3 . So, by dropping out all terms that do not depend on θ_1 from the posterior (6.14), then the full conditional $\theta_1|(\theta_2, \theta_3, \underline{\mathbf{x}})$, after some algebra, is shown to be³

$$\pi(\theta_1|\theta_2, \theta_3, \underline{\mathbf{x}}) \propto \theta_1^{-a_1-1} \exp\left\{-\frac{\theta_1}{2}\left[2b_1 + \frac{\sum_{i=1}^n x_{i2}^2}{\theta_2(1-\theta_3)^{1/2}}\right] - \frac{1}{2\theta_1}\left[\frac{\sum_{i=1}^n x_{i1}^2}{\theta_2(1-\theta_3)^{1/2}}\right]\right\}. \quad (6.15)$$

Comparing the RHSs of (6.15) and (2.51), it turns out that (6.15) is the pseudo PDF corresponds to a $\mathcal{GIG}(a, b, c)$ distribution where $a = a_1$, $b = 2b_1 + \sum_{i=1}^n x_{i2}^2/[\theta_2(1-\theta_3)^{1/2}]$, $c = \sum_{i=1}^n x_{i1}^2/[\theta_2(1-\theta_3)^{1/2}]$. So,

$$\theta_1|(\theta_2, \theta_3, \underline{\mathbf{x}}) \sim \mathcal{GIG}\left(a_1, 2b_1 + \sum_{i=1}^n x_{i2}^2/[\theta_2(1-\theta_3)^{1/2}], \sum_{i=1}^n x_{i1}^2/[\theta_2(1-\theta_3)^{1/2}]\right). \quad (6.16)$$

Likewise, we can see that the full conditional $\theta_2|(\theta_1, \theta_3, \underline{\mathbf{x}})$ follows an inverse gamma distribution. We have

$$\begin{aligned} \theta_2|(\theta_1, \theta_3, \underline{\mathbf{x}}) &\sim \mathcal{IG}\left(n + a_2, b_2 + \frac{1}{2\theta_2(1-\theta_3)^{1/2}}\right. \\ &\times \left.\left[\theta_1^{-1} \sum_{i=1}^n x_{i1}^2 + \theta_1 \sum_{i=1}^n x_{i2}^2 - 2\theta_3 \sum_{i=1}^n x_{i1}x_{i2}\right]\right), \end{aligned} \quad (6.17)$$

and finally

$$\theta_3|(\theta_1, \theta_2, \underline{\mathbf{x}}) \propto \exp\left\{-\frac{1}{2\theta_2\sqrt{1-\theta_3^2}}\left[\theta_1^{-1} \sum_{i=1}^n x_{i1}^2 + \theta_1 \sum_{i=1}^n x_{i2}^2 - 2\theta_3 \sum_{i=1}^n x_{i1}x_{i2}\right]\right\}. \quad (6.18)$$

As it is seen, the full conditional $\theta_3|(\theta_1, \theta_2, \underline{\mathbf{x}})$ has no closed form. Hence, we can use the MH within Gibbs sampling technique. Recall from Subsection ??, considering a symmetric uniform proposal on $(-1, 1)$, the transition probability $p(\theta_{3(n)} \rightarrow \theta_{3(n+1)}) = \min\{1, \exp\{R_1 - R_2\}\}$, is

$$\begin{aligned} R_1 &= -\frac{1}{2\theta_2\sqrt{1-\theta_{3(n+1)}^2}}\left[\theta_1^{-1} \sum_{i=1}^n x_{i1}^2 + \theta_1 \sum_{i=1}^n x_{i2}^2 - 2\theta_{3(n+1)} \sum_{i=1}^n x_{i1}x_{i2}\right] \\ R_2 &= -\frac{1}{2\theta_2\sqrt{1-\theta_{3(n)}^2}}\left[\theta_1^{-1} \sum_{i=1}^n x_{i1}^2 + \theta_1 \sum_{i=1}^n x_{i2}^2 - 2\theta_{3(n)} \sum_{i=1}^n x_{i1}x_{i2}\right] \end{aligned} \quad (6.19)$$

In what follows, we give the steps for computing Bayesian estimation of $\boldsymbol{\theta}$.

1. Read N, M, a_1, b_1, a_2, b_2 , and suggest $\boldsymbol{\theta}^{(0)} = (\theta_1^{(0)}, \theta_2^{(0)}, \theta_3^{(0)})^\top$ arbitrarily;
2. Set $t = 0$;

³It is noteworthy that $\underline{\mathbf{x}}$ has been observed and we no longer look at it as variable. So, the model's variables are θ_1 , θ_2 , and θ_3 .

3. Simulate $\theta_1^{(t+1)}$ from the full conditional $\theta_1^{(t+1)} | (\theta_2^{(t)}, \theta_3^{(t)}, \underline{x})$ that follows

$$\mathcal{GIG}\left(a_1, 2b_1 + \sum_{i=1}^n x_{2i}^2 / [\theta_2^{(t)}(1 - \theta_3^{(t)})^{1/2}], \sum_{i=1}^n x_{1i}^2 / [\theta_2^{(t)}(1 - \theta_3^{(t)})^{1/2}]\right);$$

4. Having $\theta_1^{(t+1)}$, simulate $\theta_2^{(t+1)}$ from the full conditional $\theta_2^{(t+1)} | (\theta_1^{(t+1)}, \theta_3^{(t)}, \underline{x})$ that follows

$$\mathcal{GIG}\left(n + a_2, 2b_2, \left[\frac{1}{\theta_1^{(t+1)}} \sum_{i=1}^n x_{i1}^2 + \theta_1^{(t+1)} \sum_{i=1}^n x_{i2}^2 - 2\theta_3^{(t)} \sum_{i=1}^n x_{i1}x_{i2}\right] \left[\theta_2^{(t)}(1 - \theta_3^{(t)})^{1/2}\right]^{-1}\right);$$

5. Having $\theta_1^{(t+1)}$ and $\theta_2^{(t+1)}$, simulate $\theta_3^{(t+1)}$ from the full conditional $\theta_3^{(t+1)} | (\theta_2^{(t+1)}, \theta_3^{(t)}, \underline{x})$ using MH-within-Gibbs sampling technique as follows.

- (a) Set $k = 0$, $K = 100$, and choose the initial state as $y_0 \sim \mathcal{U}(-1, 1)$;
- (b) Generate $y^{(k+1)} \sim \mathcal{U}(-1, 1)$
- (c) Compute $p(y^{(k)} \rightarrow y^{(k+1)}) = \min\{1, \exp\{R_1 - R_2\}\}$ where

$$R_1 = -\frac{1}{2\theta_2^{(t+1)}\sqrt{1 - (y^{(k+1)})^2}} \left[\frac{1}{\theta_1^{(t+1)}} \sum_{i=1}^n x_{i1}^2 + \theta_1 \sum_{i=1}^n x_{i1}^2 - 2y^{(k+1)} \sum_{i=1}^n x_{i1}x_{i2} \right]$$

and

$$R_2 = -\frac{1}{2\theta_2^{(t+1)}\sqrt{1 - (y^{(k)})^2}} \left[\frac{1}{\theta_1^{(t+1)}} \sum_{i=1}^n x_{i1}^2 + \theta_1^{(t+1)} \sum_{i=1}^n x_{i2}^2 - 2y^{(k)} \sum_{i=1}^n x_{i1}x_{i2} \right]$$

- (d) Generate $u \sim \mathcal{U}(0, 1)$;
- (e) If $u < p(y^{(k)}, y^{(k+1)})$ then $y^{(k)} \rightarrow y^{(k+1)}$ and $k \rightarrow k + 1$;
- (f) Repeat algorithm from step (b);
- (g) If $k = K$ then $y^{(k+1)} \rightarrow \theta_3^{(t+1)}$ and stop the algorithm.

6. If $t = N$, then go to the next step, otherwise $t + 1 \rightarrow t$ and return step 3;

7. The Bayesian estimator $\hat{\theta}_B$, is the average of $N - M + 1$ samples, that is, $\theta^{(M+1)}, \theta^{(M+2)}, \dots, \theta^{(M+N)}$ generated from target PDF $\pi(\theta | \underline{x})$.

We note that, in practice, hyperparameters a_1, b_1, a_2 and b_2 are determined based on experimenter's belief or the empirical Bayes. If sample size is small, then the improper Jeffreys *prior* is suggested. It is noteworthy that for simulating from GIG distribution, one can use the method proposed by Hörmann and Leydold [2014]. ■

The pertaining R code for implementing example above is given as follows. To this end, a sample of $N = 5000$ realizations have been generated from a zero-mean bivariate Gaussian when $\sigma_1 = 0.5$, $\sigma_2 = 1$, and $\rho = -0.75$. The output of the Gibbs sampler is displayed in Figure 6.2.

```
1 R > set.seed(20240522)
2 R > library(MASS)
3 R > library(GIGrvg)
4 R > n <- 5000 # sample size
5 R > Mu <- c(0, 0)
```

```

6 R > X <- matrix(0, nrow = n, ncol = 2)
7 R > sigma1 <- 1; sigma2 <- .5; rho <- -0.75
8 R > Sigma <- matrix( c(sigma1^2,rho*sigma1*sigma2,rho*sigma1*sigma2,sigma2^2), 2, 2)
9 R > X <- mvrnorm(n, Mu, Sigma)
10 R > n.sim <- 5000 # number of Gibbs sampling iterations
11 R > n.burn <- 2000 # length of burn-in period
12 R > K <- 100      # number of MH algorithm iterations
13 R > theta3i <- rep(0, K)
14 R > a1 <- 0.5; b1<- 0.5; a2 <- 0.5; b2 <- 0.5;
15 R > theta <- matrix(0, nrow = n.sim, ncol = 3)
16 R > theta[1, ] <- c(1, 1, 0.75)
17 R > sx1 <- sum(X[, 1]^2); sx2 <- sum(X[, 2]^2); sx12 <- sum(X[, 1]*X[, 2])
18 R > j <- 1
19 R > while(j < n.sim)
20 + {
21 +   c0 <- ( theta[j, 2]*sqrt( 1 - theta[j, 3]^2 ) )
22 +   rate1 <- function(x) (sx1/theta[j, 1] + theta[j, 1]*sx2 - 2*x*sx12)/sqrt(1-x^2)
23 +   theta[j + 1, 1] <- rgig(n = 1, lambda = a1, chi = sx1/c0, psi = 2*b1 + sx2/c0 )
24 +   theta[j + 1, 2] <- 1/rgamma(n=1, shape=n + a2, rate=rate1(theta[j, 3])/2 + b2 )
25 # start of MH-within-Gibbs sampling for sampling
26 # from full conditional of theta3
27 +   theta3i[1] <- runif(1, -1, 1)
28 +   for(k in 2:K)
29 +   {
30 +     theta3.n <- runif(1, -1, 1)
31 +     dif <- -( rate1(theta3.n) - rate1(theta3i[k - 1]) )/(2*theta[j + 1, 2])
32 +     if( runif(1) < exp( dif ) )
33 +     {
34 +       theta3i[k] <- theta3.n # theta3.n is accepted
35 +     }else{
36 +       theta3i[k] <- theta3i[ k - 1] # theta3.n is rejected
37 +     }
38 +   }
39 # start of MH-within-Gibbs sampling for sampling
40 # from full conditional of theta3
41 +   theta[j + 1, 3] <- theta3i[k]
42 +   j <- j + 1
43 + }
44 R > theta.hat <- apply(theta[(n.sim - n.burn + 1):n.sim, ], 2, mean)
45 R > rho <- theta.hat[3] #estimator of rho
46 R > sigma1 <- sqrt(theta.hat[1]*theta.hat[2]/sqrt(1-rho.hat^2)) #estimator of sigma1
47 R > sigma2 <- sigma1.hat/theta.hat[1] #estimator of sigma2

```

Example 6-4

Suppose response variable Y depends linearly on independent variable (covariate) X through a simple linear model as

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i; \quad i = 1, 2, 3, \dots, n. \quad (6.20)$$

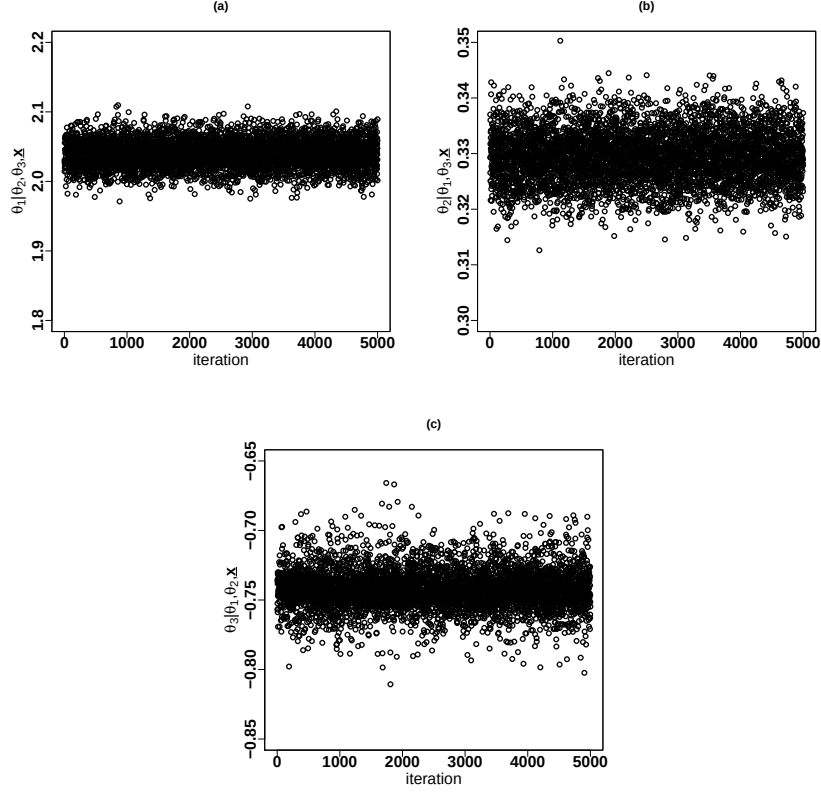


Figure 6.2: Scatterplot of 5000 samples generated from full conditionals: (a) $\theta_1 | (\theta_2, \theta_3, \mathbf{x})$, (b): $\theta_2 | (\theta_1, \theta_3, \mathbf{x})$, and (c): $\theta_3 | (\theta_1, \theta_2, \mathbf{x})$.

Recording n measurements on Y represented as $y_1, y_2, y_3, \dots, y_n$ at the corresponding levels of covariate X shown by $x_1, x_2, \dots, x_n$, the focus is placed on estimating the regression coefficients β_0 and β_1 through the Bayesian approach. Herein, ϵ_i s are the simple linear model's theoretical errors, independently (that is, $\text{cov}(\epsilon_i, \epsilon_j) = 0$ for $i \neq j = 1, 2, 3, \dots, n$) assumed to follow $\mathcal{N}(0, \sigma^2)$. Of course the skewed Student's t [Teimouri et al., 2020] and α -stable [Nolan and Ojeda-Revah, 2013] may be reasonable candidates for distribution of theoretical error in presence of outliers that yield more robust estimation of $\beta = (\beta_0, \beta_1)^\top$ than the Gaussian one. It is known that the least-square (LS) estimator $\hat{\beta}_{ls}$ and ML estimator $\hat{\beta}_{ml}$, of the regression coefficients β are the same and given by

$$\begin{aligned}\hat{\beta}_{1ML} &= \hat{\beta}_{1LS} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \\ \hat{\beta}_{0ML} &= \hat{\beta}_{0LS} = \bar{y} - \hat{\beta}_{1ML} \times \bar{x},\end{aligned}\tag{6.21}$$

where $\bar{x} = (x_1 + x_2 + \dots + x_n)/n$. Assuming independent Gaussian priors for elements of β and a mutually independent conjugate inverse gamma prior for $\delta = \sigma^2$ as

$$\begin{aligned}\pi(\beta_0) &= \mathcal{N}(\beta_0 | \mu_0, \sigma_0^2), \\ \pi(\beta_1) &= \mathcal{N}(\beta_1 | \mu_1, \sigma_1^2), \\ \pi(\delta) &= \mathcal{IG}\left(\delta \middle| \frac{a_0}{2}, \frac{b_0}{2}\right).\end{aligned}$$

It follows that

$$\begin{aligned}\pi(\boldsymbol{\theta}|\mathbf{y}, \mathbf{x}) &= L(\boldsymbol{\theta}|\mathbf{y}, \mathbf{x})\pi(\beta_0)\pi(\beta_1)\pi(\delta) \\ &\propto \frac{1}{\delta^{\frac{n}{2}}} \exp\left\{-\frac{1}{2\delta} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2\right\} \frac{1}{\sqrt{2\pi\sigma_0^2}} \exp\left\{-\frac{(\beta_0 - \mu_0)^2}{2\sigma_0^2}\right\} \\ &\quad \times \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp\left\{-\frac{(\beta_1 - \mu_1)^2}{2\sigma_1^2}\right\} \left(\frac{b_0}{2}\right)^{\frac{a_0}{2}} \frac{\delta^{-\frac{a_0}{2}-1}}{\Gamma(\frac{a_0}{2})} \exp\left\{-\frac{b_0}{2\delta}\right\},\end{aligned}\quad (6.22)$$

where $\delta = \sigma^2$ and $\boldsymbol{\theta} = (\beta_0, \beta_1, \delta)^\top$. Then more algebra shows

$$\pi(\beta_0|\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\beta_0)}) = \mathcal{N}\left(\beta_0 \middle| D_0 \left[\frac{\mu_0}{\sigma_0^2} + \frac{\sum_{i=1}^n (y_i - \beta_1 x_i)}{\sigma^2} \right], D_0\right), \quad (6.23)$$

and

$$\pi(\beta_1|\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\beta_1)}) = \mathcal{N}\left(\beta_1 \middle| D_1 \left[\frac{\mu_1}{\sigma_1^2} + \frac{\sum_{i=1}^n x_i (y_i - \beta_0)}{\sigma^2} \right], D_1\right), \quad (6.24)$$

where $D_0 = \sigma_0^2 \delta / (n\sigma_0^2 + \delta)$ and $D_1 = \sigma_1^2 \delta / (\sigma_1^2 \sum_{i=1}^n x_i^2 + \delta)$. Furthermore, by considering transformation $\delta = \sigma^2$, it can be seen easily that

$$\pi(\delta|\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\beta)}) = \mathcal{IG}\left(\delta \middle| \frac{a_0 + n}{2}, \frac{a_0 + \sum_{i=1}^n (y_i - x_i \beta)^2}{2}\right). \quad (6.25)$$

Applying a monotone transformation such used as above is a common trick in each Bayesian paradigm to yield a conjugate prior. Finally, the simple reverse transformation gives back the Bayesian estimator of σ . There follows the steps for computing Bayesian estimation of $\boldsymbol{\theta}$.

1. Read N, M , determine hyperparameters $\mu_0, \mu_1, \sigma_0, \sigma_1, a_0, b_0$, and propose $\boldsymbol{\theta}^{(0)} = (\beta_0^{(0)}, \beta_1^{(0)}, \delta^{(0)})^\top$;
2. Set $t = 0$;
3. Simulate $\beta_0^{(t+1)}$ from the full conditional $\beta_0|(\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\beta_0)})$ that follows

$$\mathcal{N}\left(D_0^{(t)} \left[\frac{\mu_0}{\sigma_0^2} + \frac{\sum_{i=1}^n (y_i - \beta_1^{(t)} x_i)}{\delta^{(t)}} \right], D_0^{(t)}\right),$$

where $D_0^{(t)} = \sigma_0^2 \delta^{(t)} / (n\sigma_0^2 + \delta^{(t)})$.

4. Having $\beta_0^{(t+1)}$, simulate $\beta_1^{(t+1)}$ from the full conditional $\beta_1|(\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\beta_1)})$ that follows

$$\mathcal{N}\left(D_1^{(t)} \left[\frac{\mu_1}{\sigma_1^2} + \frac{\sum_{i=1}^n x_i (y_i - \beta_0^{(t+1)})}{\delta^{(t)}} \right], D_1^{(t)}\right),$$

where $D_1^{(t)} = \sigma_1^2 \delta^{(t)} / (\sigma_1^2 \sum_{i=1}^n x_i^2 + \delta^{(t)})$.

5. Having $\boldsymbol{\beta}^{(t+1)}$, simulate $\delta^{(t+1)}$ from the full conditional $\delta|(\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\delta)})$ that follows

$$\mathcal{IG}\left(\delta \middle| \frac{a_0 + n}{2}, \frac{b_0 + \sum_{i=1}^n (y_i - \beta_0^{(t+1)} - x_i \beta_1^{(t+1)})^2}{2}\right).$$

6. If $t = N$, then go to the next step. Otherwise set $t + 1 \rightarrow t$ and return to step 3;

7. The average of sequence $\{\boldsymbol{\theta}^{(M+1)}, \boldsymbol{\theta}^{(M+2)}, \dots, \boldsymbol{\theta}^{(M+N)}\}$ sampled from target distribution with PDF $\pi(\boldsymbol{\theta}|\mathbf{y}, \mathbf{x})$ is the Bayesian estimator $\hat{\boldsymbol{\theta}}_B = (\hat{\beta}_{0B}, \hat{\beta}_{1B}, \hat{\sigma}_B)^\top$.

Herein, we set the hyperparameters $\{\mu_0, \mu_1\}$ are supposed to be $\widehat{\boldsymbol{\beta}}_{ml}$ and also, we set $\sigma_0 = 1$, $\sigma_1 = 1$, and $\sigma_2 = 1$. Furthermore, a_0 and b_0 are determined through the empirical Bayes as follows.

$$\begin{aligned}
m(\mathbf{y}|a_0, b_0) &= \prod_{i=1}^n m(y_i|a_0, b_0) \\
&= \prod_{i=1}^n \int_0^\infty f(y_i|\delta) \pi(\delta|a_0, b_0) d\delta \\
&= \prod_{i=1}^n \int_0^\infty \frac{\delta^{-\frac{1}{2}}}{\sqrt{2\pi}} \exp\left\{-\frac{1}{2\delta}(y_i - \beta_0 - \beta_1 x_i)^2\right\} \\
&\quad \times \left(\frac{b_0}{2}\right)^{\frac{a_0}{2}} \delta^{-\frac{a_0}{2}-1} \Gamma^{-1}\left(\frac{a_0}{2}\right) \exp\left\{-\frac{b_0}{2\delta}\right\} d\delta \\
&= \prod_{i=1}^n \left(\frac{b_0}{2}\right)^{\frac{a_0}{2}} \int_0^\infty \frac{\delta^{-\frac{a_0+1}{2}-1}}{\sqrt{2\pi} \Gamma\left(\frac{a_0}{2}\right)} \exp\left\{-\frac{1}{2\delta}[(y_i - \beta_0 - \beta_1 x_i)^2 + b_0]\right\} d\delta \\
&= \prod_{i=1}^n \frac{\Gamma\left(\frac{a_0+1}{2}\right)}{\sqrt{b_0} \pi \Gamma\left(\frac{a_0}{2}\right)} \left[1 + \frac{(y_i - \beta_0 - \beta_1 x_i)^2}{b_0}\right]^{-\frac{a_0+1}{2}}, \tag{6.26}
\end{aligned}$$

where $f(y_i|\delta) = \mathcal{N}(y_i|\beta_0 + \beta_1 x_i, \delta = \sigma^2)$. Further, taking into account the fact that if $a_0 = b_0$, the RHS of (6.26) becomes $\prod_{i=1}^n t(y_i|\beta_0 + \beta_1 x_i, b_0, a_0)$. Assuming $\boldsymbol{\beta}$ is known, the ML estimator of a_0 and b_0 is obtained by maximizing the RHS of (6.26) based on the sequence of n observed data $\{y_i - \beta_0 - \beta_1 x_i\}_{i=1}^n$. Moreover, the hyperparameters can be estimated through the the moment-based (MO) approach. While the likelihood function in the RHS of (6.26) is complicated in terms of a_0 , a simple argument shows that this function gets its maximum when b_0 is large. We consider this fact when finding the ML estimators of a_0 and b_0 . To illustrate how one can estimate the simple regression coefficients within a Bayesian framework, we generate $n = 200$ realizations $x_1, x_2, \dots, x_n$ form uniform distribution on $(-1, 1)$ and then responses $y_1, y_2, \dots, y_n$ are obtained using the linear relationship $y_i = \beta_0 + \beta_1 x_i + \epsilon_i = 5 + 2x_i + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, 1)$ for $i = 1, 2, \dots, n$. ■

The corresponding R code for implementing this example is given as follows. The output of the Gibbs sampler is displayed in Figure 6.3.

```

1  R> set.seed(20240530)
2  R> n <- 200
3  R> x <- runif( n, -4, 4)
4  R> y <- 5 + 2*x + rnorm(n, mean = 0, sd = 2)
5  R> out <- lm(y ~ x)
6  R> ML <- as.vector( coefficients(out) )
7  R> error <- as.vector( y - ML[1] - ML[2]*x )
8  R> param0 <- var(error)
9  R> obj <- function(par) -sum( -log(par[2])/2 + lgamma( (par[1]+1)/2 ) -
10 +      lgamma( par[1]/2 ) - (par[1]+1)/2*log( 1 + error^2/par[2] ) )
11 R> hyper0 <- optim( rep(param0, 2), fn = obj, method = "L-BFGS-B",
12 +      lower = rep(0.5, 2), upper = rep(n, 2) )$par
13 R> N <- 5000 # number of generations
14 R> M <- 2000 # size of burn-in period
15 R> theta <- matrix(0, nrow = N, ncol = 3 )
16 R> sigma0 <- 1; sigma1 <- 1;
17 R> Sigma0 <- c(sigma0, sigma1)
18 R> mu0 <- ML
19 R> delta <- 1
20 R> theta[1, ] <- c(ML, delta )

```

```

21 R> s.x2 <- c(n, sum(x^2) )
22 R> j <- 1
23 R> while(j < N)
24 + {
25 +   Di <- Sigma0^2*theta[j, 3]/(Sigma0^2*s.x2 + theta[j, 3])
26 +   A0 <- sum( y - x*theta[j, 2] )
27 +   Mu0 <- Di[1]*(mu0[1]/Sigma0[1]^2 + sum(A0)/delta)
28 +   hat0 <- rnorm(1, Mu0, sqrt(Di[1]))
29 +   theta[j + 1, 1] <- hat0
30 +   A1 <- sum( x*(y - theta[j + 1, 1]) )
31 +   Mu1 <- Di[2]*(mu0[2]/Sigma0[2]^2 + sum(A1)/delta)
32 +   hat1 <- rnorm(1, Mu1, sqrt(Di[2]))
33 +   theta[j + 1, 2] <- hat1
34 +   SSE <- sum(error^2)
35 +   delta <- 1/sqrt( rgamma( 1, shape = (theta0[1] + n)/2,
36 +     rate = (theta0[2] + SSE)/2 ) )
37 +   theta[j + 1, 3] <- delta
38 +   j <- j + 1
39 + }
40 R> Bayes <- apply(theta[(N - M + 1):N, ], 2, mean)
41 R> list(beta0 = Bayes[1], beta1 = Bayes[2], sigma = Bayes[3])

```

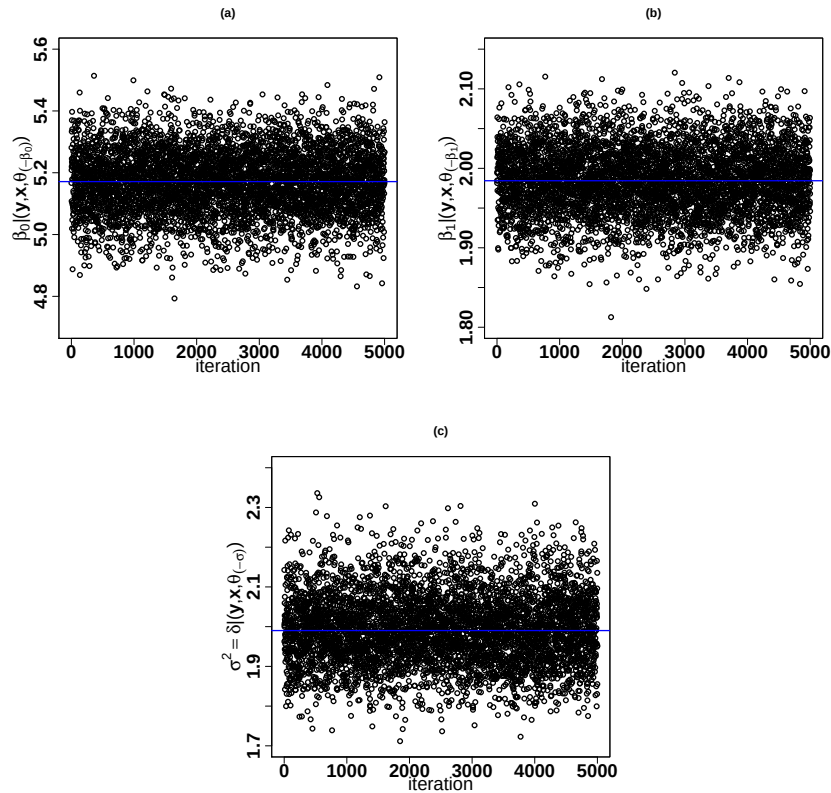


Figure 6.3: Scatterplot of 5000 samples generated from full conditionals: (a) $\beta_0|(\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\beta_0)})$, (b): $\beta_1|(\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\beta_1)})$, and (c): $\sigma^2|(\mathbf{y}, \mathbf{x}, \boldsymbol{\theta}_{(-\sigma)})$. The blue line, in each subfigure, shows the Bayesian estimator.

Example 6-5

Suppose response variable Y depends linearly on independent variable (covariate) X through a simple linear model as

$$\begin{aligned} y_i &= \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_K x_{iK} + \epsilon_i \\ &= \mathbf{x}_i^\top \boldsymbol{\beta} + \epsilon_i; \quad i = 1, 2, 3, \dots, n, \end{aligned}$$

where $\mathbf{x}_i^\top = (1, x_{i1}, \dots, x_{iK})$, $\boldsymbol{\beta}^\top = (\beta_0, \beta_1, \dots, \beta_K)$ is the vector of unknown regression coefficients, and ϵ_i s are the model's theoretical error assumed to follow $\mathcal{N}(0, \sigma^2)$ independently (that is, $\text{cov}(\epsilon_i, \epsilon_j) = 0$ for $i \neq j = 1, 2, 3, \dots, n$). Of course the skewed Student's t [Teimouri et al., 2020] and α -stable [Nolan and Ojeda-Revah, 2013] may be reasonable candidates for distribution of theoretical error in presence of outliers that yield more robust estimation of $\boldsymbol{\beta}$ than the Gaussian one. It is known that the least-square (LS) estimator $\hat{\boldsymbol{\beta}}_{ls}$ and ML estimator $\hat{\boldsymbol{\beta}}_{ml}$, of the regression coefficients $\boldsymbol{\beta}$ are the same and given by

$$\hat{\boldsymbol{\beta}}_{ML} = \hat{\boldsymbol{\beta}}_{LS} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}, \quad (6.27)$$

where $\mathbf{y}$ (vector of responses) and $\mathbf{X}$ is $n \times (K+1)$ design matrix are

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1K} \\ 1 & x_{21} & x_{22} & \cdots & x_{2K} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nK} \end{bmatrix}. \quad (6.28)$$

Although it is possible to consider a Gaussian conjugate prior for each regression coefficient separately, but we prefer to consider a $(K+1)$ -dimensional Gaussian prior for vector of regression coefficients $\boldsymbol{\beta}$ and a mutually independent inverse gamma conjugate prior for $\delta = \sigma^2$ as

$$\begin{aligned} \pi(\boldsymbol{\beta}) &= \mathcal{N}_{K+1}(\boldsymbol{\beta} | \boldsymbol{\mu}_0, \Sigma_0), \\ \pi(\delta) &= \mathcal{IG}\left(\delta \middle| \frac{a_0}{2}, \frac{b_0}{2}\right). \end{aligned} \quad (6.29)$$

It follows that

$$\begin{aligned} \pi(\boldsymbol{\Psi} | \mathbf{y}, \mathbf{x}) &= L(\boldsymbol{\Psi} | \mathbf{y}, \mathbf{x}) \pi(\boldsymbol{\beta}) \\ &\propto \delta^{-\frac{n}{2}} \exp\left\{-\frac{1}{2\delta} \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2\right\} \exp\left\{-\frac{1}{2} (\boldsymbol{\beta} - \boldsymbol{\mu}_0)^\top \Sigma_0^{-1} (\boldsymbol{\beta} - \boldsymbol{\mu}_0)\right\} \\ &\quad \times \left(\frac{a_0}{2}\right)^{\frac{a_0}{2}} \frac{\delta^{-a_0/2-1}}{\Gamma(\frac{a_0}{2})} \exp\left\{-\frac{a_0}{2\delta}\right\}, \end{aligned} \quad (6.30)$$

where $\boldsymbol{\Psi}^\top = (\boldsymbol{\beta}^\top, \delta)$. For computing the full conditional $\boldsymbol{\beta} | (\mathbf{y}, \mathbf{x}, \boldsymbol{\Psi}_{(-\boldsymbol{\beta})})$, it is worth taking into account to the fact that

$$\sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 = \sum_{i=1}^n y_i^2 - 2\boldsymbol{\beta}^\top \mathbf{X}^\top \mathbf{y} + \boldsymbol{\beta}^\top \mathbf{X}^\top \mathbf{X} \boldsymbol{\beta}.$$

Then more algebra shows

$$\pi(\boldsymbol{\beta} | \mathbf{y}, \mathbf{x}, \boldsymbol{\Psi}_{(-\boldsymbol{\beta})}) \propto \exp\left\{-\frac{1}{2} (\boldsymbol{\beta} - \boldsymbol{\mu}_\bullet)^\top \Sigma_\bullet^{-1} (\boldsymbol{\beta} - \boldsymbol{\mu}_\bullet)\right\},$$

where $\boldsymbol{\mu}_\bullet = \Sigma_\bullet [\Sigma_0^{-1} \boldsymbol{\mu}_0 + \mathbf{X}^\top \mathbf{y}]$ and $\Sigma_\bullet = [\Sigma_0^{-1} + \mathbf{X}^\top \mathbf{X}]^{-1}$. So,

$$\pi(\boldsymbol{\beta} | \mathbf{y}, \mathbf{x}, \boldsymbol{\Psi}_{(-\boldsymbol{\beta})}) = \mathcal{N}_{K+1}\left(\boldsymbol{\beta} \middle| \Sigma_\bullet [\Sigma_0^{-1} \boldsymbol{\mu}_0 + \mathbf{X}^\top \mathbf{y}], \Sigma_\bullet\right).$$

Furthermore, it can be seen easily that

$$\pi(\delta|\mathbf{y}, \mathbf{x}, \Psi_{(-\delta)}) = \mathcal{IG}\left(\delta \left| \frac{a_0 + n}{2}, \frac{b_0 + \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2}{2} \right| \right).$$

The hyperparameters a_0 and b_0 can be estimated in the same way as in Example 6-4, $\boldsymbol{\mu}_0$ is assumed to be (6.27), and $\Sigma_0 = \text{cov}(\hat{\boldsymbol{\beta}}_{ML}) = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$. In what follows, we give the steps for computing Bayesian estimation of Ψ in Example 6-5.

1. Read N, M , and determine hyperparameters $a_0, b_0, \boldsymbol{\mu}_0$, and Σ_0 ;
2. Set $t = 0$;
3. Simulate $\boldsymbol{\beta}^{(t+1)}$ from the full conditional $\boldsymbol{\beta} | (\mathbf{y}, \mathbf{x}, \Psi_{(-\boldsymbol{\beta})})$ with PDF

$$\pi(\boldsymbol{\beta} | \mathbf{y}, \mathbf{x}, \Psi_{(-\boldsymbol{\beta})}) = \mathcal{N}_{K+1}(\boldsymbol{\beta} | \boldsymbol{\mu}_\bullet, \Sigma_\bullet),$$

where $\boldsymbol{\mu}_\bullet = \Sigma_\bullet^{-1}[\Sigma_0^{-1}\boldsymbol{\mu}_0 + \mathbf{X}^\top \mathbf{y}]$ and $\Sigma_\bullet = [\Sigma_0^{-1} + \mathbf{X}^\top \mathbf{X}]^{-1}$. So,

4. Having $\boldsymbol{\beta}^{(t+1)}$, set $\delta^{(t+1)} = \sqrt{z}$ where z follows $\mathcal{IG}$ distribution with PDF

$$\mathcal{IG}\left(z \left| \frac{a_0 + n}{2}, \frac{a_0 + \sum_{i=1}^n (y_i - \mathbf{x}_i^\top \boldsymbol{\beta}^{(t+1)})^2}{2} \right| \right).$$

5. If $t = N$, then go to the next step. Otherwise set $t \leftarrow t + 1$ and return to step 3;
6. The average of sequence $\{\Psi^{(M+1)}, \Psi^{(M+2)}, \dots, \Psi^{(M+N)}\}$ sampled from target distribution with PDF $\pi(\Psi | \mathbf{y}, \mathbf{x})$ is the Bayesian estimator $\hat{\Psi}_B = (\hat{\boldsymbol{\beta}}_B^\top, \hat{\sigma}_B)$.

■

Herein, Example 6-5 will be illustrated with computing the Bayesian estimator of regression plane coefficients fitted to `trees` data. This data set is available in R environment. The corresponding R code for implementing this example is given as follows. The output of the Gibbs sampler is displayed in Figure 6.4.

```

1  R> set.seed(20240601)
2  R> data(trees)
3  R> y <- trees$Volume;
4  R> n <- length(y)
5  R> x <- cbind(trees$Girth, trees$Height)
6  R> K <- 3 # number of regression coefficients
7  R> X <- as.matrix( cbind(1, x), nrow = n, ncol = K)
8  R> out <- lm(y ~ X[, 2] + X[, 3])
9  R> ML <- as.vector( coefficients(out) )
10 R> error <- as.vector(y - X%*%ML)
11 R> param0 <- var(error)
12 R> obj <- function(par) -sum( -log(par[2])/2 + lgamma( (par[1]+1)/2 ) -
13 + lgamma( par[1]/2 ) - (par[1]+1)/2*log( 1 + error^2/par[2] ))
14 R> theta0 <- optim( rep(param0, 2), fn = obj, method = "L-BFGS-B",
15 + lower = rep(0.5, 2), upper = rep(n, 2) )$par
16 R> N <- 5000 # number of generations
17 R> M <- 2000 # size of burn-in period
18 R> Psi <- matrix(0, nrow = N, ncol = K + 1)
19 R> delta <- 1
20 R> Psi[1, ] <- c(ML, delta )
21 R> hat <- Psi[1, ]
22 R> Mu0 <- ML

```

```

23 R> sigma2_hat <- theta0[2]/abs(theta0[1] - 1)
24 R> Sigma0_inv <- solve( sigma2_hat*solve(t(X)%*%X) )
25 R> j <- 1
26 R> while(j < N)
27 + {
28 +   Sigma_bullet <- solve( Sigma0_inv + t(X)%*%X )
29 +   Mu_bullet <- Sigma_bullet%*( Sigma0_inv%*%Mu0 + t(X)%*%y )
30 +   Psi[j+1, 1:K] <- mvrnorm(1, mu = Mu_bullet, Sigma = Sigma_bullet )
31 +   error <- as.vector( y - X%*%Psi[j+1, 1:K] )
32 +   SSE <- sum(error^2)
33 +   delta <- 1/sqrt(rgamma( 1, shape = (theta0[1] + n)/2,
34 +     rate = (theta0[2] + SSE)/2 ) )
35 +   Psi[j + 1, 4] <- delta
36 +   j <- j + 1
37 + }
38 R> Bayes <- apply(Psi[(N - M + 1):N, ], 2, mean)
39 R> list(beta0=Bayes[1], beta1=Bayes[2], beta2=Bayes[3], sigma=sqrt(Bayes[4]))

```

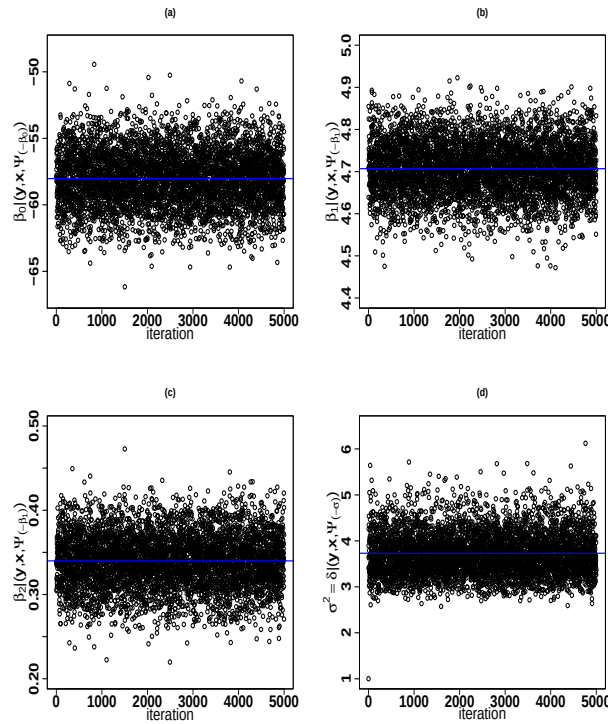


Figure 6.4: Scatterplot of 5000 samples generated from full conditionals: (a) $\beta_0|(\mathbf{y}, \mathbf{x}, \Psi_{(-\beta_0)})$, (b): $\beta_1|(\mathbf{y}, \mathbf{x}, \Psi_{(-\beta_1)})$, (c): $\beta_2|(\mathbf{y}, \mathbf{x}, \Psi_{(-\beta_2)})$, and (d): $\sigma^2 = \delta|(\mathbf{y}, \mathbf{x}, \Psi_{(-\delta)})$. The blue line, in each subfigure, shows the Bayesian estimator.

6.4 Gibbs sampling in Bayesian paradigm with one latent variable

Sometimes implementing the MCMC techniques such as MH or Gibbs sampling for drawing the Bayesian inference is computationally inefficient due to the complex form of the posterior. In such cases, the statistical model may be viewed as incomplete-data problem. From this point of view, recalling from the EM algorithm framework, inference about unknown parameter(s) is accomplished by involving some latent (*unobservable*, *missing*, or *latent*) variable(s). The rationale behind using the term “latent” traced back to this fact that the experimenter cannot observe some extra variable(s) existing in the model at quick glance, but there exists. For

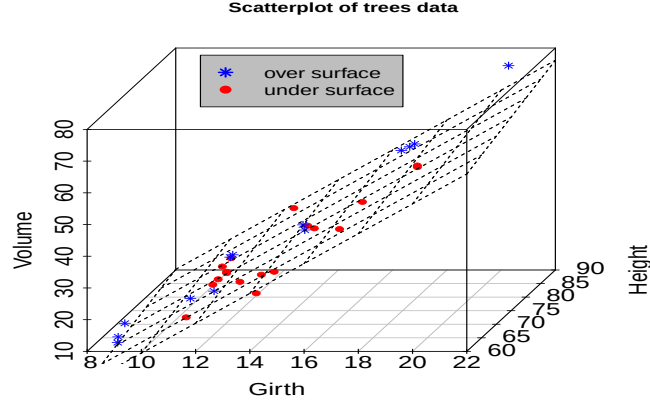


Figure 6.5: Scatterplot of trees data. Superimposed is the fitted regression plane whose coefficients are estimated through the Bayesian paradigm.

example, let random vector $\mathbf{Y}^\top = (Y_1, \dots, Y_p) \sim t_\nu(\boldsymbol{\mu}, \Sigma)$ with PDF given by (4.28). Evidently implementing the MH or Gibbs sampling in which the full conditionals are constructed based on PDF (4.28) is quite difficult. But, since each Student's t distribution is a GSM model with mixing variable $G \sim \mathcal{G}(\nu/2, \nu/2)$, then G can play the role of the latent variable. We also can consider more than one latent variable in each statistical model. Nevertheless, the Gibbs sampling is a very useful approach in presence of latent variable in the case of either sampling or Bayesian inference. In what follows, we give a definition that plays a main role in understanding Bayesian inference for the parameters of a statistical model that includes latent variable(s).

Definition 6.4.1. Let $y_1, \dots, y_n$ are a sample of n independent observed values from a model with PDF $f(\cdot|\boldsymbol{\theta})$ where $\boldsymbol{\theta}$ is the unknown parameter vector. If model includes latent variable g whose PDF is $f(g|\boldsymbol{\theta}_g)$ in which $\boldsymbol{\theta}_g$ is the unknown parameter vector of distribution of g . Furthermore, let $f(y, g|\boldsymbol{\Psi})$ is the joint PDF of vector $(y, g)^\top$ where $\boldsymbol{\Psi} = \boldsymbol{\theta} \cup \boldsymbol{\theta}_g$. Then we have:

- The sequence $\{y_1, \dots, y_n\}$ and $\{g_1, \dots, g_n\}$, are called the observed and latent data, respectively.
- The sequence of paired-data $\{(y_1, g_1), \dots, (y_n, g_n)\}$ is called complete-data that evidently is the union of observed and latent data.
- The complete-data likelihood function becomes

$$L_c(\boldsymbol{\Psi}|\mathbf{y}, \mathbf{g}) = f(\mathbf{y}, \mathbf{g}|\boldsymbol{\Psi}) = \prod_{i=1}^n f(y_i|g_i, \boldsymbol{\Psi}) = \prod_{i=1}^n f(y_i|g_i, \boldsymbol{\theta}) f(g_i|\boldsymbol{\theta}_g). \quad (6.31)$$

Definition 6.4.1 is based on just one latent variable, but can be easily extended and is valid for two or more latent variables. It is noteworthy that for a Bayesian paradigm in which we have latent variable(s), we rewrite the posterior (6.30) in terms of the *complete-data* likelihood function as follows.

$$\pi(\boldsymbol{\Psi}|\mathbf{y}, \mathbf{g}) \propto L_c(\boldsymbol{\Psi}|\mathbf{y}, \mathbf{g})\pi(\boldsymbol{\Psi}), \quad (6.32)$$

where $L_c(\boldsymbol{\Psi}|\mathbf{y}, \mathbf{g})$ is given by Definition 6.4.1. To implement the Bayesian paradigm, notice that the conditional PDF $f(y_i|g_i, \boldsymbol{\theta})$ and marginal PDF $f(g_i|\boldsymbol{\theta}_g)$ given in RHS of (6.31) must be known and possibly in closed form. In what follows, we give an example in which Bayesian inference on the parameters of a Student's t distribution is desired.

Example 6-6

Suppose $\mathbf{y}^\top = (\mathbf{y}_1^\top, \dots, \mathbf{y}_n^\top)$ is vector of n independent observations from a p -dimensional Student's t distribution. Recall from Definition 4.0.5 that for each Student's t random vector $\mathbf{Y}$, regarding $\eta(G) = G$ with $G \sim \mathcal{G}(\nu/2, \nu/2)$, then we can write

$$\begin{aligned} \mathbf{Y}|G &\sim f(\mathbf{y}_i|g_i, \boldsymbol{\theta} = (\boldsymbol{\mu}, \Sigma)) = \mathcal{N}_p(\mathbf{y}_i|\boldsymbol{\mu}, \frac{\Sigma}{G}), \\ G &\sim f(g_i|\boldsymbol{\theta}_g = \nu) = \mathcal{G}(g_i|\nu/2, \nu/2), \end{aligned} \quad (6.33)$$

Hence, the complete-data likelihood is

$$\begin{aligned} L_c(\boldsymbol{\Psi}|\mathbf{y}, \mathbf{g}) &= \prod_{i=1}^n \left[\mathcal{N}_p(\mathbf{y}_i|\boldsymbol{\mu}, [g_i]^{-1}\Sigma) f(g_i|\boldsymbol{\theta}_g) \right] \\ &= \left[\Gamma\left(\frac{\nu}{2}\right) \right]^{-n} \left(\frac{\nu}{2}\right)^{\frac{n\nu}{2}} \\ &\quad \times \prod_{i=1}^n \left\{ [g_i]^{\frac{\nu+p}{2}-1} \exp\left\{-\frac{g_i}{2} \left[\nu + (\mathbf{y}_i - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{y}_i - \boldsymbol{\mu}) \right] \right\} \right\}, \end{aligned} \quad (6.34)$$

where $\boldsymbol{\Psi}^\top = \boldsymbol{\theta}^\top \cup \boldsymbol{\theta}_g^\top = (\boldsymbol{\mu}, \Sigma, \nu)$ is the model's parameter vector. For the prior, we proceed to assume independence, that is $\pi(\boldsymbol{\Psi}) = \pi(\boldsymbol{\mu})\pi(\Sigma)\pi(\nu)$ in which $\pi(\boldsymbol{\mu})$ and $\pi(\Sigma)$ are conjugate Gaussian and inverse Wishart priors. Furthermore, we let $\pi(\nu)$ to be the PDF of a gamma distribution. We have

$$\begin{aligned} \pi(\boldsymbol{\mu}) &\sim \mathcal{N}_p(\mathcal{M}_0, \mathcal{S}_0), \\ \pi(\Sigma) &\sim \mathcal{IW}(\mathcal{D}_0, \nu_0), \\ \pi(\nu) &\sim \mathcal{G}(a_0, b_0). \end{aligned}$$

We note that in above an objective prior such as Jeffreys' prior can be chosen, however, the most concern is placed on $\pi(\nu)$ that was proposed by Fernández and Steel [1999] to be $\pi(\nu) = 10 \exp\{-10\nu\}$. It can be seen that the jeffreys' prior becomes $\pi(\boldsymbol{\mu}, \Sigma, \nu) = |\Sigma|^{-(p+1)/2}$ [Fernández and Steel, 1999], and so considering a flat prior for both of $\boldsymbol{\mu}$ and ν is acceptable. We note that for drawing inference about $\boldsymbol{\Psi}$, one needs to generate from full conditional of the latent variable too. So, by considering $\boldsymbol{\mu}$, Σ , ν , and g (latent variable) as the model's variables, within a Gibbs sampling framework, we proceed to sample from full conditionals $\boldsymbol{\mu}|\Sigma, \nu$, $\Sigma|\boldsymbol{\mu}, \nu$, $\nu|\boldsymbol{\mu}, \Sigma$. But, keep in mind that for each observed data $\mathbf{y}_i$, g_i (for $i = 1, \dots, n$) is also latent unobservable). In order to g_i being known, we proceed to simulate g_i (for $i = 1 \dots, n$) for each $\mathbf{y}_i$. To do this, in each iteration of the Gibbs sampling, we simulate (for each $\mathbf{y}_i$) from full conditional $g_i|\boldsymbol{\mu}, \Sigma, \nu, \mathbf{y}_i$ as follows. From RHS of (6.34), for a single observed value $\mathbf{y}_i$, it follows that

$$\pi(g_i|\boldsymbol{\Psi}, \mathbf{y}_i) \propto [g_i]^{\frac{\nu+p}{2}-1} \exp\left\{-\frac{g_i}{2} \left[\nu + (\mathbf{y}_i - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{y}_i - \boldsymbol{\mu}) \right] \right\}.$$

This means that

$$\pi(g_i|\boldsymbol{\Psi}, \mathbf{y}_i) \sim \mathcal{G}\left((\nu + p)/2, \nu/2 + (\mathbf{y}_i - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{y}_i - \boldsymbol{\mu})/2\right).$$

Let $\boldsymbol{\Psi}_{(-\theta)}$ denote the whole parameter vector excluding its element θ , once we have generated a whole vector of latent variable as $\mathbf{g}^\top = (g_1, \dots, g_n)$, the complete-data is known and we proceed to simulate from other full conditionals as follows.

- **full conditional of $\boldsymbol{\mu} | (\underline{\mathbf{y}}, \mathbf{g}, \Psi_{(-\boldsymbol{\mu})})$:** By considering a conjugate prior, shown earlier by $\pi(\boldsymbol{\mu}) \sim \mathcal{N}_p(\mathcal{M}_0, \mathcal{S}_0)$, we have

$$\pi(\boldsymbol{\mu} | \underline{\mathbf{y}}, \mathbf{g}, \Psi_{(-\boldsymbol{\mu})}) \propto L_c(\boldsymbol{\Psi} | \underline{\mathbf{y}}, \mathbf{g}) \mathcal{N}_p(\boldsymbol{\mu} | \mathcal{M}_0, \mathcal{S}_0).$$

More algebra shows that

$$\boldsymbol{\mu} | (\underline{\mathbf{y}}, \mathbf{g}, \Psi_{(-\boldsymbol{\mu})}) \sim \mathcal{N}_p\left(\mathcal{Q}_0 \left[\mathcal{S}_0^{-1} \mathcal{M}_0 + \Sigma^{-1} \sum_{i=1}^n g_i \mathbf{y}_i \right], \mathcal{Q}_0\right),$$

where

$$\mathcal{Q}_0 = \left[\mathcal{S}_0^{-1} + \Sigma^{-1} \sum_{i=1}^n g_i \right]^{-1}.$$

- **full conditional of $\Sigma | (\underline{\mathbf{y}}, \mathbf{g}, \Psi_{(-\Sigma)})$:** By considering a conjugate prior, that is $\Sigma \sim \mathcal{IW}(\mathcal{D}_0, \nu_0)$, we can write

$$\pi(\Sigma | \underline{\mathbf{y}}, \mathbf{g}, \Psi_{(-\Sigma)}) \propto L_c(\boldsymbol{\Psi} | \underline{\mathbf{y}}, \mathbf{g}) \mathcal{IW}(\Sigma | \mathcal{D}_0, \nu_0),$$

where $\mathcal{IW}(\Sigma | \mathcal{D}_0, \nu_0)$ denotes the PDF of an inverse Wishart distribution. More algebra shows that

$$\Sigma | (\underline{\mathbf{y}}, \mathbf{g}, \Psi_{(-\Sigma)}) \sim \mathcal{IW}(\mathcal{R}_0, \nu_0 + n),$$

where $\mathcal{R}_0 = \mathcal{D}_0 + \sum_{i=1}^n g_i (\mathbf{y}_i - \boldsymbol{\mu})(\mathbf{y}_i - \boldsymbol{\mu})^\top$.

- **full conditional of $\nu | (\underline{\mathbf{y}}, \mathbf{g}, \Psi_{(-\nu)})$:** By considering a truncated exponential distribution with PDF $\pi(\nu) = \mathcal{G}(\nu - 2 | 1, b_0)$ for $\nu \geq 2$, we can write

$$\begin{aligned} \pi(\nu | \underline{\mathbf{y}}, \mathbf{g}, \Psi_{(-\nu)}) &\propto L_c(\boldsymbol{\Psi} | \underline{\mathbf{y}}, \mathbf{g}) \pi(\nu) \\ &\propto \left[\Gamma\left(\frac{\nu}{2}\right) \right]^{-n} \left(\frac{\nu}{2}\right)^{\frac{n\nu}{2}} b_0 \exp\left\{-\frac{\nu}{2} [2b_0 + \sum_{i=1}^n g_i]\right\} \prod_{i=1}^n [g_i]^{\frac{\nu}{2}}. \end{aligned} \quad (6.35)$$

As it is seen, the full conditional (6.35) has not closed form. The MH algorithm can be suggested to draw sample from this full conditional. It is easy to check that $\pi(\nu | \underline{\mathbf{y}}, \mathbf{g}, \Psi_{(-\nu)})$ is log-concave and hence the ARS-within-Gibbs sampling algorithm can be suggested for simulating from this full conditional. For computing the vector of hyperparameter $\boldsymbol{\theta}_0 = b_0$, it follows from hierarchy (6.33) that

$$\begin{aligned} m(\underline{\mathbf{y}} | b_0) &= \Pi_{i=1}^n \int_2^\infty f(\mathbf{y}_i | \nu) \pi(\nu | b_0) d\nu \\ &= \Pi_{i=1}^n \int_2^\infty \frac{\Gamma\left(\frac{\nu+p}{2}\right) \pi(\nu | b_0) d\nu}{(\nu\pi)^{\frac{p}{2}} \Gamma\left(\frac{\nu}{2}\right) |\Sigma|^{\frac{1}{2}} \left[1 + \frac{(\mathbf{y}_i - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{y}_i - \boldsymbol{\mu})}{\nu}\right]^{\frac{\nu+p}{2}}}, \end{aligned} \quad (6.36)$$

where $\underline{\mathbf{y}}^\top = (\mathbf{y}_1^\top, \dots, \mathbf{y}_n^\top)$. Maximizing the RHS of (6.36) with respect to b_0 is not a simple task, an so we proceed to compute the moment-type estimator of b_0 as follows. First notice that

$$\begin{aligned} E[(\mathbf{Y} - \boldsymbol{\mu})(\mathbf{Y} - \boldsymbol{\mu})^\top | b_0] &= \int_2^\infty \int_{\mathbb{R}^p} (\mathbf{y} - \boldsymbol{\mu})(\mathbf{y} - \boldsymbol{\mu})^\top f(\mathbf{y}|\nu) \pi(\nu|b_0) d\mathbf{y} d\nu \\ &= \int_2^\infty \int_{\mathbb{R}^p} \frac{(\mathbf{y} - \boldsymbol{\mu})(\mathbf{y} - \boldsymbol{\mu})^\top \Gamma(\frac{\nu+p}{2})}{(\nu\pi)^{\frac{p}{2}} \Gamma(\frac{\nu}{2}) |\Sigma|^{\frac{1}{2}} [1 + \frac{(\mathbf{y}-\boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{y}-\boldsymbol{\mu})}{\nu}]^{\frac{\nu+p}{2}}} \\ &\quad \times \frac{b_0^{b_0-1}}{\Gamma(b_0)} (\nu-2)^{b_0-1} \exp\{-b_0(\nu-2)\} d\mathbf{y} d\nu \\ &= \int_2^\infty \left(\frac{\nu\Sigma}{\nu-2} \right) \frac{b_0^{b_0-1}}{\Gamma(b_0)} (\nu-2)^{b_0-1} \exp\{-b_0(\nu-2)\} d\nu \\ &= 3 - \frac{1}{b_0}. \end{aligned}$$

Let AD denote the average of diagonal elements of matrix $[\sum_{i=1}^n (\mathbf{y}_i^2 - \boldsymbol{\mu})(\mathbf{y}_i^2 - \boldsymbol{\mu})^\top / n] \Sigma^{-1}$, then b_0 can be exploited as $b_0 = 1/(3 - AD)$.

Furthermore, one can employ the MH-within-Gibbs sampling technique for drawing sample from full conditional (6.35). To this end, we assume gamma distribution for the prior with PDF $\pi(\nu) = \mathcal{G}(\cdot | b_0, b_0)$. Herein there is just one hyperparameter that we prefer to compute it as $b_0 = 1/(3 - AD)$. We further choose $\mathcal{G}(b_0, b_0)$ as the candidate. For other hyperparameters including $\mathcal{M}_0, \mathcal{S}_0, \mathcal{D}_0$, and ν_0 , in example above, we set $\mathcal{M}_0$ to be the mean of $\underline{\mathbf{y}}$ and $\nu_0 = 1$. For $\mathcal{S}_0$ and $\mathcal{D}_0$ we suggest to consider $\mathbf{I}_p$. The steps for computing the Bayesian estimator of $\boldsymbol{\Psi}$ are given by Algorithm 18. ■

The pertaining R code for implementing example above is given as follows. The R code is applied to a sample of $n = 500$ realizations have been generated from a bivariate Student's t distribution with parameters $\boldsymbol{\mu}^\top = (0, 2)$, $\sigma_{11} = 1$, $\sigma_{12} = -0.50$, $\sigma_{22} = 0.75$, and $\nu = 5$. The output of the Gibbs sampler is displayed in Figure 6.6.

```

1 R > rsgg<-function(n, nu, Mu, Sigma)
2 + {
3 +   Dim <- length(Mu)
4 +   Y <- matrix(NA, nrow = n, ncol = Dim)
5 +   for (i in 1:n)
6 +   {
7 +     Z <- rgamma(1, shape = nu/2, rate = nu/2)
8 +     X <- mvrnorm(n = 1, mu = rep(0, Dim), Sigma = Sigma )
9 +     Y[i, ] <- Mu + X/sqrt(Z)
10 +   }
11 +   Y
12 + }
13 R > f.a <- function(x, b0, y)
14 + {
15 +   n <- length(y)
16 +   -n*loggamma(x/2) + n*(x/2)*log(x/2) - x/2*( 2*b0 + sum(y) ) + x/2*sum( log(y) ) + (b0-1)*
17 +   log(x)
18 + }
19 R > fprim.a <- function(x, b0, y)
20 + {
21 +   n <- length(y)
22 +   -n*digamma(x/2)/2 + n/2 + n/2*log(x/2) - ( 2*b0 + sum(y) )/2 + sum( log(y) )/2 + (b0-1)/
23 +   x
24 + }
25 R > set.seed(20240529)

```

Algorithm 18 Bayesian inference for Student's t distribution

```
1: Read  $N, M, K, \mathcal{M}_0, \mathcal{S}_0, \mathcal{D}_0$ , and  $\nu_0$ ;
2: Set  $t \leftarrow 0$ ;
3: Set  $\boldsymbol{\mu}^{(0)} \leftarrow \mathcal{M}_0, \Sigma^{(0)} \leftarrow 1/n \sum_{i=1}^n (\mathbf{y}_i - \boldsymbol{\mu}^{(0)})(\mathbf{y}_i - \boldsymbol{\mu}^{(0)})^\top, \nu^{(0)} = 2$ ,
4: and set  $\boldsymbol{\Psi}^{(0)} = (\boldsymbol{\mu}^{(0)}, \Sigma^{(0)}, \nu^{(0)})$ ;
5: while  $t \leq N$  do
6:   Set  $i = 1$ ;
7:   while  $i \leq n$  do
8:     Simulate  $g_i$  from  $\mathcal{G}(a, b)$  where  $a = (\nu^{(t)} + p + 1)/2$  and  $b =$ 
9:      $\left[ \nu^{(t)}/2 + (\mathbf{y}_i - \boldsymbol{\mu}^{(t)})^\top [\Sigma^{(t)}]^{-1} (\mathbf{y}_i - \boldsymbol{\mu}^{(t)}) \right]/2$ ;
10:    Set  $i \leftarrow i + 1$ ;
11:   end
12:   Generate  $\boldsymbol{\mu}^{(t+1)}$  from  $\mathcal{N}_p(\mathcal{Q}_0 [\mathcal{S}_0^{-1} \mathcal{M}_0 + [\Sigma^{(t)}]^{-1} \sum_{i=1}^n g_i \mathbf{y}_i], \mathcal{Q}_0)$  where
13:    $\mathcal{Q}_0 = [\mathcal{S}_0^{-1} + [\Sigma^{(t)}]^{-1} \sum_{i=1}^n g_i]^{-1}$ ;
14:   Generate  $\Sigma^{(t+1)}$  from  $\mathcal{IW}(\mathcal{R}_0, \nu_0 + n)$  where  $\mathcal{R}_0 = \mathcal{D}_0 +$ 
15:    $\sum_{i=1}^n g_i (\mathbf{y}_i - \boldsymbol{\mu}^{(t+1)})(\mathbf{y}_i - \boldsymbol{\mu}^{(t+1)})^\top$ ;
16:   Use the MH-within-Gibbs sampling technique for simulating  $\nu^{(t+1)}$ 
17:   as follows;
18:   Set  $k = 0$  and  $x_0 \leftarrow \nu_0$ 
19:   while  $k \leq K$  do
20:      $x_0 \sim \mathcal{G}(a_0, b_0)$ ;
21:     Generate  $x^{(k+1)} \sim \mathcal{G}(a_0, b_0)$ ;
22:     Compute  $p(x^{(k)} \rightarrow x^{(k+1)}) = \min\{1, \exp\{R_1 - R_2\}\}$  where
23:      $R_1 = -n \log \Gamma(\frac{x^{(k+1)}}{2}) + \frac{nx^{(k+1)}}{2} \log(\frac{x^{(k+1)}}{2}) + (b_0 - 1) \log x^{(k+1)}$ 
24:      $- \frac{x^{(k+1)}}{2} [2b_0 + \sum_{i=1}^n g_i] + \frac{x^{(k+1)}}{2} \sum_{i=1}^n \log g_i$ 
25:     and
26:      $R_2 = -n \log \Gamma(\frac{x^{(k)}}{2}) + \frac{nx^{(k)}}{2} \log(\frac{x^{(k)}}{2}) + (b_0 - 1) \log x^{(k)}$ 
27:      $- \frac{x^{(k)}}{2} [2b_0 + \sum_{i=1}^n g_i] + \frac{x^{(k)}}{2} \sum_{i=1}^n \log g_i$ ;
28:     Generate  $u \sim \mathcal{U}(0, 1)$ ;
29:     If  $u < p(x^{(k)}, x^{(k+1)})$  then  $x^{(k)} \leftarrow x^{(k+1)}$  and  $k \leftarrow k + 1$ ;
30:     Repeat algorithm from line (21);
31:     If  $k = K$  then  $\nu^{(t+1)} \leftarrow x^{(k+1)}$  and stop the algorithm;
32:   end
33: Set  $\boldsymbol{\Psi}^{(t+1)} \leftarrow (\boldsymbol{\mu}^{(t+1)}, \Sigma^{(t+1)}, \nu^{(t+1)})$  and  $t \leftarrow t + 1$ ;
34: end
35: Sequence  $\{\boldsymbol{\Psi}^{(M)}, \boldsymbol{\Psi}^{(M+1)}, \dots, \boldsymbol{\Psi}^{(N)}\}$  is a sample of size  $N - M + 1$  from
36: posterior with PDF  $\pi(\boldsymbol{\Psi}|\mathbf{x})$ ;
37: The Bayesian estimator of  $\hat{\boldsymbol{\Psi}}_B$  is given by  $(N - M + 1)^{-1} \sum_{t=1}^{N-M+1} \boldsymbol{\Psi}^{(t)}$ .
```

```

24 R > library(MASS); library(ars)
25 R > n <- 500; nu <- 5; Mu <- c(0, 2)
26 R > Sigma <- matrix( c(1,-0.5,-0.5,0.75), 2, 2)
27 R > Y <- rssg(n, nu, Mu, Sigma)
28 R > N <- 5000; M <- 2000; p <- length(Mu)
29 R > Psi <- matrix(0, nrow = N, ncol = p + 1 + p^2)
30 R > g <- rate <- rep(0, n)
31 R > SOMu <- diag(2); MOMu <- rep(0, 2); nu0 <- 2
32 R > DO <- matrix(diag(2), 2, 2)
33 R > for(j in 1:N)
34 + {
35 +   rate <- mahalanobis(x = Y, center = Mu, cov = Sigma )
36 +   for(i in 1:n) g[i] <- rgamma( 1, shape = (nu + p)/2, rate = rate[i]/2 + nu/2 )
37 +   Sigmainv <- solve( Sigma )
38 +   dy <- mahalanobis( Y, center = Mu, cov = Sigma )
39 +   Sum.g <- sum(g)
40 +   M_sum <- rowSums( sapply(1:n, function(i) g[i]*c( Y[i, ] ) ) )
41 +   Q <- solve( solve( SOMu ) + Sigmainv*Sum.g )
42 +   Mu <- c( mvrnorm(1, mu = Q%*( solve( SOMu )%*%MOMu +
43 +     Sigmainv%*%M_sum ) , Sigma = Q) )
44 +   Psi[j, 1:p] <- Mu
45 +   R0 <- matrix(0, nrow = p, ncol = p)
46 +   for(i in 1:n){R0 <- R0 + g[i]*c( Y[i, ] - Mu )%*%t(c( Y[i, ] - Mu ))}
47 +   Sigma <- solve(rWishart(1, df = nu0 + n, Sigma = solve(DO + R0) )[,1])
48 +   Psi[j, (p+1):(p + p^2)] <- as.numeric(Sigma)
49 +   b0 <- 1/(3-(cov(Y)%*%solve(Sigma))[1,1])
50 +   nu <- ars(1, f.a, fprim.a, x = c(2, 5, 10, 50), m = 4,
51 +     lb = TRUE, xlb = 2, b0 = b0, y = g)
52 +   Psi[j, p + 1 + p^2] <- nu
53 + }
54 R > Psi.hat <- apply(Psi[(N - M + 1):N, ], 2, mean)
55 R > Mu <- Psi.hat[1:p]
56 R > Sigma <- Psi.hat[(p+1):(p + p^2)]
57 R > nu <- Psi.hat[p + 1 + p^2]

```

6.5 Gibbs sampling in Bayesian paradigm with two latent variables

The Bayesian paradigm implemented with one latent variable can be adopted further for two or more latent variables. The Bayesian inference on parameters of CFUSG model with PDF (4.5) has been studied by Liseo and Parisi [2013]. Herein, we are willing to implement the Bayesian paradigm for the CFUSG scale mixture model (4.25). To this end, having $\{\mathbf{y}_1^\top, \dots, \mathbf{y}_n^\top\}$ as the sequence of observed data and, we regard $\{\mathbf{u}_1^\top, \dots, \mathbf{u}_n^\top\}$ and $\{g_1, \dots, g_n\}$ as two sequences of latent variables associated model (4.25) or (4.26). Based on hierarchy (4.27), the complete-data log-likelihood is

$$\begin{aligned}
L_c(\Psi|\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}) &= \prod_{i=1}^n \left[\phi_p(\mathbf{y}_i | \boldsymbol{\mu} + \Lambda \mathbf{u}_i, [\eta(g_i)]^{-1} \Sigma) \phi_q(\mathbf{u}_i | \mathbf{0}_q, [\eta(g_i)]^{-1} \mathbf{I}_q) f(g_i | \boldsymbol{\theta}) \right] \\
&\propto \prod_{i=1}^n [\eta(g_i)]^{\frac{p+q}{2}} \exp \left\{ -\frac{h(g_i)}{2} \left[(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1} (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i) \right. \right. \\
&\quad \left. \left. + \mathbf{u}_i^\top \mathbf{I}_q \mathbf{u}_i \right] \right\} f(g_i | \boldsymbol{\theta}),
\end{aligned} \tag{6.37}$$

where $\Psi = (\boldsymbol{\mu}, \Sigma, \Lambda, \boldsymbol{\theta})$ is the model's parameter vector. In what follows, first we consider three scenarios for distribution of mixing variable as shown in Table 4.3 and represent the full conditional thereof. Then, the Bayesian paradigm is implemented for the skewed Student's t distribution as a study case.

Full conditional $G|(Y = \mathbf{y}_i, \dots)$

In this part, the full conditionals $G|(Y = \mathbf{y}_i, U = \mathbf{u}_i, \Psi)$ and $U|(Y = \mathbf{y}_i, G = g_i, \Psi)$ are presented and then we describe how to sample from both full conditionals under three scenarios shown in Table 4.3.

i. **Scenario 1 (mixing random variable is $\mathcal{GL}(\omega, \beta, \gamma)$):** By definition we have

$$\begin{aligned}\pi(g_i|\mathbf{y}_i, \mathbf{u}_i, \Psi) &= \frac{f(\mathbf{y}_i, \mathbf{u}_i, g_i|\Psi)}{f(\mathbf{y}_i, \mathbf{u}_i|\Psi)} \\ &\propto f(\mathbf{y}_i, \mathbf{u}_i, g_i|\Psi) \\ &= f(\mathbf{y}_i|\mathbf{u}_i, g_i|\Psi) f(\mathbf{u}_i|g_i) f(g_i|\boldsymbol{\theta}) \\ &\propto \frac{\beta^{\omega+1}}{(\beta + \gamma)\Gamma(\omega + 1)} [\eta(g_i)]^{\frac{p+q}{2}} [g_i^{\omega-1}(\omega + \gamma g_i)] \exp\{-\beta g_i \\ &\quad - \frac{\eta(g_i)}{2} [(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1}(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i) + \mathbf{u}_i^\top \mathbf{u}_i]\}. \end{aligned} \quad (6.38)$$

Setting $\eta(g_i) = g_i$ in the RHS of (6.38), it turns out that

$$\begin{aligned}\pi(g_i|\mathbf{y}_i, \mathbf{u}_i, \Psi) &\propto g_i^{\frac{p+q}{2} + \omega - 1} (\omega + \gamma g_i) \exp\left\{-g_i \left[\beta + \frac{\mathbf{u}_i^\top \mathbf{u}_i}{2} + (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \right. \right. \\ &\quad \left. \left. \Sigma^{-1}(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)/2\right]\right\}. \end{aligned} \quad (6.39)$$

Compare the RHS of (6.39) with the PDF $f(g|\boldsymbol{\theta})$ given in the first column of Table 4.3. Then more algebra shows

$$G|(Y = \mathbf{y}_i, U = \mathbf{u}_i, \Psi) \sim \mathcal{GL}(\omega^*, \beta^*, \gamma^*), \quad (6.40)$$

where $\omega^* = (p + q)/2 + \omega$, $\beta^* = \beta + (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1}(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)/2 + \mathbf{u}_i^\top \mathbf{u}_i/2$, and $\gamma^* = \gamma/\omega$.

ii. **Scenario 2 (mixing random variable is $\mathcal{G}(\eta, \zeta)$):** Following the same argument as in Scenario 1, for $\gamma = 0$, $\omega = \eta$, and $\beta = \zeta$, it turns out from RHS of (6.40) that

$$G|(Y = \mathbf{y}_i, U = \mathbf{u}_i, \Psi) \sim \mathcal{G}(\eta^*, \zeta^*), \quad (6.41)$$

where $\eta^* = (p + q)/2 + \eta$ and $\zeta^* = 2\zeta + (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1}(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)/2 + \mathbf{u}_i^\top \mathbf{u}_i/2$.

iii. **Scenario 3 (mixing random variable is $\mathcal{GIG}(\tau, \chi, \chi)$):** If we let $f(g_i|\boldsymbol{\theta})$ in the RHS of (6.37) to be PDF of $\mathcal{GIG}$ distribution whose form is given in third column of Table 4.3, we have

$$\begin{aligned}\pi(g_i|\mathbf{y}_i, \mathbf{u}_i, \Psi) &\propto \left[\frac{g_i^{\tau-1}}{2\mathcal{K}_\tau(\chi)} \exp\left\{-\frac{\chi g_i}{2} - \frac{\chi}{2g_i}\right\} \right] [\eta(g_i)]^{\frac{p+q}{2}} \exp\left\{-\frac{\eta(g_i)}{2} \right. \\ &\quad \left. \times [(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1}(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i) + \mathbf{u}_i^\top \mathbf{u}_i]\right\}, \end{aligned} \quad (6.42)$$

If in the RHS of (6.42) we let $\eta(g_i) = g_i$, then it turns out that

$$\pi(g_i|\mathbf{y}_i, \mathbf{u}_i, \Psi) \propto g_i^{\frac{p+q}{2} + \tau - 1} \exp\left\{-\epsilon \frac{g_i}{2} - \frac{\chi}{2g_i}\right\}. \quad (6.43)$$

Evidently,

$$G|(Y = \mathbf{y}_i, U = \mathbf{u}_i, \Psi) \sim \mathcal{GIG}\left(\frac{p}{2} + \frac{q}{2} + \tau, \epsilon, \chi\right),$$

where $\epsilon = [(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1}(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i) + \mathbf{u}_i^\top \mathbf{u}_i + \chi]$.

Full conditional $U|(Y = \mathbf{y}_i, \dots)$

By definition we have

$$\begin{aligned}
\pi(\mathbf{u}_i | \mathbf{y}_i, g_i, \Psi) &= \frac{f(\mathbf{y}_i, \mathbf{u}_i, g_i | \Psi)}{f(\mathbf{y}_i, g_i | \Psi)} \times \mathbb{I}_{\mathbb{R}^{q+}}(\mathbf{u}_i) \\
&\propto f(\mathbf{y}_i, \mathbf{u}_i, g_i | \Psi) \times \mathbb{I}_{\mathbb{R}^{q+}}(\mathbf{u}_i) \\
&= f(\mathbf{y}_i | \mathbf{u}_i, g_i | \Psi) f(\mathbf{u}_i | g_i) f(g_i | \theta) \times \mathbb{I}_{\mathbb{R}^{q+}}(\mathbf{u}_i) \\
&\propto f(\mathbf{y}_i | \mathbf{u}_i, g_i | \Psi) f(\mathbf{u}_i | g_i) \times \mathbb{I}_{\mathbb{R}^{q+}}(\mathbf{u}_i) \\
&\propto -\exp\left\{\frac{\eta(g_i)}{2} \left[(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1} (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i) \right. \right. \\
&\quad \left. \left. + \mathbf{u}_i^\top \mathbf{u}_i \right] \right\} \times \mathbb{I}_{\mathbb{R}^{q+}}(\mathbf{u}_i),
\end{aligned}$$

where $\mathbb{I}_{\mathcal{S}}(x)$ is defined in (3.24). We use the fact that

$$\begin{aligned}
&(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1} (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i) + \mathbf{u}_i^\top \mathbf{u}_i \\
&= d(\mathbf{y}_i) + (\mathbf{u}_i - \mathbf{m})^\top \Delta^{-1} (\mathbf{u}_i - \mathbf{m}),
\end{aligned} \tag{6.44}$$

where $d(\mathbf{y}_i)$, $\mathbf{m}$, and Δ are defined in Theorem 4.1.2. Hence,

$$\pi(\mathbf{u}_i | \mathbf{y}_i, g_i, \Psi) \propto \exp\left\{-\frac{\eta(g_i)}{2} (\mathbf{u}_i - \mathbf{m})^\top \Delta^{-1} (\mathbf{u}_i - \mathbf{m})\right\} \times \mathbb{I}_{\mathbb{R}^{q+}}(\mathbf{u}_i). \tag{6.45}$$

This means that

$$U|(Y = \mathbf{y}_i, G = g_i, \Psi) \sim \mathcal{TN}_q\left(\mathbf{m}, \frac{\Delta}{\eta(g_i)} \mathbf{0}_q, \infty\right), \tag{6.46}$$

where $\mathcal{TN}_q(\cdot, \cdot, \cdot, \cdot)$ refers to the family of truncated q -dimensional Gaussian distributions introduced in Sub-section 4.10.

Bayesian inference for skew Student's t distribution

As it is seen from second column of Table 4.3, when parameters of gamma distribution restricted to be $\alpha = \beta = \nu/2$, then the associated CFUSG scale mixture model is skewed Student's t distribution with ν degrees of freedom for which a Bayesian paradigm has been developed by [Parisi and Liseo, 2018]. Therefore, setting $\theta = \nu$, we simulate from full conditionals $g_i | (\boldsymbol{\mu}, \Sigma, \nu, \mathbf{u}_i, \mathbf{y}_i)$ and $\mathbf{u}_i | (\boldsymbol{\mu}, \Sigma, \nu, g_i, \mathbf{y}_i)$ (for $i = 1 \dots, n$) in order to construct the complete-data represented as $\{(\mathbf{y}_1^\top, \mathbf{u}_1^\top, g_1), \dots, (\mathbf{y}_n^\top, \mathbf{u}_n^\top, g_n)\}$. To this end, it follows from the RHS of (6.41) that

$$\pi(g_i | \mathbf{y}_i, \mathbf{u}_i, \Psi) = \mathcal{G}(\alpha^*, \beta^*), \tag{6.47}$$

where $\alpha^* = (p + q)/2 + \alpha$ and $\beta^* = 2\beta + (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1} (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)/2 + \mathbf{u}_i^\top \mathbf{u}_i/2$. Moreover, using $\eta(g_i) = g_i$, it turns out from (6.46) that

$$\pi(\mathbf{u}_i | \mathbf{y}_i, g_i, \Psi) = \mathcal{TN}_q\left(\mathbf{m}_i, \frac{\Delta}{g_i} \mathbf{0}_q, \infty\right), \tag{6.48}$$

where $\mathbf{m}_i = \Lambda^\top \Omega^{-1} (\mathbf{y}_i - \boldsymbol{\mu})$, $\Delta = \mathbf{I}_q - \Lambda^\top \Omega^{-1} \Lambda$, and $\Omega = \Sigma + \Lambda \Lambda^\top$. Constructing the complete data, we proceed to generate from the remainder full conditionals $\boldsymbol{\mu} | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \Psi_{(-\boldsymbol{\mu})})$, $\Sigma | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \Psi_{(-\Sigma)})$, $\Lambda | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \Psi_{(-\Lambda)})$, $\alpha | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \Psi_{(-\alpha)})$, and $\beta | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \Psi_{(-\beta)})$ in which $\underline{\mathbf{y}} = (\mathbf{y}_1^\top, \dots, \mathbf{y}_n^\top)^\top$, $\underline{\mathbf{u}} = (\mathbf{u}_1^\top, \dots, \mathbf{u}_n^\top)^\top$, and $\mathbf{g}^\top = (g_1, \dots, g_n)$. First, we notice that

$$\begin{aligned}
\pi(\Psi | \underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}) &\propto L_c(\Psi | \underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}) \pi(\theta) \\
&= L_c(\Psi | \underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}) \pi(\boldsymbol{\mu}) \pi(\Sigma) \pi(\Lambda) \pi(\alpha) \pi(\beta),
\end{aligned} \tag{6.49}$$

where, as usual, we assumed that priors are independent and furthermore

$$\begin{aligned}\pi(\boldsymbol{\mu}) &\sim \mathcal{N}_p(\mathcal{M}_0, \mathcal{S}_0), \\ \pi(\Sigma) &\sim \mathcal{IW}(\mathcal{D}_0, \nu_0), \\ \pi(\Lambda) &\sim \mathcal{N}_q(\mathcal{L}_0, \mathcal{P}_0), \\ \pi(\alpha) &\sim \mathcal{G}(a_0, a_0), \\ \pi(\beta) &\sim \mathcal{G}(b_0, b_0).\end{aligned}$$

As it is seen, for priors of α and β , we assumed a gamma distribution with common shape and rate parameters. The results below give the full conditionals needed for implementing the Bayesian paradigm.

- **full conditional of $\boldsymbol{\mu} | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \boldsymbol{\Psi}_{(-\boldsymbol{\mu})})$:** By considering a conjugate prior, that is $\pi(\boldsymbol{\mu}) \sim \mathcal{N}_p(\mathcal{M}_0, \mathcal{S}_0)$, more algebra shows that

$$\pi(\boldsymbol{\mu} | \underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \boldsymbol{\Psi}_{(-\boldsymbol{\mu})}) = \mathcal{N}_p(\boldsymbol{\mu} | \mathcal{Q}_0 [\mathcal{S}_0^{-1} \mathcal{M}_0 + \Sigma^{-1} \sum_{i=1}^n g_i (\mathbf{y}_i - \Lambda \mathbf{u}_i)], \mathcal{Q}_0),$$

where

$$\mathcal{Q}_0 = \left[\mathcal{S}_0^{-1} + \Sigma^{-1} \sum_{i=1}^n g_i \right]^{-1}.$$

- **full conditional of $\Sigma | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \boldsymbol{\Psi}_{(-\Sigma)})$:** By considering a conjugate prior, that is $\Sigma \sim \mathcal{IW}(\mathcal{D}_0, \nu_0)$, we can write

$$\pi(\Sigma | \underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \boldsymbol{\Psi}_{(-\Sigma)}) = \mathcal{IW}(\cdot | \mathcal{R}_0, \nu_0 + n),$$

where $\mathcal{R}_0 = \mathcal{D}_0 + \sum_{i=1}^n g_i (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i) (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top$.

- **full conditional of $\Lambda | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \boldsymbol{\Psi}_{(-\Lambda)})$:** Although a possible choice for prior of Λ is a matrix normal normal distribution, but we prefer rather to sample from this full conditional in a slightly different method. We perform this by sampling from each column of Λ separately. Let $\Lambda_{(c)}$ denote the c -th column (for $c = 1, \dots, q$) of matrix Λ and $\Lambda_{(-c)}$ is the matrix Λ when its c -th column is removed. Likewise, assume $u_{i(c)}$ denote the c -th element of vector $\mathbf{u}_i$ and $\mathbf{u}_{i(-c)}$ is vector $\mathbf{u}_i$ when its c -th element is removed. Suppose $\boldsymbol{\xi}_i = \mathbf{y}_i - \boldsymbol{\mu} - \Lambda_{(-c)} \mathbf{u}_{i(-c)}$, then it can be seen that

$$\begin{aligned}(\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i)^\top \Sigma^{-1} (\mathbf{y}_i - \boldsymbol{\mu} - \Lambda \mathbf{u}_i) \\ = (\Lambda_{(c)} u_{i(c)} - \boldsymbol{\xi}_i)^\top \Sigma^{-1} (\Lambda_{(c)} u_{i(c)} - \boldsymbol{\xi}_i).\end{aligned}$$

Replacing above identity in the RHS of (6.37) and simultaneously assuming a multivariate Gaussian conjugate prior for $\Lambda_{(c)}$, that is $\Lambda_{(c)} \sim \mathcal{N}_p(\mathcal{L}_{0(c)}, \mathcal{P}_{0(c)})$, it turns out that

$$\Lambda_{(c)} | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \boldsymbol{\Psi}_{(-\Lambda_{(c)})}) \sim \mathcal{N}_p(\mathcal{V}, \mathcal{W}),$$

where, for $c = 1, \dots, q$, we have

$$\begin{aligned}\mathcal{V} &= \mathcal{W} \left[\mathcal{P}_{0(c)}^{-1} \mathcal{L}_{0(c)} + \Sigma^{-1} \sum_{i=1}^n g_i u_{i(c)} \boldsymbol{\xi}_i \right], \\ \mathcal{W} &= \left[\mathcal{P}_{0(c)}^{-1} + \Sigma^{-1} \sum_{i=1}^n g_i u_{i(c)}^2 \right]^{-1}.\end{aligned}$$

After simulating sequence $\{\Lambda_{(1)}, \Lambda_{(2)}, \dots, \Lambda_{(q)}\}$, the generated skewness matrix Λ is reconstructed as $\Lambda = [\Lambda_{(1)} | \Lambda_{(2)} | \dots | \Lambda_{(q)}]$.

- **full conditionals** $\alpha|(\underline{y}, \underline{u}, \underline{g}, \Psi_{(-\alpha)})$ and $\beta|(\underline{y}, \underline{u}, \underline{g}, \Psi_{(-\beta)})$: We have

$$\theta|(\underline{y}, \underline{u}, \underline{g}, \Psi_{(-\theta)}) \propto L_c(\Psi|\underline{y}, \underline{u}, \underline{g})\pi(\theta), \quad (6.50)$$

By assuming independence between marginals of $\pi(\theta)$, it follows from RHS of in (6.50) that

$$\begin{aligned} \pi(\alpha|\underline{y}, \underline{u}, \underline{g}, \Psi_{(-\alpha)}) &\propto \prod_{i=1}^n f(g_i|\theta)\pi(\alpha) \\ &= \frac{\beta^{n\alpha}\alpha_0^{\alpha_0}}{[\Gamma(\alpha)]^n\Gamma(\alpha_0)} \left(\prod_{i=1}^n g_i\right)^{\alpha-1} \alpha^{\alpha_0-1} \exp\{-\alpha_0\alpha\}. \end{aligned} \quad (6.51)$$

It can be easily seen that $\pi(\alpha|\underline{y}, \underline{u}, \underline{g}, \Psi_{(-\alpha)})$ is log-concave for $\alpha_0 \geq 1$, and hence the ARS approach is suggested for sampling from this full conditional. If one would like to employ the MH approach, one choice for candidate may be $\mathcal{G}(\alpha_0, \alpha_0)$. This yields the the acceptance ratio as

$$\begin{aligned} p(\alpha_k, \alpha_{k+1}) &= \min \left\{ 1, \left(\frac{\alpha_{k+1}}{\alpha_k} \right)^{\alpha_0-1} \left[\frac{\Gamma(\alpha_k)}{\Gamma(\alpha_{k+1})} \right]^n \beta^{n(\alpha_{k+1}-\alpha_k)} \right. \\ &\quad \left. \times \exp\left\{ \alpha_0(\alpha_k - \alpha_{k+1}) \right\} \left(\prod_{i=1}^n g_i \right)^{\alpha_{k+1}-\alpha_k} \right\}. \end{aligned} \quad (6.52)$$

Similarly, assuming $\pi(\beta) \sim \mathcal{G}(\beta_0, \beta_0)$. We have

$$\begin{aligned} \pi(\beta|\underline{y}, \underline{u}, \underline{g}, \Psi_{(-\beta)}) &\propto \prod_{i=1}^n f(g_i|\theta)\pi(\beta) \\ &\propto \beta^{n\alpha+\beta_0-1} \exp\left\{ -\beta(\beta_0 + \sum_{i=1}^n g_i) \right\}. \end{aligned} \quad (6.53)$$

From the RHS of (6.53), it turns out that $\beta|(\underline{y}, \underline{u}, \underline{g}, \Psi_{(-\beta)}) \sim \mathcal{G}(\beta_0 + n\alpha, \beta_0 + \sum_{i=1}^n g_i)$.

In what follows, Algorithm 19 describes how to compute the Bayesian estimator for parameters of skewed Student's t distribution.

Example 6-7

Herein, we suppose a sample of $n = 500$ observations are drawn from skewed Student's t distribution with parameters

$$\mu = \begin{bmatrix} 2 \\ -2 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 0.4 & 0 \\ 0 & 0.4 \end{bmatrix}, \quad \Lambda = \begin{bmatrix} -2 & 2 \\ 3 & 0 \end{bmatrix}, \quad \theta = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}.$$

We follow the steps of Algorithm 7 to obtain Bayesian estimator of Ψ . The pertaining R code for implementing this example is given below. As it may seen from R code, we hyperparameters as $\mathcal{M}_0 = (0, 0)^\top$, $\mathcal{S}_0 = \mathcal{D}_0 = \mathcal{P}_0 = \mathbf{I}_p$, $\nu_0 = 2$, $\mathcal{L}_0 = (0, 0)^\top$, $\beta_0 = 2$, and $\alpha_0 = 2$. The output of the Gibbs sampler is displayed in Figure 6.6.

```

1 R > f.a <- function(x, beta, alpha0, y)
2 +   {
3 +     n = length(y)
4 +     n*x*log(beta) - n*lgamma(x) + (x-1)*sum(log(y)) +
5 +       (alpha0-1)*log(x) - alpha0*x
6 +   }

```

Algorithm 19 Bayesian inference for skewed Student's t distribution

```

1: Read  $N$ ,  $M$ , and determine quantities  $\mathcal{M}_0$ ,  $\mathcal{S}_0$ ,  $\mathcal{D}_0$ ,  $\nu_0$ ,  $\mathcal{L}_0$ ,  $\mathcal{P}_0$ ,  $a_0$ , and
2:  $\boldsymbol{\theta}_0 = (a_0, b_0)^\top$ ;
3: Set  $t \leftarrow 0$ ;
4: Set  $\boldsymbol{\mu}^{(0)} \leftarrow \mathcal{M}^0$ ,  $\Sigma^{(0)} \leftarrow 1/n \sum_{i=1}^n (\mathbf{y}_i - \boldsymbol{\mu}^{(0)})(\mathbf{y}_i - \boldsymbol{\mu}^{(0)})^\top$ ,  $\nu^{(0)} = 2$ ,
5: and set  $\boldsymbol{\Psi}^{(0)} = (\boldsymbol{\mu}^{(0)}, \Sigma^{(0)}, \Lambda^{(0)}, \boldsymbol{\theta}^{(0)})$ ;
6: while  $t \leq N$  do
7:   Set  $i \leftarrow 1$ ;
8:   while  $i \leq n$  do
9:     Simulate  $g_i$  from  $\mathcal{G}(a, b)$  where  $a = \alpha^{(t)} + (p + q)/2$  and  $b =$ 
10:     $\left[ 2\beta^{(t)} + (\mathbf{y}_i - \boldsymbol{\mu}^{(t)} - \Lambda^{(t)}\mathbf{u}_i)^\top [\Sigma^{(t)}]^{-1} (\mathbf{y}_i - \boldsymbol{\mu}^{(t)} - \Lambda^{(t)}\mathbf{u}_i) \right] / 2 + \mathbf{u}_i^\top \mathbf{u}_i / 2$ ;
11:     Simulate  $\mathbf{u}_i$  from  $\mathcal{HN}_q(\mathbf{m}_i^{(t)}, \frac{\Delta^{(t)}}{g_i})$  where  $\mathbf{m}_i^{(t)} = [\Lambda^{(t)}]^\top [\Omega^{(t)}]^{-1} (\mathbf{y}_i -$ 
12:      $\boldsymbol{\mu})$  with  $\Omega^{(t)} = \Sigma^{(t)} + \Lambda^{(t)} [\Lambda^{(t)}]^\top$  and  $\Delta^{(t)} = \mathbf{I}_q - [\Lambda^{(t)}]^\top [\Omega^{(t)}]^{-1} \Lambda^{(t)}$ ;
13:     Set  $i \leftarrow i + 1$ ;
14:   end
15:   Generate  $\boldsymbol{\mu}^{(t+1)} \sim \mathcal{N}_p(\mathcal{Q}_0 [\mathcal{S}_0^{-1} \mathcal{M}_0 + [\Sigma^{(t)}]^{-1} \sum_{i=1}^n g_i (\mathbf{y}_i - \Lambda^{(t)} \mathbf{u}_i)], \mathcal{Q}_0)$ 
16:   where  $\mathcal{Q}_0 = [\mathcal{S}_0^{-1} + [\Sigma^{(t)}]^{-1} \sum_{i=1}^n g_i]^{-1}$ ;
17:   Generate  $\Sigma^{(t+1)}$  from  $\mathcal{IW}(\mathcal{R}_0, \nu_0 + n)$  where  $\mathcal{R}_0 = \mathcal{D}_0 + \sum_{i=1}^n g_i (\mathbf{y}_i -$ 
18:    $\boldsymbol{\mu}^{(t+1)}) (\mathbf{y}_i - \boldsymbol{\mu}^{(t+1)})^\top$ ;
19:   Set  $j \leftarrow 1$ ;
20:   while  $j \leq q$  do
21:     Set  $\mathcal{W}^{-1} = [\mathcal{P}_{0(j)}^{-1} + [\Sigma^{-1}]^{(t+1)} \sum_{i=1}^n g_i u_{i(j)}^2]$ ,  $\mathcal{V} = \mathcal{W} [\mathcal{P}_{0(j)}^{-1} \mathcal{L}_{0(j)} +$ 
22:      $[\Sigma^{(t+1)}]^{-1} \sum_{i=1}^n g_i u_{i(j)} \boldsymbol{\xi}_i]$ , and  $\boldsymbol{\xi}_i = \mathbf{y}_i - \boldsymbol{\mu}^{(t+1)} - \Lambda_{(-j)}^{(t)} \mathbf{u}_{i(-j)}$ ;
23:     Generate  $\Lambda_{(j)} \sim \mathcal{N}_p(\mathcal{V}, \mathcal{W})$ ;
24:   end
25:   Construct  $\Lambda^{(t+1)} = [\Lambda_{(1)} | \Lambda_{(2)} | \dots | \Lambda_{(q)}]$ ;
26:   Use the ARS-within-Gibbs sampling technique for simulating  $\alpha^{(t+1)}$  while
27:    $\log \pi(\alpha | \underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \boldsymbol{\Psi}_{(-\alpha)}) \propto n\alpha \log \beta - n \log \Gamma(\alpha) + (\alpha - 1) \sum_{i=1}^n \log g_i +$ 
28:    $(\alpha_0 - 1) \log \alpha - \alpha_0 \alpha$ 
29:   Simulate  $\beta^{(t+1)}$  from full conditional  $\beta | (\underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g}, \boldsymbol{\Psi}_{(-\beta)}) \sim \mathcal{G}(\beta_0 +$ 
30:    $n\alpha^{(t+1)}, \beta_0 + \sum_{i=1}^n g_i)$ ;
31:   end
32:   Set  $\boldsymbol{\Psi}^{(t+1)} \leftarrow (\boldsymbol{\mu}^{(t+1)}, \Sigma^{(t+1)}, \Lambda^{(t+1)}, \boldsymbol{\theta}^{(t+1)})$  and  $t \leftarrow t + 1$ ;
33: end
34: Sequence  $\{\boldsymbol{\Psi}^{(M)}, \boldsymbol{\Psi}^{(M+1)}, \dots, \boldsymbol{\Psi}^{(N)}\}$  is a sample of size  $N - M + 1$  from
35: posterior  $\pi(\boldsymbol{\Psi} | \underline{\mathbf{y}}, \underline{\mathbf{u}}, \mathbf{g})$ ;
36: The Bayesian estimator of  $\boldsymbol{\Psi}$  is given by  $(N - M + 1)^{-1} \sum_{t=1}^{N-M+1} \boldsymbol{\Psi}^{(t)}$ .

```

```

7 R > fprim.a <- function(x, beta, alpha0, y)
8 + {
9 +   n = length(y)
10 +   n*log(beta) - n*digamma(x) + sum(log(y)) + (alpha0-1)/x - alpha0
11 + }
12 R> rssg <- function(n, alpha, beta, Mu, Sigma, Lambda)
13 + {
14 +   Dim <- length(Mu)
15 +   Q <- length(Lambda[1,])
16 +   Y <- matrix(NA, nrow = n, ncol = Dim)
17 +   for (i in 1:n)
18 +   {
19 +     Z <- rgamma(1, shape = alpha, rate = beta)
20 +     X <- mvrnorm(n= 1, mu = rep(0, Dim), Sigma = Sigma )
21 +     u <- abs( rnorm(Q) )
22 +     Y[i, ] <- Mu + Lambda%*%u/sqrt(Z) + X/sqrt(Z)
23 +   }
24 +   Y
25 + }
26 R> set.seed(20240520)
27 R> library(MASS); library(tmvtnorm); library(ars)
28 R> n <- 500; alpha <- 5; beta <- 5; Sigma <- matrix( c(.4,0,0,.4), 2, 2);
29 R> Mu <- c(2, -2); Lambda <- matrix( c(-2,3,2,0), 2, 2)
30 R> Y <- rssg(n, alpha, beta, Mu, Sigma, Lambda );
31 R> g <- rgamma(n, shape = alpha, rate = beta);
32 R> Q <- dim(Lambda)[2]
33 R> p <- length(Mu)
34 R> T <- matrix( abs(rnorm(n*Q, mean = 0, sd = 1)), nrow = n, ncol = Q)
35 R> N <-10000; M <-5000; Mu0c <-rep(0, p); beta0 <-2; alpha0 <-2; nu0 <-2;
36 R> SOMu <- diag(p); MOMu <- rep(0, p);
37 R> D0 <- S0c <- P0c <-diag(p);
38 R> Psi <-matrix(0, nrow = N, ncol = p + p^2 + p*Q + 2 )
39 R> for(k in 1:N)
40 + {
41 +   Sigmainv <- solve(Sigma)
42 +   Omega <- Sigma + tcrossprod( Lambda )
43 +   Omegainv <- solve(Omega)
44 +   Delta <- diag(Q) - (t(Lambda)%*%Omegainv%*%Lambda)
45 +   for(i in 1:n)
46 +   {
47 +     rate <- mahalanobis(x = Y[i, ], center = Mu + Lambda%*%T[i, ], cov = Sigma )/2 + (T[i,]
48 + %*%T[i,])[1]/2 + beta
49 +     g[i] <- rgamma(1, shape = alpha + (p + Q)/2, rate = rate )
50 +     T[i,] <- rtmvnorm(1, mean = c(t(Lambda)%*%Omegainv%*%c( Y[i, ] - Mu )), sigma = Delta/g
51 + [i,
52 + lower = rep(0, length = Q),upper = rep( Inf, length = Q), algorithm = c("rejection") )
53 +   }
54 +   Sum_g <- sum(g)
55 +   M_sum <- rowSums( sapply(1:n, function(i) g[i]*c( Y[i, ] -
56 + Lambda%*%T[i, ] ) ) )
57 +   Q0 <- solve( solve( SOMu ) + Sigmainv*Sum_g )
58 +   Mu <- c( mvrnorm(1, mu = Q0%*%( solve( SOMu )%*%MOMu + Sigmainv%*%M_sum ),
59 + Sigma = Q0 ) );
60 +   Psi[k, 1:p] <- Mu
61 +   R <- matrix(0, nrow = p, ncol = p)
62 +   for(i in 1:n)
63 +   {
64 +     R <- R + g[i]*c( Y[i, ] - Mu - Lambda%*%T[i, ])%*%t( c( Y[i, ] - Mu -
65 + Lambda%*%T[i, ] )
66 +   )}
67 +   Sigma <- solve( rWishart(1, df = n+nu0+1, Sigma = solve(D0 + R) )[,1] )
68 +   Psi[k, (p + 1):(p + p^2)] <- as.numeric(Sigma)
69 +   for(j in 1:Q)

```

```

68 + {
69 +   Sumgt <- sum( g*T[, j]^2 )
70 +   Sc <- solve( solve( S0c ) + Sigmainv*Sumgt )
71 +   if(Q >=3)
72 +   {
73 +     Mc_sum <- rowSums( sapply(1:n,function(i) g[i]*T[i, j]*c( Y[i, ] -
74 +       Mu - Lambda[,-j]*%T[i, -j]) ) )
75 +   }else{
76 +     Mc_sum <- rowSums( sapply(1:n,
77 +       function(i) g[i]*T[i, j]*c( Y[i, ] - Mu - Lambda[,-j]*T[i, -j]) ) ) )
78 +   }
79 +   Mc <- Sc%*( solve( S0c )%Mu0c + Sigmainv%Mc_sum )
80 +   Lambda[, j] <- mvrnorm(n= 1, mu = Mc , Sigma = Sc )
81 + }
82 + Psi[k, (p + p^2 + 1):(p + p^2 + p*Q)] <- as.numeric(Lambda)
83 + alpha <- ars(1, f.a, fprim.a, x=c(0.5, 4, 10, 30), m=4, lb=TRUE, xlb=0,
84 +   ub=FALSE, beta=beta, alpha0=alpha0, y=g )
85 + beta <- rgamma( 1, n*alpha + beta0, beta0 + sum(g) )
86 + # b0 <- 1/(3-(cov(Y)%solve(Sigma))[1,1])
87 + Psi[k, (p + p^2 + p*Q + 1)] <- alpha
88 + Psi[k, (p + p^2 + p*Q + 2)] <- beta
89 + }
90 R > Psi.hat <- apply(Psi[(N-M+1):N,], 2, mean)
91 R > Mu <- Psi.hat[1:p]
92 R > Sigma <- Psi.hat[(p+1):(p + p^2)]
93 R > Lambda <- Psi.hat[(p + p^2 + 1):(p + p^2 + p*Q)]
94 R > alpha <- Psi.hat[p + p^2 + p*Q + 1]
95 R > beta <- Psi.hat[p + p^2 + p*Q + 2]

```



6.6 Gibbs sampling in Bayesian paradigm with more than two latent variables: Model-based Clustering

The aim of clustering technique is to divide a non-homogeneous population into K numbers of homogeneous sub-populations. Obviously, members of a non-homogeneous population have dissimilarities while the clustered or partitioned sub-populations have more similarities. Herein, the term “similarity” may refer to some measurable attribute or criterion such as Euclidean distance. If two observations are similar, then they are close together (high degree of similarity) and otherwise, two observations are dissimilar (low degree of similarity).

Figure 6.9(a) displays a set of ten observations, labeled with numbers $1, 2, \dots, 10$, that are similar and so can constitute a cluster. On the other hand, consider the situation in which ten observations are non-homogeneous or dissimilar as shown by Figure 6.9(b). Herein, ten observations with labels $\{7, 6, 1, 4\}$ are similar together, but quite dissimilar with observations whose label are $\{2, 8, 1, 3, 9, 5\}$. So, observations in Figure 6.9(b) should be groped into two clusters. We emphasize that the objective of clustering technique is to determine the label of each observation. Furthermore, sometimes the yardstick for measuring the similarity between observations may not be a quantitative tool such as the Euclidean distance. The clustering technique proceeds to divide observations into K groups such that the observations within each group have the maximum degree of similarity while the observations between groups have the maximum degree of dissimilarity. Sometimes the clustering technique is known as semi-supervised or unsupervised machine learning since label of observations are not known and the only available information are observed data McNicholas [2016].

The model-based clustering (or non-hierarchical clustering) uses a finite mixture of multivariate statistical distributions of the same family for the purpose of clustering. Hence, the model-based clustering and finite mixture models are often used interchangeably in the litera-

ture. As mentioned above, the aim of clustering is to determine the label or the origin of an observation is coming from. It is worth to note that the concepts “origin”, “cluster”, “label”, and “component” may be used interchangeably. For more clarifications, Figure 6.9(c) shows a set of ten observations with perfect similarity and, within the model-based clustering framework, we assume that these ten observations follow independently the same multivariate distribution. On the other hand, Figure 6.9(d) shows another set of ten observations with two clusters, and hence within the model-based clustering framework, we prefer to assume these data points are coming from two different multivariate statistical distribution of the same family. In other words, for the first and second clusters we consider two multivariate statistical distributions of the same family whose PDFs are $f(\mathbf{y}|\Theta_1)$ and $f(\mathbf{y}|\Theta_2)$, respectively. The corresponding weights are 0.4 and 0.6 and hence the distribution of ten observations is a mixture of two distributions fitted to two clusters whose PDF is

$$g(\mathbf{y}|\Psi) = 0.4f(\mathbf{y}|\Theta_1) + 0.6f(\mathbf{y}|\Theta_2). \quad (6.54)$$

In (6.54), the vector $\omega = (0.40, 0.60)^\top$, is known as the weight vector that means 40% of observations in Figure 6.9(d) are coming from the first component and the remaining 60% are coming from the second component. Overall, for p -dimensional random vector $\mathbf{Y}$, the PDF of a K-component mixture model is given by

$$g(\mathbf{y}|\Psi) = \sum_{k=1}^K \omega_k f(\mathbf{y}|\Theta_k), \quad (6.55)$$

where $\Psi = (\omega^\top, \Theta_1, \dots, \Theta_K)$ is the parameter space in which $\omega = (\omega_1, \dots, \omega_K)^\top$ is vector of mixing parameters such that $\sum_{k=1}^K \omega_k = 1$. In (6.55), $f_{\mathbf{Y}}(\mathbf{y}|\Theta_k)$ is the PDF of k -th component and hence the PDF $g(\mathbf{y}|\Psi)$ is proper since

$$\int_{\mathbb{R}^p} g(\mathbf{y}|\Psi) d\mathbf{y} = \sum_{k=1}^K \omega_k \int_{\mathbb{R}^p} f(\mathbf{y}|\Theta_k) d\mathbf{y} = \sum_{k=1}^K \omega_k = 1.$$

The weight parameter ω provides the chance of observing sample from components. Sometimes, representation (6.55) is called the *finite mixture model* since the quantity K or number of clusters is assumed to be finite. In next two subsections, we proceed to discuss about generating realization from a finite mixture model and how to determine quantity K.

6.6.1 Generating from Gaussian finite mixture model

For generating n realizations from a K-component mixture model, we may follow one of the two following approaches. In the first method, one can generate independently $n_k = \lceil n \times \omega_k \rceil$ (for $k = 1, \dots, K-1$) realizations from PDF $f(\mathbf{y}|\Theta_k)$ in which $\lceil n \times \omega_k \rceil$ denotes the smallest integer value equal or greater than $n \times \omega_k$, and finally $n_K = n - \sum_{k=1}^{K-1} n_k$ realizations are drawn independently from PDF $f(\mathbf{y}|\Theta_K)$. The following pseudo code describes how to draw a sample of size n from a K-component Gaussian mixture model under the second method.

Example 6-8

Suppose we are interested in generating a sample of $n = 300$ observations from observations are coming from a three-component bivariate Gaussian mixture model with PDF given by

$$g(\mathbf{y}|\Psi) = \frac{1}{3}\mathcal{N}_2(\boldsymbol{\mu}_1, \Sigma_1) + \frac{1}{3}\mathcal{N}_2(\boldsymbol{\mu}_2, \Sigma_2) + \frac{1}{3}\mathcal{N}_2(\boldsymbol{\mu}_3, \Sigma_3), \quad (6.56)$$

Algorithm 20 Simulating from K-component Gaussian mixture model.

- 1: Read n , K (number of components), and $\Psi = (\omega, \Theta_1, \dots, \Theta_K)$;
 - 2: Set $\mathbf{Y}$ a vector of length n ;
 - 3: Set $i = 1$;
 - 4: **while** $i \leq n$ **do**
 - 5: Sample $\mathbf{u} \sim \text{MUL}(1, \omega)$;
 - 6: Set $k_{\max} = \{k | \mathbf{u}[k] = 1\}$ where $\mathbf{u}[k]$ is the k th element of vector $\mathbf{u}$ for
 - 7: $k = 1, \dots, K$;
 - 8: Sample $\mathbf{y}$ from PDF $f(\cdot | \Theta_{k_{\max}})$
 - 9: Set $\mathbf{Y}[i] \leftarrow \mathbf{y}$;
 - 10: Set $i \leftarrow i + 1$;
 - 11: **end**
 - 12: Accept $\mathbf{Y}$ as sample of size n from K-component mixture model.
-

where $\Psi = (\omega_1, \omega_2, \omega_3, \Theta_1, \Theta_2, \Theta_3)$ is whole parameter vector whose elements are

$$\begin{aligned}\Theta_1 &= \omega_1 = \frac{1}{3}, \boldsymbol{\mu}_1 = (-3, -3)^\top, \Sigma_1 = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}, \\ \Theta_2 &= \omega_2 = \frac{1}{3}, \boldsymbol{\mu}_2 = (0, 0)^\top, \quad \Sigma_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \\ \Theta_3 &= \omega_3 = \frac{1}{3}, \boldsymbol{\mu}_3 = (3, 3)^\top, \quad \Sigma_3 = \begin{bmatrix} 1 & -0.25 \\ -0.25 & 1 \end{bmatrix}.\end{aligned}\tag{6.57}$$

Based on Algorithm 20, we use the following R code to generate from three-component bivariate Gaussian mixture model with whole parameter given as above. Figure 6.10(a) displays the corresponding scatterplot. ■

```
1 R> library(MASS) # for generating realization from multivariate Gaussian
2 R> set.seed(20240713)
3 R> K <- 3
4 R> omega <- rep(1/3, K)
5 R> Mu1 <- rep(-3, p); Mu2 <- rep(0, p); Mu3 <- c(3, -3)
6 R> Sigma1 <- diag(0.5, p) + matrix(.5, p, p)
7 R> Sigma2 <- diag(1, p)
8 R> Sigma3 <- diag(1.5, p) + matrix(-0.5, p, p)
9 R> Sigma2 <- matrix(c(1, 0, 0, 1), 2, 2);
10 R> Sigma3 <- matrix(c(1, -0.25, -0.25, 1), 2, 2);
11 R> Mu <- list(Mu1, Mu2, Mu3)
12 R> Sigma <- list(Sigma1, Sigma2, Sigma3)
13 R> rmixnorm <- function(n, Mu, Sigma, omega)
14 + {
15 +   p <- length( Mu[[1]] )
16 +   label <- rep(0, n)
17 +   Y <- matrix(0, nrow = n, ncol = p)
18 +   for(i in 1:n)
19 +   {
20 +     r_MUL <- rmultinom(n = 1, size = 1, omega)
21 +     k <- apply(r_MUL, 2, which.max)
22 +     label[i] <- k
23 +     Y[i, ] <- mvrnorm(n = 1, mu = Mu[[k]], Sigma = Sigma[[k]])
24 +   }
25 +   ls <- list( "sample" = Y, "label" = label )
26 +   return(ls)
27 + }
28 R> Y <- rmixnorm(n, Mu, Sigma, omega)
29 R> plot( Y$sample, col = Y$label )
```

6.6.2 Model selection

The true value of K is not known in practice. This quantity can be determined through the Bayesian information criterion (BIC) [Schwarz et al., 1978]. The process of finding best K is known as the *model selection* in the literature. Based on n realizations $\mathbf{y}_1, \dots, \mathbf{y}_n$ that are assumed to follow independently a distribution with PDF $g(\mathbf{y}|\Psi)$, the BIC is defined as

$$\text{BIC} = 2 \sum_{i=1}^n \log \sum_{k=1}^K \widehat{\omega}_k f(\mathbf{y}_i | \widehat{\Theta}_k) - N \log n, \quad (6.58)$$

where $\widehat{\omega}_k$ and $\widehat{\Theta}_k$ (for $k = 1, \dots, K$) account for the ML estimators of ω_k and Θ_k , respectively. Furthermore, constant N denotes number of model's free parameters that must be estimated. The BIC is an effective criterion for model selection in Gaussian mixture models, see Fraley and Raftery [2002], McNicholas and Murphy [2008], and has been used frequently for other model-based clustering, see Keribin [2000], Fraley and Raftery [1998, 2002], McNicholas and Murphy [2008], McNicholas and Murphy [2010]. In practice, for determining K , we compute BIC defined by (6.58) when data points are clustered through other clustering approaches such as k-mean [Hartigan and Wong, 1979] or other approaches of clustering, for $k = 1, 2, \dots$. We choose the mixture model for which BIC is the largest. For instance, suppose that we do not know the true value of K when 300 observations are coming from a three-component bivariate Gaussian mixture model with true parameters given by (6.57). As expected, when $K = 3$ the BIC gets its maximum value, see Figure 6.10(b). The R function BIC has been developed for determining the true value of K for a K -component Gaussian mixture model.

```

1  R> dmvnorm <- function(x, mean, sigma)
2  + {
3  +   p <- length(mean)
4  +   1/sqrt((2*pi)^p*det(sigma))*exp(-mahalanobis(x, mean, sigma)/2)
5  + }
6  R> BIC <- function(Y, param = NULL, k_max = 6)
7  + {
8  +   dim_y <- dim(Y)
9  +   n <- dim_y[1]
10 +   p <- dim_y[2]
11 +   if( is.null(param) )
12 +   {
13 +     bic <- rep(0, k_max)
14 +     for(k in 1:k_max)
15 +     {
16 +       Mu_hat <- matrix(0, nrow = k, ncol = p)
17 +       Sigma_hat <- array(0, dim = c(p, p, k) )
18 +       N <- (k - 1) + k*( p + p*(p + 1)/2 )
19 +       cluster <- kmeans(Y, k)
20 +       omega <- cluster$size/n
21 +       Sum_bic <- matrix(0, nrow = n, ncol = k)
22 +       for(i in 1:k)
23 +       {
24 +         clust <- Y[cluster$cluster == i, ]
25 +         Mu_hat[i,] <- apply(clust, 2, mean)
26 +         Sigma_hat[,i] <- cov(clust)
27 +         Sum_bic[, i] <- omega[i]*dmvnorm(Y, mean = Mu_hat[i,],
28 +                                         sigma = Sigma_hat[,i])
29 +       }
30 +       bic[k] <- 2*sum( log( apply(Sum_bic, 1, sum) ) ) - N*log(n)
31 +     }
32 +     out <- data.frame(k = 1:k, bic = bic)
33 +     ls <- list( "model" = out, "K" = which.max(bic) )
34 +   }else{

```

```

35 + omega <- param[[1]]
36 + Mu <- param[[2]]
37 + Sigma <- param[[3]]
38 + K <- length( Mu )
39 + Sum_bic <- matrix(0, nrow = n, ncol = K)
40 + N <- (K - 1) + K*( p + p*(p + 1)/2 )
41 + for(k in 1:K)
42 + {
43 +   Sum_bic[, k] <- omega[k]*dmvnorm(Y, mean = Mu[[k]],
44 +                                     sigma = Sigma[[k]])
45 + }
46 + out <- 2*sum( log( apply(Sum_bic, 1, sum) ) ) - N*log(n)
47 + ls <- list( "bic" = out, "K" = K )
48 + }
49 + return(ls)
50 + }

```

As noted earlier, determining the label of each observation is the main task of each clustering technique. To this end, we may use the Bayesian (Gibbs sampling) or the EM algorithm. If the former applied, depending on the model structure, the Gibbs sampling involves one, two, or more latent variables.

6.6.3 Finite mixture of Gaussian distributions

Evidently the most commonly used statistical distribution for the purpose of model-based clustering is Gaussian mixture model. Suppose p -dimensional random vector $\mathbf{Y}$ follows a K -component Gaussian mixture model. It is clear that

$$\begin{aligned} \mathbf{Y}|H_{ik} = 1 &\sim \mathcal{N}_p(\boldsymbol{\mu}_k, \Sigma_k), \\ \mathbf{H}_i &\sim \mathcal{MUL}(\boldsymbol{\omega}|1). \end{aligned}$$

Since $\mathbf{H}_i$ is latent variable, hence we encounter with a incomplete data problem. Here $(\mathbf{y}_1^\top, \dots, \mathbf{y}_n^\top)$ is the sequence of observed data and $\mathbf{H}_i^\top = (H_{i1}, \dots, H_{iK})$ is the missing component's label. Herein $H_{ik} = 1$ takes on values zero and one. If $H_{ik} = 1$, then $\mathbf{y}_i^\top$ comes from k -th component, otherwise $H_{i1} = 0$. Hence, the complete data PDF can be represented as

$$g_c(\underline{\mathbf{y}}|\boldsymbol{\Psi}) = \prod_{i=1}^n g(\mathbf{y}_i, \mathbf{H}_i|\boldsymbol{\Psi}), \quad (6.59)$$

where $\underline{\mathbf{y}}^\top = \{\mathbf{y}_1^\top, \dots, \mathbf{y}_n^\top\}$ and

$$\begin{aligned} g(\mathbf{y}_i, \mathbf{H}_i|\boldsymbol{\Psi}) &= g(\mathbf{y}_i|\mathbf{H}_i, \boldsymbol{\Psi}) \times \mathcal{MUL}(\boldsymbol{\omega}|1) \\ &= \left[\mathcal{N}_p(\mathbf{y}_i|\boldsymbol{\mu}_k, \Sigma_k) \right]^{H_{ik}} \times \omega_1^{H_{i1}} \times \dots \times \omega_K^{H_{iK}} \\ &= \prod_{k=1}^K \left[\omega_k \mathcal{N}_p(\mathbf{y}_i|\boldsymbol{\mu}_k, \Sigma_k) \right]^{H_{ik}}. \end{aligned} \quad (6.60)$$

Replacing the RHS of (6.60) into the RHS of (6.59), the PDF of complete-data is

$$g_c(\underline{\mathbf{y}}|\boldsymbol{\Psi}) = \prod_{i=1}^n \prod_{k=1}^K \left[\omega_k \mathcal{N}_p(\mathbf{y}_i|\boldsymbol{\mu}_k, \Sigma_k) \right]^{H_{ik}}. \quad (6.61)$$

Thus, the complete-data likelihood function is

$$\begin{aligned}
L_c(\Psi|\underline{\mathbf{y}}, \underline{\mathbf{H}}) &= (2\pi)^{-\frac{np}{2}} \times \omega_k^{\sum_{i=1}^n H_{ik}} \times \dots \times \omega_K^{\sum_{i=1}^n H_{iK}} \times |\Sigma_k|^{-\frac{1}{2} \sum_{i=1}^n \sum_{k=1}^K H_{ik}} \\
&\times \exp\left\{-\frac{1}{2} \sum_{i=1}^n \sum_{k=1}^K H_{ik} (\mathbf{y}_i - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1} (\mathbf{y}_i - \boldsymbol{\mu}_k)\right\}, \\
&\propto \omega_1^{n_1} \times \dots \times \omega_K^{n_K} \times |\Sigma_1|^{-\frac{n_1}{2}} \times \dots \times |\Sigma_K|^{-\frac{n_K}{2}} \\
&\times \exp\left\{-\frac{1}{2} \sum_{i=1}^n \sum_{k=1}^K H_{ik} (\mathbf{y}_i - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1} (\mathbf{y}_i - \boldsymbol{\mu}_k)\right\}, \tag{6.62}
\end{aligned}$$

where $\underline{\mathbf{H}}^\top = (\mathbf{H}_1^\top, \dots, \mathbf{H}_n^\top)$ in which $\mathbf{H}_i^\top = (H_{i1}, \dots, H_{iK})$ and $n_k = \sum_{i=1}^n H_{ik}$ provided that $n = \sum_{k=1}^K n_k$. Given i th observed data $\mathbf{y}_i^\top$, for computing PDF of $\mathbf{H}_i|\mathbf{Y} = \mathbf{y}_i$, it follows from (6.60) that

$$L_c(\Psi|\mathbf{y}_i, \mathbf{H}_i) \propto g(\mathbf{y}_i|\mathbf{H}_i, \Psi) \times \mathcal{MUL}(\omega|1).$$

Obviously,

$$\mathbf{H}_i|\Psi, \mathbf{Y} = \mathbf{y}_i \sim \mathcal{MUL}(\boldsymbol{\tau}_i|1), \tag{6.63}$$

where $\boldsymbol{\tau}_i^\top = (\tau_{i1}, \dots, \tau_{iK})$ denotes the i th row of $n \times K$ matrix $\boldsymbol{\tau}$ whose (i, k) th element is

$$\tau_{i,k} = \frac{\omega_k \mathcal{N}_p(\mathbf{y}_i|\boldsymbol{\mu}_k, \Sigma_k)}{\sum_{k=1}^K \omega_k \mathcal{N}_p(\mathbf{y}_i|\boldsymbol{\mu}_k, \Sigma_k)}. \tag{6.64}$$

Once we have generated the entire sample (for $i = 1, \dots, n$) from full conditional $\mathbf{H}_i|(\Psi, \mathbf{y}_i)$, we proceed to generate from full conditionals $\boldsymbol{\mu}_k|(\underline{\mathbf{y}}, \underline{\mathbf{H}}, \Psi_{(-\boldsymbol{\mu}_k)})$ and $\Sigma_k|(\underline{\mathbf{y}}, \underline{\mathbf{H}}, \Psi_{(-\Sigma_k)})$ (for $k = 1, \dots, K$). To this end, first we have

$$\begin{aligned}
\pi(\Psi|\underline{\mathbf{y}}, \underline{\mathbf{H}}) &\propto L_c(\Psi|\underline{\mathbf{y}}, \underline{\mathbf{H}}) \times \pi(\Psi_1) \times \dots \times \pi(\Psi_K) \\
&= L_c(\Psi|\underline{\mathbf{y}}, \underline{\mathbf{H}}) \times \pi(\omega) \times \pi(\boldsymbol{\mu}_1) \times \dots \times \pi(\boldsymbol{\mu}_K) \times \pi(\Sigma_1) \times \dots \times \pi(\Sigma_K). \tag{6.65}
\end{aligned}$$

Substituting $L_c(\Psi|\underline{\mathbf{y}}, \underline{\mathbf{H}})$ given by (6.62) into the RHSS of (6.65) yields

$$\begin{aligned}
\pi(\Psi|\underline{\mathbf{y}}, \underline{\mathbf{H}}) &\propto \pi(\omega) \times \pi(\boldsymbol{\mu}_1) \times \dots \times \pi(\boldsymbol{\mu}_K) \times \pi(\Sigma_1) \times \dots \times \pi(\Sigma_K) \times \\
&\omega_1^{n_1} \times \dots \times \omega_K^{n_K} \times |\Sigma_1|^{-\frac{n_1}{2}} \times \dots \times |\Sigma_K|^{-\frac{n_K}{2}} \\
&\times \exp\left\{-\frac{1}{2} \sum_{i=1}^n \sum_{k=1}^K H_{ik} (\mathbf{y}_i - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1} (\mathbf{y}_i - \boldsymbol{\mu}_k)\right\}. \tag{6.66}
\end{aligned}$$

We consider (for $k = 1, \dots, K$) independent conjugate priors as follows.

$$\pi(\omega) \sim \mathcal{DIR}(\boldsymbol{\rho}), \tag{6.67}$$

$$\pi(\boldsymbol{\mu}_k) \sim \mathcal{N}_p(\mathcal{M}_k, \mathcal{S}_k), \tag{6.68}$$

$$\pi(\Sigma_k) \sim \mathcal{IW}(\mathcal{D}_k, \nu_k), \tag{6.69}$$

where $\mathcal{DIR}(\cdot)$ is defined in (2.20). In what follows, we give the full conditionals needed for implementing the Gibbs sampling.

- **full conditional of $\omega|(\underline{\mathbf{y}}, \underline{\mathbf{H}}, \Psi_{(-\omega)})$:** Substituting prior $\pi(\omega)$ given by (6.66) into the RHS of (6.65) yields

$$\pi(\omega|\underline{\mathbf{y}}, \underline{\mathbf{H}}, \Psi_{(-\omega)}) \propto \mathcal{DIR}(\omega|\boldsymbol{\rho}) \times \omega_1^{n_1} \times \dots \times \omega_K^{n_K} = \mathcal{DIR}(\omega|\boldsymbol{\rho}_*),$$

where

$$\boldsymbol{\rho}_* = \left(\rho_1 + \sum_{i=1}^n H_{i1}, \dots, \rho_K + \sum_{i=1}^n H_{iK} \right)^\top.$$

- **full conditional of $\mu_k | (\underline{y}, \underline{H}, \Psi_{(-\mu_k)})$:** Substitute the prior $\pi(\mu_k)$ given by (6.67) into the RHS of (6.65). Then more algebra shows

$$\pi(\mu | \underline{y}, \underline{H}, \Psi_{(-\mu_k)}) = \mathcal{N}_p(\mu | \mathcal{Q}_k [\mathcal{S}_k^{-1} \mathcal{M}_k + \Sigma_k^{-1} \sum_{i=1}^n H_{ik} \mathbf{y}_i], \mathcal{Q}_k),$$

where

$$\mathcal{Q}_k = [\mathcal{S}_k^{-1} + \Sigma_k^{-1} \times \sum_{i=1}^n H_{ik}]^{-1}.$$

- **full conditional of $\Sigma_k | (\underline{y}, \underline{H}, \Psi_{(-\Sigma_k)})$:** Substitute the prior $\pi(\Sigma_k)$ given by (6.68) into the RHS of (6.65). Then more algebra shows

$$\pi(\Sigma | \underline{y}, \underline{H}, \Psi_{(-\Sigma_k)}) \sim \mathcal{IW}(\mathcal{R}_k, \nu_k + \sum_{i=1}^n H_{ik}),$$

where $\mathcal{R}_k = \mathcal{D}_k + \sum_{i=1}^n H_{ik} (\mathbf{y}_i - \mu_k)(\mathbf{y}_i - \mu_k)^\top$.

The following pseudo code gives the details for implementing the model-based clustering for finite mixture of Gaussian distributions.

Algorithm 21 Bayesian inference for finite mixture of Gaussian distributions

- 1: Read N , M , and determine quantities K , ω_k , $\mathcal{M}_k$, $\mathcal{S}_k$, $\mathcal{D}_k$, ν_k , for $k = 1 \dots K$
 - 2: Set $t \leftarrow 0$;
 - 3: Determine initial values $\omega^{(0)}$, $\mu^{(0)}$, $\Sigma^{(0)}$, and set $\Psi^{(0)} = (\omega^{(0)}, \mu^{(0)}, \Sigma^{(0)})$;
 - 4: **while** $t \leq N$ **do** ▷ start Gibbs sampling
 - 5: Set $i \leftarrow 1$;
 - 6: **while** $i \leq n$ **do** ▷ start for sampling form missing labels
 - 7: Simulate the i row of $n \times K$ matrix $\mathbf{H}$ as $\mathbf{H}_i \sim \mathcal{MUL}(\cdot | \boldsymbol{\tau}_i)$ where
 - 8: $\boldsymbol{\tau}_i^\top = (\tau_{i1}, \dots, \tau_{iK})$ is the i th row of matrix $\boldsymbol{\tau}$ is given by (6.64);
 - 9: Set $i \leftarrow i + 1$;
 - 10: **end**
 - 11: Set, for $k = 1, \dots, K$, $n_k = \sum_{i=1}^n H_{i,k}$ where $H_{i,k}$ denotes the (i, k) th
 - 12: element of $\mathbf{H}$;
 - 13: Simulate $\omega \sim \mathcal{DIR}(1 | \rho_*)$ where $\rho_*^\top = (\rho_1 + n_1, \dots, \rho_K + n_K)$;
 - 14: Set $k \leftarrow 1$;
 - 15: **while** $k \leq K$ **do** ▷ start sampling from full conditionals of μ_k , and Σ_k
 - 16: Generate $\mu_k^{(t+1)} \sim \mathcal{N}_p(\mathcal{Q}_k [\mathcal{S}_k^{-1} \mathcal{M}_k + [\Sigma_k^{(t)}]^{-1} \sum_{i=1}^n H_{i,k} \times \mathbf{y}_i], \mathcal{Q}_k)$
 - 17: where $\mathcal{Q}_0 = [\mathcal{S}_0^{-1} + [\Sigma_k^{(t)}]^{-1} \times n_k]^{-1}$;
 - 18: Generate $\Sigma_k^{(t+1)}$ from $\mathcal{IW}(\mathcal{R}_k, \nu_k + n_k)$ where $\mathcal{R}_k = \mathcal{D}_k + \sum_{i=1}^n H_{i,k} \times$
 - 19: $(\mathbf{y}_i - \mu_k^{(t+1)})(\mathbf{y}_i - \mu_k^{(t+1)})^\top$;
 - 20: Set $k \leftarrow k + 1$;
 - 21: **end**
 - 22: Set $\Psi^{(t+1)} = (\omega^{(t+1)}, \mu_1^{(t+1)}, \dots, \mu_K^{(t+1)}, \Sigma_1^{(t+1)}, \dots, \Sigma_K^{(t+1)})$ and $t \leftarrow t + 1$;
 - 23: **end**
 - 24: Sequence $\{\Psi^{(M+1)}, \dots, \Psi^{(N)}\}$ is a sample of size $N - M$ from
 - 25: posterior $\pi(\Psi | \underline{y}, \underline{H})$;
 - 26: The Bayesian estimator of Ψ is given by $(N - M)^{-1} \sum_{t=1}^{N-M} \Psi^{(t)}$.
-

While Algorithm 21 works reasonably well, its performance is subject to outliers. If clusters have large variances, then the Gibbs sampling may fail to allocate true label to each cluster. This mainly because this fact that Gaussian distribution has light tails and the PDF decays exponentially for observation are far from the center of distribution. For instance, consider a two-component bivariate mixture model with equal weights $\omega_1 = \omega_2 = 0.5$ while the second cluster has large locations and variances. It is easy to check that the elements of second column of matrix $\boldsymbol{\tau}$ in (6.64) are very small and hence, $P(H_{i,2} = 1) \rightarrow 1$ for $i = 1, \dots, n$. Hence, all observations are allocated to the first cluster. One way to tackle this issue is standardization of observations before clustering [Tortora et al., 2021]. The following example gives more detailed information about this case.

Example 6-9

Herein, we are interested in clustering the famous Old Faithful Geyser Data (known as faithful data). This faithful data consists of 272 observations on two variables *duration of the eruption* in minutes and *waiting time between eruptions* for the Old Faithful geyser in Yellowstone National Park, Wyoming, USA, [Azzalini and Bowman, 1990]. Figure 6.11(a) displays the scatterplot of this dataset. Our investigation shows that the BIC of -833.7747 suggests that $K = 2$ for which the k-mean algorithm pre-clustering results shows that the variances of second variable in both clusters are significantly large as 31.66 and 34.75. Using the R code after standardization given by

```
R> data(faithful)
R> Y <- matrix( cbind(faithful$eruptions, faithful$waiting),
+             nrow = 272, ncol = 2)
R> Z <- apply( Y, 2, function(x)( x - mean(x) )/sd(x) )
R> bic <- BIC(Z)
R> out_kmeans <- kmeans(Z, bic$K)
R> label <- out_kmeans$cluster
R> sapply(1:K, function(x) var( Z[label == x, ] ))
```

then, it can be seen that variances of second variable in both clusters are 0.1876 and 0.1874, correspondingly. The R package `mclust` [Scrucca et al., 2016] applies the EM algorithm for model-based clustering based on the finite mixture of Gaussian distributions. We notice that the clustering results based on both the Gibbs sampling proposed here and EM algorithm proposed in [Scrucca et al., 2016] are the same. The most commonly used criterion for measuring the degree of agreement between clustering approaches is the adjusted Rand index (ARI) that varies from 0 (for zero agreement in clustering) to 1 (for perfect agreement in clustering) [Hubert and Arabie, 1985]. Herein, we use the ARI to compare the Bayesian paradigm and EM algorithm for clustering the faithful data. To this end, among several R packages that can compute the ARI, we call the package `MixGHD`. Fortunately, the agreement between two above approaches in clustering faithful data is perfect, that is $ARI=1$. Figure 6.11(b) and (c) display the contour and scatterplot fitted to standardized faithful data. The R function `FGM` given by the following can be used for applying the model-based clustering to the Gaussian finite mixture model.

```
1 R> FGM <- function(Y, K, n_burn, n_sim, param, hyper = NULL, scale = FALSE )
2 + {
3 +   if( scale == TRUE ) Y <- apply( Y, 2, function(x)( x - mean(x) )/sd(x) )
4 +   dim_y <- dim(Y)
5 +   n <- dim_y[1]
6 +   p <- dim_y[2]
7 +   dim_Psi <- K + K*p + K*p^2
8 +   Psi <- matrix(0, nrow = n_sim, ncol = dim_Psi)
9 +   if( is.null(hyper) )
```

```

10 + {
11 +   rho <- rep(1, K)
12 +   M <- rep( list( rep(0, p) ), K )
13 +   S <- rep( list( diag(1, p) ), K )
14 +   nu <- rep(1, K)
15 +   D0 <- rep( list( diag(10, p) ), K )
16 + }else{
17 +   rho <- hyper[[1]]
18 +   M <- hyper[[2]]
19 +   S <- hyper[[3]]
20 +   D0 <- hyper[[4]]
21 +   nu <- hyper[[5]]
22 + }
23 + omega <- param[[1]]
24 + Mu <- param[[2]]
25 + Sigma <- param[[3]]
26 + index_Mu <- (K + 1):(K*p + K)
27 + index_Sigma <- (K*p + K + 1):(K*(p^2+p+1))
28 + Psi[1, 1:K ] <- omega
29 + Psi[1, index_Mu] <- as.numeric( unlist( Mu ) )
30 + Psi[1, index_Sigma] <- as.numeric( unlist(Sigma) )
31 + A <- tau <- H <- matrix(0, nrow = n, ncol = K)
32 +   for(j in 2:n_sim)
33 +   {
34 +     for(k in 1:K) A[, k] <- omega[k]*dmvnorm(Y, mean = Mu[[k]], sigma =
35 +       Sigma[[k]])
36 +     tau <- A/apply(A, 1, sum)
37 +     for(i in 1:n) H[i, ] <- rmultinom(n = 1, size = 1, tau[i, ])
38 +     n_k <- apply(H, 2, sum)
39 +     omega <- rDIR( rho + n_k )
40 +     for(k in 1:K){
41 +       index_H <- which(H[, k] == 1)
42 +       Y_k <- Y[index_H, ]
43 +       Q_k <- solve( solve(S[[k]]) + solve(Sigma[[k]])*n_k[k] )
44 +       Sum_Yk <- apply(Y_k, 2, sum)
45 +       Mu_k <- Q_k%*( solve(S[[k]])%*M[[k]] + solve(Sigma[[k]])%*Sum_Yk )
46 +       Mu[[k]] <- mvrnorm(n = 1, mu = Mu_k, Sigma = Q_k )
47 +       R <- matrix(0, nrow = p, ncol = p)
48 +       for(i in index_H){R <- R + c( Y[i, ] - Mu[[k]] )%o%( Y[i, ] - Mu[[k]] )}
49 +       Sigma[[k]] <- solve( rWishart(1, df = n_k[k] + nu[k], Sigma =
50 +         solve(D0[[k]] + R) )[, ,1] )
51 +       #print(Sigma[[k]])
52 +     }
53 +     Psi[j, 1:K ] <- omega
54 +     Psi[j, index_Mu] <- as.numeric( unlist( Mu ) )
55 +     Psi[j, index_Sigma] <- as.numeric( unlist(Sigma) )
56 +   }
57 +   omega_hat <- rep(1/K, K)
58 +   Mu_hat <- rep( list( rep(0, p) ), K )
59 +   Sigma_hat <- rep( list( diag(p) ), K )
60 +   out <- apply(Psi[n_burn:n_sim, ], 2, mean)
61 +   for(k in 1:K)
62 +   {
63 +     omega_hat[k] <- out[k]
64 +     Mu_hat[[k]] <- out[index_Mu[((k-1)*p + 1):(k*p)]]
65 +     Sigma_hat[[k]] <- matrix( out[index_Sigma[((k-1)*p^2 + 1):(k*p^2)]] ,
66 +       nrow = p, ncol = p )
67 +     A[, k] <- omega_hat[k]*dmvnorm(Y, mean = Mu_hat[[k]], sigma =
68 +       Sigma_hat[[k]])
69 +   }
70 +   label <- apply(A, 1, which.max)
71 +   ls <- list( "omega" = omega, "Mu" = Mu_hat, "Sigma" = Sigma_hat,
72 +     "label" = label )

```

```

73 + return(ls)
74 + }

```



6.7 Simulating from truncated Gaussian distribution

A mentioned in Section 4.8, the truncated distribution is widely used in practice. Generating from truncated multivariate Gaussian distribution may arise in various contexts such as probit regression [Albert and Chib, 1993], likelihood estimation for max-stable processes [Genton et al., 2011], fitting mixed effects models with censored data [Grün and Hornik, 2012], model-based clustering [Maleki et al., 2019], regression analysis [Rodriguez-Yam et al., 2004], logistic regression with covariates subject to measurement error and left censoring [Teimouri and Sinha, 2024]. For more detailed information on application of truncated multivariate Gaussian distribution, we refer to Botev [2017] and references therein. Section 4.8 gives the first and second moments of a Gaussian distribution under rectangular constraint. Suppose p -dimensional random vector $\mathbf{X}^\top = (X_1, \dots, X_p)$ follows a truncated Gaussian distribution on region with positive Lebesgue measure $\mathbf{R}$ denoted as

$$\mathcal{TN}_{\mathbf{R}}(\boldsymbol{\mu}, \Sigma), \quad \mathbf{R} = \{\mathbf{x} \in \mathbb{R}^p | \mathbf{a} \leq A\mathbf{x} \leq \mathbf{b}\}, \quad (6.70)$$

where $\mathbf{a}^\top = (a_1, \dots, a_p)$ and $\mathbf{b}^\top = (b_1, \dots, b_p)$ are vectors of finite or infinite constants. Moreover, matrix A is a $p \times p$ full rank matrix that constructs a set of p independent linear constraints. One can investigate easily, as pointed out by [Geweke, 1991], that the marginals of $\mathbf{X}$ with distribution given by (6.70) are not truncated Gaussian, but the full conditionals, that is distribution of X_i (for $i = 1, \dots, p$) conditional on all of other members of $\mathbf{X}$ are truncated Gaussian. To verify this claim, we shall confine ourselves here to suppose $p = 2$ and $A = \mathbf{I}_p$ in (6.70). This means that

$$\mathcal{TN}_{\mathbf{R}}(\boldsymbol{\mu}, \Sigma), \quad \mathbf{R} = \{\mathbf{x} = (x_1, x_2)^\top \in \mathbb{R}^2 | a_1 \leq x_1 \leq b_1, a_2 \leq x_2 \leq b_2\}, \quad (6.71)$$

where $\boldsymbol{\mu}^\top = (\mu_1, \mu_2)$ and $\Sigma = [(\sigma_{11}, \sigma_{12})^\top, (\sigma_{21}, \sigma_{22})^\top]$. For computing the marginal distribution of (6.71), e.g. X_1 , whose PDF is denoted by $\mathcal{TN}_{R_1}(x_1 | \boldsymbol{\mu}, \Sigma)$, we have

$$\begin{aligned} \mathcal{TN}_{R_1}(x_1 | \boldsymbol{\mu}, \Sigma) &= \frac{1}{\mathcal{P}_G} \int_{a_2}^{b_2} \phi_2(\mathbf{x} | \boldsymbol{\mu}, \Sigma) dx_2, \\ &= \frac{1}{\mathcal{P}_G} \phi(x_1 | \mu_1, \sigma_{11}) \int_{a_2}^{b_2} \phi(x_2 | \mu_2^*, \sigma_{22}^*) dx_2, \\ &= \frac{1}{\mathcal{P}_G} \phi(x_1 | \mu_1, \sigma_{11}) \left[\Phi(b_2 | \mu_2^*, \sigma_{22}^*) - \Phi(a_2 | \mu_2^*, \sigma_{22}^*) \right], \end{aligned} \quad (6.72)$$

where $x_1 \in R_1$, $\mu_2^* = \mu_2 + \sigma_{12}\sigma_{11}^{-1}(x_1 - \mu_1)$, $\sigma_{22}^* = \sigma_{22} - \sigma_{12}\sigma_{11}^{-1}\sigma_{21}$, and $\mathcal{P}_G = \Phi_2(\mathbf{b} | \boldsymbol{\mu}, \Sigma) - \Phi_2(\mathbf{a} | \boldsymbol{\mu}, \Sigma)$. Obviously, since both terms within the square bracket in the RHS of (6.72) depend on x_1 through μ_2^* , hence $\mathcal{TN}_{R_1}(x_1 | \boldsymbol{\mu}, \Sigma)$ is no longer the PDF of a Gaussian distribution unless μ_2^* is independent of x_1 that occurs only when $\sigma_{12} = \sigma_{21} = 0$. This is while both of the full conditionals $X_1|X_2$ and $X_2|X_1$ correspond to the joint PDF in (6.71) are Gaussian. For example, $X_1|X_2 \sim \mathcal{TN}_{R_1}(\mu_1^*, \sigma_{11}^*)$ where $R_1 = \{x_1 \in \mathbb{R} | a_1 \leq x_1 \leq b_1\}$, $\mu_1^* = \mu_1 + \sigma_{12}\sigma_{22}^{-1}(x_2 - \mu_2)$, $\sigma_{11}^* = \sigma_{11} - \sigma_{12}\sigma_{22}^{-1}\sigma_{21}$, and $a_2 \leq x_2 \leq b_2$. The following proposition generalizes the fact described above for higher dimensions.

Proposition 6.7.1. *Rodriguez-Yam et al. [2004] Suppose $\mathbf{X} \sim \mathcal{TN}_{\mathbf{R}}(\boldsymbol{\mu}, \Sigma)$ as defined in (6.70). Furthermore $\mathbf{Y} = \mathbf{c} + B\mathbf{X}$ where $\mathbf{c} = (c_1, \dots, c_p)^\top$ is a vector of finite constants and B is a $p \times p$ full rank matrix. Then,*

i. $\mathbf{Y} \sim \mathcal{TN}_{\mathbf{T}}(\mathbf{c} + B\boldsymbol{\mu}, B\Sigma B^\top)$, $\mathbf{T} = \{\mathbf{y} \in \mathbb{R}^p | \mathbf{c} + BA^{-1}\mathbf{a} \leq \mathbf{y} \leq \mathbf{c} + BA^{-1}\mathbf{b}\}$.

ii. For a given partition of $\mathbf{X}$, $\boldsymbol{\mu}$, and Σ given by

$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ X_p \end{bmatrix}, \boldsymbol{\mu} = \begin{bmatrix} \boldsymbol{\mu}_1 \\ \mu_p \end{bmatrix}, \Sigma = \begin{bmatrix} \Sigma_{11} & \Sigma_1 \\ \Sigma_1^\top & \sigma_{pp} \end{bmatrix},$$

we have

$$X_p | \mathbf{X}_1 = \mathbf{x}_1 \sim \mathcal{TN}_{R_p}(\mu_p^*, \sigma_{pp}^*), \quad (6.73)$$

where

$$\mu_p^* = \mu_p + \Sigma_1^\top \Sigma_{11}^{-1}(\mathbf{x}_1 - \boldsymbol{\mu}_1), \quad (6.74)$$

$$\sigma_{pp}^* = \sigma_{pp} - \Sigma_1^\top \Sigma_{11}^{-1} \Sigma_1, \quad (6.75)$$

$$R_p = \{x_p \in \mathbb{R} | (\mathbf{x}_1, x_p) \in \mathbf{R}\}.$$

In what follows, we review two algorithms for generating from truncated multivariate Gaussian distribution on a set of linearly independent constraints using the Gibbs sampling. The first algorithm is due to Geweke [1991]. For a given covariance matrix Σ and a $p \times p$ full rank matrix A , suppose $\mathbf{Z} \sim \mathcal{TN}_{\mathbf{T}}(\mathbf{0}, A\Sigma A^\top)$ follows a truncated Gaussian distribution on region $\mathbf{T} = \{\mathbf{z} \in \mathbb{R}^p | \boldsymbol{\alpha} \leq \mathbf{z} \leq \boldsymbol{\beta}\}$ with positive Lebesgue measure in which

$$\boldsymbol{\alpha} = \mathbf{a} - A\boldsymbol{\mu}, \quad (6.76)$$

$$\boldsymbol{\beta} = \mathbf{b} - A\boldsymbol{\mu}, \quad (6.77)$$

are vectors of finite or infinite constants. It follows from Proposition (6.7.1) (i) that $\mathbf{X} = \boldsymbol{\mu} + A^{-1}\mathbf{Z} \sim \mathcal{TN}_{\mathbf{R}}(\boldsymbol{\mu}, \Sigma)$ where $\mathbf{R} = \{\mathbf{x} \in \mathbb{R}^p | \mathbf{a} \leq A\mathbf{x} \leq \mathbf{b}\}$ as (6.70). Let $\mathbf{z}_{-i}$ denote the realization associated with $\mathbf{Z}_{-i} = (Z_1, \dots, Z_{i-1}, Z_{i+1}, \dots, Z_p)^\top$. It follows from Proposition (6.7.1) (ii) that

$$Z_i | (\mathbf{Z}_{-i} = \mathbf{z}_{-i}) \sim \mathcal{TN}_{T_i}(\mu_i^*, \sigma_{ii}^*), \quad T_i = \{z_i \in \mathbb{R} | c_i \leq z_i \leq d_i\}, \quad (6.78)$$

where elements of vectors $\mathbf{c}^\top = (c_1, \dots, c_p)$ and $\mathbf{d}^\top = (d_1, \dots, d_p)$ are given by (6.76) and (6.77), respectively. Furthermore, based on (6.74) and (6.75), for $i = 1, \dots, p$, we have

$$\mu_i^* = V_1^\top V_{11}^{-1} \mathbf{z}_{-i}, \quad (6.79)$$

$$\sigma_{ii}^* = V_{ii} - V_1^\top V_{11}^{-1} V_1, \quad (6.80)$$

in which $V = A\Sigma A^\top$. Algorithm 22 describes the method proposed by Geweke [1991] for simulating from truncated Gaussian distribution. The second algorithm is due to Rodriguez-

Yam et al. [2004]. Let p -dimensional random vector $\mathbf{X}^\top = (X_1, \dots, X_p)$ follows a truncated Gaussian distribution on region $\mathbf{R}$ denoted as

$$\mathcal{TN}_{\mathbf{R}}(\boldsymbol{\mu}, \Sigma), \quad \mathbf{R} = \{\mathbf{x} \in \mathbb{R}^p | C\mathbf{x} \leq \mathbf{b}\}, \quad (6.81)$$

where rows of matrix B are not restricted to be linearly independent. If D is a square matrix of full rank for which we have $D\Sigma D^\top = \mathbf{I}_p$, then for transformation $\mathbf{Z} = D\mathbf{X}$ it follows from Proposition 6.7.1 (i) that

$$\mathbf{Z} \sim \mathcal{TN}_{\mathbf{T}}(D\boldsymbol{\mu}, \mathbf{I}_p), \quad \mathbf{T} = \{\mathbf{z} = D\mathbf{x} \in \mathbb{R}^p | E\mathbf{z} \leq \mathbf{b}\}, \quad (6.82)$$

Algorithm 22 Simulating truncated Gaussian distribution: Geweke's method

```
1: Read  $\mathbf{a}, \mathbf{b}, \boldsymbol{\mu}, \Sigma, A$  given by (6.70), and  $N$  for number of Gibbs sampler runs;
2: Set  $t \leftarrow 0$  and suggest the initial vector  $\mathbf{z}^{(t)} = (z_1^{(t)}, \dots, z_p^{(t)})^\top \in \mathbf{T}$  where  $T_i$  (for  $i = 1, \dots, p$ ) is given by (6.78);
3: while  $t < N$  do
4:   set  $i \leftarrow 1$ ;
5:   while  $i \leq p$  do
6:     Draw  $z_i^{(t+1)}$  form full conditional  $Z_i^{(t+1)} | (Z_1^{(t+1)} = z_1^{(t+1)}, \dots, Z_{i-1}^{(t+1)} = z_{i-1}^{(t+1)}, Z_{i+1}^{(t)} = z_{i+1}^{(t)}, \dots, Z_p^{(t)} = z_p^{(t)})$  that follows a truncated Gaussian distribution with PDF given by (6.78);
7:     set  $i \leftarrow i + 1$ ;
8:   end
9:    $\mathbf{z}^{(t+1)} \leftarrow \mathbf{z}^{(t)}$ ;
10:  Accept  $\mathbf{x}^{(t+1)} = \boldsymbol{\mu} + A^{-1}\mathbf{z}^{(t+1)}$  as a sample of from distribution in (6.70);
11:  set  $t \leftarrow t + 1$ ;
12: end
```

where

$$E = CD^{-1}. \quad (6.83)$$

It is not hard to see that the only choice for D is inverse of the lower-triangular Cholesky decomposition of Σ . Using the transformation $\mathbf{Z} = D\mathbf{X}$ avoids computing μ_i^* and σ_{ii}^* given in (6.78) when applying Algorithm 22 proposed by Geweke [1991]. Suppose $\boldsymbol{\alpha} = D\boldsymbol{\mu}$, $\mathbf{Z}_{-i} = (Z_1, \dots, Z_{i-1}, Z_{i+1}, \dots, Z_p)^\top$, and $\mathbf{z}_{-i}$ denotes the realization of $\mathbf{Z}_{-i}$. Based on Proposition 6.7.1 (ii), we have

$$Z_i | (\mathbf{Z}_{-i} = \mathbf{z}_{-i}) \sim \mathcal{TN}_{T_i}(\alpha_i, 1), \quad T_i = \{z_i \in \mathbb{R} | E\mathbf{z} \leq \mathbf{b}\}, \quad (6.84)$$

where α_i is the i th element of vector $\boldsymbol{\alpha}$. Let E_{-i} denote the matrix E given in (6.83) when its i -th column is removed and $\mathbf{e}_i$ is the i -th column of matrix E . Then, the region T_i given in (6.84) can be represented as

$$T_i = \{z_i \in \mathbb{R} | \mathbf{e}_i z_i \leq \mathbf{b} - E_{-i}\mathbf{z}_{-i}\}. \quad (6.85)$$

As pointed out by Rodriguez-Yam et al. [2004], since constraints on $\mathbf{X}$ construct a convex region in $\mathbb{R}^p$, hence the region T_i given by (6.85) takes the forms $l_i \leq z_i \leq u_i$, $-\infty \leq z_i \leq u_i$, or $l_i \leq z_i \leq \infty$ in which l_i and u_i are, accordingly, the lower and upper bounds of inequalities obtained from (6.85). Algorithm 23 describes how to implement the method proposed by Rodriguez-Yam et al. [2004] for simulating from a truncated Gaussian distribution. In what

follows, we are willing to review an example given by [Rodriguez-Yam et al., 2004] for simulating from a bivariate truncated Gaussian distribution. It is worthwhile to note that the methods proposed by Geweke [1991] and [Rodriguez-Yam et al., 2004] hereafter are denoted as Method 1 and Method 2, respectively.

Example 6-10

Suppose we are interested in simulating from $\mathbf{X}^\top = (X_1, X_2) \sim \mathcal{TN}_{\mathbf{R}}(\boldsymbol{\mu}, \Sigma)$ where $\mathbf{R} = \{\mathbf{x} \in \mathbb{R}^2 | \mathbf{x} \geq \mathbf{0}\}$, $\boldsymbol{\mu}^\top = (\mu_1, \mu_2)$ and $\Sigma = [(1, 0.8)^\top, (0.8, 1)^\top]$. We discuss the key features for implementing Algorithm 22 and Algorithm 23 as follows.

Algorithm 23 Simulating truncated Gaussian distribution: Rodriguez-Yam et al. method

```
1: Read  $\mathbf{b}$ ,  $\boldsymbol{\mu}$ ,  $\Sigma$ ,  $C$  given by (6.81), and  $N$  for number of Gibbs sampler runs;
2: Compute  $\boldsymbol{\alpha} = D\boldsymbol{\mu}$  and  $E = CD^{-1}$  in which  $D$  is the Cholesky decomposition of  $\Sigma$ ;
3: Set  $t = 0$  and suggest the initial value  $\mathbf{z}^{(t)} = (z_1^{(t)}, \dots, z_p^{(t)})^\top \in \mathbf{T}$  where  $\mathbf{T}$  is given by (6.85);
4: while  $t \leq N$  do
5:   set  $i = 1$ ;
6:   while  $i < p$  do
7:     Draw  $z_i^{(t+1)}$  form full conditional  $Z_i^{(t+1)} | (Z_1^{(t+1)} = z_1^{(t+1)}, \dots, Z_{i-1}^{(t+1)} = z_{i-1}^{(t+1)}, Z_{i+1}^{(t)} = z_{i+1}^{(t)}, \dots, Z_p^{(t)} = z_p^{(t)})$  that follows a truncated Gaussian distribution with PDF given by (6.84);
8:     set  $i = i + 1$ ;
9:   end
10:   $\mathbf{Z}^{(t+1)} \leftarrow \mathbf{Z}^{(t)}$ ;
11:   $\mathbf{X}^{(t+1)} \leftarrow D^{-1}\mathbf{Z}^{(t+1)}$ 
12:  set  $t \leftarrow t + 1$ ;
13: end
```

- **Method 1:** For implementing the first step of Algorithm 22, we set $A = [(1, 0)^\top, (0, 1)^\top]$, $\mathbf{a}^\top = (0, 0)$, $\mathbf{b}^\top = (+\infty, +\infty)$, and $N = 2000$. The vector of initial values in the second step can be $\mathbf{z}^{(0)\top} = (1, 1)$ and further, using $\boldsymbol{\mu}^\top = (0, 0)$, it follows from (6.76) and (6.77) that $T_1 = T_2 = [0, \infty)$. Based on (6.74) and (6.75), the full conditionals in the sixth step of Algorithm 22 are given as follows.

$$Z_1 | (\mathbf{Z}_{-1} = \mathbf{z}_{-1}) \sim \mathcal{TN}_{T_1}\left(\frac{4}{5}z_2, \frac{9}{25}\right),$$
$$Z_2 | (\mathbf{Z}_{-2} = \mathbf{z}_{-2}) \sim \mathcal{TN}_{T_2}\left(\frac{4}{5}z_1, \frac{9}{25}\right).$$

The R function called `rtrunnorm_gibbs(.)` for implementing Method 1 is given as follows.

```
1 R> rtrunnorm_gibbs <- function(N, Mu, Sigma, a, b, A)
2 +{
3 + p <- length(Mu)
4 + V <- A%*%Sigma%*%t(A)
5 + T1 <- matrix( cbind(a[1:p] - A%*%Mu[1:p], b[1:p] - A%*%Mu[1:p]), nrow = p, ncol = 2)
6 + Z <- matrix(0, nrow = N, ncol = p)
7 + Z[1, ] <- (a + b)/2
8 + Vinv <- array( 0, dim = c(p-1, p-1, p) )
9 + V1 <- matrix(0, nrow = p, ncol = p-1)
10 + for(i in 1:p)
11 + {
12 +   Vinv[, , i] <- solve( V[-i, -i] )
13 +   V1[i, ] <- as.numeric( V[i, -i]%*%Vinv[, , i] )
14 + }
15 + j <- 1
16 + while(j < N)
17 + {
18 +   for(i in p:1)
19 +   {
20 +     mu.star <- V1[i, ]%*%Z[j, -i]
21 +     sigma.star <- sqrt( V[i, i] - V1[i, ]%*%V[i, -i] )
22 +     u <- runif(1)
23 +     cdf.a <- pnorm(T1[i, 1], mu.star, sigma.star)
```

```

24 +   cdf.b <- pnorm(T1[i, 2], mu.star, sigma.star)
25 +   Z[j, i] <- qnorm( u*(cdf.b - cdf.a) + cdf.a,
26 +                   mean = mu.star, sd = sigma.star )
27 +   Z[j+1, i] <- Z[j, i]
28 + }
29 + j <- j + 1
30 + }
31 + Z <- matrix(Mu, nrow = N, ncol = p, byrow = T) + Z*%solve(A)
32 + return(Z)
33 +}

```

Hence, for simulating truncated multivariate Gaussian using Method 1, we proceed by the following.

```

1 R > N <- 20000 # number of Gibbs sampler iterations
2 R > X <- Z <- matrix(0, nrow = N, ncol = 2)
3 R > Mu <- c(0, 0)
4 R > Sigma <- matrix( c(1, 0.8, 0.8, 1), nrow = 2, ncol = 2)
5 R > A <- diag(2)
6 R > a <- rep(0, 2) # truncation lower bounds
7 R > b <- rep(Inf, 2) # truncation upper bounds
8 R > rtrunnorm_gibbs(N, Mu, Sigma, a, b, A)

```

- **Method 2:** For implementing the first step of Algorithm 23, we set $\mathbf{b} = \mathbf{0}$, $C = -\mathbf{I}_2$, and $N = 2000$. For completing the second step, the Cholesky decomposition D , of Σ and its inverse, are $D = [(1, 4/5)^\top, (0, 3/5)^\top]$ and $D^{-1} = [(1, -4/3)^\top, (0, 5/3)^\top]$, respectively. Hence, for $\boldsymbol{\mu}^\top = (0, 0)$, we have $\boldsymbol{\alpha} = D^{-1}\boldsymbol{\mu} = (0, 0)^\top$ and $E = CD = [(-1, -4/5)^\top, (0, -3/5)^\top]$ where $E_{-1}^\top = (0, -3/5)$ and $E_{-2}^\top = (-1, -4/5)$. Moreover,

$$\begin{aligned}
T_1 &= \left\{ z_1 \in \mathbb{R} \mid \begin{bmatrix} -1 \\ -\frac{4}{5} \end{bmatrix} z_1 \leq - \begin{bmatrix} 0 \\ -\frac{3}{5} \end{bmatrix} z_2 \right\}, \\
&= \left\{ z_1 \in \mathbb{R} \mid z_1 \geq 0, z_1 \geq -\frac{3}{4}z_2 \right\}, \\
&= \left[\max\left\{0, -\frac{3}{4}z_2\right\}, \infty \right),
\end{aligned} \tag{6.86}$$

and

$$\begin{aligned}
T_2 &= \left\{ z_2 \in \mathbb{R} \mid \begin{bmatrix} 0 \\ -\frac{3}{5} \end{bmatrix} z_2 \leq - \begin{bmatrix} -1 \\ -\frac{4}{5} \end{bmatrix} z_1 \right\}, \\
&= \left\{ z_2 \in \mathbb{R} \mid z_2 \geq -\frac{4}{3}z_1 \right\}, \\
&= \left[-\frac{4}{3}z_1, \infty \right).
\end{aligned} \tag{6.87}$$

Using (6.86), (6.87), and $\mathbf{z}^{(0)} = (1, 1)^\top$ the third step of the Algorithm 23 is complete. Finally, the required full conditionals in the seventh step of this algorithm are given as follows.

$$\begin{aligned}
Z_1 | (\mathbf{Z}_{-1} = \mathbf{z}_{-1}) &\sim \mathcal{TN}_{T_1}(\mu_1, 1), \\
Z_2 | (\mathbf{Z}_{-2} = \mathbf{z}_{-2}) &\sim \mathcal{TN}_{T_2}\left(-\frac{4}{3}\mu_1 + \frac{5}{3}\mu_2, 1\right).
\end{aligned}$$

The R code for implementing Method 2, when $\boldsymbol{\mu}^\top = (0, 0)$, is given as follows.

```

1 N <- 20000 # number of Gibbs sampler iterations
2 Z <- matrix(0, nrow = N, ncol = 2)
3 Mu <- c(0, 0) # mean vector can be changed arbitrarily
4 Sigma <- matrix( c(1, 0.8, 0.8, 1), nrow = 2, ncol = 2)
5 C <- -diag(2)
6 D0 <- t( chol(Sigma) )
7 D0inv <- solve( D0 )
8 E <- C%*%D0
9 alpha <- D0inv%*%Mu
10 Z[1, ] <- rep(1, 2) # vector of initial values
11 j <- 1
12 while(j < N)
13 {
14   Z[j + 1, 1] <- rtnorm(1, alpha[1], 1, max(0, -3/4*Z[j, 2]), Inf)
15   Z[j + 1, 2] <- rtnorm(1, alpha[2], 1, -4/3*Z[j + 1, 1], Inf)
16   j <- j + 1
17 }
18 X <- Z%*%t(D0) # X is vector of realizations following truncated
19               # multivariate Gaussian distribution

```



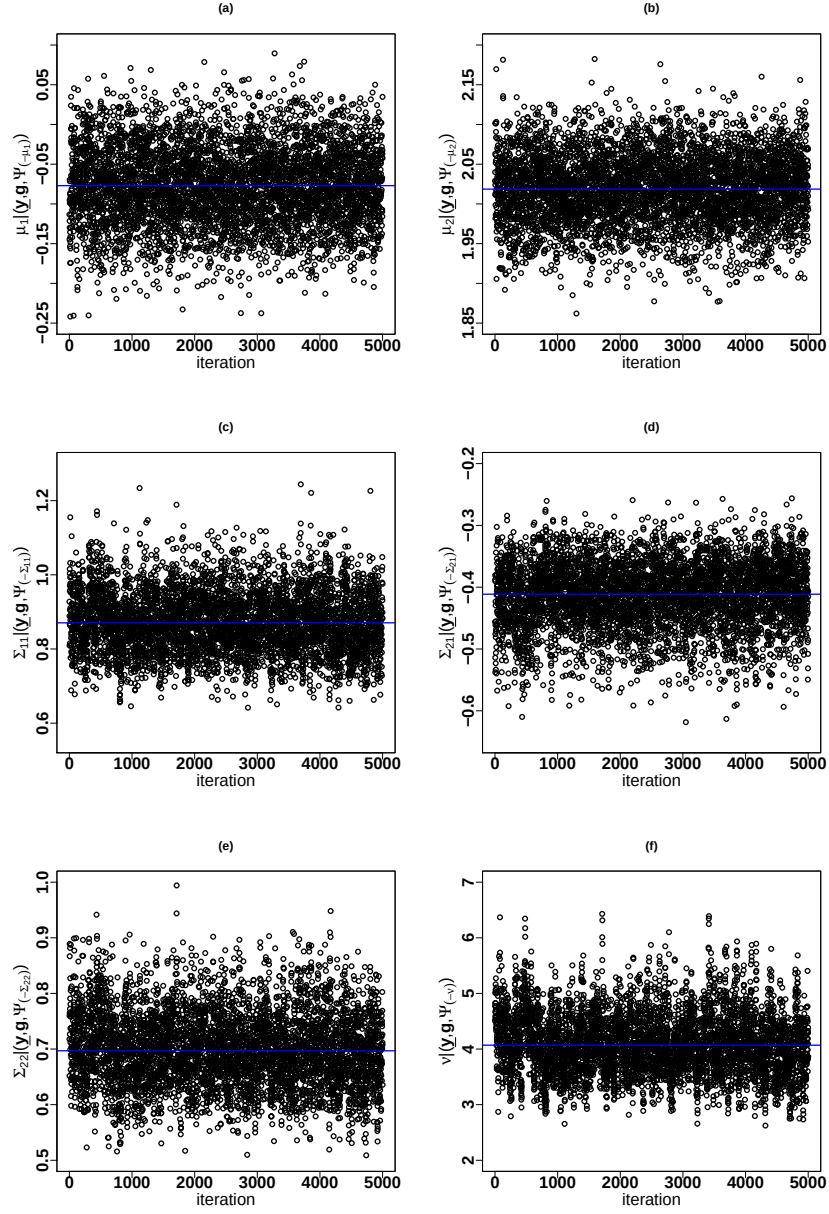


Figure 6.6: Scatterplot of 5000 samples generated from full conditionals: (a) $\mu_1|(\mathbf{y}, \mathbf{g}, \Psi_{(-\mu_1)})$, (b): $\mu_2|(\mathbf{y}, \mathbf{g}, \Psi_{(-\mu_2)})$, (c): $\Sigma_{11}|(\mathbf{y}, \mathbf{g}, \Psi_{(-\Sigma_{11})})$, (d) $\Sigma_{12}|(\mathbf{y}, \mathbf{g}, \Psi_{(-\Sigma_{12})})$, (e) $\Sigma_{22}|(\mathbf{y}, \mathbf{g}, \Psi_{(-\Sigma_{22})})$, and (f) $\nu|(\mathbf{y}, \mathbf{g}, \Psi_{(-\nu)})$. The blue line, in each subfigure, shows the Bayesian estimator.

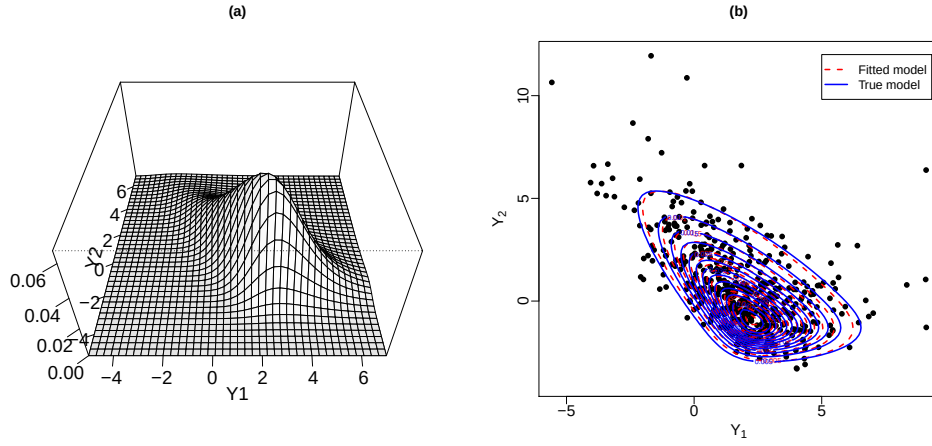


Figure 6.7: The PDF of skewed Student's t distribution. (a): true model and (b): true and fitted contour plots.

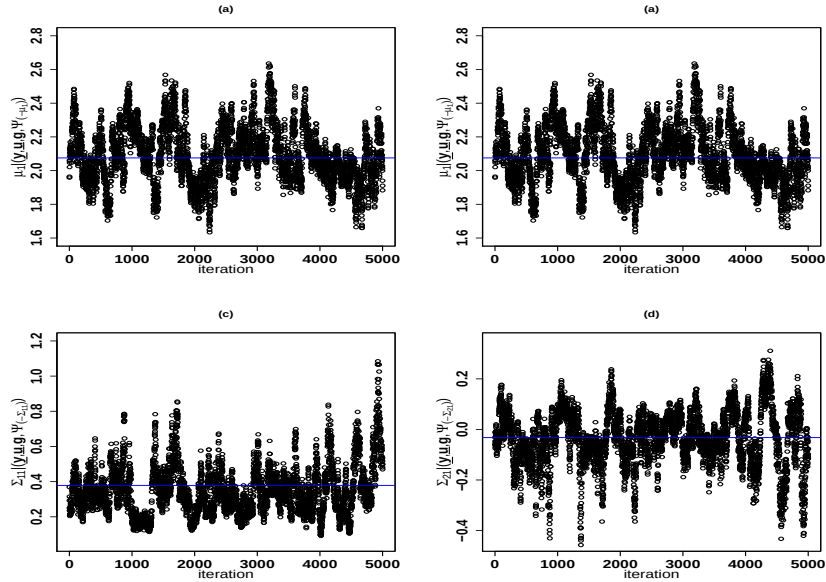


Figure 6.8: Scatterplot of 5000 samples generated from full conditionals involved in Bayesian paradigm for skewed Student's t distribution in Example 6-7.

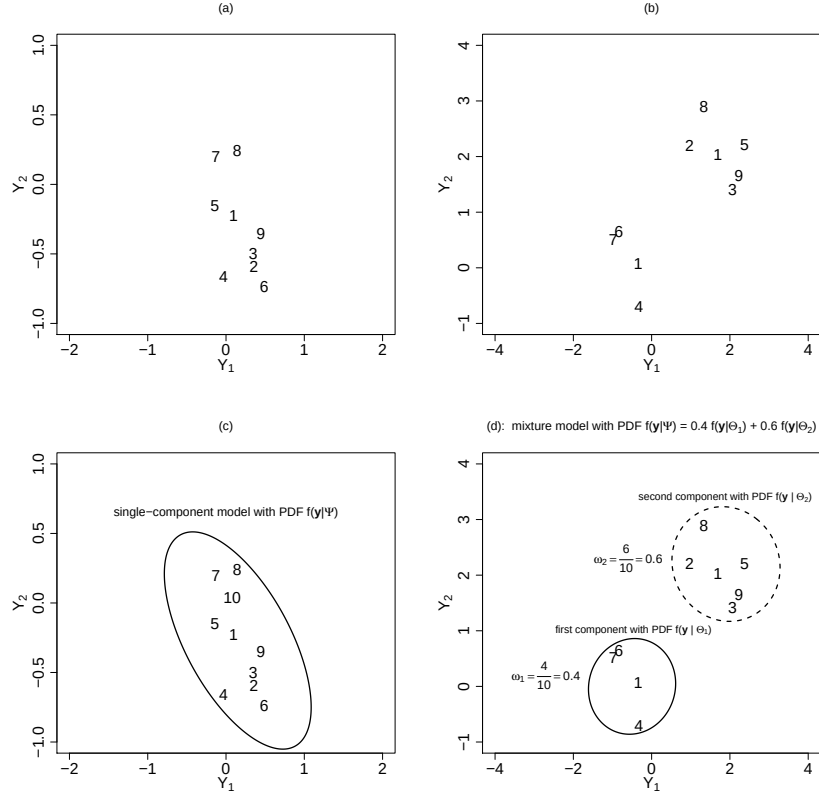


Figure 6.9: (a): Ten data points that can constitute a cluster, (b): Ten data points that should be clustered into two groups, (c): Ten data points in part (a) that can be modelled through a single-component (homogeneous) model with PDF $f(\mathbf{y}|\Theta)$, and (d): Ten data points in part (b) that can be modelled through a two-component (non-homogeneous) mixture model with PDF $0.4f(\mathbf{y}|\Theta_1) + 0.6f(\mathbf{y}|\Theta_2)$.

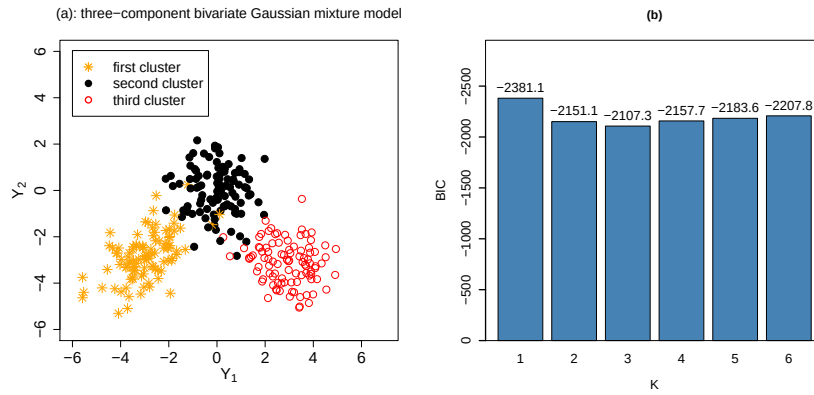
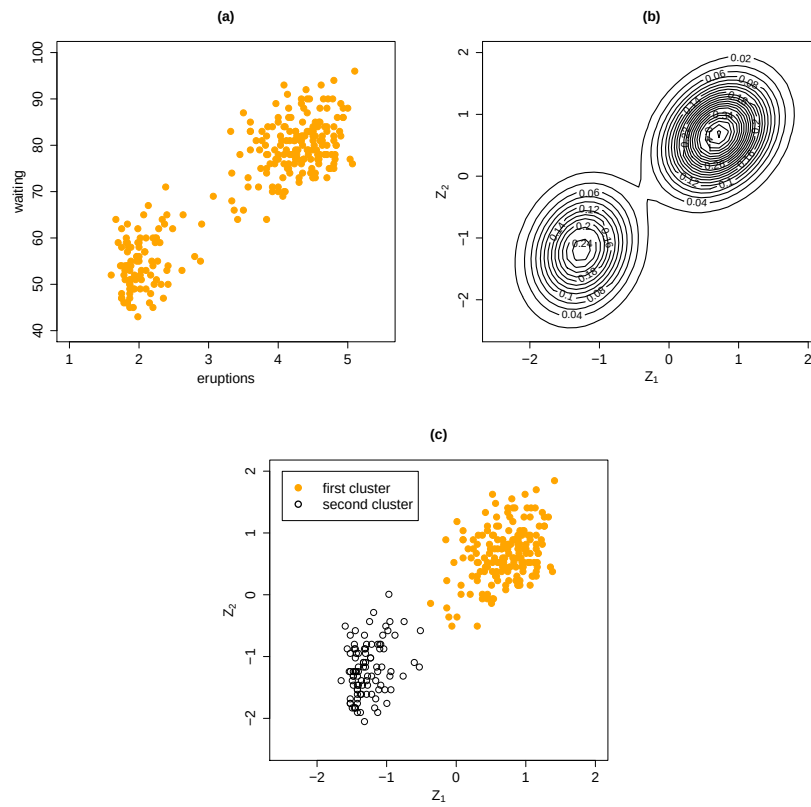


Figure 6.10: (a): Scatterplot of 300 data points drawn from $\mathbf{Y} = (Y_1, Y_2)^\top$ following a three-component Gaussian mixture model and (b): Computed BIC versus number of clusters.



Bibliography

- Milton Abramowitz and Irene A Stegun. Handbook of mathematical functions with formulas, graphs, and mathematical tables, volume 55. US Government printing office, 1948.
- Rita Aggarwala*. Progressive interval censoring: some mathematical results with applications to inference. Communications in Statistics-Theory and Methods, 30(8-9):1921–1935, 2001.
- Hirotugu Akaike. A new look at the statistical model identification. IEEE Transactions on Automatic Control, 19(6):716–723, 2003.
- James H Albert and Siddhartha Chib. Bayesian analysis of binary and polychotomous response data. Journal of the American Statistical Association, 88(422):669–679, 1993.
- Theodore W Anderson and Donald A Darling. A test of goodness of fit. Journal of the American Statistical Association, 49(268):765–769, 1954.
- Reinaldo B Arellano-Valle and Marc G Genton. On fundamental skew distributions. Journal of Multivariate Analysis, 96(1):93–116, 2005.
- George B Arfken, Hans J Weber, and Frank E Harris. Mathematical Methods for Physicists: A Comprehensive Guide. Academic Press, 2011.
- Adelchi Azzalini. A class of distributions which includes the normal ones. Scandinavian Journal of Statistics, pages 171–178, 1985.
- Adelchi Azzalini. sn: The skew-normal and related distributions such as the skew-t and the sun. R package version, 2(0), 2022.
- Adelchi Azzalini and Adrian W Bowman. A look at some data on the old faithful geyser. Journal of the Royal Statistical Society: Series C (Applied Statistics), 39(3):357–365, 1990.
- Adelchi Azzalini and Antonella Capitanio. Distributions generated by perturbation of symmetry with emphasis on a multivariate skew t-distribution. Journal of the Royal Statistical Society Series B: Statistical Methodology, 65(2):367–389, 2003.
- Ole Barndorff-Nielsen. Exponentially decreasing distributions for the logarithm of particle size. Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences, 353(1674):401–419, 1977a.
- Ole Barndorff-Nielsen. Hyperbolic distributions and distributions on hyperbolae. Scandinavian Journal of Statistics, 5:151–157, 1978.
- Ole Eiler Barndorff-Nielsen. Contribution to the discussion of dr cox: The role of significance tests. Scandinavian Journal of Statistics, 4:67–69, 1977b.
- Christian Bauer. Value at risk using hyperbolic distributions. Journal of Economics and Business, 52(5):455–467, 2000.

- Richard A. Becker, John M. Chambers, and Allan R. Wilks. The New S Language. Chapman & Hall, London, 1988.
- James O Berger, José M Bernardo, and Dongchu Sun. The formal definition of reference priors. The Annals of Statistics, 37(2):905–938, 2009.
- Jose M Bernardo. Reference posterior distributions for bayesian inference. Journal of the Royal Statistical Society Series B: Statistical Methodology, 41(2):113–128, 1979.
- José M Bernardo. Reference analysis. Handbook of Statistics, 25:17–90, 2005.
- Marta Blangiardo and Michela Cameletti. Spatial and Spatio-temporal Bayesian Models with R-INLA. Wiley, Chichester, West Sussex, United Kingdom, 1st edition, 2015. ISBN 978-1-118-32655-8. URL <https://eu.wiley.com/WileyCDA/WileyTitle/productCd-1118326555.html>.
- Victor A. Bloomfield. Using R for Numerical Analysis in Science and Engineering. Chapman & Hall/CRC, 2014. ISBN 978-1439884485. URL <http://www.crcpress.com/product/isbn/9781439884485>.
- James G Booth and James P Hobert. Maximizing generalized linear mixed model likelihoods with an automated monte carlo em algorithm. Journal of the Royal Statistical Society Series B: Statistical Methodology, 61(1):265–285, 1999.
- Zdravko I Botev. The normal law under linear restrictions: simulation and estimation via minimax tilting. Journal of the Royal Statistical Society Series B: Statistical Methodology, 79(1):125–148, 2017.
- George EP Box and Mervin E Muller. A note on the generation of random normal deviates. The Annals of Mathematical Statistics, 29(2):610–611, 1958.
- George EP Box and George C Tiao. Bayesian Inference in Statistical Analysis. John Wiley & Sons, 2011.
- DJ Buckle. Bayesian inference for stable distributions. Journal of the American Statistical Association, 90(430):605–613, 1995.
- Paul P Carbone, Leland E Kellerhouse, and Edmund A Gehan. Plasmacytic myeloma: A study of the relationship of survival to various clinical manifestations and anomalous protein type in 112 patients. The American Journal of Medicine, 42(6):937–948, 1967.
- George Casella. An introduction to empirical bayes data analysis. The American Statistician, 39(2):83–87, 1985.
- Gilles Celeux. The sem algorithm: a probabilistic teacher algorithm derived from the em algorithm for the mixture problem. Computational Statistics Quarterly, 2:73–82, 1986.
- John M. Chambers. Programming with Data. Springer, New York, 1998. ISBN 978-0-387-98503-9.
- John M Chambers. Graphical Methods for Data Analysis. Chapman and Hall/CRC, 2018.
- John M. Chambers. S, R, and Data Science. The R Journal, 12(1):462–476, 2020. doi: 10.32614/RJ-2020-028. URL <https://doi.org/10.32614/RJ-2020-028>.
- John M. Chambers and Trevor J. Hastie. Statistical Models in S. Chapman & Hall, London, 1992. ISBN 9780412830402.

- John M Chambers, Colin L Mallows, and BW4159820341 Stuck. A method for simulating stable random variables. Journal of the American Statistical Association, 71(354):340–344, 1976.
- Ding-Geng Chen and YL Lio. Parameter estimations for generalized exponential distribution under progressive type-i interval censoring. Computational Statistics & Data Analysis, 54(6): 1581–1591, 2010.
- KE Chernin and IA Ibragimov. On the unimodality of stable laws. Teor. Veroyatnost. i Primenen, 4:453–456, 1959.
- A Clifford Cohen, Betty Jones Whitten, and Yihua Ding. Modified moment estimation for the three-parameter weibull distribution. Journal of Quality Technology, 16(3):159–167, 1984.
- Edmund A Cornish. The multivariate t-distribution associated with a set of normal sample deviates. Australian Journal of Physics, vol. 7, p. 531, 7:531–542, 1954.
- Harald Cramér. On the composition of elementary errors: First paper: Mathematical deductions. Scandinavian Actuarial Journal, 1928(1):13–74, 1928.
- Germund Dahlquist and Åke Björck. Numerical Methods. Courier Corporation, 2003.
- Utkarsh J Dang, Ryan P Browne, and Paul D McNicholas. Mixtures of multivariate power exponential distributions. Biometrics, 71(4):1081–1089, 2015.
- Gaston Darboux. Mémoire sur l’approximation des fonctions de très-grands nombres, et sur une classe étendue de développements en série. Journal de Mathématiques pures et appliquées, 4:5–56, 1878.
- Gergely Daróczi. Mastering Data Analysis with R. Packt Publishing, 9 2015. ISBN 9781783982028. URL <https://www.packtpub.com/product/mastering-data-analysis-with-r/9781783982028>.
- Vikram Dayal. An Introduction to R for Quantitative Economics: Graphing, Simulating and Computing. Springer, 2015. ISBN 978-81-322-2340-5. URL <https://www.springer.com/978-81-322-2340-5>.
- Arthur P Dempster, Nan M Laird, and Donald B Rubin. Maximum likelihood from incomplete data via the em algorithm. Journal of the Royal Statistical Society. Series B (Methodological), pages 1–38, 1977.
- Luc Devroye. Non-Uniform Random Variate Generation. Springer, New York, 1986.
- Luc Devroye. Random variate generation for exponentially and polynomially tilted stable distributions. ACM Transactions on Modeling and Computer Simulation (TOMACS), 19(4): 1–20, 2009.
- MUDHOLKAR GSAND SRIVASTAVA DK. Exponentiated weibull family for analyzing bath-tub failure data. IEEE Trans Reliab, 42:299–302, 1993.
- Joseph L Doob. Application of the theory of martingales. Le calcul des probabilités et ses applications, pages 23–27, 1949.
- Zvi Drezner. Computation of the multivariate normal integral. ACM Transactions on Mathematical Software (TOMS), 18(4):470–480, 1992.
- Zvi Drezner. Computation of the trivariate normal integral. Mathematics of Computation, 62 (205):289–294, 1994.

- Zvi Drezner and George O Wesolowsky. On the computation of the bivariate normal integral. Journal of Statistical Computation and Simulation, 35(1-2):101–107, 1990.
- Ernst Eberlein and Ulrich Keller. Hyperbolic distributions in finance. Bernoulli, pages 281–299, 1995.
- Torbjørn Eltoft, Taesu Kim, and Te-Won Lee. On the multivariate laplace distribution. IEEE Signal Processing Letters, 13(5):300–303, 2006.
- Carmen Fernández and Mark FJ Steel. Multivariate student-t regression models: Pitfalls and inference. Biometrika, 86(1):153–167, 1999.
- Jeffrey A Fessler and Alfred O Hero. Space-alternating generalized expectation-maximization algorithm. IEEE Transactions on Signal Processing, 42(10):2664–2677, 1994.
- Ronald A Fisher. The use of multiple measurements in taxonomic problems. Annals of Eugenics, 7.2:179–188, 1936.
- Florence Forbes and Darren Wraith. A new family of multivariate heavy-tailed distributions with variable marginal amounts of tailweight: application to robust clustering. Statistics and Computing, 24(6):971–984, 2014.
- Mama Foupouagnigni and Wolfram Koepf. Orthogonal Polynomials. Springer, 2018.
- John Fox. An R and S-Plus Companion to Applied Regression. Sage, 2002.
- John Fox and Sanford Weisberg. An R Companion to Applied Regression. Sage publications, 2018.
- Chris Fraley and Adrian E Raftery. How many clusters? which clustering method? answers via model-based cluster analysis. The computer journal, 41(8):578–588, 1998.
- Chris Fraley and Adrian E Raftery. Model-based clustering, discriminant analysis, and density estimation. Journal of the American statistical Association, 97(458):611–631, 2002.
- James Gareth, Witten Daniela, Hastie Trevor, and Tibshirani Robert. An Introduction to Statistical Learning: with Applications in R. Spinger, 2013.
- HI Gassmann. Multivariate normal probabilities: implementing an old idea of plackett’s. Journal of Computational and Graphical Statistics, 12(3):731–752, 2003.
- Alan E Gelfand and Adrian FM Smith. Sampling-based approaches to calculating marginal densities. Journal of the American Statistical Association, 85(410):398–409, 1990.
- Andrew Gelman. Prior distributions for variance parameters in hierarchical models. Bayesian Analysis, 1(3):551–533, 2006.
- Andrew Gelman, John B Carlin, Hal S Stern, and Donald B Rubin. Bayesian Data Analysis. Chapman and Hall/CRC, 1995.
- Stuart Geman and Donald Geman. Stochastic relaxation, gibbs distributions, and the bayesian restoration of images. IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-6(6):721–741, 1984.
- Marc G Genton, Yanyuan Ma, and Huiyan Sang. On the likelihood function of gaussian max-stable processes. Biometrika, 98(2):481–488, 2011.
- Alan Genz. Numerical computation of multivariate normal probabilities. Journal of Computational and Graphical Statistics, 1(2):141–149, 1992.

- Alan Genz. Numerical computation of rectangular bivariate and trivariate normal and t probabilities. Statistics and Computing, 14(3):251–260, 2004.
- Alan Genz and Frank Bretz. Numerical computation of multivariate t-probabilities with application to power calculation of multiple contrasts. Journal of Statistical Computation and Simulation, 63(4):103–117, 1999.
- John Geweke. Efficient simulation from the multivariate normal and student-t distributions subject to linear constraints and the evaluation of constraint probabilities. In Computing science and statistics: Proceedings of the 23rd symposium on the interface, volume 571, page 578. Fairfax, Virginia: Interface Foundation of North America, Inc, 1991.
- Amparo Gil, Javier Segura, and Nico M Temme. Numerical Methods for Special Functions. SIAM, 2007.
- Walter R Gilks and Pascal Wild. Adaptive rejection sampling for gibbs sampling. Journal of the Royal Statistical Society: Series C (Applied Statistics), 41(2):337–348, 1992.
- Simon Godsill and Ercan E Kuruoglu. Bayesian inference for time series with heavy-tailed symmetric α -stable noise processes. Proc. Applications of heavy tailed distributions in economics, engineering and statistics, 1999.
- Gene H Golub and John H Welsch. Calculation of gauss quadrature rules. Mathematics of Computation, 23(106):221–230, 1969.
- Eusebio Gómez, MA Gomez-Vilegas, and J Miguel Marín. A multivariate generalization of the power exponential family of distributions. Communications in Statistics-Theory and Methods, 27(3):589–600, 1998.
- Irving J Good. The population frequencies of species and the estimation of population parameters. Biometrika, 40(3-4):237–264, 1953.
- Bettina Grün and Kurt Hornik. Modelling human immunodeficiency virus ribonucleic acid levels with finite mixtures for censored longitudinal data. Journal of the Royal Statistical Society Series C: Applied Statistics, 61(2):201–218, 2012.
- Shanti S Gupta. Probability integrals of multivariate normal and multivariate t1. The Annals of Mathematical Statistics, pages 792–828, 1963.
- Frank E. Harrell. Regression Modeling Strategies, with Applications to Linear Models, Survival Analysis and Logistic Regression. Springer, 2001. ISBN 978-3-319-19425-7. URL <https://hbiostat.org/doc/rms/>.
- John A Hartigan and Manchek A Wong. Algorithm as 136: A k-means clustering algorithm. Journal of the Royal Statistical Society. Series c (Applied Statistics), 28(1):100–108, 1979.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. The elements of statistical learning: Data mining, inference, and prediction, 2017.
- W Keith Hastings. Monte carlo sampling methods using markov chains and their applications. Biometrika, 57(1):97–109, 1970.
- Norbert Henze. A probabilistic representation of the ‘skew-normal’ distribution. Scandinavian Journal of Statistics, pages 271–275, 1986.
- James P Hobert and Christian P Robert. A mixture representation of π with applications in markov chain monte carlo and perfect sampling. Annals of Applied Probability, pages 1295–1305, 2004.

- Marius Hofert. Sampling exponentially tilted stable distributions. ACM Transactions on Modeling and Computer Simulation (TOMACS), 22(1):1–11, 2011.
- Wolfgang Hörmann and Josef Leydold. Generating generalized inverse gaussian random variates. Statistics and Computing, 24:547–557, 2014.
- Torsten Hothorn and Brian S. Everitt. A Handbook of Statistical Analyses Using R. Chapman & Hall/CRC Press, Boca Raton, Florida, USA, 3rd edition, 2014. ISBN 978-1-4822-0458-2. URL <http://www.crcpress.com/product/isbn/9781482204582>.
- Lawrence Hubert and Phipps Arabie. Comparing partitions. Journal of Classification, 2:193–218, 1985.
- Ross Ihaka and Robert Gentleman. R: A language for data analysis and graphics. Journal of Computational and Graphical Statistics, 5(3):299–314, 1996.
- Edwin T Jaynes. Prior probabilities. IEEE Transactions on Systems Science and Cybernetics, 4(3):227–241, 1968.
- Harold Jeffreys. An invariant form for the prior probability in estimation problems. Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences, 186(1007):453–461, 1946.
- Adam M Johansen, Ludger Evers, and N Whiteley. Monte carlo methods. International Encyclopedia of Education, pages 296–303, 2010.
- Norman L Johnson, Samuel Kotz, and Narayanaswamy Balakrishnan. Continuous Univariate Distributions, volume 1. John Wiley & Sons, 1995.
- Norman Lloyd Johnson, Samuel Kotz, and Narayanaswamy Balakrishnan. Continuous Multivariate Distributions. John Wiley & Sons New York, 1972.
- Bent Jorgensen. Statistical properties of the generalized inverse Gaussian distribution, volume 9. Springer Science & Business Media, 2012.
- David Kahaner, Cleve Moler, and Stephen Nash. Numerical Methods and Software. Prentice-Hall, Inc., 1989.
- M. Kanter. Stable densities under change of scale and total variation inequalities. Annals of Probability, 3:4, 1975.
- Dan E. Kelley. Oceanographic Analysis with R. Springer-Verlag, New York, 2018. ISBN 978-1-4939-8842-6. URL <https://www.springer.com/us/book/9781493988426>.
- Christine Keribin. Consistent estimation of the order of mixture models. Sankhyā: The Indian Journal of Statistics, Series A, pages 49–66, 2000.
- Nam-Hwui Kim and Ryan P Browne. Flexible mixture regression with the generalized hyperbolic distribution. Advances in Data Analysis and Classification, 18(1):33–60, 2024.
- Albert J Kinderman and John F Monahan. Computer generation of random variables using the ratio of uniform deviates. ACM Transactions on Mathematical Software (TOMS), 3(3):257–260, 1977.
- Matthias Kohl. Introduction to statistical data analysis with R. bookboon.com, London, 2015. ISBN 978-87-403-1123-5.

- Samuel Kotz and Saralees Nadarajah. Multivariate t-Distributions and Their Applications. Cambridge university press, 2004.
- Samuel Kotz, Tomasz Kozubowski, and Krzysztof Podgorski. The Laplace Distribution and Generalizations: A Revisit With Applications To Communications, Economics, Engineering, and Finance. Springer Science & Business Media, 2012.
- Debasis Kundu, Narayanaswamy Balakrishnan, and Ahad Jamalizadeh. Bivariate birnbaum-saunders distribution and associated inference. Journal of Multivariate Analysis, 101(1): 113–125, 2010.
- Ercan E Kuruoglu. Density parameter estimation of skewed/spl alpha/-stable distributions. IEEE Transactions on signal processing, 49(10):2192–2201, 2001.
- Sharon Lee and Geoffrey J McLachlan. Finite mixtures of canonical fundamental skew t distributions: The unification of the restricted and unrestricted skew t -mixture models. Statistics and Computing, 26(3):573–589, 2016a.
- Sharon X Lee and Geoffrey J McLachlan. Model-based clustering and classification with non-normal mixture distributions. Statistical Methods & Applications, 22(4):427–454, 2013.
- Sharon X Lee and Geoffrey J McLachlan. Finite mixtures of canonical fundamental skew t -distributions: The unification of the restricted and unrestricted skew t -mixture models. Statistics and computing, 26(3):573–589, 2016b.
- Lawrence Leemis. Learning Base R. Lightning Source, 2016. ISBN 978-0-9829174-8-0. URL <https://www.amazon.com/Learning-Base-Lawrence-Mark-Leemis/dp/0982917481>.
- Hubert W Lilliefors. On the kolmogorov-smirnov test for normality with mean and variance unknown. Journal of the American Statistical Association, 62(318):399–402, 1967.
- Brunero Liseo and Antonio Parisi. Bayesian inference for the multivariate skew-normal model: A population monte carlo approach. Computational Statistics & Data Analysis, 63:125–138, 2013.
- Chuanhai Liu and Donald B Rubin. The ecme algorithm: a simple extension of em and ecm with faster monotone convergence. Biometrika, 81(4):633–648, 1994.
- Chuanhai Liu, Donald B Rubin, and Ying Nian Wu. Parameter expansion to accelerate em: the px-em algorithm. Biometrika, pages 755–770, 1998.
- Jun S Liu and Jun S Liu. Monte Carlo Strategies in Scientific Computing, volume 10. Springer, 2001.
- Marco J Lombardi. Bayesian inference for α -stable distributions: A random walk mcmc approach. Computational Statistics & Data Analysis, 51(5):2688–2700, 2007.
- Mohsen Maleki, Darren Wraith, and Reinaldo B Arellano-Valle. Robust finite mixture modeling of multivariate unrestricted skew-normal generalized hyperbolic distributions. Statistics and Computing, 29:415–428, 2019.
- George Marsaglia and Thomas A Bray. A convenient method for generating normal variables. SIAM Review, 6(3):260–264, 1964.
- JI McCool. Inferential techniques for weibull populations. aerospace research laboratories report. Technical report, ARL TR 74-0180, Wright-Patterson AFB, Ohio, 1974.

- Geoffrey J McLachlan and Thriyambakam Krishnan. The EM Algorithm and Extensions. John Wiley & Sons, New York, 1997.
- Geoffrey J McLachlan, Thriyambakam Krishnan, and See Ket Ng. The em algorithm. Technical report, Papers, 2004.
- Alexander J McNeil, Rüdiger Frey, and Paul Embrechts. Quantitative Risk Management: Concepts, Techniques and Tools. Princeton University Press, Princeton, NJ, 2005.
- Paul D McNicholas. Mixture Model-Based Classification. Chapman and Hall/CRC, 2016.
- Paul D McNicholas and T Brendan Murphy. Model-based clustering of longitudinal data. Canadian Journal of Statistics, 38(1):153–168, 2010.
- Paul David McNicholas and Thomas Brendan Murphy. Parsimonious gaussian mixture models. Statistics and Computing, 18(3):285–296, 2008.
- Robert W Mee and DB Owen. A simple approximation for bivariate normal probabilities. Journal of Quality Technology, 15(2):72–75, 1983.
- Xiao-Li Meng and Donald B Rubin. Using em to obtain asymptotic variance-covariance matrices: The sem algorithm. Journal of the American Statistical Association, 86(416):899–909, 1991.
- Xiao-Li Meng and Donald B Rubin. Maximum likelihood estimation via the ecm algorithm: A general framework. Biometrika, 80(2):267–278, 1993.
- Xiao-Li Meng and David Van Dyk. The em algorithm-an old folk-song sung to a fast new tune. Journal of the Royal Statistical Society Series B: Statistical Methodology, 59(3):511–567, 1997.
- Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. Equation of state calculations by fast computing machines. The Journal of Chemical Physics, 21(6):1087–1092, 1953.
- John R Michael, William R Schucany, and Roy W Haas. Generating random variates using transformations with multiple roots. The American Statistician, 30(2):88–90, 1976.
- Tetsuhisa Miwa, AJ Hayter, and Satoshi Kuriki. The evaluation of general non-centred orthant probabilities. Journal of the Royal Statistical Society Series B: Statistical Methodology, 65(1):223–234, 2003.
- Christian E Galarza Morales, Larissa A Matos, Dipak K Dey, and Victor H Lachos. On moments of folded and doubly truncated multivariate extended skew-normal distributions. Journal of Computational and Graphical Statistics, 31(2):455–465, 2022.
- Carl N Morris. Parametric empirical bayes inference: theory and applications. Journal of the American Statistical Association, 78(381):47–65, 1983.
- Paula M Murray, Ryan P Browne, and Paul D McNicholas. Hidden truncation hyperbolic distributions, finite mixtures thereof, and their application for clustering. Journal of Multivariate Analysis, 161:141–156, 2017.
- Steven Murray. Apprendre R en un Jour. SJ Murray, 2017. URL https://www.amazon.com/dp/B071W6ZJCV/ref=sr_1_1?s=digital-text&ie=UTF8&qid=1496261881&sr=1-1. Ebook.

- HKT Ng, Debasis Kundu, and N Balakrishnan. Modified moment estimation for the two-parameter birnbaum–saunders distribution. Computational Statistics & Data Analysis, 43(3):283–298, 2003.
- Deborah Nolan and Terry Speed. Stat Labs: Mathematical Statistics Through Applications. Springer Texts in Statistics. Springer, 2000. ISBN 978-0-387-22743-6. URL <https://www.stat.berkeley.edu/users/statlabs/>.
- John Nolan. Stable distributions: Models for Heavy Tailed Data. Birkhauser New York, 2003.
- John P Nolan. Parameterizations and modes of stable distributions. Statistics & probability letters, 38(2):187–195, 1998.
- John P Nolan. Univariate stable distributions: Models for heavy tailed data. Springer Series in Operations Research and Financial Engineering, 2020.
- John P Nolan and Diana Ojeda-Revah. Linear and nonlinear regression with stable errors. Journal of Econometrics, 172(2):186–194, 2013.
- Donald B Owen. Tables for computing bivariate normal probabilities. The Annals of Mathematical Statistics, 27(4):1075–1090, 1956.
- Antonio Parisi and Brunero Liseo. Objective bayesian analysis for the multivariate skew-t model. Statistical Methods & Applications, 27(2):277–295, 2018.
- Hyun Park and Te Lee. Modeling nonlinear dependencies in natural images using mixture of laplacian distribution. Advances in Neural Information Processing Systems, 17, 2004.
- Mike Patefield. Fast and accurate calculation of owen’s t function. Journal of Statistical Software, 5(5):1–25, 2000. doi: 10.18637/jss.v005.i05. URL <https://www.jstatsoft.org/index.php/jss/article/view/v005i05>.
- David Peel and Geoffrey J McLachlan. Robust mixture modelling using the t distribution. Statistics and Computing, 10(4):339–348, 2000.
- Kaare Brandt Petersen, Michael Syskind Pedersen, et al. The matrix cookbook. Technical University of Denmark, 7(15):510, 2008.
- Jose C. Pinheiro and Douglas M. Bates. Mixed-Effects Models in S and S-Plus. Springer, 2000. ISBN 978-0-387-22747-4.
- Robin L Plackett. A reduction formula for normal multivariate integrals. Biometrika, 41(3/4):351–360, 1954.
- Saumyadipta Pyne, Xinli Hu, Kui Wang, Elizabeth Rossin, Tsung-I Lin, Lisa M Maier, Clare Baecher-Allan, Geoffrey J McLachlan, Pablo Tamayo, David A Hafler, et al. Automated high-dimensional flow cytometric data analysis. Proceedings of the National Academy of Sciences, 106(21):8519–8524, 2009.
- Yan Qu, Angelos Dassios, and Hongbiao Zhao. Random variate generation for exponential and gamma tilted stable distributions. ACM Transactions on Modeling and Computer Simulation (TOMACS), 31(4):1–21, 2021.
- Thomas Rahlf. Data Visualisation with R. Springer International Publishing, New York, 2017. ISBN 978-3-319-49750-1. URL <http://www.datavisualisation-r.com>.
- Theodore J Rivlin. Chebyshev Polynomials. Courier Dover Publications, 2020.

- Herbert Robbins. An empirical bayes approach to statistics. In Proceedings of the Third Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 157–163, 1956.
- Christian P Robert, George Casella, Christian P Robert, and George Casella. The metropolis-hastings algorithm. Monte Carlo Statistical Methods, pages 267–320, 2004.
- Christian P Robert, George Casella, and George Casella. Introducing Monte Carlo Methods with R, volume 18. Springer, 2010.
- Gareth O Roberts and Jeffrey S Rosenthal. Examples of adaptive mcmc. Journal of Computational and Graphical Statistics, 18(2):349–367, 2009.
- Gareth O Roberts and Adrian FM Smith. Simple conditions for the convergence of the gibbs sampler and metropolis-hastings algorithms. Stochastic processes and their applications, 49(2):207–216, 1994.
- Gabriel Rodriguez-Yam, Richard A Davis, and Louis L Scharf. Efficient gibbs sampling of truncated multivariate normal with application to constrained linear regression. Unpublished manuscript, 2004.
- Sheldon M Ross. Simulation. academic press, 2022.
- Sujit K Sahu, Dipak K Dey, and Márcia D Branco. A new class of multivariate skew distributions with applications to bayesian regression models. Canadian Journal of Statistics, 31(2):129–150, 2003.
- G. Samorodnitsky and Murad. S. Taqqu. Stable Non-Gaussian Random Processes: Stochastic Models with Infinite Variance. Chapman & Hall, New York, 1994.
- Gideon Schwarz et al. Estimating the dimension of a model. The Annals of Statistics, 6(2):461–464, 1978.
- Luca Scrucca, Michael Fop, Thomas Brendan Murphy, and Adrian E. Raftery. mclust 5: clustering, classification and density estimation using gaussian finite mixture models. The R Journal, 8(1):205–233, 2016. URL <https://journal.r-project.org/archive/2016-1/scrucca-fop-murphy-et-al.pdf>.
- Jie Shen, Tao Tang, and Li-Lian Wang. Spectral Methods: Algorithms, Analysis and Applications, volume 41. Springer Science & Business Media, 2011.
- William Fleetwood Sheppard and Karl Pearson. On the calculation of the double integral expressing normal correlation. Transactions of the Cambridge Philosophical Society, 19:23–69, 1900.
- Galen R Shorack and Jon A Wellner. Empirical Processes with Applications to Statistics. SIAM, 2009.
- Jonathan Shuster. On the inverse gaussian distribution function. Journal of the American Statistical Association, 63(324):1514–1516, 1968.
- David Slepian. The one-sided barrier problem for gaussian noise. Bell System Technical Journal, 41(2):463–501, 1962.
- NM Steen, GD Byrne, and EM Gelbard. Gaussian quadratures for the integrals $\int_0^\infty \exp(-x^2) f(x) dx$ and $\int_0^\infty \exp(-x^2) f(x) dx$. Mathematics of Computation, pages 661–671, 1969.

- Michael A Stephens. Tests based on edf statistics. In Goodness-of-fit-techniques, pages 97–194. Routledge, 2017.
- C. Sun. Empirical Research in Economics: Growing up with R. Pine Square, Starkville, Mississippi, USA, 1st edition, 2015. ISBN 978-0-9965854-0-8. URL https://www.amazon.com/Empirical-Research-Economics-Changyou-Sun/dp/0996585400/ref=aag_m_pw_dp?ie=UTF8&m=A1TZL30UWYSSR8.
- Benaglia Tatiana, Chauveau Didier, RH David, and Young Derek. mixtools: an r package for analyzing finite mixture models. Journal of Statistical Software, 32(6):1–29, 2009.
- Mahdi Teimouri. Fast bayesian inference for birnbaum-saunders distribution. Computational Statistics, 38(2):569–601, 2023.
- Mahdi Teimouri and Saralees Nadarajah. Maximum likelihood estimation for the asymmetric exponential power distribution. Computational Economics, 60(2):665–692, 2022.
- Mahdi Teimouri and Sanjoy K Sinha. Inference for logistic regression with covariates subject to limit of detection and measurement error. METRON, 82(2):161–182, 2024.
- Mahdi Teimouri, Jeffrey W Doser, and Andrew O Finley. Forestfit: An r package for modeling plant size distributions. Environmental Modelling & Software, 131:104668, 2020.
- Mahdi Teimouri, Seyed Mehdi Hoseini, and Maria Sabrina Greco. Bayesian inference for amplitude distribution with application to radar clutter. Digital Signal Processing, 148:104443, 2024.
- Terry M. Therneau and Patricia M. Grambsch. Modeling Survival Data: Extending the Cox Model. Statistics for Biology and Health. Springer, 2000. ISBN 978-1-4757-3294-8.
- HC Thode. Testing for normality, 2002.
- GE Thomas. Remark asr 65: A remark on algorithm as76: An integral useful in calculating non-central t and bivariate normal probabilities. Journal of the Royal Statistical Society. Series C (Applied Statistics), 35(3):310–312, 1986.
- Cristina Tortora, Paul D McNicholas, and Ryan P Browne. A mixture of generalized hyperbolic factor analyzers. Advances in Data Analysis and Classification, 10(4):423–440, 2016.
- Cristina Tortora, Ryan P Browne, Aisha ElSherbiny, Brian C Franczak, and Paul D McNicholas. Model-based clustering, classification, and discriminant analysis using the generalized hyperbolic distribution: Mixghd r package. Journal of Statistical Software, 98(1):1–24, 2021.
- William N. Venables and Brian D. Ripley. S Programming. Springer, New York, 2000. ISBN 978-0-387-21856-4. URL <http://www.stats.ox.ac.uk/pub/MASS3/Sprog/>.
- Jonathan C Wakefield, Alan E Gelfand, and Adrian FM Smith. Efficient generation of random variates via the ratio-of-uniforms method. Statistics and Computing, 1:129–133, 1991.
- Greg CG Wei and Martin A Tanner. A monte carlo implementation of the em algorithm and the poor man’s data augmentation algorithms. Journal of the American Statistical Association, 85(411):699–704, 1990.
- Marc Weibel, Wolfgang Breymann, and David Lüthi. ghyp: A package on generalized hyperbolic distributions. Manual for the R Package GHYP, 1:1–27, 2020.
- R Weron. On the chambers-mallows-stuck method for simulating skewed stable random variables. Statistics and Probability Letters, 28:165–171, 1996.

- CF Jeff Wu. On the convergence properties of the em algorithm. The Annals of Statistics, pages 95–103, 1983.
- Richard C Yang, Antel Kozak, and J Harry G Smith. The potential of weibull-type functions as flexible growth curves. Canadian Journal of Forest Research, 8(4):424–431, 1978.
- JC Young and Ch E Minder. As 76: An integral useful in calculating non-central t and bivariate normal probabilities. Journal of the Royal Statistical Society Series C: Applied Statistics, 23(3):455–457, 1974.
- Vladimir M Zolotarev. One-dimensional stable distributions, volume 65. American Mathematical Soc., The American Mathematical Society, 1986.